

TrioCFD Reference Manual V1.9.7

Support team: trust@cea.fr

December 15, 2025

Contents

1	Syntax to define a mathematical function	21
2	Existing & predefined fields names	23
3	interpret	24
3.1	Ale_neumann_bc_for_grid_problem	25
3.2	Bloc_lecture	25
3.2.1	Solveur_petsc_option_cli	26
3.2.2	Bloc_criteres_convergence	26
3.3	Beam_model	26
3.4	Bloc_lecture_beam_model	26
3.4.1	Bloc_poutre	27
3.4.2	Newmarktimescheme_deriv	28
3.4.3	Hht	28
3.4.4	Ma	28
3.4.5	Fd	28
3.4.6	Listpoints	29
3.4.7	Un_point	29
3.5	Create_domain_from_sub_domain	29
3.6	Dns_qc_double	30
3.7	Debogft	30
3.8	Debogijk	30
3.9	Write_med	31
3.10	Extraire_surface_ale	31
3.11	Link_cgns_files	32
3.12	Merge_med	32
3.13	Multiplefiles	32
3.14	My_comm_group	32
3.15	Op_conv_ef_stab_polymac_face	33
3.16	Op_conv_ef_stab_polymac_p0p1nc_elem	33
3.17	Op_conv_ef_stab_polymac_p0p1nc_face	33
3.18	Op_conv_ef_stab_polymac_p0_face	33
3.19	Option_cgns	34
3.20	Option_dg	34
3.21	Option_ijk	35
3.22	Option_interpolation	35
3.23	Option_polymac	35
3.24	Parallel_io_parameters	36
3.25	Projection_ale_boundary	36
3.26	Raffiner_isotrope_parallele	37
3.27	Read_med	37
3.28	Solver_moving_mesh_ale	38
3.29	Structural_dynamic_mesh_model	38
3.30	Bloc_lecture_structural_dynamic_mesh_model	38
3.31	Switch_ft_double	39
3.32	Switch_double	39
3.33	Test_sse_kernels	40
3.34	Analyse_angle	40
3.35	Associate	40
3.36	Associer_algo	41
3.37	Associer_pbmng_pbfin	41
3.38	Associer_pbmng_pbgglobal	41

3.39	Axi	42
3.40	Bidim_axi	42
3.41	Calculer_moments	42
3.42	Lecture_bloc_moment_base	42
3.42.1	Calcul	42
3.42.2	Centre_de_gravite	42
3.43	Corriger_frontiere_periodique	43
3.44	Criteres_convergence	43
3.45	Debog	44
3.46	{	44
3.47	Decoupebord_pour_rayonnement	44
3.48	Decouper_bord_coincident	45
3.49	Dilate	45
3.50	Dimension	46
3.51	Disable_tu	46
3.52	Discretiser_domaine	46
3.53	Discretize	46
3.54	Distance_paro	47
3.55	Ecrire_champ_med	47
3.56	Ecrire_fichier_formatte	47
3.57	Ecrire_fichier_xyz_valeur	47
3.58	Ecriturelecturespecial	48
3.59	Espece	48
3.60	Execute_parallel	49
3.61	Export	49
3.62	Extract_2d_from_3d	49
3.63	Extract_2daxi_from_3d	49
3.64	Extraire_domaine	50
3.65	Extraire_plan	50
3.66	Extraire_surface	51
3.67	Extrudebord	52
3.68	Extrudeparoi	52
3.69	Extruder	53
3.70	Troisf	53
3.71	Extruder_en20	54
3.72	Extruder_en3	54
3.73	Facsec_expert	54
3.74	End	55
3.75	}	55
3.76	Imposer_vit_bords_ale	56
3.77	Imprimer_flux	56
3.78	Imprimer_flux_sum	56
3.79	Integrer_champ_med	57
3.80	Interfaces	57
3.81	Interprete_geometrique_base	58
3.82	Lata_to_cgns	58
3.83	Format_lata_to_cgns	58
3.84	Lata_2_med	58
3.85	Format_lata_to_med	59
3.86	Lata_2_other	59
3.87	Lire_ideas	59
3.88	Lml_2_lata	60
3.89	Mailler	60
3.90	List_bloc_mailler	60

3.90.1	Mailler_base	60
3.90.2	Pave	61
3.90.3	Bloc_pave	61
3.90.4	List_bord	62
3.90.5	Bord_base	62
3.90.6	Bord	62
3.90.7	Defbord	63
3.90.8	Defbord_2	63
3.90.9	Defbord_3	63
3.90.10	Raccord	64
3.90.11	Internes	64
3.90.12	Epsilon	64
3.90.13	Domain	64
3.91	Maillerparallel	65
3.92	Mass_source	66
3.93	Mkdir	66
3.94	Modif_bord_to_raccord	66
3.95	Modifydomaineaxi1d	67
3.96	Moyenne_volumique	67
3.97	Multigrid_solver	68
3.98	Coarsen_operators	69
3.98.1	Coarsen_operator_uniform	69
3.99	Nettoiepasnoeuds	70
3.100	Option_vdf	70
3.101	Orientefacesbord	70
3.102	Partition	71
3.103	Bloc_decouper	71
3.104	Partition_multi	72
3.105	Pilote_icoco	73
3.106	Polyedriser	73
3.107	Postraiter_domaine	73
3.108	Precisiongeom	74
3.109	Raffiner_anisotrope	74
3.110	Raffiner_isotrope	75
3.111	Read	76
3.112	Read_file	76
3.113	Read_file_binary	77
3.114	Lire_tgrid	77
3.115	Read_unsupported_ascii_file_from_icem	77
3.116	Orienter_simplexes	77
3.117	Redresser_hexaedres_vdf	78
3.118	Refine_mesh	78
3.119	Regroupebord	78
3.120	Remaillage_ft_ijk	78
3.121	Remove_elem	79
3.122	Remove_elem_bloc	80
3.123	Remove_invalid_internal_boundaries	80
3.124	Reorienter_tetraedres	80
3.125	Reorienter_triangles	81
3.126	Reordonner	81
3.127	Residuals	81
3.128	Rotation	82
3.129	Scatter	82
3.130	Scatteredmed	82

3.131	Solve	82
3.132	Stat_per_proc_perf_log	83
3.133	Supprime_bord	83
3.134	List_nom	83
3.135	System	83
3.136	Test_solveur	84
3.137	Testeur	84
3.138	Testeur_medcoupling	85
3.139	Tetraedriser	85
3.140	Tetraedriser_homogene	85
3.141	Tetraedriser_homogene_compact	86
3.142	Tetraedriser_homogene_fin	86
3.143	Tetraedriser_par_prisme	87
3.144	Transformer	88
3.145	Trianguler	88
3.146	Trianguler_fin	89
3.147	Trianguler_h	89
3.148	Verifier_qualite_raffinements	90
3.149	Vect_nom	90
3.150	Verifier_simplexes	90
3.151	Verifiercoin	90
3.152	Verifiercoin_bloc	91
3.153	Ecrire	91
3.154	Ecrire_fichier_bin	91
4	pb_gen_base	91
4.1	Pb_conduction	92
4.2	Corps_postraitement	93
4.2.1	Interface_posts	94
4.2.2	Champs_a_post	95
4.2.3	Champ_a_post	95
4.2.4	Definition_champs	95
4.2.5	Definition_champ	95
4.2.6	Definition_champs_fichier	96
4.2.7	Sondes	96
4.2.8	Sonde	96
4.2.9	Sonde_base	97
4.2.10	Segmentfacesx	97
4.2.11	Segmentfacesy	97
4.2.12	Segmentfacesz	97
4.2.13	Radius	98
4.2.14	Points	98
4.2.15	Segmentpoints	98
4.2.16	Point	98
4.2.17	Numero_elem_sur_maitre	99
4.2.18	Position_like	99
4.2.19	Plan	99
4.2.20	Volume	99
4.2.21	Circle	100
4.2.22	Circle_3	100
4.2.23	Segment	100
4.2.24	Sondes_fichier	101
4.2.25	Champs_posts	101
4.2.26	Champs_posts_fichier	101

4.2.27	Bloc_fichier	101
4.2.28	Stats_posts	102
4.2.29	List_stat_post	102
4.2.30	Stat_post_deriv	103
4.2.31	T_deb	103
4.2.32	T_fin	103
4.2.33	Moyenne	103
4.2.34	Ecart_type	104
4.2.35	Correlation	104
4.2.36	Stats_posts_fichier	104
4.2.37	Stats_serie_posts	105
4.2.38	Stats_serie_posts_fichier	106
4.3	Post_processings	106
4.3.1	Un_postraitement	106
4.4	Liste_post_ok	107
4.4.1	Nom_postraitement	107
4.4.2	Postraitement_base	107
4.4.3	Post_processing	107
4.4.4	Postraitement_ft_lata	109
4.5	Liste_post	109
4.5.1	Un_postraitement_spec	109
4.5.2	Type_un_post	109
4.5.3	Type_postraitement_ft_lata	110
4.6	Format_file_base	110
4.6.1	Binaire	110
4.6.2	Formatte	110
4.6.3	Xyz	111
4.6.4	Single_hdf	111
4.6.5	Pdi	111
4.6.6	Pdi_expert	111
4.7	Pb_conduction_ibm	112
4.8	Pb_fronttracking_disc	113
4.9	Listdeuxmots_acc	114
4.9.1	Deuxmots	114
4.10	Pb_hydraulique_cloned_concentration	115
4.11	Pb_hydraulique_cloned_concentration_turbulent	116
4.12	Pb_hydraulique_ibm_turbulent	117
4.13	Pb_hydraulique_list_concentration	118
4.14	Listeqn	119
4.15	Pb_hydraulique_list_concentration_turbulent	119
4.16	Pb_hydraulique_turbulent_ale	121
4.17	Pb_hydraulique_sensibility	122
4.18	Pb_multiphase	123
4.19	Pb_multiphase_h	124
4.20	Pb_hem	126
4.21	Pb_rayo_conduction	128
4.22	Pb_rayo_hydraulique	129
4.23	Pb_rayo_hydraulique_turbulent	130
4.24	Pb_rayo_thermohydraulique	131
4.25	Pb_rayo_thermohydraulique_qc	132
4.26	Pb_rayo_thermohydraulique_turbulent	133
4.27	Pb_rayo_thermohydraulique_turbulent_qc	134
4.28	Pb_thermohydraulique_cloned_concentration	136
4.29	Pb_thermohydraulique_cloned_concentration_turbulent	137

4.30	Pb_thermohydraulique_ibm_turbulent	138
4.31	Pb_thermohydraulique_list_concentration	139
4.32	Pb_thermohydraulique_list_concentration_turbulent	140
4.33	Pb_thermohydraulique_sensibility	142
4.34	Pb_base	143
4.35	Probleme_ftd_ijk_cut_cell	144
4.36	Probleme_couple	146
4.37	List_list_nom	146
4.38	Modele_rayo_semi_transp	147
4.39	Eq_rayo_semi_transp	148
4.39.1	Condlims	148
4.39.2	Condlimlu	148
4.40	Pb_avec_liste_conc	148
4.41	Pb_avec_passif	150
4.42	Pb_couple_rayo_semi_transp	151
4.43	Pb_hydraulique	151
4.44	Pb_hydraulique_ale	152
4.45	Pb_hydraulique_aposteriori	153
4.46	Pb_hydraulique_concentration	155
4.47	Pb_hydraulique_concentration_scalaires_passifs	156
4.48	Pb_hydraulique_concentration_turbulent	157
4.49	Pb_hydraulique_concentration_turbulent_scalaires_passifs	158
4.50	Pb_hydraulique_ibm	160
4.51	Pb_hydraulique_melange_binaire_qc	161
4.52	Pb_hydraulique_melange_binaire_wc	162
4.53	Pb_hydraulique_melange_binaire_turbulent_qc	163
4.54	Pb_hydraulique_turbulent	164
4.55	Pb_mg	166
4.56	Pb_phase_field	166
4.57	Pb_post	167
4.58	Pb_thermohydraulique	168
4.59	Pb_thermohydraulique_qc	169
4.60	Pb_thermohydraulique_wc	171
4.61	Pb_thermohydraulique_concentration	172
4.62	Pb_thermohydraulique_concentration_scalaires_passifs	173
4.63	Pb_thermohydraulique_concentration_turbulent	174
4.64	Pb_thermohydraulique_concentration_turbulent_scalaires_passifs	176
4.65	Pb_thermohydraulique_especes_qc	177
4.66	Pb_thermohydraulique_especes_wc	178
4.67	Pb_thermohydraulique_especes_turbulent_qc	180
4.68	Pb_thermohydraulique_ibm	181
4.69	Pb_thermohydraulique_scalaires_passifs	182
4.70	Pb_thermohydraulique_turbulent	183
4.71	Pb_thermohydraulique_turbulent_qc	185
4.72	Pb_thermohydraulique_turbulent_scalaires_passifs	186
4.73	Pbc_med	187
4.74	List_info_med	187
4.74.1	Info_med	187
4.75	Problem_read_generic	188
4.76	Pb_couple_rayonnement	189
4.77	Probleme_ftd_ijk	189

5	mor_eqn	191
5.1	Conduction	191
5.2	Bloc_convection	192
5.2.1	Convection_deriv	192
5.2.2	Negligeable	192
5.2.3	Amont	192
5.2.4	Centre	192
5.2.5	Centre4	193
5.2.6	Ef	193
5.2.7	Bloc_ef	193
5.2.8	Muscl_old	194
5.2.9	Muscl	194
5.2.10	Di_l2	194
5.2.11	Quick	194
5.2.12	Centre_old	194
5.2.13	Amont_old	194
5.2.14	Generic	195
5.2.15	Muscl_new	195
5.2.16	Kquick	195
5.2.17	Muscl3	195
5.2.18	Ef_stab	196
5.2.19	Listsous_zone_valeur	196
5.2.20	Sous_zone_valeur	197
5.2.21	Btd	197
5.2.22	Supg	197
5.2.23	Ale	197
5.2.24	Rt	198
5.2.25	Sensibility	198
5.3	Bloc_diffusion	198
5.3.1	Diffusion_deriv	198
5.3.2	Negligeable	199
5.3.3	Option	199
5.3.4	Stab	199
5.3.5	P1ncp1b	200
5.3.6	P1b	200
5.3.7	Standard	200
5.3.8	Bloc_diffusion_standard	200
5.3.9	Turbulente	201
5.3.10	Type_diffusion_turbulente_multiphase_deriv	201
5.3.11	Wale	201
5.3.12	Sgdh	201
5.3.13	Smago	202
5.3.14	L_melange	202
5.3.15	Prandtl	203
5.3.16	Interfacial_area	203
5.3.17	Multiple	203
5.3.18	K_omega	204
5.3.19	Sato	204
5.3.20	K_omega	204
5.3.21	K_tau	204
5.3.22	Tenseur_reynolds_externe	205
5.3.23	Op_implicit	205
5.4	Condinit	205
5.4.1	Condinit	205

5.5	Sources	205
5.6	Parametre_equation_base	206
5.6.1	Parametre_diffusion_implicite	206
5.6.2	Parametre_implicite	206
5.7	Conduction_ibm	207
5.8	Convection_diffusion_concentration_turbulent_ft_disc	208
5.9	Convection_diffusion_espece_binaire_turbulent_qc	209
5.10	Convection_diffusion_temperature_sensibility	210
5.11	Echelle_temporelle_turbulente	212
5.12	Energie_multiphase	212
5.13	Energie_multiphase_h	213
5.14	Energie_cinetique_turbulente	214
5.15	Energie_cinetique_turbulente_wit	215
5.16	Masse_multiphase	216
5.17	Navier_stokes_aposteriori	217
5.18	Traitement_particulier	218
5.18.1	Traitement_particulier_base	219
5.18.2	Profils_thermo	219
5.18.3	Canal	219
5.18.4	Ec	220
5.18.5	Temperature	220
5.18.6	Thi	221
5.18.7	Thi_thermo	221
5.18.8	Chmoy_faceperio	222
5.18.9	Brech	222
5.18.10	Ceg	223
5.18.11	Ceg_areva	223
5.18.12	Ceg_cea_jaea	223
5.19	Floatfloat	224
5.20	Navier_stokes_ftd_ijk	224
5.21	Navier_stokes_turbulent_ale	227
5.22	Modele_turbulence_hyd_deriv	229
5.22.1	Dt_impr_ustar_mean_only	230
5.22.2	Mod_turb_hyd_ss_maille	230
5.22.3	Form_a_nb_points	231
5.22.4	Sous_maille_wale	231
5.22.5	Sous_maille_smago	232
5.22.6	Longueur_melange	234
5.22.7	Sous_maille_selectif_mod	235
5.22.8	Deuxentiers	236
5.22.9	Floatentier	236
5.22.10	Sous_maille_selectif	237
5.22.11	Sous_maille_1elt	238
5.22.12	Sous_maille_1elt_selectif_mod	239
5.22.13	Sous_maille_axi	240
5.22.14	Sous_maille_smago_filtre	241
5.22.15	Sous_maille_smago_dyn	242
5.22.16	Combinaison	243
5.22.17	Sous_maille	244
5.22.18	Null	245
5.22.19	Mod_turb_hyd_rans	246
5.22.20	K_omega	247
5.22.21	Mod_turb_hyd_rans_keps	248
5.22.22	K_epsilon	249

5.22.23	Modele_fonction_bas_reynolds_base	250
5.22.24	Lam_bremhorst	250
5.22.25	Easm_baglietto	251
5.22.26	Standard_keps	251
5.22.27	Jones_launder	251
5.22.28	Launder_sharma	252
5.22.29	Mod_turb_hyd_rans_bicephale	252
5.22.30	K_epsilon_bicephale	253
5.22.31	Mod_turb_hyd_rans_komega	254
5.22.32	K_epsilon_realisable_bicephale	255
5.22.33	K_epsilon_realisable	256
5.23	Navier_stokes_standard_sensibility	257
5.24	Navier_stokes_std_ale	259
5.25	Qdm_multiphase	261
5.26	Taux_dissipation_turbulent	261
5.27	Convection_diffusion_chaleur_qc	262
5.28	Convection_diffusion_chaleur_wc	263
5.29	Convection_diffusion_chaleur_turbulent_qc	264
5.30	Convection_diffusion_concentration	265
5.31	Convection_diffusion_concentration_ft_disc	266
5.32	Convection_diffusion_concentration_turbulent	268
5.33	Convection_diffusion_espece_binaire_qc	269
5.34	Convection_diffusion_espece_binaire_wc	270
5.35	Convection_diffusion_espece_multi_qc	270
5.36	Convection_diffusion_espece_multi_wc	271
5.37	Convection_diffusion_espece_multi_turbulent_qc	272
5.38	Convection_diffusion_phase_field	273
5.39	Convection_diffusion_temperature	274
5.40	Convection_diffusion_temperature_ft_disc	275
5.41	Objet_lecture_maintien_temperature	277
5.42	Convection_diffusion_temperature_ibm	277
5.43	Convection_diffusion_temperature_ibm_turbulent	278
5.44	Convection_diffusion_temperature_turbulent	279
5.45	Eqn_base	280
5.46	Ijk_interfaces	281
5.47	Ijk_thermals	282
5.48	Navier_stokes_qc	283
5.49	Navier_stokes_wc	285
5.50	Navier_stokes_ft_disc	286
5.51	Penalisation_forage	289
5.52	Navier_stokes_ibm	290
5.53	Navier_stokes_ibm_turbulent	292
5.54	Navier_stokes_phase_field	293
5.55	Approx_boussinesq	295
5.55.1	Bloc_boussinesq	295
5.55.2	Bloc_rho_fonc_c	296
5.56	Visco_dyn_cons	296
5.56.1	Bloc_visco2	297
5.56.2	Bloc_mu_fonc_c	297
5.57	Navier_stokes_standard	297
5.58	Navier_stokes_turbulent	299
5.59	Navier_stokes_turbulent_qc	301
5.60	Transport_epsilon	302
5.61	Transport_interfaces_ft_disc	303

5.62	Methode_transport_deriv	307
5.62.1	Vitesse_imposee	307
5.62.2	Vitesse_interpolee	307
5.62.3	Loi_horaire	308
5.63	Bloc_lecture_remaillage	308
5.64	Parcours_interface	309
5.65	Interpolation_champ_face_deriv	310
5.65.1	Base	310
5.65.2	Lineaire	310
5.66	Type_indic_faces_deriv	310
5.66.1	Standard	310
5.66.2	Modifiee	310
5.66.3	Ai_based	311
5.67	Transport_k	311
5.68	Transport_marqueur_ft	312
5.69	Injection_marqueur	313
6	collision_model_ft_base	314
7	domaine_base	314
7.1	Domaine_ijk	315
7.2	Troismots	315
8	interface_base	315
8.1	Interface_sigma_constant	316
8.2	Saturation_base	316
8.3	Saturation_constant	316
8.4	Saturation_sodium	317
9	triple_line_model_ft_disc	317
10	algo_base	319
10.1	Algo_couple_1	319
11	/*	320
11.1	/*	320
12	champ_generique_base	320
12.1	Champ_post_de_champs_post	320
12.2	Listchamp_generique	320
12.3	List_nom_virgule	321
12.4	Champ_post_operateur_base	321
12.5	Champ_post_operateur_eqn	321
12.6	Champ_post_statistiques_base	322
12.7	Correlation	323
12.8	Champ_post_operateur_divergence	323
12.9	Ecart_type	324
12.10	Champ_post_extraction	324
12.11	Champ_post_operateur_gradient	325
12.12	Interpolation	326
12.13	Champ_post_morceau_equation	326
12.14	Moyenne	327
12.15	Predefini	328
12.16	Champ_post_reduction_0d	328
12.17	Champ_post_refchamp	329

12.18	Champ_post_tparoi_vef	329
12.19	Champ_post_transformation	330
13	chimie	331
13.1	Reactions	331
13.1.1	Reaction	332
14	class_generic	332
14.1	Modele_fonc_realisable_base	332
14.2	Modele_shih_zhu_lumley_vdf	333
14.3	Shih_zhu_lumley	333
14.4	Amg	333
14.5	Amgx	333
14.6	Cholesky	334
14.7	Cudss	334
14.8	Dt_calc	334
14.9	Dt_fixe	334
14.10	Dt_min	335
14.11	Dt_start	335
14.12	Gcp_ns	335
14.13	Gen	336
14.14	Gmres	337
14.15	Optimal	337
14.16	Petsc	338
14.17	Petsc_gpu	338
14.18	Gcp	338
14.19	Solveur_sys_base	339
15	collision_model_fpi_deriv	339
15.1	Collision_model_ft_sphere	339
16	#	340
16.1	#	340
17	condlim_base	340
17.1	Cond_lim_k_complique_transition_flux_nul_demi	341
17.2	Cond_lim_k_simple_flux_nul	341
17.3	Cond_lim_omega_demi	341
17.4	Cond_lim_omega_dix	341
17.5	Echange_couplage_thermique	341
17.6	Paroi_echange_interne_global_impose	342
17.7	Paroi_echange_interne_global_parfait	342
17.8	Paroi_echange_interne_impose	342
17.9	Paroi_echange_interne_parfait	342
17.10	Mixte_shear	343
17.11	Neumann_homogene	343
17.12	Neumann_paro	343
17.13	Neumann_paro_adiabatique	343
17.14	Paroi	343
17.15	Paroi_frottante_loi	343
17.16	Paroi_frottante_simple	344
17.17	Contact_vdf_vef	344
17.18	Contact_vef_vdf	344
17.19	Dirichlet	344

17.20Echange_contact_rayo_transp_vdf	344
17.21Echange_contact_vdf_ft_disc	345
17.22Echange_contact_vdf_ft_disc_solid	345
17.23Paroi_echange_externeradiatif	346
17.24Entree_temperature_imposee_h	346
17.25Flux_radiatif	347
17.26Flux_radiatif_vdf	347
17.27Flux_radiatif_vef	347
17.28Frontiere_ouverte	348
17.29Frontiere_ouverte_alpha_impose	348
17.30Frontiere_ouverte_concentration_imposee	348
17.31Frontiere_ouverte_fraction_massique_imposee	348
17.32Frontiere_ouverte_gradient_pression_impose	349
17.33Frontiere_ouverte_gradient_pression_impose_vefprep1b	349
17.34Frontiere_ouverte_gradient_pression_libre_vef	349
17.35Frontiere_ouverte_gradient_pression_libre_vefprep1b	349
17.36Frontiere_ouverte_k_eps_impose	349
17.37Frontiere_ouverte_k_omega_impose	350
17.38Frontiere_ouverte_pression_imposee	350
17.39Frontiere_ouverte_pression_imposee_orlansky	350
17.40Frontiere_ouverte_pression_moyenne_imposee	350
17.41Frontiere_ouverte_rayo_semi_transp	351
17.42Frontiere_ouverte_rayo_transp	351
17.43Frontiere_ouverte_rayo_transp_vdf	351
17.44Frontiere_ouverte_rayo_transp_vef	352
17.45Frontiere_ouverte_rho_u_impose	352
17.46Frontiere_ouverte_enthalpie_imposee	352
17.47Frontiere_ouverte_temperature_imposee_rayo_semi_transp	352
17.48Frontiere_ouverte_temperature_imposee_rayo_transp	353
17.49Frontiere_ouverte_vitesse_imposee	353
17.50Frontiere_ouverte_vitesse_imposee_ale	353
17.51Frontiere_ouverte_vitesse_imposee_sortie	353
17.52Neumann	354
17.53Paroi_adiabatique	354
17.54Paroi_contact	354
17.55Paroi_contact_fictif	355
17.56Paroi_contact_rayo	355
17.57Paroi_decalee_robin	355
17.58Paroi_defilante	356
17.59Paroi_echange_contact_correlation_vdf	356
17.60Paroi_echange_contact_correlation_vef	357
17.61Paroi_echange_contact_odvm_vdf	358
17.62Paroi_echange_contact_rayo_semi_transp_vdf	358
17.63Paroi_echange_contact_vdf	359
17.64Paroi_echange_contact_vdf_ft	359
17.65Paroi_echange_externer_impose	360
17.66Paroi_echange_externer_impose_h	360
17.67Paroi_echange_externer_impose_rayo_semi_transp	360
17.68Paroi_echange_externer_impose_rayo_transp	360
17.69Paroi_echange_global_impose	361
17.70Paroi_fixe	361
17.71Paroi_fixe_iso_genepi2_sans_contribution_aux_vitesses_sommets	361
17.72Paroi_flux_impose	361
17.73Paroi_flux_impose_rayo_semi_transp_vdf	362

17.74	Paroi_flux_impose_rayo_semi_transp_vef	362
17.75	Paroi_flux_impose_rayo_transp	362
17.76	Paroi_ft_disc	363
17.77	Paroi_ft_disc_deriv	363
17.77.1	Symetrie	363
17.77.2	Constant	363
17.78	Paroi_knudsen_non_negligeable	363
17.79	Paroi_rugueuse	364
17.80	Paroi_temperature_imposee	364
17.81	Paroi_temperature_imposee_rayo_semi_transp	364
17.82	Paroi_temperature_imposee_rayo_transp	365
17.83	Perio	365
17.84	Periodique	365
17.85	Robin_vef	365
17.86	Scalaire_impose_paro	366
17.87	Sortie_libre_rho_variable	366
17.88	Sortie_libre_temperature_imposee_h	366
17.89	Symetrie	366
17.90	Enthalpie_imposee_paro	367
18	discretisation_base	367
18.1	Dg	367
18.2	Ef_axi	367
18.3	Ef	367
18.4	Ijk	367
18.5	Polymac	368
18.6	Polymac_p0p1nc	368
18.7	Polymac_p0	368
18.8	Vdf	368
18.9	Vef	368
19	domaine	369
19.1	Domaineaxi1d	369
19.2	Ijk_grid_geometry	369
19.3	Domaine_ale	370
20	champ_base	370
20.1	Champ_base	370
20.2	Champ_fonc_interp	370
20.3	Champ_fonc_med_table_temps	371
20.4	Champ_fonc_med_tabule	372
20.5	Champ_tabule_morceaux	373
20.6	Champ_fonc_tabule_morceaux_interp	373
20.7	Champ_parametrique	373
20.8	Champ_composite	374
20.9	Champ_don_base	374
20.10	Champ_don_lu	374
20.11	Champ_fonc_fonction	374
20.12	Champ_fonc_fonction_txyz	375
20.13	Champ_fonc_fonction_txyz_morceaux	375
20.14	Champ_fonc_med	375
20.15	Champ_fonc_reprise	376
20.16	Fonction_champ_reprise	377
20.17	Champ_fonc_t	377

20.18	Champ_fonc_tabule	377
20.19	Champ_init_canal_sinal	377
20.20	Bloc_lec_champ_init_canal_sinal	378
20.21	Champ_input_base	379
20.22	Champ_input_p0	379
20.23	Champ_input_p0_composite	379
20.24	Champ_musig	380
20.25	Champ_ostwald	380
20.26	Champ_som_lu_vdf	380
20.27	Champ_som_lu_vef	381
20.28	Champ_tabule_lu	381
20.29	Champ_tabule_temps	381
20.30	Champ_uniforme_morceaux	382
20.31	Champ_uniforme_morceaux_tabule_temps	382
20.32	Champ_fonc_txyz	382
20.33	Champ_fonc_xyz	383
20.34	Field_uniform_keps_from_ud	383
20.35	Init_par_partie	383
20.36	Tayl_green	384
20.37	Uniform_field	384
20.38	Valeur_totale_sur_volume	384
21	champ_front_base	384
21.1	Champ_front_base	384
21.2	Boundary_field_keps_from_ud	385
21.3	Ch_front_input_ale	385
21.4	Champ_front_xyz_tabule	385
21.5	Champ_front_ale_beam	386
21.6	Champ_front_parametrique	386
21.7	Champ_front_ale	386
21.8	Champ_front_debit_qc_vdf	387
21.9	Champ_front_debit_qc_vdf_fonc_t	387
21.10	Champ_front_synt	387
21.11	Bloc_lecture_turb_synt	387
21.12	Boundary_field_inward	388
21.13	Boundary_field_uniform_keps_from_ud	388
21.14	Ch_front_input	389
21.15	Ch_front_input_uniforme	389
21.16	Champ_front_med	390
21.17	Champ_front_bruit	390
21.18	Champ_front_calc	390
21.19	Champ_front_composite	391
21.20	Champ_front_contact_rayo_semi_transp_vef	391
21.21	Champ_front_contact_rayo_transp_vef	391
21.22	Champ_front_contact_vef	392
21.23	Champ_front_debit	392
21.24	Champ_front_debit_massique	392
21.25	Champ_front_fonc_pois_ipsn	393
21.26	Champ_front_fonc_pois_tube	393
21.27	Champ_front_fonc_t	393
21.28	Champ_front_fonc_txyz	393
21.29	Champ_front_fonc_xyz	394
21.30	Champ_front_fonction	394
21.31	Champ_front_lu	394

21.32	Champ_front_musig	394
21.33	Champ_front_normal_vef	395
21.34	Champ_front_pression_from_u	395
21.35	Champ_front_recyclage	395
21.36	Champ_front_tabule	396
21.37	Champ_front_tabule_lu	396
21.38	Champ_front_tangentiel_vef	397
21.39	Champ_front_uniforme	397
21.40	Champ_front_vortex	397
21.41	Champ_front_xyz_debit	397
22	interpolation_ibm_base	398
22.1	Interpolation_ibm_power_law_tbl_u_star	398
22.2	Ibm_aucune	399
22.3	Ibm_element_fluide	399
22.4	Ibm_hybride	399
22.5	Ibm_gradient_moyen	400
22.6	Ibm_power_law_tbl	401
23	loi_etat_base	401
23.1	Eos_qc	402
23.2	Eos_wc	402
23.3	Binaire_gaz_parfait_qc	402
23.4	Binaire_gaz_parfait_wc	403
23.5	Coolprop_qc	403
23.6	Coolprop_wc	404
23.7	Loi_etat_gaz_parfait_base	404
23.8	Loi_etat_gaz_reel_base	404
23.9	Loi_etat_tppi_base	405
23.10	Multi_gaz_parfait_qc	405
23.11	Multi_gaz_parfait_wc	405
23.12	Gaz_parfait_qc	406
23.13	Gaz_parfait_wc	406
23.14	Rhot_gaz_parfait_qc	407
23.15	Rhot_gaz_reel_qc	407
24	loi_fermeture_base	407
24.1	Loi_fermeture_test	408
25	loi_horaire	408
26	milieu_base	408
26.1	Fluide_diphasique_ijk	409
26.2	Solid_particle_base	410
26.3	Solid_particle_sphere	411
26.4	Solid_particle_spheroid	411
26.5	Constituant	412
26.6	Fluide_base	413
26.7	Fluide_dilatable_base	414
26.8	Fluide_diphasique	414
26.9	Fluide_incompressible	415
26.10	Fluide_ostwald	416
26.11	Fluide_quasi_compressible	417
26.12	Bloc_sutherland	418

26.13	Fluide_reel_base	419
26.14	Fluide_sodium_gaz	419
26.15	Fluide_sodium_liquide	420
26.16	Fluide_stiffened_gas	421
26.17	Fluide_weakly_compressible	422
26.18	Solide	423
27	milieu_v2_base	424
28	modele_rayonnement_base	424
28.1	Modele_rayonnement_milieu_transparent	424
29	modele_turbulence_scal_base	426
29.1	Dt_impr_nusselt_mean_only	426
29.2	Null	427
29.3	Prandtl	427
29.4	Schmidt	428
29.5	Sous_maille_dyn	429
30	moyenne_imposee_deriv	429
30.1	Connexion_approchee	430
30.2	Connexion_exacte	430
30.3	Interpolation	431
30.4	Logarithmique	431
30.5	Profil	431
31	nom	432
31.1	Nom_anonyme	432
32	partitionneur_deriv	432
32.1	Fichier_med	433
32.2	Fichier_decoupage	433
32.3	Metis	434
32.4	Partition	434
32.5	Sous_dom	435
32.6	Sous_zones	435
32.7	Tranche	436
32.8	Union	436
33	pb_champ_evaluateur	436
34	porosites	437
34.1	Bloc_lecture_poro	437
35	precond_base	438
35.1	Ilu	438
35.2	Precondsolv	438
35.3	Ssor	438
35.4	Ssor_bloc	439
36	preconditionneur_petsc_deriv	439
36.1	Block_jacobi_icc	439
36.2	Eisntat	440
36.3	Block_jacobi_ilu	440
36.4	Boomeramg	440

36.5	C-amg	440
36.6	Diag	441
36.7	Jacobi	441
36.8	Lu	441
36.9	Null	441
36.10	Pilut	441
36.11	Sa-amg	442
36.12	Spai	442
36.13	Ssor	442
37	schema_temps_base	442
37.1	Implicit_euler_steady_scheme	444
37.2	Sch_cn_ex_iteratif	447
37.3	Sch_cn_iteratif	449
37.4	Scheme_euler_explicit	452
37.5	Leap_frog	454
37.6	Rk3_ft	456
37.7	Runge_kutta_ordre_2	458
37.8	Runge_kutta_ordre_2_classique	460
37.9	Runge_kutta_ordre_3	462
37.10	Runge_kutta_ordre_3_classique	464
37.11	Runge_kutta_ordre_4_d3p	466
37.12	Runge_kutta_ordre_4_classique	468
37.13	Runge_kutta_ordre_4_classique_3_8	470
37.14	Runge_kutta_rationnel_ordre_2	472
37.15	Schema_adams_bashforth_order_2	474
37.16	Schema_adams_bashforth_order_3	476
37.17	Schema_adams_moulton_order_2	478
37.18	Schema_adams_moulton_order_3	480
37.19	Schema_backward_differentiation_order_2	483
37.20	Schema_backward_differentiation_order_3	485
37.21	Scheme_euler_implicit	488
37.22	Schema_implicite_base	491
37.23	Schema_phase_field	493
37.24	Schema_predictor_corrector	495
37.25	Schema_euler_explicite_ale	497
38	schema_temps_base_ijk	499
38.1	Schema_rk3_ijk	499
38.2	Schema_euler_explicite_ijk	500
39	solveur_implicite_base	501
39.1	Ice	501
39.2	Implicit_steady	502
39.3	Implicite	503
39.4	Implicite_ale	504
39.5	Piso	505
39.6	Sets	506
39.7	Simple	507
39.8	Simpler	508
39.9	Solveur_lineaire_std	509
39.10	Solveur_u_p	509

40	solveur_petsc_deriv	510
40.1	Bicgstab	511
40.2	Cholesky_out_of_core	511
40.3	Cholesky_pastix	512
40.4	Cholesky_superlu	512
40.5	Cholesky_umfpack	513
40.6	Ibicgstab	513
40.7	Pipecg	514
40.8	Cholesky	515
40.9	Cholesky_mumps_blr	516
40.10	Cli	517
40.11	Cli_quiet	517
40.12	Gcp	518
40.13	Gmres	519
40.14	Lu	519
41	source_base	520
41.1	Coalescence_bulles_1groupe	521
41.2	Coalescence_bulles_2groupes	521
41.3	Correction_antal	521
41.4	Correction_lubchenko	522
41.5	Correction_tomiyama	522
41.6	Dp_impose	522
41.7	Type_perte_charge_deriv	522
41.7.1	Dp	523
41.7.2	Dp_regul	523
41.8	Diffusion_croisee_echelle_temp_taux_diss_turb	523
41.9	Diffusion_supplementaire_echelle_temp_turb	524
41.10	Dispersion_bulles	524
41.11	Dissipation_echelle_temp_taux_diss_turb	524
41.12	Flux_2groupes	524
41.13	Injection_qdm_nulle	525
41.14	Portance_interfaciale	525
41.15	Production_hzdr	525
41.16	Production_echelle_temp_taux_diss_turb	525
41.17	Production_energie_cin_turb	526
41.18	Rupture_bulles_1groupe	526
41.19	Rupture_bulles_2groupes	526
41.20	Source_bif	526
41.21	Source_constituant_vortex	527
41.22	Source_dissipation_hzdr	527
41.23	Source_dissipation_echelle_temp_taux_diss_turb	527
41.24	Source_transport_k_eps_anisotherme	528
41.25	Source_dep_inco_bases	528
41.26	Terme_dissipation_energie_cinetique_turbulente	528
41.27	Acceleration	529
41.28	Boussinesq_concentration	529
41.29	Boussinesq_temperature	530
41.30	Canal_perio	530
41.31	Coriolis	531
41.32	Darcy	531
41.33	Dirac	531
41.34	Flux_interfacial	532
41.35	Forchheimer	532

41.36	Frottement_interfacial	532
41.37	Perte_charge_anisotrope	533
41.38	Perte_charge_circulaire	533
41.39	Perte_charge_directionnelle	534
41.40	Perte_charge_isotrope	534
41.41	Perte_charge_reguliere	534
41.42	Spec_pdc_base	535
41.42.1	Longitudinale	535
41.42.2	Transversale	535
41.43	Perte_charge_singuliere	535
41.44	Puissance_thermique	536
41.45	Radioactive_decay	536
41.46	Source_con_phase_field	537
41.47	Systeme_naire_deriv	538
41.47.1	Non	538
41.47.2	Bloc_kappa_variable	538
41.47.3	Bloc_potentiel_chim	538
41.48	Source_constituant	539
41.49	Flottabilite	539
41.50	Source_generique	539
41.51	Masse_ajoutee	539
41.52	Source_pdf	539
41.53	Bloc_pdf_model	540
41.54	Source_pdf_base	540
41.55	Source_qdm	541
41.56	Source_qdm_lambdaup	541
41.57	Source_qdm_phase_field	542
41.58	Source_rayo_semi_transp	542
41.59	Source_robin	542
41.60	Source_robin_scalaire	543
41.61	Listdeuxmots_sacc	543
41.62	Source_th_tdivu	543
41.63	Trainee	543
41.64	Source_transport_eps	543
41.65	Source_transport_k	544
41.66	Source_transport_k_eps	544
41.67	Source_transport_k_eps_aniso_concen	544
41.68	Source_transport_k_eps_aniso_therm_concen	545
41.69	Source_transport_k_eps_realisable	545
41.70	Tenseur_reynolds_externes	546
41.71	Terme_puissance_thermique_echange_impose	546
41.72	Travail_pression	546
41.73	Vitesse_derive_base	547
41.74	Vitesse_relative_base	547
42	sous_zone	547
42.1	Bloc_origine_cotes	548
42.2	Bloc_couronne	548
42.3	Bloc_tube	548
43	transport_equation_deriv	549
43.1	Transport_k_epsilon_realisable	550
43.2	Transport_k_epsilon	550
43.3	Transport_k_omega	552

44	turbulence_paroι_base	553
44.1	Loι_ciofalo_hydr	553
44.2	Loι_expert_hydr	553
44.3	Loι_puissance_hydr	554
44.4	Loι_standard_hydr	554
44.5	Loι_standard_hydr_old	554
44.6	Loι_ww_hydr	554
44.7	Negligeable	554
44.8	Paroi_tble	554
44.9	Twofloat	555
44.10	Liste_sonde_tble	555
44.10.1	Sonde_tble	556
44.11	Utau_imp	556
45	turbulence_paroι_scalaire_base	556
45.1	Loι_ww_scalaire	557
45.2	Loι_analytique_scalaire	557
45.3	Loι_expert_scalaire	557
45.4	Loι_odvm	557
45.5	Loι_paroι_nu_impose	558
45.6	Loι_standard_hydr_scalaire	558
45.7	Negligeable_scalaire	558
45.8	Paroi_tble_scal	559
45.9	Fourfloat	559
46	listobj_impl	560
46.1	Milieu_musig	560
46.2	Milieu_composite	560
46.3	List_un_pb	560
46.4	Un_pb	560
46.5	Listobj	561
47	objet_lecture	561
47.1	Quatremots	561
47.2	Entierfloat	562
47.3	Type_diffusion_turbulente_multiphase_multiple_deriv	562
48	index	562

1 Syntax to define a mathematical function

In a mathematical function, used for example in field definition, it's possible to use the predefined function (an object parser is used to evaluate the functions) :

ABS : absolute value function

COS : cosine function

SIN : sine function

TAN : tangent function

ATAN : arctangent function

EXP : exponential function

LN : natural logarithm function

SQRT : square root function

INT : integer function

ERF : error function

RND(x) : random function (values between 0 and x)

COSH : hyperbolic cosine function
 SINH : hyperbolic sine function
 TANH : hyperbolic tangent function
 ACOS : inverse cosine function
 ASIN : inverse sine function
 ATANH : inverse hyperbolic tangent function
 NOT(x) : NOT x (returns 1 if x is false, 0 otherwise)
 SGN(x) : SGN x (returns 1 if x is positive, -1 if negative, 0 if zero)
 x_AND_y : boolean logical operation AND (returns 1 if both x and y are true, else 0)
 x_OR_y : boolean logical operation OR (returns 1 if x or y is true, else 0)
 x_GT_y : greater than (returns 1 if $x > y$, else 0)
 x_GE_y : greater than or equal to (returns 1 if $x \geq y$, else 0)
 x_LT_y : less than (returns 1 if $x < y$, else 0)
 x_LE_y : less than or equal to (returns 1 if $x \leq y$, else 0)
 x_MIN_y : returns the smallest of x and y
 x_MAX_y : returns the largest of x and y
 x_MOD_y : modular division of x per y
 x_EQ_y : equal to (returns 1 if $x == y$, else 0)
 x_NEQ_y : not equal to (returns 1 if $x \neq y$, else 0)

You can also use the following operations:

+ : addition
 - : subtraction
 / : division
 * : multiplication
 % : modulo
 \$: max
 ^ : power
 < : less than
 > : greater than
 [: less than or equal to
] : greater than or equal to

You can also use the following constants:

Pi : pi value (3,1415...)

The variables which can be used are:

x,y,z : coordinates
 t : time

Examples:

Champ_front_fonc_txyz 2 cos(y+x^2) t+ln(y)
 Champ_fonc_xyz dom 2 tanh(4*y)*(0.95+0.1*rnd(1)) 0.

Possible errors:

Error 1:

Champ_fonc_txyz 1 cos(10*t)*(1<x<2)*(1<y<2)
 Previous line is wrong. It should be written as:
 Champ_fonc_txyz 1 cos(10*t)*(1<x)*(x<2)*(1<y)*(y<2)

Error 2:

Champ_front_fonc_xyz 1 20*(x<-2)+10*(y]-5)+3*(z>0)
 Previous line is wrong because negative values are not written between parentheses. It should be written as:
 Champ_front_fonc_xyz 1 20*(x<(-2))+10*(y](-5))+3*(z>0)

2 Existing & predefined fields names

Here is a list of post-processable fields, but it is not the only ones.

Physical values	Keyword for field_name	Unit
Velocity	Vitesse or Velocity	$m.s^{-1}$
Velocity residual	Vitesse_residu	$m.s^{-2}$
Kinetic energy per elements ($0.5\rho u_i ^2$)	Energie_cinetique_elem	$kg.m^{-1}.s^{-2}$
Total kinetic energy $\left(\frac{\sum_{i=1}^{nb_elem} 0.5\rho u_i ^2 vol_i}{\sum_{i=1}^{nb_elem} vol_i} \right)$	Energie_cinetique_totale	$kg.m^{-1}.s^{-2}$
Vorticity	Vorticite	s^{-1}
Pressure in incompressible flow ($P/\rho + gz$) For Front Tracking probleme ($P + \rho gz$)	Pression ¹	$Pa.m^3.kg^{-1}$ or Pa
Pressure in incompressible flow ($P+\rho gz$)	Pression_pa or Pressure	Pa
Pressure in compressible flow	Pression	Pa
Hydrostatic pressure (ρgz)	Pression_hydrostatique	Pa
Totale pressure (when quasi compressible model is used)=Pth+P	Pression_tot	Pa
Pressure gradient ($\nabla(P/\rho + gz)$)	Gradient_pression	$m.s^{-2}$
Velocity gradient	gradient_vitesse	s^{-1}
Temperature	Temperature	$^{\circ}C$ or K
Temperature residual	Temperature_residu	$^{\circ}C.s^{-1}$ or $K.s^{-1}$
Phase temperature of a two phases flow	Temperature_EquationName	$^{\circ}C$ or K
Mass transfer rate between two phases	Temperature_mpoint	$kg.m^{-2}.s^{-1}$
Temperature variance	Variance_Temperature	K^2
Temperature dissipation rate	Taux_Dissipation_Temperature	$K^2.s^{-1}$
Temperature gradient	Gradient_temperature	$K.m^{-1}$
Heat exchange coefficient	H_echange_Tref ²	$W.m^{-2}.K^{-1}$
Turbulent heat flux	Flux_Chaleur_Turbulente	$m.K.s^{-1}$
Turbulent viscosity	Viscosite_turbulente	$m^2.s^{-1}$
Turbulent dynamic viscosity (when quasi compressible model is used)	Viscosite_dynamique_turbulente	$kg.m.s^{-1}$
Turbulent kinetic energy	K	$m^2.s^{-2}$
Turbulent dissipation rate	Eps	$m^3.s^{-1}$
Turbulent quantities K and Epsilon	K_Eps	$(m^2.s^{-2}, m^3.s^{-1})$
Residuals of turbulent quantities		
... continued on next page ...		

¹The post-processed pressure is the pressure divided by the fluid's density ($P/\rho + gz$) on incompressible laminar calculation. For turbulent, pressure is $P/\rho + gz + 2/3 * k$ cause the turbulent kinetic energy is in the pressure gradient.

²Tref indicates the value of a reference temperature and must be specified by the user. For example, H_echange_293 is the keyword to use for Tref=293K.

Physical values	Keyword for field_name	Unit
K and Epsilon residuals	K_Eps_residu	$(m^2.s^{-3}, m^3.s^{-2})$
Constituent concentration	Concentration	
Constituent concentration residual	Concentration_residu	
Component velocity along X	VitesseX	$m.s^{-1}$
Component velocity along Y	VitesseY	$m.s^{-1}$
Component velocity along Z	VitesseZ	$m.s^{-1}$
Mass balance on each cell	Divergence_U	$m^3.s^{-1}$
Irradiancy	Irradiance	$W.m^{-2}$
Q-criteria	Critere_Q	s^{-1}
Distance to the wall $Y^+ = yU/\nu$ (only computed on boundaries of wall type)	Y_plus	dimensionless
Friction velocity	U_star	$m.s^{-1}$
Void fraction	alpha	dimensionless
Cell volumes	Volume_maille	m^3
Chemical potential	Potentiel_Chimique_Generalise	
Source term in non Galilean referential	Acceleration_terme_source	$m.s^{-2}$
Stability time steps	Pas_de_temps	S
Listing of boundary fluxes	Flux_bords	cf each *.out file
Volumetric porosity	Porosite_volumique	dimensionless
Distance to the wall	Distance_Paroi³	m
Volumic thermal power	Puissance_volumique	$W.m^{-3}$
Local shear strain rate defined as $\sqrt{(2S_{ij}S_{ij})}$	Taux_cisaillement	s^{-1}
Cell Courant number (VDF only)	Courant_maille	dimensionless
Cell Reynolds number (VDF only)	Reynolds_maille	dimensionless
Viscous force	viscous_force	$kg.m^2.s^{-1}$
Pressure force	pressure_force	$kg.m^2.s^{-1}$
Total force	total_force	$kg.m^2.s^{-1}$
Viscous force along X	viscous_force_x	$kg.m^2.s^{-1}$
Viscous force along Y	viscous_force_y	$kg.m^2.s^{-1}$
Viscous force along Z	viscous_force_z	$kg.m^2.s^{-1}$
Pressure force along X	pressure_force_x	$kg.m^2.s^{-1}$
Pressure force along Y	pressure_force_y	$kg.m^2.s^{-1}$
Pressure force along Z	pressure_force_z	$kg.m^2.s^{-1}$
Total force along X	total_force_x	$kg.m^2.s^{-1}$
Total force along Y	total_force_y	$kg.m^2.s^{-1}$
Total force along Z	total_force_z	$kg.m^2.s^{-1}$

3 interpret

Description: Basic class for interpreting a data file. Interpreters allow some operations to be carried out on objects.

See also: objet_u (48) { (3.46) } (3.75) export (3.61) Option_DG (3.20) ecrire_fichier_xyz_valeur (3.57) option_vdf (3.100) Option_PolyMAC (3.23) Op_Conv_EF_Stab_PolyMAC_P0P1NC_Face (3.17) Op_Conv_EF_Stab_PolyMAC_Face (3.15) Op_Conv_EF_Stab_PolyMAC_P0P1NC_Elem (3.16) Op_Conv_EF_Stab-

³distance_paroi is a field which can be used only if the mixing length model (see 2.15.1.2) is used in the data file.

[_PolyMAC_P0_Face \(3.18\)](#) [Option_IJK \(3.21\)](#) [Test_SSE_Kernels \(3.33\)](#) [multigrid_solver \(3.97\)](#) [test_solveur \(3.136\)](#) [residuals \(3.127\)](#) [facsec_expert \(3.73\)](#) [postraiter_domaine \(3.107\)](#) [Option_CGNS \(3.19\)](#) [lata_to_CGNS \(3.82\)](#) [Link_CGNS_Files \(3.11\)](#) [lata_2_other \(3.86\)](#) [lml_2_lata \(3.88\)](#) [Merge_MED \(3.12\)](#) [ecrire_champ_med \(3.55\)](#) [Write_MED \(3.9\)](#) [lata_2_med \(3.84\)](#) [debog \(3.45\)](#) [testeur \(3.137\)](#) [solve \(3.131\)](#) [associate \(3.35\)](#) [discretize \(3.53\)](#) [ecrire \(3.153\)](#) [ecrire_fichier_bin \(3.154\)](#) [read_file \(3.112\)](#) [system \(3.135\)](#) [My_Comm_Group \(3.14\)](#) [Option_Interpolation \(3.22\)](#) [execute_parallel \(3.60\)](#) [disable_TU \(3.51\)](#) [mkdir \(3.93\)](#) [stat_per_proc_perf_log \(3.132\)](#) [MultipleFiles \(3.13\)](#) [read \(3.111\)](#) [end \(3.74\)](#) [pilote_icoco \(3.105\)](#) [testeur_medcoupling \(3.138\)](#) [raffiner_isotrope \(3.110\)](#) [extruder_en20 \(3.71\)](#) [nettoiepasnoeuds \(3.99\)](#) [lire_tgrid \(3.114\)](#) [rotation \(3.128\)](#) [extract_2d_from_3d \(3.62\)](#) [decouper_bord_coincident \(3.48\)](#) [extrudeparoi \(3.68\)](#) [tetraedriser \(3.139\)](#) [regroupebord \(3.119\)](#) [corriger_frontiere_periodique \(3.43\)](#) [scatter \(3.129\)](#) [refine_mesh \(3.118\)](#) [extrudebord \(3.67\)](#) [Raffiner_isotrope_parallele \(3.26\)](#) [dilate \(3.49\)](#) [maillerparallel \(3.91\)](#) [mailler \(3.89\)](#) [discretiser_domaine \(3.52\)](#) [modifydomaineAxisId \(3.95\)](#) [triangler \(3.145\)](#) [reorienter_tetraedres \(3.124\)](#) [extraire_surface \(3.66\)](#) [lire_ideas \(3.87\)](#) [decoupebord_pour_rayonnement \(3.47\)](#) [precisiongeom \(3.108\)](#) [imprimer_flux \(3.77\)](#) [verifier_qualite_raffinements \(3.148\)](#) [polyedriser \(3.106\)](#) [extruder \(3.69\)](#) [calculer_moments \(3.41\)](#) [extraire_plan \(3.65\)](#) [distance_pari \(3.54\)](#) [axi \(3.39\)](#) [raffiner_anisotrope \(3.109\)](#) [verifier_simplexes \(3.150\)](#) [modif_bord_to_raccord \(3.94\)](#) [dimension \(3.50\)](#) [remove_elem \(3.121\)](#) [transformer \(3.144\)](#) [redresser_hexaedres_vdf \(3.117\)](#) [supprime_bord \(3.133\)](#) [bidim_axi \(3.40\)](#) [reorienter_triangles \(3.125\)](#) [interprete_geometrique_base \(3.81\)](#) [analyse_angle \(3.34\)](#) [extraire_domaine \(3.64\)](#) [reordonner \(3.126\)](#) [orientefacesbord \(3.101\)](#) [remove_invalid_internal_boundaries \(3.123\)](#) [orienter_simplexes \(3.116\)](#) [partition_multi \(3.104\)](#) [partition \(3.102\)](#) [ecriturelecturespecial \(3.58\)](#) [moyenne_volumique \(3.96\)](#) [read_med \(3.27\)](#) [integrer_champ_med \(3.79\)](#) [Parallel_io_parameters \(3.24\)](#) [verifiercoin \(3.151\)](#) [mass_source \(3.92\)](#) [espece \(3.59\)](#) [criteres_convergence \(3.44\)](#) [DebogIJK \(3.8\)](#) [Extraire_surface_ALE \(3.10\)](#) [Structural_dynamic_mesh_model \(3.29\)](#) [remaillage_ft_ijk \(3.120\)](#) [interfaces \(3.80\)](#) [DNS_QC_double \(3.6\)](#) [ALE_Neumann_BC_for_grid_problem \(3.1\)](#) [Switch_double \(3.32\)](#) [Solver_moving_mesh_ALE \(3.28\)](#) [Projection_ALE_boundary \(3.25\)](#) [Beam_model \(3.3\)](#) [DebogFT \(3.7\)](#) [imposer_vit_bords_ale \(3.76\)](#)

Usage:

interprete

3.1 Ale_neumann_bc_for_grid_problem

Description: block to indicates the names of the boundary with Neumann BC for the grid problem. By default, in the ALE grid problem, we impose a homogeneous Dirichelt-type BC on the fix boundary. This option allows you to impose also Neumann-type BCs on certain boundary.

See also: [interprete \(3\)](#)

Usage:

ALE_Neumann_BC_for_grid_problem dom bloc

where

- **dom** *str*: Name of domain.
- **bloc** *bloc_lecture (3.2)*: between the braces, you must specify the numbers of the mobile borders then list these mobile borders.
Example: `ALE_Neumann_BC_for_grid_problem dom_name { 1 boundary_name }`

3.2 Bloc_lecture

Description: to read between two braces

See also: [objet_lecture \(47\)](#) [solveur_petsc_option_cli \(3.2.1\)](#) [bloc_criteres_convergence \(3.2.2\)](#)

Usage:

bloc_lecture

where

- **bloc_lecture** *str*

3.2.1 Solveur_petsc_option_cli

Description: solver

See also: (3.2)

Usage:

bloc_lecture

where

- **bloc_lecture** *str*

3.2.2 Bloc_criteres_convergence

Description: Not set

See also: (3.2)

Usage:

bloc_lecture

where

- **bloc_lecture** *str*

3.3 Beam_model

Description: Reduced mechanical model: a beam model. Resolution based on a modal analysis. Temporal discretization: Newmark or Hilber-Hughes-Taylor (HHT)

See also: `interpret` (3)

Usage:

Beam_model dom bloc

where

- **dom** *str*: domain name
- **bloc** *bloc_lecture_beam_model* (3.4)

3.4 Bloc_lecture_beam_model

Description: bloc

See also: `objet_lecture` (47)

Usage:

aco nb_beam nb_beam_val Name Name_of_beam bloc [Name2] [Name_of_beam2] [bloc2] acof

where

- **aco** *str* into [`'`]: Opening curly bracket.
- **nb_beam** *str* into [`'nb_beam'`]: Keyword to specify the number of beams

- **nb_beam_val** *int*: Number of beams
- **Name** *str* into ['name']: keyword to specify the Name of the beam (the name must match with the name of the edge in the fluid domain)
- **Name_of_beam** *str*: keyword to specify the Name of the beam (the name must match with the name of the edge in the fluid domain)
- **bloc** *bloc_poutre* (3.4.1)
- **Name2** *str* into ['name']: keyword to specify the Name of the beam (the name must match with the name of the edge in the fluid domain)
- **Name_of_beam2** *str*: keyword to specify the Name of the beam (the name must match with the name of the edge in the fluid domain)
- **bloc2** *bloc_poutre* (3.4.1)
- **acof** *str* into ['']: Closing curly bracket.

3.4.1 Bloc_poutre

Description: Read poutre bloc

See also: objet_lecture (47)

Usage:

```
{
    nb_modes int
    longitudinal_axis str
    bendingDirection str
    nb_planes int
    NewmarkTimeScheme newmarktimescheme_deriv
    Mass_and_stiffness_file_name str
    Absc_file_name str
    Modal_deformation_file_name n word1 word2 ... wordn
    [ Young_Module float]
    [ Rho_beam float]
    [ BaseCenterCoordinates x1 x2 (x3)]
    [ CI_file_name str]
    [ Restart_file_name str]
    [ Output_position_1D n x1 x2 ... xn]
    [ Output_position_3D listpoints]
}
```

where

- **nb_modes** *int*: Number of modes. If there are two planes, indicate the total for both planes.
- **longitudinal_axis** *str*: x, y, z. Axis along the length of the beam
- **bendingDirection** *str*: x, y, z. Direction of bending. If two planes, give the first one, the second direction is determine automatically
- **nb_planes** *int*: Number of planes used in the beam dynamic model
- **NewmarkTimeScheme** *newmarktimescheme_deriv* (3.4.2): Solve the beam dynamics. Time integration scheme: choice between MA (Newmark mean acceleration), FD (Newmark finite differences), and HHT alpha (Hilber-Hughes-Taylor, alpha usually -0.1)
- **Mass_and_stiffness_file_name** *str*: Name of the file containing the diagonal modal mass, stiffness, and damping matrices. The file must contain nb_modes lines. When several bending planes are defined, the modes must be ordered plane-by-plane. The first modes_per_plane lines correspond to plane 0, the next modes_per_plane lines correspond to plane 1.
- **Absc_file_name** *str*: Name of the file containing the coordinates of the Beam

- **Modal_deformation_file_name** *n word1 word2 ... wordn*: Name of the file containing the modal deformation of the Beam (Two-column modal deformation file: W(X) modal displacement and the rotation dW/dX modal rotation).
- **Young_Module** *float*: Young Module
- **Rho_beam** *float*: Beam density
- **BaseCenterCoordinates** *x1 x2 (x3)*: position of the base center coordinates on the Beam
- **CI_file_name** *str*: Name of the file containing the initial condition of the Beam. The initial condition file must contain nb_modes displacement values, one per line. When several bending planes are defined, the modes must be ordered plane-by-plane. The first modes_per_plane lines correspond to plane 0, the next modes_per_plane lines correspond to plane 1.
- **Restart_file_name** *str*: SaveBeamForRestart.txt file to restart the calculation. The file must contain nb_modes displacement values, one per line. When several bending planes are defined, the modes must be ordered plane-by-plane. The first modes_per_plane lines correspond to plane 0, the next modes_per_plane lines correspond to plane 1.
- **Output_position_1D** *n x1 x2 ... xn*: nb_points position Post-traitement of specific points on the Beam
- **Output_position_3D** *listpoints* (3.4.6): nb_points position Post-traitement of specific points on the 3d FSI boundary

3.4.2 Newmarktimescheme_deriv

Description: Solve the beam dynamics. Selection of time integration scheme.

See also: objet_lecture (47) HHT (3.4.3) MA (3.4.4) FD (3.4.5)

Usage:

3.4.3 Hht

Description: HHT alpha (Hilber-Hughes-Taylor, alpha usually -0.1) time integration scheme.

See also: NewmarkTimeScheme_deriv (3.4.2)

Usage:

HHT [**alpha**]

where

- **alpha** *float*: usually, alpha is set to -0.1

3.4.4 Ma

Description: MA (Newmark mean acceleration) time integration scheme.

See also: NewmarkTimeScheme_deriv (3.4.2)

Usage:

MA

3.4.5 Fd

Description: FD (Newmark finite differences) time integration scheme. Warning: this scheme is conditionally stable. The time step should satisfy the corresponding stability constraint, but this implementation does not automatically enforce it. The Newmark finite difference scheme is retained primarily for advanced

users and benchmarking purposes.

See also: NewmarkTimeScheme_deriv ([3.4.2](#))

Usage:

FD

3.4.6 Listpoints

Description: Points.

See also: listobj ([46.5](#))

Usage:

n object1 object2

list of *un_point* ([3.4.7](#))

3.4.7 Un_point

Description: A point.

See also: objet_lecture ([47](#))

Usage:

pos

where

- **pos** *x1 x2 (x3)*: Point coordinates.

3.5 Create_domain_from_sub_domain

Description: This keyword fills the domain *domaine_final* with the subdomaine *par_sous_zone* from the domain *domaine_init*. It is very useful when meshing several mediums with Gmsh. Each medium will be defined as a subdomaine into Gmsh. A MED mesh file will be saved from Gmsh and read with Lire_Med keyword by the TRUST data file. And with this keyword, a domain will be created for each medium in the TRUST data file.

See also: interprete_geometrique_base ([3.81](#))

Usage:

Create_domain_from_sub_domain {

 [**domaine_final** *str*]

 [**par_sous_dom**|**par_sous_zone** *str*]

domaine_init *str*

}

where

- **domaine_final** *str*: new domain in which faces are stored
- **par_sous_dom**|**par_sous_zone** *str*: a sub-area (a group in a MED file) allowing to choose the elements
- **domaine_init** *str*: initial domain

3.6 Dns_qc_double

Description: not_set

See also: [interpret \(3\)](#)

Usage:

DNS_QC_double bloc

where

- **bloc** *bloc_lecture* ([3.2](#))

3.7 Debugft

Description: not_set

See also: [interpret \(3\)](#)

Usage:

DebugFT {

```
[ mode str into ['disabled', 'write_pass', 'check_pass']]  
[ filename str]  
[ seuil_absolu float]  
[ seuil_relatif float]  
[ seuil_minimum_relatif float]
```

}

where

- **mode** *str* into ['disabled', 'write_pass', 'check_pass']
- **filename** *str*
- **seuil_absolu** *float*
- **seuil_relatif** *float*
- **seuil_minimum_relatif** *float*

3.8 Debugijk

Description: not_set

See also: [interpret \(3\)](#)

Usage:

DebugIJK {

```
[ mode str]  
[ filename str]  
[ seuil_absolu float]  
[ seuil_relatif float]  
[ seuil_minimum_relatif float]
```

}

where

- **mode** *str*

- **filename** *str*
- **seuil_absolu** *float*
- **seuil_relatif** *float*
- **seuil_minimum_relatif** *float*

3.9 Write_med

Description: Write a domain to MED format into a file.

See also: [interpret](#) (3)

Usage:

Write_MED **nom_dom** **file**

where

- **nom_dom** *str*: Name of domain.
- **file** *str*: Name of file.

3.10 Extraire_surface_ale

Description: Extraire_surface_ALE in order to extract a surface on a mobile boundary (with ALE description).

Keyword to specify that the extract surface is done on a mobile domain. The surface mesh is defined by one or two conditions. The first condition is about elements with Condition_elements. For example: Condition_elements $x*x+y*y+z*z<1$

Will define a surface mesh with external faces of the mesh elements inside the sphere of radius 1 located at (0,0,0). The second condition Condition_faces is useful to give a restriction.

By default, the faces from the boundaries are not added to the surface mesh excepted if option avec_les_bords is given (all the boundaries are added), or if the option avec_certaines_bords is used to add only some boundaries.

Keyword Discretize should have already been used to read the object.

See also: [interpret](#) (3)

Usage:

Extraire_surface_ALE {

```

    domaine str
    probleme str
    [ condition_elements str]
    [ condition_faces str]
    [ avec_les_bords ]
    [ avec_certaines_bords n word1 word2 ... wordn]

```

}

where

- **domaine** *str*: Domain in which faces are saved
- **probleme** *str*: Problem from which faces should be extracted
- **condition_elements** *str*
- **condition_faces** *str*
- **avec_les_bords**
- **avec_certaines_bords** *n word1 word2 ... wordn*

3.11 Link_cgns_files

Description: Creates a single CGNS xxxx.cgns file that links to a xxxx.grid.cgns and xxxx.solution.*.cgns files

See also: [interpret \(3\)](#)

Usage:

Link_CGNS_Files **base_name** **output_name**

where

- **base_name** *str*: Base name of the gid/solution cgns files.
- **output_name** *str*: Name of the output cgns file.

3.12 Merge_med

Description: This keyword allows to merge multiple MED files produced during a parallel computation into a single MED file.

See also: [interpret \(3\)](#)

Usage:

Merge_MED **med_files_base_name** **time_iterations**

where

- **med_files_base_name** *str*: Base name of multiple med files that should appear as base_name-
_xxxxx.med, where xxxxx denotes the MPI rank number. If you specify NOM_DU_CAS, it will automatically take the basename from your datafile's name.
- **time_iterations** *str* into ['all_times', 'last_time']: Identifies whether to merge all time iterations present in the MED files or only the last one.

3.13 Multiplefiles

Description: Change MPI rank limit for multiple files during I/O

See also: [interpret \(3\)](#)

Usage:

MultipleFiles **type**

where

- **type** *int*: New MPI rank limit

3.14 My_comm_group

Description: This keyword allows to create a user MPI Comm Group of size N using the processors allocated to TRUST. The set of processors is split in N subsets.

See also: [interpret \(3\)](#)

Usage:

My_Comm_Group {


```

    group_nb int
}
where

```

- **group_nb** *int*: Number of groups to define in your Comm Group.

3.15 Op_conv_ef_stab_polymac_face

Description: Class Op_Conv_EF_Stab_PolyMAC_Face_PolyMAC

See also: [interpret \(3\)](#)

Usage:

```

Op_Conv_EF_Stab_PolyMAC_Face {
    [ alpha float ]
}
where

```

- **alpha** *float*: parametre ajustant la stabilisation de 0 (schema centre) a 1 (schema amont)

3.16 Op_conv_ef_stab_polymac_p0p1nc_elem

Description: Class Op_Conv_EF_Stab_PolyMAC_P0P1NC_Elem

See also: [interpret \(3\)](#)

Usage:

```

Op_Conv_EF_Stab_PolyMAC_P0P1NC_Elem {
    [ alpha float ]
}
where

```

- **alpha** *float*: parametre ajustant la stabilisation de 0 (schema centre) a 1 (schema amont)

3.17 Op_conv_ef_stab_polymac_p0p1nc_face

Description: Class Op_Conv_EF_Stab_PolyMAC_P0P1NC_Face

See also: [interpret \(3\)](#)

Usage:

3.18 Op_conv_ef_stab_polymac_p0_face

Description: Class Op_Conv_EF_Stab_PolyMAC_P0_Face

See also: [interpret \(3\)](#)

Usage:

3.19 Option_cgns

Description: Class for CGNS options.

See also: [interpret](#) (3)

Usage:

```
Option_CGNS {  
    [ single_precision ]  
    [ parallel_over_zone ]  
    [ use_links ]  
    [ file_per_comm_group ]  
    [ single_safe_file ]  
    [ close_every_n int]  
    [ flush_every_n int]  
}
```

where

- **single_precision** : If used, data will be written with a single_precision format inside the CGNS file (it concerns both mesh coordinates and field values).
- **parallel_over_zone** : If used, data will be written in separate zones (ie: one zone per processor). This is not so performant but easier to read later ...
- **use_links** : If used, data will be written in separate files; one file for mesh, and then one file for solution time. Links will be used.
- **file_per_comm_group** : If used, data will be written (at each comm group) in separate files; one file for mesh, and then one file for solution time. Links will be used.
- **single_safe_file** : If used, data will be written in a single file that will be opened and closed at each dt post so that file can be visualized in live. Safer if simulation stops, the file can be used.
- **close_every_n** *int*: Used to fix the opening/closing frequency when the SINGLE_SAFE_FILE option is used.
- **flush_every_n** *int*: Used to fix the flush-to-disc frequency when the SINGLE_SAFE_FILE option is used.

3.20 Option_dg

Description: Class for DG options.

See also: [interpret](#) (3)

Usage:

```
Option_DG {  
    [ order int]  
    [ velocity_order int]  
    [ pressure_order int]  
    [ temperature_order int]  
    [ gram_schmidt int]  
}
```

where

- **order** *int*: global order for the DG unknowns (1 by default)
- **velocity_order** *int*: optional order for DG velocity unknown

- **pressure_order** *int*: optional order for DG pressure unknown
- **temperature_order** *int*: optional order for DG temperature unknown
- **gram_schmidt** *int*: Gram Schmidt orthogonalization (1 by default)

3.21 Option_ijk

Description: Class of IJK options.

See also: [interpret \(3\)](#)

Usage:

```
Option_IJK {
    [ check_divergence ]
    [ disable_diphasique ]
```

```
}
```

where

- **check_divergence** : Flag to compute and print the value of $\text{div}(\mathbf{u})$ after each pressure-correction
- **disable_diphasique** : Disable all calculations related to interfaces (phase properties, interfacial force, ...)

3.22 Option_interpolation

Description: Class for interpolation fields using MEDCoupling.

See also: [interpret \(3\)](#)

Usage:

```
Option_Interpolation {
    [ without_declsans_dec ]
    [ sharing_algo int]
```

```
}
```

where

- **without_declsans_dec** : Use remapper even for a parallel calculation
- **sharing_algo** *int*: Setting the DEC sharing algo : 0,1,2

3.23 Option_polymac

Description: Class of PolyMAC options.

See also: [interpret \(3\)](#)

Usage:

```
Option_PolyMAC {
    [ use_osqp ]
    [ vdf_mesh|maillage_vdf ]
    [ interp_ve1 ]
    [ traitement_axi ]
```

}
where

- **use_osqp** : Flag to use the old formulation of the M2 matrix provided by the OSQP library. Only useful for PolyMAC version.
- **vdf_meshmaillage_vdf** : Flag used to force the calculation of the equiv tab.
- **interp_ve1** : Flag to enable a first-order face-to-element velocity interpolation. By default, it is not activated which means a second order interpolation. Only useful for PolyMAC_P0 version.
- **traitement_axi** : Flag used to relax the time-step stability criterion in case of a thin slice geometry while modelling an axi-symmetrical case. Only useful for PolyMAC_P0 version.

3.24 Parallel_io_parameters

Description: Object to handle parallel files in IJK discretization

See also: [interpret \(3\)](#)

Usage:

```
Parallel_io_parameters {  
    [ block_size_bytes int]  
    [ block_size_megabytes int]  
    [ writing_processes int]  
    [ bench_ijk_splitting_write str]  
    [ bench_ijk_splitting_read str]  
}
```

where

- **block_size_bytes** *int*: File writes will be performed by chunks of this size (in bytes). This parameter will not be taken into account if **block_size_megabytes** has been defined
- **block_size_megabytes** *int*: File writes will be performed by chunks of this size (in megabytes). The size should be a multiple of the GPFS block size or lustre stripping size (typically several megabytes)
- **writing_processes** *int*: This is the number of processes that will write concurrently to the file system (this must be set according to the capacity of the filesystem, set to 1 on small computers, can be up to 64 or 128 on very large systems).
- **bench_ijk_splitting_write** *str*: Name of the splitting object we want to use to run a parallel write bench (optional parameter)
- **bench_ijk_splitting_read** *str*: Name of the splitting object we want to use to run a parallel read bench (optional parameter)

3.25 Projection_ale_boundary

Description: block to compute the projection of a modal function on a mobile boundary. Use to compute modal added coefficients in FSI.

See also: [interpret \(3\)](#)

Usage:

```
Projection_ALE_boundary dom bloc  
where
```

- **dom** *str*: Name of domain.

- **bloc** *bloc_lecture* (3.2): between the braces, you must specify the numbers of the mobile borders then list these mobile borders and indicate the modal function which must be projected on these boundaries.

Example: Projection_ALE_boundary dom_name { 1 boundary_name 3 0.sin(pi*x)*1.e-4 0. }

3.26 Raffiner_isotrope_parallele

Description: Refine parallel mesh in parallel

See also: [interpret \(3\)](#)

Usage:

```
Raffiner_isotrope_parallele {
    name_of_initial_zones|name_of_initial_domaines str
    name_of_new_zones|name_of_new_domaines str
    [ ascii ]
    [ single_hdf ]
}
```

where

- **name_of_initial_zones|name_of_initial_domaines** *str*: name of initial Domaines
- **name_of_new_zones|name_of_new_domaines** *str*: name of new Domaines
- **ascii** : writing Domaines in ascii format
- **single_hdf** : writing Domaines in hdf format

3.27 Read_med

Synonymous: **lire_med**

Description: Keyword to read MED mesh files where 'domain' corresponds to the domain name, 'file' corresponds to the file (written in the MED format) containing the mesh named mesh_name.

Note about naming boundaries: When reading 'file', TRUST will detect boundaries between domains (Raccord) when the name of the boundary begins by 'type_raccord

-'_. For example, a boundary named type_raccord_wall in 'file' will be considered by TRUST as a boundary named 'wall' between two domains.

NB: To read several domains from a mesh issued from a MED file, use Read_Med to read the mesh then use Create_domain_from_sub_domain keyword.

NB: If the MED file contains one or several subdomaine defined as a group of volumes, then Read_MED will read it and will create two files domain_name_ssz.geo and domain_name_ssz_par.geo defining the subdomaines for sequential and/or parallel calculations. These subdomaines will be read in sequential in the datafile by including (after Read_Med keyword) something like:

Read_Med

Read_file domain_name_ssz.geo ;

During the parallel calculation, you will include something:

Scatter { ... }

Read_file domain_name_ssz_par.geo ;

See also: [interpret \(3\)](#)

Usage:

```
read_med {
```

```

[ convertalltopoly ]
domain|domain str
fichier|file str
[ maillage|mesh str]
[ excluder_groupe|exclude_groups n word1 word2 ... wordn]
[ includer_groupe|faces_additionnels|include_additional_face_groups n word1 word2 ... wordn]
}
where

```

- **convertalltopoly** : Option to convert mesh with mixed cells into polyhedral/polygonal cells
- **domain|domain *str*** : Corresponds to the domain name.
- **fichier|file *str*** : File (written in the MED format, with extension '.med') containing the mesh
- **maillage|mesh *str*** : Name of the mesh in med file. If not specified, the first mesh will be read.
- **excluder_groupe|exclude_groups *n word1 word2 ... wordn*** : List of face groups to skip in the MED file.
- **includer_groupe|faces_additionnels|include_additional_face_groups *n word1 word2 ... wordn*** : List of face groups to read and register in the MED file.

3.28 Solver_moving_mesh_ale

Description: Solver used to solve the system giving the mesh velocity for the ALE (Arbitrary Lagrangian-Eulerian) framework.

See also: [interpret \(3\)](#)

Usage:

Solver_moving_mesh_ALE dom bloc

where

- **dom *str*** : Name of domain.
- **bloc *bloc_lecture* (3.2)**: Example: { PETSC GCP { precondition ssor { omega 1.5 } seuil 1e-7 impr } }

3.29 Structural_dynamic_mesh_model

Description: Fictitious structural model for mesh motion. Link with MGIS library

See also: [interpret \(3\)](#)

Usage:

Structural_dynamic_mesh_model dom bloc

where

- **dom *str*** : domain name
- **bloc *bloc_lecture_structural_dynamic_mesh_model* (3.30)**

3.30 Bloc_lecture_structural_dynamic_mesh_model

Description: bloc

See also: [objet_lecture \(47\)](#)

Usage:

aco **Mfront_library** **Mfront_model_name** **Mfront_material_property** [**YoungModulus**] [**Density**] [**Inertial_Damping**] [**Grid_dt_min**] **acof**
 where

- **aco** *str* into [' ']: Opening curly bracket.
- **Mfront_library** *str* into ['Mfront_library']: Keyword to specify the path_to_libBehaviour.so If the user does not define the MFront library path, we use the default one instead: path = \$project-_directory/share/MFront_material_libraries/<MFront_library_keyword>/src/libBehaviour.so
- **Mfront_model_name** *str* into ['Mfront_model_name']: keyword to specify the Mfront model. Choice between Ogden and SaintVenantKirchhoffElasticity.
- **Mfront_material_property** *str* into ['Mfront_material_property']: keyword to specify the material property. Eg. Ogden_alpha_, Ogden_mu_, Ogden_K
- **YoungModulus** *float*: Young Module
- **Density** *float*: fictitious structural density
- **Inertial_Damping** *float*: fictitious structural inertial damping
- **Grid_dt_min** *float*: fictitious structural time step
- **acof** *str* into [' ']: Closing curly bracket.

3.31 Switch_ft_double

Description: not_set

See also: Switch_double (3.32)

Usage:

```
Switch_FT_double {
    [ old_ijk_splitting_ft_extension int]
    old_ijk_splitting str
    new_ijk_splitting str
    nom_sauvegarde str
    nom_reprise str
    [ direct_write int]
}
```

where

- **old_ijk_splitting_ft_extension** *int*
- **old_ijk_splitting** *str* for inheritance
- **new_ijk_splitting** *str* for inheritance
- **nom_sauvegarde** *str* for inheritance
- **nom_reprise** *str* for inheritance
- **direct_write** *int* for inheritance

3.32 Switch_double

Description: not_set

See also: interpret (3) Switch_FT_double (3.31)

Usage:

```
Switch_double {
    old_ijk_splitting str
```

```

    new_ijk_splitting str
    nom_sauvegarde str
    nom_reprise str
    [ direct_write int]
}
where

```

- **old_ijk_splitting** *str*
- **new_ijk_splitting** *str*
- **nom_sauvegarde** *str*
- **nom_reprise** *str*
- **direct_write** *int*

3.33 Test_sse_kernels

Description: Object to test the different kernel methods used in the multigrid solver in IJK discretization

See also: [interpret \(3\)](#)

Usage:

```

Test_SSE_Kernels {
    [ nmax int]
}
where

```

- **nmax** *int*: Number of tests we want to perform

3.34 Analyse_angle

Description: Keyword Analyse_angle prints the histogram of the largest angle of each mesh elements of the domain named name_domain. nb_histo is the histogram number of bins. It is called by default during the domain discretization with nb_histo set to 18. Useful to check the number of elements with angles above 90 degrees.

See also: [interpret \(3\)](#)

Usage:

```

analyse_angle domain_name nb_histo
where

```

- **domain_name** *str*: Name of domain to resequence.
- **nb_histo** *int*

3.35 Associate

Synonymous: **associer**

Description: This interpreter allows one object to be associated with another. The order of the two objects in this instruction is not important. The object objet_2 is associated to objet_1 if this makes sense; if not either objet_1 is associated to objet_2 or the program exits with error because it cannot execute the

Associate (Associer) instruction. For example, to calculate water flow in a pipe, a Pb_Hydraulique type object needs to be defined. But also a Domaine type object to represent the pipe, a Scheme_euler_explicit type object for time discretization, a discretization type object (VDF or VEF) and a Fluide_Incompressible type object which will contain the water properties. These objects must then all be associated with the problem.

See also: [interpret](#) (3) [associer_pbmng_pbgglobal](#) (3.38) [associer_pbmng_pbfin](#) (3.37) [associer_algo](#) (3.36)

Usage:

associate objet_1 objet_2

where

- **objet_1** *str*: Objet_1
- **objet_2** *str*: Objet_2

3.36 Associer_algo

Description: This interpreter allows an algorithm to be associated with multi-grid problem.

See also: [associate](#) (3.35)

Usage:

associer_algo objet_1 objet_2

where

- **objet_1** *str*: Objet_1
- **objet_2** *str*: Objet_2

3.37 Associer_pbmng_pbfin

Description: This interpreter allows a local problem to be associated with multi-grid problem.

See also: [associate](#) (3.35)

Usage:

associer_pbmng_pbfin objet_1 objet_2

where

- **objet_1** *str*: Objet_1
- **objet_2** *str*: Objet_2

3.38 Associer_pbmng_pbgglobal

Description: This interpreter allows a global problem to be associated with multi-grid problem.

See also: [associate](#) (3.35)

Usage:

associer_pbmng_pbgglobal objet_1 objet_2

where

- **objet_1** *str*: Objet_1
- **objet_2** *str*: Objet_2

3.39 Axi

Description: This keyword allows a 3D calculation to be executed using cylindrical coordinates (R, θ, Z) . If this instruction is not included, calculations are carried out using Cartesian coordinates.

See also: [interpret](#) (3)

Usage:

axi

3.40 Bidim_axi

Description: Keyword allowing a 2D calculation to be executed using axisymmetric coordinates (R, Z) . If this instruction is not included, calculations are carried out using Cartesian coordinates.

See also: [interpret](#) (3)

Usage:

bidim_axi

3.41 Calculer_moments

Description: Calculates and prints the torque (moment of force) exerted by the fluid on each boundary in output files (.out) of the domain `nom_dom`.

See also: [interpret](#) (3)

Usage:

calculer_moments nom_dom mot

where

- **nom_dom** *str*: Name of domain.
- **mot** *lecture_bloc_moment_base* (3.42): Keyword.

3.42 Lecture_bloc_moment_base

Description: Auxiliary class to compute and print the moments.

See also: [objet_lecture](#) (47) [calcul](#) (3.42.1) [centre_de_gravite](#) (3.42.2)

Usage:

3.42.1 Calcul

Description: The centre of gravity will be calculated.

See also: (3.42)

Usage:

calcul

3.42.2 Centre_de_gravite

Description: To specify the centre of gravity.

See also: (3.42)

Usage:

centre_de_gravite point
where

- **point** *un_point* (3.4.7): A centre of gravity.

3.43 Corriger_frontiere_periodique

Description: The `Corriger_frontiere_periodique` keyword is mandatory to first define the periodic boundaries, to reorder the faces and eventually fix unaligned nodes of these boundaries. Faces on one side of the periodic domain are put first, then the faces on the opposite side, in the same order. It must be run in sequential before mesh splitting.

See also: `interprete` (3)

Usage:

corriger_frontiere_periodique {
 domaine *str*
 bord *str*
 [**direction** *n x1 x2 ... xn*]
 [**fichier_post** *str*]
}
where

- **domaine** *str*: Name of domain.
- **bord** *str*: the name of the boundary (which must contain two opposite sides of the domain)
- **direction** *n x1 x2 ... xn*: defines the periodicity direction vector (a vector that points from one node on one side to the opposite node on the other side). This vector must be given if the automatic algorithm fails, that is:
 - when the node coordinates are not perfectly periodic
 - when the periodic direction is not aligned with the normal vector of the boundary faces
- **fichier_post** *str*: .

3.44 Criteres_convergence

Description: convergence criteria

See also: `interprete` (3)

Usage:

aco [**inco**] [**val**] **acof**
where

- **aco** *str* into ['{']: Opening curly bracket.
- **inco** *str*: Unknown (i.e.: *alpha*, *temperature*, *velocity* and *pressure*)
- **val** *float*: Convergence threshold
- **acof** *str* into ['}']: Closing curly bracket.

3.45 Debug

Description: Class to debug some differences between two TRUST versions on a same data file.

If you want to compare the results of the same code in sequential and parallel calculation, first run (mode=0) in sequential mode (the files fichier1 and fichier2 will be written first) then the second run in parallel calculation (mode=1).

During the first run (mode=0), it prints into the file DEBOG, values at different points of the code thanks to the C++ instruction call. see for example in Kernel/Framework/Resoudre.cpp file the instruction: `Debug::verifier(msg,value);` Where msg is a string and value may be a double, an integer or an array.

During the second run (mode=1), it prints into a file Err_Debug.dbg the same messages than in the DEBOG file and checks if the differences between results from both codes are less than a given value (error). If not, it prints Ok else show the differences and the lines where it occurred.

See also: [interpret \(3\)](#)

Usage:

debug pb fichier1 fichier2 seuil mode

where

- **pb** *str*: Name of the problem to debug.
- **fichier1** *str*: Name of the file where domain will be written in sequential calculation.
- **fichier2** *str*: Name of the file where faces will be written in sequential calculation.
- **seuil** *float*: Minimal value (by default 1.e-20) for the differences between the two codes.
- **mode** *int*: By default -1 (nothing is written in the different files), you will set 0 for the sequential run, and 1 for the parallel run.

3.46 {

Description: Block's beginning.

See also: [interpret \(3\)](#)

Usage:

{

3.47 Decoupebord_pour_rayonnement

Synonymous: **decoupebord**

Description: To subdivide the external boundary of a domain into several parts (may be useful for better accuracy when using radiation model in transparent medium). To specify the boundaries of the fine_domain_name domain to be splitted. These boundaries will be cut according the coarse mesh defined by either the keyword `domaine_grossier` (each boundary face of the coarse mesh coarse_domain_name will be used to group boundary faces of the fine mesh to define a new boundary), either by the keyword `nb_parts_naif` (each boundary of the fine mesh is splitted into a partition with nx*ny*nz elements), either by a geometric condition given by a formulae with the keyword `condition_geometrique`. If used, the coarse_domain_name domain should have the same boundaries name of the fine_domain_name domain.

A mesh file (ASCII format, except if binaire option is specified) named by default newgeom (or specified by the `nom_fichier_sortie` keyword) will be created and will contain the fine_domain_name domain with the splitted boundaries named boundary_name

See also: [interpret \(3\)](#)

Usage:

decoupebord_pour_rayonnement {

```

    domaine str
    [ domaine_grossier str]
    [ nb_parts_naif n n1 n2 ... nn]
    [ nb_parts_geom n n1 n2 ... nn]
    [ condition_geometrique n word1 word2 ... wordn]
    bords_a_decouper n word1 word2 ... wordn
    [ nom_fichier_sortie str]
    [ binaire int]
}
where

```

- **domaine** *str*
- **domaine_grossier** *str*
- **nb_parts_naif** *n n1 n2 ... nn*
- **nb_parts_geom** *n n1 n2 ... nn*
- **condition_geometrique** *n word1 word2 ... wordn*
- **bords_a_decouper** *n word1 word2 ... wordn*
- **nom_fichier_sortie** *str*
- **binaire** *int*

3.48 Decouper_bord_coincident

Description: In case of non-coincident meshes and a `paroi_contact` condition, run is stopped and two external files are automatically generated in VEF (`connectivity_failed_boundary_name` and `connectivity_failed_pb_name.med`). In 2D, the keyword `Decouper_bord_coincident` associated to the `connectivity_failed_boundary_name` file allows to generate a new coincident mesh.

See also: [interpret \(3\)](#)

Usage:

```

decouper_bord_coincident domain_name bord
where

```

- **domain_name** *str*: Name of domain.
- **bord** *str*: `connectivity_failed_boundary_name`

3.49 Dilate

Description: Keyword to multiply the whole coordinates of the geometry.

See also: [interpret \(3\)](#)

Usage:

```

dilate domain_name alpha
where

```

- **domain_name** *str*: Name of domain.
- **alpha** *float*: Value of dilatation coefficient.

3.50 Dimension

Description: Keyword allowing calculation dimensions to be set (2D or 3D), where `dim` is an integer set to 2 or 3. This instruction is mandatory.

See also: [interpret \(3\)](#)

Usage:

dimension dim

where

- **dim** *int into [2, 3]*: Number of dimensions.

3.51 Disable_tu

Description: Flag to disable the writing of the .TU files

See also: [interpret \(3\)](#)

Usage:

disable_TU

3.52 Discretiser_domaine

Description: Useful to discretize the domain `domain_name` (faces will be created) without defining a problem.

See also: [interpret \(3\)](#)

Usage:

discretiser_domaine domain_name

where

- **domain_name** *str*: Name of the domain.

3.53 Discretize

Synonymous: **discretiser**

Description: Keyword to discretise a problem `problem_name` according to the discretization `dis`.

IMPORTANT: A number of objects must be already associated (a domain, time scheme, central object) prior to invoking the Discretize (Discretiser) keyword. The physical properties of this central object must also have been read.

See also: [interpret \(3\)](#)

Usage:

discretize problem_name dis

where

- **problem_name** *str*: Name of problem.
- **dis** *str*: Name of the discretization object.

3.54 Distance_pari

Description: Class to generate external file Wall_length.xyz devoted for instance, for mixing length modelling. In this file, are saved the coordinates of each element (center of gravity) of dom domain and minimum distance between this point and boundaries (specified bords) that user specifies in data file (typically, those associated to walls). A field Distance_pari is available to post process the distance to the wall.

See also: [interpret \(3\)](#)

Usage:

distance_pari dom bords format

where

- **dom** *str*: Name of domain.
- **bords** *n word1 word2 ... wordn*: Boundaries.
- **format** *str* into [*'binaire'*, *'formatte'*]: Value for format may be binaire (a binary file Wall_length.xyz is written) or formatte (moreover, a formatted file Wall_length_formatted.xyz is written).

3.55 Ecrire_champ_med

Description: Keyword to write a field to MED format into a file.

See also: [interpret \(3\)](#)

Usage:

ecrire_champ_med nom_dom nom_chp file

where

- **nom_dom** *str*: domain name
- **nom_chp** *str*: field name
- **file** *str*: file name

3.56 Ecrire_fichier_formatte

Description: Keyword to write the object of name name_obj to a file filename in ASCII format.

See also: [ecrire_fichier_bin \(3.154\)](#)

Usage:

ecrire_fichier_formatte name_obj filename

where

- **name_obj** *str*: Name of the object to be written.
- **filename** *str*: Name of the file.

3.57 Ecrire_fichier_xyz_valeur

Description: This keyword is used to write the values of a field only for some boundaries in a text file with the following format: n_valeur

x_1 y_1 [z_1] val_1

...

x_n y_n [z_n] val_n

The created files are named : pbname_fieldname_[boundaryname]_time.dat

See also: [interpret \(3\)](#)

Usage:

```
ecrire_fichier_xyz_valeur {  
    [ binary_file ]  
    [ dt float]  
    [ fields n word1 word2 ... wordn]  
    [ boundaries n word1 word2 ... wordn]  
}
```

where

- **binary_file** : To write file in binary format
- **dt** *float*: File writing frequency
- **fields** *n word1 word2 ... wordn*: Names of the fields we want to write
- **boundaries** *n word1 word2 ... wordn*: Names of the boundaries on which to write fields

3.58 Ecriturelecturespecial

Description: Class to write or not to write a .xyz file on the disk at the end of the calculation.

See also: [interpret \(3\)](#)

Usage:

```
ecriturelecturespecial type  
where
```

- **type** *str*: If set to 0, no xyz file is created. If set to 1 (the default) the .xyz file is written at the end of the computation.

3.59 Espece

Description: not_set

See also: [interpret \(3\)](#)

Usage:

```
espece {  
    mu champ_base  
    cp champ_base  
    masse_molaire float  
}
```

where

- **mu** *champ_base* (20.1): Species dynamic viscosity value (kg.m-1.s-1).
- **cp** *champ_base* (20.1): Species specific heat value (J.kg-1.K-1).
- **masse_molaire** *float*: Species molar mass.

3.60 Execute_parallel

Description: This keyword allows to run several computations in parallel on processors allocated to TRUST. The set of processors is split in N subsets and each subset will read and execute a different data file. Error messages usually written to stderr and stdout are redirected to .log files (journaling must be activated).

See also: [interpret \(3\)](#)

Usage:

```
execute_parallel {  
    liste_cas  n word1 word2 ... wordn  
    [ nb_procs  n n1 n2 ... nn ]  
}  
where
```

- **liste_cas** *n word1 word2 ... wordn*: N datafile1 ... datafileN. datafileX the name of a TRUST data file without the .data extension.
- **nb_procs** *n n1 n2 ... nn*: nb_procs is the number of processors needed to run each data file. If not given, TRUST assumes that computations are sequential.

3.61 Export

Description: Class to make the object have a global range, if not its range will apply to the block only (the associated object will be destroyed on exiting the block).

See also: [interpret \(3\)](#)

Usage:

export

3.62 Extract_2d_from_3d

Description: Keyword to extract a 2D mesh by selecting a boundary of the 3D mesh. To generate a 2D axisymmetric mesh prefer Extract_2Daxi_from_3D keyword.

See also: [interpret \(3\)](#) [extract_2daxi_from_3d \(3.63\)](#)

Usage:

```
extract_2d_from_3d dom3D bord dom2D  
where
```

- **dom3D** *str*: Domain name of the 3D mesh
- **bord** *str*: Boundary name. This boundary becomes the new 2D mesh and all the boundaries, in 3D, attached to the selected boundary, give their name to the new boundaries, in 2D.
- **dom2D** *str*: Domain name of the new 2D mesh

3.63 Extract_2daxi_from_3d

Description: Keyword to extract a 2D axisymmetric mesh by selecting a boundary of the 3D mesh.

See also: [extract_2d_from_3d \(3.62\)](#)

Usage:

extract_2daxi_from_3d dom3D bord dom2D

where

- **dom3D** *str*: Domain name of the 3D mesh
- **bord** *str*: Boundary name. This boundary becomes the new 2D mesh and all the boundaries, in 3D, attached to the selected boundary, give their name to the new boundaries, in 2D.
- **dom2D** *str*: Domain name of the new 2D mesh

3.64 Extraire_domaine

Description: Keyword to create a new domain built with the domain elements of the pb_name problem verifying the two conditions given by Condition_elements. The problem pb_name should have been discretized.

Keyword Discretize should have already been used to read the object.

See also: [interprete \(3\)](#)

Usage:

extraire_domaine {

domaine *str*

probleme *str*

 [**condition_elements** *str*]

 [**sous_zonelsous_domaine** *str*]

}

where

- **domaine** *str*: Domain in which faces are saved
- **probleme** *str*: Problem from which faces should be extracted
- **condition_elements** *str*
- **sous_zonelsous_domaine** *str*

3.65 Extraire_plan

Description: This keyword extracts a plane mesh named domain_name (this domain should have been declared before) from the mesh of the pb_name problem. The plane can be either a triangle (defined by the keywords Origine, Point1, Point2 and Triangle), either a regular quadrangle (with keywords Origine, Point1 and Point2), or either a generalized quadrangle (with keywords Origine, Point1, Point2, Point3). The keyword Epaisseur specifies the thickness of volume around the plane which contains the faces of the extracted mesh. The keyword via_extraire_surface will create a plan and use Extraire_surface algorithm. Inverse_condition_element keyword then will be used in the case where the plane is a boundary not well oriented, and avec_certain_bords_pour_extraire_surface is the option related to the Extraire_surface option named avec_certain_bords.

Keyword Discretize should have already been used to read the object.

See also: [interprete \(3\)](#)

Usage:

extraire_plan {

domaine *str*

```

probleme str
origine n x1 x2 ... xn
point1 n x1 x2 ... xn
point2 n x1 x2 ... xn
[ point3 n x1 x2 ... xn ]
[ triangle ]
epaisseur float
[ via_extraire_surface ]
[ inverse_condition_element ]
[ avec_certains_bords_pour_extraire_surface n word1 word2 ... wordn ]

}
where

```

- **domaine** *str*: domain name
- **probleme** *str*: pb_name
- **origine** *n x1 x2 ... xn*
- **point1** *n x1 x2 ... xn*
- **point2** *n x1 x2 ... xn*
- **point3** *n x1 x2 ... xn*
- **triangle**
- **epaisseur** *float*: thickness
- **via_extraire_surface**
- **inverse_condition_element**
- **avec_certains_bords_pour_extraire_surface** *n word1 word2 ... wordn*: name of boundaries to include when extracting plan

3.66 Extraire_surface

Description: This keyword extracts a surface mesh named domain_name (this domain should have been declared before) from the mesh of the pb_name problem. The surface mesh is defined by one or two conditions. The first condition is about elements with Condition_elements $x*y+z<1$

Will define a surface mesh with external faces of the mesh elements inside the sphere of radius 1 located at (0,0,0). The second condition Condition_faces is useful to give a restriction.

By default, the faces from the boundaries are not added to the surface mesh excepted if option avec_les_bords is given (all the boundaries are added), or if the option avec_certains_bords is used to add only some boundaries.

Keyword Discretize should have already been used to read the object.

See also: [interpret \(3\)](#)

Usage:

```

extraire_surface {
    domaine str
    probleme str
    [ condition_elements str ]
    [ condition_faces str ]
    [ avec_les_bords ]
    [ avec_certains_bords n word1 word2 ... wordn ]
}
where

```

- **domaine** *str*: Domain in which faces are saved
- **probleme** *str*: Problem from which faces should be extracted
- **condition_elements** *str*: condition on center of elements
- **condition_faces** *str*
- **avec_les_bords**
- **avec_certains_bords** *n word1 word2 ... wordn*

3.67 Extrudebord

Description: Class to generate an extruded mesh from a boundary of a tetrahedral or an hexahedral mesh.
Warning: If the initial domain is a tetrahedral mesh, the boundary will be moved in the XY plane then extrusion will be applied (you should maybe use the Transformer keyword on the final domain to have the domain you really want). You can use the keyword Postraiter_domaine to generate a latalmedl... file to visualize your initial and final meshes.

This keyword can be used for example to create a periodic box extracted from a boundary of a tetrahedral or a hexahedral mesh. This periodic box may be used then to engender turbulent inlet flow condition for the main domain.

Note that ExtrudeBord in VEF generates 3 or 14 tetrahedra from extruded prisms.

See also: [interprete \(3\)](#)

Usage:

```
extrudebord {
    domaine_init str
    direction x1 x2 (x3)
    nb_tranches int
    domaine_final str
    nom_bord str
    [ hexa_old ]
    [ trois_tetra ]
    [ vingt_tetra ]
    [ sans_passer_par_le2d int]
}
where
```

- **domaine_init** *str*: Initial domain with hexaedras or tetrahedras.
- **direction** *x1 x2 (x3)*: Directions for the extrusion.
- **nb_tranches** *int*: Number of elements in the extrusion direction.
- **domaine_final** *str*: Extruded domain.
- **nom_bord** *str*: Name of the boundary of the initial domain where extrusion will be applied.
- **hexa_old** : Old algorithm for boundary extrusion from a hexahedral mesh.
- **trois_tetra** : To extrude in 3 tetrahedras instead of 14 tetrahedras.
- **vingt_tetra** : To extrude in 20 tetrahedras instead of 14 tetrahedras.
- **sans_passer_par_le2d** *int*: Only for non-regression

3.68 Extrudeparoi

Description: Keyword dedicated in 3D (VEF) to create prismatic layer at wall. Each prism is cut into 3 tetraedra.

See also: [interprete \(3\)](#)

Usage:

```
extrudeparoi {  
    domaine str  
    nom_bord str  
    [ epaisseur n x1 x2 ... xn ]  
    [ critere_absolu ]  
    [ projection_normale_bord ]  
}
```

where

- **domaine** *str*: Name of the domain.
- **nom_bord** *str*: Name of the (no-slip) boundary for creation of prismatic layers.
- **epaisseur** *n x1 x2 ... xn*: n r_1 r_2 r_n : (relative or absolute) width for each layer.
- **critere_absolu** : use absolute width for each layer instead of relative.
- **projection_normale_bord** : keyword to project layers on the same plane that contiguous boundaries. default values are : epaisseur_relative 1 0.5 projection_normale_bord 1

3.69 Extruder

Description: Class to create a 3D tetrahedral/hexahedral mesh (a prism is cut in 14) from a 2D triangular/quadrangular mesh.

See also: interpret (3) extruder_en3 (3.72)

Usage:

```
extruder {  
    domaine str  
    nb_tranches int  
    direction troisf  
}
```

where

- **domaine** *str*: Name of the domain.
- **nb_tranches** *int*: Number of elements in the extrusion direction.
- **direction** *troisf* (3.70): Direction of the extrude operation.

3.70 Troisf

Description: Auxiliary class to extrude.

See also: objet_lecture (47)

Usage:

```
lx ly lz  
where
```

- **lx** *float*: X direction of the extrude operation.
- **ly** *float*: Y direction of the extrude operation.
- **lz** *float*: Z direction of the extrude operation.

3.71 Extruder_en20

Description: It does the same task as Extruder except that a prism is cut into 20 tetraedra instead of 3. The name of the boundaries will be devant (front) and derriere (back). But you can change these names with the keyword RegroupeBord.

See also: [interpret](#) (3)

Usage:

```
extruder_en20 {  
    domaine str  
    nb_tranches int  
    [ direction troisf]  
}  
where
```

- **domaine** *str*: Name of the domain.
- **nb_tranches** *int*: Number of elements in the extrusion direction.
- **direction** *troisf* (3.70): 0 Direction of the extrude operation.

3.72 Extruder_en3

Description: Class to create a 3D tetrahedral/hexahedral mesh (a prism is cut in 3) from a 2D triangular/quadrangular mesh. The names of the boundaries (by default, devant (front) and derriere (back)) may be edited by the keyword **nom_cl_devant** and **nom_cl_derriere**. If 'null' is written for **nom_cl**, then no boundary condition is generated at this place.

Recommendation : to ensure conformity between meshes (in case of fluid/solid coupling) it is recommended to extrude all the domains at the same time.

See also: [extruder](#) (3.69)

Usage:

```
extruder_en3 {  
    domaine n word1 word2 ... wordn  
    [ nom_cl_devant str]  
    [ nom_cl_derriere str]  
    nb_tranches int  
    direction troisf  
}  
where
```

- **domaine** *n word1 word2 ... wordn*: List of the domains
- **nom_cl_devant** *str*: New name of the first boundary.
- **nom_cl_derriere** *str*: New name of the second boundary.
- **nb_tranches** *int* for inheritance: Number of elements in the extrusion direction.
- **direction** *troisf* (3.70) for inheritance: Direction of the extrude operation.

3.73 Facsec_expert

Description: To parameter the safety factor for the time step during the simulation.

See also: [interpret \(3\)](#)

Usage:

```
facsec_expert {  
    [ facsec_ini float]  
    [ facsec_max float]  
    [ rapport_residus float]  
    [ nb_ite_sans_accel_max int]  
}
```

where

- **facsec_ini** *float*: Initial facsec taken into account at the beginning of the simulation.
- **facsec_max** *float*: Maximum ratio allowed between time step and stability time returned by CFL condition. The initial ratio given by facsec keyword is changed during the calculation with the implicit scheme but it couldn't be higher than facsec_max value.

Warning: Some implicit schemes do not permit high facsec_max, example Schema_Adams_Moulton_order_3 needs facsec=facsec_max=1.

Advice:

The calculation may start with a facsec specified by the user and increased by the algorithm up to the facsec_max limit. But the user can also choose to specify a constant facsec (facsec_max will be set to facsec value then). Faster convergence has been seen and depends on the kind of calculation:

- Hydraulic only or thermal hydraulic with forced convection and low coupling between velocity and temperature (Boussinesq value beta low), facsec between 20-30
- Thermal hydraulic with forced convection and strong coupling between velocity and temperature (Boussinesq value beta high), facsec between 90-100
- Thermohydraulic with natural convection, facsec around 300
- Conduction only, facsec can be set to a very high value (1e8) as if the scheme was unconditionally stable

These values can also be used as rule of thumb for initial facsec with a facsec_max limit higher.

- **rapport_residus** *float*: Ratio between the residual at time n and the residual at time n+1 above which the facsec is increased by multiplying by sqrt(rapport_residus) (1.2 by default).
- **nb_ite_sans_accel_max** *int*: Maximum number of iterations without facsec increases (20000 by default): if facsec does not increase with the previous condition (ration between 2 consecutive residuals too high), we increase it by force after nb_ite_sans_accel_max iterations.

3.74 End

Synonymous: **fin**

Description: Keyword which must complete the data file. The execution of the data file stops when reaching this keyword.

See also: [interpret \(3\)](#)

Usage:

end

3.75 }

Description: Block's end.

See also: [interpret \(3\)](#)

Usage:
}

3.76 Imposer_vit_bords_ale

Description: For the Arbitrary Lagrangian-Eulerian framework: block to indicate the number of mobile boundaries of the domain and specify the speed that must be imposed on them.

See also: [interpret \(3\)](#)

Usage:

imposer_vit_bords_ale **dom** **bloc**
where

- **dom** *str*: Name of domain.
- **bloc** *bloc_lecture* (3.2): between the braces, you must specify the numbers of the mobile borders of the domain then list these mobile borders and indicate the speed which must be imposed on them
Example: Imposer_vit_bords_ALE dom_name { 1 boundary_name Champ_front_ALE 2 -(y-0.1)*0.01 (x-0.1)*0.01 }

3.77 Imprimer_flux

Description: This keyword prints the flux per face at the specified domain boundaries in the data set. The fluxes are written to the .face files at a frequency defined by dt_impr, the evaluation printing frequency (refer to time scheme keywords). By default, fluxes are incorporated onto the edges before being displayed.

See also: [interpret \(3\)](#) [imprimer_flux_sum \(3.78\)](#)

Usage:

imprimer_flux **domain_name** **noms_bord**
where

- **domain_name** *str*: Name of the domain.
- **noms_bord** *bloc_lecture* (3.2): List of boundaries, for ex: { Bord1 Bord2 }

3.78 Imprimer_flux_sum

Description: This keyword prints the sum of the flux per face at the domain boundaries defined by the user in the data set. The fluxes are written into the .out files at a frequency defined by dt_impr, the evaluation printing frequency (refer to time scheme keywords).

See also: [imprimer_flux \(3.77\)](#)

Usage:

imprimer_flux_sum **domain_name** **noms_bord**
where

- **domain_name** *str*: Name of the domain.
- **noms_bord** *bloc_lecture* (3.2): List of boundaries, for ex: { Bord1 Bord2 }

3.79 Integrer_champ_med

Description: this keyword is used to calculate a flow rate from a velocity MED field read before. The method is either `debit_total` to calculate the flow rate on the whole surface, either `integrale_en_z` to calculate flow rates between $z=z_{min}$ and $z=z_{max}$ on `nb_tranche` surfaces. The output file indicates first the flow rate for the whole surface and then lists for each tranche : the height z , the surface average value, the surface area and the flow rate. For the `debit_total` method, only one tranche is considered.
file : $z \text{ Sum}(u.dS)/\text{Sum}(dS) \text{ Sum}(dS) \text{ Sum}(u.dS)$

See also: [interpret](#) (3)

Usage:

```
integrer_champ_med {  
    champ_med str  
    methode str into ['integrale_en_z', 'debit_total']  
    [ zmin float]  
    [ zmax float]  
    [ nb_tranche int]  
    [ fichier_sortie str]  
}
```

where

- **champ_med** *str*
- **methode** *str* into ['integrale_en_z', 'debit_total']: to choose between the integral following z or over the entire height (`debit_total` corresponds to $z_{min}=-D_{MAXFLOAT}$, $z_{max}=D_{MAXFLOAT}$, `nb_tranche=1`)
- **zmin** *float*
- **zmax** *float*
- **nb_tranche** *int*
- **fichier_sortie** *str*: name of the output file, by default: `integrale`.

3.80 Interfaces

Description: `not_set`

See also: [interpret](#) (3)

Usage:

```
interfaces {  
    fichier_reprise_interface str  
    [ timestep_reprise_interface int]  
    [ lata_meshname str]  
    [ remaillage_ft_ijk remaillage_ft_ijk]  
    [ use_tryggvason_interfacial_source remaillage_ft_ijk]  
    [ no_octree_method int]  
    [ compute_distance_autres_interfaces ]  
    [ terme_gravite str into ['rho_g', 'grad_i']]  
}
```

where

- **fichier_reprise_interface** *str*
- **timestep_reprise_interface** *int*

- **lata_meshname** *str*
- **remaillage_ft_ijk** *remaillage_ft_ijk* (3.120)
- **use_tryggvason_interfacial_source** *remaillage_ft_ijk* (3.120)
- **no_octree_method** *int*: if the bubbles repel each other, what method should be used to compute relative velocities? Octree method by default, otherwise we used the IJK discretization
- **compute_distance_autres_interfaces**
- **terme_gravite** *str* into [*'rho_g'*, *'grad_i'*]

3.81 Interpret_geometrique_base

Description: Class for interpreting a data file

See also: [interpret](#) (3) [Create_domain_from_sub_domain](#) (3.5)

Usage:

interpret_geometrique_base

3.82 Lata_to_cgns

Description: To convert results file written with LATA format to CGNS file. Warning: Fields located on faces are not supported yet.

See also: [interpret](#) (3)

Usage:

lata_to_CGNS [**format**] **file** **file_CGNS**

where

- **format** *format_lata_to_cgns* (3.83): generated file post_CGNS.data use format (CGNS or LATA or LML keyword).
- **file** *str*: LATA file to convert to the new format.
- **file_CGNS** *str*: Name of the CGNS file.

3.83 Format_lata_to_cgns

Description: not_set

See also: [objet_lecture](#) (47)

Usage:

mot [**format**]

where

- **mot** *str* into [*'format_post_sup'*]
- **format** *str* into [*'lml'*, *'lata'*, *'lata_v2'*, *'med'*, *'cgns'*]: generated file post_CGNS.data use format (CGNS or LATA or LML keyword).

3.84 Lata_2_med

Synonymous: **lata_to_med**

Description: To convert results file written with LATA format to MED file. Warning: Fields located on faces are not supported yet.

See also: [interpret \(3\)](#)

Usage:

lata_2_med [**format**] **file** **file_med**

where

- **format** *format_lata_to_med* (3.85): generated file post_med.data use format (MED or LATA or LML keyword).
- **file** *str*: LATA file to convert to the new format.
- **file_med** *str*: Name of the MED file.

3.85 Format_lata_to_med

Description: not_set

See also: [objet_lecture \(47\)](#)

Usage:

mot [**format**]

where

- **mot** *str* into ['format_post_sup']
- **format** *str* into ['lml', 'lata', 'lata_v2', 'med']: generated file post_med.data use format (MED or LATA or LML keyword).

3.86 Lata_2_other

Synonymous: **lata_to_other**

Description: To convert results file written with LATA format to CGNS, MED or LML format. Warning: Fields located at faces are not supported yet.

See also: [interpret \(3\)](#)

Usage:

lata_2_other [**format**] **file** **file_post**

where

- **format** *str* into ['lml', 'lata', 'lata_v2', 'med', 'cgns']: Results format (CGNS, MED or LATA or LML keyword).
- **file** *str*: LATA file to convert to the new format.
- **file_post** *str*: Name of file post.

3.87 Lire_ideas

Description: Read a geom in a unv file. 3D tetra mesh elements only may be read by TRUST.

See also: [interpret \(3\)](#)

Usage:

lire_ideas **nom_dom** **file**

where

- **nom_dom** *str*: Name of domain.
- **file** *str*: Name of file.

3.88 Lml_2_lata

Synonymous: **lml_to_lata**

Description: To convert results file written with LML format to a single LATA file.

See also: [interpret](#) (3)

Usage:

lml_2_lata file_lml file_lata

where

- **file_lml** *str*: LML file to convert to the new format.
- **file_lata** *str*: Name of the single LATA file.

3.89 Mailler

Description: The Mailler (Mesh) interpreter allows a Domain type object domaine to be meshed with objects objet_1, objet_2, etc...

See also: [interpret](#) (3)

Usage:

mailler domaine bloc

where

- **domaine** *str*: Name of domain.
- **bloc** *list_bloc_mailler* (3.90): Instructions to mesh.

3.90 List_bloc_mailler

Description: List of block mesh.

See also: [listobj](#) (46.5)

Usage:

{ object1 , object2 }

list of *mailler_base* (3.90.1) separated with ,

3.90.1 Mailler_base

Description: Basic class to mesh.

See also: [objet_lecture](#) (47) [pave](#) (3.90.2) [epsilon](#) (3.90.12) [domain](#) (3.90.13)

Usage:

3.90.2 Pave

Description: Class to create a pave (block) with boundaries.

See also: `mailler_base` (3.90.1)

Usage:

pave name bloc list_bord

where

- **name** *str*: Name of the pave (block).
- **bloc** *bloc_pave* (3.90.3): Definition of the pave (block).
- **list_bord** *list_bord* (3.90.4): Domain boundaries definition.

3.90.3 Bloc_pave

Description: Class to create a pave.

See also: `objet_lecture` (47)

Usage:

```
{  
    [ Origine x1 x2 (x3)]  
    [ longueurs x1 x2 (x3)]  
    [ nombre_de_noeuds n1 n2 (n3)]  
    [ facteurs x1 x2 (x3)]  
    [ symx ]  
    [ symy ]  
    [ symz ]  
    [ xtanh float]  
    [ xtanh_dilatation int into [-1, 0, 1]]  
    [ xtanh_taille_premiere_maille float]  
    [ ytanh float]  
    [ ytanh_dilatation int into [-1, 0, 1]]  
    [ ytanh_taille_premiere_maille float]  
    [ ztanh float]  
    [ ztanh_dilatation int into [-1, 0, 1]]  
    [ ztanh_taille_premiere_maille float]  
}
```

where

- **Origine** *x1 x2 (x3)*: Keyword to define the pave (block) origin, that is to say one of the 8 block points (or 4 in a 2D coordinate system).
- **longueurs** *x1 x2 (x3)*: Keyword to define the block dimensions, that is to say knowing the origin, length along the axes.
- **nombre_de_noeuds** *n1 n2 (n3)*: Keyword to define the discretization (nodenumber) in each direction.
- **facteurs** *x1 x2 (x3)*: Keyword to define stretching factors for mesh discretization in each direction. This is a real number which must be positive (by default 1.0). A stretching factor other than 1 allows refinement on one edge in one direction.
- **symx**: Keyword to define a block mesh that is symmetrical with respect to the YZ plane (respectively Y-axis in 2D) passing through the block centre.
- **symy**: Keyword to define a block mesh that is symmetrical with respect to the XZ plane (respectively X-axis in 2D) passing through the block centre.

- **symz** : Keyword defining a block mesh that is symmetrical with respect to the XY plane passing through the block centre.
- **xtanh** *float*: Keyword to generate mesh with tanh (hyperbolic tangent) variation in the X-direction.
- **xtanh_dilatation** *int into [-1, 0, 1]*: Keyword to generate mesh with tanh (hyperbolic tangent) variation in the X-direction. **xtanh_dilatation**: The value may be -1,0,1 (0 by default): 0: coarse mesh at the middle of the channel and smaller near the walls -1: coarse mesh at the left side of the channel and smaller at the right side 1: coarse mesh at the right side of the channel and smaller near the left side of the channel.
- **xtanh_taille_premiere_maille** *float*: Size of the first cell of the mesh with tanh (hyperbolic tangent) variation in the X-direction.
- **ytanh** *float*: Keyword to generate mesh with tanh (hyperbolic tangent) variation in the Y-direction.
- **ytanh_dilatation** *int into [-1, 0, 1]*: Keyword to generate mesh with tanh (hyperbolic tangent) variation in the Y-direction. **ytanh_dilatation**: The value may be -1,0,1 (0 by default): 0: coarse mesh at the middle of the channel and smaller near the walls -1: coarse mesh at the bottom of the channel and smaller near the top 1: coarse mesh at the top of the channel and smaller near the bottom.
- **ytanh_taille_premiere_maille** *float*: Size of the first cell of the mesh with tanh (hyperbolic tangent) variation in the Y-direction.
- **ztanh** *float*: Keyword to generate mesh with tanh (hyperbolic tangent) variation in the Z-direction.
- **ztanh_dilatation** *int into [-1, 0, 1]*: Keyword to generate mesh with tanh (hyperbolic tangent) variation in the Z-direction. **ztanh_dilatation**: The value may be -1,0,1 (0 by default): 0: coarse mesh at the middle of the channel and smaller near the walls -1: coarse mesh at the back of the channel and smaller near the front 1: coarse mesh at the front of the channel and smaller near the back.
- **ztanh_taille_premiere_maille** *float*: Size of the first cell of the mesh with tanh (hyperbolic tangent) variation in the Z-direction.

3.90.4 List_bord

Description: The block sides.

See also: listobj ([46.5](#))

Usage:

{ object1 object2 }

list of *bord_base* ([3.90.5](#))

3.90.5 Bord_base

Description: Basic class for block sides. Block sides that are neither edges nor connectors are not specified. The duplicate nodes of two blocks in contact are automatically recognized and deleted.

See also: objet_lecture ([47](#)) bord ([3.90.6](#)) raccord ([3.90.10](#)) internes ([3.90.11](#))

Usage:

3.90.6 Bord

Description: The block side is not in contact with another block and boundary conditions are applied to it.

See also: bord_base ([3.90.5](#))

Usage:

bord nom defbord

where

- **nom** *str*: Name of block side.
- **defbord** *defbord* (3.90.7): Definition of block side.

3.90.7 Defbord

Description: Class to define an edge.

See also: [objet_lecture \(47\)](#) [defbord_2 \(3.90.8\)](#) [defbord_3 \(3.90.9\)](#)

Usage:

3.90.8 Defbord_2

Description: 1-D edge (straight line) in the 2-D space.

See also: (3.90.7)

Usage:

dir eq pos pos2_min inf1 dir2 inf2 pos2_max
where

- **dir** *str into* ['X', 'Y']: Edge is perpendicular to this direction.
- **eq** *str into* ['=']: Equality sign.
- **pos** *float*: Position value.
- **pos2_min** *float*: Minimal value.
- **inf1** *str into* ['<=']: Less than or equal to sign.
- **dir2** *str into* ['X', 'Y']: Edge is parallel to this direction.
- **inf2** *str into* ['<=']: Less than or equal to sign.
- **pos2_max** *float*: Maximal value.

3.90.9 Defbord_3

Description: 2-D edge (plane) in the 3-D space.

See also: (3.90.7)

Usage:

dir eq pos pos2_min inf1 dir2 inf2 pos2_max pos3_min inf3 dir3 inf4 pos3_max
where

- **dir** *str into* ['X', 'Y', 'Z']: Edge is perpendicular to this direction.
- **eq** *str into* ['=']: Equality sign.
- **pos** *float*: Position value.
- **pos2_min** *float*: Minimal value.
- **inf1** *str into* ['<=']: Less than or equal to sign.
- **dir2** *str into* ['X', 'Y']: Edge is parallel to this direction.
- **inf2** *str into* ['<=']: Less than or equal to sign.
- **pos2_max** *float*: Maximal value.
- **pos3_min** *float*: Minimal value.
- **inf3** *str into* ['<=']: Less than or equal to sign.
- **dir3** *str into* ['Y', 'Z']: Edge is parallel to this direction.
- **inf4** *str into* ['<=']: Less than or equal to sign.
- **pos3_max** *float*: Maximal value.

3.90.10 Raccord

Description: The block side is in contact with the block of another domain (case of two coupled problems).

See also: `bord_base` ([3.90.5](#))

Usage:

raccord **type1** **type2** **nom** **defbord**

where

- **type1** *str* into ['local', 'distant']: Contact type.
- **type2** *str* into ['homogene']: Contact type.
- **nom** *str*: Name of block side.
- **defbord** *defbord* ([3.90.7](#)): Definition of block side.

3.90.11 Internes

Description: To indicate that the block has a set of internal faces (these faces will be duplicated automatically by the program and will be processed in a manner similar to edge faces).

Two boundaries with the same boundary conditions may have the same name (whether or not they belong to the same block).

The keyword Internes (Internal) must be used to execute a calculation with plates, followed by the equation of the surface area covered by the plates.

See also: `bord_base` ([3.90.5](#))

Usage:

internes **nom** **defbord**

where

- **nom** *str*: Name of block side.
- **defbord** *defbord* ([3.90.7](#)): Definition of block side.

3.90.12 Epsilon

Description: Two points will be confused if the distance between them is less than `eps`. By default, `eps` is set to $1e-12$. The keyword Epsilon allows an alternative value to be assigned to `eps`.

See also: `mailler_base` ([3.90.1](#))

Usage:

epsilon **eps**

where

- **eps** *float*: New value of precision.

3.90.13 Domain

Description: Class to reuse a domain.

See also: `mailler_base` ([3.90.1](#))

Usage:

domain **domain_name**

where

- **domain_name** *str*: Name of domain.

3.91 Maillerparallel

Description: creates a parallel distributed hexaedral mesh of a parallelepipedic box. It is equivalent to creating a mesh with a single Pave, splitting it with Decouper and reloading it in parallel with Scatter. It only works in 3D at this time. It can also be used for a sequential computation (with all NPARTS=1)}

See also: [interpret \(3\)](#)

Usage:

```
maillerparallel {
    domain str
    nb_nodes n n1 n2 ... nn
    splitting n n1 n2 ... nn
    ghost_thickness int
    [ perio_x ]
    [ perio_y ]
    [ perio_z ]
    [ function_coord_x str ]
    [ function_coord_y str ]
    [ function_coord_z str ]
    [ file_coord_x str ]
    [ file_coord_y str ]
    [ file_coord_z str ]
    [ boundary_xmin str ]
    [ boundary_xmax str ]
    [ boundary_ymin str ]
    [ boundary_ymax str ]
    [ boundary_zmin str ]
    [ boundary_zmax str ]
}
```

where

- **domain** *str*: the name of the domain to mesh (it must be an empty domain object).
- **nb_nodes** *n n1 n2 ... nn*: dimension defines the spatial dimension (currently only dimension=3 is supported), and nX, nY and nZ defines the total number of nodes in the mesh in each direction.
- **splitting** *n n1 n2 ... nn*: dimension is the spatial dimension and npartsX, npartsY and npartsZ are the number of parts created. The product of the number of parts must be equal to the number of processors used for the computation.
- **ghost_thickness** *int*: the number of ghost cells (equivalent to the `epaisseur_joint` parameter of Decouper).
- **perio_x** : change the splitting method to provide a valid mesh for periodic boundary conditions.
- **perio_y** : change the splitting method to provide a valid mesh for periodic boundary conditions.
- **perio_z** : change the splitting method to provide a valid mesh for periodic boundary conditions.
- **function_coord_x** *str*: By default, the meshing algorithm creates nX nY nZ coordinates ranging between 0 and 1 (eg a unity size box). If `function_coord_x` is specified, it is used to transform the [0,1] segment to the coordinates of the nodes. `funcX` must be a function of the x variable only.
- **function_coord_y** *str*: like `function_coord_x` for y
- **function_coord_z** *str*: like `function_coord_x` for z
- **file_coord_x** *str*: Keyword to read the Nx floating point values used as nodes coordinates in the file.

- **file_coord_y** *str*: idem file_coord_x for y
- **file_coord_z** *str*: idem file_coord_x for z
- **boundary_xmin** *str*: the name of the boundary at the minimum X direction. If it not provided, the default boundary names are xmin, xmax, ymin, ymax, zmin and zmax. If the mesh is periodic in a given direction, only the MIN boundary name is used, for both sides of the box.
- **boundary_xmax** *str*
- **boundary_ymin** *str*
- **boundary_ymax** *str*
- **boundary_zmin** *str*
- **boundary_zmax** *str*

3.92 Mass_source

Description: Mass source used in a dilatable simulation to add/reduce a mass at the boundary (volumetric source in the first cell of a given boundary).

See also: [interpret \(3\)](#)

Usage:

```
mass_source {
    bord str
    surfacic_flux champ_front_base
}
```

where

- **bord** *str*: Name of the boundary where the source term is applied
- **surfacic_flux** *champ_front_base* (21.1): The boundary field that the user likes to apply: for example, champ_front_uniforme, ch_front_input_uniform or champ_front_fonc_t

3.93 Mkdir

Description: equivalent to system mkdir

See also: [interpret \(3\)](#)

Usage:

```
mkdir directory
```

where

- **directory** *str*: directory to create

3.94 Modif_bord_to_raccord

Description: Keyword to convert a boundary of domain_name domain of kind Bord to a boundary of kind Raccord (named boundary_name). It is useful when using meshes with boundaries of kind Bord defined and to run a coupled calculation.

See also: [interpret \(3\)](#)

Usage:

```
modif_bord_to_raccord domaine nom_bord
```

where

- **domaine** *str*: Name of domain
- **nom_bord** *str*: Name of the boundary to transform.

3.95 Modifydomaineaxi1d

Description: Convert a 1D mesh to 1D axisymmetric mesh

See also: [interpret \(3\)](#)

Usage:

modifydomaineAx1d **dom** **bloc**

where

- **dom** *str*
- **bloc** *bloc_lecture* ([3.2](#))

3.96 Moyenne_volumique

Description: This keyword should be used after Resoudre keyword. It computes the convolution product of one or more fields with a given filtering function.

See also: [interpret \(3\)](#)

Usage:

```
moyenne_volumique {
    nom_pb str
    nom_domaine str
    noms_champs n word1 word2 ... wordn
    [ format_post str ]
    [ nom_fichier_post str ]
    fonction_filtre bloc_lecture
    [ localisation str into ['elem', 'som']]
}
```

where

- **nom_pb** *str*: name of the problem where the source fields will be searched.
 - **nom_domaine** *str*: name of the destination domain (for example, it can be a coarser mesh, but for optimal performance in parallel, the domain should be split with the same algorithm as the computation mesh, eg, same tranche parameters for example)
 - **noms_champs** *n word1 word2 ... wordn*: name of the source fields (these fields must be accessible from the postraitement) N source_field1 source_field2 ... source_fieldN
 - **format_post** *str*: gives the fileformat for the result (by default : lata)
 - **nom_fichier_post** *str*: indicates the filename where the result is written
 - **fonction_filtre** *bloc_lecture* ([3.2](#)): to specify the given filter
- ```
Fonction_filtre {
 type filter_type
 demie-largeur l
 [omega w]
 [expression string]
}
```

type filter\_type : This parameter specifies the filtering function. Valid filter\_type are:  
 Boite is a box filter,  $f(x, y, z) = (abs(x) < l) * (abs(y) < l) * (abs(z) < l) / (8l^3)$   
 Chapeau is a hat filter (product of hat filters in each direction) centered on the origin, the half-width of the filter being l and its integral being 1.  
 Quadra is a 2nd order filter.  
 Gaussienne is a normalized gaussian filter of standard deviation sigma in each direction (all field elements outside a cubic box defined by clipping\_half\_width are ignored, hence, taking clipping\_half\_width=2.5\*sigma yields an integral of 0.99 for a uniform unity field).  
 Parser allows a user defined function of the x,y,z variables. All elements outside a cubic box defined by clipping\_half\_width are ignored. The parser is much slower than the equivalent c++ coded function...

demie-largeur l : This parameter specifies the half width of the filter

[ omega w ] : This parameter must be given for the gaussienne filter. It defines the standard deviation of the gaussian filter.

[ expression string ] : This parameter must be given for the parser filter type. This expression will be interpreted by the math parser with the predefined variables x, y and z.

- **localisation** str into ['elem', 'som']: indicates where the convolution product should be computed: either on the elements or on the nodes of the destination domain.

### 3.97 Multigrid\_solver

Description: Object defining a multigrid solver in IJK discretization

See also: interpret (3)

Usage:

```
multigrid_solver {
 [coarsen_operators coarsen_operators]
 [ghost_size int]
 [relax_jacobi n x1 x2 ... xn]
 [pre_smooth_steps n n1 n2 ... nn]
 [smooth_steps n n1 n2 ... nn]
 [nb_full_mg_steps n n1 n2 ... nn]
 [solveur_grossier solveur_sys_base]
 [seuil float]
 [impr]
 [solver_precision str into ['mixed', 'double']]
 [iterations_mixed_solver int]
}
```

where

- **coarsen\_operators** coarsen\_operators (3.98): Definition of the number of grids that will be used, in addition to the finest (original) grid, followed by the list of the coarsen operators that will be applied to get those grids
- **ghost\_size** int: Number of ghost cells known by each processor in each of the three directions
- **relax\_jacobi** n x1 x2 ... xn: Parameter between 0 and 1 that will be used in the Jacobi method to solve equation on each grid. Should be around 0.7
- **pre\_smooth\_steps** n n1 n2 ... nn: First integer of the list indicates the numbers of integers that has to be read next. Following integers define the numbers of iterations done before solving the equation on each grid. For example, 2 7 8 means that we have a list of 2 integers, the first one tells us to perform 7 pre-smooth steps on the first grid, the second one tells us to perform 8 pre-smooth steps

on the second grid. If there are more than 2 grids in the solver, then the remaining ones will have as many pre-smooth steps as the last mentioned number (here, 8)

- **smooth\_steps** *n n1 n2 ... nn*: First integer of the list indicates the numbers of integers that has to be read next. Following integers define the numbers of iterations done after solving the equation on each grid. Same behavior as `pre_smooth_steps`
- **nb\_full\_mg\_steps** *n n1 n2 ... nn*: Number of multigrid iterations at each level
- **solveur\_grossier** *solveur\_sys\_base* (14.19): Name of the iterative solver that will be used to solve the system on the coarsest grid. This resolution must be more precise than the ones occurring on the fine grids. The threshold of this solver must therefore be lower than `seuil` defined above.
- **seuil** *float*: Define an upper bound on the norm of the final residue (i.e. the one obtained after applying the multigrid solver). With hybrid precision, as long as we have not obtained a residue whose norm is lower than the imposed threshold, we keep applying the solver
- **impr** : Flag to display some info on the resolution on each grid
- **solver\_precision** *str into ['mixed', 'double']*: Precision with which the variables at stake during the resolution of the system will be stored. We can have a simple or floatant precision or both. In the case of a hybrid precision, the multigrid solver is launched in simple precision, but the residual is calculated in floatant precision.
- **iterations\_mixed\_solver** *int*: Define the maximum number of iterations in mixed precision solver

### 3.98 Coarsen\_operators

Description: `not_set`

See also: `listobj` (46.5)

Usage:

`n object1 object2 ....`

list of `coarsen_operator_uniform` (3.98.1)

#### 3.98.1 Coarsen\_operator\_uniform

Description: Object defining the uniform coarsening process of the given grid in IJK discretization

See also: `objet_lecture` (47)

Usage:

`[ Coarsen_Operator_Uniform ] aco [ coarsen_i ] [ coarsen_i_val ] [ coarsen_j ] [ coarsen_j_val ] [ coarsen_k ] [ coarsen_k_val ] acof`

where

- **Coarsen\_Operator\_Uniform** *str*
- **aco** *str into ['{']*: opening curly brace
- **coarsen\_i** *str into ['coarsen\_i']*
- **coarsen\_i\_val** *int*: Integer indicating the number by which we will divide the number of elements in the I direction (in order to obtain a coarser grid)
- **coarsen\_j** *str into ['coarsen\_j']*
- **coarsen\_j\_val** *int*: Integer indicating the number by which we will divide the number of elements in the J direction (in order to obtain a coarser grid)
- **coarsen\_k** *str into ['coarsen\_k']*
- **coarsen\_k\_val** *int*: Integer indicating the number by which we will divide the number of elements in the K direction (in order to obtain a coarser grid)
- **acof** *str into [']*: closing curly brace

### 3.99 Nettoiepasnoeuds

Description: Keyword NettoiePasNoeuds does not delete useless nodes (nodes without elements) from a domain.

See also: [interpret \(3\)](#)

Usage:

**nettoiepasnoeuds** **domain\_name**

where

- **domain\_name** *str*: Name of domain.

### 3.100 Option\_vdf

Description: Class of VDF options.

See also: [interpret \(3\)](#)

Usage:

**option\_vdf** {

```
[traitement_coins str into ['oui', 'non']]
[traitement_gradients str into ['oui', 'non']]
[p_imposee_aux_faces str into ['oui', 'non']]
[deactivate_arete_mixte]
[toutes_les_options | all_options]
```

}

where

- **traitement\_coins** *str* into ['oui', 'non']: Treatment of corners (yes or no). This option modifies slightly the calculations at the outlet of the plane channel. It supposes that the boundary continues after channel outlet (i.e. velocity vector remains parallel to the boundary).
- **traitement\_gradients** *str* into ['oui', 'non']: Treatment of gradient calculations (yes or no). This option modifies slightly the gradient calculation at the corners and activates also the corner treatment option.
- **p\_imposee\_aux\_faces** *str* into ['oui', 'non']: Pressure imposed at the faces (yes or no).
- **deactivate\_arete\_mixte** : Deactivate the arete\_mixte contribution in the conv op of the momentum equation.
- **toutes\_les\_options** | **all\_options** : Activates all Option\_VDF options. If used, must be used alone without specifying the other options, nor combinations.

### 3.101 Orientefacesbord

Description: Keyword to modify the order of the boundary vertices included in a domain, such that the surface normals are outer pointing.

See also: [interpret \(3\)](#)

Usage:

**orientefacesbord** **domain\_name**

where

- **domain\_name** *str*: Name of domain.

### 3.102 Partition

Synonymous: **decouper**

Description: Class for parallel calculation to cut a domain for each processor. By default, this keyword is commented in the reference test cases.

See also: [interprete \(3\)](#)

Usage:

**partition** **domaine** **bloc\_decouper**

where

- **domaine** *str*: Name of the domain to be cut.
- **bloc\_decouper** *bloc\_decouper* ([3.103](#)): Description how to cut a domain.

### 3.103 Bloc\_decouper

Description: Auxiliary class to cut a domain.

See also: [objet\\_lecture \(47\)](#)

Usage:

```
{
 [Partition_toolpartitionneur partitionneur_deriv]
 [larg_joint int]
 [nom_zones str]
 [ecrire_decoupage str]
 [ecrire_lata str]
 [ecrire_med str]
 [nb_parts_tot int]
 [periodique n word1 word2 ... wordn]
 [reorder int]
 [single_hdf]
 [print_more_infos int]
```

}

where

- **Partition\_tool****partitionneur** *partitionneur\_deriv* ([32](#)): Defines the partitionning algorithm (the effective C++ object used is 'Partitionneur\_ALGORITHM\_NAME').
- **larg\_joint** *int*: This keyword specifies the thickness of the virtual ghost domaine (data known by one processor though not owned by it). The default value is 1 and is generally correct for all algorithms except the QUICK convection scheme that require a thickness of 2. Since the 1.5.5 version, the VEF discretization imply also a thickness of 2 (except VEF P0). Any non-zero positive value can be used, but the amount of data to store and exchange between processors grows quickly with the thickness.
- **nom\_zones** *str*: Name of the files containing the different partition of the domain. The files will be:  
:  
name\_0001.Zones  
name\_0002.Zones  
...  
name\_000n.Zones. If this keyword is not specified, the geometry is not written on disk (you might just want to generate a 'ecrire\_decoupage' or 'ecrire\_lata').

- **ecrire\_decoupage** *str*: After having called the partitionning algorithm, the resulting partition is written on disk in the specified filename. See also `partitionneur Fichier_Decoupage`. This keyword is useful to change the partition numbers: first, you write the partition into a file with the option `ecrire_decoupage`. This file contains the domaine number for each element's mesh. Then you can easily permute domaine numbers in this file. Then read the new partition to create the `.Zones` files with the `Fichier_Decoupage` keyword.
- **ecrire\_lata** *str*: Save the partition field in a LATA format file for visualization
- **ecrire\_med** *str*: Save the partition field in a MED format file for visualization
- **nb\_parts\_tot** *int*: Keyword to generates N `.Domaine` files, instead of the default number M obtained after the partitionning algorithm. N must be greater or equal to M. This option might be used to perform coupled parallel computations. Supplemental empty domaines from M to N-1 are created. This keyword is used when you want to run a parallel calculation on several domains with for example, 2 processors on a first domain and 10 on the second domain because the first domain is very small compare to second one. You will write `Nb_parts 2` and `Nb_parts_tot 10` for the first domain and `Nb_parts 10` for the second domain.
- **periodique** *n word1 word2 ... wordn*: N `BOUNDARY_NAME_1 BOUNDARY_NAME_2 ...` : N is the number of boundary names given. Periodic boundaries must be declared by this method. The partitionning algorithm will ensure that facing nodes and faces in the periodic boundaries are located on the same processor.
- **reorder** *int*: If this option is set to 1 (0 by default), the partition is renumbered in order that the processes which communicate the most are nearer on the network. This may slightly improves parallel performance.
- **single\_hdf** : Optional keyword to enable you to write the partitioned domaines in a single file in hdf5 format.
- **print\_more\_infos** *int*: If this option is set to 1 (0 by default), print infos about number of remote elements (ghosts) and additional infos about the quality of partitionning. Warning, it slows down the cutting operations.

### 3.104 Partition\_multi

Synonymous: **decouper\_multi**

Description: allows to partition multiple domains in contact with each other in parallel: necessary for resolution monolithique in implicit schemes and for all coupled problems using `PolyMAC_POPINC`. By default, this keyword is commented in the reference test cases.

See also: [interpret \(3\)](#)

Usage:

**partition\_multi aco domaine1 dom blocdecouppdom1 domaine2 dom2 blocdecouppdom2 acof**  
where

- **aco** *str* into `[ '{' ]`: Opening curly bracket.
- **domaine1** *str* into `[ 'domaine' ]`: not set.
- **dom** *str*: Name of the first domain to be cut.
- **blocdecouppdom1** *bloc\_decouper (3.103)*: Partition bloc for the first domain.
- **domaine2** *str* into `[ 'domaine' ]`: not set.
- **dom2** *str*: Name of the second domain to be cut.
- **blocdecouppdom2** *bloc\_decouper (3.103)*: Partition bloc for the second domain.
- **acof** *str* into `[ '}' ]`: Closing curly bracket.



### 3.105 Pilote\_icoco

Description: not\_set

See also: interpret (3)

Usage:

```
pilote_icoco {
 pb_name str
 main str
}
where
```

- **pb\_name** *str*
- **main** *str*

### 3.106 Polyedriser

Description: cast hexahedra into polyhedra so that the indexing of the mesh vertices is compatible with PolyMAC\_POPINC discretization. Must be used in PolyMAC\_POPINC discretization if a hexahedral mesh has been produced with TRUST's internal mesh generator.

See also: interpret (3)

Usage:

```
polyedriser domain_name
where
```

- **domain\_name** *str*: Name of domain.

### 3.107 Postraiter\_domaine

Description: To write one or more domains in a file with a specified format (MED,LML,LATA,SINGLE-LATA,CGNS).

See also: interpret (3)

Usage:

```
postraiter_domaine {
 format str into ['lml', 'lata', 'single_lata', 'lata_v2', 'med', 'cgns']
 [binaire int into [0, 1]]
 [ecrire_frontiere int into [0, 1]]
 [dual int into [0, 1]]
 [file|fichier str]
 [joints_non_postraites int into [0, 1]]
 [domain|domaine str]
 [domaines bloc_lecture]
}
where
```

- **format** *str* into ['lml', 'lata', 'single\_lata', 'lata\_v2', 'med', 'cgns']: File format.

- **binaire** *int into [0, 1]*: Binary (binaire 1) or ASCII (binaire 0) may be used. By default, it is 0 for LATA and only ASCII is available for LML and only binary is available for MED.
- **ecrire\_frontiere** *int into [0, 1]*: This option will write (if set to 1, the default) or not (if set to 0) the boundaries as fields into the file (it is useful to not add the boundaries when writing a domain extracted from another domain)
- **dual** *int into [0, 1]*: This option indicates whether the original mesh (default) or the dual one (the one used for postprocessing of field faces) is to be written.
- **filefichier** *str*: The file name can be changed with the fichier option.
- **joints\_non\_postraites** *int into [0, 1]*: The joints\_non\_postraites (1 by default) will not write the boundaries between the partitioned mesh.
- **domain|domaine** *str*: Name of domain
- **domaines** *bloc\_lecture (3.2)*: Names of domains : { name1 name2 }

### 3.108 Precisiongeom

Description: Class to change the way floating-point number comparison is done. By default, two numbers are equal if their absolute difference is smaller than  $1e-10$ . The keyword is useful to modify this value. Moreover, nodes coordinates will be written in .geom files with this same precision.

See also: [interpret \(3\)](#)

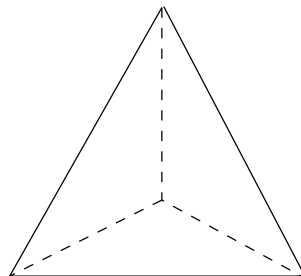
Usage:

**precisiongeom precision**  
where

- **precision** *float*: New value of precision.

### 3.109 Raffiner\_anisotrope

Description: Only for VEF discretizations, allows to cut triangle elements in 3, or tetrahedra in 4 parts, by defining a new summit located at the center of the element:



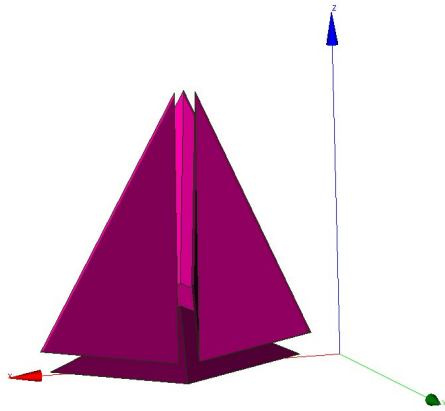
Note that such a cut creates flat elements (anisotropic).

See also: [interpret \(3\)](#)

Usage:

**raffiner\_anisotrope domain\_name**  
where

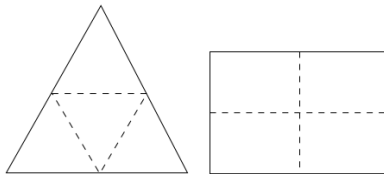
- **domain\_name** *str*: Name of domain.



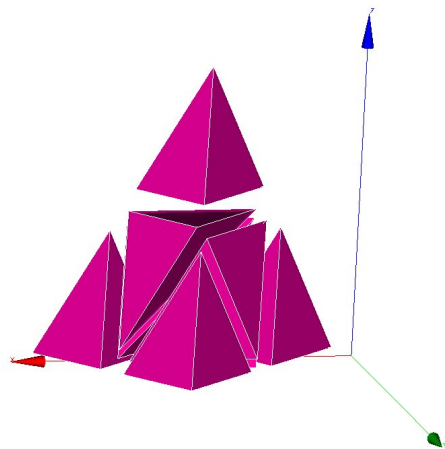
### 3.110 Raffiner\_isotrope

Synonymous: **raffiner\_simplexes**

Description: For VDF and VEF discretizations, allows to cut triangles/quadrangles or tetrahedral/hexaedras elements respectively in 4 or 8 new ones by defining new summits located at the middle of edges (and center of faces and elements for quadrangles and hexaedra). Such a cut preserves the shape of original elements (isotropic). For 2D elements:



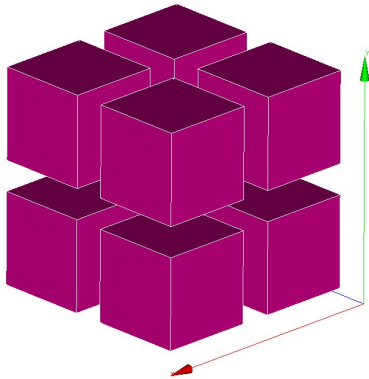
For 3D elements:



.

See also: [interpret \(3\)](#)

Usage:



**raffiner\_isotrope domain\_name**

where

- **domain\_name** *str*: Name of domain.

### 3.111 Read

Synonymous: **lire**

Description: Interpreter to read the **a\_object** object defined between the braces.

See also: [interpret \(3\)](#)

Usage:

**read a\_object bloc**

where

- **a\_object** *str*: Object to be read.
- **bloc** *str*: Definition of the object.

### 3.112 Read\_file

Synonymous: **lire\_fichier**

Description: Keyword to read the object **name\_obj** contained in the file **filename**.

This is notably used when the calculation domain has already been meshed and the mesh contains the file **filename**, simply write **read\_file dom filename** (where **dom** is the name of the meshed domain).

If the filename is **;**, is to execute a data set given in the file of name **name\_obj** (a space must be entered between the semi-colon and the file name).

See also: [interpret \(3\)](#) [read\\_unsupported\\_ascii\\_file\\_from\\_icem \(3.115\)](#) [read\\_file\\_binary \(3.113\)](#)

Usage:

**read\_file name\_obj filename**

where

- **name\_obj** *str*: Name of the object to be read.
- **filename** *str*: Name of the file.

### 3.113 Read\_file\_binary

Synonymous: **lire\_fichier\_bin**

Description: Keyword to read an object name\_obj in the unformatted type file filename.

See also: read\_file ([3.112](#))

Usage:

**read\_file\_binary name\_obj filename**

where

- **name\_obj** *str*: Name of the object to be read.
- **filename** *str*: Name of the file.

### 3.114 Lire\_tgrid

Description: Keyword to read Tgrid/Gambit mesh files. 2D (triangles or quadrangles) and 3D (tetra or hexa elements) meshes, may be read by TRUST.

See also: interpret ([3](#))

Usage:

**lire\_tgrid dom filename**

where

- **dom** *str*: Name of domaine.
- **filename** *str*: Name of file containing the mesh.

### 3.115 Read\_unsupported\_ascii\_file\_from\_icem

Description: not\_set

See also: read\_file ([3.112](#))

Usage:

**read\_unsupported\_ascii\_file\_from\_icem name\_obj filename**

where

- **name\_obj** *str*: Name of the object to be read.
- **filename** *str*: Name of the file.

### 3.116 Orienter\_simplexes

Synonymous: **rectify\_mesh**

Description: Keyword to refine a mesh

See also: interpret ([3](#))

Usage:

**orienter\_simplexes domain\_name**

where

- **domain\_name** *str*: Name of domain.

### 3.117 Redresser\_hexaedres\_vdf

Description: Keyword to convert a domain (named `domain_name`) with quadrilaterals/VEF hexaedras which looks like rectangles/VDF hexaedras into a domain with real rectangles/VDF hexaedras.

See also: [interpret \(3\)](#)

Usage:

**redresser\_hexaedres\_vdf** `domain_name`

where

- **domain\_name** *str*: Name of domain to resequence.

### 3.118 Refine\_mesh

Description: `not_set`

See also: [interpret \(3\)](#)

Usage:

**refine\_mesh** `domaine`

where

- **domaine** *str*

### 3.119 Regroupebord

Description: Keyword to build one boundary `new_bord` with several boundaries of the domain named `domaine`.

See also: [interpret \(3\)](#)

Usage:

**regroupebord** `domaine` `new_bord` `bords`

where

- **domaine** *str*: Name of domain
- **new\_bord** *str*: Name of the new boundary
- **bords** *bloc\_lecture* (3.2): { Bound1 Bound2 }

### 3.120 Remaillage\_ft\_ijk

Description: `not_set`

See also: [interpret \(3\)](#)

Usage:

**remaillage\_ft\_ijk** {

```
[pas_remaillage float]
[nb_iter_barycentrage int]
[relax_barycentrage float]
[critere_arete float]
[seuil_dvolume_residuel float]
```

```

[nb_iter_correction_volume int]
[nb_iter_remaillage int]
[facteur_longueur_ideale float]
[equilateral int]
[lissage_courbure_coeff float]
[lissage_courbure_iterations_systematique int]
[lissage_courbure_iterations_si_remaillage int]
}
where

```

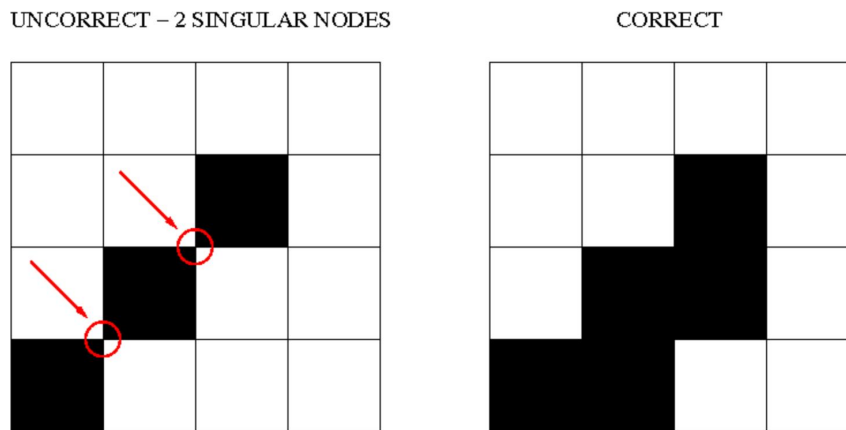
- pas\_remaillage *float*
- nb\_iter\_barycentrage *int*
- relax\_barycentrage *float*
- critere\_arete *float*
- seuil\_dvolume\_residuel *float*
- nb\_iter\_correction\_volume *int*
- nb\_iter\_remaillage *int*
- facteur\_longueur\_ideale *float*
- equilateral *int*
- lissage\_courbure\_coeff *float*
- lissage\_courbure\_iterations\_systematique *int*
- lissage\_courbure\_iterations\_si\_remaillage *int*

### 3.121 Remove\_elem

Description: Keyword to remove element from a VDF mesh (named *domaine\_name*), either from an explicit list of elements or from a geometric condition defined by a condition  $f(x,y)>0$  in 2D and  $f(x,y,z)>0$  in 3D. All the new borders generated are gathered in one boundary called : *newBord* (to rename it, use *RegroupeBord* keyword). To split it to different boundaries, use *DecoupeBord\_Pour\_Rayonnement* keyword). Example of a removed zone of radius 0.2 centered at  $(x,y)=(0.5,0.5)$ :

Remove\_elem dom { fonction  $0.2 * 0.2 - (x - 0.5)^2 - (y - 0.5)^2 > 0$  }

Warning : the thickness of removed zone has to be large enough to avoid singular nodes as decribed below :



See also: [interpret \(3\)](#)

Usage:

**remove\_elem** **domaine** **bloc**

where

- **domaine** *str*: Name of domain
- **bloc** *remove\_elem\_bloc* ([3.122](#))

### 3.122 Remove\_elem\_bloc

Description: not\_set

See also: objet\_lecture ([47](#))

Usage:

```
{

 [liste n n1 n2 ... nn]
 [fonction str]

}
```

where

- **liste** *n n1 n2 ... nn*
- **fonction** *str*

### 3.123 Remove\_invalid\_internal\_boundaries

Description: Keyword to suppress an internal boundary of the domain\_name domain. Indeed, some mesh tools may define internal boundaries (eg: for post processing task after the calculation) but TRUST does not support it yet.

See also: interpret ([3](#))

Usage:

**remove\_invalid\_internal\_boundaries** **domain\_name**

where

- **domain\_name** *str*: Name of domain.

### 3.124 Reorienter\_tetraedres

Description: This keyword is mandatory for front-tracking computations with the VEF discretization. For each tetrahedral element of the domain, it checks if it has a positive volume. If the volume (determinant of the three vectors) is negative, it swaps two nodes to reverse the orientation of this tetrahedron.

See also: interpret ([3](#))

Usage:

**reorienter\_tetraedres** **domain\_name**

where

- **domain\_name** *str*: Name of domain.



### 3.125 Reorienter\_triangles

Description: not\_set

See also: interpreté (3)

Usage:

**reorienter\_triangles** **domain\_name**

where

- **domain\_name** *str*: Name of domain.

### 3.126 Reordonner

Description: The Reordonner\_32\_64 interpreter is required sometimes for a VDF mesh which is not produced by the internal mesher. Example where this is used:

Read\_file dom fichier.geom

Reordonner\_32\_64 dom

Observations: This keyword is redundant when the mesh that is read is correctly sequenced in the TRUST sense. This significant mesh operation may take some time... The message returned by TRUST is not explicit when the Reordonner\_32\_64 (Resequencing) keyword is required but not included in the data set...

See also: interpreté (3)

Usage:

**reordonner** **domain\_name**

where

- **domain\_name** *str*: Name of domain to resequence.

### 3.127 Residuals

Description: To specify how the residuals will be computed.

See also: interpreté (3)

Usage:

**residuals** {

[ **norm** *str* into ['L2', 'max']

[ **relative** *str* into ['0', '1', '2']

}

where

- **norm** *str* into ['L2', 'max']: allows to choose the norm we want to use (max norm by default). Possible to specify L2-norm.
- **relative** *str* into ['0', '1', '2']: This is the old keyword seuil\_statio\_relatif\_deconseille. If it is set to 1, it will normalize the residuals with the residuals of the first 5 timesteps (default is 0). if set to 2, residual will be computed as  $R/(\max - \min)$ .

### 3.128 Rotation

Description: Keyword to rotate the geometry of an arbitrary angle around an axis aligned with Ox, Oy or Oz axis.

See also: [interpret](#) (3)

Usage:

**rotation domain\_name dir coord1 coord2 angle**

where

- **domain\_name** *str*: Name of domain to which the transformation is applied.
- **dir** *str* into ['X', 'Y', 'Z']: X, Y or Z to indicate the direction of the rotation axis
- **coord1** *float*: coordinates of the center of rotation in the plane orthogonal to the rotation axis. These coordinates must be specified in the direct triad sense.
- **coord2** *float*
- **angle** *float*: angle of rotation (in degrees)

### 3.129 Scatter

Description: Class to read a partitioned mesh from the files during a parallel calculation. The files are in binary format.

See also: [interpret](#) (3) [scattermed](#) (3.130)

Usage:

**scatter file domaine**

where

- **file** *str*: Name of file.
- **domaine** *str*: Name of domain.

### 3.130 Scattermed

Description: This keyword will read the partition of the domain\_name domain into a the MED format files file.med created by Medsplitter.

See also: [scatter](#) (3.129)

Usage:

**scattermed file domaine**

where

- **file** *str*: Name of file.
- **domaine** *str*: Name of domain.

### 3.131 Solve

Synonymous: **resoudre**

Description: Interpreter to start calculation with TRUST.

Keyword Discretize should have already been used to read the object.

See also: [interpret](#) (3)

Usage:

**solve pb**  
where

- **pb** *str*: Name of problem to be solved.

### 3.132 Stat\_per\_proc\_perf\_log

Description: Keyword allowing to activate the detailed statistics per processor (by default this is false, and only the master proc will produce stats).

See also: [interpret](#) (3)

Usage:

**stat\_per\_proc\_perf\_log flg**  
where

- **flg** *int*: A rien that can be either 0 or 1 to turn off (default) or on the detailed stats.

### 3.133 Supprime\_bord

Description: Keyword to remove boundaries (named Boundary\_name1 Boundary\_name2 ) of the domain named domain\_name.

See also: [interpret](#) (3)

Usage:

**supprime\_bord domaine bords**  
where

- **domaine** *str*: Name of domain
- **bords** *list\_nom* (3.134): { Boundary\_name1 Boundaray\_name2 }

### 3.134 List\_nom

Description: List of name.

See also: [listobj](#) (46.5)

Usage:

{ object1 object2 .... }  
list of *nom\_anonyme* (31.1)

### 3.135 System

Description: To run Unix commands from the data file. Example: System 'echo The End | mail trust@cea.fr'

See also: [interpret](#) (3)

Usage:

**system cmd**  
where

- **cmd** *str*: command to execute.

### 3.136 Test\_solveur

Description: To test several solvers

See also: [interpret](#) (3)

Usage:

```
test_solveur {
 [fichier_secmem str]
 [fichier_matrice str]
 [fichier_solution str]
 [nb_test int]
 [impr]
 [solveur solveur_sys_base]
 [fichier_solveur str]
 [genere_fichier_solveur float]
 [seuil_verification float]
 [pas_de_solution_initiale]
 [ascii]
}
```

where

- **fichier\_secmem** *str*: Filename containing the second member B
- **fichier\_matrice** *str*: Filename containing the matrix A
- **fichier\_solution** *str*: Filename containing the solution x
- **nb\_test** *int*: Number of tests to measure the time resolution (one preconditionnement)
- **impr** : To print the convergence solver
- **solveur** *solveur\_sys\_base* (14.19): To specify a solver
- **fichier\_solveur** *str*: To specify a file containing a list of solvers
- **genere\_fichier\_solveur** *float*: To create a file of the solver with a threshold convergence
- **seuil\_verification** *float*: Check if the solution satisfy  $\|Ax-B\| < \text{precision}$
- **pas\_de\_solution\_initiale** : Resolution isn't initialized with the solution x
- **ascii** : Ascii files

### 3.137 Testeur

Description: not\_set

See also: [interpret](#) (3)

Usage:

```
testeur data
where
```

- **data** *bloc\_lecture* (3.2)

### 3.138 Testeur\_medcoupling

Description: not\_set

See also: interpret (3)

Usage:

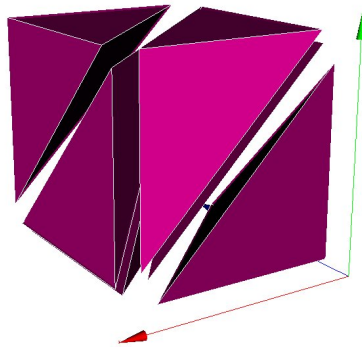
**testeur\_medcoupling pb\_name field\_name**

where

- **pb\_name** *str*: Name of domain.
- **field\_name** *str*: Name of domain.

### 3.139 Tetraedriser

Description: To achieve a tetrahedral mesh based on a mesh comprising blocks, the Tetraedriser (Tetrahe-  
dralise) interpreter is used in VEF discretization. Initial block is divided in 6 tetrahedra:



See also: interpret (3) tetraedriser\_homogene (3.140) tetraedriser\_homogene\_compact (3.141) tetraedriser-  
\_homogene\_fin (3.142) tetraedriser\_par\_prisme (3.143)

Usage:

**tetraedriser domain\_name**

where

- **domain\_name** *str*: Name of domain.

### 3.140 Tetraedriser\_homogene

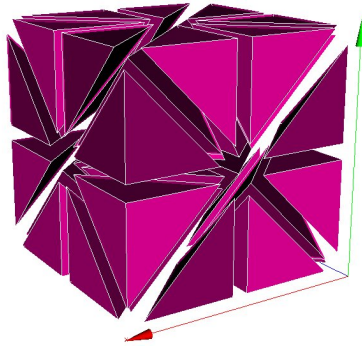
Description: Use the Tetraedriser\_homogene (Homogeneous\_Tetrahedralisation) interpreter in VEF dis-  
cretization to mesh a block in tetrahedrals. Each block hexahedral is no longer divided into 6 tetrahedrals  
(keyword Tetraedriser (Tetrahedralise)), it is now broken down into 40 tetrahedrals. Thus a block defined  
with 11 nodes in each X, Y, Z direction will contain  $10*10*10*40=40,000$  tetrahedrals. This also allows  
problems in the mesh corners with the P1NC/P1iso/P1bulle or P1/P1 discretization items to be avoided.  
Initial block is divided in 40 tetrahedra:

See also: tetraedriser (3.139)

Usage:

**tetraedriser\_homogene domain\_name**

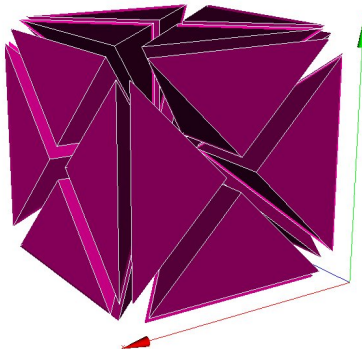
where



- **domain\_name** *str*: Name of domain.

### 3.141 Tetraedriser\_homogene\_compact

Description: This new discretization generates tetrahedral elements from cartesian or non-cartesian hexahedral elements. The process cut each hexahedral in 6 pyramids, each of them being cut then in 4 tetrahedral. So, in comparison with tetra\_homogene, less elements (\*24 instead of\*40) with more homogeneous volumes are generated. Moreover, this process is done in a faster way. Initial block is divided in 24 tetrahedra:



See also: tetraedriser ([3.139](#))

Usage:

**tetraedriser\_homogene\_compact domain\_name**  
where

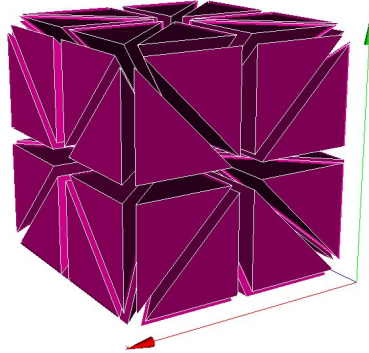
- **domain\_name** *str*: Name of domain.

### 3.142 Tetraedriser\_homogene\_fin

Description: Tetraedriser\_homogene\_fin is the recommended option to tetrahedralise blocks. As an extension (subdivision) of Tetraedriser\_homogene\_compact, this last one cut each initial block in 48 tetrahedra (against 24, previously). This cutting ensures :

- a correct cutting in the corners (in respect to pressure discretization PreP1B),
- a better isotropy of elements than with Tetraedriser\_homogene\_compact,
- a better alignment of summits (this could have a benefit effect on calculation near walls since first elements in contact with it are all contained in the same constant thickness and ii/ by the way, a 3D cartesian

grid based on summits can be engendered and used to realise spectral analysis in HIT for instance). Initial block is divided in 48 tetrahedra:



See also: `tetraedriser` ([3.139](#))

Usage:

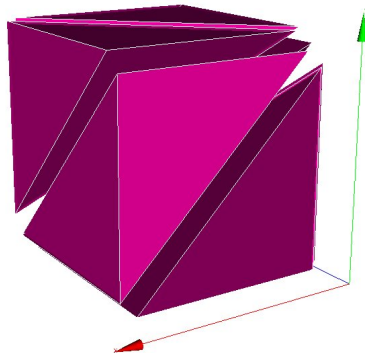
**`tetraedriser_homogene_fin`** **`domain_name`**

where

- **`domain_name`** *str*: Name of domain.

### 3.143 `Tetraedriser_par_prisme`

Description: `Tetraedriser_par_prisme` generates 6 iso-volume tetrahedral element from primary hexahedral one (contrarily to the 5 elements ordinarily generated by `tetraedriser`). This element is suitable for calculation of gradients at the summit (coincident with the gravity centre of the jointed elements related with) and spectra (due to a better alignment of the points).



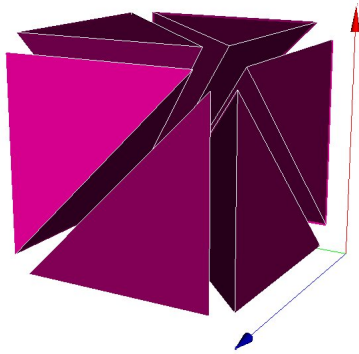
Initial block is divided in 6 prisms.

See also: `tetraedriser` ([3.139](#))

Usage:

**`tetraedriser_par_prisme`** **`domain_name`**

where



- **domain\_name** *str*: Name of domain.

### 3.144 Transformer

Description: Keyword to transform the coordinates of the geometry.

Exemple to rotate your mesh by a 90o rotation and to scale the z coordinates by a factor 2: Transformer domain\_name -y -x 2\*z

See also: interpret (3)

Usage:

**transformer domain\_name formule**

where

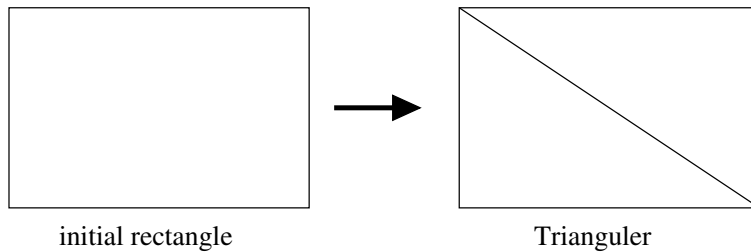
- **domain\_name** *str*: Name of domain.
- **formule** *word1 word2 (word3)*: Function\_for\_x Function\_for\_y

*Function\_forz*

### 3.145 Trianguler

Description: To achieve a triangular mesh from a mesh comprising rectangles (2 triangles per rectangle).

Should be used in VEF discretization. Principle:



See also: interpret (3) trianguler\_fin (3.146) trianguler\_h (3.147)

Usage:

**triangler domain\_name**

where



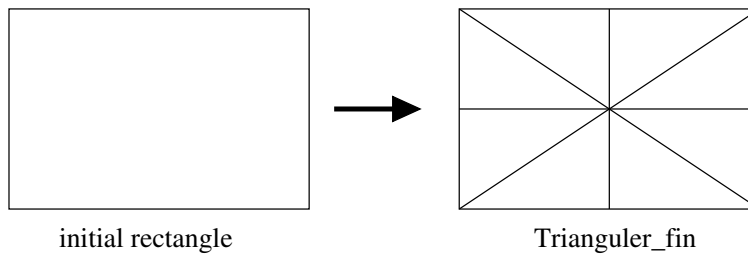
- **domain\_name** *str*: Name of domain.

### 3.146 Trianguler\_fin

Description: Trianguler\_fin is the recommended option to triangulate rectangles.

As an extension (subdivision) of Triangulate\_h option, this one cut each initial rectangle in 8 triangles (against 4, previously). This cutting ensures :

- a correct cutting in the corners (in respect to pressure discretization PreP1B).
- a better isotropy of elements than with Trianguler\_h option.
- a better alignment of summits (this could have a benefit effect on calculation near walls since first elements in contact with it are all contained in the same constant thickness, and, by this way, a 2D cartesian grid based on summits can be engendered and used to realize statistical analysis in plane channel configuration for instance). Principle:



See also: [trianguler \(3.145\)](#)

Usage:

**trianguler\_fin** **domain\_name**

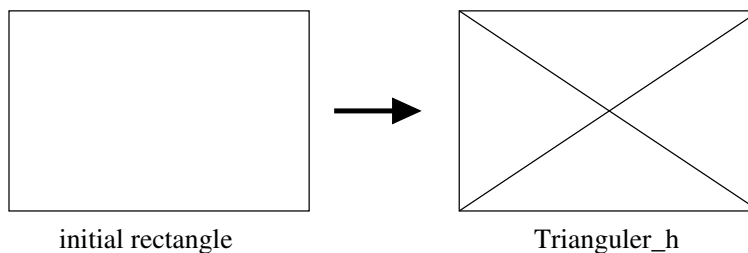
where

- **domain\_name** *str*: Name of domain.

### 3.147 Trianguler\_h

Description: To achieve a triangular mesh from a mesh comprising rectangles (4 triangles per rectangle).

Should be used in VEF discretization. Principle:



See also: [trianguler \(3.145\)](#)

Usage:

**trianguler\_h domain\_name**

where

- **domain\_name** *str*: Name of domain.

### 3.148 Verifier\_qualite\_raffinements

Description: not\_set

See also: interpret (3)

Usage:

**verifier\_qualite\_raffinements domain\_names**

where

- **domain\_names** *vect\_nom* (3.149)

### 3.149 Vect\_nom

Description: Vect of name.

See also: listobj (46.5)

Usage:

n object1 object2 ....

list of *nom\_anonyme* (31.1)

### 3.150 Verifier\_simplexes

Description: Keyword to refine a simplexes

See also: interpret (3)

Usage:

**verifier\_simplexes domain\_name**

where

- **domain\_name** *str*: Name of domain.

### 3.151 Verifiercoin

Description: This keyword subdivides inconsistent 2D/3D cells used with VEFPreP1B discretization. Must be used before the mesh is discretized. The Read\_file option can be used only if the file.decoupage\_som was previously created by TRUST. This option, only in 2D, reverses the common face at two cells (at least one is inconsistent), through the nodes opposed. In 3D, the option has no effect.

See also: interpret (3)

Usage:

**verifiercoin domain\_name bloc**

where

- **domain\_name** *str*: Name of the domaine
- **bloc** *verifiercoin\_bloc* (3.152)

### 3.152 Verifiercoin\_bloc

Description: not\_set

See also: objet\_lecture ([47](#))

Usage:

```
{
 [Lire_fichier|Read_file str]
}
where
```

- **Lire\_fichier|Read\_file** *str*: name of the \*.decoupage\_som file

### 3.153 Ecrire

Description: Keyword to write the object of name name\_obj to a standard outlet.

See also: interprete ([3](#))

Usage:

```
ecrire name_obj
where
```

- **name\_obj** *str*: Name of the object to be written.

### 3.154 Ecrire\_fichier\_bin

Synonymous: **ecrire\_fichier**

Description: Keyword to write the object of name name\_obj to a file filename. Since the v1.6.3, the default format is now binary format file.

See also: interprete ([3](#)) **ecrire\_fichier\_formatte** ([3.56](#))

Usage:

```
ecrire_fichier_bin name_obj filename
where
```

- **name\_obj** *str*: Name of the object to be written.
- **filename** *str*: Name of the file.

## 4 pb\_gen\_base

Description: Basic class for problems.

See also: objet\_u ([48](#)) **Pb\_base** ([4.34](#)) **probleme\_couple** ([4.36](#)) **pbc\_med** ([4.73](#)) **pb\_mg** ([4.55](#))

Usage:

## 4.1 Pb\_conduction

Description: Resolution of the heat equation.

Keyword Discretize should have already been used to read the object.

See also: Pb\_base (4.34) Pb\_Rayo\_Conduction (4.21)

Usage:

**Pb\_Conduction** *str*

**Read** *str* {

```
[solide solide]
[Conduction conduction]
[milieu milieu_base]
[constituant constituant]
[Post_processing|postraitement corps_postraitement]
[Post_processings|postraitements post_processings]
[liste_de_postraitements liste_post_ok]
[liste_postraitements liste_post]
[sauvegarde format_file_base]
[sauvegarde_simple format_file_base]
[reprise format_file_base]
[resume_last_time format_file_base]
```

}

where

- **solide** *solide* (26.18): The medium associated with the problem.
- **Conduction** *conduction* (5.1): Heat equation.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \geq P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.

- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the *name\_file* file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.2 Corps\_postraitement

Description: not\_set

See also: post\_processing (4.4.3)

Usage:

```
{
 [t_debut_statistiques float]
 [nb_pas_dt_post_stats_plans float]
 [nb_pas_dt_post_stats_bulles float]
 [expression_vx_ana str]
 [expression_vy_ana str]
 [expression_vz_ana str]
 [expression_p_ana str]
 [interfaces interface_posts]
 [fichier str]
 [format str into ['lml', 'lata', 'single_lata', 'lata_v2', 'med', 'cgns']]
 [dt_post str]
 [nb_pas_dt_post int]
 [domaine str]
 [sous_zone|sous_domaine str]
 [parallele str into ['simple', 'multiple', 'mpi-io']]
 [definition_champs definition_champs]
 [definition_champs_file|definition_champs_fichier definition_champs_fichier]
 [probes|sondes sondes]
 [probes_file|sondes_fichier sondes_fichier]
 [mobile_probes|sondes mobiles sondes]
 [mobile_probes_file|sondes mobiles_fichier sondes_fichier]
 [deprecatedkeepduplicatedprobes int]
 [fields|champs champs_posts]
 [fields_file|champs_fichier champs_posts_fichier]
 [statistics|statistiques stats_posts]
 [statistics_file|statistiques_fichier stats_posts_fichier]
 [serial_statistics|statistiques_en_serie stats_serie_posts]
 [serial_statistics_file|statistiques_en_serie_fichier stats_serie_posts_fichier]
 [suffix_for_reset str]
}
```

where

- **t\_debut\_statistiques** *float* for inheritance: not\_set (for IJK)
- **nb\_pas\_dt\_post\_stats\_plans** *float* for inheritance: not\_set (for IJK)
- **nb\_pas\_dt\_post\_stats\_bulles** *float* for inheritance: not\_set (for IJK)
- **expression\_vx\_ana** *str* for inheritance: not\_set (for IJK)
- **expression\_vy\_ana** *str* for inheritance: not\_set (for IJK)
- **expression\_vz\_ana** *str* for inheritance: not\_set (for IJK)
- **expression\_p\_ana** *str* for inheritance: not\_set (for IJK)
- **interfaces** *interface\_posts* (4.2.1) for inheritance: Keyword to read all the characteristics of the interfaces. Different kind of interfaces exist as well as different interface initialisations.

- **fichier** *str* for inheritance: Name of file.
- **format** *str* into ['lml', 'lata', 'single\_lata', 'lata\_v2', 'med', 'cgns'] for inheritance: This optional parameter specifies the format of the output file. The basename used for the output file is the base-name of the data file. For the fmt parameter, choices are lml or lata. A short description of each format can be found below. The default value is lml.
- **dt\_post** *str* for inheritance: Field's write frequency (as a time period) - can also be specified after the 'field' keyword.
- **nb\_pas\_dt\_post** *int* for inheritance: Field's write frequency (as a number of time steps) - can also be specified after the 'field' keyword.
- **domaine** *str* for inheritance: This optional parameter specifies the domain on which the data should be interpolated before it is written in the output file. The default is to write the data on the domain of the current problem (no interpolation).
- **sous\_zonelsous\_domaine** *str* for inheritance: This optional parameter specifies the sub\_domaine on which the data should be interpolated before it is written in the output file. It is only available for sequential computation.
- **parallele** *str* into ['simple', 'multiple', 'mpi-io'] for inheritance: Select simple (single file, sequential write), multiple (several files, parallel write), or mpi-io (single file, parallel write) for LATA format
- **definition\_champs** *definition\_champs* (4.2.4) for inheritance: Keyword to create new or more complex field for advanced postprocessing.
- **definition\_champs\_filedefinition\_champs\_fichier** *definition\_champs\_fichier* (4.2.6) for inheritance: Definition\_champs read from file.
- **probesondes** *sondes* (4.2.7) for inheritance: Probe.
- **probes\_filesondes\_fichier** *sondes\_fichier* (4.2.24) for inheritance: Probe read from a file.
- **mobile\_probesondes\_mobiles** *sondes* (4.2.7) for inheritance: Mobile probes useful for ALE, their positions will be updated in the mesh.
- **mobile\_probes\_filesondes\_mobiles\_fichier** *sondes\_fichier* (4.2.24) for inheritance: Mobile probes read in a file
- **deprecatedkeepduplicatedprobes** *int* for inheritance: Flag to not remove duplicated probes in .son files (1: keep duplicate probes, 0: remove duplicate probes)
- **fieldslchamps** *champs\_posts* (4.2.25) for inheritance: Field's write mode.
- **fields\_filelchamps\_fichier** *champs\_posts\_fichier* (4.2.26) for inheritance: Fields read from file.
- **statisticslstatistiques** *stats\_posts* (4.2.28) for inheritance: Statistics between two points fixed : start of integration time and end of integration time.
- **statistics\_filelstatistiques\_fichier** *stats\_posts\_fichier* (4.2.36) for inheritance: Statistics read from file.
- **serial\_statisticslstatistiques\_en\_serie** *stats\_serie\_posts* (4.2.37) for inheritance: Statistics between two points not fixed : on period of integration.
- **serial\_statistics\_filelstatistiques\_en\_serie\_fichier** *stats\_serie\_posts\_fichier* (4.2.38) for inheritance: Serial\_statistics read from a file
- **suffix\_for\_reset** *str* for inheritance: Suffix used to modify the postprocessing file name if the ICoCo resetTime() method is invoked.

#### 4.2.1 Interface\_posts

Description: not set

See also: objet\_lecture (47)

Usage:

[ **nom\_interf** ] **blocs**

where

- **nom\_interf** *str*: name of the interface to post process

- **blocs** *champs\_a\_post* (4.2.2): Post-processed fields.

#### 4.2.2 Champs\_a\_post

Description: Fields to be post-processed.

See also: *listobj* (46.5)

Usage:

{ object1 object2 .... }

list of *champ\_a\_post* (4.2.3)

#### 4.2.3 Champ\_a\_post

Description: Field to be post-processed.

See also: *objet\_lecture* (47)

Usage:

**champ** [ **localisation** ]

where

- **champ** *str*: Name of the post-processed field.
- **localisation** *str into ['elem', 'som', 'faces']*: Localisation of post-processed field values: The two available values are elem, som, or faces (LATA format only) used respectively to select field values at mesh centres (CHAMPMAILLE type field in the lml file) or at mesh nodes (CHAMPPPOINT type field in the lml file). If no selection is made, localisation is set to som by default.

#### 4.2.4 Definition\_champs

Description: List of definition champ

See also: *listobj* (46.5)

Usage:

{ object1 object2 .... }

list of *definition\_champ* (4.2.5)

#### 4.2.5 Definition\_champ

Description: Keyword to create new complex field for advanced postprocessing.

See also: *objet\_lecture* (47)

Usage:

**name** **champ\_generique**

where

- **name** *str*: The name of the new created field.
- **champ\_generique** *champ\_generique\_base* (12)

#### 4.2.6 Definition\_champs\_fichier

Description: Keyword to read definition\_champs from a file

See also: objet\_lecture (47)

Usage:

```
{
 filefichier str
}
where
```

- **filefichier** *str*: name of file

#### 4.2.7 Sondes

Description: List of probes.

See also: listobj (46.5)

Usage:

```
{ object1 object2 }
list of sonde (4.2.8)
```

#### 4.2.8 Sonde

Description: Keyword is used to define the probes. Observations: the probe coordinates should be given in Cartesian coordinates (X, Y, Z), including axisymmetric.

See also: objet\_lecture (47)

Usage:

```
nom_sonde [special] nom_inco mperiode prd type
where
```

- **nom\_sonde** *str*: Name of the file in which the values taken over time will be saved. The complete file name is nom\_sonde.son.
- **special** *str into* ['grav', 'som', 'nodes', 'chsom', 'gravcl']: Option to change the positions of the probes. Several options are available:
  - grav : each probe is moved to the nearest cell center of the mesh;
  - som : each probe is moved to the nearest vertex of the mesh
  - nodes : each probe is moved to the nearest face center of the mesh;
  - chsom : only available for P1NC sampled field. The values of the probes are calculated according to P1-Conform corresponding field.
  - gravcl : Extend to the domain face boundary a cell-located segment probe in order to have the boundary condition for the field. For this type the extreme probe point has to be on the face center of gravity.
- **nom\_inco** *str*: Name of the sampled field.
- **mperiode** *str into* ['periode']: Keyword to set the sampled field measurement frequency.
- **prd** *float*: Period value. Every prd seconds, the field value calculated at the previous time step is written to the nom\_sonde.son file.
- **type** *sonde\_base* (4.2.9): Type of probe.



#### 4.2.9 Sonde\_base

Description: Basic probe. Probes refer to sensors that allow a value or several points of the domain to be monitored over time. The probes may be a set of points defined one by one (keyword Points) or a set of points evenly distributed over a straight segment (keyword Segment) or arranged according to a layout (keyword Plan) or according to a parallelepiped (keyword Volume). The fields allow all the values of a physical value on the domain to be known at several moments in time.

See also: `objet_lecture` (47) `segmentfacesx` (4.2.10) `segmentfacesy` (4.2.11) `segmentfacesz` (4.2.12) `radius` (4.2.13) `points` (4.2.14) `numero_elem_sur_maitre` (4.2.17) `position_like` (4.2.18) `plan` (4.2.19) `volume` (4.2.20) `circle` (4.2.21) `circle_3` (4.2.22) `segment` (4.2.23)

Usage:

**sonde\_base**

#### 4.2.10 Segmentfacesx

Description: Segment probe where points are moved to the nearest x faces

See also: `sonde_base` (4.2.9)

Usage:

**segmentfacesx nbr point\_deb point\_fin**

where

- **nbr** *int*: Number of probe points of the segment, evenly distributed.
- **point\_deb** *un\_point* (3.4.7): First outer probe segment point.
- **point\_fin** *un\_point* (3.4.7): Second outer probe segment point.

#### 4.2.11 Segmentfacesy

Description: Segment probe where points are moved to the nearest y faces

See also: `sonde_base` (4.2.9)

Usage:

**segmentfacesy nbr point\_deb point\_fin**

where

- **nbr** *int*: Number of probe points of the segment, evenly distributed.
- **point\_deb** *un\_point* (3.4.7): First outer probe segment point.
- **point\_fin** *un\_point* (3.4.7): Second outer probe segment point.

#### 4.2.12 Segmentfacesz

Description: Segment probe where points are moved to the nearest z faces

See also: `sonde_base` (4.2.9)

Usage:

**segmentfacesz nbr point\_deb point\_fin**

where

- **nbr** *int*: Number of probe points of the segment, evenly distributed.

- **point\_deb** *un\_point* (3.4.7): First outer probe segment point.
- **point\_fin** *un\_point* (3.4.7): Second outer probe segment point.

#### 4.2.13 Radius

Description: not\_set

See also: sonde\_base (4.2.9)

Usage:

**radius nbr point\_deb radius teta1 teta2**

where

- **nbr** *int*: Number of probe points of the segment, evenly distributed.
- **point\_deb** *un\_point* (3.4.7): First outer probe segment point.
- **radius** *float*
- **teta1** *float*
- **teta2** *float*

#### 4.2.14 Points

Description: Keyword to define the number of probe points. The file is arranged in columns.

See also: sonde\_base (4.2.9) segmentpoints (4.2.15) point (4.2.16)

Usage:

**points points**

where

- **points** *listpoints* (3.4.6): Probe points.

#### 4.2.15 Segmentpoints

Description: This keyword is used to define a probe segment from specifics points. The nom\_champ field is sampled at ns specifics points.

See also: points (4.2.14)

Usage:

**segmentpoints points**

where

- **points** *listpoints* (3.4.6): Probe points.

#### 4.2.16 Point

Description: Point as class-daughter of Points.

See also: points (4.2.14)

Usage:

**point points**

where

- **points** *listpoints* (3.4.6): Probe points.

#### 4.2.17 Numero\_elem\_sur\_maitre

Description: Keyword to define a probe at the special element. Useful for min/max sonde.

See also: sonde\_base ([4.2.9](#))

Usage:

**numero\_elem\_sur\_maitre** **numero**  
where

- **numero** *int*: element number

#### 4.2.18 Position\_like

Description: Keyword to define a probe at the same position of another probe named autre\_sonde.

See also: sonde\_base ([4.2.9](#))

Usage:

**position\_like** **autre\_sonde**  
where

- **autre\_sonde** *str*: Name of the other probe.

#### 4.2.19 Plan

Description: Keyword to set the number of probe layout points. The file format is type .lml

See also: sonde\_base ([4.2.9](#))

Usage:

**plan** **nbr** **nbr2** **point\_deb** **point\_fin** **point\_fin\_2**  
where

- **nbr** *int*: Number of probes in the first direction.
- **nbr2** *int*: Number of probes in the second direction.
- **point\_deb** *un\_point* ([3.4.7](#)): First point defining the angle. This angle should be positive.
- **point\_fin** *un\_point* ([3.4.7](#)): Second point defining the angle. This angle should be positive.
- **point\_fin\_2** *un\_point* ([3.4.7](#)): Third point defining the angle. This angle should be positive.

#### 4.2.20 Volume

Description: Keyword to define the probe volume in a parallelepiped passing through 4 points and the number of probes in each direction.

See also: sonde\_base ([4.2.9](#))

Usage:

**volume** **nbr** **nbr2** **nbr3** **point\_deb** **point\_fin** **point\_fin\_2** **point\_fin\_3**  
where

- **nbr** *int*: Number of probes in the first direction.
- **nbr2** *int*: Number of probes in the second direction.

- **nbr3** *int*: Number of probes in the third direction.
- **point\_deb** *un\_point* (3.4.7): Point of origin.
- **point\_fin** *un\_point* (3.4.7): Point defining the first direction (from point of origin).
- **point\_fin\_2** *un\_point* (3.4.7): Point defining the second direction (from point of origin).
- **point\_fin\_3** *un\_point* (3.4.7): Point defining the third direction (from point of origin).

#### 4.2.21 Circle

Description: Keyword to define several probes located on a circle.

See also: *sonde\_base* (4.2.9)

Usage:

**circle** **nbr** **point\_deb** [ **direction** ] **radius** **theta1** **theta2**

where

- **nbr** *int*: Number of probes between teta1 and teta2 (angles given in degrees).
- **point\_deb** *un\_point* (3.4.7): Center of the circle.
- **direction** *int into* [0, 1, 2]: Axis normal to the circle plane (0:x axis, 1:y axis, 2:z axis).
- **radius** *float*: Radius of the circle.
- **theta1** *float*: First angle.
- **theta2** *float*: Second angle.

#### 4.2.22 Circle\_3

Description: Keyword to define several probes located on a circle (in 3-D space).

See also: *sonde\_base* (4.2.9)

Usage:

**circle\_3** **nbr** **point\_deb** **direction** **radius** **theta1** **theta2**

where

- **nbr** *int*: Number of probes between teta1 and teta2 (angles given in degrees).
- **point\_deb** *un\_point* (3.4.7): Center of the circle.
- **direction** *int into* [0, 1, 2]: Axis normal to the circle plane (0:x axis, 1:y axis, 2:z axis).
- **radius** *float*: Radius of the circle.
- **theta1** *float*: First angle.
- **theta2** *float*: Second angle.

#### 4.2.23 Segment

Description: Keyword to define the number of probe segment points. The file is arranged in columns.

See also: *sonde\_base* (4.2.9)

Usage:

**segment** **nbr** **point\_deb** **point\_fin**

where

- **nbr** *int*: Number of probe points of the segment, evenly distributed.
- **point\_deb** *un\_point* (3.4.7): First outer probe segment point.
- **point\_fin** *un\_point* (3.4.7): Second outer probe segment point.

#### 4.2.24 Sondes\_fichier

Description: Keyword to read probes from a file

See also: [objet\\_lecture \(47\)](#)

Usage:

```
{

 filefichier str

}
```

where

- **filefichier** *str*: name of file

#### 4.2.25 Champs\_posts

Description: Field's write mode.

See also: [objet\\_lecture \(47\)](#)

Usage:

```
[format] [mot] [period] fields|champs
where
```

- **format** *str* into ['binaire', 'formatte']: Type of file.
- **mot** *str* into ['dt\_post', 'nb\_pas\_dt\_post']: Keyword to set the kind of the field's write frequency. Either a time period or a time step period. it can be specified either here, or at the beginning of the postprocessing bloc.
- **period** *str*: Value of the period which can be like (2.\*t).
- **fields|champs** *champs\_a\_post* ([4.2.2](#)): Post-processed fields.

#### 4.2.26 Champs\_posts\_fichier

Description: Fields read from file.

See also: [objet\\_lecture \(47\)](#)

Usage:

```
[format] [mot] [period] fichier
where
```

- **format** *str* into ['binaire', 'formatte']: Type of file.
- **mot** *str* into ['dt\_post', 'nb\_pas\_dt\_post']: Keyword to set the kind of the field's write frequency. Either a time period or a time step period.
- **period** *str*: Value of the period which can be like (2.\*t).
- **fichier** *bloc\_fichier* ([4.2.27](#)): name of file

#### 4.2.27 Bloc\_fichier

Description: Block containing the name of the file

See also: [objet\\_lecture \(47\)](#)

Usage:

```
{
```

**fichier** *str*  
 }  
 where

- **fichier** *str*: File name

#### 4.2.28 Stats\_posts

Description: Post-processing for statistics.

Example:

```
Statistiques Dt_post dtst {
 t_deb 0.1 t_fin 0.12
Moyenne Pression
Ecart_type Pression
Correlation Vitesse Vitesse }
```

It will write every **dt\_post** the mean, standard deviation and correlation value:

$$\begin{aligned}
 & t \leq t_{\text{deb}} \text{ or } t \geq t_{\text{fin}} : \\
 & \text{average: } \overline{P(t)} = 0 \\
 & \text{std\_deviation: } \langle P(t) \rangle = 0 \\
 & \text{correlation: } \langle U(t).V(t) \rangle = 0 \\
 \\
 & t > t_{\text{deb}} \text{ and } t < t_{\text{fin}} : \\
 & \text{average: } \overline{P(t)} = \frac{1}{t - t_{\text{deb}}} \int_{t_{\text{deb}}}^t P(s) ds \\
 & \text{std\_deviation: } \langle P(t) \rangle = \sqrt{\frac{1}{t - t_{\text{deb}}} \int_{t_{\text{deb}}}^t [P(s) - \overline{P(t)}]^2 ds} \\
 & \text{correlation: } \langle U(t).V(t) \rangle = \frac{1}{t - t_{\text{deb}}} \int_{t_{\text{deb}}}^t [U(s) - \overline{U(t)}] \cdot [V(s) - \overline{V(t)}] ds
 \end{aligned}$$

See also: [objet\\_lecture \(47\)](#)

Usage:

[ **mot** ] [ **period** ] **fields|champs**  
 where

- **mot** *str* into ['dt\_post', 'nb\_pas\_dt\_post']: Keyword to set the kind of the field's write frequency. Either a time period or a time step period.
- **period** *str*: Value of the period which can be like (2.\*t).
- **fields|champs** *list\_stat\_post* (4.2.29): Post-processed fields.

#### 4.2.29 List\_stat\_post

Description: Post-processing for statistics

See also: [listobj \(46.5\)](#)

Usage:

```
{ object1 object2 }
list of stat_post_deriv (4.2.30)
```

#### 4.2.30 Stat\_post\_deriv

Description: not\_set

See also: objet\_lecture (47) t\_deb (4.2.31) t\_fin (4.2.32) moyenne (4.2.33) ecart\_type (4.2.34) correlation (4.2.35)

Usage:

**stat\_post\_deriv**

#### 4.2.31 T\_deb

Description: Start of integration time

See also: stat\_post\_deriv (4.2.30)

Usage:

**t\_deb val**

where

- **val** *float*

#### 4.2.32 T\_fin

Description: End of integration time

See also: stat\_post\_deriv (4.2.30)

Usage:

**t\_fin val**

where

- **val** *float*

#### 4.2.33 Moyenne

Synonymous: **champ\_post\_statistiques\_moyenne**

Description: to calculate the average of the field over time

See also: stat\_post\_deriv (4.2.30)

Usage:

**moyenne field [ localisation ]**

where

- **field** *str*: name of the field on which statistical analysis will be performed. Possible keywords are Vitesse (velocity), Pression (pressure), Temperature, Concentration, ...
- **localisation** *str* into [*'elem'*, *'som'*, *'faces'*]: Localisation of post-processed field value

#### 4.2.34 Ecart\_type

Synonymous: **champ\_post\_statistiques\_ecart\_type**

Description: to calculate the standard deviation (statistic rms) of the field

See also: stat\_post\_deriv (4.2.30)

Usage:

**ecart\_type field [ localisation ]**

where

- **field** *str*: name of the field on which statistical analysis will be performed. Possible keywords are Vitesse (velocity), Pression (pressure), Temperature, Concentration, ...
- **localisation** *str* into [*'elem'*, *'som'*, *'faces'*]: Localisation of post-processed field value

#### 4.2.35 Correlation

Synonymous: **champ\_post\_statistiques\_correlation**

Description: correlation between the two fields

See also: stat\_post\_deriv (4.2.30)

Usage:

**correlation first\_field second\_field [ localisation ]**

where

- **first\_field** *str*: first field
- **second\_field** *str*: second field
- **localisation** *str* into [*'elem'*, *'som'*, *'faces'*]: Localisation of post-processed field value

#### 4.2.36 Stats\_posts\_fichier

Description: Statistics read from file..

Example:

```
Statistiques Dt_post dtst {
 t_deb 0.1 t_fin 0.12
```

```
 Moyenne Pression
```

```
 Ecart_type Pression
```

```
 Correlation Vitesse Vitesse }
}
```

It will write every **dt\_post** the mean, standard deviation and correlation value:



$t \leq t_{\text{deb}}$  or  $t \geq t_{\text{fin}}$  :

average:  $\overline{P(t)} = 0$

std\_deviation:  $\langle P(t) \rangle = 0$

correlation:  $\langle U(t).V(t) \rangle = 0$

$t > t_{\text{deb}}$  and  $t < t_{\text{fin}}$  :

average:  $\overline{P(t)} = \frac{1}{t - t_{\text{deb}}} \int_{t_{\text{deb}}}^t P(s) ds$

std\_deviation:  $\langle P(t) \rangle = \sqrt{\frac{1}{t - t_{\text{deb}}} \int_{t_{\text{deb}}}^t [P(s) - \overline{P(t)}]^2 ds}$

correlation:  $\langle U(t).V(t) \rangle = \frac{1}{t - t_{\text{deb}}} \int_{t_{\text{deb}}}^t [U(s) - \overline{U(t)}] \cdot [V(s) - \overline{V(t)}] ds$

See also: [objet\\_lecture \(47\)](#)

Usage:

**mot period fichier**

where

- **mot** *str* into ['dt\_post', 'nb\_pas\_dt\_post']: Keyword to set the kind of the field's write frequency. Either a time period or a time step period.
- **period** *str*: Value of the period which can be like (2.\*t).
- **fichier** *bloc\_fichier* ([4.2.27](#)): name of file

#### 4.2.37 Stats\_serie\_posts

Description: This keyword is used to set the statistics. Average on dt\_integr time interval is post-processed every dt\_integr seconds.

Example:

```
Statistiques_en_serie Dt_integr dtst {
Moyenne Pression
}
```

Will calculate and write every dtst seconds the mean value:

$$(n + 1)dt_{\text{integr}} > t > n * dt_{\text{integr}}, \overline{P(t)} = \frac{1}{t - n * dt_{\text{integr}}} \int_{t_n * dt_{\text{integr}}}^t P(t) dt$$

See also: [objet\\_lecture \(47\)](#)

Usage:

**mot dt\_integr stat**

where

- **mot** *str* into ['dt\_integr']: Keyword is used to set the statistics period of integration and write period.
- **dt\_integr** *float*: Average on dt\_integr time interval is post-processed every dt\_integr seconds.
- **stat** *list\_stat\_post* ([4.2.29](#))

#### 4.2.38 Stats\_serie\_posts\_fichier

Description: This keyword is used to set the statistics read from a file. Average on dt\_integr time interval is post-processed every dt\_integr seconds.

Example:

```
Statistiques_en_serie Dt_integr dtst {
Moyenne Pression
}
```

Will calculate and write every dtst seconds the mean value:

$$(n + 1)dt\_integr > t > n * dt\_integr, \overline{P(t)} = \frac{1}{t - n * dt\_integr} \int_{t_n * dt\_integr}^t P(t)dt$$

See also: objet\_lecture (47)

Usage:

**mot dt\_integr fichier**  
where

- **mot** *str* into [*'dt\_integr'*]: Keyword is used to set the statistics period of integration and write period.
- **dt\_integr** *float*: Average on dt\_integr time interval is post-processed every dt\_integr seconds.
- **fichier** *bloc\_fichier* (4.2.27): name of file

### 4.3 Post\_processings

Synonymous: **postraitements**

Description: Keyword to use several results files. List of objects of post-processing (with name).

See also: listobj (46.5)

Usage:

```
{ object1 object2 }
list of un_postraitement (4.3.1)
```

#### 4.3.1 Un\_postraitement

Description: An object of post-processing (with name).

See also: objet\_lecture (47)

Usage:

**nom post**  
where

- **nom** *str*: Name of the post-processing.
- **post** *corps\_postraitement* (4.2): Definition of the post-processing.

## 4.4 Liste\_post\_ok

Description: Keyword to use several results files. List of objects of post-processing (with name)

See also: listobj ([46.5](#))

Usage:

{ object1 object2 .... }

list of *nom\_postraitement* ([4.4.1](#))

### 4.4.1 Nom\_postraitement

Description: not\_set

See also: objet\_lecture ([47](#))

Usage:

**nom post**

where

- **nom** *str*: Name of the post-processing.
- **post** *postraitement\_base* ([4.4.2](#)): the post

### 4.4.2 Postraitement\_base

Description: not\_set

See also: objet\_lecture ([47](#)) post\_processing ([4.4.3](#)) postraitement\_ft\_lata ([4.4.4](#))

Usage:

### 4.4.3 Post\_processing

Synonymous: **postraitement**

Description: An object of post-processing (without name).

See also: postraitement\_base ([4.4.2](#)) corps\_postraitement ([4.2](#))

Usage:

**post\_processing** {

[ **t\_debut\_statistiques** *float*]  
[ **nb\_pas\_dt\_post\_stats\_plans** *float*]  
[ **nb\_pas\_dt\_post\_stats\_bulles** *float*]  
[ **expression\_vx\_ana** *str*]  
[ **expression\_vy\_ana** *str*]  
[ **expression\_vz\_ana** *str*]  
[ **expression\_p\_ana** *str*]  
[ **interfaces** *interface\_posts*]  
[ **fichier** *str*]  
[ **format** *str* into [*'lml'*, *'lata'*, *'single\_lata'*, *'lata\_v2'*, *'med'*, *'cgns'*]]  
[ **dt\_post** *str*]  
[ **nb\_pas\_dt\_post** *int*]  
[ **domaine** *str*]

```

[sous_zone|sous_domaine str]
[parallele str into ['simple', 'multiple', 'mpi-io']]
[definition_champs definition_champs]
[definition_champs_file|definition_champs_fichier definition_champs_fichier]
[probes|sondes sondes]
[probes_file|sondes_fichier sondes_fichier]
[mobile_probes|sondes_mobiles sondes]
[mobile_probes_file|sondes_mobiles_fichier sondes_fichier]
[deprecatedkeepduplicatedprobes int]
[fields|champs champs_posts]
[fields_file|champs_fichier champs_posts_fichier]
[statistics|statistiques stats_posts]
[statistics_file|statistiques_fichier stats_posts_fichier]
[serial_statistics|statistiques_en_serie stats_serie_posts]
[serial_statistics_file|statistiques_en_serie_fichier stats_serie_posts_fichier]
[suffix_for_reset str]
}
where

```

- **t\_debut\_statistiques** *float*: not\_set (for IJK)
- **nb\_pas\_dt\_post\_stats\_plans** *float*: not\_set (for IJK)
- **nb\_pas\_dt\_post\_stats\_bulles** *float*: not\_set (for IJK)
- **expression\_vx\_ana** *str*: not\_set (for IJK)
- **expression\_vy\_ana** *str*: not\_set (for IJK)
- **expression\_vz\_ana** *str*: not\_set (for IJK)
- **expression\_p\_ana** *str*: not\_set (for IJK)
- **interfaces** *interface\_posts* (4.2.1): Keyword to read all the characteristics of the interfaces. Different kind of interfaces exist as well as different interface initialisations.
- **fichier** *str*: Name of file.
- **format** *str into* ['lml', 'lata', 'single\_lata', 'lata\_v2', 'med', 'cgns']: This optional parameter specifies the format of the output file. The basename used for the output file is the basename of the data file. For the fmt parameter, choices are lml or lata. A short description of each format can be found below. The default value is lml.
- **dt\_post** *str*: Field's write frequency (as a time period) - can also be specified after the 'field' keyword.
- **nb\_pas\_dt\_post** *int*: Field's write frequency (as a number of time steps) - can also be specified after the 'field' keyword.
- **domaine** *str*: This optional parameter specifies the domain on which the data should be interpolated before it is written in the output file. The default is to write the data on the domain of the current problem (no interpolation).
- **sous\_zone|sous\_domaine** *str*: This optional parameter specifies the sub\_domaine on which the data should be interpolated before it is written in the output file. It is only available for sequential computation.
- **parallele** *str into* ['simple', 'multiple', 'mpi-io']: Select simple (single file, sequential write), multiple (several files, parallel write), or mpi-io (single file, parallel write) for LATA format
- **definition\_champs** *definition\_champs* (4.2.4): Keyword to create new or more complex field for advanced postprocessing.
- **definition\_champs\_file|definition\_champs\_fichier** *definition\_champs\_fichier* (4.2.6): Definition\_champs read from file.
- **probes|sondes** *sondes* (4.2.7): Probe.
- **probes\_file|sondes\_fichier** *sondes\_fichier* (4.2.24): Probe read from a file.
- **mobile\_probes|sondes\_mobiles** *sondes* (4.2.7): Mobile probes useful for ALE, their positions will be updated in the mesh.
- **mobile\_probes\_file|sondes\_mobiles\_fichier** *sondes\_fichier* (4.2.24): Mobile probes read in a file

- **deprecatedkeepduplicatedprobes** *int*: Flag to not remove duplicated probes in .son files (1: keep duplicate probes, 0: remove duplicate probes)
- **fields|champs** *champs\_posts* (4.2.25): Field's write mode.
- **fields\_file|champs\_fichier** *champs\_posts\_fichier* (4.2.26): Fields read from file.
- **statistics|statistiques** *stats\_posts* (4.2.28): Statistics between two points fixed : start of integration time and end of integration time.
- **statistics\_file|statistiques\_fichier** *stats\_posts\_fichier* (4.2.36): Statistics read from file.
- **serial\_statistics|statistiques\_en\_serie** *stats\_serie\_posts* (4.2.37): Statistics between two points not fixed : on period of integration.
- **serial\_statistics\_file|statistiques\_en\_serie\_fichier** *stats\_serie\_posts\_fichier* (4.2.38): Serial\_statistics read from a file
- **suffix\_for\_reset** *str*: Suffix used to modify the postprocessing file name if the ICoCo resetTime() method is invoked.

#### 4.4.4 Postraitement\_ft\_lata

Description: not\_set

See also: postraitement\_base (4.4.2)

Usage:

**postraitement\_ft\_lata bloc**  
where

- **bloc** *str*

### 4.5 Liste\_post

Description: Keyword to use several results files. List of objects of post-processing (with name)

See also: listobj (46.5)

Usage:

{ object1 object2 .... }  
list of *un\_postraitement\_spec* (4.5.1)

#### 4.5.1 Un\_postraitement\_spec

Description: An object of post-processing (with type +name).

See also: objet\_lecture (47)

Usage:

[ **type\_un\_post** ] [ **type\_postraitement\_ft\_lata** ]  
where

- **type\_un\_post** *type\_un\_post* (4.5.2)
- **type\_postraitement\_ft\_lata** *type\_postraitement\_ft\_lata* (4.5.3)

#### 4.5.2 Type\_un\_post

Description: not\_set

See also: [objet\\_lecture \(47\)](#)

Usage:

**type post**  
where

- **type** *str* into ['postraitement', 'post\_processing']
- **post** *un\_postraitement* ([4.3.1](#))

#### 4.5.3 Type\_postraitement\_ft\_lata

Description: not\_set

See also: [objet\\_lecture \(47\)](#)

Usage:

**type nom bloc**  
where

- **type** *str* into ['postraitement\_ft\_lata']
- **nom** *str*: Name of the post-processing.
- **bloc** *bloc\_lecture* ([3.2](#))

### 4.6 Format\_file\_base

Description: Format of the file

See also: [objet\\_lecture \(47\)](#) [binaire \(4.6.1\)](#) [formatte \(4.6.2\)](#) [xyz \(4.6.3\)](#) [single\\_hdf \(4.6.4\)](#) [pdi \(4.6.5\)](#) [pdi-expert \(4.6.6\)](#)

Usage:

**checkpoint\_fname**  
where

- **checkpoint\_fname** *str*: Name of file.

#### 4.6.1 Binaire

Description: Format of the file - binary version

See also: ([4.6](#))

Usage:

**binaire checkpoint\_fname**  
where

- **checkpoint\_fname** *str*: Name of file.

#### 4.6.2 Formatte

Description: Format of the file - formatte version

See also: ([4.6](#))

Usage:

**formatte checkpoint\_fname**

where

- **checkpoint\_fname** *str*: Name of file.

#### 4.6.3 Xyz

Description: Format of the file - xyz version

See also: (4.6)

Usage:

**xyz checkpoint\_fname**

where

- **checkpoint\_fname** *str*: Name of file.

#### 4.6.4 Single\_hdf

Description: Format of the file - single\_hdf version

See also: (4.6)

Usage:

**single\_hdf checkpoint\_fname**

where

- **checkpoint\_fname** *str*: Name of file.

#### 4.6.5 Pdi

Description: Format of the file - pdi version

See also: (4.6)

Usage:

**pdi checkpoint\_fname**

where

- **checkpoint\_fname** *str*: Name of file.

#### 4.6.6 Pdi\_expert

Description: Format of the file - PDI expert version

See also: (4.6)

Usage:

**pdi\_expert {**

**yaml\_fname** *str*

```

 checkpoint_fname str
}
where

```

- **yaml\_fname** *str*: YAML file name
- **checkpoint\_fname** *str* for inheritance: Name of file.

## 4.7 Pb\_conduction\_ibm

Description: Resolution of the IBM heat equation.

Keyword Discretize should have already been used to read the object.

See also: Pb\_base (4.34)

Usage:

**Pb\_Conduction\_ibm** *str*

```

Read str {
 [solide solide]
 [Conduction_ibm conduction_ibm]
 [milieu milieu_base]
 [constituant constituant]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]
}
where

```

- **solide** *solide* (26.18): The medium associated with the problem.
- **Conduction\_ibm** *conduction\_ibm* (5.7): IBM Heat equation.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.



- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the *name\_file* file (see the class *format\_file*). If *format\_reprise* is xyz, the *name\_file* file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \leq P$ ) processors. Should the calculation be resumed, values for the *tinit* (see *schema\_temps\_base*) time fields are taken from the *name\_file* file. If there is no backup corresponding to this time in the *name\_file*, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the *name\_file* file, resume the calculation at the last time found in the file (*tinit* is set to last time of saved files).

## 4.8 Pb\_fronttracking\_disc

Synonymous: **probleme\_ft\_disc\_gen**

Description: The generic Front-Tracking problem in the discontinuous version. It differs from the rest of the TRUST code : The problem does not state the number of equations that are enclosed in the problem. Two equations are compulsory : a momentum balance equation (alias Navier-Stokes equation) and an interface tracking equation. The list of equations to be solved is declared in the beginning of the data file. Another difference with more classical TRUST data file, lies in the fluids definition. The two-phase fluid (*Fluide\_Diphasique*) is made with two usual single-phase fluids (*Fluide\_Incompressible*). As the list of equations to be solved in the generic Front-Tracking problem is declared in the data file and not pre-defined in the structure of the problem, each equation has to be distinctively associated with the problem with the *Associer* keyword.

Keyword *Discretize* should have already been used to read the object.

See also: *problem\_read\_generic* (4.75) *probleme\_ftd\_ijk* (4.77)

Usage:

**pb\_fronttracking\_disc** *str*

**Read** *str* {

```

 solved_equations listdeuxmots_acc
 [fluide_incompressible fluide_incompressible]
 [fluide_diphasique fluide_diphasique]
 [constituant constituant]
 [Triple_Line_Model_FT_Disc triple_line_model_ft_disc]
 [milieu milieu_base]
 [Post_processing|postraitements corps_postraitements]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]

```

}

where

- **solved\_equations** *listdeuxmots\_acc* (4.9): List of solved equations in the form 'equation\_type' 'equation\_alias'
- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem.

- **fluide\_diphasique** *fluide\_diphasique* (26.8): The diphasic fluid medium associated with the problem.
- **constituant** *constituant* (26.5): Constituent.
- **Triple\_Line\_Model\_FT\_Disc** *triple\_line\_model\_ft\_disc* (9)
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processing|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N < P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.9 Listdeuxmots\_acc

Description: List of groups of two words (with curly brackets).

See also: listobj (46.5)

Usage:

```
{ object1 object2 }
```

list of *deuxmots* (4.9.1)

### 4.9.1 Deuxmots

Description: Two words.

See also: objet\_lecture (47)

Usage:

```
mot_1 mot_2
```

where

- **mot\_1** *str*: First word.
- **mot\_2** *str*: Second word.

## 4.10 Pb\_hydraulique\_cloned\_concentration

Description: Resolution of Navier-Stokes/multiple constituent transport equations.

Keyword Discretize should have already been used to read the object.

See also: Pb\_base (4.34)

Usage:

**Pb\_Hydraulique\_Cloned\_Concentration** *str*

**Read** *str* {

```
 fluide_incompressible fluide_incompressible
 [constituant constituant]
 [navier_stokes_standard navier_stokes_standard]
 [convection_diffusion_concentration convection_diffusion_concentration]
 [milieu milieu_base]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]
```

}

where

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem.
- **constituant** *constituant* (26.5): Constituents.
- **navier\_stokes\_standard** *navier\_stokes\_standard* (5.57): Navier-Stokes equations.
- **convection\_diffusion\_concentration** *convection\_diffusion\_concentration* (5.30): Constituent transport vectorial equation (concentration diffusion convection).
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N (N<>P) processors. Should the

calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.

- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.11 Pb\_hydraulique\_cloned\_concentration\_turbulent

Description: Resolution of Navier-Stokes/multiple constituent transport equations, with turbulence modelling.

Keyword Discretize should have already been used to read the object.

See also: Pb\_base (4.34)

Usage:

**Pb\_Hydraulique\_Cloned\_Concentration\_Turbulent** *str*

**Read** *str* {

```

 fluide_incompressible fluide_incompressible
 [constituant constituant]
 [navier_stokes_turbulent navier_stokes_turbulent]
 [convection_diffusion_concentration_turbulent convection_diffusion_concentration_turbulent]
 [milieu milieu_base]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]

```

}

where

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem.
- **constituant** *constituant* (26.5): Constituents.
- **navier\_stokes\_turbulent** *navier\_stokes\_turbulent* (5.58): Navier-Stokes equations as well as the associated turbulence model equations.
- **convection\_diffusion\_concentration\_turbulent** *convection\_diffusion\_concentration\_turbulent* (5.32): Constituent transport equations (concentration diffusion convection) as well as the associated turbulence model equations.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.

- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \leq P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.12 Pb\_hydraulique\_ibm\_turbulent

Description: Resolution of Navier-Stokes equations with turbulence modelling.

Keyword Discretize should have already been used to read the object.

See also: Pb\_base (4.34)

Usage:

**Pb\_Hydraulique\_IBM\_Turbulent** *str*

**Read** *str* {

```

 fluide_incompressible fluide_incompressible
 navier_stokes_ibm_turbulent navier_stokes_ibm_turbulent
 [milieu milieu_base]
 [constituant constituant]
 [Post_processing|postraitemment corps_postraitemment]
 [Post_processings|postraitemments post_processings]
 [liste_de_postraitemments liste_post_ok]
 [liste_postraitemments liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]

```

}

where

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem.
- **navier\_stokes\_ibm\_turbulent** *navier\_stokes\_ibm\_turbulent* (5.53): IBM Navier-Stokes equations as well as the associated turbulence model equations.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitemment** *corps\_postraitemment* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitemments** *post\_processings* (4.3) for inheritance: List of Postraitemment objects (with name).

- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N < P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

### 4.13 Pb\_hydraulique\_list\_concentration

Description: Resolution of Navier-Stokes/multiple constituent transport equations.

Keyword Discretize should have already been used to read the object.

See also: pb\_avec\_liste\_conc (4.40)

Usage:

**Pb\_Hydraulique\_List\_Concentration** *str*

```
Read str {
 fluide_incompressible fluide_incompressible
 [constituant constituant]
 [navier_stokes_standard navier_stokes_standard]
 list_equations listeqn
 [milieu milieu_base]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]
}
```

where

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem.
- **constituant** *constituant* (26.5): Constituents.

- **navier\_stokes\_standard** *navier\_stokes\_standard* (5.57): Navier-Stokes equations.
- **list\_equations** *listeqn* (4.14) for inheritance: convection\_diffusion\_concentration equations. The unknown of the concentration equation number N is named concentrationN. This keyword is used to define initial conditions and the post processing fields. This kind of problem is very useful to test in only one data file (and then only one calculation) different schemes or different boundary conditions for the scalar transport equation.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processing|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N < P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

#### 4.14 Listeqn

Description: List of equations.

See also: listobj (46.5)

Usage:

```
{ object1 object2 }
```

list of *eqn\_base* (5.45)

#### 4.15 Pb\_hydraulique\_list\_concentration\_turbulent

Description: Resolution of Navier-Stokes/multiple constituent transport equations, with turbulence modelling.

Keyword Discretize should have already been used to read the object.

See also: pb\_avec\_liste\_conc (4.40)

Usage:



**Pb\_Hydraulique\_List\_Concentration\_Turbulent** *str*

**Read** *str* {

```

 fluide_incompressible fluide_incompressible
 [constituant constituant]
 [navier_stokes_turbulent navier_stokes_turbulent]
 list_equations listeqn
 [milieu milieu_base]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]

```

}

where

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem.
- **constituant** *constituant* (26.5): Constituents.
- **navier\_stokes\_turbulent** *navier\_stokes\_turbulent* (5.58): Navier-Stokes equations as well as the associated turbulence model equations.
- **list\_equations** *listeqn* (4.14) for inheritance: convection\_diffusion\_concentration equations. The unknown of the concentration equation number N is named concentrationN. This keyword is used to define initial conditions and the post processing fields. This kind of problem is very useful to test in only one data file (and then only one calculation) different schemes or different boundary conditions for the scalar transport equation.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \geq P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.



- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the *name\_file* file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.16 Pb\_hydraulique\_turbulent\_ale

Description: Resolution of hydraulic turbulent problems for ALE

Keyword Discretize should have already been used to read the object.

See also: **Pb\_base** (4.34)

Usage:

**Pb\_Hydraulique\_Turbulent\_ALE** *str*

**Read** *str* {

```

 fluide_incompressible fluide_incompressible
 Navier_Stokes_Turbulent_ALE navier_stokes_turbulent_ale
 [milieu milieu_base]
 [constituant constituant]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]

```

}

where

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem.
- **Navier\_Stokes\_Turbulent\_ALE** *navier\_stokes\_turbulent\_ale* (5.21): Navier-Stokes\_ALE equations as well as the associated turbulence model equations on mobile domain (ALE)
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.

- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the *name\_file* file (see the class *format\_file*). If *format\_reprise* is xyz, the *name\_file* file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \leq P$ ) processors. Should the calculation be resumed, values for the *tinit* (see *schema\_temps\_base*) time fields are taken from the *name\_file* file. If there is no backup corresponding to this time in the *name\_file*, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the *name\_file* file, resume the calculation at the last time found in the file (*tinit* is set to last time of saved files).

## 4.17 Pb\_hydraulique\_sensibility

Description: Resolution of hydraulic sensibility problems

Keyword Discretize should have already been used to read the object.

See also: *Pb\_base* (4.34)

Usage:

**Pb\_Hydraulique\_sensibility** *str*

**Read** *str* {

```

 fluide_incompressible fluide_incompressible
 Navier_Stokes_standard_sensibility navier_stokes_standard_sensibility
 [milieu milieu_base]
 [constituant constituant]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]

```

}

where

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem.
- **Navier\_Stokes\_standard\_sensibility** *navier\_stokes\_standard\_sensibility* (5.23): Navier-Stokes sensibility equations
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.

- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \leq P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.18 Pb\_multiphase

Description: A problem that allows the resolution of N-phases with  $3 \times N$  equations

Keyword Discretize should have already been used to read the object.

See also: Pb\_base (4.34) Pb\_Multiphase\_h (4.19) Pb\_HEM (4.20)

Usage:

**Pb\_Multiphase** *str*

**Read** *str* {

```
[milieu_composite bloc_lecture]
[Milieu_MUSIG bloc_lecture]
[correlations bloc_lecture]
[models bloc_lecture]
QDM_Multiphase qdm_multiphase
Masse_Multiphase masse_multiphase
Energie_Multiphase energie_multiphase
[Echelle_temporelle_turbulente echelle_temporelle_turbulente]
[Energie_cinetique_turbulente energie_cinetique_turbulente]
[Energie_cinetique_turbulente_WIT energie_cinetique_turbulente_wit]
[Taux_dissipation_turbulent taux_dissipation_turbulent]
[milieu milieu_base]
[constituant constituant]
[Post_processing|postraitements corps_postraitements]
[Post_processing|postraitements post_processings]
[liste_de_postraitements liste_post_ok]
[liste_postraitements liste_post]
[sauvegarde format_file_base]
[sauvegarde_simple format_file_base]
[reprise format_file_base]
[resume_last_time format_file_base]
```

}

where

- **milieu\_composite** *bloc\_lecture* (3.2): The composite medium associated with the problem.

- **Milieu\_MUSIG** *bloc\_lecture* (3.2): The composite medium associated with the problem.
- **correlations** *bloc\_lecture* (3.2): List of correlations used in specific source terms (i.e. interfacial flux, interfacial friction, ...)
- **models** *bloc\_lecture* (3.2): List of models used in specific source terms (i.e. interfacial flux, interfacial friction, ...)
- **QDM\_Multiphase** *qdm\_multiphase* (5.25): Momentum conservation equation for a multi-phase problem where the unknown is the velocity
- **Masse\_Multiphase** *masse\_multiphase* (5.16): Mass conservation equation for a multi-phase problem where the unknown is the alpha (void fraction)
- **Energie\_Multiphase** *energie\_multiphase* (5.12): Internal energy conservation equation for a multi-phase problem where the unknown is the temperature
- **Echelle\_temporelle\_turbulente** *echelle\_temporelle\_turbulente* (5.11): Turbulent Dissipation time scale equation for a turbulent mono/multi-phase problem (available in TrioCFD)
- **Energie\_cinetique\_turbulente** *energie\_cinetique\_turbulente* (5.14): Turbulent kinetic Energy conservation equation for a turbulent mono/multi-phase problem (available in TrioCFD)
- **Energie\_cinetique\_turbulente\_WIT** *energie\_cinetique\_turbulente\_wit* (5.15): Bubble Induced Turbulent kinetic Energy equation for a turbulent multi-phase problem (available in TrioCFD)
- **Taux\_dissipation\_turbulent** *taux\_dissipation\_turbulent* (5.26): Turbulent Dissipation frequency equation for a turbulent mono/multi-phase problem (available in TrioCFD)
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitemnt** *corps\_postraitemnt* (4.2) for inheritance: One post-processing (without name).
- **Post\_processing|postraitemnts** *post\_processings* (4.3) for inheritance: List of Postraitemnt objects (with name).
- **liste\_de\_postraitemnts** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitemnts** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N < P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.19 Pb\_multiphase\_h

Description: A problem that allows the resolution of N-phases with  $3*N$  equations

Keyword Discretize should have already been used to read the object.

See also: `Pb_Multiphase` (4.18)

Usage:

**Pb\_Multiphase\_h** *str*

**Read** *str* {

```
[milieu_composite bloc_lecture]
[correlations bloc_lecture]
QDM_Multiphase qdm_multiphase
Masse_Multiphase masse_multiphase
Energie_Multiphase_h energie_multiphase_h
[Milieu_MUSIG bloc_lecture]
[models bloc_lecture]
[Echelle_temporelle_turbulente echelle_temporelle_turbulente]
[Energie_cinetique_turbulente energie_cinetique_turbulente]
[Energie_cinetique_turbulente_WIT energie_cinetique_turbulente_wit]
[Taux_dissipation_turbulent taux_dissipation_turbulent]
[milieu milieu_base]
[constituant constituant]
[Post_processing|postraitement corps_postraitement]
[Post_processings|postraitements post_processings]
[liste_de_postraitements liste_post_ok]
[liste_postraitements liste_post]
[sauvegarde format_file_base]
[sauvegarde_simple format_file_base]
[reprise format_file_base]
[resume_last_time format_file_base]
```

}

where

- **milieu\_composite** *bloc\_lecture* (3.2): The composite medium associated with the problem.
- **correlations** *bloc\_lecture* (3.2): List of correlations used in specific source terms (i.e. interfacial flux, interfacial friction, ...)
- **QDM\_Multiphase** *qdm\_multiphase* (5.25): Momentum conservation equation for a multi-phase problem where the unknown is the velocity
- **Masse\_Multiphase** *masse\_multiphase* (5.16): Mass conservation equation for a multi-phase problem where the unknown is the alpha (void fraction)
- **Energie\_Multiphase\_h** *energie\_multiphase\_h* (5.13): Internal energy conservation equation for a multi-phase problem where the unknown is the enthalpy
- **Milieu\_MUSIG** *bloc\_lecture* (3.2) for inheritance: The composite medium associated with the problem.
- **models** *bloc\_lecture* (3.2) for inheritance: List of models used in specific source terms (i.e. interfacial flux, interfacial friction, ...)
- **Echelle\_temporelle\_turbulente** *echelle\_temporelle\_turbulente* (5.11) for inheritance: Turbulent Dissipation time scale equation for a turbulent mono/multi-phase problem (available in TrioCFD)
- **Energie\_cinetique\_turbulente** *energie\_cinetique\_turbulente* (5.14) for inheritance: Turbulent kinetic Energy conservation equation for a turbulent mono/multi-phase problem (available in TrioCFD)
- **Energie\_cinetique\_turbulente\_WIT** *energie\_cinetique\_turbulente\_wit* (5.15) for inheritance: Bubble Induced Turbulent kinetic Energy equation for a turbulent multi-phase problem (available in TrioCFD)
- **Taux\_dissipation\_turbulent** *taux\_dissipation\_turbulent* (5.26) for inheritance: Turbulent Dissipation frequency equation for a turbulent mono/multi-phase problem (available in TrioCFD)
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.

- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processing|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N < P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.20 Pb\_hem

Description: A problem that allows the resolution of 2-phases mechanically and thermally coupled with 3 equations

Keyword Discretize should have already been used to read the object.

See also: Pb\_Multiphase (4.18)

Usage:

**Pb\_HEM** *str*

**Read** *str* {

```
[milieu_composite bloc_lecture]
[Milieu_MUSIG bloc_lecture]
[correlations bloc_lecture]
[models bloc_lecture]
QDM_Multiphase qdm_multiphase
Masse_Multiphase masse_multiphase
Energie_Multiphase energie_multiphase
[Echelle_temporelle_turbulente echelle_temporelle_turbulente]
[Energie_cinetique_turbulente energie_cinetique_turbulente]
[Energie_cinetique_turbulente_WIT energie_cinetique_turbulente_wit]
[Taux_dissipation_turbulent taux_dissipation_turbulent]
[milieu milieu_base]
[constituant constituant]
```

```

[Post_processing|postraitement corps_postraitement]
[Post_processings|postraitements post_processings]
[liste_de_postraitements liste_post_ok]
[liste_postraitements liste_post]
[sauvegarde format_file_base]
[sauvegarde_simple format_file_base]
[reprise format_file_base]
[resume_last_time format_file_base]

```

}

where

- **milieu\_composite** *bloc\_lecture* (3.2) for inheritance: The composite medium associated with the problem.
- **Milieu\_MUSIG** *bloc\_lecture* (3.2) for inheritance: The composite medium associated with the problem.
- **correlations** *bloc\_lecture* (3.2) for inheritance: List of correlations used in specific source terms (i.e. interfacial flux, interfacial friction, ...)
- **models** *bloc\_lecture* (3.2) for inheritance: List of models used in specific source terms (i.e. interfacial flux, interfacial friction, ...)
- **QDM\_Multiphase** *qdm\_multiphase* (5.25) for inheritance: Momentum conservation equation for a multi-phase problem where the unknown is the velocity
- **Masse\_Multiphase** *masse\_multiphase* (5.16) for inheritance: Mass conservation equation for a multi-phase problem where the unknown is the alpha (void fraction)
- **Energie\_Multiphase** *energie\_multiphase* (5.12) for inheritance: Internal energy conservation equation for a multi-phase problem where the unknown is the temperature
- **Echelle\_temporelle\_turbulente** *echelle\_temporelle\_turbulente* (5.11) for inheritance: Turbulent Dissipation time scale equation for a turbulent mono/multi-phase problem (available in TrioCFD)
- **Energie\_cinetique\_turbulente** *energie\_cinetique\_turbulente* (5.14) for inheritance: Turbulent kinetic Energy conservation equation for a turbulent mono/multi-phase problem (available in TrioCFD)
- **Energie\_cinetique\_turbulente\_WIT** *energie\_cinetique\_turbulente\_wit* (5.15) for inheritance: Bubble Induced Turbulent kinetic Energy equation for a turbulent multi-phase problem (available in TrioCFD)
- **Taux\_dissipation\_turbulent** *taux\_dissipation\_turbulent* (5.26) for inheritance: Turbulent Dissipation frequency equation for a turbulent mono/multi-phase problem (available in TrioCFD)
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.



- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the *name\_file* file (see the class *format\_file*). If *format\_reprise* is xyz, the *name\_file* file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \ll P$ ) processors. Should the calculation be resumed, values for the *tinit* (see *schema\_temps\_base*) time fields are taken from the *name\_file* file. If there is no backup corresponding to this time in the *name\_file*, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the *name\_file* file, resume the calculation at the last time found in the file (*tinit* is set to last time of saved files).

## 4.21 Pb\_rayo\_conduction

Description: Resolution of the heat equation with rayonnement.

Keyword Discretize should have already been used to read the object.

See also: *Pb\_Conduction* (4.1)

Usage:

**Pb\_Rayo\_Conduction** *str*

```
Read str {
 [Conduction conduction]
 [milieu milieu_base]
 [constituant constituant]
 [Post_processing|postraitemment corps_postraitemment]
 [Post_processings|postraitemments post_processings]
 [liste_de_postraitemments liste_post_ok]
 [liste_postraitemments liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]
}
```

where

- **Conduction** *conduction* (5.1) for inheritance: Heat equation.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitemment** *corps\_postraitemment* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitemments** *post\_processings* (4.3) for inheritance: List of Postraitemment objects (with name).
- **liste\_de\_postraitemments** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitemments** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.



- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \leq P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.22 Pb\_rayo\_hydraulique

Description: Resolution of the Navier-Stokes equations with rayonnement.

Keyword Discretize should have already been used to read the object.

See also: pb\_hydraulique (4.43)

Usage:

**Pb\_Rayo\_Hydraulique** *str*

```
Read str {
 navier_stokes_standard navier_stokes_standard
 [milieu milieu_base]
 [constituant constituant]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]
}
```

where

- **navier\_stokes\_standard** *navier\_stokes\_standard* (5.57) for inheritance: Navier-Stokes equations.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified

for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.

- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \ll P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.23 Pb\_rayo\_hydraulique\_turbulent

Description: Resolution of pb\_hydraulique\_turbulent with rayonnement.

Keyword Discretize should have already been used to read the object.

See also: pb\_hydraulique\_turbulent (4.54)

Usage:

**Pb\_Rayo\_Hydraulique\_Turbulent** *str*

```
Read str {
 navier_stokes_turbulent navier_stokes_turbulent
 [milieu milieu_base]
 [constituant constituant]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]
}
```

where

- **navier\_stokes\_turbulent** *navier\_stokes\_turbulent* (5.58) for inheritance: Navier-Stokes equations as well as the associated turbulence model equations.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.

- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the *name\_file* file (see the class *format\_file*). If *format\_reprise* is xyz, the *name\_file* file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \leq P$ ) processors. Should the calculation be resumed, values for the *tinit* (see *schema\_temps\_base*) time fields are taken from the *name\_file* file. If there is no backup corresponding to this time in the *name\_file*, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the *name\_file* file, resume the calculation at the last time found in the file (*tinit* is set to last time of saved files).

## 4.24 Pb\_rayo\_thermohydraulique

Description: Resolution of *pb\_thermohydraulique* with rayonnement.

Keyword Discretize should have already been used to read the object.

See also: *pb\_thermohydraulique* (4.58)

Usage:

**Pb\_Rayo\_Thermohydraulique** *str*

**Read** *str* {

```
[fluide_ostwald fluide_ostwald]
[fluide_sodium_liquide fluide_sodium_liquide]
[fluide_sodium_gaz fluide_sodium_gaz]
[correlations bloc_lecture]
[navier_stokes_standard navier_stokes_standard]
[convection_diffusion_temperature convection_diffusion_temperature]
[milieu milieu_base]
[constituant constituant]
[Post_processing|postraitements corps_postraitements]
[Post_processings|postraitements post_processings]
[liste_de_postraitements liste_post_ok]
[liste_postraitements liste_post]
[sauvegarde format_file_base]
[sauvegarde_simple format_file_base]
[reprise format_file_base]
[resume_last_time format_file_base]
```

}

where

- **fluide\_ostwald** *fluide\_ostwald* (26.10) for inheritance: The fluid medium associated with the problem (only one possibility).
- **fluide\_sodium\_liquide** *fluide\_sodium\_liquide* (26.15) for inheritance: The fluid medium associated with the problem (only one possibility).
- **fluide\_sodium\_gaz** *fluide\_sodium\_gaz* (26.14) for inheritance: The fluid medium associated with the problem (only one possibility).

- **correlations** *bloc\_lecture* (3.2) for inheritance: List of correlations used in specific source terms (i.e. interfacial flux, interfacial friction, ...)
- **navier\_stokes\_standard** *navier\_stokes\_standard* (5.57) for inheritance: Navier-Stokes equations.
- **convection\_diffusion\_temperature** *convection\_diffusion\_temperature* (5.39) for inheritance: Energy equation (temperature diffusion convection).
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N < P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.25 Pb\_rayo\_thermohydraulique\_qc

Description: Resolution of pb\_thermohydraulique\_QC with rayonnement.

Keyword Discretize should have already been used to read the object.

See also: pb\_thermohydraulique\_QC (4.59)

Usage:

**Pb\_Rayo\_Thermohydraulique\_QC** *str*

**Read** *str* {

```

 navier_stokes_QC navier_stokes_qc
 convection_diffusion_chaleur_QC convection_diffusion_chaleur_qc
 [milieu milieu_base]
 [constituant constituant]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]

```

```

[sauvegarde format_file_base]
[sauvegarde_simple format_file_base]
[reprise format_file_base]
[resume_last_time format_file_base]
}
where

```

- **navier\_stokes\_QC** *navier\_stokes\_qc* (5.48) for inheritance: Navier-Stokes equation for a quasi-compressible fluid.
- **convection\_diffusion\_chaleur\_QC** *convection\_diffusion\_chaleur\_qc* (5.27) for inheritance: Temperature equation for a quasi-compressible fluid.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitemnt** *corps\_postraitemnt* (4.2) for inheritance: One post-processing (without name).
- **Post\_processing|postraitemnts** *post\_processings* (4.3) for inheritance: List of Postraitemnt objects (with name).
- **liste\_de\_postraitemnts** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitemnts** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N < P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.26 Pb\_rayo\_thermohydraulique\_turbulent

Description: Resolution of pb\_thermohydraulique\_turbulent with rayonnement.

Keyword Discretize should have already been used to read the object.

See also: pb\_thermohydraulique\_turbulent (4.70)

Usage:

**Pb\_Rayo\_Thermohydraulique\_Turbulent** *str*

**Read** *str* {

```

 navier_stokes_turbulent navier_stokes_turbulent
 convection_diffusion_temperature_turbulent convection_diffusion_temperature_turbulent

```

```

[milieu milieu_base]
[constituant constituant]
[Post_processing|postraitement corps_postraitement]
[Post_processings|postraitements post_processings]
[liste_de_postraitements liste_post_ok]
[liste_postraitements liste_post]
[sauvegarde format_file_base]
[sauvegarde_simple format_file_base]
[reprise format_file_base]
[resume_last_time format_file_base]
}
where

```

- **navier\_stokes\_turbulent** *navier\_stokes\_turbulent* (5.58) for inheritance: Navier-Stokes equations as well as the associated turbulence model equations.
- **convection\_diffusion\_temperature\_turbulent** *convection\_diffusion\_temperature\_turbulent* (5.44) for inheritance: Energy equation (temperature diffusion convection) as well as the associated turbulence model equations.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \leq P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.27 Pb\_rayo\_thermohydraulique\_turbulent\_qc

Description: Resolution of pb\_thermohydraulique\_turbulent\_qc with rayonnement.

Keyword Discretize should have already been used to read the object.

See also: `pb_thermohydraulique_turbulent_qc` (4.71)

Usage:

**Pb\_Rayo\_Thermohydraulique\_Turbulent\_QC** *str*

**Read** *str* {

```
 navier_stokes_turbulent_qc navier_stokes_turbulent_qc
 convection_diffusion_chaleur_turbulent_qc convection_diffusion_chaleur_turbulent_qc
 [milieu milieu_base]
 [constituant constituant]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]
```

}

where

- **navier\_stokes\_turbulent\_qc** *navier\_stokes\_turbulent\_qc* (5.59) for inheritance: Navier-Stokes equations under low Mach number as well as the associated turbulence model equations.
- **convection\_diffusion\_chaleur\_turbulent\_qc** *convection\_diffusion\_chaleur\_turbulent\_qc* (5.29) for inheritance: Energy equation under low Mach number as well as the associated turbulence model equations.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N < P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).



## 4.28 Pb\_thermohydraulique\_cloned\_concentration

Description: Resolution of Navier-Stokes/energy/multiple constituent transport equations.

Keyword Discretize should have already been used to read the object.

See also: Pb\_base (4.34)

Usage:

**Pb\_Thermohydraulique\_Cloned\_Concentration** *str*

**Read** *str* {

```
 fluide_incompressible fluide_incompressible
 [constituant constituant]
 [navier_stokes_standard navier_stokes_standard]
 [convection_diffusion_concentration convection_diffusion_concentration]
 [convection_diffusion_temperature convection_diffusion_temperature]
 [milieu milieu_base]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]
```

}

where

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem.
- **constituant** *constituant* (26.5): Constituents.
- **navier\_stokes\_standard** *navier\_stokes\_standard* (5.57): Navier-Stokes equations.
- **convection\_diffusion\_concentration** *convection\_diffusion\_concentration* (5.30): Constituent transport equations (concentration diffusion convection).
- **convection\_diffusion\_temperature** *convection\_diffusion\_temperature* (5.39): Energy equation (temperature diffusion convection).
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz



file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \leq P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.

- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.29 Pb\_thermohydraulique\_cloned\_concentration\_turbulent

Description: Resolution of Navier-Stokes/energy/multiple constituent transport equations, with turbulence modelling.

Keyword Discretize should have already been used to read the object.

See also: Pb\_base (4.34)

Usage:

**Pb\_Thermohydraulique\_Cloned\_Concentration\_Turbulent** *str*

**Read** *str* {

```

 fluide_incompressible fluide_incompressible
 [constituant constituant]
 [navier_stokes_turbulent navier_stokes_turbulent]
 [convection_diffusion_concentration_turbulent convection_diffusion_concentration_turbulent]
 [convection_diffusion_temperature_turbulent convection_diffusion_temperature_turbulent]
 [milieu milieu_base]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]

```

}

where

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem.
- **constituant** *constituant* (26.5): Constituents.
- **navier\_stokes\_turbulent** *navier\_stokes\_turbulent* (5.58): Navier-Stokes equations as well as the associated turbulence model equations.
- **convection\_diffusion\_concentration\_turbulent** *convection\_diffusion\_concentration\_turbulent* (5.32): Constituent transport equations (concentration diffusion convection) as well as the associated turbulence model equations.
- **convection\_diffusion\_temperature\_turbulent** *convection\_diffusion\_temperature\_turbulent* (5.44): Energy equation (temperature diffusion convection) as well as the associated turbulence model equations.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).

- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \leq P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

### 4.30 Pb\_thermohydraulique\_ibm\_turbulent

Description: Resolution of thermohydraulic problem, with turbulence modelling.

Keyword Discretize should have already been used to read the object.

See also: Pb\_base (4.34)

Usage:

**Pb\_Thermohydraulique\_IBM\_Turbulent** *str*

**Read** *str* {

```

 fluide_incompressible fluide_incompressible
 navier_stokes_ibm_turbulent navier_stokes_ibm_turbulent
 convection_diffusion_temperature_ibm_turbulent convection_diffusion_temperature_ibm_turbulent
 [milieu milieu_base]
 [constituant constituant]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]

```

}

where

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem.

- **navier\_stokes\_ibm\_turbulent** *navier\_stokes\_ibm\_turbulent* (5.53): IBM Navier-Stokes equations as well as the associated turbulence model equations.
- **convection\_diffusion\_temperature\_ibm\_turbulent** *convection\_diffusion\_temperature\_ibm\_turbulent* (5.43): Energy equation (temperature diffusion convection) as well as the associated turbulence model equations.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N < P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

### 4.31 Pb\_thermohydraulique\_list\_concentration

Description: Resolution of Navier-Stokes/energy/multiple constituent transport equations.

Keyword Discretize should have already been used to read the object.

See also: pb\_avec\_liste\_conc (4.40)

Usage:

**Pb\_Thermohydraulique\_List\_Concentration** *str*

**Read** *str* {

```

 fluide_incompressible fluide_incompressible
 [constituant constituant]
 [navier_stokes_standard navier_stokes_standard]
 [convection_diffusion_temperature convection_diffusion_temperature]
 list_equations listeqn
 [milieu milieu_base]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]

```

```

[liste_de_postraitements liste_post_ok]
[liste_postraitements liste_post]
[sauvegarde format_file_base]
[sauvegarde_simple format_file_base]
[reprise format_file_base]
[resume_last_time format_file_base]
}
where

```

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem.
- **constituant** *constituant* (26.5): Constituents.
- **navier\_stokes\_standard** *navier\_stokes\_standard* (5.57): Navier-Stokes equations.
- **convection\_diffusion\_temperature** *convection\_diffusion\_temperature* (5.39): Energy equation (temperature diffusion convection).
- **list\_equations** *listeqn* (4.14) for inheritance: convection\_diffusion\_concentration equations. The unknown of the concentration equation number N is named concentrationN. This keyword is used to define initial conditions and the post processing fields. This kind of problem is very useful to test in only one data file (and then only one calculation) different schemes or different boundary conditions for the scalar transport equation.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \leq P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

### 4.32 Pb\_thermohydraulique\_list\_concentration\_turbulent

Description: Resolution of Navier-Stokes/energy/multiple constituent transport equations, with turbulence modelling.

Keyword Discretize should have already been used to read the object.  
See also: `pb_avec_liste_conc` (4.40)

Usage:

**Pb\_Thermohydraulique\_List\_Concentration\_Turbulent** *str*

**Read** *str* {

```

 fluide_incompressible fluide_incompressible
 [constituant constituant]
 [navier_stokes_turbulent navier_stokes_turbulent]
 [convection_diffusion_temperature_turbulent convection_diffusion_temperature_turbulent]
 list_equations listeqn
 [milieu milieu_base]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]

```

}

where

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem.
- **constituant** *constituant* (26.5): Constituents.
- **navier\_stokes\_turbulent** *navier\_stokes\_turbulent* (5.58): Navier-Stokes equations as well as the associated turbulence model equations.
- **convection\_diffusion\_temperature\_turbulent** *convection\_diffusion\_temperature\_turbulent* (5.44): Energy equation (temperature diffusion convection) as well as the associated turbulence model equations.
- **list\_equations** *listeqn* (4.14) for inheritance: convection\_diffusion\_concentration equations. The unknown of the concentration equation number N is named concentrationN. This keyword is used to define initial conditions and the post processing fields. This kind of problem is very useful to test in only one data file (and then only one calculation) different schemes or different boundary conditions for the scalar transport equation.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.

- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \leq P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

### 4.33 Pb\_thermohydraulique\_sensibility

Description: Resolution of Resolution of thermohydraulic sensitivity problem

Keyword Discretize should have already been used to read the object.

See also: pb\_thermohydraulique (4.58)

Usage:

**Pb\_Thermohydraulique\_sensibility** *str*

**Read** *str* {

```

 fluide_incompressible fluide_incompressible
 Convection_Diffusion_Temperature_Sensibility convection_diffusion_temperature_sensibility
 Navier_Stokes_standard_sensibility navier_stokes_standard_sensibility
 [fluide_ostwald fluide_ostwald]
 [fluide_sodium_liquide fluide_sodium_liquide]
 [fluide_sodium_gaz fluide_sodium_gaz]
 [correlations bloc_lecture]
 [navier_stokes_standard navier_stokes_standard]
 [milieu milieu_base]
 [constituant constituant]
 [Post_processing|postraitemment corps_postraitemment]
 [Post_processings|postraitemments post_processings]
 [liste_de_postraitemments liste_post_ok]
 [liste_postraitemments liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]

```

}

where

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem.
- **Convection\_Diffusion\_Temperature\_Sensibility** *convection\_diffusion\_temperature\_sensibility* (5.10): Convection diffusion temperature sensitivity equation
- **Navier\_Stokes\_standard\_sensibility** *navier\_stokes\_standard\_sensibility* (5.23): Navier Stokes sensitivity equation
- **fluide\_ostwald** *fluide\_ostwald* (26.10) for inheritance: The fluid medium associated with the problem (only one possibility).

- **fluide\_sodium\_liquide** *fluide\_sodium\_liquide* (26.15) for inheritance: The fluid medium associated with the problem (only one possibility).
- **fluide\_sodium\_gaz** *fluide\_sodium\_gaz* (26.14) for inheritance: The fluid medium associated with the problem (only one possibility).
- **correlations** *bloc\_lecture* (3.2) for inheritance: List of correlations used in specific source terms (i.e. interfacial flux, interfacial friction, ...)
- **navier\_stokes\_standard** *navier\_stokes\_standard* (5.57) for inheritance: Navier-Stokes equations.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitemnt** *corps\_postraitemnt* (4.2) for inheritance: One post-processing (without name).
- **Post\_processing|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N < P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

### 4.34 Pb\_base

Description: Resolution of equations on a domain. A problem is defined by creating an object and assigning the problem type that the user wishes to resolve. To enter values for the problem objects created, the Lire (Read) interpreter is used with a data block.

Keyword Discretize should have already been used to read the object.

See also: *pb\_gen\_base* (4) *Pb\_Conduction* (4.1) *Pb\_Conduction\_ibm* (4.7) *problem\_read\_generic* (4.75) *pb\_post* (4.57) *pb\_thermohydraulique\_ibm* (4.68) *Pb\_Thermohydraulique\_IBM\_Turbulent* (4.30) *pb\_hydraulique\_ibm* (4.50) *Pb\_Hydraulique\_IBM\_Turbulent* (4.12) *pb\_avec\_passif* (4.41) *pb\_thermohydraulique\_concentration* (4.61) *pb\_hydraulique\_concentration* (4.46) *Pb\_Thermohydraulique\_Cloned\_Concentration* (4.28) *Pb\_Hydraulique\_Cloned\_Concentration* (4.10) *pb\_thermohydraulique* (4.58) *pb\_hydraulique* (4.43) *pb\_avec\_liste\_conc* (4.40) *pb\_hydraulique\_melange\_binaire\_WC* (4.52) *pb\_thermohydraulique\_WC* (4.60) *pb\_hydraulique\_melange\_binaire\_QC* (4.51) *pb\_thermohydraulique\_QC* (4.59) *Pb\_Multiphase* (4.18) *pb\_thermohydraulique\_turbulent\_qc* (4.71) *pb\_hydraulique\_concentration\_turbulent* (4.48) *Pb\_Thermohydraulique\_Cloned\_Concentration\_Turbulent* (4.29) *Pb\_Hydraulique\_Cloned\_Concentration\_Turbulent* (4.11) *pb\_thermohydraulique\_concentration\_turbulent* (4.63) *pb\_hydraulique\_melange\_binaire\_turbulent\_qc* (4.53) *pb\_thermohydraulique\_turbulent*



(4.70) `pb_hydraulique_turbulent` (4.54) `modele_rayo_semi_transp` (4.38) `pb_phase_field` (4.56) `pb_hydraulique-_aposteriori` (4.45) `Pb_Hydraulique_Turbulent_ALE` (4.16) `pb_hydraulique_ALE` (4.44) `Pb_Hydraulique-_sensitivity` (4.17)

Usage:

**Pb\_base** *str*

**Read** *str* {

```
[milieu milieu_base]
[constituant constituant]
[Post_processing|postraitement corps_postraitement]
[Post_processings|postraitements post_processings]
[liste_de_postraitements liste_post_ok]
[liste_postraitements liste_post]
[sauvegarde format_file_base]
[sauvegarde_simple format_file_base]
[reprise format_file_base]
[resume_last_time format_file_base]
```

}

where

- **milieu** *milieu\_base* (26): The medium associated with the problem.
- **constituant** *constituant* (26.5): Constituent.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2): One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3): List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4): This
- **liste\_postraitements** *liste\_post* (4.5): This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6): Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6): The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6): Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N < P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6): Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

### 4.35 Probleme\_ftd\_ijk\_cut\_cell

Description: not\_set

Keyword Discretize should have already been used to read the object.



See also: `probleme_ftd_ijk` (4.77)

Usage:

**Probleme\_FTD\_IJK\_cut\_cell** *str*

```
Read str {
 [Fluide_Diphasique_IJK fluide_diphasique_ijk]
 [nom_sauvegarde str]
 [sauvegarder_xyz]
 [nom_reprise str]
 solved_equations listdeuxmots_acc
 [fluide_incompressible fluide_incompressible]
 [fluide_diphasique fluide_diphasique]
 [constituant constituant]
 [Triple_Line_Model_FT_Disc triple_line_model_ft_disc]
 [milieu milieu_base]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]
}
```

where

- **Fluide\_Diphasique\_IJK** *fluide\_diphasique\_ijk* (26.1) for inheritance: The diphasic fluid medium associated with the problem.
- **nom\_sauvegarde** *str* for inheritance: Definition of filename to save the calculation
- **sauvegarder\_xyz** for inheritance: save in xyz format
- **nom\_reprise** *str* for inheritance: Enable restart from filename given
- **solved\_equations** *listdeuxmots\_acc* (4.9) for inheritance: List of solved equations in the form 'equation\_type' 'equation\_alias'
- **fluide\_incompressible** *fluide\_incompressible* (26.9) for inheritance: The fluid medium associated with the problem.
- **fluide\_diphasique** *fluide\_diphasique* (26.8) for inheritance: The diphasic fluid medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Triple\_Line\_Model\_FT\_Disc** *triple\_line\_model\_ft\_disc* (9) for inheritance
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified

for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.

- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \leq P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

### 4.36 Probleme\_couple

Description: This instruction causes a probleme\_couple type object to be created. This type of object has an associated problem list, that is, the coupling of n problems among them may be processed. Coupling between these problems is carried out explicitly via conditions at particular contact limits. Each problem may be associated either with the Associate keyword or with the Read/groupes keywords. The difference is that in the first case, the four problems exchange values then calculate their timestep, rather in the second case, the same strategy is used for all the problems listed inside one group, but the second group of problem exchange values with the first group of problems after the first group did its timestep. So, the first case may then also be written like this:

Probleme\_Couple pbc

Read pbc { groupes { { pb1 , pb2 , pb3 , pb4 } } }

There is a physical environment per problem (however, the same physical environment could be common to several problems).

Each problem is resolved in a domain.

Warning : Presently, coupling requires coincident meshes. In case of non-coincident meshes, boundary condition 'paroi\_contact' in VEF returns error message (see paroi\_contact for correcting procedure).

See also: pb\_gen\_base (4) pb\_couple\_rayonnement (4.76) pb\_couple\_rayo\_semi\_transp (4.42)

Usage:

**probleme\_couple** *str*

**Read** *str* {

[ **groupes** *list\_list\_nom*]

}

where

- **groupes** *list\_list\_nom* (4.37): { groupes { { pb1 , pb2 } , { pb3 , pb4 } } }

### 4.37 List\_list\_nom

Description: pour les groupes

See also: listobj (46.5)

Usage:

{ object1 , object2 .... }

list of *list\_un\_pb* (46.3) separated with ,

## 4.38 Modele\_rayo\_semi\_transp

Description: Radiation model for semi transparent gas. The model should be associated to the coupling problem BEFORE the time scheme.

Keyword Discretize should have already been used to read the object.

See also: Pb\_base (4.34)

Usage:

**modele\_rayo\_semi\_transp** *str*

**Read** *str* {

```
[eq_rayo_semi_transp eq_rayo_semi_transp]
[milieu milieu_base]
[constituant constituant]
[Post_processing|postraitement corps_postraitement]
[Post_processings|postraitements post_processings]
[liste_de_postraitements liste_post_ok]
[liste_postraitements liste_post]
[sauvegarde format_file_base]
[sauvegarde_simple format_file_base]
[reprise format_file_base]
[resume_last_time format_file_base]
```

}

where

- **eq\_rayo\_semi\_transp** *eq\_rayo\_semi\_transp* (4.39): Irradiancy G equation. Radiative flux equals  $-\text{grad}(G)/3/kappa$ .
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \geq P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.

- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the *name\_file* file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

### 4.39 Eq\_rayo\_semi\_transp

Description: Irradiancy equation.

See also: *objet\_lecture* (47)

Usage:

```
{
 solveur solveur_sys_base
 [boundary_conditions|conditions_limites condlims]
}
```

where

- **solveur** *solveur\_sys\_base* (14.19): Solver of the irradiancy equation.
- **boundary\_conditions**|**conditions\_limites** *condlims* (4.39.1): Boundary conditions.

#### 4.39.1 Condlims

Description: Boundary conditions.

See also: *listobj* (46.5)

Usage:

```
{ object1 object2 }
list of condlimlu (4.39.2)
```

#### 4.39.2 Condlimlu

Description: Boundary condition specified.

See also: *objet\_lecture* (47)

Usage:

```
bord cl
where
```

- **bord** *str*: Name of the edge where the boundary condition applies.
- **cl** *condlim\_base* (17): Boundary condition at the boundary called *bord* (edge).

### 4.40 Pb\_avec\_liste\_conc

Description: Class to create a classical problem with a list of scalar concentration equations.

Keyword Discretize should have already been used to read the object.

See also: *Pb\_base* (4.34) *Pb\_Thermohydraulique\_List\_Concentration* (4.31) *Pb\_Hydraulique\_List\_Concentration* (4.13) *Pb\_Thermohydraulique\_List\_Concentration\_Turbulent* (4.32) *Pb\_Hydraulique\_List\_Concentration-Turbulent* (4.15)

Usage:

**pb\_avec\_liste\_conc** *str*

**Read** *str* {

```
 list_equations listeqn
 [milieu milieu_base]
 [constituant constituant]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]
```

}

where

- **list\_equations** *listeqn* (4.14): convection\_diffusion\_concentration equations. The unknown of the concentration equation number N is named concentrationN. This keyword is used to define initial conditions and the post processing fields. This kind of problem is very useful to test in only one data file (and then only one calculation) different schemes or different boundary conditions for the scalar transport equation.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \geq P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.41 Pb\_avec\_passif

Description: Class to create a classical problem with a scalar transport equation (e.g: temperature or concentration) and an additional set of passive scalars (e.g: temperature or concentration) equations.

Keyword Discretize should have already been used to read the object.

See also: Pb\_base (4.34) pb\_thermohydraulique\_concentration\_scalaires\_passifs (4.62) pb\_thermohydraulique\_scalaires\_passifs (4.69) pb\_hydraulique\_concentration\_scalaires\_passifs (4.47) pb\_thermohydraulique\_especes\_QC (4.65) pb\_thermohydraulique\_especes\_WC (4.66) pb\_thermohydraulique\_turbulent\_scalaires\_passifs (4.72) pb\_thermohydraulique\_especes\_turbulent\_qc (4.67) pb\_hydraulique\_concentration\_turbulent\_scalaires\_passifs (4.49) pb\_thermohydraulique\_concentration\_turbulent\_scalaires\_passifs (4.64)

Usage:

**pb\_avec\_passif** *str*

```
Read str {
 equations_scalaires_passifs listeqn
 [milieu milieu_base]
 [constituant constituant]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]
}
```

where

- **equations\_scalaires\_passifs** *listeqn* (4.14): Passive scalar equations. The unknowns of the passive scalar equation number N are named temperatureN or concentrationN or fraction\_massiqueN. This keyword is used to define initial conditions and the post processing fields. This kind of problem is very useful to test in only one data file (and then only one calculation) different schemes or different boundary conditions for the scalar transport equation.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz

file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \ll P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.

- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.42 Pb\_couple\_rayo\_semi\_transp

Description: Problem coupling several other problems to which radiation coupling is added (for semi transparent gas).

You have to associate a modele\_rayo\_semi\_transp

You have to add a radiative term source in energy equation

Warning: Calculation with semi transparent gas model may lead to divergence when high temperature differences are used. Indeed, the calculation of the stability time step of the equation does not take in account the source term. In semi transparent gas model, energy equation source term depends strongly of temperature via irradiance and stability is not guaranteed by the calculated time step. Reducing the facsec of the time scheme is a good tip to reach convergence when divergence is encountered.

See also: probleme\_couple (4.36)

Usage:

**pb\_couple\_rayo\_semi\_transp** *str*

**Read** *str* {

[ **groupes** *list\_list\_nom*]

}

where

- **groupes** *list\_list\_nom* (4.37) for inheritance: { groupes { { pb1 , pb2 } , { pb3 , pb4 } } }

## 4.43 Pb\_hydraulique

Description: Resolution of the Navier-Stokes equations.

Keyword Discretize should have already been used to read the object.

See also: Pb\_base (4.34) Pb\_Rayo\_Hydraulique (4.22)

Usage:

**pb\_hydraulique** *str*

**Read** *str* {

**fluide\_incompressible** *fluide\_incompressible*  
**navier\_stokes\_standard** *navier\_stokes\_standard*  
[ **milieu** *milieu\_base*]  
[ **constituant** *constituant*]  
[ **Post\_processing|postraitement** *corps\_postraitement*]  
[ **Post\_processings|postraitements** *post\_processings*]  
[ **liste\_de\_postraitements** *liste\_post\_ok*]  
[ **liste\_postraitements** *liste\_post*]  
[ **sauvegarde** *format\_file\_base*]

```

[sauvegarde_simple format_file_base]
[reprise format_file_base]
[resume_last_time format_file_base]
}
where

```

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem.
- **navier\_stokes\_standard** *navier\_stokes\_standard* (5.57): Navier-Stokes equations.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitemnt** *corps\_postraitemnt* (4.2) for inheritance: One post-processing (without name).
- **Post\_processing|postraitemnts** *post\_processings* (4.3) for inheritance: List of Postraitemnt objects (with name).
- **liste\_de\_postraitemnts** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitemnts** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N < P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

#### 4.44 Pb\_hydraulique\_ale

Description: Resolution of hydraulic problems for ALE

Keyword Discretize should have already been used to read the object.

See also: Pb\_base (4.34)

Usage:

**pb\_hydraulique\_ALE** *str*

**Read** *str* {

```

 fluide_incompressible fluide_incompressible
 navier_stokes_standard_ALE navier_stokes_standard
 [milieu milieu_base]
 [constituant constituant]

```



```

[Post_processing|postraitement corps_postraitement]
[Post_processings|postraitements post_processings]
[liste_de_postraitements liste_post_ok]
[liste_postraitements liste_post]
[sauvegarde format_file_base]
[sauvegarde_simple format_file_base]
[reprise format_file_base]
[resume_last_time format_file_base]
}
where

```

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem.
- **navier\_stokes\_standard\_ALE** *navier\_stokes\_standard* (5.57): Navier-Stokes equations for ALE problems
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N < P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

#### 4.45 Pb\_hydraulique\_aposteriori

Description: Modification of the pb\_hydraulique problem in order to accept the estimateur\_aposteriori post-processing.

Keyword Discretize should have already been used to read the object.

See also: Pb\_base (4.34)

Usage:

```

pb_hydraulique_aposteriori str
Read str {
 fluide_incompressible fluide_incompressible
 Navier_Stokes_Aposteriori navier_stokes_aposteriori
 [milieu milieu_base]
 [constituant constituant]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]
}

```

where

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem.
- **Navier\_Stokes\_Aposteriori** *navier\_stokes\_aposteriori* (5.17): Modification of the `Navier_Stokes_standard` class in order to accept the `estimateur_aposteriori` post-processing. To post-process `estimateur_aposteriori`, add this keyword into the list of fields to be post-processed. This estimator will generate a map of `aposteriori` error estimators; it is defined on each mesh cell and is a measure of the local discretisation error. This will serve for adaptive mesh refinement
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is `lata` in order to use `OpenDX` to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory `lata` used in this example should be created before running the computation or the `lata` files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than `Sauvegarde` except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the `name_file` file (see the class `format_file`). If `format_reprise` is `xyz`, the `name_file` file should be the `.xyz` file created by the previous calculation. With this file, it is possible to resume a parallel calculation on `P` processors, whereas the previous calculation has been run on `N` ( $N < P$ ) processors. Should the calculation be resumed, values for the `tinit` (see `schema_temps_base`) time fields are taken from the `name_file` file. If there is no backup corresponding to this time in the `name_file`, `TRUST` exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the `name_file` file, resume the calculation at the last time found in the file (`tinit` is set to last time of saved files).

## 4.46 Pb\_hydraulique\_concentration

Description: Resolution of Navier-Stokes/multiple constituent transport equations.

Keyword Discretize should have already been used to read the object.

See also: Pb\_base (4.34)

Usage:

**pb\_hydraulique\_concentration** *str*

**Read** *str* {

```
 fluide_incompressible fluide_incompressible
 [constituant constituant]
 [navier_stokes_standard navier_stokes_standard]
 [convection_diffusion_concentration convection_diffusion_concentration]
 [milieu milieu_base]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]
```

}

where

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem.
- **constituant** *constituant* (26.5): Constituents.
- **navier\_stokes\_standard** *navier\_stokes\_standard* (5.57): Navier-Stokes equations.
- **convection\_diffusion\_concentration** *convection\_diffusion\_concentration* (5.30): Constituent transport vectorial equation (concentration diffusion convection).
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N (N<>P) processors. Should the

calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.

- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

#### 4.47 Pb\_hydraulique\_concentration\_scalaires\_passifs

Description: Resolution of Navier-Stokes/multiple constituent transport equations with the additional passive scalar equations.

Keyword Discretize should have already been used to read the object.

See also: pb\_avec\_passif (4.41)

Usage:

**pb\_hydraulique\_concentration\_scalaires\_passifs** *str*

**Read** *str* {

```

 fluide_incompressible fluide_incompressible
 [constituant constituant]
 [navier_stokes_standard navier_stokes_standard]
 [convection_diffusion_concentration convection_diffusion_concentration]
 equations_scalaires_passifs listeqn
 [milieu milieu_base]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]

```

}

where

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem.
- **constituant** *constituant* (26.5): Constituents.
- **navier\_stokes\_standard** *navier\_stokes\_standard* (5.57): Navier-Stokes equations.
- **convection\_diffusion\_concentration** *convection\_diffusion\_concentration* (5.30): Constituent transport equations (concentration diffusion convection).
- **equations\_scalaires\_passifs** *listeqn* (4.14) for inheritance: Passive scalar equations. The unknowns of the passive scalar equation number N are named temperatureN or concentrationN or fraction\_masseN. This keyword is used to define initial conditions and the post processing fields. This kind of problem is very useful to test in only one data file (and then only one calculation) different schemes or different boundary conditions for the scalar transport equation.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This

- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N < P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

#### 4.48 Pb\_hydraulique\_concentration\_turbulent

Description: Resolution of Navier-Stokes/multiple constituent transport equations, with turbulence modelling.

Keyword Discretize should have already been used to read the object.

See also: Pb\_base (4.34)

Usage:

**pb\_hydraulique\_concentration\_turbulent** *str*

**Read** *str* {

```

 fluide_incompressible fluide_incompressible
 [constituant constituant]
 [navier_stokes_turbulent navier_stokes_turbulent]
 [convection_diffusion_concentration_turbulent convection_diffusion_concentration_turbulent]
 [milieu milieu_base]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]

```

}

where

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem.
- **constituant** *constituant* (26.5): Constituents.

- **navier\_stokes\_turbulent** *navier\_stokes\_turbulent* (5.58): Navier-Stokes equations as well as the associated turbulence model equations.
- **convection\_diffusion\_concentration\_turbulent** *convection\_diffusion\_concentration\_turbulent* (5.32): Constituent transport equations (concentration diffusion convection) as well as the associated turbulence model equations.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N < P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

#### 4.49 Pb\_hydraulique\_concentration\_turbulent\_scalaires\_passifs

Description: Resolution of Navier-Stokes/multiple constituent transport equations, with turbulence modelling and with the additional passive scalar equations.

Keyword Discretize should have already been used to read the object.

See also: pb\_avec\_passif (4.41)

Usage:

**pb\_hydraulique\_concentration\_turbulent\_scalaires\_passifs** *str*

Read *str* {

```

 fluide_incompressible fluide_incompressible
 [constituant constituant]
 [navier_stokes_turbulent navier_stokes_turbulent]
 [convection_diffusion_concentration_turbulent convection_diffusion_concentration_turbulent]
 equations_scalaires_passifs listeqn
 [milieu milieu_base]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]

```

```

[liste_de_postraitements liste_post_ok]
[liste_postraitements liste_post]
[sauvegarde format_file_base]
[sauvegarde_simple format_file_base]
[reprise format_file_base]
[resume_last_time format_file_base]
}
where

```

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem.
- **constituant** *constituant* (26.5): Constituents.
- **navier\_stokes\_turbulent** *navier\_stokes\_turbulent* (5.58): Navier-Stokes equations as well as the associated turbulence model equations.
- **convection\_diffusion\_concentration\_turbulent** *convection\_diffusion\_concentration\_turbulent* (5.32): Constituent transport equations (concentration diffusion convection) as well as the associated turbulence model equations.
- **equations\_scalaires\_passifs** *listeqn* (4.14) for inheritance: Passive scalar equations. The unknowns of the passive scalar equation number N are named temperatureN or concentrationN or fraction\_masseN. This keyword is used to define initial conditions and the post processing fields. This kind of problem is very useful to test in only one data file (and then only one calculation) different schemes or different boundary conditions for the scalar transport equation.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processing|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \leq P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).



## 4.50 Pb\_hydraulique\_ibm

Description: Resolution of the IBM Navier-Stokes equations.

Keyword Discretize should have already been used to read the object.

See also: Pb\_base (4.34)

Usage:

**pb\_hydraulique\_ibm** *str*

**Read** *str* {

```
 fluide_incompressible fluide_incompressible
 navier_stokes_ibm navier_stokes_ibm
 [milieu milieu_base]
 [constituant constituant]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]
```

}

where

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem.
- **navier\_stokes\_ibm** *navier\_stokes\_ibm* (5.52): IBM Navier-Stokes equations.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \geq P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.



- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the *name\_file* file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

#### 4.51 Pb\_hydraulique\_melange\_binaire\_qc

Description: Resolution of a binary mixture problem for a quasi-compressible fluid with an iso-thermal condition.

Keywords for the unknowns other than pressure, velocity, fraction\_massique are :

masse\_volumique : density

pression : reduced pressure

pression\_tot : total pressure.

Keyword Discretize should have already been used to read the object.

See also: Pb\_base (4.34)

Usage:

**pb\_hydraulique\_melange\_binaire\_QC** *str*

**Read** *str* {

```

 fluide_quasi_compressible fluide_quasi_compressible
 [constituant constituant]
 navier_stokes_QC navier_stokes_qc
 convection_diffusion_espece_binaire_QC convection_diffusion_espece_binaire_qc
 [milieu milieu_base]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]

```

}

where

- **fluide\_quasi\_compressible** *fluide\_quasi\_compressible* (26.11): The fluid medium associated with the problem.
- **constituant** *constituant* (26.5): The various constituents associated to the problem.
- **navier\_stokes\_QC** *navier\_stokes\_qc* (5.48): Navier-Stokes equation for a quasi-compressible fluid.
- **convection\_diffusion\_espece\_binaire\_QC** *convection\_diffusion\_espece\_binaire\_qc* (5.33): Species conservation equation for a binary quasi-compressible fluid.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.

- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \leq P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.52 Pb\_hydraulique\_melange\_binaire\_wc

Description: Resolution of a binary mixture problem for a weakly-compressible fluid with an iso-thermal condition.

Keywords for the unknowns other than pressure, velocity, fraction\_massique are :

masse\_volumique : density

pression : reduced pressure

pression\_tot : total pressure

pression\_hydro : hydro-static pressure

pression\_eos : pressure used in state equation.

Keyword Discretize should have already been used to read the object.

See also: Pb\_base (4.34)

Usage:

**pb\_hydraulique\_melange\_binaire\_WC** *str*

**Read** *str* {

```

 fluide_weakly_compressible fluide_weakly_compressible
 navier_stokes_WC navier_stokes_wc
 convection_diffusion_espece_binaire_WC convection_diffusion_espece_binaire_wc
 [milieu milieu_base]
 [constituant constituant]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]

```

}

where

- **fluide\_weakly\_compressible** *fluide\_weakly\_compressible* (26.17): The fluid medium associated with the problem.

- **navier\_stokes\_WC** *navier\_stokes\_wc* (5.49): Navier-Stokes equation for a weakly-compressible fluid.
- **convection\_diffusion\_espece\_binaire\_WC** *convection\_diffusion\_espece\_binaire\_wc* (5.34): Species conservation equation for a binary weakly-compressible fluid.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N < P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

### 4.53 Pb\_hydraulique\_melange\_binaire\_turbulent\_qc

Description: Resolution of a turbulent binary mixture problem for a quasi-compressible fluid with an isothermal condition.

Keyword Discretize should have already been used to read the object.

See also: Pb\_base (4.34)

Usage:

**pb\_hydraulique\_melange\_binaire\_turbulent\_qc** *str*

Read *str* {

```

fluide_quasi_compressible fluide_quasi_compressible
navier_stokes_turbulent_qc navier_stokes_turbulent_qc
Convection_Diffusion_Espece_Binaire_Turbulent_QC convection_diffusion_espece_binaire_turbulent-
_qc
[milieu milieu_base]
[constituant constituant]
[Post_processing|postraitement corps_postraitement]
[Post_processings|postraitements post_processings]

```

```

[liste_de_postraitements liste_post_ok]
[liste_postraitements liste_post]
[sauvegarde format_file_base]
[sauvegarde_simple format_file_base]
[reprise format_file_base]
[resume_last_time format_file_base]
}
where

```

- **fluide\_quasi\_compressible** *fluide\_quasi\_compressible* (26.11): The fluid medium associated with the problem.
- **navier\_stokes\_turbulent\_qc** *navier\_stokes\_turbulent\_qc* (5.59): Navier-Stokes equation for a quasi-compressible fluid as well as the associated turbulence model equations.
- **Convection\_Diffusion\_Espece\_Binaire\_Turbulent\_QC** *convection\_diffusion\_espece\_binaire\_turbulent\_qc* (5.9): Species conservation equation for a quasi-compressible fluid as well as the associated turbulence model equations.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N < P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.54 Pb\_hydraulique\_turbulent

Description: Resolution of Navier-Stokes equations with turbulence modelling.

Keyword Discretize should have already been used to read the object.

See also: Pb\_base (4.34) Pb\_Rayo\_Hydraulique\_Turbulent (4.23)

Usage:

```

pb_hydraulique_turbulent str
Read str {

 fluide_incompressible fluide_incompressible
 navier_stokes_turbulent navier_stokes_turbulent
 [milieu milieu_base]
 [constituant constituant]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]

}

```

where

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem.
- **navier\_stokes\_turbulent** *navier\_stokes\_turbulent* (5.58): Navier-Stokes equations as well as the associated turbulence model equations.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \geq P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.55 Pb\_mg

Description: Multi-grid problem.

Keyword Discretize should have already been used to read the object.

See also: `pb_gen_base` (4)

Usage:

**pb\_mg**

## 4.56 Pb\_phase\_field

Description: Problem to solve local instantaneous incompressible-two-phase-flows. Complete description of the Phase Field model for incompressible and immiscible fluids can be found into this PDF: [TRUST-ROOT/doc/TRUST/phase\\_field\\_non\\_miscible\\_manuel.pdf](#)

Keyword Discretize should have already been used to read the object.

See also: `Pb_base` (4.34)

Usage:

**pb\_phase\_field** *str*

**Read** *str* {

```
 fluide_incompressible fluide_incompressible
 [constituant constituant]
 [navier_stokes_phase_field navier_stokes_phase_field]
 [convection_diffusion_phase_field convection_diffusion_phase_field]
 [milieu milieu_base]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]
```

}

where

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem.
- **constituant** *constituant* (26.5): Constituents.
- **navier\_stokes\_phase\_field** *navier\_stokes\_phase\_field* (5.54): Navier Stokes equation for the Phase Field problem.
- **convection\_diffusion\_phase\_field** *convection\_diffusion\_phase\_field* (5.38): Cahn-Hilliard equation of the Phase Field problem. The unknown of this equation is the concentration C.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This

block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.

- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \leq P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.57 Pb\_post

Description: not\_set

Keyword Discretize should have already been used to read the object.

See also: Pb\_base (4.34)

Usage:

**pb\_post** *str*

**Read** *str* {

```
[milieu milieu_base]
[constituant constituant]
[Post_processing|postraitement corps_postraitement]
[Post_processings|postraitements post_processings]
[liste_de_postraitements liste_post_ok]
[liste_postraitements liste_post]
[sauvegarde format_file_base]
[sauvegarde_simple format_file_base]
[reprise format_file_base]
[resume_last_time format_file_base]
```

}

where

- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and



in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.

- **sauegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N < P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.58 Pb\_thermohydraulique

Description: Resolution of thermohydraulic problem.

Keyword Discretize should have already been used to read the object.

See also: Pb\_base (4.34) Pb\_Rayo\_Thermohydraulique (4.24) Pb\_Thermohydraulique\_sensibility (4.33)

Usage:

**pb\_thermohydraulique** *str*

**Read** *str* {

```
[fluide_incompressible fluide_incompressible]
[fluide_ostwald fluide_ostwald]
[fluide_sodium_liquide fluide_sodium_liquide]
[fluide_sodium_gaz fluide_sodium_gaz]
[correlations bloc_lecture]
[navier_stokes_standard navier_stokes_standard]
[convection_diffusion_temperature convection_diffusion_temperature]
[milieu milieu_base]
[constituant constituant]
[Post_processing|postraitemment corps_postraitemment]
[Post_processings|postraitemments post_processings]
[liste_de_postraitemments liste_post_ok]
[liste_postraitemments liste_post]
[sauegarde format_file_base]
[sauegarde_simple format_file_base]
[reprise format_file_base]
[resume_last_time format_file_base]
```

}

where

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem (only one possibility).



- **fluide\_ostwald** *fluide\_ostwald* (26.10): The fluid medium associated with the problem (only one possibility).
- **fluide\_sodium\_liquide** *fluide\_sodium\_liquide* (26.15): The fluid medium associated with the problem (only one possibility).
- **fluide\_sodium\_gaz** *fluide\_sodium\_gaz* (26.14): The fluid medium associated with the problem (only one possibility).
- **correlations** *bloc\_lecture* (3.2): List of correlations used in specific source terms (i.e. interfacial flux, interfacial friction, ...)
- **navier\_stokes\_standard** *navier\_stokes\_standard* (5.57): Navier-Stokes equations.
- **convection\_diffusion\_temperature** *convection\_diffusion\_temperature* (5.39): Energy equation (temperature diffusion convection).
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitemnt** *corps\_postraitemnt* (4.2) for inheritance: One post-processing (without name).
- **Post\_processing|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N < P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.59 Pb\_thermohydraulique\_qc

Description: Resolution of thermo-hydraulic problem for a quasi-compressible fluid.

Keywords for the unknowns other than pressure, velocity, temperature are :

masse\_volumique : density

enthalpie : enthalpy

pression : reduced pressure

pression\_tot : total pressure.

Keyword Discretize should have already been used to read the object.

See also: Pb\_base (4.34) Pb\_Rayo\_Thermohydraulique\_QC (4.25)

Usage:

```

pb_thermohydraulique_QC str
Read str {
 fluide_quasi_compressible fluide_quasi_compressible
 navier_stokes_QC navier_stokes_qc
 convection_diffusion_chaleur_QC convection_diffusion_chaleur_qc
 [milieu milieu_base]
 [constituant constituant]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]
}
where

```

- **fluide\_quasi\_compressible** *fluide\_quasi\_compressible* (26.11): The fluid medium associated with the problem.
- **navier\_stokes\_QC** *navier\_stokes\_qc* (5.48): Navier-Stokes equation for a quasi-compressible fluid.
- **convection\_diffusion\_chaleur\_QC** *convection\_diffusion\_chaleur\_qc* (5.27): Temperature equation for a quasi-compressible fluid.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N < P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.60 Pb\_thermohydraulique\_wc

Description: Resolution of thermo-hydraulic problem for a weakly-compressible fluid.

Keywords for the unknowns other than pressure, velocity, temperature are :

masse\_volumique : density

pression : reduced pressure

pression\_tot : total pressure

pression\_hydro : hydro-static pressure

pression\_eos : pressure used in state equation.

Keyword Discretize should have already been used to read the object.

See also: Pb\_base (4.34)

Usage:

**pb\_thermohydraulique\_WC** *str*

**Read** *str* {

```
 fluide_weakly_compressible fluide_weakly_compressible
 navier_stokes_WC navier_stokes_wc
 convection_diffusion_chaleur_WC convection_diffusion_chaleur_wc
 [milieu milieu_base]
 [constituant constituant]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]
```

}

where

- **fluide\_weakly\_compressible** *fluide\_weakly\_compressible* (26.17): The fluid medium associated with the problem.
- **navier\_stokes\_WC** *navier\_stokes\_wc* (5.49): Navier-Stokes equation for a weakly-compressible fluid.
- **convection\_diffusion\_chaleur\_WC** *convection\_diffusion\_chaleur\_wc* (5.28): Temperature equation for a weakly-compressible fluid.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.

- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \leq P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.61 Pb\_thermohydraulique\_concentration

Description: Resolution of Navier-Stokes/energy/multiple constituent transport equations.

Keyword Discretize should have already been used to read the object.

See also: Pb\_base (4.34)

Usage:

**pb\_thermohydraulique\_concentration** *str*

**Read** *str* {

```

 fluide_incompressible fluide_incompressible
 [constituant constituant]
 [navier_stokes_standard navier_stokes_standard]
 [convection_diffusion_concentration convection_diffusion_concentration]
 [convection_diffusion_temperature convection_diffusion_temperature]
 [milieu milieu_base]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]

```

}

where

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem.
- **constituant** *constituant* (26.5): Constituents.
- **navier\_stokes\_standard** *navier\_stokes\_standard* (5.57): Navier-Stokes equations.
- **convection\_diffusion\_concentration** *convection\_diffusion\_concentration* (5.30): Constituent transport equations (concentration diffusion convection).
- **convection\_diffusion\_temperature** *convection\_diffusion\_temperature* (5.39): Energy equation (temperature diffusion convection).
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).

- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N < P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.62 Pb\_thermohydraulique\_concentration\_scalaires\_passifs

Description: Resolution of Navier-Stokes/energy/multiple constituent transport equations, with the additional passive scalar equations.

Keyword Discretize should have already been used to read the object.

See also: pb\_avec\_passif (4.41)

Usage:

**pb\_thermohydraulique\_concentration\_scalaires\_passifs** *str*

**Read** *str* {

```

 fluide_incompressible fluide_incompressible
 [constituant constituant]
 [navier_stokes_standard navier_stokes_standard]
 [convection_diffusion_concentration convection_diffusion_concentration]
 [convection_diffusion_temperature convection_diffusion_temperature]
 equations_scalaires_passifs listeqn
 [milieu milieu_base]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]

```

}

where

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem.
- **constituant** *constituant* (26.5): Constituents.
- **navier\_stokes\_standard** *navier\_stokes\_standard* (5.57): Navier-Stokes equations.
- **convection\_diffusion\_concentration** *convection\_diffusion\_concentration* (5.30): Constituent transport equations (concentration diffusion convection).
- **convection\_diffusion\_temperature** *convection\_diffusion\_temperature* (5.39): Energy equations (temperature diffusion convection).
- **equations\_scalaires\_passifs** *listeqn* (4.14) for inheritance: Passive scalar equations. The unknowns of the passive scalar equation number N are named temperatureN or concentrationN or fraction-massiqueN. This keyword is used to define initial conditions and the post processing fields. This kind of problem is very useful to test in only one data file (and then only one calculation) different schemes or different boundary conditions for the scalar transport equation.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N < P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

#### 4.63 Pb\_thermohydraulique\_concentration\_turbulent

Description: Resolution of Navier-Stokes/energy/multiple constituent transport equations, with turbulence modelling.

Keyword Discretize should have already been used to read the object.

See also: Pb\_base (4.34)

Usage:

```
pb_thermohydraulique_concentration_turbulent str
Read str {
```



```

fluide_incompressible fluide_incompressible
[constituant constituant]
[navier_stokes_turbulent navier_stokes_turbulent]
[convection_diffusion_concentration_turbulent convection_diffusion_concentration_turbulent]
[convection_diffusion_temperature_turbulent convection_diffusion_temperature_turbulent]
[milieu milieu_base]
[Post_processing|postraitement corps_postraitement]
[Post_processings|postraitements post_processings]
[liste_de_postraitements liste_post_ok]
[liste_postraitements liste_post]
[sauvegarde format_file_base]
[sauvegarde_simple format_file_base]
[reprise format_file_base]
[resume_last_time format_file_base]
}
where

```

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem.
- **constituant** *constituant* (26.5): Constituents.
- **navier\_stokes\_turbulent** *navier\_stokes\_turbulent* (5.58): Navier-Stokes equations as well as the associated turbulence model equations.
- **convection\_diffusion\_concentration\_turbulent** *convection\_diffusion\_concentration\_turbulent* (5.32): Constituent transport equations (concentration diffusion convection) as well as the associated turbulence model equations.
- **convection\_diffusion\_temperature\_turbulent** *convection\_diffusion\_temperature\_turbulent* (5.44): Energy equation (temperature diffusion convection) as well as the associated turbulence model equations.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N (N<>P) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time

of saved files).

#### 4.64 Pb\_thermohydraulique\_concentration\_turbulent\_scalaires\_passifs

Description: Resolution of Navier-Stokes/energy/multiple constituent transport equations, with turbulence modelling and with the additional passive scalar equations.

Keyword Discretize should have already been used to read the object.

See also: pb\_avec\_passif (4.41)

Usage:

**pb\_thermohydraulique\_concentration\_turbulent\_scalaires\_passifs** *str*

**Read** *str* {

```
fluide_incompressible fluide_incompressible
[constituant constituant]
[navier_stokes_turbulent navier_stokes_turbulent]
[convection_diffusion_concentration_turbulent convection_diffusion_concentration_turbulent]
[convection_diffusion_temperature_turbulent convection_diffusion_temperature_turbulent]
equations_scalaires_passifs listeqn
[milieu milieu_base]
[Post_processing|postraitement corps_postraitement]
[Post_processings|postraitements post_processings]
[liste_de_postraitements liste_post_ok]
[liste_postraitements liste_post]
[sauvegarde format_file_base]
[sauvegarde_simple format_file_base]
[reprise format_file_base]
[resume_last_time format_file_base]
```

}

where

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem.
- **constituant** *constituant* (26.5): Constituents.
- **navier\_stokes\_turbulent** *navier\_stokes\_turbulent* (5.58): Navier-Stokes equations as well as the associated turbulence model equations.
- **convection\_diffusion\_concentration\_turbulent** *convection\_diffusion\_concentration\_turbulent* (5.32): Constituent transport equations (concentration diffusion convection) as well as the associated turbulence model equations.
- **convection\_diffusion\_temperature\_turbulent** *convection\_diffusion\_temperature\_turbulent* (5.44): Energy equations (temperature diffusion convection) as well as the associated turbulence model equations.
- **equations\_scalaires\_passifs** *listeqn* (4.14) for inheritance: Passive scalar equations. The unknowns of the passive scalar equation number N are named temperatureN or concentrationN or fraction\_masseN. This keyword is used to define initial conditions and the post processing fields. This kind of problem is very useful to test in only one data file (and then only one calculation) different schemes or different boundary conditions for the scalar transport equation.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).



- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N < P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.65 Pb\_thermohydraulique\_especes\_qc

Description: Resolution of thermo-hydraulic problem for a multi-species quasi-compressible fluid.

Keyword Discretize should have already been used to read the object.

See also: pb\_avec\_passif (4.41)

Usage:

**pb\_thermohydraulique\_especes\_QC** *str*

**Read** *str* {

```

 fluide_quasi_compressible fluide_quasi_compressible
 navier_stokes_QC navier_stokes_qc
 convection_diffusion_chaleur_QC convection_diffusion_chaleur_qc
 equations_scalaires_passifs listeqn
 [milieu milieu_base]
 [constituant constituant]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]

```

}

where

- **fluide\_quasi\_compressible** *fluide\_quasi\_compressible* (26.11): The fluid medium associated with the problem.

- **navier\_stokes\_QC** *navier\_stokes\_qc* (5.48): Navier-Stokes equation for a quasi-compressible fluid.
- **convection\_diffusion\_chaleur\_QC** *convection\_diffusion\_chaleur\_qc* (5.27): Temperature equation for a quasi-compressible fluid.
- **equations\_scalaires\_passifs** *listeqn* (4.14) for inheritance: Passive scalar equations. The unknowns of the passive scalar equation number N are named temperatureN or concentrationN or fraction\_massiqueN. This keyword is used to define initial conditions and the post processing fields. This kind of problem is very useful to test in only one data file (and then only one calculation) different schemes or different boundary conditions for the scalar transport equation.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processing|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N < P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.66 Pb\_thermohydraulique\_especes\_wc

Description: Resolution of thermo-hydraulic problem for a multi-species weakly-compressible fluid.

Keyword Discretize should have already been used to read the object.

See also: pb\_avec\_passif (4.41)

Usage:

**pb\_thermohydraulique\_especes\_WC** *str*

**Read** *str* {

```

 fluide_weakly_compressible fluide_weakly_compressible
 navier_stokes_WC navier_stokes_wc
 convection_diffusion_chaleur_WC convection_diffusion_chaleur_wc
 equations_scalaires_passifs listeqn

```

```

[milieu milieu_base]
[constituant constituant]
[Post_processing|postraitement corps_postraitement]
[Post_processings|postraitements post_processings]
[liste_de_postraitements liste_post_ok]
[liste_postraitements liste_post]
[sauvegarde format_file_base]
[sauvegarde_simple format_file_base]
[reprise format_file_base]
[resume_last_time format_file_base]
}
where

```

- **fluide\_weakly\_compressible** *fluide\_weakly\_compressible* (26.17): The fluid medium associated with the problem.
- **navier\_stokes\_WC** *navier\_stokes\_wc* (5.49): Navier-Stokes equation for a weakly-compressible fluid.
- **convection\_diffusion\_chaleur\_WC** *convection\_diffusion\_chaleur\_wc* (5.28): Temperature equation for a weakly-compressible fluid.
- **equations\_scalaires\_passifs** *listeqn* (4.14) for inheritance: Passive scalar equations. The unknowns of the passive scalar equation number N are named temperatureN or concentrationN or fraction-massiqueN. This keyword is used to define initial conditions and the post processing fields. This kind of problem is very useful to test in only one data file (and then only one calculation) different schemes or different boundary conditions for the scalar transport equation.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \leq P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.67 Pb\_thermohydraulique\_especes\_turbulent\_qc

Description: Resolution of turbulent thermohydraulic problem under low Mach number with passive scalar equations.

Keyword Discretize should have already been used to read the object.

See also: pb\_avec\_passif (4.41)

Usage:

**pb\_thermohydraulique\_especes\_turbulent\_qc** *str*

**Read** *str* {

```
 fluide_quasi_compressible fluide_quasi_compressible
 navier_stokes_turbulent_qc navier_stokes_turbulent_qc
 convection_diffusion_chaleur_turbulent_qc convection_diffusion_chaleur_turbulent_qc
 equations_scalaires_passifs listeqn
 [milieu milieu_base]
 [constituant constituant]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]
```

}

where

- **fluide\_quasi\_compressible** *fluide\_quasi\_compressible* (26.11): The fluid medium associated with the problem.
- **navier\_stokes\_turbulent\_qc** *navier\_stokes\_turbulent\_qc* (5.59): Navier-Stokes equations under low Mach number as well as the associated turbulence model equations.
- **convection\_diffusion\_chaleur\_turbulent\_qc** *convection\_diffusion\_chaleur\_turbulent\_qc* (5.29): Energy equation under low Mach number as well as the associated turbulence model equations.
- **equations\_scalaires\_passifs** *listeqn* (4.14) for inheritance: Passive scalar equations. The unknowns of the passive scalar equation number N are named temperatureN or concentrationN or fraction-massiqueN. This keyword is used to define initial conditions and the post processing fields. This kind of problem is very useful to test in only one data file (and then only one calculation) different schemes or different boundary conditions for the scalar transport equation.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified

for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.

- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \leq P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.68 Pb\_thermohydraulique\_ibm

Description: Resolution of IBM thermohydraulic problem.

Keyword Discretize should have already been used to read the object.

See also: Pb\_base (4.34)

Usage:

**pb\_thermohydraulique\_ibm** *str*

**Read** *str* {

```
[fluide_incompressible fluide_incompressible]
[fluide_ostwald fluide_ostwald]
[navier_stokes_ibm navier_stokes_ibm]
[convection_diffusion_temperature_ibm convection_diffusion_temperature_ibm]
[milieu milieu_base]
[constituant constituant]
[Post_processing|postraitemment corps_postraitemment]
[Post_processings|postraitemments post_processings]
[liste_de_postraitemments liste_post_ok]
[liste_postraitemments liste_post]
[sauvegarde format_file_base]
[sauvegarde_simple format_file_base]
[reprise format_file_base]
[resume_last_time format_file_base]
```

}

where

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem (only one possibility).
- **fluide\_ostwald** *fluide\_ostwald* (26.10): The fluid medium associated with the problem (only one possibility).
- **navier\_stokes\_ibm** *navier\_stokes\_ibm* (5.52): IBM Navier-Stokes equations.
- **convection\_diffusion\_temperature\_ibm** *convection\_diffusion\_temperature\_ibm* (5.42): IBM Energy equation (temperature diffusion convection).
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.

- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N < P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.69 Pb\_thermohydraulique\_scalaires\_passifs

Description: Resolution of thermohydraulic problem, with the additional passive scalar equations.

Keyword Discretize should have already been used to read the object.

See also: pb\_avec\_passif (4.41)

Usage:

**pb\_thermohydraulique\_scalaires\_passifs** *str*

**Read** *str* {

```

 fluide_incompressible fluide_incompressible
 [constituant constituant]
 [navier_stokes_standard navier_stokes_standard]
 [convection_diffusion_temperature convection_diffusion_temperature]
 equations_scalaires_passifs listeqn
 [milieu milieu_base]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]

```

}

where

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem.
- **constituant** *constituant* (26.5): Constituents.
- **navier\_stokes\_standard** *navier\_stokes\_standard* (5.57): Navier-Stokes equations.
- **convection\_diffusion\_temperature** *convection\_diffusion\_temperature* (5.39): Energy equations (temperature diffusion convection).
- **equations\_scalaires\_passifs** *listeqn* (4.14) for inheritance: Passive scalar equations. The unknowns of the passive scalar equation number N are named temperatureN or concentrationN or fraction\_massiqueN. This keyword is used to define initial conditions and the post processing fields. This kind of problem is very useful to test in only one data file (and then only one calculation) different schemes or different boundary conditions for the scalar transport equation.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \leq P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.70 Pb\_thermohydraulique\_turbulent

Description: Resolution of thermohydraulic problem, with turbulence modelling.

Keyword Discretize should have already been used to read the object.

See also: Pb\_base (4.34) Pb\_Rayo\_Thermohydraulique\_Turbulent (4.26)

Usage:

**pb\_thermohydraulique\_turbulent** *str*

**Read** *str* {

**fluide\_incompressible** *fluide\_incompressible*  
**navier\_stokes\_turbulent** *navier\_stokes\_turbulent*



```

convection_diffusion_temperature_turbulent convection_diffusion_temperature_turbulent
[milieu milieu_base]
[constituant constituant]
[Post_processing|postraitement corps_postraitement]
[Post_processings|postraitements post_processings]
[liste_de_postraitements liste_post_ok]
[liste_postraitements liste_post]
[sauvegarde format_file_base]
[sauvegarde_simple format_file_base]
[reprise format_file_base]
[resume_last_time format_file_base]
}
where

```

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem.
- **navier\_stokes\_turbulent** *navier\_stokes\_turbulent* (5.58): Navier-Stokes equations as well as the associated turbulence model equations.
- **convection\_diffusion\_temperature\_turbulent** *convection\_diffusion\_temperature\_turbulent* (5.44): Energy equation (temperature diffusion convection) as well as the associated turbulence model equations.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N (N<>P) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).



## 4.71 Pb\_thermohydraulique\_turbulent\_qc

Description: Resolution of turbulent thermohydraulic problem under low Mach number.

Warning : Available for VDF and VEF P0/P1NC discretization only.

Keyword Discretize should have already been used to read the object.

See also: Pb\_base (4.34) Pb\_Rayo\_Thermohydraulique\_Turbulent\_QC (4.27)

Usage:

**pb\_thermohydraulique\_turbulent\_qc** *str*

**Read** *str* {

```
 fluide_quasi_compressible fluide_quasi_compressible
 navier_stokes_turbulent_qc navier_stokes_turbulent_qc
 convection_diffusion_chaleur_turbulent_qc convection_diffusion_chaleur_turbulent_qc
 [milieu milieu_base]
 [constituant constituant]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]
```

}

where

- **fluide\_quasi\_compressible** *fluide\_quasi\_compressible* (26.11): The fluid medium associated with the problem.
- **navier\_stokes\_turbulent\_qc** *navier\_stokes\_turbulent\_qc* (5.59): Navier-Stokes equations under low Mach number as well as the associated turbulence model equations.
- **convection\_diffusion\_chaleur\_turbulent\_qc** *convection\_diffusion\_chaleur\_turbulent\_qc* (5.29): Energy equation under low Mach number as well as the associated turbulence model equations.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz

file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \ll P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.

- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.72 Pb\_thermohydraulique\_turbulent\_scalaires\_passifs

Description: Resolution of thermohydraulic problem, with turbulence modelling and with the additional passive scalar equations.

Keyword Discretize should have already been used to read the object.

See also: pb\_avec\_passif (4.41)

Usage:

**pb\_thermohydraulique\_turbulent\_scalaires\_passifs** *str*

**Read** *str* {

```

 fluide_incompressible fluide_incompressible
 [constituant constituant]
 [navier_stokes_turbulent navier_stokes_turbulent]
 [convection_diffusion_temperature_turbulent convection_diffusion_temperature_turbulent]
 equations_scalaires_passifs listeqn
 [milieu milieu_base]
 [Post_processing|postraitement corps_postraitement]
 [Post_processings|postraitements post_processings]
 [liste_de_postraitements liste_post_ok]
 [liste_postraitements liste_post]
 [sauvegarde format_file_base]
 [sauvegarde_simple format_file_base]
 [reprise format_file_base]
 [resume_last_time format_file_base]

```

}

where

- **fluide\_incompressible** *fluide\_incompressible* (26.9): The fluid medium associated with the problem.
- **constituant** *constituant* (26.5): Constituents.
- **navier\_stokes\_turbulent** *navier\_stokes\_turbulent* (5.58): Navier-Stokes equations as well as the associated turbulence model equations.
- **convection\_diffusion\_temperature\_turbulent** *convection\_diffusion\_temperature\_turbulent* (5.44): Energy equations (temperature diffusion convection) as well as the associated turbulence model equations.
- **equations\_scalaires\_passifs** *listeqn* (4.14) for inheritance: Passive scalar equations. The unknowns of the passive scalar equation number N are named temperatureN or concentrationN or fraction\_masseN. This keyword is used to define initial conditions and the post processing fields. This kind of problem is very useful to test in only one data file (and then only one calculation) different schemes or different boundary conditions for the scalar transport equation.
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.

- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processing|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \leq P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

### 4.73 Pbc\_med

Description: Allows to read med files and post-process them.

See also: pb\_gen\_base (4)

Usage:

**pb\_med list\_info\_med**  
where

- **list\_info\_med** *list\_info\_med* (4.74)

### 4.74 List\_info\_med

Description: not\_set

See also: listobj (46.5)

Usage:

{ object1 , object2 .... }  
list of *info\_med* (4.74.1) separated with ,

#### 4.74.1 Info\_med

Description: not\_set

See also: `objet_lecture` (47)

Usage:

**file\_med** **domaine** **pb\_post**

where

- **file\_med** *str*: Name of the MED file.
- **domaine** *str*: Name of domain.
- **pb\_post** *pb\_post* (4.57)

## 4.75 Problem\_read\_generic

Description: The `probleme_read_generic` differs from the rest of the TRUST code : The problem does not state the number of equations that are enclosed in the problem. As the list of equations to be solved in the generic read problem is declared in the data file and not pre-defined in the structure of the problem, each equation has to be distinctively associated with the problem with the Associate keyword.

Keyword Discretize should have already been used to read the object.

See also: `Pb_base` (4.34) `pb_fronttracking_disc` (4.8)

Usage:

**problem\_read\_generic** *str*

**Read** *str* {

```
[milieu milieu_base]
[constituant constituant]
[Post_processing|postraitement corps_postraitement]
[Post_processings|postraitements post_processings]
[liste_de_postraitements liste_post_ok]
[liste_postraitements liste_post]
[sauvegarde format_file_base]
[sauvegarde_simple format_file_base]
[reprise format_file_base]
[resume_last_time format_file_base]
```

}

where

- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.

- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \leq P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 4.76 Pb\_couple\_rayonnement

Description: This keyword is used to define a problem coupling several other problems to which radiation coupling is added.

See also: probleme\_couple (4.36)

Usage:

**pb\_couple\_rayonnement** *str*

**Read** *str* {

    [ **groupes** *list\_list\_nom*]

}

where

- **groupes** *list\_list\_nom* (4.37) for inheritance: { groupes { { pb1 , pb2 } , { pb3 , pb4 } } }

## 4.77 Probleme\_ftd\_ijk

Description: not\_set

Keyword Discretize should have already been used to read the object.

See also: pb\_fronttracking\_disc (4.8) Probleme\_FTD\_IJK\_cut\_cell (4.35)

Usage:

**probleme\_ftd\_ijk** *str*

**Read** *str* {

    [ **Fluide\_Diphasique\_IJK** *fluide\_diphasique\_ijk*]

    [ **nom\_sauvegarde** *str*]

    [ **sauvegarder\_xyz** ]

    [ **nom\_reprise** *str*]

**solved\_equations** *listdeuxmots\_acc*

    [ **fluide\_incompressible** *fluide\_incompressible*]

    [ **fluide\_diphasique** *fluide\_diphasique*]

    [ **constituant** *constituant*]

    [ **Triple\_Line\_Model\_FT\_Disc** *triple\_line\_model\_ft\_disc*]

    [ **milieu** *milieu\_base*]

    [ **Post\_processing|postraitement** *corps\_postraitement*]

    [ **Post\_processings|postraitements** *post\_processings*]

```

[liste_de_postraitements liste_post_ok]
[liste_postraitements liste_post]
[sauvegarde format_file_base]
[sauvegarde_simple format_file_base]
[reprise format_file_base]
[resume_last_time format_file_base]
}
where

```

- **Fluide\_Diphasique\_IJK** *fluide\_diphasique\_ijk* (26.1): The diphasic fluid medium associated with the problem.
- **nom\_sauvegarde** *str*: Definition of filename to save the calculation
- **sauvegarder\_xyz** : save in xyz format
- **nom\_reprise** *str*: Enable restart from filename given
- **solved\_equations** *listdeuxmots\_acc* (4.9) for inheritance: List of solved equations in the form 'equation\_type' 'equation\_alias'
- **fluide\_incompressible** *fluide\_incompressible* (26.9) for inheritance: The fluid medium associated with the problem.
- **fluide\_diphasique** *fluide\_diphasique* (26.8) for inheritance: The diphasic fluid medium associated with the problem.
- **constituant** *constituant* (26.5) for inheritance: Constituent.
- **Triple\_Line\_Model\_FT\_Disc** *triple\_line\_model\_ft\_disc* (9) for inheritance
- **milieu** *milieu\_base* (26) for inheritance: The medium associated with the problem.
- **Post\_processing|postraitement** *corps\_postraitement* (4.2) for inheritance: One post-processing (without name).
- **Post\_processings|postraitements** *post\_processings* (4.3) for inheritance: List of Postraitement objects (with name).
- **liste\_de\_postraitements** *liste\_post\_ok* (4.4) for inheritance: This
- **liste\_postraitements** *liste\_post* (4.5) for inheritance: This block defines the output files to be written during the computation. The output format is lata in order to use OpenDX to draw the results. This block can be divided in one or several sub-blocks that can be written at different frequencies and in different directories. Attention. The directory lata used in this example should be created before running the computation or the lata files will be lost.
- **sauvegarde** *format\_file\_base* (4.6) for inheritance: Keyword used when calculation results are to be backed up. When a coupling is performed, the backup-recovery file name must be well specified for each problem. In this case, you must save to different files and correctly specify these files when resuming the calculation.
- **sauvegarde\_simple** *format\_file\_base* (4.6) for inheritance: The same keyword than Sauvegarde except, the last time step only is saved.
- **reprise** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file (see the class format\_file). If format\_reprise is xyz, the name\_file file should be the .xyz file created by the previous calculation. With this file, it is possible to resume a parallel calculation on P processors, whereas the previous calculation has been run on N ( $N \leq P$ ) processors. Should the calculation be resumed, values for the tinit (see schema\_temps\_base) time fields are taken from the name\_file file. If there is no backup corresponding to this time in the name\_file, TRUST exits in error.
- **resume\_last\_time** *format\_file\_base* (4.6) for inheritance: Keyword to resume a calculation based on the name\_file file, resume the calculation at the last time found in the file (tinit is set to last time of saved files).

## 5 mor\_eqn

Description: Class of equation pieces (morceaux d'equation).

See also: `objet_u` (48) `eqn_base` (5.45)

Usage:

### 5.1 Conduction

Description: Heat equation.

Keyword Discretize should have already been used to read the object.

See also: `eqn_base` (5.45) `Conduction_ibm` (5.7)

Usage:

**Conduction** *str*

**Read** *str* {

```
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
```

}

where

- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.2 Bloc\_convection

Description: not\_set

See also: objet\_lecture ([47](#))

Usage:

**aco** **opérateur** **acof**

where

- **aco** *str* into [**'**]: Opening curly bracket.
- **opérateur** *convection\_deriv* ([5.2.1](#))
- **acof** *str* into [**'**]: Closing curly bracket.

### 5.2.1 Convection\_deriv

Description: not\_set

See also: objet\_lecture ([47](#)) [negligeable](#) ([5.2.2](#)) [amont](#) ([5.2.3](#)) [centre](#) ([5.2.4](#)) [centre4](#) ([5.2.5](#)) [ef](#) ([5.2.6](#)) [muscl\\_old](#) ([5.2.8](#)) [muscl](#) ([5.2.9](#)) [di\\_l2](#) ([5.2.10](#)) [quick](#) ([5.2.11](#)) [centre\\_old](#) ([5.2.12](#)) [amont\\_old](#) ([5.2.13](#)) [generic](#) ([5.2.14](#)) [muscl\\_new](#) ([5.2.15](#)) [kquick](#) ([5.2.16](#)) [muscl3](#) ([5.2.17](#)) [ef\\_stab](#) ([5.2.18](#)) [btd](#) ([5.2.21](#)) [supg](#) ([5.2.22](#)) [ale](#) ([5.2.23](#)) [RT](#) ([5.2.24](#)) [sensibility](#) ([5.2.25](#))

Usage:

**convection\_deriv**

### 5.2.2 Negligeable

Description: For VDF and VEF discretizations. Suppresses the convection operator.

See also: [convection\\_deriv](#) ([5.2.1](#))

Usage:

**negligeable**

### 5.2.3 Amont

Description: Keyword for upwind scheme for VDF or VEF discretizations. In VEF discretization equivalent to generic [amont](#) for TRUST version 1.5 or later. The previous upwind scheme can be used with the obsolete [amont\\_old](#) keyword.

See also: [convection\\_deriv](#) ([5.2.1](#))

Usage:

**amont**

### 5.2.4 Centre

Description: For VDF and VEF discretizations.

See also: [convection\\_deriv](#) ([5.2.1](#))

Usage:

**centre**



### 5.2.5 Centre4

Description: For VDF and VEF discretizations.

See also: `convection_deriv` ([5.2.1](#))

Usage:

**centre4**

### 5.2.6 Ef

Description: For VEF calculations, a centred convective scheme based on Finite Elements formulation can be called through the following data:

Convection { EF transportant\_bar val transporte\_bar val antisym val filtrer\_resu val }

This scheme is 2nd order accuracy (and get better the property of kinetic energy conservation). Due to possible problems of instabilities phenomena, this scheme has to be coupled with stabilisation process (see `Source_Qdm_lambdaup`). These two last data are equivalent from a theoretical point of view in variational writing to :  $\text{div}((u \cdot \text{grad } ub, vb) - (u \cdot \text{grad } vb, ub))$ , where  $vb$  corresponds to the filtered reference test functions.

Remark:

This class requires to define a filtering operator : see `solveur_bar`

See also: `convection_deriv` ([5.2.1](#))

Usage:

**ef** [ **mot1** ] [ **bloc\_ef** ]

where

- **mot1** *str* into [ 'defaut\_bar' ]: equivalent to `transportant_bar 0 transporte_bar 1 filtrer_resu 1 antisym 1`
- **bloc\_ef** *bloc\_ef* ([5.2.7](#))

### 5.2.7 Bloc\_ef

Description: `not_set`

See also: `objet_lecture` ([47](#))

Usage:

**mot1 val1 mot2 val2 mot3 val3 mot4 val4**

where

- **mot1** *str* into [ 'transportant\_bar', 'transporte\_bar', 'filtrer\_resu', 'antisym' ]
- **val1** *int* into [ 0, 1 ]
- **mot2** *str* into [ 'transportant\_bar', 'transporte\_bar', 'filtrer\_resu', 'antisym' ]
- **val2** *int* into [ 0, 1 ]
- **mot3** *str* into [ 'transportant\_bar', 'transporte\_bar', 'filtrer\_resu', 'antisym' ]
- **val3** *int* into [ 0, 1 ]
- **mot4** *str* into [ 'transportant\_bar', 'transporte\_bar', 'filtrer\_resu', 'antisym' ]
- **val4** *int* into [ 0, 1 ]

### 5.2.8 Muscl\_old

Description: Only for VEF discretization.

See also: convection\_deriv ([5.2.1](#))

Usage:

**muscl\_old**

### 5.2.9 Muscl

Description: Keyword for muscl scheme in VEF discretization equivalent to generic muscl vanleer 2 for the 1.5 version or later. The previous muscl scheme can be used with the obsolete in future muscl\_old keyword.

See also: convection\_deriv ([5.2.1](#))

Usage:

**muscl**

### 5.2.10 Di\_l2

Description: Only for VEF discretization.

See also: convection\_deriv ([5.2.1](#))

Usage:

**di\_l2**

### 5.2.11 Quick

Description: Only for VDF discretization.

See also: convection\_deriv ([5.2.1](#))

Usage:

**quick**

### 5.2.12 Centre\_old

Description: Only for VEF discretization.

See also: convection\_deriv ([5.2.1](#))

Usage:

**centre\_old**

### 5.2.13 Amont\_old

Description: Only for VEF discretization, obsolete keyword, see amont.

See also: convection\_deriv ([5.2.1](#))

Usage:

**amont\_old**

#### 5.2.14 Generic

Description: Keyword for generic calling of upwind and muscl convective scheme in VEF discretization. For muscl scheme, limiters and order for fluxes calculations have to be specified. The available limiters are : minmod - vanleer - vanalbada - chakravarthy - superbee, and the order of accuracy is 1 or 2. Note that chakravarthy is a non-symmetric limiter and superbee may engender results out of physical limits. By consequence, these two limiters are not recommended.

Examples:

```
convection { generic amount }
convection { generic muscl minmod 1 }
convection { generic muscl vanleer 2 }
```

In case of results out of physical limits with muscl scheme (due for instance to strong non-conformal velocity flow field), user can redefine in data file a lower order and a smoother limiter, as : convection { generic muscl minmod 1 }

See also: convection\_deriv ([5.2.1](#))

Usage:

**generic** type [ **limiteur** ] [ **ordre** ] [ **alpha** ]

where

- **type** str into ['amount', 'muscl', 'centre']: type of scheme
- **limiteur** str into ['minmod', 'vanleer', 'vanalbada', 'chakravarthy', 'superbee']: type of limiter
- **ordre** int into [1, 2, 3]: order of accuracy
- **alpha** float: alpha

#### 5.2.15 Muscl\_new

Description: Only for VEF discretization.

See also: convection\_deriv ([5.2.1](#))

Usage:

**muscl\_new**

#### 5.2.16 Kquick

Description: Only for VEF discretization.

See also: convection\_deriv ([5.2.1](#))

Usage:

**kquick**

#### 5.2.17 Muscl3

Description: Keyword for a scheme using a ponderation between muscl and center schemes in VEF.

See also: convection\_deriv ([5.2.1](#))

Usage:

**muscl3** {  
    [ **alpha** float]

}  
where

- **alpha** *float*: To weight the scheme centering with the factor floatant (between 0 (full centered) and 1 (muscl), by default 1).

### 5.2.18 Ef\_stab

Description: Keyword for a VEF convective scheme.

See also: convection\_deriv (5.2.1)

Usage:

```
ef_stab {
 [alpha float]
 [test int]
 [tdivu]
 [old]
 [volumes_etendus]
 [volumes_non_etendus]
 [amont_sous_zone str]
 [alpha_sous_zone listsous_zone_valeur]
}
```

where

- **alpha** *float*: To weight the scheme centering with the factor floatant (between 0 (full centered) and 1 (mix between upwind and centered), by default 1). For scalar equation, it is advised to use alpha=1 and for the momentum equation, alpha=0.2 is advised.
- **test** *int*: Developer option to compare old and new version of EF\_stab
- **tdivu** : To have the convective operator calculated as  $\text{div}(\text{TU}) - \text{TdivU} (= \text{UgradT})$ .
- **old** : To use old version of EF\_stab scheme (default no).
- **volumes\_etendus** : Option for the scheme to use the extended volumes (default, yes).
- **volumes\_non\_etendus** : Option for the scheme to not use the extended volumes (default, no).
- **amont\_sous\_zone** *str*: Option to degenerate EF\_stab scheme into Amont (upwind) scheme in the sub zone of name sz\_name. The sub zone may be located arbitrarily in the domain but the more often this option will be activated in a zone where EF\_stab scheme generates instabilities as for free outlet for example.
- **alpha\_sous\_zone** *listsous\_zone\_valeur* (5.2.19): Option to change locally the alpha value on N sub-zones named sub\_zone\_name\_I. Generally, it is used to prevent from a local divergence by increasing locally the alpha parameter.

### 5.2.19 Listsous\_zone\_valeur

Description: List of groups of two words.

See also: listobj (46.5)

Usage:

n object1 object2 ....

list of *sous\_zone\_valeur* (5.2.20)

### 5.2.20 Sous\_zone\_valeur

Description: Two words.

See also: [objet\\_lecture \(47\)](#)

Usage:

**sous\_zone valeur**

where

- **sous\_zone** *str*: sous zone
- **valeur** *float*: value

### 5.2.21 Btd

Description: Only for EF discretization.

See also: [convection\\_deriv \(5.2.1\)](#)

Usage:

**btd** {

**btd** *float*

**facteur** *float*

}

where

- **btd** *float*
- **facteur** *float*

### 5.2.22 Supg

Description: Only for EF discretization.

See also: [convection\\_deriv \(5.2.1\)](#)

Usage:

**supg** {

**facteur** *float*

}

where

- **facteur** *float*

### 5.2.23 Ale

Description: A convective scheme for ALE (Arbitrary Lagrangian-Eulerian) framework.

See also: [convection\\_deriv \(5.2.1\)](#)

Usage:

**ale opconv**

where

- **opconv** *bloc\_convection* (5.2): Choice between: `amont` and `muscl`

Example: `convection { ALE { amont } }`

#### 5.2.24 Rt

Description: Keyword to use RT projection for PINCP0RT discretization

See also: `convection_deriv` (5.2.1)

Usage:

**RT**

#### 5.2.25 Sensibility

Description: A convective scheme for the sensibility problem.

See also: `convection_deriv` (5.2.1)

Usage:

**sensibility opconv**

where

- **opconv** *bloc\_convection* (5.2): Choice between: `amont` and `muscl`

Example: `convection { Sensibility { amont } }`

### 5.3 Bloc\_diffusion

Description: `not_set`

See also: `objet_lecture` (47)

Usage:

**aco [ `opérateur` ] [ `op_implicite` ] `acof`**

where

- **aco** *str into ['']*: Opening curly bracket.
- **opérateur** *diffusion\_deriv* (5.3.1): if none is specified, the diffusive scheme used is a 2nd-order scheme.
- **op\_implicite** *op\_implicite* (5.3.23): To have diffusive implicitation, it use Uzawa algorithm. Very useful when viscosity has large variations.
- **acof** *str into ['']*: Closing curly bracket.

#### 5.3.1 Diffusion\_deriv

Description: `not_set`

See also: `objet_lecture` (47) `negligeable` (5.3.2) `option` (5.3.3) `stab` (5.3.4) `p1ncp1b` (5.3.5) `p1b` (5.3.6) `standard` (5.3.7) `turbulente` (5.3.9) `tenseur_Reynolds_externe` (5.3.22)

Usage:

**diffusion\_deriv**

### 5.3.2 Negligeable

Description: the diffusivity will not be taken into account

See also: `diffusion_deriv` (5.3.1)

Usage:

**negligeable**

### 5.3.3 Option

Description: `not_set`

See also: `diffusion_deriv` (5.3.1)

Usage:

**option bloc\_lecture**

where

- **bloc\_lecture** *bloc\_lecture* (3.2)

### 5.3.4 Stab

Description: keyword allowing consistent and stable calculations even in case of obtuse angle meshes.

See also: `diffusion_deriv` (5.3.1)

Usage:

**stab** {

```
[standard int]
[info int]
[new_jacobian int]
[nu int]
[nut int]
[nu_transp int]
[nut_transp int]
```

}

where

- **standard** *int*: to recover the same results as calculations made by standard laminar diffusion operator. However, no stabilization technique is used and calculations may be unstable when working with obtuse angle meshes (by default 0)
- **info** *int*: developer option to get the stabilizing ratio (by default 0)
- **new\_jacobian** *int*: when implicit time schemes are used, this option defines a new jacobian that may be more suitable to get stationary solutions (by default 0)
- **nu** *int*: (respectively `nut` 1) takes the molecular viscosity (resp. eddy viscosity) into account in the velocity gradient part of the diffusion expression (by default `nu`=1 and `nut`=1)
- **nut** *int*
- **nu\_transp** *int*: (respectively `nut_transp` 1) takes the molecular viscosity (resp. eddy viscosity) into account in the transposed velocity gradient part of the diffusion expression (by default `nu_transp`=0 and `nut_transp`=1)
- **nut\_transp** *int*

### 5.3.5 P1ncp1b

Description: not\_set

See also: diffusion\_deriv (5.3.1)

Usage:

### 5.3.6 P1b

Description: not\_set

See also: diffusion\_deriv (5.3.1)

Usage:

**p1b**

### 5.3.7 Standard

Description: A new keyword, intended for LES calculations, has been developed to optimise and parameterise each term of the diffusion operator. Remark:

1. This class requires to define a filtering operator : see solveur\_bar
2. The former (original) version: diffusion { } -which omitted some of the term of the diffusion operator- can be recovered by using the following parameters in the new class :  
diffusion { standard grad\_Ubar 0 nu 1 nut 1 nu\_transp 0 nut\_transp 1 filtrer\_resu 0 }.

See also: diffusion\_deriv (5.3.1)

Usage:

**standard** [ **mot1** ] [ **bloc\_diffusion\_standard** ]

where

- **mot1** str into ['default\_bar']: equivalent to grad\_Ubar 1 nu 1 nut 1 nu\_transp 1 nut\_transp 1 filtrer\_resu 1
- **bloc\_diffusion\_standard** bloc\_diffusion\_standard (5.3.8)

### 5.3.8 Bloc\_diffusion\_standard

Description: grad\_Ubar 1 makes the gradient calculated through the filtered values of velocity (P1-conform). nu 1 (respectively nut 1) takes the molecular viscosity (eddy viscosity) into account in the velocity gradient part of the diffusion expression.

nu\_transp 1 (respectively nut\_transp 1) takes the molecular viscosity (eddy viscosity) into account according in the TRANSPOSED velocity gradient part of the diffusion expression.

filtrer\_resu 1 allows to filter the resulting diffusive fluxes contribution.

See also: objet\_lecture (47)

Usage:

**mot1 val1 mot2 val2 mot3 val3 mot4 val4 mot5 val5 mot6 val6**

where

- **mot1** str into ['grad\_Ubar', 'nu', 'nut', 'nu\_transp', 'nut\_transp', 'filtrer\_resu']
- **val1** int into [0, 1]
- **mot2** str into ['grad\_Ubar', 'nu', 'nut', 'nu\_transp', 'nut\_transp', 'filtrer\_resu']



- **val2** *int* into [0, 1]
- **mot3** *str* into ['grad\_Ubar', 'nu', 'nut', 'nu\_transp', 'nut\_transp', 'filtrer\_resu']
- **val3** *int* into [0, 1]
- **mot4** *str* into ['grad\_Ubar', 'nu', 'nut', 'nu\_transp', 'nut\_transp', 'filtrer\_resu']
- **val4** *int* into [0, 1]
- **mot5** *str* into ['grad\_Ubar', 'nu', 'nut', 'nu\_transp', 'nut\_transp', 'filtrer\_resu']
- **val5** *int* into [0, 1]
- **mot6** *str* into ['grad\_Ubar', 'nu', 'nut', 'nu\_transp', 'nut\_transp', 'filtrer\_resu']
- **val6** *int* into [0, 1]

### 5.3.9 Turbulente

Description: Turbulent diffusion operator for multiphase problem

See also: `diffusion_deriv` (5.3.1)

Usage:

**turbulente** [ *type* ]

where

- **type** *type\_diffusion\_turbulente\_multiphase\_deriv* (5.3.10): Turbulence model for multiphase problem

### 5.3.10 Type\_diffusion\_turbulente\_multiphase\_deriv

Description: not\_set

See also: `objet_lecture` (47) `wale` (5.3.11) `SGDH` (5.3.12) `smago` (5.3.13) `l_melange` (5.3.14) `Prandtl` (5.3.15) `interfacial_area` (5.3.16) `multiple` (5.3.17) `k_omega` (5.3.20) `k_tau` (5.3.21)

Usage:

### 5.3.11 Wale

Description: LES WALE type.

See also: `type_diffusion_turbulente_multiphase_deriv` (5.3.10)

Usage:

**wale** {

    [ **cw** *float*]

}

where

- **cw** *float*: WALE's model constant. By default it is set to 0.5.

### 5.3.12 Sgdh

Description: not\_set

See also: `type_diffusion_turbulente_multiphase_deriv` (5.3.10)

Usage:

```
SGDH {
 [Pr_t float]
 [sigma_turbulent|sigma float]
 [no_alpha]
 [gas_turb]
}
```

where

- **Pr\_t** *float*
- **sigma\_turbulent**|**sigma** *float*
- **no\_alpha**
- **gas\_turb**

### 5.3.13 Smago

Description: LES Smagorinsky type.

See also: `type_diffusion_turbulente_multiphase_deriv` ([5.3.10](#))

Usage:

```
smago {
 [cs float]
}
```

where

- **cs** *float*: Smagorinsky's model constant. By default it is set to 0.18.

### 5.3.14 L\_melange

Description: `not_set`

See also: `type_diffusion_turbulente_multiphase_deriv` ([5.3.10](#))

Usage:

```
L_melange {
 l_melange float
}
```

where

- **l\_melange** *float*

### 5.3.15 Prandtl

Description: Scalar Prandtl model.

See also: `type_diffusion_turbulente_multiphase_deriv` ([5.3.10](#))

Usage:

**Prandtl** {

    [ **prandtl\_turbulent|pr\_t** *float*]

}

where

- **prandtl\_turbulent|pr\_t** *float*: Prandtl's model constant. By default it is set to 0.9.

### 5.3.16 Interfacial\_area

Synonymous: **aire\_interfaciale**

Description: not\_set

See also: `type_diffusion_turbulente_multiphase_deriv` ([5.3.10](#))

Usage:

**interfacial\_area** {

    [ **cstdiff** *float*]

    [ **ng2** *float*]

}

where

- **cstdiff** *float*: Kataoka diffusion model constant. By default it is set to 0.236.
- **ng2** *float*

### 5.3.17 Multiple

Description: See `TrioCFD_Pb_multiphase.pdf`

See also: `type_diffusion_turbulente_multiphase_deriv` ([5.3.10](#))

Usage:

**multiple** {

    [ **k\_omega** *type\_diffusion\_turbulente\_multiphase\_multiple\_deriv\_\_k\_omega*]

    [ **sato** *type\_diffusion\_turbulente\_multiphase\_multiple\_deriv\_\_sato*]

}

where

- **k\_omega** *type\_diffusion\_turbulente\_multiphase\_multiple\_deriv\_\_k\_omega* ([5.3.18](#)): first correlation
- **sato** *type\_diffusion\_turbulente\_multiphase\_multiple\_deriv\_\_sato* ([5.3.19](#))

### 5.3.18 K\_omega

Description: not\_set

See also: type\_diffusion\_turbulente\_multiphase\_multiple\_deriv ([47.3](#))

Usage:

### 5.3.19 Sato

Description: not\_set

See also: type\_diffusion\_turbulente\_multiphase\_multiple\_deriv ([47.3](#))

Usage:

### 5.3.20 K\_omega

Description: not\_set

See also: type\_diffusion\_turbulente\_multiphase\_deriv ([5.3.10](#))

Usage:

```
k_omega {
 [limiteurlimiter str]
 [sigma float]
 [beta_k float]
 [gas_turb]
}
```

where

- **limiteurlimiter** *str*
- **sigma** *float*
- **beta\_k** *float*
- **gas\_turb**

### 5.3.21 K\_tau

Description: not\_set

See also: type\_diffusion\_turbulente\_multiphase\_deriv ([5.3.10](#))

Usage:

```
k_tau {
 [limiteurlimiter str]
 [sigma float]
 [beta_k float]
}
```

where

- **limiteurlimiter** *str*
- **sigma** *float*
- **beta\_k** *float*

### 5.3.22 Tenseur\_reynolds\_externe

Description: Estimate the values of the Reynolds tensor.

See also: `diffusion_deriv` ([5.3.1](#))

Usage:

**tenseur\_Reynolds\_externe**

### 5.3.23 Op\_implicite

Description: `not_set`

See also: `objet_lecture` ([47](#))

Usage:

**implicite mot solveur**

where

- **implicite** *str* into [*'implicite'*]
- **mot** *str* into [*'solveur'*]
- **solveur** *solveur\_sys\_base* ([14.19](#))

## 5.4 Condinits

Description: Initial conditions.

See also: `listobj` ([46.5](#))

Usage:

{ *object1 object2 ....* }

list of *condinit* ([5.4.1](#))

### 5.4.1 Condinit

Description: Initial condition.

See also: `objet_lecture` ([47](#))

Usage:

**nom ch**

where

- **nom** *str*: Name of initial condition field.
- **ch** *champ\_base* ([20.1](#)): Type field and the initial values.

## 5.5 Sources

Description: The sources.

See also: `listobj` ([46.5](#))

Usage:

{ *object1 , object2 ....* }

list of *source\_base* ([41](#)) separated with ,

## 5.6 Parametre\_equation\_base

Description: Basic class for parametre\_equation

See also: objet\_lecture (47) parametre\_diffusion\_implicit (5.6.1) parametre\_implicit (5.6.2)

Usage:

### 5.6.1 Parametre\_diffusion\_implicit

Description: To specify additional parameters for the equation when using implicit diffusion

See also: parametre\_equation\_base (5.6)

Usage:

```
parametre_diffusion_implicit {
 [crank int into [0, 1]]
 [preconditionnement_diag int into [0, 1]]
 [niter_max_diffusion_implicit int]
 [seuil_diffusion_implicit float]
 [solveur solveur_sys_base]
}
```

where

- **crank** *int into [0, 1]*: Use (1) or not (0, default) a Crank Nicholson method for the diffusion implicitation algorithm. Setting crank to 1 increases the order of the algorithm from 1 to 2.
- **preconditionnement\_diag** *int into [0, 1]*: The CG used to solve the implicitation of the equation diffusion operator is not preconditioned by default. If this option is set to 1, a diagonal preconditioning is used. Warning: this option is not necessarily more efficient, depending on the treated case.
- **niter\_max\_diffusion\_implicit** *int*: Change the maximum number of iterations for the CG (Conjugate Gradient) algorithm when solving the diffusion implicitation of the equation.
- **seuil\_diffusion\_implicit** *float*: Change the threshold convergence value used by default for the CG resolution for the diffusion implicitation of this equation.
- **solveur** *solveur\_sys\_base* (14.19): Method (different from the default one, Conjugate Gradient) to solve the linear system.

### 5.6.2 Parametre\_implicit

Description: Keyword to change for this equation only the parameter of the implicit scheme used to solve the problem.

See also: parametre\_equation\_base (5.6)

Usage:

```
parametre_implicit {
 [seuil_convergence_implicit float]
 [seuil_convergence_solveur float]
 [solveur solveur_sys_base]
 [resolution_explicite]
 [equation_non_resolue]
 [equation_frequence_resolue str]
}
```

}  
where

- **seuil\_convergence\_implicite** *float*: Keyword to change for this equation only the value of `seuil_convergence_implicite` used in the implicit scheme.
- **seuil\_convergence\_solveur** *float*: Keyword to change for this equation only the value of `seuil_convergence_solveur` used in the implicit scheme
- **solveur** *solveur\_sys\_base* (14.19): Keyword to change for this equation only the solver used in the implicit scheme
- **resolution\_explicite** : To solve explicitly the equation whereas the scheme is an implicit scheme.
- **equation\_non\_resolue** : Keyword to specify that the equation is not solved.
- **equation\_frequence\_resolue** *str*: Keyword to specify that the equation is solved only every *n* time steps (*n* is an integer or given by a time-dependent function *f(t)*).

## 5.7 Conduction\_ibm

Description: IBM Heat equation.

Keyword Discretize should have already been used to read the object.  
See also: Conduction (5.1)

Usage:

**Conduction\_ibm** *str*

**Read** *str* {

```
[correction_variable_initiale int]
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
```

}  
where

- **correction\_variable\_initiale** *int*: Modify initial variable
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation

- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.8 Convection\_diffusion\_concentration\_turbulent\_ft\_disc

Description: equation\_non\_resolue

Keyword Discretize should have already been used to read the object.

See also: convection\_diffusion\_concentration\_turbulent (5.32)

Usage:

**Convection\_Diffusion\_Concentration\_Turbulent\_FT\_Disc** *str*

```
Read str {
 [equation_interface str]
 phase int into [0, 1]
 [option str]
 [equations_source_chimie n word1 word2 ... wordn]
 [modele_cinetique int]
 [equation_nu_t str]
 [constante_cinetique float]
 [modele_turbulence modele_turbulence_scal_base]
 [nom_inconnue str]
 [alias str]
 [masse_molaire float]
 [is_multi_scalar_diffusion|is_multi_scalar]
 [disable_equation_residual int]
 [convection bloc_convection]
 [diffusion bloc_diffusion]
 [boundary_conditions|conditions_limites condlims]
 [initial_conditions|conditions_initiales condinits]
 [sources sources]
 [ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
 [parametre_equation parametre_equation_base]
 [equation_non_resolue str]
 [renommer_equation str]
}
```

where

- **equation\_interface** *str*: this is the name of the interface tracking equation to watch. The scalar will not diffuse through the interface of this equation.
- **phase** *int* into [0, 1]: tells whether the scalar must be confined in phase 0 or in phase 1
- **option** *str*: Experimental features used to prevent the concentration to leak through the interface between phases due to numerical diffusion.  
RIEN: do nothing  
RAMASSE\_MIETTES\_SIMPLE: at each timestep, this algorithm takes all the mass located in the opposite phase and spreads it uniformly in the given phase.
- **equations\_source\_chimie** *n word1 word2 ... wordn*: This term specifies the name of the concentration equation of the reagents. It should be specified only in the bloc that concerns the convection/diffusion equation of the product.



- **modele\_cinetique** *int*: This is the keyword that the user defines for the reaction model that he wants to use. Four reaction models are currently offered (1 to 4). Model 1 is the default one and is based on the laminar rate formulation. Model 2 employs an LES diffusive EDC formulation. Model 3 defines an LES variance formulation. Model 4 is a mix between models 2 and 3.
- **equation\_nu\_t** *str*: This specifies the name of the hydraulic equation used which defines the turbulent (basically SGS) viscosity.
- **constante\_cinetique** *float*: This is the constant kinetic rate of the reaction and is used for the laminar model 1 only.
- **modele\_turbulence** *modele\_turbulence\_scal\_base* (29) for inheritance: Turbulence model to be used in the constituent transport equations. The only model currently available is Schmidt.
- **nom\_inconnue** *str* for inheritance: Keyword *Nom\_inconnue* will rename the unknown of this equation with the given name. In the postprocessing part, the concentration field will be accessible with this name. This is useful if you want to track more than one concentration (otherwise, only the concentration field in the first concentration equation can be accessed).
- **alias** *str* for inheritance
- **masse\_molaire** *float* for inheritance
- **is\_multi\_scalar\_diffusion** *is\_multi\_scalar* for inheritance: Flag to activate the multi\_scalar diffusion operator
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions** *conditions\_limites* *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions** *conditions\_initiales* *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if *equation\_non\_resolue* keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.9 Convection\_diffusion\_espece\_binaire\_turbulent\_qc

Description: Species conservation equation for a binary quasi-compressible fluid as well as the associated turbulence model equations.

Keyword Discretize should have already been used to read the object.

See also: *convection\_diffusion\_espece\_binaire\_QC* (5.33)

Usage:

**Convection\_Diffusion\_Espece\_Binaire\_Turbulent\_QC** *str*

**Read** *str* {

[ **modele\_turbulence** *modele\_turbulence\_scal\_base*]  
[ **disable\_equation\_residual** *int*]

```

[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]

```

}

where

- **modele\_turbulence** *modele\_turbulence\_scal\_base* (29): Turbulence model for the species conservation equation.
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.10 Convection\_diffusion\_temperature\_sensibility

Description: Energy sensitivity equation (temperature diffusion convection)

Keyword Discretize should have already been used to read the object.

See also: convection\_diffusion\_temperature (5.39)

Usage:

**Convection\_Diffusion\_Temperature\_sensibility** *str*

**Read** *str* {

```

[convection_sensibility convection_deriv]
velocity_state bloc_lecture
temperature_state bloc_lecture
uncertain_variable bloc_lecture
[polynomial_chaos float]
[penalisation_l2_ftd bloc_lecture]
[disable_equation_residual int]

```

```

[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limit condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]

```

}

where

- **convection\_sensibility** *convection\_deriv* (5.2.1): Choice between: *amont* and *muscl*  
Example: `convection { Sensibility { amont } }`
- **velocity\_state** *bloc\_lecture* (3.2): Block to indicate the state problem. Between the braces, you must specify the key word '*pb\_champ\_evaluateur*' then the name of the state problem and the velocity unknown  
Example: `velocity_state { pb_champ_evaluateur pb_state velocity }`
- **temperature\_state** *bloc\_lecture* (3.2): Block to indicate the state problem. Between the braces, you must specify the key word '*pb\_champ\_evaluateur*' then the name of the state problem and the temperature unknown  
Example: `velocity_state { pb_champ_evaluateur pb_state temperature }`
- **uncertain\_variable** *bloc\_lecture* (3.2): Block to indicate the name of the uncertain variable. Between the braces, you must specify the name of the unknown variable (choice between: *temperature*, *beta\_th*, *boussinesq\_temperature*, *Cp* and *lambda* ).  
Example: `uncertain_variable { temperature }`
- **polynomial\_chaos** *float*: It is the method that we will use to study the sensitivity of the
- **penalisation\_l2\_ftd** *bloc\_lecture* (3.2) for inheritance: to activate or not (the default is Direct Forcing method) the Penalized Direct Forcing method to impose the specified temperature on the solid-fluid interface.
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limit** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if *equation\_non\_resolue* keyword is used. Example: The Navier-Stokes equations are not solved between time *t0* and *t1*.  
`Navier_Sokes_Standard`  
`{ equation_non_resolue (t>t0)*(t<t1) }`
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.11 Echelle\_temporelle\_turbulente

Description: Turbulent Dissipation time scale equation for a turbulent mono/multi-phase problem (available in TrioCFD)

Keyword Discretize should have already been used to read the object.

See also: eqn\_base (5.45)

Usage:

**Echelle\_temporelle\_turbulente** *str*

**Read** *str* {

```
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
```

}

where

- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.12 Energie\_multiphase

Description: Internal energy conservation equation for a multi-phase problem where the unknown is the temperature

Keyword Discretize should have already been used to read the object.

See also: eqn\_base (5.45)

Usage:

**Energie\_Multiphase** *str*

**Read** *str* {

```
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
```

}

where

- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

### 5.13 Energie\_multiphase\_h

Description: Internal energy conservation equation for a multi-phase problem where the unknown is the enthalpy

Keyword Discretize should have already been used to read the object.

See also: eqn\_base (5.45)

Usage:

**Energie\_Multiphase\_h** *str*

**Read** *str* {

```
[disable_equation_residual int]
```

```

[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]

```

}

where

- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.14 Energie\_cinetique\_turbulente

Description: Turbulent kinetic Energy conservation equation for a turbulent mono/multi-phase problem (available in TrioCFD)

Keyword Discretize should have already been used to read the object.

See also: eqn\_base (5.45)

Usage:

**Energie\_cinetique\_turbulente** *str*

**Read** *str* {

```

[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]

```

```

[equation_non_resolue str]
[renommer_equation str]
}

```

where

- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.15 Energie\_cinetique\_turbulente\_wit

Description: Bubble Induced Turbulent kinetic Energy equation for a turbulent multi-phase problem (available in TrioCFD)

Keyword Discretize should have already been used to read the object.

See also: eqn\_base (5.45)

Usage:

**Energie\_cinetique\_turbulente\_WIT** *str*

**Read** *str* {

```

[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]

```

}

where

- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step

- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.16 Masse\_multiphase

Description: Mass conservation equation for a multi-phase problem where the unknown is the alpha (void fraction)

Keyword Discretize should have already been used to read the object.

See also: eqn\_base (5.45)

Usage:

**Masse\_Multiphase** *str*

```
Read str {
 [disable_equation_residual int]
 [convection bloc_convection]
 [diffusion bloc_diffusion]
 [boundary_conditions|conditions_limites condlims]
 [initial_conditions|conditions_initiales condinits]
 [sources sources]
 [ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
 [parametre_equation parametre_equation_base]
 [equation_non_resolue str]
 [renommer_equation str]
}
```

where

- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)



- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.17 Navier\_stokes\_aposteriori

Description: Modification of the Navier\_Stokes\_standard class in order to accept the estimateur\_aposteriori post-processing. To post-process estimateur\_aposteriori, add this keyword into the list of fields to be post-processed. This estimator whill generate a map of aposteriori error estimators; it is defined on each mesh cell and is a measure of the local discretisation error. This will serve for adaptive mesh refinement

Keyword Discretize should have already been used to read the object.

See also: navier\_stokes\_standard (5.57)

Usage:

**Navier\_Stokes\_Aposteriori** *str*

**Read** *str* {

```
[solveur_pression solveur_sys_base]
[dt_projection deuxmots]
[traitement_particulier traitement_particulier]
[seuil_divU floatfloat]
[solveur_bar solveur_sys_base]
[projection_initiale int]
[postraiter_gradient_pression_sans_masse]
[methode_calcul_pression_initiale str into ['avec_les_cl', 'avec_sources', 'avec_sources_et-
_operateurs', 'sans_rien']]
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
```

}

where

- **solveur\_pression** *solveur\_sys\_base* (14.19) for inheritance: Linear pressure system resolution method.
- **dt\_projection** *deuxmots* (4.9.1) for inheritance: nb value : This keyword checks every nb time-steps the equality of velocity divergence to zero. value is the criteria convergency for the solver used.
- **traitement\_particulier** *traitement\_particulier* (5.18) for inheritance: Keyword to post-process particular values.

- **seuil\_divU** *floatfloat* (5.19) for inheritance: value factor : this keyword is intended to minimise the number of iterations during the pressure system resolution. The convergence criteria during this step ('seuil' in solveur\_pression) is dynamically adapted according to the mass conservation. At  $t_n$ , the linear system  $Ax=B$  is considered as solved if the residual  $\|Ax-B\| < \text{seuil}(t_n)$ . For  $t_{n+1}$ , the threshold value  $\text{seuil}(t_{n+1})$  will be evaluated as:  

```
If (lmax(DivU)*dtl<value)
Seuil(tn+1)= Seuil(tn)*factor
Else
Seuil(tn+1)= Seuil(tn)*factor
Endif
```

The first parameter (value) is the mass evolution the user is ready to accept per timestep, and the second one (factor) is the factor of evolution for 'seuil' (for example 1.1, so 10)
- **solveur\_bar** *solveur\_sys\_base* (14.19) for inheritance: This keyword is used to define when filtering operation is called (typically for EF convective scheme, standard diffusion operator and Source\_Qdm\_lambdaup ). A file (solveur.bar) is then created and used for inversion procedure. Syntax is the same then for pressure solver (GCP is required for multi-processor calculations and, in a general way, for big meshes).
- **projection\_initiale** *int* for inheritance: Keyword to suppress, if boolean equals 0, the initial projection which checks  $\text{Div}U=0$ . By default, boolean equals 1.
- **postraiter\_gradient\_pression\_sans\_masse** for inheritance: Avoid mass matrix multiplication for the gradient postprocessing
- **methode\_calcul\_pression\_initiale** *str into ['avec\_les\_cl', 'avec\_sources', 'avec\_sources\_et\_operateurs', 'sans\_rien']* for inheritance: Keyword to select an option for the pressure calculation before the first time step. Options are : avec\_les\_cl (default option lapP=0 is solved with Neuman boundary conditions on pressure if any), avec\_sources (lapP=f is solved with Neuman boundaries conditions and f integrating the source terms of the Navier-Stokes equations) and avec\_sources\_et\_operateurs (lapP=f is solved as with the previous option avec\_sources but f integrating also some operators of the Navier-Stokes equations). The two last options are useful and sometime necessary when source terms are implicated when using an implicit time scheme to solve the Navier-Stokes equations.
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limite** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time  $t_0$  and  $t_1$ .  

```
Navier_Sokes_Standard
{ equation_non_resolue (t>t0)*(t<t1) }
```
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.18 Traitement\_particulier

Description: Auxiliary class to post-process particular values.

See also: `objet_lecture` ([47](#))

Usage:

**aco trait\_part acof**  
where

- **aco** *str* into `['']`: Opening curly bracket.
- **trait\_part** *traitement\_particulier\_base* ([5.18.1](#)): Type of *traitement\_particulier*.
- **acof** *str* into `['']`: Closing curly bracket.

### 5.18.1 Traitement\_particulier\_base

Description: Basic class to post-process particular values.

See also: `objet_lecture` ([47](#)) `profils_thermo` ([5.18.2](#)) `canal` ([5.18.3](#)) `ec` ([5.18.4](#)) `temperature` ([5.18.5](#)) `thi` ([5.18.6](#)) `chmoy_faceperio` ([5.18.8](#)) `brech` ([5.18.9](#)) `ceg` ([5.18.10](#))

Usage:

### 5.18.2 Profils\_thermo

Description: non documente

See also: `traitement_particulier_base` ([5.18.1](#))

Usage:

**profils\_thermo bloc**  
where

- **bloc** *bloc\_lecture* ([3.2](#))

### 5.18.3 Canal

Description: Keyword for statistics on a periodic plane channel.

See also: `traitement_particulier_base` ([5.18.1](#))

Usage:

**canal** {  
    [ **dt\_impr\_moy\_spat** *float*]  
    [ **dt\_impr\_moy\_temp** *float*]  
    [ **debut\_stat** *float*]  
    [ **fin\_stat** *float*]  
    [ **pulsation\_w** *float*]  
    [ **nb\_points\_par\_phase** *int*]  
    [ **reprise** *str*]  
}

where

- **dt\_impr\_moy\_spat** *float*: Period to print the spatial average (default value is 1e6).
- **dt\_impr\_moy\_temp** *float*: Period to print the temporal average (default value is 1e6).
- **debut\_stat** *float*: Time to start the temporal averaging (default value is 1e6).

- **fin\_stat** *float*: Time to end the temporal averaging (default value is 1e6).
- **pulsation\_w** *float*: Pulsation for phase averaging (in case of pulsating forcing term) (no default value).
- **nb\_points\_par\_phase** *int*: Number of samples to represent phase average all along a period (no default value).
- **reprise** *str*: val\_moy\_temp\_XXXXXX.sauv : Keyword to resume a calculation with previous averaged quantities.

Note that for thermal and turbulent problems, averages on temperature and turbulent viscosity are automatically calculated. To resume a calculation with phase averaging, val\_moy\_temp\_XXXXXX.sauv-\_phase file is required on the directory where the job is submitted (this last file will be then automatically loaded by TRUST).

#### 5.18.4 Ec

Description: Keyword to print total kinetic energy into the referential linked to the domain (keyword Ec). In the case where the domain is moving into a Galilean referential, the keyword Ec\_dans\_repere\_fixe will print total kinetic energy in the Galilean referential whereas Ec will print the value calculated into the moving referential linked to the domain

See also: traitement\_particulier\_base ([5.18.1](#))

Usage:

```
ec {
 [Ec]
 [Ec_dans_repere_fixe]
 [periode float]
}
```

where

- **Ec**
- **Ec\_dans\_repere\_fixe**
- **periode** *float*: periode is the keyword to set the period of printing into the file datafile\_Ec.son or datafile\_Ec\_dans\_repere\_fixe.son.

#### 5.18.5 Temperature

Description: not\_set

See also: traitement\_particulier\_base ([5.18.1](#))

Usage:

```
temperature {
 bord str
 direction int
}
```

where

- **bord** *str*
- **direction** *int*

### 5.18.6 Thi

Description: Keyword for a THI (Homogeneous Isotropic Turbulence) calculation.

See also: `traitement_particulier_base` (5.18.1) `thi_thermo` (5.18.7)

Usage:

```
thi {

 init_Ec int
 [val_Ec float]
 [facon_init int into [0, 1]]
 [calc_spectre int into [0, 1]]
 [periode_calc_spectre float]
 [spectre_3D int into [0, 1]]
 [spectre_1D int into [0, 1]]
 [conservation_Ec]
 [longueur_boite float]

}
```

where

- **init\_Ec** *int*: Keyword to renormalize initial velocity so that kinetic energy equals to the value given by keyword `val_Ec`.
- **val\_Ec** *float*: Keyword to impose a value for kinetic energy by velocity renormalized if `init_Ec` value is 1.
- **facon\_init** *int into [0, 1]*: Keyword to specify how kinetic energy is computed (0 or 1).
- **calc\_spectre** *int into [0, 1]*: Calculate or not the spectrum of kinetic energy.  
Files called `Sorties_THI` are written with inside four columns :  
time:t global\_kinetic\_energy:Ec enstrophy:D skewness:S  
If `calc_spectre` is set to 1, a file `Sorties_THI2_2` is written with three columns :  
time:t kinetic\_energy\_at\_kc=32 enstrophy\_at\_kc=32  
If `calc_spectre` is set to 1, a file `spectre_XXXXX` is written with two columns at each time `XXXXX` :  
frequency:k energy:E(k).
- **periode\_calc\_spectre** *float*: Period for calculating spectrum of kinetic energy
- **spectre\_3D** *int into [0, 1]*: Calculate or not the 3D spectrum
- **spectre\_1D** *int into [0, 1]*: Calculate or not the 1D spectrum
- **conservation\_Ec** : If set to 1, velocity field will be changed as to have a constant kinetic energy (default 0)
- **longueur\_boite** *float*: Length of the calculation domain

### 5.18.7 Thi\_thermo

Description: Treatment for the temperature field.

It offers the possibility to :

- evaluate the probability density function on temperature field,
- give in a file the temperature field for a future spectral analysis,
- monitor the evolution of the max and min temperature on the whole domain.

See also: `thi` (5.18.6)

Usage:

```
thi_thermo {

 init_Ec int
```

```

[val_Ec float]
[facon_init int into [0, 1]]
[calc_spectre int into [0, 1]]
[periode_calc_spectre float]
[spectre_3D int into [0, 1]]
[spectre_1D int into [0, 1]]
[conservation_Ec]
[longueur_boite float]
}
where

```

- **init\_Ec** *int* for inheritance: Keyword to renormalize initial velocity so that kinetic energy equals to the value given by keyword **val\_Ec**.
- **val\_Ec** *float* for inheritance: Keyword to impose a value for kinetic energy by velocity renormalized if **init\_Ec** value is 1.
- **facon\_init** *int into [0, 1]* for inheritance: Keyword to specify how kinetic energy is computed (0 or 1).
- **calc\_spectre** *int into [0, 1]* for inheritance: Calculate or not the spectrum of kinetic energy.  
Files called **Sorties\_THI** are written with inside four columns :  
time:t global\_kinetic\_energy:Ec enstrophy:D skewness:S  
If **calc\_spectre** is set to 1, a file **Sorties\_THI2\_2** is written with three columns :  
time:t kinetic\_energy\_at\_kc=32 enstrophy\_at\_kc=32  
If **calc\_spectre** is set to 1, a file **spectre\_XXXXX** is written with two columns at each time **XXXXX** :  
frequency:k energy:E(k).
- **periode\_calc\_spectre** *float* for inheritance: Period for calculating spectrum of kinetic energy
- **spectre\_3D** *int into [0, 1]* for inheritance: Calculate or not the 3D spectrum
- **spectre\_1D** *int into [0, 1]* for inheritance: Calculate or not the 1D spectrum
- **conservation\_Ec** for inheritance: If set to 1, velocity field will be changed as to have a constant kinetic energy (default 0)
- **longueur\_boite** *float* for inheritance: Length of the calculation domain

### 5.18.8 Chmoy\_faceperio

Description: non documente

See also: **traitement\_particulier\_base** (5.18.1)

Usage:

**chmoy\_faceperio bloc**  
where

- **bloc** *bloc\_lecture* (3.2)

### 5.18.9 Brech

Description: non documente

See also: **traitement\_particulier\_base** (5.18.1)

Usage:

**brech bloc**  
where

- **bloc** *bloc\_lecture* (3.2)

#### 5.18.10 Ceg

Description: Keyword for a CEG ( Gas Entrainment Criteria) calculation. An objective is deepening gas entrainment on the free surface. Numerical analysis can be performed to predict the hydraulic and geometric conditions that can handle gas entrainment from the free surface.

See also: `traitement_particulier_base` ([5.18.1](#))

Usage:

```
ceg {
 frontiere str
 t_deb float
 [t_fin float]
 [dt_post float]
 haspi float
 [debug int]
 [areva ceg_areva]
 [cea_jaea ceg_cea_jaea]
```

```
}
```

where

- **frontiere** *str*: To specify the boundaries conditions representing the free surfaces
- **t\_deb** *float*: value of the CEG's initial calculation time
- **t\_fin** *float*: not\_set time during which the CEG's calculation was stopped
- **dt\_post** *float*: periode refers to the printing period, this value is expressed in seconds
- **haspi** *float*: The suction height required to calculate AREVA's criterion
- **debug** *int*
- **areva** *ceg\_areva* ([5.18.11](#)): AREVA's criterion
- **cea\_jaea** *ceg\_cea\_jaea* ([5.18.12](#)): CEA\_JAEA's criterion

#### 5.18.11 Ceg\_areva

Description: not\_set

See also: `objet_lecture` ([47](#))

Usage:

```
{
 [c float]
```

```
}
```

where

- **c** *float*

#### 5.18.12 Ceg\_cea\_jaea

Description: not\_set

See also: `objet_lecture` ([47](#))

Usage:

```
{
```

```

[normalise int]
[nb_mailles_mini int]
[min_critere_q_sur_max_critere_q float]
}
where

```

- **normalise** *int*: renormalize (1) or not (0) values alpha and gamma
- **nb\_mailles\_mini** *int*: Sets the minimum number of cells for the detection of a vortex.
- **min\_critere\_q\_sur\_max\_critere\_q** *float*: Is an optional keyword used to correct the minimum values of Q's criterion taken into account in the detection of a vortex

## 5.19 Floatfloat

Description: Two reals.

See also: `objet_lecture` ([47](#))

Usage:

**a b**  
where

- **a** *float*: First real.
- **b** *float*: Second real.

## 5.20 Navier\_stokes\_ftd\_ijk

Description: Navier-Stokes equations.

Keyword Discretize should have already been used to read the object.

See also: `eqn_base` ([5.45](#))

Usage:

**Navier\_Stokes\_FTD\_IJK** *str*

**Read** *str* {

```

 multigrid_solver multigrid_solver
 [vitesse_entree float]
 [expression_vx_init str]
 [expression_vy_init str]
 [expression_vz_init str]
 [expression_p_init str]
 [velocity_diffusion_op str]
 [velocity_convection_op str]
 [fichier_reprise_vitesse str]
 [timestep_reprise_vitesse str]
 [disable_solveur_poisson]
 [disable_diffusion_qdm]
 [disable_convection_qdm]
 [frozen_velocity str]
 [velocity_reset str]
 [resolution_fluctuations]
 [harmonic_nu_in_diff_operator]
 [use_inv_rho_for_mass_solver_and_calculer_rho_v str]

```



```

[use_inv_rho_in_poisson_solver]
[diffusion_alternative str]
[test_etapes_et_bilan]
[ajout_init_a_reprise str]
[improved_initial_pressure_guess str]
[include_pressure_gradient_in_ustar str]
[upstream_dir int]
[vitesse_upstream float]
[expression_vitesse_upstream str]
[upstream_stencil int]
[nb_diam_upstream float]
[nb_diam_ortho_shear_perio str]
[vol_bulle_monodisperse str]
[diam_bulle_monodisperse str]
[coeff_evol_volume str]
[vol_bulles str]
[reprise_vap_velocity_tmoy str]
[reprise_liq_velocity_tmoy str]
[disable_source_interf]
[harmonic_nu_in_calc_with_indicatrice]
[refuse_patch_conservation_qdm_rk3_source_interf]
[suppression_rejets str]
[p_seuil_max float]
[p_seuil_min float]
[coef_ammortissement float]
[coef_immobilisation float]
[expression_derivee_force str]
[terme_force_init str]
[correction_force str]
[compute_force_init]
[expression_variable_source_x str]
[expression_variable_source_y str]
[expression_variable_source_z str]
[facteur_variable_source_init str]
[expression_derivee_facteur_variable_source str]
[expression_potential_phi str]
[forage bloc_lecture]
[corrections_qdm bloc_lecture]
[coef_mean_force float]
[coef_force_time_n float]
[coef_rayon_force_rappel float]
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]

```

}

where

- **multigrid\_solver** *multigrid\_solver* (3.97)
- **vitesse\_entree** *float*: Velocity to prescribe at inlet
- **expression\_vx\_init** *str*: initial field for x-velocity component (parser of x,y,z)
- **expression\_vy\_init** *str*: initial field for y-velocity component (parser of x,y,z)
- **expression\_vz\_init** *str*: initial field for z-velocity component (parser of x,y,z)
- **expression\_p\_init** *str*: initial pressure field (optional)
- **velocity\_diffusion\_op** *str*: Type of velocity diffusion scheme
- **velocity\_convection\_op** *str*: Type of velocity convection scheme
- **fichier\_reprise\_vitesse** *str*
- **timestep\_reprise\_vitesse** *str*
- **disable\_solveur\_poisson** : Disable pressure poisson solver
- **disable\_diffusion\_qdm** : Disable diffusion operator in momentum
- **disable\_convection\_qdm** : Disable convection operator in momentum
- **frozen\_velocity** *str*
- **velocity\_reset** *str*
- **resolution\_fluctuations** : Disable pressure poisson solver
- **harmonic\_nu\_in\_diff\_operator** : Disable pressure poisson solver
- **use\_inv\_rho\_for\_mass\_solver\_and\_calculer\_rho\_v** *str*
- **use\_inv\_rho\_in\_poisson\_solver**
- **diffusion\_alternative** *str*
- **test\_etapes\_et\_bilan**
- **ajout\_init\_a\_reprise** *str*
- **improved\_initial\_pressure\_guess** *str*
- **include\_pressure\_gradient\_in\_ustar** *str*
- **upstream\_dir** *int*: Direction to prescribe the velocity
- **vitesse\_upstream** *float*: Velocity to prescribe at 'nb\_diam\_upstream\_' before bubble 0.
- **expression\_vitesse\_upstream** *str*: Analytical expression to set the upstream velocity
- **upstream\_stencil** *int*: Width on which the velocity is set
- **nb\_diam\_upstream** *float*: Number of bubble diameters upstream of bubble 0 to prescribe the velocity.
- **nb\_diam\_ortho\_shear\_perio** *str*
- **vol\_bulle\_monodisperse** *str*
- **diam\_bulle\_monodisperse** *str*
- **coeff\_evol\_volume** *str*
- **vol\_bulles** *str*
- **reprise\_vap\_velocity\_tmoy** *str*
- **reprise\_liq\_velocity\_tmoy** *str*
- **disable\_source\_interf** : Disable computation of the interfacial source term
- **harmonic\_nu\_in\_calc\_with\_indicatrice** : Disable pressure poisson solver
- **refuse\_patch\_conservation\_qdm\_rk3\_source\_interf** : experimental Keyword, not for use
- **suppression\_rejetons** *str*
- **p\_seuil\_max** *float*: not\_set, default 10000000
- **p\_seuil\_min** *float*: not\_set, default -10000000
- **coef\_ammortissement** *float*
- **coef\_immobilisation** *float*
- **expression\_derivee\_force** *str*: expression of the time-derivative of the X-component of a source-term (see **terme\_force\_ini** for the initial value). **terme\_force\_ini** : initial value of the X-component of the source term (see **expression\_derivee\_force** for time evolution)
- **terme\_force\_init** *str*
- **correction\_force** *str*
- **compute\_force\_init**
- **expression\_variable\_source\_x** *str*
- **expression\_variable\_source\_y** *str*
- **expression\_variable\_source\_z** *str*

- **facteur\_variable\_source\_init** *str*
- **expression\_derivee\_facteur\_variable\_source** *str*
- **expression\_potential\_phi** *str*: parser to define phi and make a momentum source Nabla phi.
- **forage** *bloc\_lecture* (3.2)
- **corrections\_qdm** *bloc\_lecture* (3.2)
- **coef\_mean\_force** *float*
- **coef\_force\_time\_n** *float*
- **coef\_rayon\_force\_rappel** *float*
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limite** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.21 Navier\_stokes\_turbulent\_ale

Description: Resolution of hydraulic turbulent Navier-Stokes eq. on mobile domain (ALE)

Keyword Discretize should have already been used to read the object.

See also: Navier\_Stokes\_std\_ALE (5.24)

Usage:

**Navier\_Stokes\_Turbulent\_ALE** *str*

**Read** *str* {

```
[modele_turbulence modele_turbulence_hyd_deriv]
[solveur_pression solveur_sys_base]
[dt_projection deuxmots]
[traitement_particulier traitement_particulier]
[seuil_divU floatfloat]
[solveur_bar solveur_sys_base]
[projection_initiale int]
[postraiter_gradient_pression_sans_masse]
[methode_calcul_pression_initiale str into ['avec_les_cl', 'avec_sources', 'avec_sources_et-
_operateurs', 'sans_rien']]
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
```

```

[boundary_conditions|conditions_limités condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
}

```

where

- **modele\_turbulence** *modele\_turbulence\_hyd\_deriv* (5.22): Turbulence model for Navier-Stokes equations.
- **solveur\_pression** *solveur\_sys\_base* (14.19) for inheritance: Linear pressure system resolution method.
- **dt\_projection** *deuxmots* (4.9.1) for inheritance: nb value : This keyword checks every nb time-steps the equality of velocity divergence to zero. value is the criteria convergency for the solver used.
- **traitement\_particulier** *traitement\_particulier* (5.18) for inheritance: Keyword to post-process particular values.
- **seuil\_divU** *floatfloat* (5.19) for inheritance: value factor : this keyword is intended to minimise the number of iterations during the pressure system resolution. The convergence criteria during this step ('seuil' in *solveur\_pression*) is dynamically adapted according to the mass conservation. At  $t_n$ , the linear system  $Ax=B$  is considered as solved if the residual  $\|Ax-B\| < \text{seuil}(t_n)$ . For  $t_{n+1}$ , the threshold value  $\text{seuil}(t_{n+1})$  will be evaluated as:  
 If (  $\text{lmax}(\text{DivU}) \cdot dt < \text{value}$  )  
 Seuil( $t_{n+1}$ ) = Seuil( $t_n$ ) \* factor  
 Else  
 Seuil( $t_{n+1}$ ) = Seuil( $t_n$ ) \* factor  
 Endif  
 The first parameter (value) is the mass evolution the user is ready to accept per timestep, and the second one (factor) is the factor of evolution for 'seuil' (for example 1.1, so 10)
- **solveur\_bar** *solveur\_sys\_base* (14.19) for inheritance: This keyword is used to define when filtering operation is called (typically for EF convective scheme, standard diffusion operator and *Source\_Qdm\_lambdaup*). A file (*solveur.bar*) is then created and used for inversion procedure. Syntax is the same then for pressure solver (GCP is required for multi-processor calculations and, in a general way, for big meshes).
- **projection\_initiale** *int* for inheritance: Keyword to suppress, if boolean equals 0, the initial projection which checks  $\text{DivU}=0$ . By default, boolean equals 1.
- **postraiter\_gradient\_pression\_sans\_masse** for inheritance: Avoid mass matrix multiplication for the gradient postprocessing
- **methode\_calcul\_pression\_initiale** *str into* ['avec\_les\_cl', 'avec\_sources', 'avec\_sources\_et\_operateurs', 'sans\_rien'] for inheritance: Keyword to select an option for the pressure calculation before the first time step. Options are : *avec\_les\_cl* (default option  $\text{lapP}=0$  is solved with Neuman boundary conditions on pressure if any), *avec\_sources* ( $\text{lapP}=f$  is solved with Neuman boundaries conditions and  $f$  integrating the source terms of the Navier-Stokes equations) and *avec\_sources\_et\_operateurs* ( $\text{lapP}=f$  is solved as with the previous option *avec\_sources* but  $f$  integrating also some operators of the Navier-Stokes equations). The two last options are useful and sometime necessary when source terms are implicated when using an implicit time scheme to solve the Navier-Stokes equations.
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limités condlims** (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales condinits** (5.4) for inheritance: Initial conditions.

- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.22 Modele\_turbulence\_hyd\_deriv

Description: Basic class for turbulence model for Navier-Stokes equations.

See also: objet\_lecture (47) mod\_turb\_hyd\_ss\_maille (5.22.2) null (5.22.18) mod\_turb\_hyd\_rans (5.22.19)

Usage:

```
modele_turbulence_hyd_deriv {
 [turbulence_paro_i turbulence_paro_i_base]
 [dt_impr_ustar float]
 [dt_impr_ustar_mean_only dt_impr_ustar_mean_only]
 [nut_max float]
 [correction_visco_turb_pour_controle_pas_de_temps]
 [correction_visco_turb_pour_controle_pas_de_temps_parametre float]
}
```

where

- **turbulence\_paro\_i** *turbulence\_paro\_i\_base* (44): Keyword to set the wall law.
- **dt\_impr\_ustar** *float*: This keyword is used to print the values ( $U^+$ ,  $d^+$ ,  $u^*$ ) obtained with the wall laws into a file named datafile\_ProblemName\_Ustar.face and periode refers to the printing period, this value is expressed in seconds.
- **dt\_impr\_ustar\_mean\_only** *dt\_impr\_ustar\_mean\_only* (5.22.1): This keyword is used to print the mean values of  $u^*$  ( obtained with the wall laws) on each boundary, into a file named datafile\_ProblemName\_Ustar\_mean\_only.out. periode refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword boundaries, all the boundaries will be considered. If you use it, you must specify nb\_boundaries which is the number of boundaries on which you want to calculate the mean values of  $u^*$ , then you have to specify their names.
- **nut\_max** *float*: Upper limitation of turbulent viscosity (default value 1.e8).
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps** : Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is calculated so that diffusive time-step is equal or higher than convective time-step. For a stationary flow, the correction for turbulent viscosity should apply only during the first time steps and not when permanent state is reached. To check that, we could post process the corr\_visco\_turb field which is the correction of turbulent viscosity: it should be 1. on the whole domain.
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps\_parametre** *float*: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is the ratio between diffusive time-step and convective time-step is higher or equal to the given value [0-1]

### 5.22.1 Dt\_impr\_ustar\_mean\_only

Description: not\_set

See also: objet\_lecture (47)

Usage:

```
{

 dt_impr float
 [boundaries n word1 word2 ... wordn]

}
```

where

- **dt\_impr** float
- **boundaries** n word1 word2 ... wordn

### 5.22.2 Mod\_turb\_hyd\_ss\_maille

Description: Class for sub-grid turbulence model for Navier-Stokes equations.

See also: modele\_turbulence\_hyd\_deriv (5.22) sous\_maille\_wale (5.22.4) sous\_maille\_smago (5.22.5) longueur\_melange (5.22.6) sous\_maille\_selectif\_mod (5.22.7) sous\_maille\_selectif (5.22.10) sous\_maille\_1elt (5.22.11) sous\_maille\_axi (5.22.13) sous\_maille\_smago\_filtre (5.22.14) sous\_maille\_smago\_dyn (5.22.15) combinaison (5.22.16) sous\_maille (5.22.17)

Usage:

```
mod_turb_hyd_ss_maille {

 [formulation_a_nb_points form_a_nb_points]
 [longueur_maille str into ['volume', 'volume_sans_lissage', 'scotti', 'arrete']]
 [turbulence_paroit turbulence_paroit_base]
 [dt_impr_ustar float]
 [dt_impr_ustar_mean_only dt_impr_ustar_mean_only]
 [nut_max float]
 [correction_visco_turb_pour_controle_pas_de_temps]
 [correction_visco_turb_pour_controle_pas_de_temps_parametre float]

}
```

where

- **formulation\_a\_nb\_points** form\_a\_nb\_points (5.22.3): The structure fonction is calculated on nb points and we should add the 2 directions (0:OX, 1:OY, 2:OZ) constituting the homegeneity planes. Example for channel flows, planes parallel to the walls.
- **longueur\_maille** str into ['volume', 'volume\_sans\_lissage', 'scotti', 'arrete']: Different ways to calculate the characteristic length may be specified :
  - volume : It is the default option. Characteristic length is based on the cubic root of the volume cells. A smoothing procedure is applied to avoid discontinuities of this quantity in VEF from a cell to another.
  - volume\_sans\_lissage : For VEF only. Characteristic length is based on the cubic root of the volume cells (without smoothing procedure).
  - scotti : Characteristic length is based on the cubic root of the volume cells and the Scotti correction is applied to take into account the stretching of the cell in the case of anisotropic meshes.
  - arete : For VEF only. Characteristic length relies on the max edge (+ smoothing procedure) is taken into account.

- **turbulence\_paro** *turbulence\_paro\_base* (44) for inheritance: Keyword to set the wall law.
- **dt\_impr\_ustar** *float* for inheritance: This keyword is used to print the values ( $U^+$ ,  $d^+$ ,  $u^*$ ) obtained with the wall laws into a file named `datafile_ProblemName_Ustar.face` and `periode` refers to the printing period, this value is expressed in seconds.
- **dt\_impr\_ustar\_mean\_only** *dt\_impr\_ustar\_mean\_only* (5.22.1) for inheritance: This keyword is used to print the mean values of  $u^*$  (obtained with the wall laws) on each boundary, into a file named `datafile_ProblemName_Ustar_mean_only.out`. `periode` refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword `boundaries`, all the boundaries will be considered. If you use it, you must specify `nb_boundaries` which is the number of boundaries on which you want to calculate the mean values of  $u^*$ , then you have to specify their names.
- **nut\_max** *float* for inheritance: Upper limitation of turbulent viscosity (default value  $1.e8$ ).
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps** for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is calculated so that diffusive time-step is equal or higher than convective time-step. For a stationary flow, the correction for turbulent viscosity should apply only during the first time steps and not when permanent state is reached. To check that, we could post process the `corr_visco_turb` field which is the correction of turbulent viscosity: it should be 1. on the whole domain.
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps\_parametre** *float* for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is the ratio between diffusive time-step and convective time-step is higher or equal to the given value [0-1]

### 5.22.3 Form\_a\_nb\_points

Description: The structure function is calculated on `nb` points and we should add the 2 directions (0:OX, 1:OY, 2:OZ) constituting the homogeneity planes. Example for channel flows, planes parallel to the walls.

See also: `objet_lecture` (47)

Usage:

**nb dir1 dir2**

where

- **nb** *int into [4]*: Number of points.
- **dir1** *int*: First direction.
- **dir2** *int*: Second direction.

### 5.22.4 Sous\_maille\_wale

Description: This is the WALE-model. It is a new sub-grid scale model for eddy-viscosity in LES that has the following properties :

- it goes naturally to 0 at the wall (it doesn't need any information on the wall position or geometry)
- it has the proper wall scaling in  $o(y^3)$  in the vicinity of the wall
- it reproduces correctly the laminar to turbulent transition.

See also: `mod_turb_hyd_ss_maille` (5.22.2)

Usage:

**sous\_maille\_wale** {

```
[cw float]
[formulation_a_nb_points form_a_nb_points]
[longueur_maille str into ['volume', 'volume_sans_lissage', 'scotti', 'arrete']]
[turbulence_paro turbulence_paro_base]
```

```

[dt_impr_ustar float]
[dt_impr_ustar_mean_only dt_impr_ustar_mean_only]
[nut_max float]
[correction_visco_turb_pour_controle_pas_de_temps]
[correction_visco_turb_pour_controle_pas_de_temps_parametre float]
}
where

```

- **cw** *float*: The unique parameter (constant) of the WALE-model (by default value 0.5).
- **formulation\_a\_nb\_points** *form\_a\_nb\_points* (5.22.3) for inheritance: The structure fonction is calculated on nb points and we should add the 2 directions (0:OX, 1:OY, 2:OZ) constituting the homogeneity planes. Example for channel flows, planes parallel to the walls.
- **longueur\_maille** *str into ['volume', 'volume\_sans\_lissage', 'scotti', 'arrete']* for inheritance: Different ways to calculate the characteristic length may be specified :  
*volume* : It is the default option. Characteristic length is based on the cubic root of the volume cells. A smoothing procedure is applied to avoid discontinuities of this quantity in VEF from a cell to another.  
*volume\_sans\_lissage* : For VEF only. Characteristic length is based on the cubic root of the volume cells (without smoothing procedure).  
*scotti* : Characteristic length is based on the cubic root of the volume cells and the Scotti correction is applied to take into account the stretching of the cell in the case of anisotropic meshes.  
*arete* : For VEF only. Characteristic length relies on the max edge (+ smoothing procedure) is taken into account.
- **turbulence\_paro** *turbulence\_paro\_base* (44) for inheritance: Keyword to set the wall law.
- **dt\_impr\_ustar** *float* for inheritance: This keyword is used to print the values (U +, d+, u\*) obtained with the wall laws into a file named datafile\_ProblemName\_Ustar.face and periode refers to the printing period, this value is expressed in seconds.
- **dt\_impr\_ustar\_mean\_only** *dt\_impr\_ustar\_mean\_only* (5.22.1) for inheritance: This keyword is used to print the mean values of u\* ( obtained with the wall laws) on each boundary, into a file named datafile\_ProblemName\_Ustar\_mean\_only.out. periode refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword boundaries, all the boundaries will be considered. If you use it, you must specify nb\_boundaries which is the number of boundaries on which you want to calculate the mean values of u\*, then you have to specify their names.
- **nut\_max** *float* for inheritance: Upper limitation of turbulent viscosity (default value 1.e8).
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps** for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is calculated so that diffusive time-step is equal or higher than convective time-step. For a stationary flow, the correction for turbulent viscosity should apply only during the first time steps and not when permanent state is reached. To check that, we could post process the corr\_visco\_turb field which is the correction of turbulent viscosity: it should be 1. on the whole domain.
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps\_parametre** *float* for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is the ratio between diffusive time-step and convective time-step is higher or equal to the given value [0-1]

### 5.22.5 Sous\_maille\_smago

Description: Smagorinsky sub-grid turbulence model.

$$\text{Nut} = \text{Cs1} * \text{Cs1} * \text{l} * \sqrt{2 * \text{S} * \text{S}}$$

$$\text{K} = \text{Cs2} * \text{Cs2} * \text{l} * 2 * \text{S}$$

See also: mod\_turb\_hyd\_ss\_maille (5.22.2)



Usage:

```
sous_maille_smago {
 [cs float]
 [formulation_a_nb_points form_a_nb_points]
 [longueur_maille str into ['volume', 'volume_sans_lissage', 'scotti', 'arrete']]
 [turbulence_paroit turbulence_paroit_base]
 [dt_impr_ustar float]
 [dt_impr_ustar_mean_only dt_impr_ustar_mean_only]
 [nut_max float]
 [correction_visco_turb_pour_controle_pas_de_temps]
 [correction_visco_turb_pour_controle_pas_de_temps_parametre float]
}
where
```

- **cs float**: This is an optional keyword and the value is used to set the constant used in the Smagorinsky model (This is currently only valid for Smagorinsky models and it is set to 0.18 by default) .
- **formulation\_a\_nb\_points form\_a\_nb\_points** (5.22.3) for inheritance: The structure function is calculated on nb points and we should add the 2 directions (0:OX, 1:OY, 2:OZ) constituting the homogeneity planes. Example for channel flows, planes parallel to the walls.
- **longueur\_maille str into ['volume', 'volume\_sans\_lissage', 'scotti', 'arrete']** for inheritance: Different ways to calculate the characteristic length may be specified :  
 volume : It is the default option. Characteristic length is based on the cubic root of the volume cells. A smoothing procedure is applied to avoid discontinuities of this quantity in VEF from a cell to another.  
 volume\_sans\_lissage : For VEF only. Characteristic length is based on the cubic root of the volume cells (without smoothing procedure).  
 scotti : Characteristic length is based on the cubic root of the volume cells and the Scotti correction is applied to take into account the stretching of the cell in the case of anisotropic meshes.  
 arete : For VEF only. Characteristic length relies on the max edge (+ smoothing procedure) is taken into account.
- **turbulence\_paroit turbulence\_paroit\_base** (44) for inheritance: Keyword to set the wall law.
- **dt\_impr\_ustar float** for inheritance: This keyword is used to print the values ( $U^+$ ,  $d^+$ ,  $u^*$ ) obtained with the wall laws into a file named datafile\_ProblemName\_Ustar.face and periode refers to the printing period, this value is expressed in seconds.
- **dt\_impr\_ustar\_mean\_only dt\_impr\_ustar\_mean\_only** (5.22.1) for inheritance: This keyword is used to print the mean values of  $u^*$  ( obtained with the wall laws) on each boundary, into a file named datafile\_ProblemName\_Ustar\_mean\_only.out. periode refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword boundaries, all the boundaries will be considered. If you use it, you must specify nb\_boundaries which is the number of boundaries on which you want to calculate the mean values of  $u^*$ , then you have to specify their names.
- **nut\_max float** for inheritance: Upper limitation of turbulent viscosity (default value 1.e8).
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps** for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is calculated so that diffusive time-step is equal or higher than convective time-step. For a stationary flow, the correction for turbulent viscosity should apply only during the first time steps and not when permanent state is reached. To check that, we could post process the corr\_visco\_turb field which is the correction of turbulent viscosity: it should be 1. on the whole domain.
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps\_parametre float** for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is the ratio between diffusive time-step and convective time-step is higher or equal to the given value [0-1]

### 5.22.6 Longueur\_melange

Description: This model is based on mixing length modelling. For a non academic configuration, formulation used in the code can be expressed basically as :

$$\nu_{u,t} = (Kappa.y)^2.dU/dy$$

Till a maximum distance (dmax) set by the user in the data file, y is set equal to the distance from the wall (dist\_w) calculated previously and saved in file Wall\_length.xyz. [see Distance\_paro keyword]

Then (from y=dmax), y decreases as an exponential function :  $y = dmax * \exp[-2. * (dist\_w - dmax) / dmax]$

See also: mod\_turb\_hyd\_ss\_maille (5.22.2)

Usage:

```
longueur_melange {
 [canalx float]
 [tuyauz float]
 [dmax float]
 [fichier str]
 [fichier_ecriture_K_Eps str]
 [formulation_a_nb_points form_a_nb_points]
 [longueur_maille str into ['volume', 'volume_sans_lissage', 'scotti', 'arrete']]
 [turbulence_paro turbulence_paro_base]
 [dt_impr_ustar float]
 [dt_impr_ustar_mean_only dt_impr_ustar_mean_only]
 [nut_max float]
 [correction_visco_turb_pour_controle_pas_de_temps]
 [correction_visco_turb_pour_controle_pas_de_temps_parametre float]
}
where
```

- **canalx float**: [height] : plane channel according to Ox direction (for the moment, formulation in the code relies on fixed height : H=2).
- **tuyauz float**: [diameter] : pipe according to Oz direction (for the moment, formulation in the code relies on fixed diameter : D=2).
- **dmax float**: Maximum distance.
- **fichier str**
- **fichier\_ecriture\_K\_Eps str**: When a resume with k-epsilon model is envisaged, this keyword allows to generate external MED-format file with evaluation of k and epsilon quantities (based on eddy turbulent viscosity and turbulent characteristic length returned by mixing length model). The frequency of the MED file print is set equal to dt\_impr\_ustar. Moreover, k-eps MED field is automatically saved at the last time step. MED file is then used for resuming a K-Epsilon calculation with the Champ\_Fonc\_Med keyword.
- **formulation\_a\_nb\_points form\_a\_nb\_points** (5.22.3) for inheritance: The structure function is calculated on nb points and we should add the 2 directions (0:OX, 1:OY, 2:OZ) constituting the homogeneity planes. Example for channel flows, planes parallel to the walls.
- **longueur\_maille str into ['volume', 'volume\_sans\_lissage', 'scotti', 'arrete']** for inheritance: Different ways to calculate the characteristic length may be specified :
  - volume : It is the default option. Characteristic length is based on the cubic root of the volume cells. A smoothing procedure is applied to avoid discontinuities of this quantity in VEF from a cell to another.
  - volume\_sans\_lissage : For VEF only. Characteristic length is based on the cubic root of the volume cells (without smoothing procedure).
  - scotti : Characteristic length is based on the cubic root of the volume cells and the Scotti correction is applied to take into account the stretching of the cell in the case of anisotropic meshes.

arete : For VEF only. Characteristic length relies on the max edge (+ smoothing procedure) is taken into account.

- **turbulence\_paro** *turbulence\_paro\_base* (44) for inheritance: Keyword to set the wall law.
- **dt\_impr\_ustar** *float* for inheritance: This keyword is used to print the values ( $U^+$ ,  $d^+$ ,  $u^*$ ) obtained with the wall laws into a file named `datafile_ProblemName_Ustar.face` and `periode` refers to the printing period, this value is expressed in seconds.
- **dt\_impr\_ustar\_mean\_only** *dt\_impr\_ustar\_mean\_only* (5.22.1) for inheritance: This keyword is used to print the mean values of  $u^*$  (obtained with the wall laws) on each boundary, into a file named `datafile_ProblemName_Ustar_mean_only.out`. `periode` refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword `boundaries`, all the boundaries will be considered. If you use it, you must specify `nb_boundaries` which is the number of boundaries on which you want to calculate the mean values of  $u^*$ , then you have to specify their names.
- **nut\_max** *float* for inheritance: Upper limitation of turbulent viscosity (default value 1.e8).
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps** for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is calculated so that diffusive time-step is equal or higher than convective time-step. For a stationary flow, the correction for turbulent viscosity should apply only during the first time steps and not when permanent state is reached. To check that, we could post process the `corr_visco_turb` field which is the correction of turbulent viscosity: it should be 1. on the whole domain.
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps\_parametre** *float* for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is the ratio between diffusive time-step and convective time-step is higher or equal to the given value [0-1]

### 5.22.7 Sous\_maille\_selectif\_mod

Description: Selective structure sub-grid function model (modified).

See also: `mod_turb_hyd_ss_maille` (5.22.2)

Usage:

```
sous_maille_selectif_mod {
 [thi deuxentiers]
 [canal floatentier]
 [formulation_a_nb_points form_a_nb_points]
 [longueur_maille str into ['volume', 'volume_sans_lissage', 'scotti', 'arrete']]
 [turbulence_paro turbulence_paro_base]
 [dt_impr_ustar float]
 [dt_impr_ustar_mean_only dt_impr_ustar_mean_only]
 [nut_max float]
 [correction_visco_turb_pour_controle_pas_de_temps]
 [correction_visco_turb_pour_controle_pas_de_temps_parametre float]
}
```

where

- **thi** *deuxentiers* (5.22.8): For homogeneous isotropic turbulence (THI), two integers  $k_i$  and  $k_c$  are needed in VDF (not in VEF).
- **canal** *floatentier* (5.22.9): `h` `dir_faces_paro`: For a channel flow, the half width  $h$  and the orientation of the wall `dir_faces_paro` are needed.
- **formulation\_a\_nb\_points** *form\_a\_nb\_points* (5.22.3) for inheritance: The structure function is calculated on `nb_points` and we should add the 2 directions (0:OX, 1:OY, 2:OZ) constituting the homogeneity planes. Example for channel flows, planes parallel to the walls.

- **longueur\_maille** *str* into [*'volume'*, *'volume\_sans\_lissage'*, *'scotti'*, *'arrete'*] for inheritance: Different ways to calculate the characteristic length may be specified :  
*volume* : It is the default option. Characteristic length is based on the cubic root of the volume cells. A smoothing procedure is applied to avoid discontinuities of this quantity in VEF from a cell to another.  
*volume\_sans\_lissage* : For VEF only. Characteristic length is based on the cubic root of the volume cells (without smoothing procedure).  
*scotti* : Characteristic length is based on the cubic root of the volume cells and the Scotti correction is applied to take into account the stretching of the cell in the case of anisotropic meshes.  
*arete* : For VEF only. Characteristic length relies on the max edge (+ smoothing procedure) is taken into account.
- **turbulence\_paro** *turbulence\_paro\_base* (44) for inheritance: Keyword to set the wall law.
- **dt\_impr\_ustar** *float* for inheritance: This keyword is used to print the values ( $U^+$ ,  $d^+$ ,  $u^*$ ) obtained with the wall laws into a file named `datafile_ProblemName_Ustar.face` and *periode* refers to the printing period, this value is expressed in seconds.
- **dt\_impr\_ustar\_mean\_only** *dt\_impr\_ustar\_mean\_only* (5.22.1) for inheritance: This keyword is used to print the mean values of  $u^*$  ( obtained with the wall laws) on each boundary, into a file named `datafile_ProblemName_Ustar_mean_only.out`. *periode* refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword *boundaries*, all the boundaries will be considered. If you use it, you must specify *nb\_boundaries* which is the number of boundaries on which you want to calculate the mean values of  $u^*$ , then you have to specify their names.
- **nut\_max** *float* for inheritance: Upper limitation of turbulent viscosity (default value 1.e8).
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps** for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is calculated so that diffusive time-step is equal or higher than convective time-step. For a stationary flow, the correction for turbulent viscosity should apply only during the first time steps and not when permanent state is reached. To check that, we could post process the `corr_visco_turb` field which is the correction of turbulent viscosity: it should be 1. on the whole domain.
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps\_parametre** *float* for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is the ratio between diffusive time-step and convective time-step is higher or equal to the given value [0-1]

### 5.22.8 Deuxentiers

Description: Two integers.

See also: `objet_lecture` (47)

Usage:

**int1 int2**

where

- **int1** *int*: First integer.
- **int2** *int*: Second integer.

### 5.22.9 Floatentier

Description: A real and an integer.

See also: `objet_lecture` (47)

Usage:

**the\_float the\_int**

where

- **the\_float** *float*: Real.
- **the\_int** *int*: Integer.

#### 5.22.10 Sous\_maille\_selectif

Description: Selective structure sub-grid function model (a filter is applied to the structure function).

See also: `mod_turb_hyd_ss_maille` (5.22.2)

Usage:

```
sous_maille_selectif {
 [formulation_a_nb_points form_a_nb_points]
 [longueur_maille str into ['volume', 'volume_sans_lissage', 'scotti', 'arrete']]
 [turbulence_paro turbulence_paro_base]
 [dt_impr_ustar float]
 [dt_impr_ustar_mean_only dt_impr_ustar_mean_only]
 [nut_max float]
 [correction_visco_turb_pour_controle_pas_de_temps]
 [correction_visco_turb_pour_controle_pas_de_temps_parametre float]
}
```

where

- **formulation\_a\_nb\_points** *form\_a\_nb\_points* (5.22.3) for inheritance: The structure function is calculated on nb points and we should add the 2 directions (0:OX, 1:OY, 2:OZ) constituting the homogeneity planes. Example for channel flows, planes parallel to the walls.
- **longueur\_maille** *str into* ['volume', 'volume\_sans\_lissage', 'scotti', 'arrete'] for inheritance: Different ways to calculate the characteristic length may be specified :  
 volume : It is the default option. Characteristic length is based on the cubic root of the volume cells. A smoothing procedure is applied to avoid discontinuities of this quantity in VEF from a cell to another.  
 volume\_sans\_lissage : For VEF only. Characteristic length is based on the cubic root of the volume cells (without smoothing procedure).  
 scotti : Characteristic length is based on the cubic root of the volume cells and the Scotti correction is applied to take into account the stretching of the cell in the case of anisotropic meshes.  
 arete : For VEF only. Characteristic length relies on the max edge (+ smoothing procedure) is taken into account.
- **turbulence\_paro** *turbulence\_paro\_base* (44) for inheritance: Keyword to set the wall law.
- **dt\_impr\_ustar** *float* for inheritance: This keyword is used to print the values (U +, d+, u\*) obtained with the wall laws into a file named `datafile_ProblemName_Ustar.face` and `periode` refers to the printing period, this value is expressed in seconds.
- **dt\_impr\_ustar\_mean\_only** *dt\_impr\_ustar\_mean\_only* (5.22.1) for inheritance: This keyword is used to print the mean values of u\* ( obtained with the wall laws) on each boundary, into a file named `datafile_ProblemName_Ustar_mean_only.out`. `periode` refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword `boundaries`, all the boundaries will be considered. If you use it, you must specify `nb_boundaries` which is the number of boundaries on which you want to calculate the mean values of u\*, then you have to specify their names.
- **nut\_max** *float* for inheritance: Upper limitation of turbulent viscosity (default value 1.e8).
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps** for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is calculated so that diffusive time-step is equal or higher than convective time-step. For a stationary flow, the correction for turbulent viscosity should apply only during the first time steps and not when permanent state is reached. To check that, we could post process the `corr_visco_turb` field which is the correction of turbulent viscosity: it should be 1. on the whole domain.

- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps\_parametre** *float* for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is the ratio between diffusive time-step and convective time-step is higher or equal to the given value [0-1]

### 5.22.11 Sous\_maille\_1elt

Description: Turbulence model sous\_maille\_1elt.

See also: mod\_turb\_hyd\_ss\_maille (5.22.2) sous\_maille\_1elt\_selectif\_mod (5.22.12)

Usage:

```
sous_maille_1elt {
 [formulation_a_nb_points form_a_nb_points]
 [longueur_maille str into ['volume', 'volume_sans_lissage', 'scotti', 'arrete']]
 [turbulence_paroit turbulence_paroit_base]
 [dt_impr_ustar float]
 [dt_impr_ustar_mean_only dt_impr_ustar_mean_only]
 [nut_max float]
 [correction_visco_turb_pour_controle_pas_de_temps]
 [correction_visco_turb_pour_controle_pas_de_temps_parametre float]
}
where
```

- **formulation\_a\_nb\_points** *form\_a\_nb\_points* (5.22.3) for inheritance: The structure function is calculated on nb points and we should add the 2 directions (0:OX, 1:OY, 2:OZ) constituting the homogeneity planes. Example for channel flows, planes parallel to the walls.
- **longueur\_maille** *str into ['volume', 'volume\_sans\_lissage', 'scotti', 'arrete']* for inheritance: Different ways to calculate the characteristic length may be specified :  
 volume : It is the default option. Characteristic length is based on the cubic root of the volume cells. A smoothing procedure is applied to avoid discontinuities of this quantity in VEF from a cell to another.  
 volume\_sans\_lissage : For VEF only. Characteristic length is based on the cubic root of the volume cells (without smoothing procedure).  
 scotti : Characteristic length is based on the cubic root of the volume cells and the Scotti correction is applied to take into account the stretching of the cell in the case of anisotropic meshes.  
 arete : For VEF only. Characteristic length relies on the max edge (+ smoothing procedure) is taken into account.
- **turbulence\_paroit** *turbulence\_paroit\_base* (44) for inheritance: Keyword to set the wall law.
- **dt\_impr\_ustar** *float* for inheritance: This keyword is used to print the values ( $U^+$ ,  $d^+$ ,  $u^*$ ) obtained with the wall laws into a file named datafile\_ProblemName\_Ustar.face and periode refers to the printing period, this value is expressed in seconds.
- **dt\_impr\_ustar\_mean\_only** *dt\_impr\_ustar\_mean\_only* (5.22.1) for inheritance: This keyword is used to print the mean values of  $u^*$  ( obtained with the wall laws) on each boundary, into a file named datafile\_ProblemName\_Ustar\_mean\_only.out. periode refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword boundaries, all the boundaries will be considered. If you use it, you must specify nb\_boundaries which is the number of boundaries on which you want to calculate the mean values of  $u^*$ , then you have to specify their names.
- **nut\_max** *float* for inheritance: Upper limitation of turbulent viscosity (default value 1.e8).
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps** for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is calculated so that diffusive time-step is equal or higher than convective time-step. For a stationary

flow, the correction for turbulent viscosity should apply only during the first time steps and not when permanent state is reached. To check that, we could post process the `corr_visco_turb` field which is the correction of turbulent viscosity: it should be 1. on the whole domain.

- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps\_parametre** *float* for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is the ratio between diffusive time-step and convective time-step is higher or equal to the given value [0-1]

### 5.22.12 Sous\_maille\_1elt\_selectif\_mod

Description: Turbulence model `sous_maille_1elt_selectif_mod`.

See also: `sous_maille_1elt` (5.22.11)

Usage:

```
sous_maille_1elt_selectif_mod {
 [formulation_a_nb_points form_a_nb_points]
 [longueur_maille str into ['volume', 'volume_sans_lissage', 'scotti', 'arrete']]
 [turbulence_paroit turbulence_paroit_base]
 [dt_impr_ustar float]
 [dt_impr_ustar_mean_only dt_impr_ustar_mean_only]
 [nut_max float]
 [correction_visco_turb_pour_controle_pas_de_temps]
 [correction_visco_turb_pour_controle_pas_de_temps_parametre float]
}
```

where

- **formulation\_a\_nb\_points** *form\_a\_nb\_points* (5.22.3) for inheritance: The structure fonction is calculated on nb points and we should add the 2 directions (0:OX, 1:OY, 2:OZ) constituting the homegeneity planes. Example for channel flows, planes parallel to the walls.
- **longueur\_maille** *str into ['volume', 'volume\_sans\_lissage', 'scotti', 'arrete']* for inheritance: Different ways to calculate the characteristic length may be specified :  
*volume* : It is the default option. Characteristic length is based on the cubic root of the volume cells. A smoothing procedure is applied to avoid discontinuities of this quantity in VEF from a cell to another.  
*volume\_sans\_lissage* : For VEF only. Characteristic length is based on the cubic root of the volume cells (without smoothing procedure).  
*scotti* : Characteristic length is based on the cubic root of the volume cells and the Scotti correction is applied to take into account the stretching of the cell in the case of anisotropic meshes.  
*arete* : For VEF only. Characteristic length relies on the max edge (+ smoothing procedure) is taken into account.
- **turbulence\_paroit** *turbulence\_paroit\_base* (44) for inheritance: Keyword to set the wall law.
- **dt\_impr\_ustar** *float* for inheritance: This keyword is used to print the values ( $U^+$ ,  $d^+$ ,  $u^*$ ) obtained with the wall laws into a file named `datafile_ProblemName_Ustar.face` and `periode` refers to the printing period, this value is expressed in seconds.
- **dt\_impr\_ustar\_mean\_only** *dt\_impr\_ustar\_mean\_only* (5.22.1) for inheritance: This keyword is used to print the mean values of  $u^*$  ( obtained with the wall laws) on each boundary, into a file named `datafile_ProblemName_Ustar_mean_only.out`. `periode` refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword `boundaries`, all the boundaries will be considered. If you use it, you must specify `nb_boundaries` which is the number of boundaries on which you want to calculate the mean values of  $u^*$ , then you have to specify their names.
- **nut\_max** *float* for inheritance: Upper limitation of turbulent viscosity (default value 1.e8).



- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps** for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is calculated so that diffusive time-step is equal or higher than convective time-step. For a stationary flow, the correction for turbulent viscosity should apply only during the first time steps and not when permanent state is reached. To check that, we could post process the `corr_visco_turb` field which is the correction of turbulent viscosity; it should be 1. on the whole domain.
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps\_parametre** *float* for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is the ratio between diffusive time-step and convective time-step is higher or equal to the given value [0-1]

### 5.22.13 Sous\_maille\_axi

Description: Structure sub-grid function turbulence model available in cylindrical co-ordinates.

See also: `mod_turb_hyd_ss_maille` (5.22.2)

Usage:

```
sous_maille_axi {
 [formulation_a_nb_points form_a_nb_points]
 [longueur_maille str into ['volume', 'volume_sans_lissage', 'scotti', 'arrete']]
 [turbulence_paro turbulence_paro_base]
 [dt_impr_ustar float]
 [dt_impr_ustar_mean_only dt_impr_ustar_mean_only]
 [nut_max float]
 [correction_visco_turb_pour_controle_pas_de_temps]
 [correction_visco_turb_pour_controle_pas_de_temps_parametre float]
```

}

where

- **formulation\_a\_nb\_points** *form\_a\_nb\_points* (5.22.3) for inheritance: The structure fonction is calculated on nb points and we should add the 2 directions (0:OX, 1:OY, 2:OZ) constituting the homegeneity planes. Example for channel flows, planes parallel to the walls.
- **longueur\_maille** *str into ['volume', 'volume\_sans\_lissage', 'scotti', 'arrete']* for inheritance: Different ways to calculate the characteristic length may be specified :  
*volume* : It is the default option. Characteristic length is based on the cubic root of the volume cells. A smoothing procedure is applied to avoid discontinuities of this quantity in VEF from a cell to another.  
*volume\_sans\_lissage* : For VEF only. Characteristic length is based on the cubic root of the volume cells (without smoothing procedure).  
*scotti* : Characteristic length is based on the cubic root of the volume cells and the Scotti correction is applied to take into account the stretching of the cell in the case of anisotropic meshes.  
*arete* : For VEF only. Characteristic length relies on the max edge (+ smoothing procedure) is taken into account.
- **turbulence\_paro** *turbulence\_paro\_base* (44) for inheritance: Keyword to set the wall law.
- **dt\_impr\_ustar** *float* for inheritance: This keyword is used to print the values ( $U^+$ ,  $d^+$ ,  $u^*$ ) obtained with the wall laws into a file named `datafile_ProblemName_Ustar.face` and *periode* refers to the printing period, this value is expressed in seconds.
- **dt\_impr\_ustar\_mean\_only** *dt\_impr\_ustar\_mean\_only* (5.22.1) for inheritance: This keyword is used to print the mean values of  $u^*$  ( obtained with the wall laws) on each boundary, into a file named `datafile_ProblemName_Ustar_mean_only.out`. *periode* refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword *boundaries*, all the boundaries will



be considered. If you use it, you must specify `nb_boundaries` which is the number of boundaries on which you want to calculate the mean values of  $u^*$ , then you have to specify their names.

- **nut\_max** *float* for inheritance: Upper limitation of turbulent viscosity (default value 1.e8).
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps** for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is calculated so that diffusive time-step is equal or higher than convective time-step. For a stationary flow, the correction for turbulent viscosity should apply only during the first time steps and not when permanent state is reached. To check that, we could post process the `corr_visco_turb` field which is the correction of turbulent viscosity; it should be 1. on the whole domain.
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps\_parametre** *float* for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is the ratio between diffusive time-step and convective time-step is higher or equal to the given value [0-1]

### 5.22.14 Sous\_maille\_smago\_filtre

Description: Smagorinsky sub-grid turbulence model should be used with low-filter.

See also: `mod_turb_hyd_ss_maille` ([5.22.2](#))

Usage:

```
sous_maille_smago_filtre {
 [formulation_a_nb_points form_a_nb_points]
 [longueur_maille str into ['volume', 'volume_sans_lissage', 'scotti', 'arrete']]
 [turbulence_paro turbulence_paro_base]
 [dt_impr_ustar float]
 [dt_impr_ustar_mean_only dt_impr_ustar_mean_only]
 [nut_max float]
 [correction_visco_turb_pour_controle_pas_de_temps]
 [correction_visco_turb_pour_controle_pas_de_temps_parametre float]
}
```

where

- **formulation\_a\_nb\_points** *form\_a\_nb\_points* ([5.22.3](#)) for inheritance: The structure function is calculated on `nb_points` and we should add the 2 directions (0:OX, 1:OY, 2:OZ) constituting the homogeneity planes. Example for channel flows, planes parallel to the walls.
- **longueur\_maille** *str into ['volume', 'volume\_sans\_lissage', 'scotti', 'arrete']* for inheritance: Different ways to calculate the characteristic length may be specified :  
*volume* : It is the default option. Characteristic length is based on the cubic root of the volume cells. A smoothing procedure is applied to avoid discontinuities of this quantity in VEF from a cell to another.  
*volume\_sans\_lissage* : For VEF only. Characteristic length is based on the cubic root of the volume cells (without smoothing procedure).  
*scotti* : Characteristic length is based on the cubic root of the volume cells and the Scotti correction is applied to take into account the stretching of the cell in the case of anisotropic meshes.  
*arete* : For VEF only. Characteristic length relies on the max edge (+ smoothing procedure) is taken into account.
- **turbulence\_paro** *turbulence\_paro\_base* ([44](#)) for inheritance: Keyword to set the wall law.
- **dt\_impr\_ustar** *float* for inheritance: This keyword is used to print the values ( $U^+$ ,  $d^+$ ,  $u^*$ ) obtained with the wall laws into a file named `datafile_ProblemName_Ustar.face` and `periode` refers to the printing period, this value is expressed in seconds.

- **dt\_impr\_ustar\_mean\_only** *dt\_impr\_ustar\_mean\_only* (5.22.1) for inheritance: This keyword is used to print the mean values of  $u^*$  (obtained with the wall laws) on each boundary, into a file named `datafile_ProblemName_Ustar_mean_only.out`. `periode` refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword `boundaries`, all the boundaries will be considered. If you use it, you must specify `nb_boundaries` which is the number of boundaries on which you want to calculate the mean values of  $u^*$ , then you have to specify their names.
- **nut\_max** *float* for inheritance: Upper limitation of turbulent viscosity (default value 1.e8).
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps** for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is calculated so that diffusive time-step is equal or higher than convective time-step. For a stationary flow, the correction for turbulent viscosity should apply only during the first time steps and not when permanent state is reached. To check that, we could post process the `corr_visco_turb` field which is the correction of turbulent viscosity: it should be 1. on the whole domain.
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps\_parametre** *float* for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is the ratio between diffusive time-step and convective time-step is higher or equal to the given value [0-1]

### 5.22.15 Sous\_maille\_smago\_dyn

Description: Dynamic Smagorinsky sub-grid turbulence model (available in VDF discretization only).

See also: `mod_turb_hyd_ss_maille` (5.22.2)

Usage:

```
sous_maille_smago_dyn {
 [stabilise str into ['6_points', 'moy_euler', 'plans_paralleles']]
 [nb_points int]
 [formulation_a_nb_points form_a_nb_points]
 [longueur_maille str into ['volume', 'volume_sans_lissage', 'scotti', 'arrete']]
 [turbulence_parois turbulence_parois_base]
 [dt_impr_ustar float]
 [dt_impr_ustar_mean_only dt_impr_ustar_mean_only]
 [nut_max float]
 [correction_visco_turb_pour_controle_pas_de_temps]
 [correction_visco_turb_pour_controle_pas_de_temps_parametre float]
}
```

where

- **stabilise** *str into ['6\_points', 'moy\_euler', 'plans\_paralleles']*
- **nb\_points** *int*
- **formulation\_a\_nb\_points** *form\_a\_nb\_points* (5.22.3) for inheritance: The structure function is calculated on `nb_points` and we should add the 2 directions (0:OX, 1:OY, 2:OZ) constituting the homogeneity planes. Example for channel flows, planes parallel to the walls.
- **longueur\_maille** *str into ['volume', 'volume\_sans\_lissage', 'scotti', 'arrete']* for inheritance: Different ways to calculate the characteristic length may be specified :  
*volume* : It is the default option. Characteristic length is based on the cubic root of the volume cells. A smoothing procedure is applied to avoid discontinuities of this quantity in VEF from a cell to another.  
*volume\_sans\_lissage* : For VEF only. Characteristic length is based on the cubic root of the volume cells (without smoothing procedure).  
*scotti* : Characteristic length is based on the cubic root of the volume cells and the Scotti correction

is applied to take into account the stretching of the cell in the case of anisotropic meshes.

*arete* : For VEF only. Characteristic length relies on the max edge (+ smoothing procedure) is taken into account.

- **turbulence\_paro** *turbulence\_paro\_base* (44) for inheritance: Keyword to set the wall law.
- **dt\_impr\_ustar** *float* for inheritance: This keyword is used to print the values ( $U^+$ ,  $d^+$ ,  $u^*$ ) obtained with the wall laws into a file named `datafile_ProblemName_Ustar.face` and *periode* refers to the printing period, this value is expressed in seconds.
- **dt\_impr\_ustar\_mean\_only** *dt\_impr\_ustar\_mean\_only* (5.22.1) for inheritance: This keyword is used to print the mean values of  $u^*$  (obtained with the wall laws) on each boundary, into a file named `datafile_ProblemName_Ustar_mean_only.out`. *periode* refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword *boundaries*, all the boundaries will be considered. If you use it, you must specify *nb\_boundaries* which is the number of boundaries on which you want to calculate the mean values of  $u^*$ , then you have to specify their names.
- **nut\_max** *float* for inheritance: Upper limitation of turbulent viscosity (default value 1.e8).
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps** for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is calculated so that diffusive time-step is equal or higher than convective time-step. For a stationary flow, the correction for turbulent viscosity should apply only during the first time steps and not when permanent state is reached. To check that, we could post process the `corr_visco_turb` field which is the correction of turbulent viscosity: it should be 1. on the whole domain.
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps\_parametre** *float* for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is the ratio between diffusive time-step and convective time-step is higher or equal to the given value [0-1]

### 5.22.16 Combinaison

Description: This keyword specifies a turbulent viscosity model where the turbulent viscosity is user-defined.

See also: `mod_turb_hyd_ss_maille` (5.22.2)

Usage:

```
combinaison {
 [nb_var n word1 word2 ... wordn]
 [fonction str]
 [formulation_a_nb_points form_a_nb_points]
 [longueur_maille str into ['volume', 'volume_sans_lissage', 'scotti', 'arrete']]
 [turbulence_paro turbulence_paro_base]
 [dt_impr_ustar float]
 [dt_impr_ustar_mean_only dt_impr_ustar_mean_only]
 [nut_max float]
 [correction_visco_turb_pour_controle_pas_de_temps]
 [correction_visco_turb_pour_controle_pas_de_temps_parametre float]
}
```

where

- **nb\_var** *n word1 word2 ... wordn*: Number and names of variables which will be used in the turbulent viscosity definition (by default 0)
- **fonction** *str*: Fonction for turbulent viscosity. X,Y,Z and variables defined previously can be used.
- **formulation\_a\_nb\_points** *form\_a\_nb\_points* (5.22.3) for inheritance: The structure function is calculated on *nb* points and we should add the 2 directions (0:OX, 1:OY, 2:OZ) constituting the homogeneity planes. Example for channel flows, planes parallel to the walls.

- **longueur\_maille** *str* into ['volume', 'volume\_sans\_lissage', 'scotti', 'arrete'] for inheritance: Different ways to calculate the characteristic length may be specified :  
 volume : It is the default option. Characteristic length is based on the cubic root of the volume cells. A smoothing procedure is applied to avoid discontinuities of this quantity in VEF from a cell to another.  
 volume\_sans\_lissage : For VEF only. Characteristic length is based on the cubic root of the volume cells (without smoothing procedure).  
 scotti : Characteristic length is based on the cubic root of the volume cells and the Scotti correction is applied to take into account the stretching of the cell in the case of anisotropic meshes.  
 arete : For VEF only. Characteristic length relies on the max edge (+ smoothing procedure) is taken into account.
- **turbulence\_paro** *turbulence\_paro\_base* (44) for inheritance: Keyword to set the wall law.
- **dt\_impr\_ustar** *float* for inheritance: This keyword is used to print the values (U +, d+, u\*) obtained with the wall laws into a file named datafile\_ProblemName\_Ustar.face and periode refers to the printing period, this value is expressed in seconds.
- **dt\_impr\_ustar\_mean\_only** *dt\_impr\_ustar\_mean\_only* (5.22.1) for inheritance: This keyword is used to print the mean values of u\* ( obtained with the wall laws) on each boundary, into a file named datafile\_ProblemName\_Ustar\_mean\_only.out. periode refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword boundaries, all the boundaries will be considered. If you use it, you must specify nb\_boundaries which is the number of boundaries on which you want to calculate the mean values of u\*, then you have to specify their names.
- **nut\_max** *float* for inheritance: Upper limitation of turbulent viscosity (default value 1.e8).
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps** for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is calculated so that diffusive time-step is equal or higher than convective time-step. For a stationary flow, the correction for turbulent viscosity should apply only during the first time steps and not when permanent state is reached. To check that, we could post process the corr\_visco\_turb field which is the correction of turbulent viscosity: it should be 1. on the whole domain.
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps\_parametre** *float* for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is the ratio between diffusive time-step and convective time-step is higher or equal to the given value [0-1]

### 5.22.17 Sous\_maille

Description: Structure sub-grid function model.

See also: mod\_turb\_hyd\_ss\_maille (5.22.2)

Usage:

```
sous_maille {
 [formulation_a_nb_points form_a_nb_points]
 [longueur_maille str into ['volume', 'volume_sans_lissage', 'scotti', 'arrete']]
 [turbulence_paro turbulence_paro_base]
 [dt_impr_ustar float]
 [dt_impr_ustar_mean_only dt_impr_ustar_mean_only]
 [nut_max float]
 [correction_visco_turb_pour_controle_pas_de_temps]
 [correction_visco_turb_pour_controle_pas_de_temps_parametre float]
}
```

where

- **formulation\_a\_nb\_points** *form\_a\_nb\_points* (5.22.3) for inheritance: The structure function is calculated on nb points and we should add the 2 directions (0:OX, 1:OY, 2:OZ) constituting the homogeneity planes. Example for channel flows, planes parallel to the walls.
- **longueur\_maille** *str* into ['volume', 'volume\_sans\_lissage', 'scotti', 'arrete'] for inheritance: Different ways to calculate the characteristic length may be specified :  
 volume : It is the default option. Characteristic length is based on the cubic root of the volume cells. A smoothing procedure is applied to avoid discontinuities of this quantity in VEF from a cell to another.  
 volume\_sans\_lissage : For VEF only. Characteristic length is based on the cubic root of the volume cells (without smoothing procedure).  
 scotti : Characteristic length is based on the cubic root of the volume cells and the Scotti correction is applied to take into account the stretching of the cell in the case of anisotropic meshes.  
 arete : For VEF only. Characteristic length relies on the max edge (+ smoothing procedure) is taken into account.
- **turbulence\_paro** *turbulence\_paro\_base* (44) for inheritance: Keyword to set the wall law.
- **dt\_impr\_ustar** *float* for inheritance: This keyword is used to print the values (U +, d+, u\*) obtained with the wall laws into a file named datafile\_ProblemName\_Ustar.face and periode refers to the printing period, this value is expressed in seconds.
- **dt\_impr\_ustar\_mean\_only** *dt\_impr\_ustar\_mean\_only* (5.22.1) for inheritance: This keyword is used to print the mean values of u\* ( obtained with the wall laws) on each boundary, into a file named datafile\_ProblemName\_Ustar\_mean\_only.out. periode refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword boundaries, all the boundaries will be considered. If you use it, you must specify nb\_boundaries which is the number of boundaries on which you want to calculate the mean values of u\*, then you have to specify their names.
- **nut\_max** *float* for inheritance: Upper limitation of turbulent viscosity (default value 1.e8).
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps** for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is calculated so that diffusive time-step is equal or higher than convective time-step. For a stationary flow, the correction for turbulent viscosity should apply only during the first time steps and not when permanent state is reached. To check that, we could post process the corr\_visco\_turb field which is the correction of turbulent viscosity: it should be 1. on the whole domain.
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps\_parametre** *float* for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is the ratio between diffusive time-step and convective time-step is higher or equal to the given value [0-1]

### 5.22.18 Null

Description: Null turbulence model (turbulent viscosity = 0) which can be used with a turbulent problem.

See also: modele\_turbulence\_hyd\_deriv (5.22)

Usage:

```

null {
 [turbulence_paro turbulence_paro_base]
 [dt_impr_ustar float]
 [dt_impr_ustar_mean_only dt_impr_ustar_mean_only]
 [nut_max float]
 [correction_visco_turb_pour_controle_pas_de_temps]
 [correction_visco_turb_pour_controle_pas_de_temps_parametre float]

```

}

where

- **turbulence\_paro** *turbulence\_paro\_base* (44) for inheritance: Keyword to set the wall law.
- **dt\_impr\_ustar** *float* for inheritance: This keyword is used to print the values ( $U +$ ,  $d+$ ,  $u*$ ) obtained with the wall laws into a file named `datafile_ProblemName_Ustar.face` and `periode` refers to the printing period, this value is expressed in seconds.
- **dt\_impr\_ustar\_mean\_only** *dt\_impr\_ustar\_mean\_only* (5.22.1) for inheritance: This keyword is used to print the mean values of  $u*$  (obtained with the wall laws) on each boundary, into a file named `datafile_ProblemName_Ustar_mean_only.out`. `periode` refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword `boundaries`, all the boundaries will be considered. If you use it, you must specify `nb_boundaries` which is the number of boundaries on which you want to calculate the mean values of  $u*$ , then you have to specify their names.
- **nut\_max** *float* for inheritance: Upper limitation of turbulent viscosity (default value  $1.e8$ ).
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps** for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is calculated so that diffusive time-step is equal or higher than convective time-step. For a stationary flow, the correction for turbulent viscosity should apply only during the first time steps and not when permanent state is reached. To check that, we could post process the `corr_visco_turb` field which is the correction of turbulent viscosity: it should be 1. on the whole domain.
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps\_parametre** *float* for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is the ratio between diffusive time-step and convective time-step is higher or equal to the given value [0-1]

### 5.22.19 Mod\_turb\_hyd\_rans

Description: Class for RANS turbulence model for Navier-Stokes equations.

See also: `modele_turbulence_hyd_deriv` (5.22) `k_omega` (5.22.20) `mod_turb_hyd_rans_keps` (5.22.21) `mod_turb_hyd_rans_bicephale` (5.22.29) `mod_turb_hyd_rans_komega` (5.22.31) `K_Epsilon_Realisable_Bicephale` (5.22.32) `K_Epsilon_Realisable` (5.22.33)

Usage:

```
mod_turb_hyd_rans {
 [transport_equation transport_equation_deriv]
 [k_min float]
 [quiet]
 [turbulence_paro turbulence_paro_base]
 [dt_impr_ustar float]
 [dt_impr_ustar_mean_only dt_impr_ustar_mean_only]
 [nut_max float]
 [correction_visco_turb_pour_controle_pas_de_temps]
 [correction_visco_turb_pour_controle_pas_de_temps_parametre float]
}
```

where

- **transport\_equation** *transport\_equation\_deriv* (43)
- **k\_min** *float*: Lower limitation of  $k$  (default value  $1.e-10$ ).
- **quiet**: To disable printing of information about  $K$  and  $Epsilon/Omega$ .
- **turbulence\_paro** *turbulence\_paro\_base* (44) for inheritance: Keyword to set the wall law.
- **dt\_impr\_ustar** *float* for inheritance: This keyword is used to print the values ( $U +$ ,  $d+$ ,  $u*$ ) obtained with the wall laws into a file named `datafile_ProblemName_Ustar.face` and `periode` refers to the printing period, this value is expressed in seconds.

- **dt\_impr\_ustar\_mean\_only** *dt\_impr\_ustar\_mean\_only* (5.22.1) for inheritance: This keyword is used to print the mean values of  $u^*$  ( obtained with the wall laws) on each boundary, into a file named `datafile_ProblemName_Ustar_mean_only.out`. `periode` refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword `boundaries`, all the boundaries will be considered. If you use it, you must specify `nb_boundaries` which is the number of boundaries on which you want to calculate the mean values of  $u^*$ , then you have to specify their names.
- **nut\_max** *float* for inheritance: Upper limitation of turbulent viscosity (default value 1.e8).
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps** for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is calculated so that diffusive time-step is equal or higher than convective time-step. For a stationary flow, the correction for turbulent viscosity should apply only during the first time steps and not when permanent state is reached. To check that, we could post process the `corr_visco_turb` field which is the correction of turbulent viscosity: it should be 1. on the whole domain.
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps\_parametre** *float* for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is the ratio between diffusive time-step and convective time-step is higher or equal to the given value [0-1]

### 5.22.20 K\_omega

Description: Turbulence model (k-omega).

See also: `mod_turb_hyd_rans` (5.22.19)

Usage:

```
k_omega {
 [sigma_k|prandtl_k float]
 [sigma_omegalprandtl_omega float]
 [model_variant str]
 [sigma_k1 float]
 [sigma_k2 float]
 [sigma_omega1 float]
 [sigma_omega2 float]
 [expert_mode bloc_lecture]
 [transport_equation transport_equation_deriv]
 [k_min float]
 [quiet]
 [turbulence_pari turbulence_pari_base]
 [dt_impr_ustar float]
 [dt_impr_ustar_mean_only dt_impr_ustar_mean_only]
 [nut_max float]
 [correction_visco_turb_pour_controle_pas_de_temps]
 [correction_visco_turb_pour_controle_pas_de_temps_parametre float]
}
```

where

- **sigma\_k|prandtl\_k** *float*: Prandtl\_K|Sigma\_K (default value 1./2.). See <https://turbmodels.larc.nasa.gov/wilcox.html>
- **sigma\_omegalprandtl\_omega** *float*: Prandtl\_Omega|Sigma\_Omega (default value 1./2.).
- **model\_variant** *str*: Model variant for k-omega (default value STD)
- **sigma\_k1** *float*: Sigma\_K1 for SST model (default value 0.85). See <https://turbmodels.larc.nasa.gov/sst.html>



- **sigma\_k2** *float*: Sigma\_K2 for SST model (default value 1.).
- **sigma\_omega1** *float*: Sigma\_Omega1 for SST model (default value 1./2.).
- **sigma\_omega2** *float*: Prandtl\_Omega2 for SST model (default value 0.856).
- **expert\_mode** *bloc\_lecture* (3.2)
- **transport\_equation** *transport\_equation\_deriv* (43) for inheritance
- **k\_min** *float* for inheritance: Lower limitation of k (default value 1.e-10).
- **quiet** for inheritance: To disable printing of information about K and Epsilon/Omega.
- **turbulence\_paro** *turbulence\_paro\_base* (44) for inheritance: Keyword to set the wall law.
- **dt\_impr\_ustar** *float* for inheritance: This keyword is used to print the values (U +, d+, u\*) obtained with the wall laws into a file named datafile\_ProblemName\_Ustar.face and periode refers to the printing period, this value is expressed in seconds.
- **dt\_impr\_ustar\_mean\_only** *dt\_impr\_ustar\_mean\_only* (5.22.1) for inheritance: This keyword is used to print the mean values of u\* ( obtained with the wall laws) on each boundary, into a file named datafile\_ProblemName\_Ustar\_mean\_only.out. periode refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword boundaries, all the boundaries will be considered. If you use it, you must specify nb\_boundaries which is the number of boundaries on which you want to calculate the mean values of u\*, then you have to specify their names.
- **nut\_max** *float* for inheritance: Upper limitation of turbulent viscosity (default value 1.e8).
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps** for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is calculated so that diffusive time-step is equal or higher than convective time-step. For a stationary flow, the correction for turbulent viscosity should apply only during the first time steps and not when permanent state is reached. To check that, we could post process the corr\_visco\_turb field which is the correction of turbulent viscosity: it should be 1. on the whole domain.
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps\_parametre** *float* for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is the ratio between diffusive time-step and convective time-step is higher or equal to the given value [0-1]

### 5.22.21 Mod\_turb\_hyd\_rans\_keps

Description: Class for RANS turbulence model for Navier-Stokes equations.

See also: mod\_turb\_hyd\_rans (5.22.19) k\_epsilon (5.22.22)

Usage:

```
mod_turb_hyd_rans_keps {
 [eps_min float]
 [eps_max float]
 [transport_equation transport_equation_deriv]
 [k_min float]
 [quiet]
 [turbulence_paro turbulence_paro_base]
 [dt_impr_ustar float]
 [dt_impr_ustar_mean_only dt_impr_ustar_mean_only]
 [nut_max float]
 [correction_visco_turb_pour_controle_pas_de_temps]
 [correction_visco_turb_pour_controle_pas_de_temps_parametre float]
}
where
```

- **eps\_min** *float*: Lower limitation of epsilon (default value 1.e-20).



- **eps\_max** *float*: Upper limitation of epsilon (default value 1.e+10).
- **transport\_equation** *transport\_equation\_deriv* (43) for inheritance
- **k\_min** *float* for inheritance: Lower limitation of k (default value 1.e-10).
- **quiet** for inheritance: To disable printing of information about K and Epsilon/Omega.
- **turbulence\_paro** *turbulence\_paro\_base* (44) for inheritance: Keyword to set the wall law.
- **dt\_impr\_ustar** *float* for inheritance: This keyword is used to print the values (U +, d+, u\*) obtained with the wall laws into a file named *datafile\_ProblemName\_Ustar.face* and *periode* refers to the printing period, this value is expressed in seconds.
- **dt\_impr\_ustar\_mean\_only** *dt\_impr\_ustar\_mean\_only* (5.22.1) for inheritance: This keyword is used to print the mean values of u\* ( obtained with the wall laws) on each boundary, into a file named *datafile\_ProblemName\_Ustar\_mean\_only.out*. *periode* refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword *boundaries*, all the boundaries will be considered. If you use it, you must specify *nb\_boundaries* which is the number of boundaries on which you want to calculate the mean values of u\*, then you have to specify their names.
- **nut\_max** *float* for inheritance: Upper limitation of turbulent viscosity (default value 1.e8).
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps** for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is calculated so that diffusive time-step is equal or higher than convective time-step. For a stationary flow, the correction for turbulent viscosity should apply only during the first time steps and not when permanent state is reached. To check that, we could post process the *corr\_visco\_turb* field which is the correction of turbulent viscosity: it should be 1. on the whole domain.
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps\_parametre** *float* for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is the ratio between diffusive time-step and convective time-step is higher or equal to the given value [0-1]

### 5.22.22 K\_epsilon

Description: Turbulence model (k-eps).

See also: *mod\_turb\_hyd\_rans\_keps* (5.22.21)

Usage:

```
k_epsilon {
 [modele_fonc_bas_reynolds modele_fonction_bas_reynolds_base]
 [cmu float]
 [sigma_klprandtl_k float]
 [sigma_epsprandtl_eps float]
 [eps_min float]
 [eps_max float]
 [transport_equation transport_equation_deriv]
 [k_min float]
 [quiet]
 [turbulence_paro turbulence_paro_base]
 [dt_impr_ustar float]
 [dt_impr_ustar_mean_only dt_impr_ustar_mean_only]
 [nut_max float]
 [correction_visco_turb_pour_controle_pas_de_temps]
 [correction_visco_turb_pour_controle_pas_de_temps_parametre float]
}
```

where

- **modele\_fonc\_bas\_reynolds** *modele\_fonction\_bas\_reynolds\_base* (5.22.23): This keyword is used to set the bas Reynolds model used.
- **cmu** *float*: Keyword to modify the Cmu constant of k-eps model :  $Nut = Cmu * k^2 / \epsilon$  Default value is 0.09
- **sigma\_klprandtl\_k** *float*: Keyword to change the PrkSigma\_K value (default 1.0). See <https://turbmodels.larc.nasa.gov/kechien.html>
- **sigma\_epsprandtl\_eps** *float*: Keyword to change the PrlSigma\_Eps value (default 1.3).
- **eps\_min** *float* for inheritance: Lower limitation of epsilon (default value 1.e-20).
- **eps\_max** *float* for inheritance: Upper limitation of epsilon (default value 1.e+10).
- **transport\_equation** *transport\_equation\_deriv* (43) for inheritance
- **k\_min** *float* for inheritance: Lower limitation of k (default value 1.e-10).
- **quiet** for inheritance: To disable printing of information about K and Epsilon/Omega.
- **turbulence\_paro** *turbulence\_paro\_base* (44) for inheritance: Keyword to set the wall law.
- **dt\_impr\_ustar** *float* for inheritance: This keyword is used to print the values (U +, d+, u\*) obtained with the wall laws into a file named datafile\_ProblemName\_Ustar.face and periode refers to the printing period, this value is expressed in seconds.
- **dt\_impr\_ustar\_mean\_only** *dt\_impr\_ustar\_mean\_only* (5.22.1) for inheritance: This keyword is used to print the mean values of u\* ( obtained with the wall laws) on each boundary, into a file named datafile\_ProblemName\_Ustar\_mean\_only.out. periode refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword boundaries, all the boundaries will be considered. If you use it, you must specify nb\_boundaries which is the number of boundaries on which you want to calculate the mean values of u\*, then you have to specify their names.
- **nut\_max** *float* for inheritance: Upper limitation of turbulent viscosity (default value 1.e8).
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps** for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is calculated so that diffusive time-step is equal or higher than convective time-step. For a stationary flow, the correction for turbulent viscosity should apply only during the first time steps and not when permanent state is reached. To check that, we could post process the corr\_visco\_turb field which is the correction of turbulent viscosity: it should be 1. on the whole domain.
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps\_parametre** *float* for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is the ratio between diffusive time-step and convective time-step is higher or equal to the given value [0-1]

### 5.22.23 Modele\_fonction\_bas\_reynolds\_base

Description: not\_set

See also: objet\_lecture (47) Lam\_Bremhorst (5.22.24) Jones\_Launder (5.22.27) Launder\_Sharma (5.22.28)

Usage:

### 5.22.24 Lam\_bremhorst

Description: Model described in ' C.K.G.Lam and K.Bremhorst, A modified form of the k- epsilon model for predicting wall turbulence, ASME J. Fluids Engng., Vol.103, p456, (1981)'. Only in VEF.

See also: modele\_fonction\_bas\_reynolds\_base (5.22.23) EASM\_Baglietto (5.22.25) standard\_KEps (5.22.26)

Usage:

```
Lam_Bremhorst {
 [fichier_distance_paro str]
 [reynolds_stress_isotrope int]
```

}  
where

- **fichier\_distance\_paro** *str*: refer to distance\_paro keyword
- **reynolds\_stress\_isotrope** *int*: keyword for isotropic Reynolds stress

#### 5.22.25 Easm\_baglietto

Description: Model described in ' E. Baglietto and H. Ninokata , A turbulence model study for simulating flow inside tight lattice rod bundles, Nuclear Engineering and Design, 773–784 (235), 2005. '

See also: Lam\_Bremhorst ([5.22.24](#))

Usage:

```
EASM_Baglietto {
 [fichier_distance_paro str]
 [reynolds_stress_isotrope int]
}
```

where

- **fichier\_distance\_paro** *str* for inheritance: refer to distance\_paro keyword
- **reynolds\_stress\_isotrope** *int* for inheritance: keyword for isotropic Reynolds stress

#### 5.22.26 Standard\_keps

Description: Model described in ' E. Baglietto , CFD and DNS methodologies development for fuel bundle simulaions, Nuclear Engineering and Design, 1503–1510 (236), 2006. '

See also: Lam\_Bremhorst ([5.22.24](#))

Usage:

```
standard_KEps {
 [fichier_distance_paro str]
 [reynolds_stress_isotrope int]
}
```

where

- **fichier\_distance\_paro** *str* for inheritance: refer to distance\_paro keyword
- **reynolds\_stress\_isotrope** *int* for inheritance: keyword for isotropic Reynolds stress

#### 5.22.27 Jones\_launder

Description: Model described in ' Jones, W. P. and Launder, B. E. (1972), The prediction of laminarization with a two-equation model of turbulence, Int. J. of Heat and Mass transfer, Vol. 15, pp. 301-314.'

See also: modele\_fonction\_bas\_reynolds\_base ([5.22.23](#))

Usage:

### 5.22.28 Launder\_sharma

Description: Model described in 'Launder, B. E. and Sharma, B. I. (1974), Application of the Energy-Dissipation Model of Turbulence to the Calculation of Flow Near a Spinning Disc, Letters in Heat and Mass Transfer, Vol. 1, No. 2, pp. 131-138.'

See also: `modele_fonction_bas_reynolds_base` ([5.22.23](#))

Usage:

### 5.22.29 Mod\_turb\_hyd\_rans\_bicephale

Description: Class for RANS turbulence model for Navier-Stokes equations.

See also: `mod_turb_hyd_rans` ([5.22.19](#)) `K_Epsilon_Bicephale` ([5.22.30](#))

Usage:

```
mod_turb_hyd_rans_bicephale {
 [eps_min float]
 [eps_max float]
 [sigma_k|prandtl_k float]
 [sigma_eps|prandtl_eps float]
 [transport_equation transport_equation_deriv]
 [k_min float]
 [quiet]
 [turbulence_paro turbulence_paro_base]
 [dt_impr_ustar float]
 [dt_impr_ustar_mean_only dt_impr_ustar_mean_only]
 [nut_max float]
 [correction_visco_turb_pour_controle_pas_de_temps]
 [correction_visco_turb_pour_controle_pas_de_temps_parametre float]
}
```

where

- **eps\_min** *float*: Lower limitation of epsilon (default value 1.e-20).
- **eps\_max** *float*: Upper limitation of epsilon (default value 1.e+10).
- **sigma\_k|prandtl\_k** *float*: Keyword to change the Prk|Sigma\_K value (default 1.0). See <https://turbmodels.larc.nasa.gov/ke-chien.html>
- **sigma\_eps|prandtl\_eps** *float*: Keyword to change the Prk|Sigma\_Eps value (default 1.3).
- **transport\_equation** *transport\_equation\_deriv* ([43](#)) for inheritance
- **k\_min** *float* for inheritance: Lower limitation of k (default value 1.e-10).
- **quiet** for inheritance: To disable printing of information about K and Epsilon/Omega.
- **turbulence\_paro** *turbulence\_paro\_base* ([44](#)) for inheritance: Keyword to set the wall law.
- **dt\_impr\_ustar** *float* for inheritance: This keyword is used to print the values (U +, d+, u\*) obtained with the wall laws into a file named `datafile_ProblemName_Ustar.face` and `periode` refers to the printing period, this value is expressed in seconds.
- **dt\_impr\_ustar\_mean\_only** *dt\_impr\_ustar\_mean\_only* ([5.22.1](#)) for inheritance: This keyword is used to print the mean values of u\* ( obtained with the wall laws) on each boundary, into a file named `datafile_ProblemName_Ustar_mean_only.out`. `periode` refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword `boundaries`, all the boundaries will be considered. If you use it, you must specify `nb_boundaries` which is the number of boundaries on which you want to calculate the mean values of u\*, then you have to specify their names.
- **nut\_max** *float* for inheritance: Upper limitation of turbulent viscosity (default value 1.e8).

- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps** for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is calculated so that diffusive time-step is equal or higher than convective time-step. For a stationary flow, the correction for turbulent viscosity should apply only during the first time steps and not when permanent state is reached. To check that, we could post process the `corr_visco_turb` field which is the correction of turbulent viscosity: it should be 1. on the whole domain.
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps\_parametre** *float* for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is the ratio between diffusive time-step and convective time-step is higher or equal to the given value [0-1]

### 5.22.30 K\_epsilon\_bicephale

Description: Turbulence model (k-eps) en formalisation bicephale.

See also: `mod_turb_hyd_rans_bicephale` (5.22.29)

Usage:

```
K_Epsilon_Bicephale {
 [modele_fonc_bas_reynolds modele_fonc_realisable_base]
 [cmu float]
 [eps_min float]
 [eps_max float]
 [sigma_klprandtl_k float]
 [sigma_eps|prandtl_eps float]
 [transport_equation transport_equation_deriv]
 [k_min float]
 [quiet]
 [turbulence_paroi turbulence_paro_base]
 [dt_impr_ustar float]
 [dt_impr_ustar_mean_only dt_impr_ustar_mean_only]
 [nut_max float]
 [correction_visco_turb_pour_controle_pas_de_temps]
 [correction_visco_turb_pour_controle_pas_de_temps_parametre float]
}
```

where

- **modele\_fonc\_bas\_reynolds** *modele\_fonc\_realisable\_base* (14.1): This keyword is used to set the model used
- **cmu** *float*: Keyword to modify the Cmu constant of k-eps model :  $Nut = Cmu * k^2 / \epsilon$  Default value is 0.09
- **eps\_min** *float* for inheritance: Lower limitation of epsilon (default value 1.e-20).
- **eps\_max** *float* for inheritance: Upper limitation of epsilon (default value 1.e+10).
- **sigma\_klprandtl\_k** *float* for inheritance: Keyword to change the PrkSigma\_K value (default 1.0). See <https://turbmodels.larc.nasa.gov/ke-chien.html>
- **sigma\_eps|prandtl\_eps** *float* for inheritance: Keyword to change the PrSigma\_Eps value (default 1.3).
- **transport\_equation** *transport\_equation\_deriv* (43) for inheritance
- **k\_min** *float* for inheritance: Lower limitation of k (default value 1.e-10).
- **quiet** for inheritance: To disable printing of information about K and Epsilon/Omega.
- **turbulence\_paro**i *turbulence\_paro\_base* (44) for inheritance: Keyword to set the wall law.

- **dt\_impr\_ustar** *float* for inheritance: This keyword is used to print the values (U +, d+, u\*) obtained with the wall laws into a file named `datafile_ProblemName_Ustar.face` and `periode` refers to the printing period, this value is expressed in seconds.
- **dt\_impr\_ustar\_mean\_only** *dt\_impr\_ustar\_mean\_only* (5.22.1) for inheritance: This keyword is used to print the mean values of  $u^*$  (obtained with the wall laws) on each boundary, into a file named `datafile_ProblemName_Ustar_mean_only.out`. `periode` refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword `boundaries`, all the boundaries will be considered. If you use it, you must specify `nb_boundaries` which is the number of boundaries on which you want to calculate the mean values of  $u^*$ , then you have to specify their names.
- **nut\_max** *float* for inheritance: Upper limitation of turbulent viscosity (default value 1.e8).
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps** for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is calculated so that diffusive time-step is equal or higher than convective time-step. For a stationary flow, the correction for turbulent viscosity should apply only during the first time steps and not when permanent state is reached. To check that, we could post process the `corr_visco_turb` field which is the correction of turbulent viscosity: it should be 1. on the whole domain.
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps\_parametre** *float* for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is the ratio between diffusive time-step and convective time-step is higher or equal to the given value [0-1]

### 5.22.31 Mod\_turb\_hyd\_rans\_komega

Description: Class for RANS turbulence model for Navier-Stokes equations.

See also: `mod_turb_hyd_rans` (5.22.19)

Usage:

```
mod_turb_hyd_rans_komega {
 [omega_min float]
 [omega_max float]
 [transport_equation transport_equation_deriv]
 [k_min float]
 [quiet]
 [turbulence_paroit turbulence_paroit_base]
 [dt_impr_ustar float]
 [dt_impr_ustar_mean_only dt_impr_ustar_mean_only]
 [nut_max float]
 [correction_visco_turb_pour_controle_pas_de_temps]
 [correction_visco_turb_pour_controle_pas_de_temps_parametre float]
}
```

where

- **omega\_min** *float*: Lower limitation of omega (default value 1.e-5).
- **omega\_max** *float*: Upper limitation of omega (default value 1.e+20).
- **transport\_equation** *transport\_equation\_deriv* (43) for inheritance
- **k\_min** *float* for inheritance: Lower limitation of k (default value 1.e-10).
- **quiet** for inheritance: To disable printing of information about K and Epsilon/Omega.
- **turbulence\_paroit** *turbulence\_paroit\_base* (44) for inheritance: Keyword to set the wall law.
- **dt\_impr\_ustar** *float* for inheritance: This keyword is used to print the values (U +, d+, u\*) obtained with the wall laws into a file named `datafile_ProblemName_Ustar.face` and `periode` refers to the printing period, this value is expressed in seconds.

- **dt\_impr\_ustar\_mean\_only** *dt\_impr\_ustar\_mean\_only* (5.22.1) for inheritance: This keyword is used to print the mean values of  $u^*$  (obtained with the wall laws) on each boundary, into a file named `datafile_ProblemName_Ustar_mean_only.out`. `periode` refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword `boundaries`, all the boundaries will be considered. If you use it, you must specify `nb_boundaries` which is the number of boundaries on which you want to calculate the mean values of  $u^*$ , then you have to specify their names.
- **nut\_max** *float* for inheritance: Upper limitation of turbulent viscosity (default value 1.e8).
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps** for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is calculated so that diffusive time-step is equal or higher than convective time-step. For a stationary flow, the correction for turbulent viscosity should apply only during the first time steps and not when permanent state is reached. To check that, we could post process the `corr_visco_turb` field which is the correction of turbulent viscosity: it should be 1. on the whole domain.
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps\_parametre** *float* for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is the ratio between diffusive time-step and convective time-step is higher or equal to the given value [0-1]

### 5.22.32 K\_epsilon\_realisable\_bicephale

Description: Realizable Two-headed K-Epsilon Turbulence Model

See also: `mod_turb_hyd_rans` (5.22.19)

Usage:

```
K_Epsilon_Realisable_Bicephale {
 modele_fonc_realisable modele_fonc_realisable_base
 sigma_klprandtl_k float
 sigma_epsprandtl_eps float
 [transport_equation transport_equation_deriv]
 [k_min float]
 [quiet]
 [turbulence_paro turbulence_paro_base]
 [dt_impr_ustar float]
 [dt_impr_ustar_mean_only dt_impr_ustar_mean_only]
 [nut_max float]
 [correction_visco_turb_pour_controle_pas_de_temps]
 [correction_visco_turb_pour_controle_pas_de_temps_parametre float]
}
```

where

- **modele\_fonc\_realisable** *modele\_fonc\_realisable\_base* (14.1): This keyword is used to set the model used
- **sigma\_klprandtl\_k** *float*: Keyword to change the `PrkSigma_K` value (default 1.0). See <https://turbmodels.larc.nasa.gov/ke-chien.html>
- **sigma\_epsprandtl\_eps** *float*: Keyword to change the `PrelSigma_Eps` value (default 1.3)
- **transport\_equation** *transport\_equation\_deriv* (43) for inheritance
- **k\_min** *float* for inheritance: Lower limitation of `k` (default value 1.e-10).
- **quiet** for inheritance: To disable printing of information about `K` and `Epsilon/Omega`.
- **turbulence\_paro** *turbulence\_paro\_base* (44) for inheritance: Keyword to set the wall law.
- **dt\_impr\_ustar** *float* for inheritance: This keyword is used to print the values ( $U^+$ ,  $d^+$ ,  $u^*$ ) obtained with the wall laws into a file named `datafile_ProblemName_Ustar.face` and `periode` refers to the printing period, this value is expressed in seconds.



- **dt\_impr\_ustar\_mean\_only** *dt\_impr\_ustar\_mean\_only* (5.22.1) for inheritance: This keyword is used to print the mean values of  $u^*$  (obtained with the wall laws) on each boundary, into a file named `datafile_ProblemName_Ustar_mean_only.out`. `periode` refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword `boundaries`, all the boundaries will be considered. If you use it, you must specify `nb_boundaries` which is the number of boundaries on which you want to calculate the mean values of  $u^*$ , then you have to specify their names.
- **nut\_max** *float* for inheritance: Upper limitation of turbulent viscosity (default value 1.e8).
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps** for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is calculated so that diffusive time-step is equal or higher than convective time-step. For a stationary flow, the correction for turbulent viscosity should apply only during the first time steps and not when permanent state is reached. To check that, we could post process the `corr_visco_turb` field which is the correction of turbulent viscosity: it should be 1. on the whole domain.
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps\_parametre** *float* for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is the ratio between diffusive time-step and convective time-step is higher or equal to the given value [0-1]

### 5.22.33 K\_epsilon\_realisable

Description: Realizable K-Epsilon Turbulence Model.

See also: `mod_turb_hyd_rans` (5.22.19)

Usage:

```
K_Epsilon_Realisable {
 modele_fonc_realisable modele_fonc_realisable_base
 sigma_klprandtl_k float
 sigma_epsprandtl_eps float
 [transport_equation transport_equation_deriv]
 [k_min float]
 [quiet]
 [turbulence_parois turbulence_parois_base]
 [dt_impr_ustar float]
 [dt_impr_ustar_mean_only dt_impr_ustar_mean_only]
 [nut_max float]
 [correction_visco_turb_pour_controle_pas_de_temps]
 [correction_visco_turb_pour_controle_pas_de_temps_parametre float]
}
```

where

- **modele\_fonc\_realisable** *modele\_fonc\_realisable\_base* (14.1): This keyword is used to set the model used
- **sigma\_klprandtl\_k** *float*: Keyword to change the `PrkSigma_K` value (default 1.0). See <https://turbmodels.larc.nasa.gov/ke-chien.html>
- **sigma\_epsprandtl\_eps** *float*: Keyword to change the `PrelSigma_Eps` value (default 1.3)
- **transport\_equation** *transport\_equation\_deriv* (43) for inheritance
- **k\_min** *float* for inheritance: Lower limitation of `k` (default value 1.e-10).
- **quiet** for inheritance: To disable printing of information about `K` and `Epsilon/Omega`.
- **turbulence\_parois** *turbulence\_parois\_base* (44) for inheritance: Keyword to set the wall law.
- **dt\_impr\_ustar** *float* for inheritance: This keyword is used to print the values ( $U^+$ ,  $d^+$ ,  $u^*$ ) obtained with the wall laws into a file named `datafile_ProblemName_Ustar.face` and `periode` refers to the printing period, this value is expressed in seconds.



- **dt\_impr\_ustar\_mean\_only** *dt\_impr\_ustar\_mean\_only* (5.22.1) for inheritance: This keyword is used to print the mean values of  $u^*$  (obtained with the wall laws) on each boundary, into a file named `datafile_ProblemName_Ustar_mean_only.out`. `periode` refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword `boundaries`, all the boundaries will be considered. If you use it, you must specify `nb_boundaries` which is the number of boundaries on which you want to calculate the mean values of  $u^*$ , then you have to specify their names.
- **nut\_max** *float* for inheritance: Upper limitation of turbulent viscosity (default value 1.e8).
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps** for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is calculated so that diffusive time-step is equal or higher than convective time-step. For a stationary flow, the correction for turbulent viscosity should apply only during the first time steps and not when permanent state is reached. To check that, we could post process the `corr_visco_turb` field which is the correction of turbulent viscosity: it should be 1. on the whole domain.
- **correction\_visco\_turb\_pour\_controle\_pas\_de\_temps\_parametre** *float* for inheritance: Keyword to set a limitation to low time steps due to high values of turbulent viscosity. The limit for turbulent viscosity is the ratio between diffusive time-step and convective time-step is higher or equal to the given value [0-1]

## 5.23 Navier\_stokes\_standard\_sensibility

Description: Resolution of Navier-Stokes sensitivity problem

Keyword `Discretize` should have already been used to read the object.

See also: `navier_stokes_standard` (5.57)

Usage:

**Navier\_Stokes\_standard\_sensibility** *str*

**Read** *str* {

```

state bloc_lecture
[uncertain_variable bloc_lecture]
[polynomial_chaos float]
[adjoint]
[solveur_pression solveur_sys_base]
[dt_projection deuxmots]
[traitement_particulier traitement_particulier]
[seuil_divU floatfloat]
[solveur_bar solveur_sys_base]
[projection_initiale int]
[postraiter_gradient_pression_sans_masse]
[methode_calcul_pression_initiale str into ['avec_les_cl', 'avec_sources', 'avec_sources_et-
_operateurs', 'sans_rien']]
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]

```

}

where

- **state** *bloc\_lecture* (3.2): Block to indicate the state problem. Between the braces, you must specify the key word 'pb\_champ\_evaluateur' then the name of the state problem and the velocity unknown  
Example: state { pb\_champ\_evaluateur pb\_state velocity }
- **uncertain\_variable** *bloc\_lecture* (3.2): Block to indicate the name of the uncertain variable. Between the braces, you must specify the name of the unknown variable. Choice between velocity and mu.  
Example: uncertain\_variable { velocity }
- **polynomial\_chaos** *float*: It is the method that we will use to study the sensitivity of the Navier Stokes equation:  
if poly\_chaos=0, the sensitivity will be treated by the standard sensitivity method. If different than 0, it will be treated by the polynomial chaos method
- **adjoint**: A keyword to indicate that the adjoint Navier-Stokes equations will be solved
- **solveur\_pression** *solveur\_sys\_base* (14.19) for inheritance: Linear pressure system resolution method.
- **dt\_projection** *deuxmots* (4.9.1) for inheritance: nb value: This keyword checks every nb time-steps the equality of velocity divergence to zero. value is the criteria convergency for the solver used.
- **traitement\_particulier** *traitement\_particulier* (5.18) for inheritance: Keyword to post-process particular values.
- **seuil\_divU** *floatfloat* (5.19) for inheritance: value factor: this keyword is intended to minimise the number of iterations during the pressure system resolution. The convergence criteria during this step ('seuil' in solveur\_pression) is dynamically adapted according to the mass conservation. At  $t_n$ , the linear system  $Ax=B$  is considered as solved if the residual  $\|Ax-B\| < \text{seuil}(t_n)$ . For  $t_{n+1}$ , the threshold value  $\text{seuil}(t_{n+1})$  will be evaluated as:  
If (  $\text{lmax}(\text{DivU}) \cdot dt < \text{value}$  )  
Seuil( $t_{n+1}$ ) = Seuil( $t_n$ ) \* factor  
Else  
Seuil( $t_{n+1}$ ) = Seuil( $t_n$ ) \* factor  
Endif  
The first parameter (value) is the mass evolution the user is ready to accept per timestep, and the second one (factor) is the factor of evolution for 'seuil' (for example 1.1, so 10)
- **solveur\_bar** *solveur\_sys\_base* (14.19) for inheritance: This keyword is used to define when filtering operation is called (typically for EF convective scheme, standard diffusion operator and Source\_Qdm\_lambdaup ). A file (solveur.bar) is then created and used for inversion procedure. Syntax is the same then for pressure solver (GCP is required for multi-processor calculations and, in a general way, for big meshes).
- **projection\_initiale** *int* for inheritance: Keyword to suppress, if boolean equals 0, the initial projection which checks  $\text{DivU}=0$ . By default, boolean equals 1.
- **postraiter\_gradient\_pression\_sans\_masse** for inheritance: Avoid mass matrix multiplication for the gradient postprocessing
- **methode\_calcul\_pression\_initiale** *str into ['avec\_les\_cl', 'avec\_sources', 'avec\_sources\_et\_operateurs', 'sans\_rien']* for inheritance: Keyword to select an option for the pressure calculation before the first time step. Options are: avec\_les\_cl (default option  $\text{lapP}=0$  is solved with Neuman boundary conditions on pressure if any), avec\_sources ( $\text{lapP}=f$  is solved with Neuman boundaries conditions and  $f$  integrating the source terms of the Navier-Stokes equations) and avec\_sources\_et\_operateurs ( $\text{lapP}=f$  is solved as with the previous option avec\_sources but  $f$  integrating also some operators of the Navier-Stokes equations). The two last options are useful and sometime necessary when source terms are implicated when using an implicit time scheme to solve the Navier-Stokes equations.
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limite** *condlims* (4.39.1) for inheritance: Boundary conditions.

- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.24 Navier\_stokes\_std\_ale

Description: Resolution of hydraulic Navier-Stokes eq. on mobile domain (ALE)

Keyword Discretize should have already been used to read the object.

See also: navier\_stokes\_standard (5.57) Navier\_Stokes\_Turbulent\_ALE (5.21)

Usage:

**Navier\_Stokes\_std\_ALE** *str*

**Read** *str* {

```
[solveur_pression solveur_sys_base]
[dt_projection deuxmots]
[traitement_particulier traitement_particulier]
[seuil_divU floatfloat]
[solveur_bar solveur_sys_base]
[projection_initiale int]
[postraiter_gradient_pression_sans_masse]
[methode_calcul_pression_initiale str into ['avec_les_cl', 'avec_sources', 'avec_sources_et-
_operateurs', 'sans_rien']]
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
```

}

where

- **solveur\_pression** *solveur\_sys\_base* (14.19) for inheritance: Linear pressure system resolution method.
- **dt\_projection** *deuxmots* (4.9.1) for inheritance: nb value : This keyword checks every nb time-steps the equality of velocity divergence to zero. value is the criteria convergency for the solver used.

- **traitement\_particulier** *traitement\_particulier* (5.18) for inheritance: Keyword to post-process particular values.
- **seuil\_divU** *floatfloat* (5.19) for inheritance: value factor : this keyword is intended to minimise the number of iterations during the pressure system resolution. The convergence criteria during this step ('seuil' in solveur\_pression) is dynamically adapted according to the mass conservation. At  $t_n$ , the linear system  $Ax=B$  is considered as solved if the residual  $\|Ax-B\| < \text{seuil}(t_n)$ . For  $t_{n+1}$ , the threshold value  $\text{seuil}(t_{n+1})$  will be evaluated as:  
 If (  $\text{lmax}(\text{DivU}) * \text{dt} < \text{value}$  )  
 Seuil( $t_{n+1}$ )= Seuil( $t_n$ )\*factor  
 Else  
 Seuil( $t_{n+1}$ )= Seuil( $t_n$ )\*factor  
 Endif  
 The first parameter (value) is the mass evolution the user is ready to accept per timestep, and the second one (factor) is the factor of evolution for 'seuil' (for example 1.1, so 10)
- **solveur\_bar** *solveur\_sys\_base* (14.19) for inheritance: This keyword is used to define when filtering operation is called (typically for EF convective scheme, standard diffusion operator and Source\_Qdm\_lambdaup ). A file (solveur.bar) is then created and used for inversion procedure. Syntax is the same then for pressure solver (GCP is required for multi-processor calculations and, in a general way, for big meshes).
- **projection\_initiale** *int* for inheritance: Keyword to suppress, if boolean equals 0, the initial projection which checks  $\text{DivU}=0$ . By default, boolean equals 1.
- **postraiter\_gradient\_pression\_sans\_masse** for inheritance: Avoid mass matrix multiplication for the gradient postprocessing
- **methode\_calcul\_pression\_initiale** *str into ['avec\_les\_cl', 'avec\_sources', 'avec\_sources\_et\_operateurs', 'sans\_rien']* for inheritance: Keyword to select an option for the pressure calculation before the first time step. Options are : avec\_les\_cl (default option lapP=0 is solved with Neuman boundary conditions on pressure if any), avec\_sources (lapP=f is solved with Neuman boundaries conditions and f integrating the source terms of the Navier-Stokes equations) and avec\_sources\_et\_operateurs (lapP=f is solved as with the previous option avec\_sources but f integrating also some operators of the Navier-Stokes equations). The two last options are useful and sometime necessary when source terms are implicated when using an implicit time scheme to solve the Navier-Stokes equations.
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limite** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time  $t_0$  and  $t_1$ .  
 Navier\_Sokes\_Standard  
 { equation\_non\_resolue ( $t > t_0$ )\*( $t < t_1$ ) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.25 Qdm\_multiphase

Description: Momentum conservation equation for a multi-phase problem where the unknown is the velocity

Keyword Discretize should have already been used to read the object.

See also: eqn\_base (5.45)

Usage:

**QDM\_Multiphase** *str*

**Read** *str* {

```
[solveur_pression solveur_sys_base]
[evanescence bloc_lecture]
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
```

}

where

- **solveur\_pression** *solveur\_sys\_base* (14.19): Linear pressure system resolution method.
- **evanescence** *bloc\_lecture* (3.2): Management of the vanishing phase (when alpha tends to 0 or 1)
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.26 Taux\_dissipation\_turbulent

Description: Turbulent Dissipation frequency equation for a turbulent mono/multi-phase problem (available in TrioCFD)

Keyword Discretize should have already been used to read the object.  
See also: eqn\_base (5.45)

Usage:

**Taux\_dissipation\_turbulent** *str*

**Read** *str* {

```
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
```

}

where

- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.27 Convection\_diffusion\_chaleur\_qc

Description: Temperature equation for a quasi-compressible fluid.

Keyword Discretize should have already been used to read the object.  
See also: eqn\_base (5.45) convection\_diffusion\_chaleur\_turbulent\_qc (5.29)

Usage:

**convection\_diffusion\_chaleur\_QC** *str*

**Read** *str* {

```
[mode_calcul_convection str into ['ancien', 'divuT_moins_Tdivu', 'divrhout_moins_Tdivrhout']]
```

```

[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
}
where

```

- **mode\_calcul\_convection** *str* into [*'ancien'*, *'divuT\_moins\_Tdivu'*, *'divrhout\_moins\_Tdivrhout'*]: Option to set the form of the convective operator  
divrhout\_moins\_Tdivrhout (the default since 1.6.8):  $\rho u \cdot \text{grad} T = \text{div}(\rho u \cdot T) - T \text{div}(\rho u)$   
ancien:  $u \cdot \text{grad} T = \text{div}(u \cdot T) - T \text{div}(u)$   
divuT\_moins\_Tdivu :  $u \cdot \text{grad} T = \text{div}(u \cdot T) - T \text{div}(u)$
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.28 Convection\_diffusion\_chaleur\_wc

Description: Temperature equation for a weakly-compressible fluid.

Keyword Discretize should have already been used to read the object.  
See also: eqn\_base (5.45)

Usage:

```

convection_diffusion_chaleur_WC str
Read str {

```

```

[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]

```



```

[boundary_conditions|conditions_limités condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
}
where

```

- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limités condlims** (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales condinits** (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.29 Convection\_diffusion\_chaleur\_turbulent\_qc

Description: Temperature equation for a quasi-compressible fluid as well as the associated turbulence model equations.

Keyword Discretize should have already been used to read the object.

See also: convection\_diffusion\_chaleur\_QC (5.27)

Usage:

**convection\_diffusion\_chaleur\_turbulent\_qc** *str*

**Read** *str* {

```

[modele_turbulence modele_turbulence_scal_base]
[mode_calcul_convection str into ['ancien', 'divuT_moins_Tdivu', 'divrhout_moins_Tdivrhout']]
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limités condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]

```



```

[equation_non_resolue str]
[renommer_equation str]
}

```

where

- **modele\_turbulence** *modele\_turbulence\_scal\_base* (29): Turbulence model for the temperature (energy) conservation equation.
- **mode\_calcul\_convection** *str* into [*'ancien'*, *'divuT\_moins\_Tdivu'*, *'divrhout\_moins\_Tdivrhout'*] for inheritance: Option to set the form of the convective operator  
divrhout\_moins\_Tdivrhout (the default since 1.6.8):  $\rho \cdot u \cdot \text{grad} T = \text{div}(\rho \cdot u \cdot T) - T \text{div}(\rho \cdot u)$   
ancien:  $u \cdot \text{grad} T = \text{div}(u \cdot T) - T \text{div}(u)$   
divuT\_moins\_Tdivu :  $u \cdot \text{grad} T = \text{div}(u \cdot T) - T \text{div}(u)$
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

### 5.30 Convection\_diffusion\_concentration

Description: Constituent transport vectorial equation (concentration diffusion convection).

Keyword Discretize should have already been used to read the object.

See also: eqn\_base (5.45) convection\_diffusion\_concentration\_turbulent (5.32) convection\_diffusion\_concentration\_ft\_disc (5.31) convection\_diffusion\_phase\_field (5.38)

Usage:

**convection\_diffusion\_concentration** *str*

**Read** *str* {

```

[nom_inconnue str]
[alias str]
[masse_molaire float]
[is_multi_scalar_diffusion is_multi_scalar]
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]

```

```

[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
}
where

```

- **nom\_inconnue** *str*: Keyword **Nom\_inconnue** will rename the unknown of this equation with the given name. In the postprocessing part, the concentration field will be accessible with this name. This is useful if you want to track more than one concentration (otherwise, only the concentration field in the first concentration equation can be accessed).
- **alias** *str*
- **masse\_molaire** *float*
- **is\_multi\_scalar\_diffusion|is\_multi\_scalar** : Flag to activate the multi\_scalar diffusion operator
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limite** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

### 5.31 Convection\_diffusion\_concentration\_ft\_disc

Description: not\_set

Keyword Discretize should have already been used to read the object.

See also: convection\_diffusion\_concentration (5.30)

Usage:

**convection\_diffusion\_concentration\_ft\_disc** *str*

**Read** *str* {

```

[equation_interface str]
phase int into [0, 1]
[option str]
[nom_inconnue str]
[alias str]

```

```

[masse_molaire float]
[is_multi_scalar_diffusionis_multi_scalar]
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditionsconditions_limites condlims]
[initial_conditionsconditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
}
where

```

- **equation\_interface** *str*: this is the name of the interface tracking equation to watch. The scalar will not diffuse through the interface of this equation.
- **phase** *int* into  $[0, 1]$ : tells whether the scalar must be confined in phase 0 or in phase 1
- **option** *str*: Experimental features used to prevent the concentration to leak through the interface between phases due to numerical diffusion.  
RIEN: do nothing  
RAMASSE\_MIETTES\_SIMPLE: at each timestep, this algorithm takes all the mass located in the opposite phase and spreads it uniformly in the given phase.
- **nom\_inconnue** *str* for inheritance: Keyword **Nom\_inconnue** will rename the unknown of this equation with the given name. In the postprocessing part, the concentration field will be accessible with this name. This is useful if you want to track more than one concentration (otherwise, only the concentration field in the first concentration equation can be accessed).
- **alias** *str* for inheritance
- **masse\_molaire** *float* for inheritance
- **is\_multi\_scalar\_diffusion****is\_multi\_scalar** for inheritance: Flag to activate the multi\_scalar diffusion operator
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions****conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions****conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

### 5.32 Convection\_diffusion\_concentration\_turbulent

Description: Constituent transport equations (concentration diffusion convection) as well as the associated turbulence model equations.

Keyword Discretize should have already been used to read the object.

See also: convection\_diffusion\_concentration (5.30) Convection\_Diffusion\_Concentration\_Turbulent\_FT-Disc (5.8)

Usage:

**convection\_diffusion\_concentration\_turbulent** *str*

```
Read str {
 [modele_turbulence modele_turbulence_scal_base]
 [nom_inconnue str]
 [alias str]
 [masse_molaire float]
 [is_multi_scalar_diffusion lis_multi_scalar]
 [disable_equation_residual int]
 [convection bloc_convection]
 [diffusion bloc_diffusion]
 [boundary_conditions|conditions_limites condlims]
 [initial_conditions|conditions_initiales condinits]
 [sources sources]
 [ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
 [parametre_equation parametre_equation_base]
 [equation_non_resolue str]
 [renommer_equation str]
}
```

where

- **modele\_turbulence** *modele\_turbulence\_scal\_base* (29): Turbulence model to be used in the constituent transport equations. The only model currently available is Schmidt.
- **nom\_inconnue** *str* for inheritance: Keyword **Nom\_inconnue** will rename the unknown of this equation with the given name. In the postprocessing part, the concentration field will be accessible with this name. This is useful if you want to track more than one concentration (otherwise, only the concentration field in the first concentration equation can be accessed).
- **alias** *str* for inheritance
- **masse\_molaire** *float* for inheritance
- **is\_multi\_scalar\_diffusion** *lis\_multi\_scalar* for inheritance: Flag to activate the multi\_scalar diffusion operator
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation

- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

### 5.33 Convection\_diffusion\_espece\_binaire\_qc

Description: Species conservation equation for a binary quasi-compressible fluid.

Keyword Discretize should have already been used to read the object.

See also: eqn\_base (5.45) Convection\_Diffusion\_Espece\_Binaire\_Turbulent\_QC (5.9)

Usage:

**convection\_diffusion\_espece\_binaire\_QC** *str*

**Read** *str* {

```
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
```

}

where

- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

### 5.34 Convection\_diffusion\_espece\_binaire\_wc

Description: Species conservation equation for a binary weakly-compressible fluid.

Keyword Discretize should have already been used to read the object.

See also: eqn\_base (5.45)

Usage:

**convection\_diffusion\_espece\_binaire\_WC** *str*

**Read** *str* {

```
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
```

}

where

- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

### 5.35 Convection\_diffusion\_espece\_multi\_qc

Description: Species conservation equation for a multi-species quasi-compressible fluid.

Keyword Discretize should have already been used to read the object.

See also: eqn\_base (5.45)

Usage:

```

convection_diffusion_espece_multi_QC str
Read str {
 [espece espece]
 [disable_equation_residual int]
 [convection bloc_convection]
 [diffusion bloc_diffusion]
 [boundary_conditions|conditions_limites condlims]
 [initial_conditions|conditions_initiales condinits]
 [sources sources]
 [ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
 [parametre_equation parametre_equation_base]
 [equation_non_resolue str]
 [renommer_equation str]
}
where

```

- **espece** *espece* (3.59): Associate a species (with its properties) to the equation
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

### 5.36 Convection\_diffusion\_espece\_multi\_wc

Description: Species conservation equation for a multi-species weakly-compressible fluid.

Keyword Discretize should have already been used to read the object.

See also: eqn\_base (5.45)

Usage:

```

convection_diffusion_espece_multi_WC str
Read str {

```

```

 [disable_equation_residual int]
 [convection bloc_convection]
 [diffusion bloc_diffusion]

```

```

[boundary_conditions|conditions_limits condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
}
where

```

- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limit**s *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_init**iales *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

### 5.37 Convection\_diffusion\_espece\_multi\_turbulent\_qc

Description: not\_set

Keyword Discretize should have already been used to read the object.  
See also: eqn\_base (5.45)

Usage:

**convection\_diffusion\_espece\_multi\_turbulent\_qc** *str*  
**Read** *str* {

```

[modele_turbulence modele_turbulence_scal_base]
espece espece
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limits condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]

```



```
[renommer_equation str]
}
```

where

- **modele\_turbulence** *modele\_turbulence\_scal\_base* (29): Turbulence model to be used.
- **espece** *espece* (3.59)
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

### 5.38 Convection\_diffusion\_phase\_field

Description: Cahn-Hilliard equation of the Phase Field problem. The unknown of this equation is the concentration C.

Keyword Discretize should have already been used to read the object.

See also: convection\_diffusion\_concentration (5.30)

Usage:

**convection\_diffusion\_phase\_field** *str*

**Read** *str* {

```
[mu_1 float]
[mu_2 float]
[rho_1 float]
[rho_2 float]
potentiel_chimique_generalise str into ['avec_energie_cinetique', 'sans_energie_cinetique']
[nom_inconnue str]
[alias str]
[masse_molaire float]
[is_multi_scalar_diffusion|is_multi_scalar]
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
```

```

[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
}
where

```

- **mu\_1** *float*: Dynamic viscosity of the first phase.
- **mu\_2** *float*: Dynamic viscosity of the second phase.
- **rho\_1** *float*: Density of the first phase.
- **rho\_2** *float*: Density of the second phase.
- **potentiel\_chimique\_generalise** *str* into [*'avec\_energie\_cinetique'*, *'sans\_energie\_cinetique'*]: To define (chaîne set to *avec\_energie\_cinetique*) or not (chaîne set to *sans\_energie\_cinetique*) if the Cahn-Hilliard equation contains the cinetic energy term.
- **nom\_inconnue** *str* for inheritance: Keyword *Nom\_inconnue* will rename the unknown of this equation with the given name. In the postprocessing part, the concentration field will be accessible with this name. This is usefull if you want to track more than one concentration (otherwise, only the concentration field in the first concentration equation can be accessed).
- **alias** *str* for inheritance
- **masse\_molaire** *float* for inheritance
- **is\_multi\_scalar\_diffusion****is\_multi\_scalar** for inheritance: Flag to activate the multi\_scalar diffusion operator
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions****conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions****conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if *equation\_non\_resolue* keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ *equation\_non\_resolue* (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

### 5.39 Convection\_diffusion\_temperature

Description: Energy equation (temperature diffusion convection).

Keyword Discretize should have already been used to read the object.

See also: *eqn\_base* (5.45) *convection\_diffusion\_temperature\_ibm* (5.42) *convection\_diffusion\_temperature\_ft\_disc* (5.40) *Convection\_Diffusion\_Temperature\_sensibility* (5.10)

Usage:

```

convection_diffusion_temperature str
Read str {
 [penalisation_l2_ftd bloc_lecture]
 [disable_equation_residual int]
 [convection bloc_convection]
 [diffusion bloc_diffusion]
 [boundary_conditions|conditions_limites condlims]
 [initial_conditions|conditions_initiales condinits]
 [sources sources]
 [ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
 [parametre_equation parametre_equation_base]
 [equation_non_resolue str]
 [renommer_equation str]
}
where

```

- **penalisation\_l2\_ftd** *bloc\_lecture* (3.2): to activate or not (the default is Direct Forcing method) the Penalized Direct Forcing method to impose the specified temperature on the solid-fluid interface.
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.40 Convection\_diffusion\_temperature\_ft\_disc

Description: not\_set

Keyword Discretize should have already been used to read the object.  
See also: convection\_diffusion\_temperature (5.39)

Usage:

```

convection_diffusion_temperature_ft_disc str
Read str {
 [equation_interface str]
 phase int into [0, 1]
}

```

```

[equation_navier_stokes str]
[stencil_width int]
[maintien_temperature objet_lecture_maintien_temperature]
[prescribed_mpoint float]
[correction_mpoint_diff_conv_energy n x1 x2 ... xn]
[correction_gradt_ordre int]
[penalisation_l2_ftd bloc_lecture]
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
}

```

where

- **equation\_interface** *str*: The name of the interface equation should be given.
- **phase** *int* into  $[0, 1]$ : Phase in which the temperature equation will be solved. The temperature, which may be postprocessed with the keyword `temperature_EquationName`, in the other phase may be negative: the code only computes the temperature field in the specified phase. The other phase is supposed to physically stay at saturation temperature. The code uses a ghost fluid numerical method to work on a smooth temperature field at the interface. In the opposite phase (1-X) the temperature will therefore be extrapolated in the vicinity of the interface and have the opposite sign, saturation temperature is zero by convention).
- **equation\_navier\_stokes** *str*: The name of the Navier Stokes equation of the problem should be given.
- **stencil\_width** *int*: distance in mesh elements over which the temperature field should be extrapolated in the opposite phase.
- **maintien\_temperature** *objet\_lecture\_maintien\_temperature* (5.41): `maintien_temperature SOUS_ZONE_NAME VALUE` : experimental, this acts as a dynamic source term that heats or cools the fluid to maintain the average temperature to `VALUE` within the specified region. At this time, this is done by multiplying the temperature within the `SOUS_ZONE` by an appropriate uniform value at each timestep. This feature might be implemented in a separate source term in the future.
- **prescribed\_mpoint** *float*: User defined value of the phase-change rate (override the value computed based on the temperature field)
- **correction\_mpoint\_diff\_conv\_energy** *n x1 x2 ... xn*
- **correction\_gradt\_ordre** *int*
- **penalisation\_l2\_ftd** *bloc\_lecture* (3.2) for inheritance: to activate or not (the default is Direct Forcing method) the Penalized Direct Forcing method to impose the specified temperature on the solid-fluid interface.
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)

- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.41 Objet\_lecture\_maintien\_temperature

Description: not\_set

See also: objet\_lecture (47)

Usage:

**sous\_zone** **temperature\_moyenne**

where

- **sous\_zone** *str*
- **temperature\_moyenne** *float*

## 5.42 Convection\_diffusion\_temperature\_ibm

Description: IBM Energy equation (temperature diffusion convection).

Keyword Discretize should have already been used to read the object.

See also: convection\_diffusion\_temperature (5.39)

Usage:

**convection\_diffusion\_temperature\_ibm** *str*

**Read** *str* {

```
[correction_variable_initiale int]
[penalisation_l2_ftd bloc_lecture]
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
```

}

where

- **correction\_variable\_initiale** *int*: Modify initial variable
- **penalisation\_l2\_ftd** *bloc\_lecture* (3.2) for inheritance: to activate or not (the default is Direct Forcing method) the Penalized Direct Forcing method to impose the specified temperature on the solid-fluid interface.

- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limtes** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

### 5.43 Convection\_diffusion\_temperature\_ibm\_turbulent

Description: IBM Energy equation (temperature diffusion convection) as well as the associated turbulence model equations.

Keyword Discretize should have already been used to read the object.

See also: eqn\_base (5.45)

Usage:

**convection\_diffusion\_temperature\_ibm\_turbulent** *str*

**Read** *str* {

```
[modele_turbulence modele_turbulence_scal_base]
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limtes condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
```

}

where

- **modele\_turbulence** *modele\_turbulence\_scal\_base* (29): Turbulence model for the energy equation.
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.

- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.44 Convection\_diffusion\_temperature\_turbulent

Description: Energy equation (temperature diffusion convection) as well as the associated turbulence model equations.

Keyword Discretize should have already been used to read the object.

See also: eqn\_base (5.45)

Usage:

**convection\_diffusion\_temperature\_turbulent** *str*

**Read** *str* {

```
[modele_turbulence modele_turbulence_scal_base]
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
```

}

where

- **modele\_turbulence** *modele\_turbulence\_scal\_base* (29): Turbulence model for the energy equation.
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.

- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.45 Eqn\_base

Description: Basic class for equations.

Keyword Discretize should have already been used to read the object.

See also: mor\_eqn (5) Conduction (5.1) convection\_diffusion\_temperature\_ibm\_turbulent (5.43) navier\_stokes\_standard (5.57) convection\_diffusion\_temperature (5.39) convection\_diffusion\_concentration (5.30) convection\_diffusion\_espece\_binaire\_WC (5.34) convection\_diffusion\_chaleur\_WC (5.28) convection\_diffusion\_espece\_multi\_WC (5.36) convection\_diffusion\_chaleur\_QC (5.27) convection\_diffusion\_espece\_binaire\_QC (5.33) convection\_diffusion\_espece\_multi\_QC (5.35) Masse\_Multiphase (5.16) QDM\_Multiphase (5.25) Energie\_Multiphase\_h (5.13) Energie\_Multiphase (5.12) Echelle\_temporelle\_turbulente (5.11) Energie\_cinetique\_turbulente (5.14) Energie\_cinetique\_turbulente\_WIT (5.15) Taux\_dissipation\_turbulent (5.26) convection\_diffusion\_temperature\_turbulent (5.44) convection\_diffusion\_espece\_multi\_turbulent\_qc (5.37) transport\_k (5.67) transport\_epsilon (5.60) transport\_interfaces\_ft\_disc (5.61) transport\_marqueur\_ft (5.68) Navier\_Stokes\_FTD\_IJK (5.20) ijk\_thermals (5.47) ijk\_interfaces (5.46)

Usage:

**eqn\_base** *str*

**Read** *str* {

```
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
```

}

where

- **disable\_equation\_residual** *int*: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2): Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3): Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1): Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4): Initial conditions.



- **sources** *sources* (5.5): To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57): This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6): Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str*: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str*: Rename the equation with a specific name.

## 5.46 Ijk\_interfaces

Description: Interface tracking equation for Front-Tracking problem in the discontinuous version.

Keyword Discretize should have already been used to read the object.

See also: eqn\_base (5.45)

Usage:

**ijk\_interfaces** *str*

**Read** *str* {

```
[fichier_reprise_interface str]
[timestep_reprise_interface float]
[lata_meshname str]
[remaillage_ft_ijk bloc_lecture]
[parcours_interface bloc_lecture]
[maillage_ft_ijk bloc_lecture]
[terme_gravite str]
[use_tryggvason_interfacial_source]
[compute_distance_autres_interfaces]
[avoid_duplicata]
[portee_force_repulsion float]
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
```

}

where

- **fichier\_reprise\_interface** *str*
- **timestep\_reprise\_interface** *float*
- **lata\_meshname** *str*
- **remaillage\_ft\_ijk** *bloc\_lecture* (3.2)

- **parcours\_interface** *bloc\_lecture* (3.2)
- **maillage\_ft\_ijk** *bloc\_lecture* (3.2)
- **terme\_gravite** *str*
- **use\_tryggvason\_interfacial\_source**
- **compute\_distance\_autres\_interfaces**
- **avoid\_duplicata**
- **portee\_force\_repulsion** *float*
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.47 Ijk\_thermals

Description: not\_set

Keyword Discretize should have already been used to read the object.

See also: eqn\_base (5.45)

Usage:

**ijk\_thermals** *str*

**Read** *str* {

```
[ONEFLUID bloc_lecture
[disable_equation_residual int
[convection bloc_convection
[diffusion bloc_diffusion
[boundary_conditions|conditions_limites condlims
[initial_conditions|conditions_initiales condinits
[sources sources
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur
[parametre_equation parametre_equation_base
[equation_non_resolue str
[renommer_equation str
```

}

where

- **ONEFLUID** *bloc\_lecture* (3.2)
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.48 Navier\_stokes\_qc

Description: Navier-Stokes equation for a quasi-compressible fluid.

Keyword Discretize should have already been used to read the object.

See also: *navier\_stokes\_standard* (5.57)

Usage:

**navier\_stokes\_QC** *str*

**Read** *str* {

```
[solveur_pression solveur_sys_base]
[dt_projection deuxmots]
[traitement_particulier traitement_particulier]
[seuil_divU floatfloat]
[solveur_bar solveur_sys_base]
[projection_initiale int]
[postraiter_gradient_pression_sans_masse]
[methode_calcul_pression_initiale str into ['avec_les_cl', 'avec_sources', 'avec_sources_et-
_operateurs', 'sans_rien']]
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
```

}

where

- **solveur\_pression** *solveur\_sys\_base* (14.19) for inheritance: Linear pressure system resolution method.
- **dt\_projection** *deuxmots* (4.9.1) for inheritance: nb value : This keyword checks every nb time-steps the equality of velocity divergence to zero. value is the criteria convergency for the solver used.
- **traitement\_particulier** *traitement\_particulier* (5.18) for inheritance: Keyword to post-process particular values.
- **seuil\_divU** *floatfloat* (5.19) for inheritance: value factor : this keyword is intended to minimise the number of iterations during the pressure system resolution. The convergence criteria during this step ('seuil' in *solveur\_pression*) is dynamically adapted according to the mass conservation. At  $t_n$ , the linear system  $Ax=B$  is considered as solved if the residual  $\|Ax-B\| < \text{seuil}(t_n)$ . For  $t_{n+1}$ , the threshold value  $\text{seuil}(t_{n+1})$  will be evaluated as:  
 If (  $\text{lmax}(\text{DivU}) * dt < \text{value}$  )  
 $\text{Seuil}(t_{n+1}) = \text{Seuil}(t_n) * \text{factor}$   
 Else  
 $\text{Seuil}(t_{n+1}) = \text{Seuil}(t_n) * \text{factor}$   
 Endif  
 The first parameter (value) is the mass evolution the user is ready to accept per timestep, and the second one (factor) is the factor of evolution for 'seuil' (for example 1.1, so 10)
- **solveur\_bar** *solveur\_sys\_base* (14.19) for inheritance: This keyword is used to define when filtering operation is called (typically for EF convective scheme, standard diffusion operator and *Source\_Qdm\_lambdaup*). A file (*solveur.bar*) is then created and used for inversion procedure. Syntax is the same then for pressure solver (GCP is required for multi-processor calculations and, in a general way, for big meshes).
- **projection\_initiale** *int* for inheritance: Keyword to suppress, if boolean equals 0, the initial projection which checks  $\text{DivU}=0$ . By default, boolean equals 1.
- **postraiter\_gradient\_pression\_sans\_masse** for inheritance: Avoid mass matrix multiplication for the gradient postprocessing
- **methode\_calcul\_pression\_initiale** *str into ['avec\_les\_cl', 'avec\_sources', 'avec\_sources\_et\_operateurs', 'sans\_rien']* for inheritance: Keyword to select an option for the pressure calculation before the first time step. Options are : *avec\_les\_cl* (default option  $\text{lapP}=0$  is solved with Neuman boundary conditions on pressure if any), *avec\_sources* ( $\text{lapP}=f$  is solved with Neuman boundaries conditions and  $f$  integrating the source terms of the Navier-Stokes equations) and *avec\_sources\_et\_operateurs* ( $\text{lapP}=f$  is solved as with the previous option *avec\_sources* but  $f$  integrating also some operators of the Navier-Stokes equations). The two last options are useful and sometime necessary when source terms are implicated when using an implicit time scheme to solve the Navier-Stokes equations.
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if *equation\_non\_resolue* keyword is used. Example: The Navier-Stokes equations are not solved between time  $t_0$  and  $t_1$ .

```
Navier_Sokes_Standard
{ equation_non_resolue (t>t0)*(t<t1) }
```

- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.49 Navier\_stokes\_wc

Description: Navier-Stokes equation for a weakly-compressible fluid.

Keyword Discretize should have already been used to read the object.

See also: `navier_stokes_standard` (5.57)

Usage:

**navier\_stokes\_WC** *str*

**Read** *str* {

```
[mass_source mass_source]
[solveur_pression solveur_sys_base]
[dt_projection deuxmots]
[traitement_particulier traitement_particulier]
[seuil_divU floatfloat]
[solveur_bar solveur_sys_base]
[projection_initiale int]
[postraiter_gradient_pression_sans_masse]
[methode_calcul_pression_initiale str into ['avec_les_cl', 'avec_sources', 'avec_sources_et-
_operateurs', 'sans_rien']]
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
```

}

where

- **mass\_source** *mass\_source* (3.92): Mass source used in a dilatable simulation to add/reduce a mass at the boundary (volumetric source in the first cell of a given boundary).
- **solveur\_pression** *solveur\_sys\_base* (14.19) for inheritance: Linear pressure system resolution method.
- **dt\_projection** *deuxmots* (4.9.1) for inheritance: nb value : This keyword checks every nb time-steps the equality of velocity divergence to zero. value is the criteria convergency for the solver used.
- **traitement\_particulier** *traitement\_particulier* (5.18) for inheritance: Keyword to post-process particular values.
- **seuil\_divU** *floatfloat* (5.19) for inheritance: value factor : this keyword is intended to minimise the number of iterations during the pressure system resolution. The convergence criteria during this step ('seuil' in `solveur_pression`) is dynamically adapted according to the mass conservation. At  $t_n$ , the linear system  $Ax=B$  is considered as solved if the residual  $\|Ax-B\| < \text{seuil}(t_n)$ . For  $t_{n+1}$ , the threshold value  $\text{seuil}(t_{n+1})$  will be evaluated as:  
If  $(\text{lmax}(\text{DivU}) * dt < \text{value})$   
 $\text{Seuil}(t_{n+1}) = \text{Seuil}(t_n) * \text{factor}$

```
Else
Seuil(tn+1)= Seuil(tn)*factor
Endif
```

The first parameter (value) is the mass evolution the user is ready to accept per timestep, and the second one (factor) is the factor of evolution for 'seuil' (for example 1.1, so 10

- **solveur\_bar** *solveur\_sys\_base* (14.19) for inheritance: This keyword is used to define when filtering operation is called (typically for EF convective scheme, standard diffusion operator and Source\_Qdm\_lambdaup ). A file (solveur.bar) is then created and used for inversion procedure. Syntax is the same then for pressure solver (GCP is required for multi-processor calculations and, in a general way, for big meshes).
- **projection\_initiale** *int* for inheritance: Keyword to suppress, if boolean equals 0, the initial projection which checks DivU=0. By default, boolean equals 1.
- **postraiter\_gradient\_pression\_sans\_masse** for inheritance: Avoid mass matrix multiplication for the gradient postprocessing
- **methode\_calcul\_pression\_initiale** *str into* ['avec\_les\_cl', 'avec\_sources', 'avec\_sources\_et\_operateurs', 'sans\_rien'] for inheritance: Keyword to select an option for the pressure calculation before the first time step. Options are : avec\_les\_cl (default option lapP=0 is solved with Neuman boundary conditions on pressure if any), avec\_sources (lapP=f is solved with Neuman boundaries conditions and f integrating the source terms of the Navier-Stokes equations) and avec\_sources\_et\_operateurs (lapP=f is solved as with the previous option avec\_sources but f integrating also some operators of the Navier-Stokes equations). The two last options are useful and sometime necessary when source terms are implicated when using an implicit time scheme to solve the Navier-Stokes equations.
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1.  

```
Navier_Sokes_Standard
{ equation_non_resolue (t>t0)*(t<t1) }
```
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.50 Navier\_stokes\_ft\_disc

Description: Two-phase momentum balance equation.

Keyword Discretize should have already been used to read the object.

See also: navier\_stokes\_turbulent (5.58)

Usage:

**navier\_stokes\_ft\_disc** *str*

**Read** *str* {

```

[equation_interfaces_proprietes_fluide str]
[equation_interfaces_vitesse_imposee str]
[equations_interfaces_vitesse_imposee n word1 word2 ... wordn]
[clipping_courbure_interface int]
[terme_gravite str into ['rho_g', 'grad_i']]
[equation_temperature_mpoint str]
[matrice_pression_invariante]
[penalisation_forage penalisation_forage]
[equation_temperature_mpoint_vapeur str]
[mpoint_inactif_sur_qdm]
[mpoint_vapeur_inactif_sur_qdm]
[new_mass_source]
[shift_secmem2]
[interpol_indic_pour_dI_dt str into ['interp_ai_based', 'interp_standard', 'interp_modifiee']]
[OutletCorrection_pour_dI_dt str into ['CORRECTION_GHOST_INDIC']]
[boussinesq_approximation]
[modele_turbulence modele_turbulence_hyd_deriv]
[solveur_pression solveur_sys_base]
[dt_projection deuxmots]
[traitement_particulier traitement_particulier]
[seuil_divU floatfloat]
[solveur_bar solveur_sys_base]
[projection_initiale int]
[postraiter_gradient_pression_sans_masse]
[methode_calcul_pression_initiale str into ['avec_les_cl', 'avec_sources', 'avec_sources_et-
_operateurs', 'sans_rien']]
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
}
where

```

- **equation\_interfaces\_proprietes\_fluide** *str*: This keyword is used for liquid-gas, liquid-vapor and fluid-fluid deformable interface, which transported at the Eulerian velocity. When this case is selected, the keyword sequence `Methode_transport vitesse_interpolee` is used in the block `Transport_Interfaces_FT_Disc` to define the velocity field for the displacement of the interface.
- **equation\_interfaces\_vitesse\_imposee** *str*: This keyword is used to specify the velocity field to be used when using an interface that mimics a solid interface moving with a given solid speed of displacement. When this case is selected, the keyword sequence `Methode_transport vitesse_imposee` in the `Transport_Interfaces_FT_Disc` block will define the velocity field for the displacement of the interface.
- **equations\_interfaces\_vitesse\_imposee** *n word1 word2 ... wordn*: This keyword is used to specify the velocity field to be used when using an interface that mimics a solid interface moving with a given solid speed of displacement. When this case is selected, the keyword sequence `Methode_transport vitesse_imposee` in the `Transport_Interfaces_FT_Disc` block will define the velocity field for the displacement of the interface. If two or more solid interfaces are defined, then the keyword `equations_interfaces_vitesse_imposee` should be used.

- **clipping\_courbure\_interface** *int*: This keyword is used to numerically limit the values of curvature used in the momentum balance equation. Curvature is computed as usual, but values exceeding the clipping value are replaced by this threshold, before using the clipped curvature in the momentum balance. Each time a curvature value is clipped, a counter is increased by one unity and the value of the counter is written in the .err file at the end of the time step. This clipping allows not reducing drastically the time stepping when a geometrical singularity occurs in the interface mesh. However, physical phenomena may be concealed with the use of such a clipping.
- **terme\_gravite** *str into ['rho\_g', 'grad\_i']*: The Terme\_gravite keyword changes the numerical scheme used for the gravity source term. The default is grad\_i, which is designed to remove spurious currents around the interface. In this case, the pressure field does not contain the hydrostatic part but only a jump across the interface. This scheme seems not to work very well in vef. The rho\_g option uses the more traditional source term, equal to rho\*g in the volume. In this case, the hydrostatic pressure is visible in the pressure field and the boundary conditions in pressure must be set accordingly. This model produces spurious currents in the vicinity of the fluid-fluid interfaces and with the immersed boundary conditions.
- **equation\_temperature\_mpoint** *str*: The equation\_temperature\_mpoint should be used in the case of liquid-vapor flow with phase-change (see the TRUST\_ROOT/doc/TRUST/ft\_chgt\_phase.pdf written in French for more information about the model). The name of the temperature equation, defined with the convection\_diffusion\_temperature\_ft\_disc keyword, should be given.
- **matrice\_pression\_invariante**: This keyword is a shortcut to be used only when the flow is a single-phase one, with interface tracking only used for solid-fluid interfaces. In this peculiar case, the density of the fluid does not evolve during the computation and the pressure matrix does not need to be actuated at each time step.
- **penalisation\_forage** *penalisation\_forage* (5.51): This keyword is used to specify a strong formulation (value set to 0) or a weak formulation (value set to 1) for an imposed pressure boundary condition. The first formulation converges quicker and is stable in general cases except some rare cases (see Ecoulement\_Neumann test case for example) where the second one should be used despite of its slow convergence.
- **equation\_temperature\_mpoint\_vapeur** *str*
- **mpoint\_inactif\_sur\_qdm**
- **mpoint\_vapeur\_inactif\_sur\_qdm**
- **new\_mass\_source**: Flag for localised computation of velocity jump based on interfacial area AI (advanced option)
- **shift\_secmem2**
- **interp\_indic\_pour\_dI\_dt** *str into ['interp\_ai\_based', 'interp\_standard', 'interp\_modifiee']*: Specific interpolation of phase indicator function in VoF mass-preserving method (advanced option)
- **OutletCorrection\_pour\_dI\_dt** *str into ['CORRECTION\_GHOST\_INDIC']*
- **boussinesq\_approximation**
- **modele\_turbulence** *modele\_turbulence\_hyd\_deriv* (5.22) for inheritance: Turbulence model for Navier-Stokes equations.
- **solveur\_pression** *solveur\_sys\_base* (14.19) for inheritance: Linear pressure system resolution method.
- **dt\_projection** *deuxmots* (4.9.1) for inheritance: nb value: This keyword checks every nb time-steps the equality of velocity divergence to zero. value is the criteria convergency for the solver used.
- **traitement\_particulier** *traitement\_particulier* (5.18) for inheritance: Keyword to post-process particular values.
- **seuil\_divU** *floatfloat* (5.19) for inheritance: value factor: this keyword is intended to minimise the number of iterations during the pressure system resolution. The convergence criteria during this step ('seuil' in solveur\_pression) is dynamically adapted according to the mass conservation. At tn, the linear system Ax=B is considered as solved if the residual ||Ax-B||<seuil(tn). For tn+1, the threshold value seuil(tn+1) will be evaluated as:  
 If ( lmax(DivU)\*dt<value )  
 Seuil(tn+1)= Seuil(tn)\*factor  
 Else



```
Seuil(tn+1)= Seuil(tn)*factor
Endif
```

The first parameter (value) is the mass evolution the user is ready to accept per timestep, and the second one (factor) is the factor of evolution for 'seuil' (for example 1.1, so 10

- **solveur\_bar** *solveur\_sys\_base* (14.19) for inheritance: This keyword is used to define when filtering operation is called (typically for EF convective scheme, standard diffusion operator and Source\_Qdm\_lambdaup ). A file (solveur.bar) is then created and used for inversion procedure. Syntax is the same then for pressure solver (GCP is required for multi-processor calculations and, in a general way, for big meshes).
- **projection\_initiale** *int* for inheritance: Keyword to suppress, if boolean equals 0, the initial projection which checks DivU=0. By default, boolean equals 1.
- **postraiter\_gradient\_pression\_sans\_masse** for inheritance: Avoid mass matrix multiplication for the gradient postprocessing
- **methode\_calcul\_pression\_initiale** *str into* ['avec\_les\_cl', 'avec\_sources', 'avec\_sources\_et\_operateurs', 'sans\_rien'] for inheritance: Keyword to select an option for the pressure calculation before the first time step. Options are : avec\_les\_cl (default option lapP=0 is solved with Neuman boundary conditions on pressure if any), avec\_sources (lapP=f is solved with Neuman boundaries conditions and f integrating the source terms of the Navier-Stokes equations) and avec\_sources\_et\_operateurs (lapP=f is solved as with the previous option avec\_sources but f integrating also some operators of the Navier-Stokes equations). The two last options are useful and sometime necessary when source terms are implicated when using an implicit time scheme to solve the Navier-Stokes equations.
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limite** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1.  

```
Navier_Sokes_Standard
{ equation_non_resolue (t>t0)*(t<t1) }
```
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.51 Penalisation\_forcage

Description: penalisation\_forcage

See also: objet\_lecture (47)

Usage:

```
{
 [pression_reference float]
 [domaine_flottant_fluide x1 x2 (x3)]
}
```

where

- **pression\_reference** *float*
- **domaine\_flottant\_fluide** *x1 x2 (x3)*

## 5.52 Navier\_stokes\_ibm

Description: IBM Navier-Stokes equations.

Keyword Discretize should have already been used to read the object.

See also: `navier_stokes_standard` (5.57)

Usage:

**navier\_stokes\_ibm** *str*

**Read** *str* {

```
[correction_matrice_projection_initiale int]
[correction_calcul_pression_initiale int]
[correction_vitesse_projection_initiale int]
[correction_matrice_pression int]
[matrice_pression_penalisee_H1 int]
[correction_vitesse_modifie int]
[correction_pression_modifie int]
[gradient_pression_qdm_modifie int]
[correction_variable_initiale int]
[solveur_pression solveur_sys_base]
[dt_projection deuxmots]
[traitement_particulier traitement_particulier]
[seuil_divU floatfloat]
[solveur_bar solveur_sys_base]
[projection_initiale int]
[postraiter_gradient_pression_sans_masse]
[methode_calcul_pression_initiale str into ['avec_les_cl', 'avec_sources', 'avec_sources_et-
_operateurs', 'sans_rien']]
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
```

}

where

- **correction\_matrice\_projection\_initiale** *int*: (IBM advanced) fix matrix of initial projection for PDF
- **correction\_calcul\_pression\_initiale** *int*: (IBM advanced) fix initial pressure computation for PDF
- **correction\_vitesse\_projection\_initiale** *int*: (IBM advanced) fix initial velocity computation for PDF
- **correction\_matrice\_pression** *int*: (IBM advanced) fix pressure matrix for PDF

- **matrice\_pression\_penalisee\_H1** *int*: (IBM advanced) fix pressure matrix for PDF
  - **correction\_vitesse\_modifie** *int*: (IBM advanced) fix velocity for PDF
  - **correction\_pression\_modifie** *int*: (IBM advanced) fix pressure for PDF
  - **gradient\_pression\_qdm\_modifie** *int*: (IBM advanced) fix pressure gradient
  - **correction\_variable\_initiale** *int*: Modify initial variable
  - **solveur\_pression** *solveur\_sys\_base* (14.19) for inheritance: Linear pressure system resolution method.
- 
- **dt\_projection** *deuxmots* (4.9.1) for inheritance: nb value : This keyword checks every nb time-steps the equality of velocity divergence to zero. value is the criteria convergency for the solver used.
  - **traitement\_particulier** *traitement\_particulier* (5.18) for inheritance: Keyword to post-process particular values.
  - **seuil\_divU** *floatfloat* (5.19) for inheritance: value factor : this keyword is intended to minimise the number of iterations during the pressure system resolution. The convergence criteria during this step ('seuil' in *solveur\_pression*) is dynamically adapted according to the mass conservation. At  $t_n$ , the linear system  $Ax=B$  is considered as solved if the residual  $\|Ax-B\| < \text{seuil}(t_n)$ . For  $t_{n+1}$ , the threshold value  $\text{seuil}(t_{n+1})$  will be evaluated as:  
 If (  $\text{lmax}(\text{DivU}) \cdot dt < \text{value}$  )  
    $\text{Seuil}(t_{n+1}) = \text{Seuil}(t_n) \cdot \text{factor}$   
 Else  
    $\text{Seuil}(t_{n+1}) = \text{Seuil}(t_n) \cdot \text{factor}$   
 Endif  
 The first parameter (value) is the mass evolution the user is ready to accept per timestep, and the second one (factor) is the factor of evolution for 'seuil' (for example 1.1, so 10)
  - **solveur\_bar** *solveur\_sys\_base* (14.19) for inheritance: This keyword is used to define when filtering operation is called (typically for EF convective scheme, standard diffusion operator and *Source\_Qdm\_lambdaup*). A file (*solveur.bar*) is then created and used for inversion procedure. Syntax is the same then for pressure solver (GCP is required for multi-processor calculations and, in a general way, for big meshes).
  - **projection\_initiale** *int* for inheritance: Keyword to suppress, if boolean equals 0, the initial projection which checks  $\text{DivU}=0$ . By default, boolean equals 1.
  - **postraiter\_gradient\_pression\_sans\_masse** for inheritance: Avoid mass matrix multiplication for the gradient postprocessing
  - **methode\_calcul\_pression\_initiale** *str into* ['avec\_les\_cl', 'avec\_sources', 'avec\_sources\_et\_operateurs', 'sans\_rien'] for inheritance: Keyword to select an option for the pressure calculation before the first time step. Options are : *avec\_les\_cl* (default option  $\text{lapP}=0$  is solved with Neuman boundary conditions on pressure if any), *avec\_sources* ( $\text{lapP}=f$  is solved with Neuman boundaries conditions and  $f$  integrating the source terms of the Navier-Stokes equations) and *avec\_sources\_et\_operateurs* ( $\text{lapP}=f$  is solved as with the previous option *avec\_sources* but  $f$  integrating also some operators of the Navier-Stokes equations). The two last options are useful and sometime necessary when source terms are implicated when using an implicit time scheme to solve the Navier-Stokes equations.
  - **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
  - **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
  - **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
  - **boundary\_conditions|conditions\_limite** *condlims* (4.39.1) for inheritance: Boundary conditions.
- 
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
  - **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
  - **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
  - **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation

- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

### 5.53 Navier\_stokes\_ibm\_turbulent

Description: IBM Navier-Stokes equations as well as the associated turbulence model equations.

Keyword Discretize should have already been used to read the object.

See also: navier\_stokes\_standard (5.57)

Usage:

**navier\_stokes\_ibm\_turbulent** *str*

**Read** *str* {

```
[modele_turbulence modele_turbulence_hyd_deriv]
[solveur_pression solveur_sys_base]
[dt_projection deuxmots]
[traitement_particulier traitement_particulier]
[seuil_divU floatfloat]
[solveur_bar solveur_sys_base]
[projection_initiale int]
[postraiter_gradient_pression_sans_masse]
[methode_calcul_pression_initiale str into ['avec_les_cl', 'avec_sources', 'avec_sources_et-
_operateurs', 'sans_rien']]
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
```

}

where

- **modele\_turbulence** *modele\_turbulence\_hyd\_deriv* (5.22): Turbulence model for Navier-Stokes equations.
- **solveur\_pression** *solveur\_sys\_base* (14.19) for inheritance: Linear pressure system resolution method.
- **dt\_projection** *deuxmots* (4.9.1) for inheritance: nb value : This keyword checks every nb time-steps the equality of velocity divergence to zero. value is the criteria convergency for the solver used.
- **traitement\_particulier** *traitement\_particulier* (5.18) for inheritance: Keyword to post-process particular values.
- **seuil\_divU** *floatfloat* (5.19) for inheritance: value factor : this keyword is intended to minimise the number of iterations during the pressure system resolution. The convergence criteria during this step ('seuil' in solveur\_pression) is dynamically adapted according to the mass conservation. At tn , the linear system Ax=B is considered as solved if the residual ||Ax-B||<seuil(tn). For tn+1, the threshold

value `seuil(tn+1)` will be evaluated as:

```
If (lmax(DivU)*dtl<value)
Seuil(tn+1)= Seuil(tn)*factor
Else
Seuil(tn+1)= Seuil(tn)*factor
Endif
```

The first parameter (value) is the mass evolution the user is ready to accept per timestep, and the second one (factor) is the factor of evolution for 'seuil' (for example 1.1, so 10

- **solveur\_bar** *solveur\_sys\_base* (14.19) for inheritance: This keyword is used to define when filtering operation is called (typically for EF convective scheme, standard diffusion operator and `Source_Qdm_lambdaup`). A file (`solveur.bar`) is then created and used for inversion procedure. Syntax is the same then for pressure solver (GCP is required for multi-processor calculations and, in a general way, for big meshes).
- **projection\_initiale** *int* for inheritance: Keyword to suppress, if boolean equals 0, the initial projection which checks  $\text{DivU}=0$ . By default, boolean equals 1.
- **postraiter\_gradient\_pression\_sans\_masse** for inheritance: Avoid mass matrix multiplication for the gradient postprocessing
- **methode\_calcul\_pression\_initiale** *str into* [*'avec\_les\_cl', 'avec\_sources', 'avec\_sources\_et\_operateurs', 'sans\_rien'*] for inheritance: Keyword to select an option for the pressure calculation before the first time step. Options are : `avec_les_cl` (default option  $\text{lapP}=0$  is solved with Neuman boundary conditions on pressure if any), `avec_sources` ( $\text{lapP}=f$  is solved with Neuman boundaries conditions and  $f$  integrating the source terms of the Navier-Stokes equations) and `avec_sources_et_operateurs` ( $\text{lapP}=f$  is solved as with the previous option `avec_sources` but  $f$  integrating also some operators of the Navier-Stokes equations). The two last options are useful and sometime necessary when source terms are implicated when using an implicit time scheme to solve the Navier-Stokes equations.
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limite** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if `equation_non_resolue` keyword is used. Example: The Navier-Stokes equations are not solved between time  $t_0$  and  $t_1$ .  

```
Navier_Sokes_Standard
{ equation_non_resolue (t>t0)*(t<t1) }
```
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.54 Navier\_stokes\_phase\_field

Description: Navier Stokes equation for the Phase Field problem.

Keyword `Discretize` should have already been used to read the object.

See also: `navier_stokes_standard` (5.57)

Usage:

**navier\_stokes\_phase\_field** *str*

**Read** *str* {

```

 approximation_de_boussinesq approx_boussinesq
 [viscosite_dynamique_constant visco_dyn_cons]
 [gravite n x1 x2 ... xn]
 [solveur_pression solveur_sys_base]
 [dt_projection deuxmots]
 [traitement_particulier traitement_particulier]
 [seuil_divU floatfloat]
 [solveur_bar solveur_sys_base]
 [projection_initiale int]
 [postraiter_gradient_pression_sans_masse]
 [methode_calcul_pression_initiale str into ['avec_les_cl', 'avec_sources', 'avec_sources_et-
_operateurs', 'sans_rien']]
 [disable_equation_residual int]
 [convection bloc_convection]
 [diffusion bloc_diffusion]
 [boundary_conditions|conditions_limites condlims]
 [initial_conditions|conditions_initiales condinits]
 [sources sources]
 [ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
 [parametre_equation parametre_equation_base]
 [equation_non_resolue str]
 [renommer_equation str]

```

}

where

- **approximation\_de\_boussinesq** *approx\_boussinesq* (5.55): To use or not the Boussinesq approximation.
- **viscosite\_dynamique\_constant** *visco\_dyn\_cons* (5.56): To use or not a viscosity which will depends on concentration C (in fact, C is the unknown of Cahn-Hilliard equation).
- **gravite** *n x1 x2 ... xn*: Keyword to define gravity in the case Boussinesq approximation is not used.
- **solveur\_pression** *solveur\_sys\_base* (14.19) for inheritance: Linear pressure system resolution method.
- **dt\_projection** *deuxmots* (4.9.1) for inheritance: nb value : This keyword checks every nb time-steps the equality of velocity divergence to zero. value is the criteria convergency for the solver used.
- **traitement\_particulier** *traitement\_particulier* (5.18) for inheritance: Keyword to post-process particular values.
- **seuil\_divU** *floatfloat* (5.19) for inheritance: value factor : this keyword is intended to minimise the number of iterations during the pressure system resolution. The convergence criteria during this step ('seuil' in *solveur\_pression*) is dynamically adapted according to the mass conservation. At  $t_n$ , the linear system  $Ax=B$  is considered as solved if the residual  $\|Ax-B\| < \text{seuil}(t_n)$ . For  $t_{n+1}$ , the threshold value  $\text{seuil}(t_{n+1})$  will be evaluated as:  
 If (  $\text{lmax}(\text{DivU}) \cdot dt < \text{value}$  )  
 $\text{Seuil}(t_{n+1}) = \text{Seuil}(t_n) \cdot \text{factor}$   
 Else  
 $\text{Seuil}(t_{n+1}) = \text{Seuil}(t_n) \cdot \text{factor}$   
 Endif  
 The first parameter (value) is the mass evolution the user is ready to accept per timestep, and the second one (factor) is the factor of evolution for 'seuil' (for example 1.1, so 10)
- **solveur\_bar** *solveur\_sys\_base* (14.19) for inheritance: This keyword is used to define when filtering operation is called (typically for EF convective scheme, standard diffusion operator and Source-

\_Qdm\_lambdaup ). A file (solveur.bar) is then created and used for inversion procedure. Syntax is the same then for pressure solver (GCP is required for multi-processor calculations and, in a general way, for big meshes).

- **projection\_initiale** *int* for inheritance: Keyword to suppress, if boolean equals 0, the initial projection which checks DivU=0. By default, boolean equals 1.
- **postraiter\_gradient\_pression\_sans\_masse** for inheritance: Avoid mass matrix multiplication for the gradient postprocessing
- **methode\_calcul\_pression\_initiale** *str* into ['avec\_les\_cl', 'avec\_sources', 'avec\_sources\_et\_operateurs', 'sans\_rien'] for inheritance: Keyword to select an option for the pressure calculation before the first time step. Options are : avec\_les\_cl (default option lapP=0 is solved with Neuman boundary conditions on pressure if any), avec\_sources (lapP=f is solved with Neuman boundaries conditions and f integrating the source terms of the Navier-Stokes equations) and avec\_sources\_et\_operateurs (lapP=f is solved as with the previous option avec\_sources but f integrating also some operators of the Navier-Stokes equations). The two last options are useful and sometime necessary when source terms are implicated when using an implicit time scheme to solve the Navier-Stokes equations.
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limite** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.55 Approx\_boussinesq

Description: different mass density formulation are available depending if the Boussinesq approximation is made or not

See also: objet\_lecture (47)

Usage:

**yes\_or\_no** **bloc\_bouss**

where

- **yes\_or\_no** *str* into ['oui', 'non']: To use or not the Boussinesq approximation.
- **bloc\_bouss** *bloc\_boussinesq* (5.55.1): to choose the rho formulation

### 5.55.1 Bloc\_boussinesq

Description: choice of rho formulation



See also: `objet_lecture` ([47](#))

Usage:

```
{
 [probleme str]
 [rho_1 float]
 [rho_2 float]
 [rho_fonc_c bloc_rho_fonc_c]
}
```

where

- **probleme** *str*: Name of problem.
- **rho\_1** *float*: value of rho
- **rho\_2** *float*: value of rho
- **rho\_fonc\_c** *bloc\_rho\_fonc\_c* ([5.55.2](#)): to use for define a general form for rho

### 5.55.2 Bloc\_rho\_fonc\_c

Description: if rho has a general form

See also: `objet_lecture` ([47](#))

Usage:

```
[Champ_Fonc_Fonction] [problem_name] [concentration] [dim] [val] [Champ_Uniforme] [fielddim] [val2]
where
```

- **Champ\_Fonc\_Fonction** *str* into [*'Champ\_Fonc\_Fonction'*]: `Champ_Fonc_Fonction`
- **problem\_name** *str*: Name of problem.
- **concentration** *str* into [*'concentration'*]: concentration
- **dim** *int*: dimension of the problem
- **val** *str*: function of rho
- **Champ\_Uniforme** *str* into [*'Champ\_Uniforme'*]: `Champ_Uniforme`
- **fielddim** *int*: dimension of the problem
- **val2** *str*: function of rho

### 5.56 Visco\_dyn\_cons

Description: different treatment of the kinematic viscosity could be done depending of the use of the Boussinesq approximation or the constant dynamic viscosity approximation

See also: `objet_lecture` ([47](#))

Usage:

```
yes_or_no bloc_visco
where
```

- **yes\_or\_no** *str* into [*'oui'*, *'non'*]: To use or not the constant dynamic viscosity
- **bloc\_visco** *bloc\_visco2* ([5.56.1](#)): to choose the mu formulation



### 5.56.1 Bloc\_visco2

Description: choice of mu formulation

See also: `objet_lecture` ([47](#))

Usage:

```
{
 [probleme str]
 [mu_1 float]
 [mu_2 float]
 [mu_fonc_c bloc_mu_fonc_c]
}
where
```

- **probleme** *str*: Name of problem.
- **mu\_1** *float*: value of mu
- **mu\_2** *float*: value of mu
- **mu\_fonc\_c** *bloc\_mu\_fonc\_c* ([5.56.2](#)): to use for define a general form for mu

### 5.56.2 Bloc\_mu\_fonc\_c

Description: if mu has a general form

See also: `objet_lecture` ([47](#))

Usage:

```
[Champ_Fonc_Fonction] [problem_name] [concentration] [dim] [val]
where
```

- **Champ\_Fonc\_Fonction** *str* into [*Champ\_Fonc\_Fonction*]: `Champ_Fonc_Fonction`
- **problem\_name** *str*: Name of problem.
- **concentration** *str* into [*concentration*]: `concentration`
- **dim** *int*: dimension of the problem
- **val** *str*: function of mu

## 5.57 Navier\_stokes\_standard

Description: Navier-Stokes equations.

Keyword `Discretize` should have already been used to read the object.

See also: `eqn_base` ([5.45](#)) `navier_stokes_ibm_turbulent` ([5.53](#)) `navier_stokes_ibm` ([5.52](#)) `navier_stokes_WC` ([5.49](#)) `navier_stokes_QC` ([5.48](#)) `navier_stokes_turbulent` ([5.58](#)) `navier_stokes_phase_field` ([5.54](#)) `Navier_Stokes_std_ALE` ([5.24](#)) `Navier_Stokes_Aposteriori` ([5.17](#)) `Navier_Stokes_standard_sensibility` ([5.23](#))

Usage:

**navier\_stokes\_standard** *str*

```
Read str {
 [solveur_pression solveur_sys_base]
 [dt_projection deuxmots]
 [traitement_particulier traitement_particulier]
 [seuil_divU floatfloat]
}
```

```

[solveur_bar solveur_sys_base]
[projection_initiale int]
[postraiter_gradient_pression_sans_masse]
[methode_calcul_pression_initiale str into ['avec_les_cl', 'avec_sources', 'avec_sources_et-
_operateurs', 'sans_rien']]
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
}

```

where

- **solveur\_pression** *solveur\_sys\_base* (14.19): Linear pressure system resolution method.
- **dt\_projection** *deuxmots* (4.9.1): nb value : This keyword checks every nb time-steps the equality of velocity divergence to zero. value is the criteria convergency for the solver used.
- **traitement\_particulier** *traitement\_particulier* (5.18): Keyword to post-process particular values.
- **seuil\_divU** *floatfloat* (5.19): value factor : this keyword is intended to minimise the number of iterations during the pressure system resolution. The convergence criteria during this step ('seuil' in solveur\_pression) is dynamically adapted according to the mass conservation. At  $t_n$ , the linear system  $Ax=B$  is considered as solved if the residual  $\|Ax-B\| < \text{seuil}(t_n)$ . For  $t_{n+1}$ , the threshold value  $\text{seuil}(t_{n+1})$  will be evaluated as:  
 If (  $\text{lmax}(\text{DivU}) \cdot dt < \text{value}$  )  
 Seuil( $t_{n+1}$ ) = Seuil( $t_n$ ) \* factor  
 Else  
 Seuil( $t_{n+1}$ ) = Seuil( $t_n$ ) \* factor  
 Endif  
 The first parameter (value) is the mass evolution the user is ready to accept per timestep, and the second one (factor) is the factor of evolution for 'seuil' (for example 1.1, so 10)
- **solveur\_bar** *solveur\_sys\_base* (14.19): This keyword is used to define when filtering operation is called (typically for EF convective scheme, standard diffusion operator and Source\_Qdm\_lambdaup ). A file (solveur.bar) is then created and used for inversion procedure. Syntax is the same then for pressure solver (GCP is required for multi-processor calculations and, in a general way, for big meshes).
- **projection\_initiale** *int*: Keyword to suppress, if boolean equals 0, the initial projection which checks  $\text{DivU}=0$ . By default, boolean equals 1.
- **postraiter\_gradient\_pression\_sans\_masse** : Avoid mass matrix multiplication for the gradient postprocessing
- **methode\_calcul\_pression\_initiale** *str* into [*'avec\_les\_cl'*, *'avec\_sources'*, *'avec\_sources\_et\_operateurs'*, *'sans\_rien'*]: Keyword to select an option for the pressure calculation before the first time step. Options are : avec\_les\_cl (default option lapP=0 is solved with Neuman boundary conditions on pressure if any), avec\_sources (lapP=f is solved with Neuman boundaries conditions and f integrating the source terms of the Navier-Stokes equations) and avec\_sources\_et\_operateurs (lapP=f is solved as with the previous option avec\_sources but f integrating also some operators of the Navier-Stokes equations). The two last options are useful and sometime necessary when source terms are implicated when using an implicit time scheme to solve the Navier-Stokes equations.
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.

- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.58 Navier\_stokes\_turbulent

Description: Navier-Stokes equations as well as the associated turbulence model equations.

Keyword Discretize should have already been used to read the object.

See also: *navier\_stokes\_standard* (5.57) *navier\_stokes\_turbulent\_qc* (5.59) *navier\_stokes\_ft\_disc* (5.50)

Usage:

**navier\_stokes\_turbulent** *str*

**Read** *str* {

```
[modele_turbulence modele_turbulence_hyd_deriv]
[solveur_pression solveur_sys_base]
[dt_projection deuxmots]
[traitement_particulier traitement_particulier]
[seuil_divU floatfloat]
[solveur_bar solveur_sys_base]
[projection_initiale int]
[postraiter_gradient_pression_sans_masse]
[methode_calcul_pression_initiale str into ['avec_les_cl', 'avec_sources', 'avec_sources_et_
_operateurs', 'sans_rien']]
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
```

}

where

- **modele\_turbulence** *modele\_turbulence\_hyd\_deriv* (5.22): Turbulence model for Navier-Stokes equations.

- **solveur\_pression** *solveur\_sys\_base* (14.19) for inheritance: Linear pressure system resolution method.
- **dt\_projection** *deuxmots* (4.9.1) for inheritance: nb value : This keyword checks every nb time-steps the equality of velocity divergence to zero. value is the criteria convergency for the solver used.
- **traitement\_particulier** *traitement\_particulier* (5.18) for inheritance: Keyword to post-process particular values.
- **seuil\_divU** *floatfloat* (5.19) for inheritance: value factor : this keyword is intended to minimise the number of iterations during the pressure system resolution. The convergence criteria during this step ('seuil' in solveur\_pression) is dynamically adapted according to the mass conservation. At tn , the linear system  $Ax=B$  is considered as solved if the residual  $\|Ax-B\| < \text{seuil}(tn)$ . For tn+1, the threshold value  $\text{seuil}(tn+1)$  will be evaluated as:  
 If (  $\text{lmax}(\text{DivU}) * dt < \text{value}$  )  
 Seuil(tn+1)= Seuil(tn)\*factor  
 Else  
 Seuil(tn+1)= Seuil(tn)\*factor  
 Endif  
 The first parameter (value) is the mass evolution the user is ready to accept per timestep, and the second one (factor) is the factor of evolution for 'seuil' (for example 1.1, so 10)
- **solveur\_bar** *solveur\_sys\_base* (14.19) for inheritance: This keyword is used to define when filtering operation is called (typically for EF convective scheme, standard diffusion operator and Source\_Qdm\_lambdaup ). A file (solveur.bar) is then created and used for inversion procedure. Syntax is the same then for pressure solver (GCP is required for multi-processor calculations and, in a general way, for big meshes).
- **projection\_initiale** *int* for inheritance: Keyword to suppress, if boolean equals 0, the initial projection which checks  $\text{DivU}=0$ . By default, boolean equals 1.
- **postraiter\_gradient\_pression\_sans\_masse** for inheritance: Avoid mass matrix multiplication for the gradient postprocessing
- **methode\_calcul\_pression\_initiale** *str into ['avec\_les\_cl', 'avec\_sources', 'avec\_sources\_et\_operateurs', 'sans\_rien']* for inheritance: Keyword to select an option for the pressure calculation before the first time step. Options are : avec\_les\_cl (default option lapP=0 is solved with Neuman boundary conditions on pressure if any), avec\_sources (lapP=f is solved with Neuman boundaries conditions and f integrating the source terms of the Navier-Stokes equations) and avec\_sources\_et\_operateurs (lapP=f is solved as with the previous option avec\_sources but f integrating also some operators of the Navier-Stokes equations). The two last options are useful and sometime necessary when source terms are implicated when using an implicit time scheme to solve the Navier-Stokes equations.
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limite** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1.  
 Navier\_Sokes\_Standard  
 { equation\_non\_resolue (t>t0)\*(t<t1) }

- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.59 Navier\_stokes\_turbulent\_qc

Description: Navier-Stokes equations under low Mach number as well as the associated turbulence model equations.

Keyword Discretize should have already been used to read the object.

See also: `navier_stokes_turbulent` (5.58)

Usage:

**navier\_stokes\_turbulent\_qc** *str*

**Read** *str* {

```
[modele_turbulence modele_turbulence_hyd_deriv]
[solveur_pression solveur_sys_base]
[dt_projection deuxmots]
[traitement_particulier traitement_particulier]
[seuil_divU floatfloat]
[solveur_bar solveur_sys_base]
[projection_initiale int]
[postraiter_gradient_pression_sans_masse]
[methode_calcul_pression_initiale str into ['avec_les_cl', 'avec_sources', 'avec_sources_et-
_operateurs', 'sans_rien']]
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
```

}

where

- **modele\_turbulence** *modele\_turbulence\_hyd\_deriv* (5.22) for inheritance: Turbulence model for Navier-Stokes equations.
- **solveur\_pression** *solveur\_sys\_base* (14.19) for inheritance: Linear pressure system resolution method.
- **dt\_projection** *deuxmots* (4.9.1) for inheritance: nb value : This keyword checks every nb time-steps the equality of velocity divergence to zero. value is the criteria convergency for the solver used.
- **traitement\_particulier** *traitement\_particulier* (5.18) for inheritance: Keyword to post-process particular values.
- **seuil\_divU** *floatfloat* (5.19) for inheritance: value factor : this keyword is intended to minimise the number of iterations during the pressure system resolution. The convergence criteria during this step ('seuil' in `solveur_pression`) is dynamically adapted according to the mass conservation. At  $t_n$ , the linear system  $Ax=B$  is considered as solved if the residual  $\|Ax-B\| < \text{seuil}(t_n)$ . For  $t_{n+1}$ , the threshold value  $\text{seuil}(t_{n+1})$  will be evaluated as:  
 If (  $\text{lmax}(\text{DivU}) \cdot dt < \text{value}$  )  
 $\text{Seuil}(t_{n+1}) = \text{Seuil}(t_n) \cdot \text{factor}$   
 Else

```
Seuil(tn+1)= Seuil(tn)*factor
Endif
```

The first parameter (value) is the mass evolution the user is ready to accept per timestep, and the second one (factor) is the factor of evolution for 'seuil' (for example 1.1, so 10

- **solveur\_bar** *solveur\_sys\_base* (14.19) for inheritance: This keyword is used to define when filtering operation is called (typically for EF convective scheme, standard diffusion operator and Source\_Qdm\_lambdaup ). A file (solveur.bar) is then created and used for inversion procedure. Syntax is the same then for pressure solver (GCP is required for multi-processor calculations and, in a general way, for big meshes).
- **projection\_initiale** *int* for inheritance: Keyword to suppress, if boolean equals 0, the initial projection which checks DivU=0. By default, boolean equals 1.
- **postraiter\_gradient\_pression\_sans\_masse** for inheritance: Avoid mass matrix multiplication for the gradient postprocessing
- **methode\_calcul\_pression\_initiale** *str* into ['avec\_les\_cl', 'avec\_sources', 'avec\_sources\_et\_operateurs', 'sans\_rien'] for inheritance: Keyword to select an option for the pressure calculation before the first time step. Options are : avec\_les\_cl (default option lapP=0 is solved with Neuman boundary conditions on pressure if any), avec\_sources (lapP=f is solved with Neuman boundaries conditions and f integrating the source terms of the Navier-Stokes equations) and avec\_sources\_et\_operateurs (lapP=f is solved as with the previous option avec\_sources but f integrating also some operators of the Navier-Stokes equations). The two last options are useful and sometime necessary when source terms are implicated when using an implicit time scheme to solve the Navier-Stokes equations.
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1.  

```
Navier_Sokes_Standard
{ equation_non_resolue (t>t0)*(t<t1) }
```
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.60 Transport\_epsilon

Description: The eps transport equation in bicephale (standard or realisable) k-eps model.

Keyword Discretize should have already been used to read the object.

See also: eqn\_base (5.45)

Usage:

**transport\_epsilon** *str*

**Read** *str* {

[ **disable\_equation\_residual** *int*]

```

[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]

```

}

where

- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.61 Transport\_interfaces\_ft\_disc

Description: Interface tracking equation for Front-Tracking problem in the discontinuous version.

Keyword Discretize should have already been used to read the object.

See also: eqn\_base (5.45)

Usage:

**transport\_interfaces\_ft\_disc** *str*

**Read** *str* {

```

[initial_conditions|conditions_initiales bloc_lecture]
[methode_transport methode_transport_deriv]
[n_iterations_distance int]
[maillage bloc_lecture]
[remaillage bloc_lecture_remaillage]
[collisions bloc_lecture]
[methode_interpolation_v str into ['valeur_a_elem', 'vdf_lineaire', 'mean_volumic_velocity']]
[volume_impose_phase_1 float]
[parcours_interface parcours_interface]

```



```

[interpolation_repere_local]
[interpolation_champ_face interpolation_champ_face_deriv]
[n_iterations_interpolation_ibc int]
[type_vitesse_imposee str into ['uniforme', 'analytique']]
[nombre_facettes_retenues_par_cellule int]
[seuil_convergence_uzawa float]
[nb_iteration_max_uzawa int]
[injecteur_interfaces str]
[vitesse_imposee_regularisee int]
[indic_faces_modifiee bloc_lecture]
[distance_projete_faces str into ['simplifiee', 'initiale', 'modifiee']]
[voflike_correction_volume int]
[nb_lissage_correction_volume int]
[nb_iterations_correction_volume int]
[type_indic_faces type_indic_faces_deriv]
[compute_particles_rms]
[post_process_hydrodynamic_forces bloc_lecture]
[collision_model_fpi collision_model_fpi_deriv]
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
}
where

```

- **initial\_conditions|conditions\_initiales** *bloc\_lecture* (3.2): The keyword `conditions_initiales` is used to define the shape of the initial interfaces through the zero level-set of a function, or through a mesh `fichier_geom`. Indicator function is set to 0, that is `fluide0`, where the function is negative; indicator function is set to 1, that is `fluide1`, where the function is positive; the interfaces are the level-set 0 of that function:

```

conditions_initiales { fonction
 -(($(x-0.002)^2 + (y-0.002)^2 + z^2 - (0.00125)^2$))*(($x-0.005)^2 + (y-0.007)^2 + z^2(0.00150)^2$))*
 (0.020 - z))
}

```

In the above example, there are three interfaces: two bubbles in a liquid with a free surface. One bubble has a radius of 0.00125, i.e. 1.25 mm, and its center is {0.002, 0.002, 0.000}. The other bubble has a radius of 0.00150, i.e. 1.5 mm, and its center is {0.005, 0.007, 0.000}. The free surface is above the two bubble, at a level  $z=0.02$ .

Additional feature in this block concerns the keywords `ajout_phase0` and `ajout_phase1`. They can be used to simplify the composition of different interfaces. When using these keywords, the initial function defines the indicator function; `ajout_phase0` and `ajout_phase1` are used to modify this initial field. Each time `ajout_phase0` is used, the field is untouched where the function is positive whereas the indicator field is set to 0 where the function is negative. The keyword `ajout_phase1` has the symmetrical use, keeping the field value where the function is negative and setting the indicator field to 1 where the function is positive. The previous example can also be written:



```

conditions_initiales {
fonction z-0.020 , NL fonction ajout_phase1 $(x - 0.002)^2 + (y - 0.002)^2 + z^2 - (0.00125)^2$,
fonction ajout_phase1 $(x - 0.005)^2 + (y - 0.007)^2 + z^2 - (0.00150)^2$
}

```

- **methode\_transport** *methode\_transport\_deriv* (5.62): Method of transport of interface.
- **n\_iterations\_distance** *int*: Keyword to specify the number of iterations requested for the smoothing process of computing the field corresponding to the signed distance to the interfaces and located at the center of the Eulerian elements. This smoothing is necessary when there are more Lagrangian nodes than Eulerian two-phase cells.
- **maillage** *bloc\_lecture* (3.2): This optional block is used to specify that we want a Gnuplot drawing of the initial mesh. There is only one keyword, *niveau\_plot*, that is used only to define if a Gnuplot drawing is active (value 1) or not active (value -1). By default, skipping the block will produce non Gnuplot drawing. This option is to be used only in a debug process.
- **remaillage** *bloc\_lecture\_remaillage* (5.63): This block is used to specify the operations that are used to keep the solid interfaces in a proper condition. The remaillage block only contains parameter's values.
- **collisions** *bloc\_lecture* (3.2): This block is used to specify the operations that are used when a collision occurs between two parts of interfaces. When this occurs, it is necessary to build a new mesh that has locally a clear definition of what is inside and what is outside of the mesh. The collisions can either be active or inactive. If the collisions are active (highly recommended), a Juric level-set reconstruction method will be used to re-create the new mesh after each coalescence or breakup. An option *Juric\_local* *phase\_continue* *N* can be used to force the remeshing to impact only a local portion of the mesh, near the collision. The next line (*type\_remaillage*) is used to state whose field will be used for the level-set computation. Main option is Juric, a remeshing that is compatible with parallel computing. When using Juric level-set remeshing, the source field (*source\_isevaleur*) that is used to compute the level-sets is then defined. It can be either the indicator function (*indicatrice*), a choice which is the default one and the most robust, or a geometrical distance computed from the mesh at the beginning of the time step (*fonction\_distance*), a choice that may be more accurate in specific situations.  
*Type\_remaillage* can be either Juric or Thomas. When Thomas is used, it is an enhancement of the Juric remeshing algorithm designed to compensate for mass loss during remeshing. The mesh is always reconstructed with the indicator function (not with the distance function). After having reconstructed the mesh with the Juric algorithm, the difference between the old indicator function (before remeshing) and the new indicator function is computed. The differences occurring at a distance below or equal to *N* elements from the interface are summed up and used to move the interface in the normal direction. The displacement of the interface is such that the volume of each phase after displacement is equal to the volume of the phase before remeshing. *N* (default value 1) must be smaller than *n\_iterations\_distance* (suggested value: 2).
- **methode\_interpolation\_v** *str* into [*'valeur\_a\_elem'*, *'vdf\_lineaire'*, *'mean\_volumic\_velocity'*]: In this block, two keywords are possible for method to select the way the interpolation is performed. With the choice *valeur\_a\_elem* the speed of displacement of the nodes of the interfaces is the velocity at the center of the Eulerian element in which each node is located at the beginning of the time step. This choice is the default interpolation method. The choice *VDF\_lineaire* is only available with a VDF discretization (VDF). In this case, the speed of displacement of the nodes of the interfaces is linearly interpolated on the 4 (in 2D) or the 6 (in 3D) Eulerian velocities closest the location of each node at the beginning of the time step. In peculiar situation, this choice may provide a better interpolated value. Of course, this choice is not available with a VEF discretization (VEFPreP1B).
- **volume\_impose\_phase\_1** *float*: this keyword is used to specify the volume of one phase to keep the volume of the phases constant during the remeshing process. It is an alternate solution to trouble in mass conservation. This option is mainly realistic when only one inclusion of phase 1 is present in the domain. In most other situations, the *iterations\_correction\_volume* keyword seems easier to justify. The volume to be keep is in m3 and should agree with initial condition.
- **parcours\_interface** *parcours\_interface* (5.64): *Parcours\_interface* allows you to configure the algorithm that computes the surface mesh to volume mesh intersection. This algorithm has some serious

trouble when the surface mesh points coincide with some faces of the volume mesh. Effects are visible on the indicator function, in VDF when a plane interface coincides with a volume mesh surface. To overcome these problems, the keyword `correction_parcours_thomas` keyword can be used: it allows the algorithm to slightly move some mesh points. This algorithm is experimental and is NOT activated by default.

- **interpolation\_repere\_local** : Triggers a new transport algorithm for the interface: the velocity vector of lagrangian nodes is computed in the moving frame of reference of the center of each connex component, in such a way that relative displacements of nodes within a connex component of the lagrangian mesh are minimized, hence reducing the necessity of barycentering, smooting and local remeshing. Very efficient for bubbly flows.
- **interpolation\_champ\_face** *interpolation\_champ\_face\_deriv* (5.65): It is possible to compute the imposed velocity for the solid-fluid interface by direct affectation (`interpolation_scheme` would be set to `base`) or by multi-linear interpolation (`interpolation_scheme` would be set to `lineaire`). The default value is `base`.
- **n\_iterations\_interpolation\_ibc** *int*: Useful only with `interpolation_champ_face` positioned to `lineaire`. Set the value concerning the width of the region of the linear interpolation. For the Penalized Direct Forcing model, a value equals to 1 is enough.
- **type\_vitesse\_imposee** *str* into [`'uniforme'`, `'analytique'`]: Useful only with `interpolation_champ_face` positioned to `lineaire`. Value of the keyword is `uniforme` (for an uniform solid-fluide interface's velocity, i.e. zero for instance) or `analytique` (for an analytic expression of the solid-fluide interface's velocity depending on the spatial coordinates). The default value is `uniforme`.
- **nombre\_facettes\_retenues\_par\_cellule** *int*: Keyword to specify the default number (3) of facets per cell used to describe the geometry of the solid-solid interface. This number should be increased if the geometry of the solid-solid interface is complex in each cell (eulerian mesh too coarse for example).
- **seuil\_convergence\_uzawa** *float*: Optional option to change the default value (10-8) of the threshold convergence for the Uzawa algorithm if used in the Penalized Direct Forcing model. Sometime, the value should be decreased to insure a better convergence to force equality between sequential and parallel results.
- **nb\_iteration\_max\_uzawa** *int*: Optional option to change the default value (10-8) of the threshold convergence for the Uzawa algorithm if used in the Penalized Direct Forcing model. Sometime, the value should be decreased to insure a better convergence to force equality between sequential and parallel results.
- **injecteur\_interfaces** *str*
- **vitesse\_imposee\_regularisee** *int*
- **indic\_faces\_modifiee** *bloc\_lecture* (3.2)
- **distance\_projete\_faces** *str* into [`'simplifiee'`, `'initiale'`, `'modifiee'`]
- **voflike\_correction\_volume** *int*: flag (0 or 1). If activated, code will perform the requested number of iterations for the correction process that can be used to keep the volume of the phases constant during the transport process.
- **nb\_lissage\_correction\_volume** *int*: Select 0 or N, the number of smoothing to apply to avoid potential spikes due to volume correction.
- **nb\_iterations\_correction\_volume** *int*: to iterate on the volume correction until `seuil` is reached.
- **type\_indic\_faces** *type\_indic\_faces\_deriv* (5.66): kind of interpolation to compute the face value of the phase indicator function (advanced option). Could be `STANDARD`, `MODIFIEE` or `AI_BASED`
- **compute\_particles\_rms**
- **post\_process\_hydrodynamic\_forces** *bloc\_lecture* (3.2)
- **collision\_model\_fpi** *collision\_model\_fpi\_deriv* (15)
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.

- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.62 Methode\_transport\_deriv

Description: Basic class for method of transport of interface.

See also: objet\_lecture (47) vitesse\_imposee (5.62.1) vitesse\_interpolee (5.62.2) loi\_horaire (5.62.3)

Usage:

**methode\_transport\_deriv**

### 5.62.1 Vitesse\_imposee

Description: Class to specify that the speed of displacement of the nodes of the interfaces is imposed with an analytical formula.

See also: methode\_transport\_deriv (5.62)

Usage:

**vitesse\_imposee val**

where

- **val** *word1 word2 (word3)*: Analytical formula.

### 5.62.2 Vitesse\_interpolee

Description: Class to specify that the interpolation will use the velocity field of the Navier-Stokes equation named val to compute the speed of displacement of the nodes of the interfaces.

See also: methode\_transport\_deriv (5.62)

Usage:

**vitesse\_interpolee val**

where

- **val** *str*: Navier-Stokes equation.

### 5.62.3 Loi\_horaire

Description: not\_set

See also: methode\_transport\_deriv ([5.62](#))

Usage:

**loi\_horaire** **nom\_loi**

where

- **nom\_loi** *str*

### 5.63 Bloc\_lecture\_remaillage

Description: Parameters for remeshing.

See also: objet\_lecture ([47](#))

Usage:

```
{
 [pas float]
 [pas_lissage float]
 [nb_iter_remaillage int]
 [nb_iter_barycentrage int]
 [relax_barycentrage float]
 [critere_arete float]
 [impr float]
 [facteur_longueur_ideale float]
 [nb_iter_correction_volume int]
 [seuil_dvolume_residuel float]
 [lissage_courbure_coeff float]
 [lissage_courbure_iterations_systematique int]
 [lissage_courbure_iterations_si_remaillage int]
 [critere_longueur_fixe float]
}
```

where

- **pas** *float*: This keyword has default value -1.; when it is set to a negative value there is no remeshing. It is the time step in second (physical time) between two operations of remeshing.
- **pas\_lissage** *float*: This keyword has default value -1.; when it is set to a negative value there is no smoothing of mesh. It is the time step in second (physical time) between two operations of smoothing of the mesh.
- **nb\_iter\_remaillage** *int*: This keyword has default value 0; when it is set to the zero value there is no remeshing. It is the number of iterations performed during a remeshing process.
- **nb\_iter\_barycentrage** *int*: This keyword has default value 0; when it is set to the zero value there is no operation of barycentrage. The barycentrage operation consists in moving each node of the mesh tangentially to the mesh surface and in a direction that let it closer the center of gravity of its neighbors. If **relax\_barycentrage** is set to 1, the node is move to the center of gravity. For values lower than unity, the motion is limited to the corresponding fraction. The parameter **nb\_iter\_barycentrage** is the number of iteration of these node displacements.
- **relax\_barycentrage** *float*: This keyword has default value 0; when it is set to the zero value there is no motion of the nodes. When  $0 < \text{relax\_barycentrage} \leq 1$ , this parameter provides the relaxation ratio to be used in the barycentrage operation described for the keyword **nb\_iter\_barycentrage**.

- **critere\_arete** *float*: This keyword is used to compute two sub-criteria : the minimum and the maximum edge length ratios used in the process of obtaining edges of length close to `critere_longueur_fixe`. Their respective values are set to  $(1-\text{critere\_arete})^{**2}$  and  $(1+\text{critere\_arete})^{**2}$ . The default values of the minimum and the maximum are set respectively to 0.5 and 1.5. When an edge is longer than  $\text{critere\_longueur\_fixe}*(1+\text{critere\_arete})^{**2}$ , the edge is cut into two pieces; when its length is smaller than  $\text{critere\_longueur\_fixe}*(1-\text{critere\_arete})^{**2}$ , this edge has to be suppressed.
- **impr** *float*: This keyword is followed by a value that specify the printing time period given. The default value is -1, which means no printing.
- **facteur\_longueur\_ideale** *float*: This keyword is used to set a ratio between edge length and the cube root of volume cell for the remeshing process. The default value is 1.0.
- **nb\_iter\_correction\_volume** *int*: This keyword give the maximum number of iterations to be performed trying to satisfy the criterion `seuil_dvolume_residuel`. The default value is 0, which means no iteration.
- **seuil\_dvolume\_residuel** *float*: This keyword give the error volume (in m3) that is accepted to stop the iterations performed to keep the volume constant during the remeshing process. The default value is 0.0.
- **lissage\_courbure\_coeff** *float*: This keyword is used to specify the diffusion coefficient used in the diffusion process of the curvature in the curvature smoothing process with a time step. The default value is 0.05. That value usually provides a stable process. Too small values do not stabilize enough the interface, especially with several Lagrangian nodes per Eulerian cell. Too high values induce an additional macroscopic smoothing of the interface that should physically come from the surface tension and not from this numerical smoothing.
- **lissage\_courbure\_iterations\_systematique** *int*: This keyword allows a finer control to perform the curvature smoothing process. N1 iterations are applied systematically at each timestep. For proper DNS computation, N1 should be set to 0. Default value is 0.
- **lissage\_courbure\_iterations\_si\_remaillage** *int*: N2 iterations are applied only if the local or the global remeshing effectively changes the lagrangian mesh connectivity. Default value is 0.
- **critere\_longueur\_fixe** *float*: This keyword is used to specify the ideal edge length for a remeshing process. The default value is -1., which means that the remeshing does not try to have all edge lengths to tend towards a given value.

## 5.64 Parcours\_interface

Description: allows you to configure the algorithm that computes the surface mesh to volume mesh intersection. This algorithm has some serious trouble when the surface mesh points coincide with some faces of the volume mesh. Effects are visible on the indicator function, in VDF when a plane interface coincides with a volume mesh surface.

To overcome these problems, the keyword `correction_parcours_thomas` keyword can be used: it allows the algorithm to slightly move some mesh points. This algorithm, which is experimental and is NOT activated by default, triggers a correction that avoids some errors in the computation of the indicator function for surface meshes that exactly cross some eulerian mesh edges (strongly suggested !).

See also: [objet\\_lecture \(47\)](#)

Usage:

```
{
 [correction_parcours_thomas]
 [parcours_sans_tolerance]
}
```

where

- **correction\_parcours\_thomas**
- **parcours\_sans\_tolerance**

## 5.65 Interpolation\_champ\_face\_deriv

Description: not\_set

See also: objet\_lecture (47) base (5.65.1) lineaire (5.65.2)

Usage:

### 5.65.1 Base

Description: not\_set

See also: interpolation\_champ\_face\_deriv (5.65)

Usage:

**base**

### 5.65.2 Lineaire

Description: not\_set

See also: interpolation\_champ\_face\_deriv (5.65)

Usage:

**lineaire** {

    [ vitesse\_fluide\_explicite ]

}

where

- **vitesse\_fluide\_explicite**

## 5.66 Type\_indic\_faces\_deriv

Description: not\_set

See also: objet\_lecture (47) standard (5.66.1) modifiee (5.66.2) ai\_based (5.66.3)

Usage:

### 5.66.1 Standard

Description: not\_set

See also: type\_indic\_faces\_deriv (5.66)

Usage:

**standard**

### 5.66.2 Modifiee

Description: not\_set

See also: type\_indic\_faces\_deriv (5.66)

Usage:

```
modifier {
 [position float]
 [thickness float]
}
```

where

- **position** *float*
- **thickness** *float*

### 5.66.3 Ai\_based

Description: not\_set

See also: type\_indic\_faces\_deriv ([5.66](#))

Usage:

**ai\_based**

## 5.67 Transport\_k

Description: The k transport equation in bicephale (standard or realisable) k-eps model.

Keyword Discretize should have already been used to read the object.

See also: eqn\_base ([5.45](#))

Usage:

**transport\_k** *str*

```
Read str {
 [disable_equation_residual int]
 [convection bloc_convection]
 [diffusion bloc_diffusion]
 [boundary_conditions|conditions_limites condlims]
 [initial_conditions|conditions_initiales condinits]
 [sources sources]
 [ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
 [parametre_equation parametre_equation_base]
 [equation_non_resolue str]
 [renommer_equation str]
}
```

where

- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* ([5.2](#)) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* ([5.3](#)) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* ([4.39.1](#)) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* ([5.4](#)) for inheritance: Initial conditions.

- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.68 Transport\_marqueur\_ft

Description: not\_set

Keyword Discretize should have already been used to read the object.

See also: eqn\_base (5.45)

Usage:

**transport\_marqueur\_ft** *str*

**Read** *str* {

```
[initial_conditions|conditions_initiales bloc_lecture]
[injection injection_marqueur]
[transformation_bulles bloc_lecture]
[phase_marquee int]
[methode_transport str into ['vitesse_interpolee', 'vitesse_particules']]
[methode_couplage str into ['suivi', 'one_way_coupling', 'two_way_coupling']]
[nb_iterations int]
[contribution_one_way int into [0, 1]]
[implicite int into [0, 1]]
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
```

}

where

- **initial\_conditions|conditions\_initiales** *bloc\_lecture* (3.2): ne semble pas standard
- **injection** *injection\_marqueur* (5.69): The keyword injection can be used to inject periodically during the calculation some other particles. The syntax for ensemble\_points and proprietes\_particles is the same than the initial conditions for the particles. The keyword t\_debut\_injection give the injection initial time (by default, given by t\_debut\_integration) and dt\_injection gives the injection time period (by default given by dt\_min).



- **transformation\_bulles** *bloc\_lecture* (3.2): This keyword will activate the transformation of an inclusion (small bubbles) into a particle. localisation gives the sub-zones (N number of sub-zones and their names) where the transformation may happen. The diameter size for the inclusion transformation is given by either diameter\_min option, in this case the inclusion will be suppressed for a diameter less than diameter\_size, either by the beta\_transfo option, in this case the inclusion will be suppressed for a diameter less than diameter\_size\*cell\_volume (cell\_volume is the volume of the cell containing the inclusion). interface specifies the name of the inclusion interface and t\_debut\_transfo is the beginning time for the inclusion transformation operation (by default, it is t\_debut\_integr value) and dt\_transfo is the period transformation (by default, it is dt\_min value). In a two phase flow calculation, the particles will be suppressed when entering into the non marked phase
- **phase\_marquee** *int*: Phase number giving the marked phase, where the particles are located (when they leave this phase, they are suppressed). By default, for a the two phase fluide, the particles are supposed to be into the phase 0 (liquid).
- **methode\_transport** *str into ['vitesse\_interpolee', 'vitesse\_particules']*: Kind of transport method for the particles. With vitesse\_interpolee, the velocity of the particles is the velocity a fluid interpolation velocity (option by default). With vitesse\_particules, the velocity of the particles is governed by the resolution of a momentum equation for the particles.
- **methode\_couplage** *str into ['suivi', 'one\_way\_coupling', 'two\_way\_coupling']*: Way of coupling between the fluid and the particles. By default, (keyword suivi), there is no interaction between both. With one\_way\_coupling keyword, the fluid act on the particles. With two\_way\_coupling keyword, besides, particles act on the fluid.
- **nb\_iterations** *int*: Number of sub-timesteps to solve the momentum equation for the particles (1 per default).
- **contribution\_one\_way** *int into [0, 1]*: Activate (1, default) or not (0) the fluid forces on the particles when one\_way\_coupling or two\_way\_coupling coupling method is used.
- **implicite** *int into [0, 1]*: Impliciting (1) or not (0) the time scheme when weight added source term is used in the momentum equation
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limit** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

## 5.69 Injection\_marqueur

Description: not\_set

See also: objet\_lecture (47)

Usage:

```
{
 ensemble_points bloc_lecture
 proprietes_particules bloc_lecture
 [t_debut_injection float]
 [dt_injection float]
}
```

where

- **ensemble\_points** *bloc\_lecture* (3.2)
- **proprietes\_particules** *bloc\_lecture* (3.2)
- **t\_debut\_injection** *float*
- **dt\_injection** *float*

## 6 collision\_model\_ft\_base

Description: base for collision models for fluid particle interaction

See also: `objet_u` (48)

Usage:

**Collision\_Model\_FT\_base** *str*

**Read** *str* {

```
 collision_model str
 detection_method str
 collision_duration float
 activate_collision_before_impact int
 activation_distance_percentage_diameter float
 force_on_two_phase_elem int
```

}

where

- **collision\_model** *str*: name of the collision model
- **detection\_method** *str*: method to detect collisions
- **collision\_duration** *float*: duration of the collision in seconds;
- **activate\_collision\_before\_impact** *int*: activate collision before impact (1) or not (0)
- **activation\_distance\_percentage\_diameter** *float*: activation distance of the collision process as a percentage of the particle diameter
- **force\_on\_two\_phase\_elem** *int*: force on two phase elements (1) or not (0). Only valid for a single particle.

## 7 domaine\_base

Description: base for most domains

See also: `objet_u` (48) `domaine_ijk` (7.1)

Usage:

## 7.1 Domaine\_ijk

Description: domain for IJK simulation (used in TrioCFD)

See also: [Domaine\\_base \(7\)](#)

Usage:

**domaine\_ijk** *str*

**Read** *str* {

```
 nbelem n1 n2 (n3)
 size_dom x1 x2 (x3)
 perio n1 n2 (n3)
 nproc n1 n2 (n3)
 [origin x1 x2 (x3)]
 [ijk_splitting_ft_extension int]
 [file_coords troismots]
```

}

where

- **nbelem** *n1 n2 (n3)*: Number of elements in each direction (integers, 2 or 3 values depending on dimension)
- **size\_dom** *x1 x2 (x3)*: Domain size in each direction (floats, 2 or 3 values depending on dimension)
- **perio** *n1 n2 (n3)*: Is the direction periodic ? (0 or 1, 2 or 3 values depending on dimension)
- **nproc** *n1 n2 (n3)*: Number of procs in each direction (integers, 2 or 3 values depending on dimension)
- **origin** *x1 x2 (x3)*: Domain origin in each direction (floats, 2 or 3 values depending on dimension)
- **ijk\_splitting\_ft\_extension** *int*
- **file\_coords** *troismots* ([7.2](#))

## 7.2 Troismots

Description: Three words.

See also: [objet\\_lecture \(47\)](#)

Usage:

**mot\_1 mot\_2 mot\_3**

where

- **mot\_1** *str*: First word.
- **mot\_2** *str*: Snd word.
- **mot\_3** *str*: Third word.

## 8 interface\_base

Description: Basic class for a liquid-gas interface (used in pb\_multiphase)

See also: [objet\\_u \(48\)](#) [saturation\\_base \(8.2\)](#) [Interface\\_sigma\\_constant \(8.1\)](#)

Usage:

**Interface\_base** *str*

**Read** *str* {

```

 [surface_tension/tension_superficielle float]
}
where

• surface_tension/tension_superficielle float: surface tension

```

## 8.1 Interface\_sigma\_constant

Description: Liquid-gas interface with a constant surface tension sigma

See also: Interface\_base (8)

Usage:

**Interface\_sigma\_constant** *str*

**Read** *str* {

```

 [surface_tension/tension_superficielle float]

```

```

}
```

where

- surface\_tension/tension\_superficielle float for inheritance: surface tension

## 8.2 Saturation\_base

Description: fluide-gas interface with phase change (used in pb\_multiphase)

See also: Interface\_base (8) saturation\_constant (8.3) saturation\_sodium (8.4)

Usage:

**saturation\_base** *str*

**Read** *str* {

```

 [p_ref float]

```

```

 [t_ref float]

```

```

 [surface_tension/tension_superficielle float]

```

```

}
```

where

- p\_ref float
- t\_ref float
- surface\_tension/tension\_superficielle float for inheritance: surface tension

## 8.3 Saturation\_constant

Description: Class for saturation constant

See also: saturation\_base (8.2)

Usage:

**saturation\_constant** *str*

**Read** *str* {

```

[P_sat float]
[T_sat float]
[Lvap float]
[Hlsat float]
[Hvsat float]
[p_ref float]
[t_ref float]
[surface_tension/tension_superficielle float]
}
where

```

- **P\_sat** float: Define the saturation pressure value (this is a required parameter)
- **T\_sat** float: Define the saturation temperature value (this is a required parameter)
- **Lvap** float: Latent heat of vaporization
- **Hlsat** float: Liquid saturation enthalpy
- **Hvsat** float: Vapor saturation enthalpy
- **p\_ref** float for inheritance
- **t\_ref** float for inheritance
- **surface\_tension/tension\_superficielle** float for inheritance: surface tension

## 8.4 Saturation\_sodium

Description: Class for saturation sodium

See also: [saturation\\_base \(8.2\)](#)

Usage:

**saturation\_sodium** *str*

**Read** *str* {

```

[P_ref float]
[T_ref float]
[p_ref float]
[t_ref float]
[surface_tension/tension_superficielle float]

```

}  
where

- **P\_ref** float: Use to fix the pressure value in the closure law. If not specified, the value of the pressure unknown will be used
- **T\_ref** float: Use to fix the temperature value in the closure law. If not specified, the value of the temperature unknown will be used
- **p\_ref** float for inheritance
- **t\_ref** float for inheritance
- **surface\_tension/tension\_superficielle** float for inheritance: surface tension

## 9 triple\_line\_model\_ft\_disc

Description: Triple Line Model (TCL)

See also: [objet\\_u \(48\)](#)

Usage:

**Triple\_Line\_Model\_FT\_Disc** *str*

**Read** *str* {

```
[qtcl float]
[ls float]
[coeffa float]
[coeffb float]
[theta_app float]
[ylim float]
[xm float]
[ym float]
sm float
equation_navier_stokeshydraulic_equation str
equation_temperaturethermal_equation str
equation_interfaceinterface_equation str
[ymeso float]
[n_extend_meso int]
[initial_cl_xcoord float]
[rc_tcl_gridn float]
[rc_inject float]
[thetac_tcl float]
[reinjection_tcl]
[distri_first_facette]
[adjust_meso_ml]
[lissage_tcl]
[file_name float]
[deactivate]
[inout_method str into ['exact', 'approx', 'both']]
```

}

where

- **qtcl** *float*: Heat flux contribution to micro-region [W/m]
- **ls** *float*: Slip length (unused)
- **coeffa** *float*
- **coeffb** *float*
- **theta\_app** *float*: Apparent contact angle (Cox-Voinov)
- **ylim** *float*
- **xm** *float*: Wall distance [x] of the point M delimiting micro/meso transition [m]
- **ym** *float*: Wall distance [y] of the point M delimiting micro/meso transition [m]
- **sm** *float*: Curvilinear abscissa of the point M delimiting micro/meso transition [m]
- **equation\_navier\_stokes****hydraulic\_equation** *str*: Hydraulic equation name
- **equation\_temperature****thermal\_equation** *str*: Thermal equation name
- **equation\_interface****interface\_equation** *str*: Interface equation name
- **ymeso** *float*: Meso region extension in wall-normal direction [m]
- **n\_extend\_meso** *int*: Meso region extension in number of cells [-]
- **initial\_cl\_xcoord** *float*: Initial interface position (unused)
- **rc\_tcl\_gridn** *float*: Radius of nucleate site; [in number of grids]
- **rc\_inject** *float*: Radius of bubble reinjected; [in m]
- **thetac\_tcl** *float*: imposed contact angle [in degree] to force bubble pinching / necking once TCL entre nucleate site
- **reinjection\_tcl** : This rien activates the automatic injection of a new nucleate seed with a specified shape when the temperature in the nucleation site becomes higher than a certain threshold

(tempC\_tcl). The shape of the seed is determined by the radius Rc\_tcl\_GridN and the contact angle thetaC\_tcl. The nucleation site is considered free when there are no bubbles present. The site size is defined by Rc\_tcl\_GridN. This temperature threshold, termed tempC\_tcl, is the activation temperature. Setting this temperature implies a wall temperature, therefore, activating reinjection\_tcl is ONLY possible for a simulation coupled with solid conduction.

When reinjection\_tcl is activated, the values of tempC\_tcl (default 10K), Rc\_tcl\_GridN (default 4 grid sizes), and thetaC\_tcl (default 150 degrees) should be provided. Unless (STRONGLY not recommended), the default values (indicated in parentheses) will be used.

If reinjection\_tcl is not activated (by default), the mechanism of Numerically forcing bubble pinching/necking will be used for multi-cycle simulation. Once the Triple Contact Line (TCL) enters the nucleation site, a big contact angle thetaC\_tcl is imposed to initiate bubble pinching/necking. After the bubble pinching ends, the large bubble above will depart, leaving the remaining part to serve as the nucleate seed. This process is equivalent to immediately inserting a new seed with a prescribed shape (determined by the nucleation site size and contact angle) once a bubble departs. Site size is defined by Rc\_tcl\_GridN (default 4 grid sizes). Contact angle thetaC\_tcl (default 150 degrees). Useful for a standalone (not coupling with solid conduction) simulation.

- **distri\_first\_facette** : This rien determines whether to distribute the Qtcl into all grids occupied by the first facette according to their area proportions. When set, the flux is redistributed into all grids occupied by the first facette based on their area proportions. Default value is 0, the flux is distributed differently: similar to the Meso zone, it is only distributed to grids within the Micro-zone (where the height of the front y is smaller than the size of Micro ym). The distribution of this flux is logarithmically proportional to y between 5.6nm (here interpreted as the value 0 in logarithm) and ym. In practice, in most cases, it will distribute all the flux locally in the first grid.
- **adjust\_meso\_ml** : This rien determines whether to correct of qmeso in Micro-layer
- **lissage\_tcl** : This rien determines whether to active lissage of Qmicro and theta\_app
- **file\_name** *float*: Input file to set TCL model
- **deactivate** : Simple way to disable completely the TCL model contribution
- **inout\_method** *str into ['exact', 'approx', 'both']*: Type of method for in out calc. By default, exact method is used

## 10 algo\_base

Description: Basic class for multi-grid algorithms.

See also: objet\_u (48) algo\_couple\_1 (10.1)

Usage:

### 10.1 Algo\_couple\_1

Description: not\_set

See also: algo\_base (10)

Usage:

**algo\_couple\_1** *str*

**Read** *str* {

    [ **dt\_uniforme** ]

}

where

- **dt\_uniforme**

## 11 /\*

### 11.1 /\*

Description: bloc of Comment in a data file.

See also: `objet_u` (48)

Usage:

**/\* comm**

where

- **comm** *str*: Text to be commented.

## 12 champ\_generique\_base

Description: `not_set`

See also: `objet_u` (48) `champ_post_de_champs_post` (12.1) `champ_post_refchamp` (12.17) `predefini` (12.15)

Usage:

### 12.1 Champ\_post\_de\_champs\_post

Description: `not_set`

See also: `champ_generique_base` (12) `champ_post_statistiques_base` (12.6) `champ_post_operateur_base` (12.4) `interpolation` (12.12) `champ_post_reduction_0d` (12.16) `champ_post_transformation` (12.19) `champ_post_extraction` (12.10) `champ_post_morceau_equation` (12.13) `champ_post_operateur_eqn` (12.5) `champ_post_tparoi_vef` (12.18)

Usage:

**champ\_post\_de\_champs\_post** *str*

**Read** *str* {

    [ **source** *champ\_generique\_base*]

    [ **sources** *listchamp\_generique*]

    [ **nom\_source** *str*]

    [ **source\_reference** *str*]

    [ **sources\_reference** *list\_nom\_virgule*]

}

where

- **source** *champ\_generique\_base* (12): the source field.
- **sources** *listchamp\_generique* (12.2): sources { Champ\_Post.... { ... } Champ\_Post.. { ... } }
- **nom\_source** *str*: To name a source field with the `nom_source` keyword
- **source\_reference** *str*
- **sources\_reference** *list\_nom\_virgule* (12.3)

### 12.2 Listchamp\_generique

Description: XXX

See also: `listobj` (46.5)



Usage:  
 { object1 , object2 .... }  
 list of *champ\_generique\_base* (12) separated with ,

### 12.3 List\_nom\_virgule

Description: List of name.

See also: listobj (46.5)

Usage:  
 { object1 , object2 .... }  
 list of *nom\_anonyme* (31.1) separated with ,

### 12.4 Champ\_post\_operateur\_base

Description: not\_set

See also: champ\_post\_de\_champs\_post (12.1) champ\_post\_operateur\_divergence (12.8) champ\_post\_operateur\_gradient (12.11)

Usage:  
**champ\_post\_operateur\_base** *str*  
**Read** *str* {  
     [ **source** *champ\_generique\_base*]  
     [ **sources** *listchamp\_generique*]  
     [ **nom\_source** *str*]  
     [ **source\_reference** *str*]  
     [ **sources\_reference** *list\_nom\_virgule*]  
 }  
 where

- **source** *champ\_generique\_base* (12) for inheritance: the source field.
- **sources** *listchamp\_generique* (12.2) for inheritance: sources { Champ\_Post.... { ... } Champ\_Post.. { ... } }
- **nom\_source** *str* for inheritance: To name a source field with the nom\_source keyword
- **source\_reference** *str* for inheritance
- **sources\_reference** *list\_nom\_virgule* (12.3) for inheritance

### 12.5 Champ\_post\_operateur\_eqn

Synonymous: **operateur\_eqn**

Description: Post-process equation operators/sources

See also: champ\_post\_de\_champs\_post (12.1)

Usage:  
**champ\_post\_operateur\_eqn** *str*  
**Read** *str* {

    [ **numero\_source** *int*]

```

[numero_op int]
[numero_masse int]
[sans_solveur_masse]
[compo int]
[source champ_generique_base]
[sources listchamp_generique]
[nom_source str]
[source_reference str]
[sources_reference list_nom_virgule]

```

}

where

- **numero\_source** *int*: the source to be post-processed (its number). If you have only one source term, numero\_source will correspond to 0 if you want to post-process that unique source
- **numero\_op** *int*: numero\_op will be 0 (diffusive operator) or 1 (convective operator) or 2 (gradient operator) or 3 (divergence operator).
- **numero\_masse** *int*: numero\_masse will be 0 for the mass equation operator in Pb\_multiphase.
- **sans\_solveur\_masse**
- **compo** *int*: If you want to post-process only one component of a vector field, you can specify the number of the component after compo keyword. By default, it is set to -1 which means that all the components will be post-processed. This feature is not available in VDF discretization.
- **source** *champ\_generique\_base* (12) for inheritance: the source field.
- **sources** *listchamp\_generique* (12.2) for inheritance: sources { Champ\_Post... { ... } Champ\_Post.. { ... }}
- **nom\_source** *str* for inheritance: To name a source field with the nom\_source keyword
- **source\_reference** *str* for inheritance
- **sources\_reference** *list\_nom\_virgule* (12.3) for inheritance

## 12.6 Champ\_post\_statistiques\_base

Description: not\_set

See also: champ\_post\_de\_champs\_post (12.1) moyenne (12.14) correlation (12.7) ecart\_type (12.9)

Usage:

**champ\_post\_statistiques\_base** *str*

**Read** *str* {

```

t_deb float
t_fin float
[source champ_generique_base]
[sources listchamp_generique]
[nom_source str]
[source_reference str]
[sources_reference list_nom_virgule]

```

}

where

- **t\_deb** *float*: Start of integration time
- **t\_fin** *float*: End of integration time
- **source** *champ\_generique\_base* (12) for inheritance: the source field.
- **sources** *listchamp\_generique* (12.2) for inheritance: sources { Champ\_Post... { ... } Champ\_Post.. { ... }}

- **nom\_source** *str* for inheritance: To name a source field with the `nom_source` keyword
- **source\_reference** *str* for inheritance
- **sources\_reference** *list\_nom\_virgule* ([12.3](#)) for inheritance

## 12.7 Correlation

Synonymous: **champ\_post\_statistiques\_correlation**

Description: to calculate the correlation between the two fields.

See also: `champ_post_statistiques_base` ([12.6](#))

Usage:

**correlation** *str*

**Read** *str* {

```

 t_deb float
 t_fin float
 [source champ_generique_base]
 [sources listchamp_generique]
 [nom_source str]
 [source_reference str]
 [sources_reference list_nom_virgule]

```

}

where

- **t\_deb** *float* for inheritance: Start of integration time
- **t\_fin** *float* for inheritance: End of integration time
- **source** *champ\_generique\_base* ([12](#)) for inheritance: the source field.
- **sources** *listchamp\_generique* ([12.2](#)) for inheritance: sources { Champ\_Post... { ... } Champ\_Post.. { ... } }
- **nom\_source** *str* for inheritance: To name a source field with the `nom_source` keyword
- **source\_reference** *str* for inheritance
- **sources\_reference** *list\_nom\_virgule* ([12.3](#)) for inheritance

## 12.8 Champ\_post\_operateur\_divergence

Synonymous: **divergence**

Description: To calculate divergency of a given field.

See also: `champ_post_operateur_base` ([12.4](#))

Usage:

**champ\_post\_operateur\_divergence** *str*

**Read** *str* {

```

 [source champ_generique_base]
 [sources listchamp_generique]
 [nom_source str]
 [source_reference str]
 [sources_reference list_nom_virgule]

```

}  
where

- **source** *champ\_generique\_base* (12) for inheritance: the source field.
- **sources** *listchamp\_generique* (12.2) for inheritance: sources { Champ\_Post... { ... } Champ\_Post.. { ... } }
- **nom\_source** *str* for inheritance: To name a source field with the nom\_source keyword
- **source\_reference** *str* for inheritance
- **sources\_reference** *list\_nom\_virgule* (12.3) for inheritance

## 12.9 Ecart\_type

Synonymous: **champ\_post\_statistiques\_ecart\_type**

Description: to calculate the standard deviation (statistic rms) of the field nom\_champ.

See also: **champ\_post\_statistiques\_base** (12.6)

Usage:

**ecart\_type** *str*

**Read** *str* {

**t\_deb** *float*  
    **t\_fin** *float*  
    [ **source** *champ\_generique\_base* ]  
    [ **sources** *listchamp\_generique* ]  
    [ **nom\_source** *str* ]  
    [ **source\_reference** *str* ]  
    [ **sources\_reference** *list\_nom\_virgule* ]

}  
where

- **t\_deb** *float* for inheritance: Start of integration time
- **t\_fin** *float* for inheritance: End of integration time
- **source** *champ\_generique\_base* (12) for inheritance: the source field.
- **sources** *listchamp\_generique* (12.2) for inheritance: sources { Champ\_Post... { ... } Champ\_Post.. { ... } }
- **nom\_source** *str* for inheritance: To name a source field with the nom\_source keyword
- **source\_reference** *str* for inheritance
- **sources\_reference** *list\_nom\_virgule* (12.3) for inheritance

## 12.10 Champ\_post\_extraction

Synonymous: **extraction**

Description: To create a surface field (values at the boundary) of a volume field

See also: **champ\_post\_de\_champs\_post** (12.1)

Usage:

**champ\_post\_extraction** *str*

**Read** *str* {

```

domaine str
nom_frontiere str
[methode str into ['trace', 'champ_frontiere']]
[source champ_generique_base]
[sources listchamp_generique]
[nom_source str]
[source_reference str]
[sources_reference list_nom_virgule]
}
where

```

- **domaine** *str*: name of the volume field
- **nom\_frontiere** *str*: boundary name where the values of the volume field will be picked
- **methode** *str* into ['trace', 'champ\_frontiere']: name of the extraction method (trace by\_default or champ\_frontiere)
- **source** *champ\_generique\_base* (12) for inheritance: the source field.
- **sources** *listchamp\_generique* (12.2) for inheritance: sources { Champ\_Post.... { ... } Champ\_Post.. { ... } }
- **nom\_source** *str* for inheritance: To name a source field with the nom\_source keyword
- **source\_reference** *str* for inheritance
- **sources\_reference** *list\_nom\_virgule* (12.3) for inheritance

## 12.11 Champ\_post\_operateur\_gradient

Synonymous: **gradient**

Description: To calculate gradient of a given field.

See also: champ\_post\_operateur\_base (12.4)

Usage:

**champ\_post\_operateur\_gradient** *str*

**Read** *str* {

```

[source champ_generique_base]
[sources listchamp_generique]
[nom_source str]
[source_reference str]
[sources_reference list_nom_virgule]

```

```

}
where

```

- **source** *champ\_generique\_base* (12) for inheritance: the source field.
- **sources** *listchamp\_generique* (12.2) for inheritance: sources { Champ\_Post.... { ... } Champ\_Post.. { ... } }
- **nom\_source** *str* for inheritance: To name a source field with the nom\_source keyword
- **source\_reference** *str* for inheritance
- **sources\_reference** *list\_nom\_virgule* (12.3) for inheritance

## 12.12 Interpolation

Synonymous: **champ\_post\_interpolation**

Description: To create a field which is an interpolation of the field given by the keyword source.

See also: champ\_post\_de\_champs\_post ([12.1](#))

Usage:

**interpolation** *str*

**Read** *str* {

```
 localisation str
 [methode str]
 [domaine str]
 [optimisation_sous_maillage str into ['default', 'yes', 'no']]
 [source champ_generique_base]
 [sources listchamp_generique]
 [nom_source str]
 [source_reference str]
 [sources_reference list_nom_virgule]
```

}

where

- **localisation** *str*: type\_loc indicate where is done the interpolation (elem for element or som for node).
- **methode** *str*: The optional keyword methode is limited to calculer\_champ\_post for the moment.
- **domaine** *str*: the domain name where the interpolation is done (by default, the calculation domain)
- **optimisation\_sous\_maillage** *str* into ['default', 'yes', 'no']
- **source** *champ\_generique\_base* ([12](#)) for inheritance: the source field.
- **sources** *listchamp\_generique* ([12.2](#)) for inheritance: sources { Champ\_Post.... { ... } Champ\_Post.. { ... } }
- **nom\_source** *str* for inheritance: To name a source field with the nom\_source keyword
- **source\_reference** *str* for inheritance
- **sources\_reference** *list\_nom\_virgule* ([12.3](#)) for inheritance

## 12.13 Champ\_post\_morceau\_equation

Synonymous: **morceau\_equation**

Description: To calculate a field related to a piece of equation. For the moment, the field which can be calculated is the stability time step of an operator equation. The problem name and the unknown of the equation should be given by Source refChamp { Pb\_Champ problem\_name unknown\_field\_of\_equation }

See also: champ\_post\_de\_champs\_post ([12.1](#))

Usage:

**champ\_post\_morceau\_equation** *str*

**Read** *str* {

```
 type str
 [numero int]
 [unite str]
 option str into ['stabilite', 'flux_bords', 'flux_surfacique_bords']
```

```

[compo int]
[source champ_generique_base]
[sources listchamp_generique]
[nom_source str]
[source_reference str]
[sources_reference list_nom_virgule]
}
where

```

- **type** *str*: can only be `operateur` for equation operators.
- **numero** *int*: numero will be 0 (diffusive operator) or 1 (convective operator) or 2 (gradient operator) or 3 (divergence operator).
- **unite** *str*: will specify the field unit
- **option** *str* into [`'stabilite'`, `'flux_bords'`, `'flux_surfacique_bords'`]: option is stability for time steps or `flux_bords` for boundary fluxes or `flux_surfacique_bords` for boundary surfacic fluxes
- **compo** *int*: compo will specify the number component of the boundary flux (for boundary fluxes, in this case compo permits to specify the number component of the boundary flux choosen).
- **source** *champ\_generique\_base* (12) for inheritance: the source field.
- **sources** *listchamp\_generique* (12.2) for inheritance: sources { `Champ_Post...` { ... } `Champ_Post..` { ... } }
- **nom\_source** *str* for inheritance: To name a source field with the `nom_source` keyword
- **source\_reference** *str* for inheritance
- **sources\_reference** *list\_nom\_virgule* (12.3) for inheritance

## 12.14 Moyenne

Synonymous: **champ\_post\_statistiques\_moyenne**

Description: to calculate the average of the field over time

See also: **champ\_post\_statistiques\_base** (12.6)

Usage:

**moyenne** *str*

**Read** *str* {

```

[moyenne_convergee champ_base]
t_deb float
t_fin float
[source champ_generique_base]
[sources listchamp_generique]
[nom_source str]
[source_reference str]
[sources_reference list_nom_virgule]
}
where

```

- **moyenne\_convergee** *champ\_base* (20.1): This option allows to read a converged time averaged field in a .xyz file in order to calculate, when resuming the calculation, the statistics fields (rms, correlation) which depend on this average. In that case, the time averaged field is not updated during the resume of calculation. In this case, the time averaged field must be fully converged to avoid errors when calculating high order statistics.
- **t\_deb** *float* for inheritance: Start of integration time

- **t\_fin** *float* for inheritance: End of integration time
- **source** *champ\_generique\_base* (12) for inheritance: the source field.
- **sources** *listchamp\_generique* (12.2) for inheritance: sources { Champ\_Post.... { ... } Champ\_Post.. { ... } }
- **nom\_source** *str* for inheritance: To name a source field with the nom\_source keyword
- **source\_reference** *str* for inheritance
- **sources\_reference** *list\_nom\_virgule* (12.3) for inheritance

## 12.15 Predefini

Description: This keyword is used to post process predefined postprocessing fields.

See also: *champ\_generique\_base* (12)

Usage:

**predefini** *str*

**Read** *str* {

**pb\_champ** *deuxmots*

}

where

- **pb\_champ** *deuxmots* (4.9.1): { Pb\_champ nom\_pb nom\_champ } : nom\_pb is the problem name and nom\_champ is the selected field name. The available keywords for the field name are: energie\_cinetique\_totale, energie\_cinetique\_elem, viscosite\_turbulente, viscous\_force\_x, viscous\_force\_y, viscous\_force\_z, pressure\_force\_x, pressure\_force\_y, pressure\_force\_z, total\_force\_x, total\_force\_y, total\_force\_z, viscous\_force, pressure\_force, total\_force

## 12.16 Champ\_post\_reduction\_0d

Synonymous: **reduction\_0d**

Description: To calculate the min, max, sum, average, weighted sum, weighted average, weighted sum by porosity, weighted average by porosity, euclidian norm, normalized euclidian norm, L1 norm, L2 norm of a field.

See also: *champ\_post\_de\_champs\_post* (12.1)

Usage:

**champ\_post\_reduction\_0d** *str*

**Read** *str* {

**methode** *str* into ['min', 'max', 'moyenne', 'average', 'moyenne\_ponderee', 'weighted\_average', 'somme', 'sum', 'somme\_ponderee', 'weighted\_sum', 'somme\_ponderee\_porosite', 'weighted\_sum\_porosity', 'euclidian\_norm', 'normalized\_euclidian\_norm', 'L1\_norm', 'L2\_norm', 'valeur\_a\_gauche', 'left\_value']

[ **source** *champ\_generique\_base*]

[ **sources** *listchamp\_generique*]

[ **nom\_source** *str*]

[ **source\_reference** *str*]

[ **sources\_reference** *list\_nom\_virgule*]

}

where



- **methode** *str* into [*'min'*, *'max'*, *'moyenne'*, *'average'*, *'moyenne\_ponderee'*, *'weighted\_average'*, *'somme'*, *'sum'*, *'somme\_ponderee'*, *'weighted\_sum'*, *'somme\_ponderee\_porosite'*, *'weighted\_sum\_porosity'*, *'euclidian\_norm'*, *'normalized\_euclidian\_norm'*, *'L1\_norm'*, *'L2\_norm'*, *'valeur\_a\_gauche'*, *'left\_value'*]: name of the reduction method:
  - min for the minimum value,
  - max for the maximum value,
  - average (or moyenne) for a mean,
  - weighted\_average (or moyenne\_ponderee) for a mean ponderated by integration volumes, e.g: cell volumes for temperature and pressure in VDF, volumes around faces for velocity and temperature in VEF,
  - sum (or somme) for the sum of all the values of the field,
  - weighted\_sum (or somme\_ponderee) for a weighted sum (integral),
  - weighted\_average\_porosity (or moyenne\_ponderee\_porosite) and weighted\_sum\_porosity (or somme\_ponderee\_porosite) for the mean and sum weighted by the volumes of the elements, only for ELEM localisation,
  - euclidian\_norm for the euclidian norm,
  - normalized\_euclidian\_norm for the euclidian norm normalized,
  - L1\_norm for norm L1,
  - L2\_norm for norm L2
- **source** *champ\_generique\_base* (12) for inheritance: the source field.
- **sources** *listchamp\_generique* (12.2) for inheritance: sources { Champ\_Post.... { ... } Champ\_Post.. { ... } }
- **nom\_source** *str* for inheritance: To name a source field with the nom\_source keyword
- **source\_reference** *str* for inheritance
- **sources\_reference** *list\_nom\_virgule* (12.3) for inheritance

## 12.17 Champ\_post\_refchamp

Synonymous: **refchamp**

Description: Field of prolem

See also: *champ\_generique\_base* (12)

Usage:

**champ\_post\_refchamp** *str*

**Read** *str* {

[ **nom\_source** *str* ]  
**pb\_champ** *deuxmots*

}

where

- **nom\_source** *str*: The alias name for the field
- **pb\_champ** *deuxmots* (4.9.1): { Pb\_champ nom\_pb nom\_champ } : nom\_pb is the problem name and nom\_champ is the selected field name.

## 12.18 Champ\_post\_tparoi\_vef

Synonymous: **tparoi\_vef**

Description: This keyword is used to post process (only for VEF discretization) the temperature field

with a slight difference on boundaries with Neumann condition where law of the wall is applied on the temperature field. `nom_pb` is the problem name and `field_name` is the selected field name. A keyword (`temperature_physique`) is available to post process this field without using `Definition_champs`.

See also: `champ_post_de_champs_post` ([12.1](#))

Usage:

**champ\_post\_tparoi\_vef** *str*

**Read** *str* {

```
[source champ_generique_base]
[sources listchamp_generique]
[nom_source str]
[source_reference str]
[sources_reference list_nom_virgule]
```

}

where

- **source** *champ\_generique\_base* ([12](#)) for inheritance: the source field.
- **sources** *listchamp\_generique* ([12.2](#)) for inheritance: sources { Champ\_Post... { ... } Champ\_Post.. { ... } }
- **nom\_source** *str* for inheritance: To name a source field with the `nom_source` keyword
- **source\_reference** *str* for inheritance
- **sources\_reference** *list\_nom\_virgule* ([12.3](#)) for inheritance

## 12.19 Champ\_post\_transformation

Synonymous: **transformation**

Description: To create a field with a transformation using source fields and x, y, z, t. If you use in your datafile `source refChamp { Pb_champ pb pression }`, the field `pression` may be used in the expression with the name `pression_natif_dom`; this latter is the same as `pression`. If you specify `nom_source` in `refChamp` bloc, you should use the alias given to pressure field. This is avail for all equations unknowns in transformation.

See also: `champ_post_de_champs_post` ([12.1](#))

Usage:

**champ\_post\_transformation** *str*

**Read** *str* {

```
methode str into ['produit_scalaire', 'norme', 'vecteur', 'formule', 'composante']
[unite str]
[expression n word1 word2 ... wordn]
[numero int]
[localisation str]
[source champ_generique_base]
[sources listchamp_generique]
[nom_source str]
[source_reference str]
[sources_reference list_nom_virgule]
```

}

where

- **methode** *str* into ['produit\_scalaire', 'norme', 'vecteur', 'formule', 'composante']: methode 0  
methode norme : will calculate the norm of a vector given by a source field  
methode produit\_scalaire : will calculate the dot product of two vectors given by two sources fields  
methode composante numero integer : will create a field by extracting the integer component of a field given by a source field  
methode formule expression 1 : will create a scalar field located to elements using expressions with x,y,z,t parameters and field names given by a source field or several sources fields.  
methode vecteur expression N f1(x,y,z,t) fN(x,y,z,t) : will create a vector field located to elements by defining its N components with N expressions with x,y,z,t parameters and field names given by a source field or several sources fields.
- **unite** *str*: will specify the field unit
- **expression** *n word1 word2 ... wordn*: expression 1 see methodes formule and vecteur
- **numero** *int*: numero 1 see methode composante
- **localisation** *str*: localisation 1 type\_loc indicate where is done the interpolation (elem for element or som for node). The optional keyword methode is limited to calculer\_champ\_post for the moment
- **source** *champ\_generique\_base* (12) for inheritance: the source field.
- **sources** *listchamp\_generique* (12.2) for inheritance: sources { Champ\_Post.... { ... } Champ\_Post.. { ... } }
- **nom\_source** *str* for inheritance: To name a source field with the nom\_source keyword
- **source\_reference** *str* for inheritance
- **sources\_reference** *list\_nom\_virgule* (12.3) for inheritance

## 13 chimie

Description: Keyword to describe the chemical reactions

See also: objet\_u (48)

Usage:

**chimie** *str*

**Read** *str* {

```

 reactions reactions
 [modele_micro_melange int]
 [constante_modele_micro_melange float]
 [espece_en_competition_micro_melange str]

```

}

where

- **reactions** *reactions* (13.1): list of reactions
- **modele\_micro\_melange** *int*: modele\_micro\_melange (0 by default)
- **constante\_modele\_micro\_melange** *float*: constante of modele (1 by default)
- **espece\_en\_competition\_micro\_melange** *str*: espece in competition in reactions

### 13.1 Reactions

Description: list of reactions

See also: listobj (46.5)

Usage:

{ object1 , object2 .... }

list of *reaction* (13.1.1) separated with ,

### 13.1.1 Reaction

Description: Keyword to describe reaction:

$w = K \text{ pow}(T, \text{beta}) \exp(-E_a / (R T)) \prod \text{pow}(\text{Reactiv}_i, \text{activity}_i)$ .

If  $K_{\text{inv}} > 0$ ,

$w = K \text{ pow}(T, \text{beta}) \exp(-E_a / (R T)) ( \prod \text{pow}(\text{Reactiv}_i, \text{activity}_i) - K_{\text{inv}} / \exp(-c_r E_a / (R T)) \prod \text{pow}(\text{Produit}_i, \text{activity}_i) )$

See also: `objet_lecture` ([47](#))

Usage:

```
{
 reactifs str
 produits str
 [constante_taux_reaction float]
 enthalpie_reaction float
 energie_activation float
 exposant_beta float
 [coefficients_activites bloc_lecture]
 [contre_reaction float]
 [contre_energie_activation float]
```

```
}
```

where

- **reactifs** *str*: LHS of equation (ex CH4+2\*O2)
- **produits** *str*: RHS of equation (ex CO2+2\*H2O)
- **constante\_taux\_reaction** *float*: constante of cinetic K
- **enthalpie\_reaction** *float*: DH
- **energie\_activation** *float*: Ea
- **exposant\_beta** *float*: Beta
- **coefficients\_activites** *bloc\_lecture* ([3.2](#)): coefficients of activity (exemple { CH4 1 O2 2 })
- **contre\_reaction** *float*:  $K_{\text{inv}}$
- **contre\_energie\_activation** *float*:  $c_r E_a$

## 14 class\_generic

Description: `not_set`

See also: `objet_u` ([48](#)) `solveur_sys_base` ([14.19](#)) `dt_start` ([14.11](#)) `Modele_Fonc_Realisable_base` ([14.1](#))

Usage:

### 14.1 Modele\_fonc\_realisable\_base

Description: Base class for Functions necessary to Realizable K-Epsilon Turbulence Model

See also: `class_generic` ([14](#)) `Shih_Zhu_Lumley` ([14.3](#)) `Modele_Shih_Zhu_Lumley_VDF` ([14.2](#))

Usage:

## 14.2 Modele\_shih\_zhu\_lumley\_vdf

Description: Functions necessary to Realizable K-Epsilon Turbulence Model in VDF

See also: Modele\_Fonc\_Realisable\_base ([14.1](#))

Usage:

**Modele\_Shih\_Zhu\_Lumley\_VDF** *str*

**Read** *str* {

    [ **a0** *float*]

}

where

- **a0** *float*: value of parameter A0 in U\* formula

## 14.3 Shih\_zhu\_lumley

Description: Functions necessary to Realizable K-Epsilon Turbulence Model in VEF

See also: Modele\_Fonc\_Realisable\_base ([14.1](#))

Usage:

**Shih\_Zhu\_Lumley** *str*

**Read** *str* {

    [ **a0** *float*]

}

where

- **a0** *float*: value of parameter A0 in U\* formula

## 14.4 Amg

Description: Wrapper for AMG preconditioner-based solver which switch for the best one on CPU/GPU  
Nvidia/GPU AMD

See also: solveur\_sys\_base ([14.19](#))

Usage:

**amg solveur option\_solveur**

where

- **solveur** *str*
- **option\_solveur** *bloc\_lecture* ([3.2](#))

## 14.5 Amgx

Description: Solver via AmgX API

See also: petsc ([14.16](#))

Usage:

**amgx solveur option\_solveur**

where

- **solveur** *str*
- **option\_solveur** *bloc\_lecture* (3.2)

## 14.6 Cholesky

Description: Cholesky direct method.

See also: `solveur_sys_base` (14.19)

Usage:

**cholesky** *str*

**Read** *str* {

    [ **impr** ]

    [ **quiet** ]

}

where

- **impr** : Keyword which may be used to print the resolution time.
- **quiet** : To disable printing of information

## 14.7 Cudss

Description: Solver via cuDSS API

See also: `petsc` (14.16)

Usage:

**cuDSS solveur option\_solveur**

where

- **solveur** *str*
- **option\_solveur** *bloc\_lecture* (3.2)

## 14.8 Dt\_calc

Description: The time step at first iteration is calculated in agreement with CFL condition.

See also: `dt_start` (14.11)

Usage:

**dt\_calc**

## 14.9 Dt\_fixe

Description: The first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity).

See also: `dt_start` (14.11)

Usage:

**dt\_fixe value**

where

- **value** *float*: first time step.

## 14.10 Dt\_min

Description: The first iteration is based on dt\_min.

See also: dt\_start (14.11)

Usage:

**dt\_min**

## 14.11 Dt\_start

Description: not\_set

See also: class\_generic (14) dt\_calc (14.8) dt\_min (14.10) dt\_fixe (14.9)

Usage:

**dt\_start**

## 14.12 Gcp\_ns

Description: not\_set

See also: gcp (14.18)

Usage:

**gcp\_ns** *str*

**Read** *str* {

```

 solveur0 solveur_sys_base
 solveur1 solveur_sys_base
 seuil float
 [nb_it_max int]
 [impr]
 [quiet]
 [save_matrix|save_matrice int]
 [precond precond_base]
 [precond_nul]
 [precond_diagonal]
 [optimized]

```

}

where

- **solveur0** *solveur\_sys\_base* (14.19): Solver type.
- **solveur1** *solveur\_sys\_base* (14.19): Solver type.
- **seuil** *float* for inheritance: Value of the final residue. The gradient ceases iteration when the Euclidean residue standard  $\|Ax-B\|$  is less than this value.
- **nb\_it\_max** *int* for inheritance: Keyword to set the maximum iterations number for the Gcp.
- **impr** for inheritance: Keyword which is used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).
- **quiet** for inheritance: To not displaying any outputs of the solver.
- **save\_matrix|save\_matrice** *int* for inheritance: to save the matrix in a file.

- **precond** *precond\_base* (35) for inheritance: Keyword to define system preconditioning in order to accelerate resolution by the conjugated gradient. Many parallel preconditioning methods are not equivalent to their sequential counterpart, and you should therefore expect differences, especially when you select a high value of the final residue (*seuil*). The result depends on the number of processors and on the mesh splitting. It is sometimes useful to run the solver with no preconditioning at all. In particular:
  - when the solver does not converge during initial projection,
  - when comparing sequential and parallel computations.
 With no preconditioning, except in some particular cases (no open boundary), the sequential and the parallel computations should provide exactly the same results within fpu accuracy. If not, there might be a coding error or the system of equations is singular.
- **precond\_nul** for inheritance: Keyword to not use a preconditioning method.
- **precond\_diagonal** for inheritance: Keyword to use diagonal preconditioning.
- **optimized** for inheritance: This keyword triggers a memory and network optimized algorithms useful for strong scaling (when computing less than 100 000 elements per processor). The matrix and the vectors are duplicated, common items removed and only virtual items really used in the matrix are exchanged.  
Warning: this is experimental and known to fail in some VEF computations (L2 projection step will not converge). Works well in VDF.

## 14.13 Gen

Description: not\_set

See also: solveur\_sys\_base (14.19)

Usage:

```
gen str
Read str {
 solv_elem str
 precond precond_base
 [seuil float]
 [impr]
 [save_matrix|save_matrice]
 [quiet]
 [nb_it_max int]
 [force]
}
```

where

- **solv\_elem** *str*: To specify a solver among gmres or bicgstab.
- **precond** *precond\_base* (35): The only preconditionner that we can specify is *ilu*.
- **seuil** *float*: Value of the final residue. The solver ceases iterations when the Euclidean residue standard  $\|Ax-B\|$  is less than this value. default value  $1e-12$ .
- **impr** : Keyword which is used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).
- **save\_matrix**|**save\_matrice** : To save the matrix in a file.
- **quiet** : To not displaying any outputs of the solver.
- **nb\_it\_max** *int*: Keyword to set the maximum iterations number for the GEN solver.
- **force** : Keyword to set `ipar[5]=-1` in the GEN solver. This is helpful if you notice that the solver does not perform more than 100 iterations. If this keyword is specified in the datafile, you should provide *nb\_it\_max*.



## 14.14 Gmres

Description: Gmres method (for non symetric matrix).

See also: solveur\_sys\_base ([14.19](#))

Usage:

**gmres** *str*

**Read** *str* {

[ **impr** ]  
[ **quiet** ]  
[ **seuil** *float*]  
[ **diag** ]  
[ **nb\_it\_max** *int*]  
[ **controle\_residu** *int* into [0, 1]]  
[ **save\_matrix**|**save\_matrice** ]  
[ **dim\_espace\_krilov** *int*]

}

where

- **impr** : Keyword which may be used to print the convergence.
- **quiet** : To disable printing of information
- **seuil** *float*: Convergence value.
- **diag** : Keyword to use diagonal preconditionner (in place of pilut that is not parallel).
- **nb\_it\_max** *int*: Keyword to set the maximum iterations number for the Gmres.
- **controle\_residu** *int* into [0, 1]: Keyword of Boolean type (by default 0). If set to 1, the convergence occurs if the residu suddenly increases.
- **save\_matrix**|**save\_matrice** : to save the matrix in a file.
- **dim\_espace\_krilov** *int*

## 14.15 Optimal

Description: Optimal is a solver which tests several solvers of the previous list to choose the fastest one for the considered linear system.

See also: solveur\_sys\_base ([14.19](#))

Usage:

**optimal** *str*

**Read** *str* {

**seuil** *float*  
[ **impr** ]  
[ **quiet** ]  
[ **save\_matrix**|**save\_matrice** ]  
[ **frequence\_recalc** *int*]  
[ **nom\_fichier\_solveur** *str*]  
[ **fichier\_solveur\_non\_recre** ]

}

where

- **seuil** *float*: Convergence threshold

- **impr** : To print the convergency of the fastest solver
- **quiet** : To disable printing of information
- **save\_matrix|save\_matrice** : To save the linear system (A, x, B) into a file
- **frequence\_recalc** *int*: To set a time step period (by default, 100) for re-checking the fastest solver
- **nom\_fichier\_solveur** *str*: To specify the file containing the list of the tested solvers
- **fichier\_solveur\_non\_recreer** : To avoid the creation of the file containing the list

## 14.16 Petsc

Description: Solver via Petsc API

See also: `solveur_sys_base` ([14.19](#)) `petsc_gpu` ([14.17](#)) `amgx` ([14.5](#)) `cuDSS` ([14.7](#))

Usage:

**petsc solveur**

where

- **solveur** *solveur\_petsc\_deriv* ([40](#)): solver type and options

## 14.17 Petsc\_gpu

Description: GPU solver via Petsc API

See also: `petsc` ([14.16](#))

Usage:

**petsc\_gpu solveur option\_solveur [ atol ] [ rtol ]**

where

- **solveur** *str*
- **option\_solveur** *bloc\_lecture* ([3.2](#))
- **atol** *float*: Absolute threshold for convergence (same as `seuil` option)
- **rtol** *float*: Relative threshold for convergence

## 14.18 Gcp

Description: Preconditioned conjugated gradient.

See also: `solveur_sys_base` ([14.19](#)) `gcp_ns` ([14.12](#))

Usage:

**gcp** *str*

**Read** *str* {

```

 seuil float
 [nb_it_max int]
 [impr]
 [quiet]
 [save_matrix|save_matrice int]
 [precond precond_base]
 [precond_nul]
 [precond_diagonal]

```

```
[optimized]
}
```

where

- **seuil** *float*: Value of the final residue. The gradient ceases iteration when the Euclidean residue standard  $\|Ax-B\|$  is less than this value.
- **nb\_it\_max** *int*: Keyword to set the maximum iterations number for the Gcp.
- **impr** : Keyword which is used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).
- **quiet** : To not displaying any outputs of the solver.
- **save\_matrix|save\_matrice** *int*: to save the matrix in a file.
- **precond** *precond\_base* (35): Keyword to define system preconditioning in order to accelerate resolution by the conjugated gradient. Many parallel preconditioning methods are not equivalent to their sequential counterpart, and you should therefore expect differences, especially when you select a high value of the final residue (seuil). The result depends on the number of processors and on the mesh splitting. It is sometimes useful to run the solver with no preconditioning at all. In particular:
  - when the solver does not converge during initial projection,
  - when comparing sequential and parallel computations.
 With no preconditioning, except in some particular cases (no open boundary), the sequential and the parallel computations should provide exactly the same results within fpv accuracy. If not, there might be a coding error or the system of equations is singular.
- **precond\_nul** : Keyword to not use a preconditioning method.
- **precond\_diagonal** : Keyword to use diagonal preconditioning.
- **optimized** : This keyword triggers a memory and network optimized algorithms useful for strong scaling (when computing less than 100 000 elements per processor). The matrix and the vectors are duplicated, common items removed and only virtual items really used in the matrix are exchanged. Warning: this is experimental and known to fail in some VEF computations (L2 projection step will not converge). Works well in VDF.

## 14.19 Solveur\_sys\_base

Description: Basic class to solve the linear system.

See also: class\_generic (14) optimal (14.15) cholesky (14.6) petsc (14.16) gcp (14.18) gmres (14.14) amg (14.4) gen (14.13)

Usage:

## 15 collision\_model\_fpi\_deriv

Description: not\_set

See also: objet\_u (48) Collision\_Model\_FT\_sphere (15.1)

Usage:

### 15.1 Collision\_model\_ft\_sphere

Description: not\_set

See also: collision\_model\_fpi\_deriv (15)

Usage:

**Collision\_Model\_FT\_sphere** *str*

**Read** *str* {

[ **collision\_model** *str*]  
[ **detection\_method** *bloc\_lecture*]  
[ **collision\_duration** *float*]  
[ **activate\_collision\_before\_impact** *int*]  
[ **activation\_distance\_percentage\_diameter** *int*]  
[ **force\_on\_two\_phase\_elem** *int*]

}

where

- **collision\_model** *str*
- **detection\_method** *bloc\_lecture* (3.2)
- **collision\_duration** *float*
- **activate\_collision\_before\_impact** *int*
- **activation\_distance\_percentage\_diameter** *int*
- **force\_on\_two\_phase\_elem** *int*

## 16 #

### 16.1 #

Description: Comments in a data file.

See also: `objet_u` (48)

Usage:

**# comm**

where

- **comm** *str*: Text to be commented.

## 17 condlim\_base

Description: Basic class of boundary conditions.

See also: `objet_u` (48) `paroi_echange_global_impose` (17.69) `Paroi_echange_interne_global_impose` (17.6) `Paroi_echange_interne_global_parfait` (17.7) `neumann` (17.52) `paroi_echange_contact_correlation_vdf` (17.59) `paroi_echange_contact_vdf` (17.63) `Paroi_echange_interne_parfait` (17.9) `Paroi_echange_interne_impose` (17.8) `dirichlet` (17.19) `Neumann_homogene` (17.11) `frontiere_ouverte_fraction_massique_imposee` (17.31) `symetrie` (17.89) `paroi_echange_externer_impose` (17.65) `Neumann_paro` (17.12) `paroi_flux_impose` (17.72) `paroi_contact_fictif` (17.55) `paroi_adiabatique` (17.53) `paroi_contact` (17.54) `paroi_echange_externer_radiatif` (17.23) `periodique` (17.84) `paroi_echange_contact_correlation_vdf` (17.60) `robin_vdf` (17.85) `paroi_fixe` (17.70) `Paroi` (17.14) `paroi_decalee_robin` (17.57) `frontiere_ouverte_k_eps_impose` (17.36) `frontiere_ouverte_k_omega_impose` (17.37) `paroi_ft_disc` (17.76) `sortie_libre_rho_variable` (17.87) `flux_radiatif` (17.25) `paroi_contact_rayo` (17.56) `contact_vdf_vdf` (17.17) `contact_vdf_vdf` (17.18) `perio` (17.83) `Mixte_shear` (17.10) `Paroi_frottante_simple` (17.16) `Cond_lim_omega_dix` (17.4) `echange_contact_vdf_ft_disc` (17.21) `Cond_lim_omega_demi` (17.3) `Paroi_frottante_loi` (17.15) `echange_contact_vdf_ft_disc_solid` (17.22) `Cond_lim_k_complique_transition_flux_nul_demi` (17.1) `Cond_lim_k_simple_flux_nul` (17.2)

Usage:

**condlim\_base**

### 17.1 Cond\_lim\_k\_complique\_transition\_flux\_nul\_demi

Description: Adaptive wall law boundary condition for turbulent kinetic energy

See also: [condlim\\_base \(17\)](#)

Usage:

**Cond\_lim\_k\_complique\_transition\_flux\_nul\_demi**

### 17.2 Cond\_lim\_k\_simple\_flux\_nul

Description: Adaptive wall law boundary condition for turbulent kinetic energy

See also: [condlim\\_base \(17\)](#)

Usage:

**Cond\_lim\_k\_simple\_flux\_nul**

### 17.3 Cond\_lim\_omega\_demi

Description: Adaptive wall law boundary condition for turbulent dissipation rate

See also: [condlim\\_base \(17\)](#)

Usage:

### 17.4 Cond\_lim\_omega\_dix

Description: Adaptive wall law boundary condition for turbulent dissipation rate

See also: [condlim\\_base \(17\)](#)

Usage:

### 17.5 Echange\_couplage\_thermique

Description: Thermal coupling boundary condition

See also: [paroi\\_echange\\_global\\_impose \(17.69\)](#)

Usage:

**Echange\_couplage\_thermique** *str*

**Read** *str* {

    [ **temperature\_paro** *champ\_base*]

    [ **flux\_paro** *champ\_base*]

}

where

- **temperature\_paro** *champ\_base* ([20.1](#)): Temperature

- **flux\_paro** *champ\_base* (20.1): Wall heat flux

## 17.6 Paroi\_echange\_interne\_global\_impose

Description: Internal heat exchange boundary condition with global exchange coefficient.

See also: *condlim\_base* (17)

Usage:

**Paroi\_echange\_interne\_global\_impose h\_imp ch**

where

- **h\_imp** *str*: Global exchange coefficient value. The global exchange coefficient value is expressed in W.m-2.K-1.
- **ch** *champ\_front\_base* (21.1): Boundary field type.

## 17.7 Paroi\_echange\_interne\_global\_parfait

Description: Internal heat exchange boundary condition with perfect (infinite) exchange coefficient.

See also: *condlim\_base* (17)

Usage:

**Paroi\_echange\_interne\_global\_parfait**

## 17.8 Paroi\_echange\_interne\_impose

Description: Internal heat exchange boundary condition with exchange coefficient.

See also: *condlim\_base* (17)

Usage:

**Paroi\_echange\_interne\_impose h\_imp ch**

where

- **h\_imp** *str*: Exchange coefficient value expressed in W.m-2.K-1.
- **ch** *champ\_front\_base* (21.1): Boundary field type.

## 17.9 Paroi\_echange\_interne\_parfait

Description: Internal heat exchange boundary condition with perfect (infinite) exchange coefficient.

See also: *condlim\_base* (17)

Usage:

**Paroi\_echange\_interne\_parfait**

### 17.10 Mixte\_shear

Description: not\_set

See also: [condlim\\_base \(17\)](#)

Usage:

**Mixte\_shear**

### 17.11 Neumann\_homogene

Description: Homogeneous neumann boundary condition

See also: [condlim\\_base \(17\)](#) [Neumann\\_paroι\\_adiabatique \(17.13\)](#)

Usage:

**Neumann\_homogene**

### 17.12 Neumann\_paroι

Description: Neumann boundary condition for mass equation (multiphase problem)

See also: [condlim\\_base \(17\)](#)

Usage:

**Neumann\_paroι ch**

where

- **ch** *champ\_front\_base (21.1)*: Boundary field type.

### 17.13 Neumann\_paroι\_adiabatique

Description: Adiabatic wall neumann boundary condition

See also: [Neumann\\_homogene \(17.11\)](#)

Usage:

**Neumann\_paroι\_adiabatique**

### 17.14 Paroι

Description: Impermeability condition at a wall called bord (edge) (standard flux zero). This condition must be associated with a wall type hydraulic condition.

See also: [condlim\\_base \(17\)](#)

Usage:

**Paroι**

### 17.15 Paroι\_frottante\_loι

Description: Adaptive wall-law boundary condition for velocity

See also: [condlim\\_base \(17\)](#)

Usage:

### 17.16 Paroi\_frottante\_simple

Description: Adaptive wall-law boundary condition for velocity

See also: [condlim\\_base \(17\)](#)

Usage:

### 17.17 Contact\_vdf\_vef

Description: Boundary condition in the case of two problems (VDF -> VEF).

See also: [condlim\\_base \(17\)](#)

Usage:

**contact\_vdf\_vef champ**

where

- **champ** *champ\_front\_base* ([21.1](#)): Boundary field type.

### 17.18 Contact\_vef\_vdf

Description: Boundary condition in the case of two problems (VEF -> VDF).

See also: [condlim\\_base \(17\)](#)

Usage:

**contact\_vef\_vdf champ**

where

- **champ** *champ\_front\_base* ([21.1](#)): Boundary field type.

### 17.19 Dirichlet

Description: Dirichlet condition at the boundary called bord (edge) : 1). For Navier-Stokes equations, velocity imposed at the boundary; 2). For scalar transport equation, scalar imposed at the boundary.

See also: [condlim\\_base \(17\)](#) [paroi\\_temperature\\_imposee \(17.80\)](#) [frontiere\\_ouverte\\_vitesse\\_imposee \(17.49\)](#) [frontiere\\_ouverte\\_alpha\\_imposee \(17.29\)](#) [frontiere\\_ouverte\\_enthalpie\\_imposee \(17.46\)](#) [paroi\\_defilante \(17.58\)](#) [scalaire\\_imposee\\_parois \(17.86\)](#) [paroi\\_knudsen\\_non\\_negligeable \(17.78\)](#) [frontiere\\_ouverte\\_concentration\\_imposee \(17.30\)](#) [paroi\\_rugueuse \(17.79\)](#) [Frontiere\\_ouverte\\_vitesse\\_imposee\\_ALE \(17.50\)](#)

Usage:

**dirichlet**

### 17.20 Echange\_contact\_rayo\_transp\_vdf

Description: Exchange boundary condition in VDF between the transparent fluid and the solid for a problem coupled with radiation. Without radiation, it is the equivalent of the `Paroi_Echange_contact_VDF` exchange condition.



See also: `paroi_echange_contact_vdf` ([17.63](#))

Usage:

**exchange\_contact\_rayo\_transp\_vdf** **autrepb** **nameb** **temp** **h**

where

- **autrepb** *str*: Name of other problem.
- **nameb** *str*: Name of bord.
- **temp** *str*: Name of field.
- **h** *float*: Value assigned to a coefficient (expressed in W.K-1m-2) that characterises the contact between the two mediums. In order to model perfect contact, h must be taken to be infinite. This value must obviously be the same in both the two problems blocks.  
The surface thermal flux exchanged between the two mediums is represented by :  
$$f_i = h (T_1 - T_2)$$
 where  $1/h = d_1/\lambda_{d1} + 1/\text{val\_h\_contact} + d_2/\lambda_{d2}$   
where  $d_i$  : distance between the node where  $T_i$  and the wall is found.

## 17.21 Echange\_contact\_vdf\_ft\_disc

Description: `exchange_contact_vdf` en precisant la phase

See also: `condlim_base` ([17](#))

Usage:

**exchange\_contact\_vdf\_ft\_disc** *str*

**Read** *str* {

**autre\_probleme** *str*  
**autre\_bord** *str*  
**autre\_champ\_temperature** *str*  
**nom\_mon\_indicatrice** *str*  
**phase** *int*

}

where

- **autre\_probleme** *str*: name of other problem
- **autre\_bord** *str*: name of other boundary
- **autre\_champ\_temperature** *str*: name of other field
- **nom\_mon\_indicatrice** *str*: name of indicatrice
- **phase** *int*: phase

## 17.22 Echange\_contact\_vdf\_ft\_disc\_solid

Description: `exchange_contact_vdf` en precisant la phase

See also: `condlim_base` ([17](#))

Usage:

**exchange\_contact\_vdf\_ft\_disc\_solid** *str*

**Read** *str* {

**autre\_probleme** *str*  
**autre\_bord** *str*

```

 autre_champ_temperature_indic1 str
 autre_champ_temperature_indic0 str
 autre_champ_indicatrice str
}
where

```

- **autre\_probleme** *str*: name of other problem
- **autre\_bord** *str*: name of other boundary
- **autre\_champ\_temperature\_indic1** *str*: name of temperature indic 1
- **autre\_champ\_temperature\_indic0** *str*: name of temperature indic 0
- **autre\_champ\_indicatrice** *str*: name of indicatrice

### 17.23 Paroi\_echange\_externe\_radiatif

Synonymous: **echange\_externe\_radiatif**

Description: Combines radiative ( $\sigma * \epsilon * (T^4 - T_{ext}^4)$ ) and convective ( $h * (T - T_{ext})$ ) heat transfer boundary conditions, where  $\sigma$  is the Stefan-Boltzmann constant,  $\epsilon$  is the emi

See also: [condlim\\_base \(17\)](#)

Usage:

```

paroi_echange_externe_radiatif h_imp himpc emissivite emissivitebc t_ext ch temp_unit
temp_unit_val
where

```

- **h\_imp** *str* into ['h\_imp', 't\_ext', 'emissivite']: Heat exchange coefficient value (expressed in W.m-2.K-1).
- **himpc** *champ\_front\_base* (21.1): Boundary field type.
- **emissivite** *str* into ['emissivite', 'h\_imp', 't\_ext']: Emissivity coefficient value.
- **emissivitebc** *champ\_front\_base* (21.1): Boundary field type.
- **t\_ext** *str* into ['t\_ext', 'h\_imp', 'emissivite']: External temperature value (expressed in oC or K).
- **ch** *champ\_front\_base* (21.1): Boundary field type.
- **temp\_unit** *str* into ['temperature\_unit']: Temperature unit
- **temp\_unit\_val** *str* into ['kelvin', 'celsius']: Temperature unit

### 17.24 Entree\_temperature\_imposee\_h

Description: Particular case of class `frontiere_ouverte_temperature_imposee` for enthalpy equation.

See also: `frontiere_ouverte_enthalpie_imposee` ([17.46](#))

Usage:

```

entree_temperature_imposee_h ch
where

```

- **ch** *champ\_front\_base* (21.1): Boundary field type.

## 17.25 Flux\_radiatif

Description: Boundary condition for radiation equation.

See also: `condlim_base` ([17](#)) `flux_radiatif_vdf` ([17.26](#)) `flux_radiatif_vef` ([17.27](#))

Usage:

**flux\_radiatif na a ne emissivite**

where

- **na** *str into* [*A*']: Keyword for constant in boundary condition for irradiancy (sqrt(3) for half-infinite domain or 2 in closed domain).
- **a** *float*: Value of constant in boundary condition for irradiancy (sqrt(3) for half-infinite domain or 2 in closed domain).
- **ne** *str into* [*'emissivite'*]: Keyword for wall emissivity.
- **emissivite** *champ\_front\_base* ([21.1](#)): Wall emissivity, value between 0 and 1.

## 17.26 Flux\_radiatif\_vdf

Description: Boundary condition for radiation equation in VDF.

See also: `flux_radiatif` ([17.25](#))

Usage:

**flux\_radiatif\_vdf na a ne emissivite**

where

- **na** *str into* [*A*']: Keyword for constant in boundary condition for irradiancy (sqrt(3) for half-infinite domain or 2 in closed domain).
- **a** *float*: Value of constant in boundary condition for irradiancy (sqrt(3) for half-infinite domain or 2 in closed domain).
- **ne** *str into* [*'emissivite'*]: Keyword for wall emissivity.
- **emissivite** *champ\_front\_base* ([21.1](#)): Wall emissivity, value between 0 and 1.

## 17.27 Flux\_radiatif\_vef

Description: Boundary condition for radiation equation in VEF.

See also: `flux_radiatif` ([17.25](#))

Usage:

**flux\_radiatif\_vef na a ne emissivite**

where

- **na** *str into* [*A*']: Keyword for constant in boundary condition for irradiancy (sqrt(3) for half-infinite domain or 2 in closed domain).
- **a** *float*: Value of constant in boundary condition for irradiancy (sqrt(3) for half-infinite domain or 2 in closed domain).
- **ne** *str into* [*'emissivite'*]: Keyword for wall emissivity.
- **emissivite** *champ\_front\_base* ([21.1](#)): Wall emissivity, value between 0 and 1.

## 17.28 Frontiere\_ouverte

Description: Boundary outlet condition on the boundary called bord (edge) (diffusion flux zero). This condition must be associated with a boundary outlet hydraulic condition.

See also: neumann ([17.52](#)) frontiere\_ouverte\_rayo\_transp ([17.42](#)) frontiere\_ouverte\_rayo\_semi\_transp ([17.41](#))

Usage:

**frontiere\_ouverte** **var\_name** **ch**

where

- **var\_name** *str* into ['T\_ext', 'C\_ext', 'Y\_ext', 'K\_Eps\_ext', 'K\_Omega\_ext', 'Fluctu\_Temperature\_ext', 'Flux\_Chaleur\_Turb\_ext', 'V2\_ext', 'a\_ext', 'tau\_ext', 'k\_ext', 'omega\_ext', 'H\_ext', 'A\_i\_ext']: Field name.
- **ch** *champ\_front\_base* ([21.1](#)): Boundary field type.

## 17.29 Frontiere\_ouverte\_alpha\_impose

Description: Imposed alpha condition at the open boundary.

See also: dirichlet ([17.19](#))

Usage:

**frontiere\_ouverte\_alpha\_impose** **ch**

where

- **ch** *champ\_front\_base* ([21.1](#)): Boundary field type.

## 17.30 Frontiere\_ouverte\_concentration\_imposee

Description: Imposed concentration condition at an open boundary called bord (edge) (situation corresponding to a fluid inlet). This condition must be associated with an imposed inlet velocity condition.

See also: dirichlet ([17.19](#))

Usage:

**frontiere\_ouverte\_concentration\_imposee** **ch**

where

- **ch** *champ\_front\_base* ([21.1](#)): Boundary field type.

## 17.31 Frontiere\_ouverte\_fraction\_massique\_imposee

Description: not\_set

See also: condlim\_base ([17](#))

Usage:

**frontiere\_ouverte\_fraction\_massique\_imposee** **ch**

where

- **ch** *champ\_front\_base* ([21.1](#)): Boundary field type.

### 17.32 **Frontiere\_ouverte\_gradient\_pression\_impose**

Description: Normal imposed pressure gradient condition on the open boundary called bord (edge). This boundary condition may be only used in VDF discretization. The imposed  $\partial P/\partial n$  value is expressed in Pa.m-1.

See also: neumann (17.52) frontiere\_ouverte\_gradient\_pression\_impose\_vefprep1b (17.33)

Usage:

**frontiere\_ouverte\_gradient\_pression\_impose ch**

where

- **ch** *champ\_front\_base* (21.1): Boundary field type.

### 17.33 **Frontiere\_ouverte\_gradient\_pression\_impose\_vefprep1b**

Description: Keyword for an outlet boundary condition in VEF P1B/P1NC on the gradient of the pressure.

See also: frontiere\_ouverte\_gradient\_pression\_impose (17.32)

Usage:

**frontiere\_ouverte\_gradient\_pression\_impose\_vefprep1b ch**

where

- **ch** *champ\_front\_base* (21.1): Boundary field type.

### 17.34 **Frontiere\_ouverte\_gradient\_pression\_libre\_vef**

Description: Class for outlet boundary condition in VEF like Orlansky. There is no reference for pressure for these boundary conditions so it is better to add pressure condition (with *Frontiere\_ouverte\_pression\_imposee*) on one or two cells (for symmetry in a channel) of the boundary where Orlansky conditions are imposed.

See also: neumann (17.52)

Usage:

**frontiere\_ouverte\_gradient\_pression\_libre\_vef**

### 17.35 **Frontiere\_ouverte\_gradient\_pression\_libre\_vefprep1b**

Description: Class for outlet boundary condition in VEF P1B/P1NC like Orlansky.

See also: neumann (17.52)

Usage:

**frontiere\_ouverte\_gradient\_pression\_libre\_vefprep1b**

### 17.36 **Frontiere\_ouverte\_k\_eps\_impose**

Description: Turbulence condition imposed on an open boundary called bord (edge) (this situation corresponds to a fluid inlet). This condition must be associated with an imposed inlet velocity condition.

See also: condlim\_base (17)

Usage:

**frontiere\_ouverte\_k\_eps\_impose ch**

where

- **ch** *champ\_front\_base* (21.1): Boundary field type.

### 17.37 Frontiere\_ouverte\_k\_omega\_impose

Description: Turbulence condition imposed on an open boundary called bord (edge) (this situation corresponds to a fluid inlet). This condition must be associated with an imposed inlet velocity condition.

See also: *condlim\_base* (17)

Usage:

**frontiere\_ouverte\_k\_omega\_impose ch**

where

- **ch** *champ\_front\_base* (21.1): Boundary field type.

### 17.38 Frontiere\_ouverte\_pression\_imposee

Description: Imposed pressure condition at the open boundary called bord (edge). The imposed pressure field is expressed in Pa.

See also: *neumann* (17.52)

Usage:

**frontiere\_ouverte\_pression\_imposee ch**

where

- **ch** *champ\_front\_base* (21.1): Boundary field type.

### 17.39 Frontiere\_ouverte\_pression\_imposee\_orlansky

Description: This boundary condition may only be used with VDF discretization. There is no reference for pressure for this boundary condition so it is better to add pressure condition (with *Frontiere\_ouverte\_pression\_imposee*) on one or two cells (for symmetry in a channel) of the boundary where Orlansky conditions are imposed.

See also: *neumann* (17.52)

Usage:

**frontiere\_ouverte\_pression\_imposee\_orlansky**

### 17.40 Frontiere\_ouverte\_pression\_moyenne\_imposee

Description: Class for open boundary with pressure mean level imposed.

See also: *neumann* (17.52)

Usage:

**frontiere\_ouverte\_pression\_moyenne\_imposee pext**

where

- **pext** *float*: Mean pressure.

### 17.41 Frontiere\_ouverte\_rayo\_semi\_transp

Description: Keyword to set a boundary outlet temperature condition on the boundary called bord (edge) (diffusion flux zero) for a radiation problem with semi transparent gas.

See also: `frontiere_ouverte` ([17.28](#))

Usage:

**frontiere\_ouverte\_rayo\_semi\_transp** **var\_name** **ch**

where

- **var\_name** *str* into ['T\_ext', 'C\_ext', 'Y\_ext', 'K\_Eps\_ext', 'K\_Omega\_ext', 'Fluctu\_Temperature\_ext', 'Flux\_Chaleur\_Turb\_ext', 'V2\_ext', 'a\_ext', 'tau\_ext', 'k\_ext', 'omega\_ext', 'H\_ext', 'A\_i\_ext']: Field name.
- **ch** *champ\_front\_base* ([21.1](#)): Boundary field type.

### 17.42 Frontiere\_ouverte\_rayo\_transp

Description: Keyword to set a boundary outlet temperature condition on the boundary called bord (edge) (diffusion flux zero) for a radiation problem with transparent gas.

See also: `frontiere_ouverte` ([17.28](#)) `frontiere_ouverte_rayo_transp_vdf` ([17.43](#)) `frontiere_ouverte_rayo_transp_vdf` ([17.44](#))

Usage:

**frontiere\_ouverte\_rayo\_transp** **var\_name** **ch**

where

- **var\_name** *str* into ['T\_ext', 'C\_ext', 'Y\_ext', 'K\_Eps\_ext', 'K\_Omega\_ext', 'Fluctu\_Temperature\_ext', 'Flux\_Chaleur\_Turb\_ext', 'V2\_ext', 'a\_ext', 'tau\_ext', 'k\_ext', 'omega\_ext', 'H\_ext', 'A\_i\_ext']: Field name.
- **ch** *champ\_front\_base* ([21.1](#)): Boundary field type.

### 17.43 Frontiere\_ouverte\_rayo\_transp\_vdf

Description: doit disparaitre

See also: `frontiere_ouverte_rayo_transp` ([17.42](#))

Usage:

**frontiere\_ouverte\_rayo\_transp\_vdf** **var\_name** **ch**

where

- **var\_name** *str* into ['T\_ext', 'C\_ext', 'Y\_ext', 'K\_Eps\_ext', 'K\_Omega\_ext', 'Fluctu\_Temperature\_ext', 'Flux\_Chaleur\_Turb\_ext', 'V2\_ext', 'a\_ext', 'tau\_ext', 'k\_ext', 'omega\_ext', 'H\_ext', 'A\_i\_ext']: Field name.
- **ch** *champ\_front\_base* ([21.1](#)): Boundary field type.

## 17.44 **Frontiere\_ouverte\_rayo\_transp\_vef**

Description: doit disparaitre

See also: `frontiere_ouverte_rayo_transp` ([17.42](#))

Usage:

**frontiere\_ouverte\_rayo\_transp\_vef** **var\_name** **ch**

where

- **var\_name** *str* into ['T\_ext', 'C\_ext', 'Y\_ext', 'K\_Eps\_ext', 'K\_Omega\_ext', 'Fluctu\_Temperature\_ext', 'Flux\_Chaleur\_Turb\_ext', 'V2\_ext', 'a\_ext', 'tau\_ext', 'k\_ext', 'omega\_ext', 'H\_ext', 'A\_i\_ext']: Field name.
- **ch** *champ\_front\_base* ([21.1](#)): Boundary field type.

## 17.45 **Frontiere\_ouverte\_rho\_u\_impose**

Description: This keyword is used to designate a condition of imposed mass rate at an open boundary called bord (edge). The imposed mass rate field at the inlet is vectorial and the imposed velocity values are expressed in kg.s-1. This boundary condition can be used only with the Quasi compressible model.

See also: `frontiere_ouverte_vitesse_imposee_sortie` ([17.51](#))

Usage:

**frontiere\_ouverte\_rho\_u\_impose** **ch**

where

- **ch** *champ\_front\_base* ([21.1](#)): Boundary field type.

## 17.46 **Frontiere\_ouverte\_enthalpie\_imposee**

Synonymous: **frontiere\_ouverte\_temperature\_imposee**

Description: Imposed temperature condition at the open boundary called bord (edge) (in the case of fluid inlet). This condition must be associated with an imposed inlet velocity condition. The imposed temperature value is expressed in oC or K.

See also: `dirichlet` ([17.19](#)) `entree_temperature_imposee_h` ([17.24](#)) `frontiere_ouverte_temperature_imposee_rayo_transp` ([17.48](#)) `frontiere_ouverte_temperature_imposee_rayo_semi_transp` ([17.47](#))

Usage:

**frontiere\_ouverte\_enthalpie\_imposee** **ch**

where

- **ch** *champ\_front\_base* ([21.1](#)): Boundary field type.

## 17.47 **Frontiere\_ouverte\_temperature\_imposee\_rayo\_semi\_transp**

Description: Imposed temperature condition for a radiation problem with semi transparent gas.

See also: `frontiere_ouverte_enthalpie_imposee` ([17.46](#))

Usage:



**frontiere\_ouverte\_temperature\_imposee\_rayo\_semi\_transp ch**

where

- **ch** *champ\_front\_base* ([21.1](#)): Boundary field type.

## 17.48 Frontiere\_ouverte\_temperature\_imposee\_rayo\_transp

Description: Imposed temperature condition for a radiation problem with transparent gas.

See also: *frontiere\_ouverte\_enthalpie\_imposee* ([17.46](#))

Usage:

**frontiere\_ouverte\_temperature\_imposee\_rayo\_transp ch**

where

- **ch** *champ\_front\_base* ([21.1](#)): Boundary field type.

## 17.49 Frontiere\_ouverte\_vitesse\_imposee

Description: Class for velocity-inlet boundary condition. The imposed velocity field at the inlet is vectorial and the imposed velocity values are expressed in m.s-1.

See also: *dirichlet* ([17.19](#)) *frontiere\_ouverte\_vitesse\_imposee\_sortie* ([17.51](#))

Usage:

**frontiere\_ouverte\_vitesse\_imposee ch**

where

- **ch** *champ\_front\_base* ([21.1](#)): Boundary field type.

## 17.50 Frontiere\_ouverte\_vitesse\_imposee\_ale

Description: Class for velocity boundary condition on a mobile boundary (ALE framework).

The imposed velocity field is vectorial of type *Ch\_front\_input\_ALE*, *Champ\_front\_ALE* or *Champ\_front\_ALE\_Beam*.

Example: *frontiere\_ouverte\_vitesse\_imposee\_ALE Champ\_front\_ALE 2 0.5\*cos(0.5\*t) 0.0*

See also: *dirichlet* ([17.19](#))

Usage:

**Frontiere\_ouverte\_vitesse\_imposee\_ALE ch**

where

- **ch** *champ\_front\_base* ([21.1](#)): Boundary field type.

## 17.51 Frontiere\_ouverte\_vitesse\_imposee\_sortie

Description: Sub-class for velocity boundary condition. The imposed velocity field at the open boundary is vectorial and the imposed velocity values are expressed in m.s-1.

See also: *frontiere\_ouverte\_vitesse\_imposee* ([17.49](#)) *frontiere\_ouverte\_rho\_u\_imposee* ([17.45](#))

Usage:

**frontiere\_ouverte\_vitesse\_imposee\_sortie ch**

where

- **ch** *champ\_front\_base* (21.1): Boundary field type.

## 17.52 Neumann

Description: Neumann condition at the boundary called bord (edge) : 1). For Navier-Stokes equations, constraint imposed at the boundary; 2). For scalar transport equation, flux imposed at the boundary.

See also: *condlim\_base* (17) *frontiere\_ouverte\_pression\_imposee\_orlansky* (17.39) *frontiere\_ouverte\_gradient\_pression\_impose* (17.32) *frontiere\_ouverte\_gradient\_pression\_libre\_vef* (17.34) *frontiere\_ouverte\_gradient\_pression\_libre\_vefprep1b* (17.35) *frontiere\_ouverte\_pression\_imposee* (17.38) *frontiere\_ouverte\_pression\_moyenne\_imposee* (17.40) *frontiere\_ouverte* (17.28) *sortie\_libre\_temperature\_imposee\_h* (17.88)

Usage:

**neumann**

## 17.53 Paroi\_adiabatique

Description: Normal zero flux condition at the wall called bord (edge).

See also: *condlim\_base* (17)

Usage:

**paroi\_adiabatique**

## 17.54 Paroi\_contact

Description: Thermal condition between two domains. Important: the name of the boundaries in the two domains should be the same. (Warning: there is also an old limitation not yet fixed on the sequential algorithm in VDF to detect the matching faces on the two boundaries: faces should be ordered in the same way). The kind of condition depends on the discretization. In VDF, it is a heat exchange condition, and in VEF, a temperature condition.

Such a coupling requires coincident meshes for the moment. In case of non-coincident meshes, run is stopped and two external files are automatically generated in VEF (*connectivity\_failed\_boundary\_name* and *connectivity\_failed\_pb\_name.med*). In 2D, the keyword *Decouper\_bord\_coincident* associated to the *connectivity\_failed\_boundary\_name* file allows to generate a new coincident mesh.

In 3D, for a first preliminary cut domain with HOMARD (fluid for instance), the second problem associated to *pb\_name* (solide in a fluid/solid coupling problem) has to be submitted to HOMARD cutting procedure with *connectivity\_failed\_pb\_name.med*.

Such a procedure works as while the primary refined mesh (fluid in our example) impacts the fluid/solid interface with a compact shape as described below (values 2 or 4 indicates the number of division from primary faces obtained in fluid domain at the interface after HOMARD cutting):

2-2-2-2-2-2

2-4-4-4-4-2 2-2-2

2-4-4-4-4-2 2-4-2

2-2-2-2-2 2-2

OK

2-2 2-2-2

2-4-2 2-2  
2-2 2-2  
NOT OK

See also: `condlim_base` ([17](#))

Usage:

**paroi\_contact autrepb nameb**  
where

- **autrepb** *str*: Name of other problem.
- **nameb** *str*: boundary name of the remote problem which should be the same than the local name

### 17.55 Paroi\_contact\_fictif

Description: This keyword is derivated from `paroi_contact` and is especially dedicated to compute coupled fluid/solid/fluid problem in case of thin material. Thanks to this option, solid is considered as a fictitious media (no mesh, no domain associated), and coupling is performed by considering instantaneous thermal equilibrium in it (for the moment).

See also: `condlim_base` ([17](#))

Usage:

**paroi\_contact\_fictif autrepb nameb conduct\_fictif ep\_fictive**  
where

- **autrepb** *str*: Name of other problem.
- **nameb** *str*: Name of bord.
- **conduct\_fictif** *float*: thermal conductivity
- **ep\_fictive** *float*: thickness of the fictitious media

### 17.56 Paroi\_contact\_rayo

Description: Thermal condition between two domains.

See also: `condlim_base` ([17](#))

Usage:

**paroi\_contact\_rayo autrepb nameb type**  
where

- **autrepb** *str*: Name of other problem.
- **nameb** *str*: boundary name of the remote problem which should be the same than the local name
- **type** *str* into [`'TRANSP'`, `'SEMI_TRANSP'`]

### 17.57 Paroi\_decalee\_robin

Description: This keyword is used to designate a Robin boundary condition ( $a.u + b.du/dn = c$ ) associated with the Pironneau methodology for the wall laws. The value of given by the `delta` option is the distance between the mesh (where symmetry boundary condition is applied) and the fictious wall. This boundary condition needs the definition of the dedicated source terms (`Source_Robin` or `Source_Robin_Scalaire`) according the equations used.

See also: `condlim_base` ([17](#))

Usage:

**paroi\_decalee\_robin** *str*

**Read** *str* {

**delta** *float*

}

where

- **delta** *float*

### 17.58 Paroi\_defilante

Description: Keyword to designate a condition where tangential velocity is imposed on the wall called bord (edge). If the velocity components set by the user is not tangential, projection is used.

See also: `dirichlet` ([17.19](#))

Usage:

**paroi\_defilante** **ch**

where

- **ch** *champ\_front\_base* ([21.1](#)): Boundary field type.

### 17.59 Paroi\_echange\_contact\_correlation\_vdf

Description: Class to define a thermohydraulic 1D model which will apply to a boundary of 2D or 3D domain.

Warning : For parallel calculation, the only possible partition will be according the axis of the model with the keyword `Tranche`.

See also: `condlim_base` ([17](#))

Usage:

**paroi\_echange\_contact\_correlation\_vdf** *str*

**Read** *str* {

    [ **dir** *int*]

    [ **tnf** *float*]

    [ **tsup** *float*]

    [ **lambda** *str*]

    [ **rho** *str*]

    [ **dt\_impr** *float*]

    [ **cp** *float*]

    [ **mu** *str*]

    [ **debit** *float*]

    [ **dh** *float*]

    [ **volume** *str*]

    [ **nu** *str*]

    [ **reprise\_correlation** ]

```
}
where
```

- **dir** *int*: Direction (0 : axis X, 1 : axis Y, 2 : axis Z) of the 1D model.
- **tin** *float*: Inlet fluid temperature of the 1D model (oC or K).
- **tsup** *float*: Outlet fluid temperature of the 1D model (oC or K).
- **lambda** *str*: Thermal conductivity of the fluid (W.m-1.K-1).
- **rho** *str*: Mass density of the fluid (kg.m-3) which may be a function of the temperature T.
- **dt\_impr** *float*: Printing period in name\_of\_data\_file\_time.dat files of the 1D model results.
- **cp** *float*: Calorific capacity value at a constant pressure of the fluid (J.kg-1.K-1).
- **mu** *str*: Dynamic viscosity of the fluid (kg.m-1.s-1) which may be a function of the temperature T.
- **debit** *float*: Surface flow rate (kg.s-1.m-2) of the fluid into the channel.
- **dh** *float*: Hydraulic diameter may be a function f(x) with x position along the 1D axis (xinf <= x <= xsup)
- **volume** *str*: Exact volume of the 1D domain (m3) which may be a function of the hydraulic diameter (Dh) and the lateral surface (S) of the meshed boundary.
- **nu** *str*: Nusselt number which may be a function of the Reynolds number (Re) and the Prandtl number (Pr).
- **reprise\_correlation** : Keyword in the case of a resuming calculation with this correlation.

## 17.60 Paroi\_echange\_contact\_correlation\_vef

Description: Class to define a thermohydraulic 1D model which will apply to a boundary of 2D or 3D domain.

Warning : For parallel calculation, the only possible partition will be according the axis of the model with the keyword Tranche\_geom.

See also: [condlim\\_base \(17\)](#)

Usage:

**paroi\_echange\_contact\_correlation\_vef** *str*

**Read** *str* {

```
[dir int]
[tin float]
[tsup float]
[lambda str]
[rho str]
[dt_impr float]
[cp float]
[mu str]
[debit float]
[n int]
[dh str]
[surface str]
[xinf float]
[xsup float]
[nu str]
[emissivite_pour_rayonnement_entre_deux_plaques_quasi_infinies float]
[reprise_correlation]
```

```
}
where
```

- **dir** *int*: Direction (0 : axis X, 1 : axis Y, 2 : axis Z) of the 1D model.

- **tinfl** *float*: Inlet fluid temperature of the 1D model (oC or K).
- **tsup** *float*: Outlet fluid temperature of the 1D model (oC or K).
- **lambda** *str*: Thermal conductivity of the fluid (W.m-1.K-1).
- **rho** *str*: Mass density of the fluid (kg.m-3) which may be a function of the temperature T.
- **dt Impr** *float*: Printing period in name\_of\_data\_file\_time.dat files of the 1D model results.
- **cp** *float*: Calorific capacity value at a constant pressure of the fluid (J.kg-1.K-1).
- **mu** *str*: Dynamic viscosity of the fluid (kg.m-1.s-1) which may be a function of the temperature T.
- **debit** *float*: Surface flow rate (kg.s-1.m-2) of the fluid into the channel.
- **n** *int*: Number of 1D cells of the 1D mesh.
- **dh** *str*: Hydraulic diameter may be a function f(x) with x position along the 1D axis ( $x_{inf} \leq x \leq x_{sup}$ )
- **surface** *str*: Section surface of the channel which may be function f(Dh,x) of the hydraulic diameter (Dh) and x position along the 1D axis ( $x_{inf} \leq x \leq x_{sup}$ )
- **xinf** *float*: Position of the inlet of the 1D mesh on the axis direction.
- **xsup** *float*: Position of the outlet of the 1D mesh on the axis direction.
- **nu** *str*: Nusselt number which may be a function of the Reynolds number (Re) and the Prandtl number (Pr).
- **emissivite\_pour\_rayonnement\_entre\_deux\_plaques\_quasi\_infinies** *float*: Coefficient of emissivity for radiation between two quasi infinite plates.
- **reprise\_correlation** : Keyword in the case of a resuming calculation with this correlation.

## 17.61 Paroi\_echange\_contact\_odvm\_vdf

Description: not\_set

See also: paroi\_echange\_contact\_vdf ([17.63](#))

Usage:

**paroi\_echange\_contact\_odvm\_vdf autrepb nameb temp h**

where

- **autrepb** *str*: Name of other problem.
- **nameb** *str*: Name of bord.
- **temp** *str*: Name of field.
- **h** *float*: Value assigned to a coefficient (expressed in W.K-1m-2) that characterises the contact between the two mediums. In order to model perfect contact, h must be taken to be infinite. This value must obviously be the same in both the two problems blocks.  
The surface thermal flux exchanged between the two mediums is represented by :  
$$q = h (T_1 - T_2) \text{ where } 1/h = d_1/\lambda_{a1} + 1/\lambda_{h\_contact} + d_2/\lambda_{a2}$$
  
where di : distance between the node where Ti and the wall is found.

## 17.62 Paroi\_echange\_contact\_rayo\_semi\_transp\_vdf

Description: Exchange boundary condition in VDF between the semi transparent fluid and the solid for a problem coupled with radiation.

See also: paroi\_echange\_contact\_vdf ([17.63](#))

Usage:

**paroi\_echange\_contact\_rayo\_semi\_transp\_vdf autrepb nameb temp h**

where

- **autrepb** *str*: Name of other problem.

- **nameb** *str*: Name of bord.
- **temp** *str*: Name of field.
- **h** *float*: Value assigned to a coefficient (expressed in W.K-1m-2) that characterises the contact between the two mediums. In order to model perfect contact, h must be taken to be infinite. This value must obviously be the same in both the two problems blocks.  
The surface thermal flux exchanged between the two mediums is represented by :  
$$f_i = h (T_1 - T_2) \text{ where } 1/h = d_1/\lambda_{d1} + 1/\text{val\_h\_contact} + d_2/\lambda_{d2}$$
where di : distance between the node where Ti and the wall is found.

### 17.63 Paroi\_echange\_contact\_vdf

Description: Boundary condition type to model the heat flux between two problems. Important: the name of the boundaries in the two problems should be the same.

See also: [condlim\\_base \(17\)](#) [paroi\\_echange\\_contact\\_odvm\\_vdf \(17.61\)](#) [paroi\\_echange\\_contact\\_vdf\\_ft \(17.64\)](#) [echange\\_contact\\_rayo\\_transp\\_vdf \(17.20\)](#) [paroi\\_echange\\_contact\\_rayo\\_semi\\_transp\\_vdf \(17.62\)](#)

Usage:

**paroi\_echange\_contact\_vdf** **autrepb** **nameb** **temp** **h**  
where

- **autrepb** *str*: Name of other problem.
- **nameb** *str*: Name of bord.
- **temp** *str*: Name of field.
- **h** *float*: Value assigned to a coefficient (expressed in W.K-1m-2) that characterises the contact between the two mediums. In order to model perfect contact, h must be taken to be infinite. This value must obviously be the same in both the two problems blocks.  
The surface thermal flux exchanged between the two mediums is represented by :  
$$f_i = h (T_1 - T_2) \text{ where } 1/h = d_1/\lambda_{d1} + 1/\text{val\_h\_contact} + d_2/\lambda_{d2}$$
where di : distance between the node where Ti and the wall is found.

### 17.64 Paroi\_echange\_contact\_vdf\_ft

Description: This boundary condition is used between a conduction problem and a thermohydraulic problem with two phases flow (Front-Tracking method) to modelize heat exchange.

See also: [paroi\\_echange\\_contact\\_vdf \(17.63\)](#)

Usage:

**paroi\_echange\_contact\_vdf\_ft** **autrepb** **nameb** **temp** **h**  
where

- **autrepb** *str*: Name of other problem.
- **nameb** *str*: Name of bord.
- **temp** *str*: Name of field.
- **h** *float*: Value assigned to a coefficient (expressed in W.K-1m-2) that characterises the contact between the two mediums. In order to model perfect contact, h must be taken to be infinite. This value must obviously be the same in both the two problems blocks.  
The surface thermal flux exchanged between the two mediums is represented by :  
$$f_i = h (T_1 - T_2) \text{ where } 1/h = d_1/\lambda_{d1} + 1/\text{val\_h\_contact} + d_2/\lambda_{d2}$$
where di : distance between the node where Ti and the wall is found.

### 17.65 Paroi\_echange\_externe\_impose

Description: External type exchange condition with a heat exchange coefficient and an imposed external temperature.

See also: [condlim\\_base \(17\)](#) [paroi\\_echange\\_externe\\_impose\\_h \(17.66\)](#) [paroi\\_echange\\_externe\\_impose\\_rayo\\_transp \(17.68\)](#) [paroi\\_echange\\_externe\\_impose\\_rayo\\_semi\\_transp \(17.67\)](#)

Usage:

**paroi\_echange\_externe\_impose h\_or\_t himpc t\_or\_h ch**

where

- **h\_or\_t** *str into* ['h\_imp', 't\_ext']: Heat exchange coefficient value (expressed in W.m-2.K-1).
- **himpc** *champ\_front\_base* (21.1): Boundary field type.
- **t\_or\_h** *str into* ['t\_ext', 'h\_imp']: External temperature value (expressed in oC or K).
- **ch** *champ\_front\_base* (21.1): Boundary field type.

### 17.66 Paroi\_echange\_externe\_impose\_h

Description: Particular case of class `paroi_echange_externe_impose` for enthalpy equation.

See also: [paroi\\_echange\\_externe\\_impose \(17.65\)](#)

Usage:

**paroi\_echange\_externe\_impose\_h h\_or\_t himpc t\_or\_h ch**

where

- **h\_or\_t** *str into* ['h\_imp', 't\_ext']: Heat exchange coefficient value (expressed in W.m-2.K-1).
- **himpc** *champ\_front\_base* (21.1): Boundary field type.
- **t\_or\_h** *str into* ['t\_ext', 'h\_imp']: External temperature value (expressed in oC or K).
- **ch** *champ\_front\_base* (21.1): Boundary field type.

### 17.67 Paroi\_echange\_externe\_impose\_rayo\_semi\_transp

Description: External type exchange condition for a coupled problem with radiation in semi transparent gas.

See also: [paroi\\_echange\\_externe\\_impose \(17.65\)](#)

Usage:

**paroi\_echange\_externe\_impose\_rayo\_semi\_transp h\_or\_t himpc t\_or\_h ch**

where

- **h\_or\_t** *str into* ['h\_imp', 't\_ext']: Heat exchange coefficient value (expressed in W.m-2.K-1).
- **himpc** *champ\_front\_base* (21.1): Boundary field type.
- **t\_or\_h** *str into* ['t\_ext', 'h\_imp']: External temperature value (expressed in oC or K).
- **ch** *champ\_front\_base* (21.1): Boundary field type.

### 17.68 Paroi\_echange\_externe\_impose\_rayo\_transp

Description: External type exchange condition for a coupled problem with radiation in transparent gas.

See also: [paroi\\_echange\\_externe\\_impose \(17.65\)](#)



Usage:

**paroi\_echange\_externe\_impose\_rayo\_transp h\_or\_t himpc t\_or\_h ch**

where

- **h\_or\_t** *str* into ['h\_imp', 't\_ext']: Heat exchange coefficient value (expressed in W.m-2.K-1).
- **himpc** *champ\_front\_base* (21.1): Boundary field type.
- **t\_or\_h** *str* into ['t\_ext', 'h\_imp']: External temperature value (expressed in oC or K).
- **ch** *champ\_front\_base* (21.1): Boundary field type.

## 17.69 Paroi\_echange\_global\_impose

Description: Global type exchange condition (internal) that is to say that diffusion on the first fluid mesh is not taken into consideration.

See also: [condlim\\_base \(17\)](#) [Echange\\_couplage\\_thermique \(17.5\)](#)

Usage:

**paroi\_echange\_global\_impose h\_imp himpc text ch**

where

- **h\_imp** *str*: Global exchange coefficient value. The global exchange coefficient value is expressed in W.m-2.K-1.
- **himpc** *champ\_front\_base* (21.1): Boundary field type.
- **text** *str*: External temperature value. The external temperature value is expressed in oC or K.
- **ch** *champ\_front\_base* (21.1): Boundary field type.

## 17.70 Paroi\_fixe

Description: Keyword to designate a situation of adherence to the wall called bord (edge) (normal and tangential velocity at the edge is zero).

See also: [condlim\\_base \(17\)](#) [paroi\\_fixe\\_iso\\_Genepi2\\_sans\\_contribution\\_aux\\_vitesses\\_sommets \(17.71\)](#)

Usage:

**paroi\_fixe**

## 17.71 Paroi\_fixe\_iso\_genepi2\_sans\_contribution\_aux\_vitesses\_sommets

Description: Boundary condition to obtain iso Genepi2, without interest

See also: [paroi\\_fixe \(17.70\)](#)

Usage:

**paroi\_fixe\_iso\_Genepi2\_sans\_contribution\_aux\_vitesses\_sommets**

## 17.72 Paroi\_flux\_impose

Description: Normal flux condition at the wall called bord (edge). The surface area of the flux (W.m-1 in 2D or W.m-2 in 3D) is imposed at the boundary according to the following convention: a positive flux is a flux that enters into the domain according to convention.

See also: `condlim_base` (17) `paroi_flux_impose_rayo_transp` (17.75) `paroi_flux_impose_rayo_semi_transp_vdf` (17.73) `paroi_flux_impose_rayo_semi_transp_vef` (17.74)

Usage:

**paroi\_flux\_impose ch**

where

- **ch** `champ_front_base` (21.1): Boundary field type.

### 17.73 Paroi\_flux\_impose\_rayo\_semi\_transp\_vdf

Description: Normal flux condition at the wall called bord (edge) for a radiation problem in semi transparent gas (in VDF).

See also: `paroi_flux_impose` (17.72)

Usage:

**paroi\_flux\_impose\_rayo\_semi\_transp\_vdf ch**

where

- **ch** `champ_front_base` (21.1): Boundary field type.

### 17.74 Paroi\_flux\_impose\_rayo\_semi\_transp\_vef

Description: Normal flux condition at the wall called bord (edge) for a radiation problem in semi transparent gas (in VEF).

See also: `paroi_flux_impose` (17.72)

Usage:

**paroi\_flux\_impose\_rayo\_semi\_transp\_vef ch**

where

- **ch** `champ_front_base` (21.1): Boundary field type.

### 17.75 Paroi\_flux\_impose\_rayo\_transp

Description: Normal flux condition at the wall called bord (edge) for a radiation problem in transparent gas.

See also: `paroi_flux_impose` (17.72)

Usage:

**paroi\_flux\_impose\_rayo\_transp ch**

where

- **ch** `champ_front_base` (21.1): Boundary field type.

## 17.76 Paroi\_ft\_disc

Description: Boundary condition for Front-Tracking problem in the discontinuous version.

See also: `condlim_base` (17)

Usage:

**paroi\_ft\_disc type**

where

- **type** `paroi_ft_disc_deriv` (17.77): Symetrie condition.

## 17.77 Paroi\_ft\_disc\_deriv

Description: `not_set`

See also: `objet_lecture` (47) `symetrie` (17.77.1) `constant` (17.77.2)

Usage:

**paroi\_ft\_disc\_deriv**

### 17.77.1 Symetrie

Description: Symetrie condition in the case of two-phase flows

See also: `paroi_ft_disc_deriv` (17.77)

Usage:

**symetrie**

### 17.77.2 Constant

Description: condition contact angle `fidex`. The angle is measured between the wall and the interface in the phase 0.

See also: `paroi_ft_disc_deriv` (17.77)

Usage:

**constant ch**

where

- **ch** `champ_front_base` (21.1): Boundary field type.

## 17.78 Paroi\_knudsen\_non\_negligeable

Description: Boundary condition for number of Knudsen ( $Kn$ ) above 0.001 where slip-flow condition appears: the velocity near the wall depends on the shear stress :  $Kn=l/L$  with  $l$  is the mean-free-path of the molecules and  $L$  a characteristic length scale.

$U(y=0)-U_{wall}=k(dU/dY)$

Where  $k$  is a coefficient given by several laws:

Maxwell :  $k=(2-s)*l/s$

Bestok&Karniadakis :  $k=(2-s)/s*L*Kn/(1+Kn)$

Xue&Fan :  $k=(2-s)/s*L*tanh(Kn)$

s is a value between 0 and 2 named accomodation coefficient. s=1 seems a good value.

Warning : The keyword is available for VDF calculation only for the moment.

See also: [dirichlet \(17.19\)](#)

Usage:

**paroi\_knudsen\_non\_negligeable** **name\_champ\_1** **champ\_1** **name\_champ\_2** **champ\_2**

where

- **name\_champ\_1** *str* into ['vitesse\_paro', 'k']: Field name.
- **champ\_1** *champ\_front\_base* (21.1): Boundary field type.
- **name\_champ\_2** *str* into ['vitesse\_paro', 'k']: Field name.
- **champ\_2** *champ\_front\_base* (21.1): Boundary field type.

## 17.79 Paroi\_rugueuse

Description: Rough wall boundary

See also: [dirichlet \(17.19\)](#)

Usage:

**paroi\_rugueuse** *str*

**Read** *str* {

**erugu** *float*

}

where

- **erugu** *float*: Constant value for roughness

## 17.80 Paroi\_temperature\_imposee

Description: Imposed temperature condition at the wall called bord (edge).

See also: [dirichlet \(17.19\)](#) [enthalpie\\_imposee\\_paro \(17.90\)](#) [paroi\\_temperature\\_imposee\\_rayo\\_transp \(17.82\)](#)  
[paroi\\_temperature\\_imposee\\_rayo\\_semi\\_transp \(17.81\)](#)

Usage:

**paroi\_temperature\_imposee** **ch**

where

- **ch** *champ\_front\_base* (21.1): Boundary field type.

## 17.81 Paroi\_temperature\_imposee\_rayo\_semi\_transp

Description: Imposed temperature condition at the wall called bord (edge) for a radiation problem in semi transparent gas.

See also: [paroi\\_temperature\\_imposee \(17.80\)](#)

Usage:

**paroi\_temperature\_imposee\_rayo\_semi\_transp** **ch**

where

- **ch** *champ\_front\_base* (21.1): Boundary field type.

## 17.82 Paroi\_temperature\_imposee\_rayo\_transp

Description: Imposed temperature condition at the wall called bord (edge) for a radiation problem in transparent gas.

See also: `paroi_temperature_imposee` ([17.80](#))

Usage:

**paroi\_temperature\_imposee\_rayo\_transp** **ch**

where

- **ch** *champ\_front\_base* ([21.1](#)): Boundary field type.

## 17.83 Perio

Description: `not_set`

See also: `condlim_base` ([17](#))

Usage:

**perio**

## 17.84 Periodique

Description: 1). For Navier-Stokes equations, this keyword is used to indicate that the horizontal inlet velocity values are the same as the outlet velocity values, at every moment. As regards meshing, the inlet and outlet edges bear the same name.; 2). For scalar transport equation, this keyword is used to set a periodic condition on scalar. The two edges dealing with this periodic condition bear the same name.

See also: `condlim_base` ([17](#))

Usage:

**periodique**

## 17.85 Robin\_vef

Description: Robin condition at the boundary (edge)

See also: `condlim_base` ([17](#))

Usage:

**robin\_vef** *str*

**Read** *str* {

**alpha** *float*

**beta** *float*

**champ\_front\_normal\_et\_tangentiel\_robin** *champ\_front\_base*

}

where

- **alpha** *float*: Robin coefficient for the normal field
- **beta** *float*: Robin coefficient for the tangent field
- **champ\_front\_normal\_et\_tangentiel\_robin** *champ\_front\_base* ([21.1](#)): The boundary field

### 17.86 **Scalaire\_impose\_paro**

Description: Imposed temperature condition at the wall called bord (edge).

See also: [dirichlet \(17.19\)](#)

Usage:

**scalaire\_impose\_paro ch**

where

- **ch** *champ\_front\_base* ([21.1](#)): Boundary field type.

### 17.87 **Sortie\_libre\_rho\_variable**

Description: Class to define an outlet boundary condition at which the pressure is defined through the given field, whereas the density of the two-phase flow may varies (value of P/rho given in Pa/kg.m-3).

See also: [condlim\\_base \(17\)](#)

Usage:

**sortie\_libre\_rho\_variable ch**

where

- **ch** *champ\_front\_base* ([21.1](#)): Boundary field type.

### 17.88 **Sortie\_libre\_temperature\_imposee\_h**

Description: Open boundary for heat equation with enthalpy as unknown.

See also: [neumann \(17.52\)](#)

Usage:

**sortie\_libre\_temperature\_imposee\_h ch**

where

- **ch** *champ\_front\_base* ([21.1](#)): Boundary field type.

### 17.89 **Symetrie**

Description: 1). For Navier-Stokes equations, this keyword is used to designate a symmetry condition concerning the velocity at the boundary called bord (edge) (normal velocity at the edge equal to zero and tangential velocity gradient at the edge equal to zero); 2). For scalar transport equation, this keyword is used to set a symmetry condition on scalar on the boundary named bord (edge).

See also: [condlim\\_base \(17\)](#)

Usage:

**symetrie**

## 17.90 Enthalpie\_imposee\_paro

Synonymous: **temperature\_imposee\_paro**

Description: Imposed temperature condition at the wall called bord (edge).

See also: `paroi_temperature_imposee` ([17.80](#))

Usage:

**enthalpie\_imposee\_paro** **ch**

where

- **ch** *champ\_front\_base* ([21.1](#)): Boundary field type.

## 18 discretisation\_base

Description: Basic class for space discretization of thermohydraulic turbulent problems.

See also: `objet_u` ([48](#)) `DG` ([18.1](#)) `vdf` ([18.8](#)) `polymac_p0` ([18.7](#)) `polymac` ([18.5](#)) `polymac_P0P1NC` ([18.6](#)) `ijk` ([18.4](#)) `vef` ([18.9](#)) `ef` ([18.3](#)) `EF_axi` ([18.2](#))

Usage:

### 18.1 Dg

Description: DG discretization

See also: `discretisation_base` ([18](#))

Usage:

### 18.2 Ef\_axi

Description: Element Finite discretization.

See also: `discretisation_base` ([18](#))

Usage:

### 18.3 Ef

Description: Element Finite discretization.

See also: `discretisation_base` ([18](#))

Usage:

### 18.4 Ijk

Description: IJK discretization.

See also: `discretisation_base` ([18](#))

Usage:

## 18.5 Polymac

Description: polymac discretization (polymac discretization that is not compatible with pb\_multi).

See also: discretisation\_base (18)

Usage:

## 18.6 Polymac\_p0p1nc

Description: polymac\_P0P1NC discretization (previously polymac discretization compatible with pb\_multi).

See also: discretisation\_base (18)

Usage:

## 18.7 Polymac\_p0

Description: polymac\_p0 discretization (previously covimac discretization compatible with pb\_multi).

See also: discretisation\_base (18)

Usage:

## 18.8 Vdf

Description: Finite difference volume discretization.

See also: discretisation\_base (18)

Usage:

## 18.9 Vef

Synonymous: **vefprep1b**

Description: Finite element volume discretization (P1NC/P1-bubble element). Since the 1.5.5 version, several new discretizations are available thanks to the optional keyword Read. By default, the VEFPreP1B keyword is equivalent to the former VEFPreP1B formulation (v1.5.4 and sooner). P0P1 (if used with the strong formulation for imposed pressure boundary) is equivalent to VEFPreP1B but the convergence is slower. VEFPreP1B dis is equivalent to VEFPreP1B dis Read dis { P0 P1 Changement\_de\_base\_P1Bulle 1 Cl\_pression\_sommet\_faible 0 }

See also: discretisation\_base (18)

Usage:

**vef** *str*

**Read** *str* {

```
[changement_de_base_p1bulle int into [0, 1]]
[p0]
[p1]
[pa]
[rt]
[modif_div_face_dirichlet int into [0, 1]]
```



```
[cl_pression_sommet_faible int into [0, 1]]
}
```

where

- **changement\_de\_base\_p1bulle** *int into [0, 1]*: `changement_de_base_p1bulle` 1 This option may be used to have the P1NC/P0P1 formulation (value set to 0) or the P1NC/P1Bulle formulation (value set to 1, the default).
- **p0** : Pressure nodes are added on element centres
- **p1** : Pressure nodes are added on vertices
- **pa** : Only available in 3D, pressure nodes are added on bones
- **rt** : For P1NCP1B (in TrioCFD)
- **modif\_div\_face\_dirichlet** *int into [0, 1]*: This option (by default 0) is used to extend control volumes for the momentum equation.
- **cl\_pression\_sommet\_faible** *int into [0, 1]*: This option is used to specify a strong formulation (value set to 0, the default) or a weak formulation (value set to 1) for an imposed pressure boundary condition. The first formulation converges quicker and is stable in general cases. The second formulation should be used if there are several outlet boundaries with Neumann condition (see `Ecoulement_Neumann` test case for example).

## 19 domaine

Description: Keyword to create a domain.

See also: `objet_u` (48) `DomaineAxis1d` (19.1) `IJK_Grid_Geometry` (19.2) `domaine_ale` (19.3)

Usage:

### 19.1 Domaineaxis1d

Description: 1D domain

See also: `domaine` (19)

Usage:

### 19.2 Ijk\_grid\_geometry

Description: Object to define the grid that will represent the domain of the simulation in IJK discretization

See also: `domaine` (19)

Usage:

**IJK\_Grid\_Geometry** *str*

**Read** *str* {

```
[perio_i]
[perio_j]
[perio_k]
[nbelem_i int]
[nbelem_j int]
[nbelem_k int]
[uniform_domain_size_i float]
[uniform_domain_size_j float]
```

```

 [uniform_domain_size_k float]
 [origin_i float]
 [origin_j float]
 [origin_k float]
}
where

```

- **perio\_i** : rien to specify the border along the I direction is periodic
- **perio\_j** : rien to specify the border along the J direction is periodic
- **perio\_k** : rien to specify the border along the K direction is periodic
- **nbelem\_i** *int*: the number of elements of the grid in the I direction
- **nbelem\_j** *int*: the number of elements of the grid in the J direction
- **nbelem\_k** *int*: the number of elements of the grid in the K direction
- **uniform\_domain\_size\_i** *float*: the size of the elements along the I direction
- **uniform\_domain\_size\_j** *float*: the size of the elements along the J direction
- **uniform\_domain\_size\_k** *float*: the size of the elements along the K direction
- **origin\_i** *float*: I-coordinate of the origin of the grid
- **origin\_j** *float*: J-coordinate of the origin of the grid
- **origin\_k** *float*: K-coordinate of the origin of the grid

### 19.3 Domaine\_ale

Description: Domain with nodes at the interior of the domain which are displaced in an arbitrarily prescribed way thanks to ALE (Arbitrary Lagrangian-Eulerian) description.

Keyword to specify that the domain is mobile following the displacement of some of its boundaries.

See also: [domaine \(19\)](#)

Usage:

## 20 champ\_base

### 20.1 Champ\_base

Description: Basic class of fields.

See also: [objet\\_u \(48\)](#) [champ\\_don\\_base \(20.9\)](#) [champ\\_input\\_base \(20.21\)](#) [champ\\_fonc\\_med \(20.14\)](#) [champ\\_ostwald \(20.25\)](#) [field\\_uniform\\_keps\\_from\\_ud \(20.34\)](#)

Usage:

### 20.2 Champ\_fonc\_interp

Description: Field that is interpolated from a distant domain via MEDCoupling (remapper).

See also: [champ\\_don\\_base \(20.9\)](#)

Usage:

**Champ\_Fonc\_Interp** *str*

**Read** *str* {

**nom\_champ** *str*

**pb\_loc** *str*

```

 pb_dist str
 [dom_loc str]
 [dom_dist str]
 [default_value str]
 nature str
 [use_overlapdec str]
}
where

```

- **nom\_champ** *str*: Name of the field (for example: temperature).
- **pb\_loc** *str*: Name of the local problem.
- **pb\_dist** *str*: Name of the distant problem.
- **dom\_loc** *str*: Name of the local domain.
- **dom\_dist** *str*: Name of the distant domain.
- **default\_value** *str*: Name of the distant domain.
- **nature** *str*: Nature of the field (knowledge from MEDCoupling is required; IntensiveMaximum, IntensiveConservation, ...).
- **use\_overlapdec** *str*: Nature of the field (knowledge from MEDCoupling is required; IntensiveMaximum, IntensiveConservation, ...).

### 20.3 Champ\_fonc\_med\_table\_temps

Description: Field defined as a fixed spatial shape scaled by a temporal coefficient

See also: champ\_fonc\_med ([20.14](#))

Usage:

```

Champ_Fonc_MED_Table_Temps str
Read str {

```

```

 [table_temps bloc_lecture]
 [table_temps_lue str]
 [use_existing_domain]
 [last_time]
 [decoup str]
 [mesh str]
 domain str
 file str
 field str
 [loc str into ['som', 'elem']]
 [time float]

```

```

}
where

```

- **table\_temps** *bloc\_lecture* ([3.2](#)): Table containing the temporal coefficient used to scale the field
- **table\_temps\_lue** *str*: Name of the file containing the values of the temporal coefficient used to scale the field
- **use\_existing\_domain** for inheritance: whether to optimize the field loading by indicating that the field is supported by the same mesh that was initially loaded as the domain
- **last\_time** for inheritance: to use the last time of the MED file instead of the specified time. Mutually exclusive with 'time' parameter.
- **decoup** *str* for inheritance: specify a partition file.

- **mesh** *str* for inheritance: Name of the mesh supporting the field. This is the name of the mesh in the MED file, and if this mesh was also used to create the TRUST domain, loading can be optimized with option 'use\_existing\_domain'.
- **domain** *str* for inheritance: Name of the domain supporting the field. This is the name of the mesh in the MED file, and if this mesh was also used to create the TRUST domain, loading can be optimized with option 'use\_existing\_domain'.
- **file** *str* for inheritance: Name of the .med file.
- **field** *str* for inheritance: Name of field to load.
- **loc** *str* into ['som', 'elem'] for inheritance: To indicate where the field is localised. Default to 'elem'.
- **time** *float* for inheritance: Timestep to load from the MED file. Mutually exclusive with 'last\_time' flag.

## 20.4 Champ\_fonc\_med\_tabule

Description: not\_set

See also: champ\_fonc\_med ([20.14](#))

Usage:

**Champ\_Fonc\_MED\_Tabule** *str*

**Read** *str* {

```

 [use_existing_domain]
 [last_time]
 [decoup str]
 [mesh str]
 domain str
 file str
 field str
 [loc str into ['som', 'elem']]
 [time float]

```

}

where

- **use\_existing\_domain** for inheritance: whether to optimize the field loading by indicating that the field is supported by the same mesh that was initially loaded as the domain
- **last\_time** for inheritance: to use the last time of the MED file instead of the specified time. Mutually exclusive with 'time' parameter.
- **decoup** *str* for inheritance: specify a partition file.
- **mesh** *str* for inheritance: Name of the mesh supporting the field. This is the name of the mesh in the MED file, and if this mesh was also used to create the TRUST domain, loading can be optimized with option 'use\_existing\_domain'.
- **domain** *str* for inheritance: Name of the domain supporting the field. This is the name of the mesh in the MED file, and if this mesh was also used to create the TRUST domain, loading can be optimized with option 'use\_existing\_domain'.
- **file** *str* for inheritance: Name of the .med file.
- **field** *str* for inheritance: Name of field to load.
- **loc** *str* into ['som', 'elem'] for inheritance: To indicate where the field is localised. Default to 'elem'.
- **time** *float* for inheritance: Timestep to load from the MED file. Mutually exclusive with 'last\_time' flag.

## 20.5 Champ\_tabule\_morceaux

Description: Field defined by tabulated data in each sub-domaine. It makes possible the definition of a field which is a function of other fields.

See also: `champ_don_base` (20.9) `Champ_Fonc_Tabule_Morceaux_Interp` (20.6)

Usage:

**Champ\_Tabule\_Morceaux** *domain\_name* *nb\_comp* *data*

where

- **domain\_name** *str*: Name of the domain.
- **nb\_comp** *int*: Number of field components.
- **data** *bloc\_lecture* (3.2): { Default *val\_def* *sous\_domaine\_1* *val\_1* ... *sous\_domaine\_i* *val\_i* } By default, the value *val\_def* is assigned to the field. It takes the *sous\_domaine\_i* identifier `Sous_Domaine` (*sub\_area*) type object function, *val\_i*. `Sous_Domaine` (*sub\_area*) type objects must have been previously defined if the operator wishes to use a `champ_fonc_tabule_morceaux` type object.

## 20.6 Champ\_fonc\_tabule\_morceaux\_interp

Description: Field defined by tabulated data in each sub-domaine. It makes possible the definition of a field which is a function of other fields. Here we use MEDCoupling to interpolate fields between the two domains.

See also: `Champ_Tabule_Morceaux` (20.5)

Usage:

**Champ\_Fonc\_Tabule\_Morceaux\_Interp** *problem\_name* *nb\_comp* *data*

where

- **problem\_name** *str*: Name of the problem.
- **nb\_comp** *int*: Number of field components.
- **data** *bloc\_lecture* (3.2): { Default *val\_def* *sous\_domaine\_1* *val\_1* ... *sous\_domaine\_i* *val\_i* } By default, the value *val\_def* is assigned to the field. It takes the *sous\_domaine\_i* identifier `Sous_Domaine` (*sub\_area*) type object function, *val\_i*. `Sous_Domaine` (*sub\_area*) type objects must have been previously defined if the operator wishes to use a `champ_fonc_tabule_morceaux` type object.

## 20.7 Champ\_parametrique

Description: Parametric field

See also: `champ_don_base` (20.9)

Usage:

**Champ\_Parametrique** *str*

**Read** *str* {

**fichier** *str*

}

where

- **fichier** *str*: Filename where fields are read

## 20.8 Champ\_composite

Description: Composite field. Used in multiphase problems to associate data to each phase.

See also: [champ\\_don\\_base \(20.9\)](#) [champ\\_musig \(20.24\)](#)

Usage:

**champ\_composite dim bloc**

where

- **dim** *int*: Number of field components.
- **bloc** *bloc\_lecture (3.2)*: Values Various pieces of the field, defined per phase. Part 1 goes to phase 1, etc...

## 20.9 Champ\_don\_base

Description: Basic class for data fields (not calculated), p.e. physics properties.

See also: [champ\\_base \(20.1\)](#) [champ\\_som\\_lu\\_vdf \(20.26\)](#) [Champ\\_Parametrique \(20.7\)](#) [champ\\_fonc\\_tabule \(20.18\)](#) [champ\\_tabule\\_temps \(20.29\)](#) [champ\\_uniforme\\_morceaux \(20.30\)](#) [champ\\_fonc\\_txyz \(20.32\)](#) [init\\_par\\_partie \(20.35\)](#) [uniform\\_field \(20.37\)](#) [champ\\_composite \(20.8\)](#) [tayl\\_green \(20.36\)](#) [champ\\_fonc\\_t \(20.17\)](#) [Champ\\_Tabule\\_Morceaux \(20.5\)](#) [champ\\_fonc\\_xyz \(20.33\)](#) [champ\\_init\\_canal\\_sinal \(20.19\)](#) [champ\\_fonc\\_fonction\\_txyz\\_morceaux \(20.13\)](#) [champ\\_don\\_lu \(20.10\)](#) [Champ\\_Fonc\\_Interp \(20.2\)](#) [champ\\_fonc\\_reprise \(20.15\)](#) [champ\\_som\\_lu\\_vef \(20.27\)](#)

Usage:

## 20.10 Champ\_don\_lu

Description: Field to read a data field (values located at the center of the cells) in a file.

See also: [champ\\_don\\_base \(20.9\)](#)

Usage:

**champ\_don\_lu dom nb\_comp file**

where

- **dom** *str*: Name of the domain.
- **nb\_comp** *int*: Number of field components.
- **file** *str*: Name of the file.  
This file has the following format:  
nb\_val\_lues -> Number of values readen in th file  
Xi Yi Zi -> Coordinates readen in the file  
Ui Vi Wi -> Value of the field

## 20.11 Champ\_fonc\_fonction

Description: Field that is a function of another field.

See also: [champ\\_fonc\\_tabule \(20.18\)](#) [champ\\_fonc\\_fonction\\_txyz \(20.12\)](#)

Usage:

**champ\_fonc\_fonction problem\_name inco expression**

where

- **problem\_name** *str*: Name of problem.
- **inco** *str*: Name of the field (for example: temperature).
- **expression** *n word1 word2 ... wordn*: Number of field components followed by the analytical expression for each field component.

## 20.12 Champ\_fonc\_fonction\_txyz

Description: this refers to a field that is a function of another field and time and/or space coordinates

See also: `champ_fonc_fonction` ([20.11](#))

Usage:

**champ\_fonc\_fonction\_txyz problem\_name inco expression**

where

- **problem\_name** *str*: Name of problem.
- **inco** *str*: Name of the field (for example: temperature).
- **expression** *n word1 word2 ... wordn*: Number of field components followed by the analytical expression for each field component.

## 20.13 Champ\_fonc\_fonction\_txyz\_morceaux

Description: Field defined by analytical functions in each sub-domaine. On each zone, the value is defined as a function of x,y,z,t and of scalar value taken from a parameter field. This values is associated to the variable 'val' in the expression.

See also: `champ_don_base` ([20.9](#))

Usage:

**champ\_fonc\_fonction\_txyz\_morceaux problem\_name inco nb\_comp data**

where

- **problem\_name** *str*: Name of the problem.
- **inco** *str*: Name of the field (for example: temperature).
- **nb\_comp** *int*: Number of field components.
- **data** *bloc\_lecture* ([3.2](#)): { Defaut val\_def sous\_domaine\_1 val\_1 ... sous\_domaine\_i val\_i } By default, the value val\_def is assigned to the field. It takes the sous\_domaine\_i identifier Sous\_Domaine (sub\_area) type object function, val\_i. Sous\_Domaine (sub\_area) type objects must have been previously defined if the operator wishes to use a `champ_fonc_fonction_txyz_morceaux` type object.

## 20.14 Champ\_fonc\_med

Description: Field to read a data field in a MED-format file .med at a specified time. It is very useful, for example, to resume a calculation with a new or refined geometry. The field post-processed on the new geometry at med format is used as initial condition for the resume.

See also: `champ_base` ([20.1](#)) `Champ_Fonc_MED_Table_Temps` ([20.3](#)) `Champ_Fonc_MED_Tabule` ([20.4](#))

Usage:

**champ\_fonc\_med str**

**Read str {**

**[ use\_existing\_domain ]**

```

[last_time]
[decoup str]
[mesh str]
domain str
file str
field str
[loc str into ['som', 'elem']]
[time float]
}
where

```

- **use\_existing\_domain** : whether to optimize the field loading by indicating that the field is supported by the same mesh that was initially loaded as the domain
- **last\_time** : to use the last time of the MED file instead of the specified time. Mutually exclusive with 'time' parameter.
- **decoup** *str*: specify a partition file.
- **mesh** *str*: Name of the mesh supporting the field. This is the name of the mesh in the MED file, and if this mesh was also used to create the TRUST domain, loading can be optimized with option 'use\_existing\_domain'.
- **domain** *str*: Name of the domain supporting the field. This is the name of the mesh in the MED file, and if this mesh was also used to create the TRUST domain, loading can be optimized with option 'use\_existing\_domain'.
- **file** *str*: Name of the .med file.
- **field** *str*: Name of field to load.
- **loc** *str* into [*'som'*, *'elem'*]: To indicate where the field is localised. Default to 'elem'.
- **time** *float*: Timestep to load from the MED file. Mutually exclusive with 'last\_time' flag.

## 20.15 Champ\_fonc\_reprise

Description: This field is used to read a data field in a save file (.xyz or .sauv) at a specified time. It is very useful, for example, to run a thermohydraulic calculation with velocity initial condition read into a save file from a previous hydraulic calculation.

See also: champ\_don\_base (20.9)

Usage:

**champ\_fonc\_reprise** [ **format** ] **filename** **pb\_name** **champ** [ **fonction** ] **temps**  
 where

- **format** *str* into [*'binaire'*, *'formatte'*, *'xyz'*, *'single\_hdf'*, *'pdi'*]: Type of file (the file format). If xyz format is activated, the .xyz file from the previous calculation will be given for filename, and if formatte or binaire is choosen, the .sauv file of the previous calculation will be specified for filename. In the case of a parallel calculation, if the mesh partition does not changed between the previous calculation and the next one, the binaire format should be preferred, because is faster than the xyz format. If pdi is used, the same constraints/advantages as binaire apply, but it produces one (HDF5) file per node on the filesystem instead of having one file per processor. The single\_hdf format is still supported but is obsolete, the PDI format is recommended.
- **filename** *str*: Name of the save file.
- **pb\_name** *str*: Name of the problem.
- **champ** *str*: Name of the problem unknown. It may also be the temporal average of a problem unknown (like moyenne\_vitesse, moyenne\_temperature,...)
- **fonction** *fonction\_champ\_reprise* (20.16): Optional keyword to apply a function on the field being read in the save file (e.g. to read a temperature field in Celsius units and convert it for the calculation on Kelvin units, you will use: fonction 1 273.+val )



- **temps** *str*: Time of the saved field in the save file or last\_time. If you give the keyword last\_time instead, the last time saved in the save file will be used.

## 20.16 Fonction\_champ\_reprise

Description: not\_set

See also: objet\_lecture ([47](#))

Usage:

**mot fonction**

where

- **mot** *str* into ['fonction']
- **fonction** *n word1 word2 ... wordn*: n f1(val) f2(val) ... fn(val)] time

## 20.17 Champ\_fonc\_t

Description: Field that is constant in space and is a function of time.

See also: champ\_don\_base ([20.9](#))

Usage:

**champ\_fonc\_t val**

where

- **val** *n word1 word2 ... wordn*: Values of field components (time dependant functions).

## 20.18 Champ\_fonc\_tabule

Description: Field that is tabulated as a function of another field.

See also: champ\_don\_base ([20.9](#)) champ\_fonc\_fonction ([20.11](#))

Usage:

**champ\_fonc\_tabule pb\_field dim bloc**

where

- **pb\_field** *bloc\_lecture* ([3.2](#)): block similar to { pb1 field1 } or { pb1 field1 ... pbN fieldN }
- **dim** *int*: Number of field components.
- **bloc** *bloc\_lecture* ([3.2](#)): Values (the table (the value of the field at any time is calculated by linear interpolation from this table) or the analytical expression (with keyword expression to use an analytical expression)).

## 20.19 Champ\_init\_canal\_sinal

Description: For a parabolic profile on U velocity with an unpredictable disturbance on V and W and a sinusoidal disturbance on V velocity.

See also: champ\_don\_base ([20.9](#))

Usage:

**champ\_init\_canal\_sinal dim bloc**

where

- **dim** *int*: Number of field components.
- **bloc** *bloc\_lec\_champ\_init\_canal\_sinal* (20.20): Parameters for the class `champ_init_canal_sinal`.

## 20.20 Bloc\_lec\_champ\_init\_canal\_sinal

Description: Parameters for the class `champ_init_canal_sinal`.

in 2D:

$$U = u_{cent} * y(2h - y) / h / h$$

$$V = ampli\_bruit * rand + ampli\_sin * \sin(\omega * x)$$

rand: unpredictable value between -1 and 1.

in 3D:

$$U = u_{cent} * y(2h - y) / h / h$$

$$V = ampli\_bruit * rand1 + ampli\_sin * \sin(\omega * x)$$

$$W = ampli\_bruit * rand2$$

rand1 and rand2: unpredictable values between -1 and 1.

See also: `objet_lecture` (47)

Usage:

```
{
 ucent float
 h float
 ampli_bruit float
 [ampli_sin float]
 omega float
 [dir_flow int into [0, 1, 2]]
 [dir_wall int into [0, 1, 2]]
 [min_dir_flow float]
 [min_dir_wall float]
}
```

where

- **ucent** *float*: Velocity value at the center of the channel.
- **h** *float*: Half length of the channel.
- **ampli\_bruit** *float*: Amplitude for the disturbance.
- **ampli\_sin** *float*: Amplitude for the sinusoidal disturbance (by default equals to `ucent/10`).
- **omega** *float*: Value of pulsation for the of the sinusoidal disturbance.
- **dir\_flow** *int into [0, 1, 2]*: Flow direction for the initialization of the flow in a channel.
  - if `dir_flow=0`, the flow direction is X
  - if `dir_flow=1`, the flow direction is Y
  - if `dir_flow=2`, the flow direction is Z
 Default value for `dir_flow` is 0
- **dir\_wall** *int into [0, 1, 2]*: Wall direction for the initialization of the flow in a channel.
  - if `dir_wall=0`, the normal to the wall is in X direction
  - if `dir_wall=1`, the normal to the wall is in Y direction
  - if `dir_wall=2`, the normal to the wall is in Z direction
 Default value for `dir_flow` is 1
- **min\_dir\_flow** *float*: Value of the minimum coordinate in the flow direction for the initialization of the flow in a channel. Default value for `dir_flow` is 0.
- **min\_dir\_wall** *float*: Value of the minimum coordinate in the wall direction for the initialization of the flow in a channel. Default value for `dir_flow` is 0.

## 20.21 Champ\_input\_base

Description: not\_set

See also: champ\_base (20.1) champ\_input\_p0 (20.22) champ\_input\_p0\_composite (20.23)

Usage:

**champ\_input\_base** *str*

**Read** *str* {  
    **nb\_comp** *int*  
    **nom** *str*  
    [ **initial\_value** *n x1 x2 ... xn*]  
    **probleme** *str*  
    [ **sous\_zone** *str*]

}

where

- **nb\_comp** *int*
- **nom** *str*
- **initial\_value** *n x1 x2 ... xn*
- **probleme** *str*
- **sous\_zone** *str*

## 20.22 Champ\_input\_p0

Description: not\_set

See also: champ\_input\_base (20.21)

Usage:

**champ\_input\_p0** *str*

**Read** *str* {  
    **nb\_comp** *int*  
    **nom** *str*  
    [ **initial\_value** *n x1 x2 ... xn*]  
    **probleme** *str*  
    [ **sous\_zone** *str*]

}

where

- **nb\_comp** *int* for inheritance
- **nom** *str* for inheritance
- **initial\_value** *n x1 x2 ... xn* for inheritance
- **probleme** *str* for inheritance
- **sous\_zone** *str* for inheritance

## 20.23 Champ\_input\_p0\_composite

Description: Field used to define a classical champ input p0 field (for ICoCo), but with a predefined field for the initial state.

See also: `champ_input_base` (20.21)

Usage:

**champ\_input\_p0\_composite** *str*

**Read** *str* {

    [ **initial\_field** *champ\_base*]

    [ **input\_field** *champ\_input\_p0*]

**nb\_comp** *int*

**nom** *str*

    [ **initial\_value** *n x1 x2 ... xn*]

**probleme** *str*

    [ **sous\_zone** *str*]

}

where

- **initial\_field** *champ\_base* (20.1): The field used for initialization
- **input\_field** *champ\_input\_p0* (20.22): The input field for ICoCo
- **nb\_comp** *int* for inheritance
- **nom** *str* for inheritance
- **initial\_value** *n x1 x2 ... xn* for inheritance
- **probleme** *str* for inheritance
- **sous\_zone** *str* for inheritance

## 20.24 Champ\_musig

Description: MUSIG field. Used in multiphase problems to associate data to each phase.

See also: `champ_composite` (20.8)

Usage:

**champ\_musig** **bloc**

where

- **bloc** *bloc\_lecture* (3.2): Not set

## 20.25 Champ\_ostwald

Description: This keyword is used to define the viscosity variation law:

$\mu(T) = K(T) \cdot (D:D/2)^{((n-1)/2)}$

See also: `champ_base` (20.1)

Usage:

**champ\_ostwald**

## 20.26 Champ\_som\_lu\_vdf

Description: Keyword to read in a file values located at the nodes of a mesh in VDF discretization.

See also: `champ_don_base` (20.9)

Usage:

**champ\_som\_lu\_vdf** **domain\_name** **dim** **tolerance** **file**

where

- **domain\_name** *str*: Name of the domain.
- **dim** *int*: Value of the dimension of the field.
- **tolerance** *float*: Value of the tolerance to check the coordinates of the nodes.
- **file** *str*: name of the file

This file has the following format:

Xi Yi Zi -> Coordinates of the node

Ui Vi Wi -> Value of the field on this node

Xi+1 Yi+1 Zi+1 -> Next point

Ui+1 Vi+1 Zi+1 -> Next value ...

## 20.27 Champ\_som\_lu\_vdf

Description: Keyword to read in a file values located at the nodes of a mesh in VEF discretization.

See also: `champ_don_base` ([20.9](#))

Usage:

**champ\_som\_lu\_vdf** **domain\_name** **dim** **tolerance** **file**

where

- **domain\_name** *str*: Name of the domain.
- **dim** *int*: Value of the dimension of the field.
- **tolerance** *float*: Value of the tolerance to check the coordinates of the nodes.
- **file** *str*: Name of the file.

This file has the following format:

Xi Yi Zi -> Coordinates of the node

Ui Vi Wi -> Value of the field on this node

Xi+1 Yi+1 Zi+1 -> Next point

Ui+1 Vi+1 Zi+1 -> Next value ...

## 20.28 Champ\_tabule\_lu

Description: Uniform field, tabulated from a specified column file. Lines starting with # are ignored.

See also: `champ_tabule_temps` ([20.29](#))

Usage:

**champ\_tabule\_lu** **nb\_comp** **column\_file** **dim**

where

- **nb\_comp** *int*: Number of field components.
- **column\_file** *str*: Name of the column file.
- **dim** *int*: Number of field components.

## 20.29 Champ\_tabule\_temps

Description: Field that is constant in space and tabulated as a function of time.

See also: `champ_don_base` ([20.9](#)) `champ_tabule_lu` ([20.28](#))

Usage:

**champ\_tabule\_temps dim bloc**

where

- **dim** *int*: Number of field components.
- **bloc** *bloc\_lecture* (3.2): Values as a table. The value of the field at any time is calculated by linear interpolation from this table.

## 20.30 Champ\_uniforme\_morceaux

Description: Field which is partly constant in space and stationary.

See also: `champ_don_base` (20.9) `champ_uniforme_morceaux_tabule_temps` (20.31) `valeur_totale_sur_volume` (20.38)

Usage:

**champ\_uniforme\_morceaux nom\_dom nb\_comp data**

where

- **nom\_dom** *str*: Name of the domain to which the sub-areas belong.
- **nb\_comp** *int*: Number of field components.
- **data** *bloc\_lecture* (3.2): { Default val\_def sous\_zone\_1 val\_1 ... sous\_zone\_i val\_i } By default, the value val\_def is assigned to the field. It takes the sous\_zone\_i identifier Sous\_Zone (sub\_area) type object value, val\_i. Sous\_Zone (sub\_area) type objects must have been previously defined if the operator wishes to use a Champ\_Uniforme\_Morceaux(partly\_uniform\_field) type object.

## 20.31 Champ\_uniforme\_morceaux\_tabule\_temps

Description: this type of field is constant in space on one or several sub\_zones and tabulated as a function of time.

See also: `champ_uniforme_morceaux` (20.30)

Usage:

**champ\_uniforme\_morceaux\_tabule\_temps nom\_dom nb\_comp data**

where

- **nom\_dom** *str*: Name of the domain to which the sub-areas belong.
- **nb\_comp** *int*: Number of field components.
- **data** *bloc\_lecture* (3.2): { Default val\_def sous\_zone\_1 val\_1 ... sous\_zone\_i val\_i } By default, the value val\_def is assigned to the field. It takes the sous\_zone\_i identifier Sous\_Zone (sub\_area) type object value, val\_i. Sous\_Zone (sub\_area) type objects must have been previously defined if the operator wishes to use a Champ\_Uniforme\_Morceaux(partly\_uniform\_field) type object.

## 20.32 Champ\_fonc\_txyz

Description: Field defined by analytical functions. It makes it possible the definition of a field that depends on the time and the space.

See also: `champ_don_base` (20.9)

Usage:

**champ\_fonc\_txyz dom val**

where

- **dom** *str*: Name of domain of calculation
- **val** *n word1 word2 ... wordn*: List of functions on (t,x,y,z).

### 20.33 Champ\_fonc\_xyz

Description: Field defined by analytical functions. It makes it possible the definition of a field that depends on (x,y,z).

See also: `champ_don_base` ([20.9](#))

Usage:

**champ\_fonc\_xyz dom val**

where

- **dom** *str*: Name of domain of calculation.
- **val** *n word1 word2 ... wordn*: List of functions on (x,y,z).

### 20.34 Field\_uniform\_keps\_from\_ud

Description: field which allows to impose on a domain K and EPS values derived from U velocity and D hydraulic diameter

See also: `champ_base` ([20.1](#))

Usage:

**field\_uniform\_keps\_from\_ud str**

**Read** *str* {

**u** *float*

**d** *float*

}

where

- **u** *float*: value of velocity specified in boundary condition.
- **d** *float*: value of hydraulic diameter specified in boundary condition

### 20.35 Init\_par\_partie

Description: ne marche que pour n\_comp=1

See also: `champ_don_base` ([20.9](#))

Usage:

**init\_par\_partie n\_comp val1 val2 val3**

where

- **n\_comp** *int into [1]*
- **val1** *float*
- **val2** *float*
- **val3** *float*

## 20.36 Tayl\_green

Description: Class Tayl\_green.

See also: champ\_don\_base (20.9)

Usage:

**tayl\_green dim**

where

- **dim** *int*: Dimension.

## 20.37 Uniform\_field

Synonymous: **champ\_uniforme**

Description: Field that is constant in space and stationary.

See also: champ\_don\_base (20.9)

Usage:

**uniform\_field val**

where

- **val** *n x1 x2 ... xn*: Values of field components.

## 20.38 Valeur\_totale\_sur\_volume

Description: Similar as Champ\_Uniforme\_Morceaux with the same syntax. Used for source terms when we want to specify a source term with a value given for the volume (eg: heat in Watts) and not a value per volume unit (eg: heat in Watts/m3).

See also: champ\_uniforme\_morceaux (20.30)

Usage:

**valeur\_totale\_sur\_volume nom\_dom nb\_comp data**

where

- **nom\_dom** *str*: Name of the domain to which the sub-areas belong.
- **nb\_comp** *int*: Number of field components.
- **data** *bloc\_lecture* (3.2): { Defaut val\_def sous\_zone\_1 val\_1 ... sous\_zone\_i val\_i } By default, the value val\_def is assigned to the field. It takes the sous\_zone\_i identifier Sous\_Zone (sub\_area) type object value, val\_i. Sous\_Zone (sub\_area) type objects must have been previously defined if the operator wishes to use a Champ\_Uniforme\_Morceaux(partly\_uniform\_field) type object.

# 21 champ\_front\_base

## 21.1 Champ\_front\_base

Description: Basic class for fields at domain boundaries.

See also: objet\_u (48) Champ\_front\_debit\_QC\_VDF (21.8) Champ\_front\_debit\_QC\_VDF\_fonc\_t (21.9)



Champ\_front\_Parametrique (21.6) champ\_front\_fonc\_txyz (21.28) champ\_front\_bruite (21.17) ch\_front\_input (21.14) champ\_front\_calc (21.18) champ\_front\_tabule (21.36) champ\_front\_fonc\_pois\_ipsn (21.25) champ\_front\_composite (21.19) champ\_front\_fonction (21.30) champ\_front\_xyz\_debit (21.41) champ\_front\_debit\_massique (21.24) champ\_front\_uniforme (21.39) champ\_front\_lu (21.31) boundary\_field\_inward (21.12) champ\_front\_normal\_vef (21.33) champ\_front\_fonc\_t (21.27) champ\_front\_fonc\_pois\_tube (21.26) champ\_front\_debit (21.23) champ\_front\_recyclage (21.35) champ\_front\_fonc\_xyz (21.29) champ\_front\_MED (21.16) champ\_front\_tangentiel\_vef (21.38) champ\_front\_contact\_vef (21.22) champ\_front\_pression\_from\_u (21.34) champ\_front\_vortex (21.40) boundary\_field\_uniform\_keps\_from\_ud (21.13) Champ\_front\_synt (21.10) Ch\_front\_input\_ALE (21.3) Champ\_front\_ALE\_Beam (21.5) Boundary\_field\_keps\_from\_ud (21.2) Champ\_front\_ale (21.7)

Usage:

## 21.2 Boundary\_field\_keps\_from\_ud

Description: To specify a K-Eps inlet field with hydraulic diameter, speed, and turbulence intensity (VDF only)

See also: champ\_front\_base (21.1)

Usage:

**Boundary\_field\_keps\_from\_ud** *str*

**Read** *str* {

**u** *champ\_front\_base*

**d** *float*

**i** *float*

}

where

- **u** *champ\_front\_base* (21.1): U 0 Initial velocity magnitude
- **d** *float*: Hydraulic diameter
- **i** *float*: Turbulence intensity [

## 21.3 Ch\_front\_input\_ale

Description: Class to define a boundary condition on a moving boundary of a mesh (only for the Arbitrary Lagrangian-Eulerian framework).

Example: Ch\_front\_input\_ALE { nb\_comp 3 nom VITESSE\_IN\_ALE probleme pb initial\_value 3 1. 0. 0. }

See also: champ\_front\_base (21.1)

Usage:

## 21.4 Champ\_front\_xyz\_tabule

Description: Space dependent field on the boundary, tabulated as a function of time.

See also: champ\_front\_fonc\_txyz (21.28)

Usage:

**Champ\_Front\_xyz\_Tabule** **val** **bloc**

where

- **val** *n word1 word2 ... wordn*: Values of field components (mathematical expressions).
- **bloc** *bloc\_lecture* (3.2): {nt1 t2 t3 ...tn u1 [v1 w1 ...] u2 [v2 w2 ...] u3 [v3 w3 ...] ... un [vn wn ...]  
}  
Values are entered into a table based on n couples (ti, ui) if nb\_comp value is 1. The value of a field at a given time is calculated by linear interpolation from this table.

## 21.5 Champ\_front\_ale\_beam

Description: Class to define a Beam on a FSI boundary.

See also: champ\_front\_base (21.1)

Usage:

**Champ\_front\_ALE\_Beam** **val**  
where

- **val** *n word1 word2 ... wordn*:  
Example: 3 0 0 0

## 21.6 Champ\_front\_parametrique

Description: Parametric boundary field

See also: champ\_front\_base (21.1)

Usage:

**Champ\_front\_Parametrique** *str*  
**Read** *str* {  
    **fichier** *str*  
}  
where

- **fichier** *str*: Filename where boundary fields are read

## 21.7 Champ\_front\_ale

Description: Class to define a boundary condition on a moving boundary of a mesh (only for the Arbitrary Lagrangian-Eulerian framework ).

See also: champ\_front\_base (21.1)

Usage:

**Champ\_front\_ale** **val**  
where

- **val** *n word1 word2 ... wordn*:  
Example: 2 -y\*0.01 x\*0.01

## 21.8 Champ\_front\_debit\_qc\_vdf

Description: This keyword is used to define a flow rate field for quasi-compressible fluids in VDF discretization. The flow rate is kept constant during a transient.

See also: `champ_front_base` ([21.1](#))

Usage:

**Champ\_front\_debit\_QC\_VDF** **dimension** **liste** [ **moyen** ] **pb\_name**

where

- **dimension** *int*: Problem dimension
- **liste** *bloc\_lecture* ([3.2](#)): List of the mass flow rate values [kg/s/m2] with the following syntaxe: { val1 ... valdim }
- **moyen** *str*: Option to use rho mean value
- **pb\_name** *str*: Problem name

## 21.9 Champ\_front\_debit\_qc\_vdf\_fonc\_t

Description: This keyword is used to define a flow rate field for quasi-compressible fluids in VDF discretization. The flow rate could be constant or time-dependent.

See also: `champ_front_base` ([21.1](#))

Usage:

**Champ\_front\_debit\_QC\_VDF\_fonc\_t** **dimension** **liste** [ **moyen** ] **pb\_name**

where

- **dimension** *int*: Problem dimension
- **liste** *bloc\_lecture* ([3.2](#)): List of the mass flow rate values [kg/s/m2] with the following syntaxe: { val1 ... valdim } where val1 ... valdim are constant or function of time.
- **moyen** *str*: Option to use rho mean value
- **pb\_name** *str*: Problem name

## 21.10 Champ\_front\_synt

Description: Boundary condition to create the synthetic fluctuations as inlet boundary. Available only for 3D configurations.

See also: `champ_front_base` ([21.1](#))

Usage:

**Champ\_front\_synt** **dim** **bloc**

where

- **dim** *int*: Number of field components. It should be 3!
- **bloc** *bloc\_lecture\_turb\_synt* ([21.11](#)): bloc containing the parameters of the synthetic turbulence

## 21.11 Bloc\_lecture\_turb\_synt

Description: bloc containing parameters of the synthetic turbulence

See also: `objet_lecture` ([47](#))

Usage:

```
{

 moyenne x1 x2 (x3)
 lengthScale float
 nbModes int
 turbKinEn float
 turbDissRate float
 ratioCutoffWavenumber float
 KeOverKmin float
 timeScale float
 dir_fluct x1 x2 (x3)

}
```

where

- **moyenne** *x1 x2 (x3)*: components of the average velocity fields
- **lengthScale** *float*: turbulent length scale
- **nbModes** *int*: number of Fourier modes
- **turbKinEn** *float*: turbulent kinetic energy (k)
- **turbDissRate** *float*: turbulent dissipation rate (epsilon)
- **ratioCutoffWavenumber** *float*: ratio between the cut-off wavenumber and pi/delta
- **KeOverKmin** *float*: ratio of the most energetic wavenumber Ke over the minimum wavenumber Kmin representing the largest turbulent eddies
- **timeScale** *float*: turbulent time scale
- **dir\_fluct** *x1 x2 (x3)*: directions for the velocity fluctuations (e.g 1 0 0 generates velocity fluctuations in the x-direction only)

## 21.12 Boundary\_field\_inward

Description: this field is used to define the normal vector field standard at the boundary in VDF or VEF discretization.

See also: `champ_front_base` ([21.1](#))

Usage:

```
boundary_field_inward str
Read str {
```

```
 normal_value str
```

```
}
```

where

- **normal\_value** *str*: normal vector value (positive value for a vector oriented outside to inside) which can depend of the time.

## 21.13 Boundary\_field\_uniform\_keps\_from\_ud

Description: field which allows to impose on a boundary K and EPS values derived from U velocity and D hydraulic diameter

See also: `champ_front_base` ([21.1](#))

Usage:

**boundary\_field\_uniform\_keps\_from\_ud** *str*

**Read** *str* {

**u** *float*

**d** *float*

}

where

- **u** *float*: value of velocity
- **d** *float*: value of hydraulic diameter

## 21.14 Ch\_front\_input

Description: not\_set

See also: champ\_front\_base ([21.1](#)) ch\_front\_input\_uniforme ([21.15](#))

Usage:

**ch\_front\_input** *str*

**Read** *str* {

**nb\_comp** *int*

**nom** *str*

    [ **initial\_value** *n x1 x2 ... xn*]

**probleme** *str*

    [ **sous\_zone** *str*]

}

where

- **nb\_comp** *int*
- **nom** *str*
- **initial\_value** *n x1 x2 ... xn*
- **probleme** *str*
- **sous\_zone** *str*

## 21.15 Ch\_front\_input\_uniforme

Description: for coupling, you can use ch\_front\_input\_uniforme which is a champ\_front\_uniforme, which use an external value. It must be used with Problem.setInputField.

See also: ch\_front\_input ([21.14](#))

Usage:

**ch\_front\_input\_uniforme** *str*

**Read** *str* {

**nb\_comp** *int*

**nom** *str*

    [ **initial\_value** *n x1 x2 ... xn*]

**probleme** *str*

    [ **sous\_zone** *str*]

}  
where

- **nb\_comp** *int* for inheritance
- **nom** *str* for inheritance
- **initial\_value** *n x1 x2 ... xn* for inheritance
- **probleme** *str* for inheritance
- **sous\_zone** *str* for inheritance

## 21.16 Champ\_front\_med

Description: Field allowing the loading of a boundary condition from a MED file using Champ\_fonc\_med

See also: champ\_front\_base (21.1)

Usage:

**champ\_front\_MED champ\_fonc\_med**

where

- **champ\_fonc\_med** *champ\_base* (20.1): a champ\_fonc\_med loading the values of the unknown on a domain boundary

## 21.17 Champ\_front\_bruite

Description: Field which is variable in time and space in a random manner.

See also: champ\_front\_base (21.1)

Usage:

**champ\_front\_bruite nb\_comp bloc**

where

- **nb\_comp** *int*: Number of field components.
- **bloc** *bloc\_lecture* (3.2): { [N val L val ] Moyenne m\_1.....[m\_i ] Amplitude A\_1.....[A\_i ]}: Random noise: If N and L are not defined, the ith component of the field varies randomly around an average value m\_i with a maximum amplitude A\_i.  
White noise: If N and L are defined, these two additional parameters correspond to L, the domain length and N, the number of nodes in the domain. Noise frequency will be between  $2\pi/L$  and  $2\pi N/(4L)$ .  
For example, formula for velocity:  $u=U0(t)$   $v=U1(t)Uj(t)=Mj+2\cdot Aj\cdot \text{bruit\_blanc}$  where bruit\_blanc (white\_noise) is the formula given in the mettre\_a\_jour (update) method of the Champ\_front\_bruite (noise\_boundary\_field) (Refer to the Champ\_front\_bruite.cpp file).

## 21.18 Champ\_front\_calc

Description: This keyword is used on a boundary to get a field from another boundary. The local and remote boundaries should have the same mesh. If not, the Champ\_front\_recyclage keyword could be used instead. It is used in the condition block at the limits of equation which itself refers to a problem called pb1. We are working under the supposition that pb1 is coupled to another problem.

See also: champ\_front\_base (21.1)

Usage:

**champ\_front\_calc problem\_name bord field\_name**

where

- **problem\_name** *str*: Name of the other problem to which pb1 is coupled.
- **bord** *str*: Name of the side which is the boundary between the 2 domains in the domain object description associated with the problem\_name object.
- **field\_name** *str*: Name of the field containing the value that the user wishes to use at the boundary. The field\_name object must be recognized by the problem\_name object.

## 21.19 Champ\_front\_composite

Description: Composite front field. Used in multiphase problems to associate data to each phase.

See also: champ\_front\_base (21.1) champ\_front\_musig (21.32)

Usage:

**champ\_front\_composite dim bloc**

where

- **dim** *int*: Number of field components.
- **bloc** *bloc\_lecture* (3.2): Values Various pieces of the field, defined per phase. Part 1 goes to phase 1, etc...

## 21.20 Champ\_front\_contact\_rayo\_semi\_transp\_vef

Description: This field is used on a boundary between a solid and fluid domain to exchange a calculated temperature at the contact face of the two domains according to the flux of the two problems with radiation in semi transparent fluid.

See also: champ\_front\_contact\_vef (21.22)

Usage:

**champ\_front\_contact\_rayo\_semi\_transp\_vef local\_pb local\_boundary remote\_pb remote\_boundary**

where

- **local\_pb** *str*: Name of the problem.
- **local\_boundary** *str*: Name of the boundary.
- **remote\_pb** *str*: Name of the second problem.
- **remote\_boundary** *str*: Name of the boundary in the second problem.

## 21.21 Champ\_front\_contact\_rayo\_transp\_vef

Description: This field is used on a boundary between a solid and fluid domain to exchange a calculated temperature at the contact face of the two domains according to the flux of the two problems with radiation in transparent fluid.

See also: champ\_front\_contact\_vef (21.22)

Usage:

**champ\_front\_contact\_rayo\_transp\_vef local\_pb local\_boundary remote\_pb remote\_boundary**

where

- **local\_pb** *str*: Name of the problem.
- **local\_boundary** *str*: Name of the boundary.
- **remote\_pb** *str*: Name of the second problem.
- **remote\_boundary** *str*: Name of the boundary in the second problem.

## 21.22 Champ\_front\_contact\_vef

Description: This field is used on a boundary between a solid and fluid domain to exchange a calculated temperature at the contact face of the two domains according to the flux of the two problems.

See also: [champ\\_front\\_base \(21.1\)](#) [champ\\_front\\_contact\\_rayo\\_transp\\_vef \(21.21\)](#) [champ\\_front\\_contact\\_rayo\\_semi\\_transp\\_vef \(21.20\)](#)

Usage:

**champ\_front\_contact\_vef local\_pb local\_boundary remote\_pb remote\_boundary**  
where

- **local\_pb** *str*: Name of the problem.
- **local\_boundary** *str*: Name of the boundary.
- **remote\_pb** *str*: Name of the second problem.
- **remote\_boundary** *str*: Name of the boundary in the second problem.

## 21.23 Champ\_front\_debit

Description: This field is used to define a flow rate field instead of a velocity field for a Dirichlet boundary condition on Navier-Stokes equations.

See also: [champ\\_front\\_base \(21.1\)](#)

Usage:

**champ\_front\_debit ch**  
where

- **ch** *champ\_front\_base (21.1)*: uniform field in space to define the flow rate. It could be, for example, `champ_front_uniforme`, `ch_front_input_uniform` or `champ_front_fonc_txyz` that depends only on time.

## 21.24 Champ\_front\_debit\_massique

Description: This field is used to define a flow rate field using the density

See also: [champ\\_front\\_base \(21.1\)](#)

Usage:

**champ\_front\_debit\_massique ch**  
where

- **ch** *champ\_front\_base (21.1)*: uniform field in space to define the flow rate. It could be, for example, `champ_front_uniforme`, `ch_front_input_uniform` or `champ_front_fonc_txyz` that depends only on time.



## 21.25 Champ\_front\_fonc\_pois\_ipsn

Description: Boundary field champ\_front\_fonc\_pois\_ipsn.

See also: champ\_front\_base (21.1)

Usage:

**champ\_front\_fonc\_pois\_ipsn** **r\_tube** **umoy** **r\_loc**

where

- **r\_tube** *float*
- **umoy** *n x1 x2 ... xn*
- **r\_loc** *x1 x2 (x3)*

## 21.26 Champ\_front\_fonc\_pois\_tube

Description: Boundary field champ\_front\_fonc\_pois\_tube.

See also: champ\_front\_base (21.1)

Usage:

**champ\_front\_fonc\_pois\_tube** **r\_tube** **umoy** **r\_loc** **r\_loc\_mult**

where

- **r\_tube** *float*
- **umoy** *n x1 x2 ... xn*
- **r\_loc** *x1 x2 (x3)*
- **r\_loc\_mult** *n1 n2 (n3)*

## 21.27 Champ\_front\_fonc\_t

Description: Boundary field that depends only on time.

See also: champ\_front\_base (21.1)

Usage:

**champ\_front\_fonc\_t** **val**

where

- **val** *n word1 word2 ... wordn*: Values of field components (mathematical expressions).

## 21.28 Champ\_front\_fonc\_txyz

Description: Boundary field which is not constant in space and in time.

See also: champ\_front\_base (21.1) Champ\_Front\_xyz\_Tabule (21.4)

Usage:

**champ\_front\_fonc\_txyz** **val**

where

- **val** *n word1 word2 ... wordn*: Values of field components (mathematical expressions).

### 21.29 Champ\_front\_fonc\_xyz

Description: Boundary field which is not constant in space.

See also: `champ_front_base` ([21.1](#))

Usage:

**champ\_front\_fonc\_xyz** **val**

where

- **val** *n word1 word2 ... wordn*: Values of field components (mathematical expressions).

### 21.30 Champ\_front\_fonction

Description: boundary field that is function of another field

See also: `champ_front_base` ([21.1](#))

Usage:

**champ\_front\_fonction** **dim inco expression**

where

- **dim** *int*: Number of field components.
- **inco** *str*: Name of the field (for example: temperature).
- **expression** *str*: keyword to use a analytical expression like `10.*EXP(-0.1*val)` where `val` be the keyword for the field.

### 21.31 Champ\_front\_lu

Description: boundary field which is given from data issued from a read file. The format of this file has to be the same that the one generated by `Ecrire_fichier_xyz_valeur`

Example for K and epsilon quantities to be defined for inlet condition in a boundary named 'entree':

`entree frontiere_ouverte_K_Eps_impose Champ_Front_lu dom 2pb_K_EPS_PERIO_1006.306198.dat`

See also: `champ_front_base` ([21.1](#))

Usage:

**champ\_front\_lu** **domaine dim file**

where

- **domaine** *str*: Name of domain
- **dim** *int*: number of components
- **file** *str*: path for the read file

### 21.32 Champ\_front\_musig

Description: MUSIG front field. Used in multiphase problems to associate data to each phase.

See also: `champ_front_composite` ([21.19](#))

Usage:

**champ\_front\_musig** **bloc**

where

- **bloc** *bloc\_lecture* ([3.2](#)): Not set

### 21.33 Champ\_front\_normal\_vef

Description: Field to define the normal vector field standard at the boundary in VEF discretization.

See also: `champ_front_base` (21.1)

Usage:

**champ\_front\_normal\_vef** **mot** **vit\_tan**

where

- **mot** *str* into [*valeur\_normale*']: Name of vector field.
- **vit\_tan** *float*: normal vector value (positive value for a vector oriented outside to inside).

### 21.34 Champ\_front\_pression\_from\_u

Description: this field is used to define a pressure field depending of a velocity field.

See also: `champ_front_base` (21.1)

Usage:

**champ\_front\_pression\_from\_u** **expression**

where

- **expression** *str*: value depending of a velocity (like  $2 * u_{moy}^2$ ).

### 21.35 Champ\_front\_recyclage

Description: This keyword is used on a boundary to get a field from another boundary.

It is to use, in a general way, on a boundary of a local\_pb problem, a field calculated from a linear combination of an imposed field  $g(x,y,z,t)$  with an instantaneous  $f(x,y,z,t)$  and a spatial mean field  $\langle f \rangle(t)$  or a temporal mean field  $\langle f \rangle(x,y,z)$  extracted from a plane of a problem named pb (pb may be local\_pb itself): For each component i, the field F applied on the boundary will be:

$$F_i(x,y,z,t) = \alpha_i * g_i(x,y,z,t) + \chi_i * [f_i(x,y,z,t) - \beta_i * \langle f_i \rangle]$$

See also: `champ_front_base` (21.1)

Usage:

**champ\_front\_recyclage** *str*

**Read** *str* {

```
 pb_champ_evaluateur pb_champ_evaluateur
 [distance_plan x1 x2 (x3)]
 [ampli_moyenne_imposee n x1 x2 ... xn]
 [ampli_moyenne_recyclee n x1 x2 ... xn]
 [ampli_fluctuation n x1 x2 ... xn]
 [direction_anisotrope int into [1, 2, 3]]
 [moyenne_imposee moyenne_imposee_deriv]
 [moyenne_recyclee str]
 [fichier str]
```

}

where

- **pb\_champ\_evaluateur** *pb\_champ\_evaluateur* (33)

- **distance\_plan** *x1 x2 (x3)*: Vector which gives the distance between the boundary and the plane from where the field F will be extracted. By default, the vector is zero, that should imply the two domains have coincident boundaries.
- **ampli\_moyenne\_imposee** *n x1 x2 ... xn*: 2|3 alpha(0) alpha(1) [alpha(2)]: alpha\_i coefficients (by default =1)
- **ampli\_moyenne\_recyclee** *n x1 x2 ... xn*: 2|3 beta(0) beta(1) [beta(2)]: beta\_i coefficients (by default =1)
- **ampli\_fluctuation** *n x1 x2 ... xn*: 2|3 gamma(0) gamma(1) [gamma(2)]: gamma\_i coefficients (by default =1)
- **direction\_anisotrope** *int into [1, 2, 3]*: If an integer is given for direction (X:1, Y:2, Z:3, by default, direction is negative), the imposed field g will be 0 for the 2 other directions.
- **moyenne\_imposee** *moyenne\_imposee\_deriv (30)*: Value of the imposed g field.
- **moyenne\_recyclee** *str*: Method used to perform a spatial or a temporal averaging of f field to specify <f>. <f> can be the surface mean of f on the plane (surface option, see below) or it can be read from several files (for example generated by the *chmoy\_faceperio* option of the *Traitement\_particulier* keyword to obtain a temporal mean field). The option *methode\_recyc* can be: *surfacique*, Surface mean for <f> from f values on the plane ; Or one of the following *methode\_moy* options applied to read a temporal mean field <f>(x,y,z): *interpolation*, *connexion\_approchee* or *connexion\_exacte*
- **fichier** *str*

## 21.36 Champ\_front\_tabule

Description: Constant field on the boundary, tabulated as a function of time.

See also: *champ\_front\_base* (21.1) *champ\_front\_tabule\_lu* (21.37)

Usage:

**champ\_front\_tabule nb\_comp bloc**

where

- **nb\_comp** *int*: Number of field components.
  - **bloc** *bloc\_lecture (3.2)*: {nt1 t2 t3 ...tn u1 [v1 w1 ...] u2 [v2 w2 ...] u3 [v3 w3 ...] ... un [vn wn ...] }
- Values are entered into a table based on n couples (ti, ui) if nb\_comp value is 1. The value of a field at a given time is calculated by linear interpolation from this table.

## 21.37 Champ\_front\_tabule\_lu

Description: Constant field on the boundary, tabulated from a specified column file. Lines starting with # are ignored.

See also: *champ\_front\_tabule* (21.36)

Usage:

**champ\_front\_tabule\_lu nb\_comp column\_file**

where

- **nb\_comp** *int*: Number of field components.
- **column\_file** *str*: Name of the column file.

### 21.38 Champ\_front\_tangentiel\_vef

Description: Field to define the tangential velocity vector field standard at the boundary in VEF discretization.

See also: `champ_front_base` ([21.1](#))

Usage:

**champ\_front\_tangentiel\_vef** **mot** **vit\_tan**

where

- **mot** *str* into [*vitesse\_tangentielle*]: Name of vector field.
- **vit\_tan** *float*: Vector field standard [m/s].

### 21.39 Champ\_front\_uniforme

Description: Boundary field which is constant in space and stationary.

See also: `champ_front_base` ([21.1](#))

Usage:

**champ\_front\_uniforme** **val**

where

- **val** *n x1 x2 ... xn*: Values of field components.

### 21.40 Champ\_front\_vortex

Description: `not_set`

See also: `champ_front_base` ([21.1](#))

Usage:

**champ\_front\_vortex** **dom** **geom** **nu** **utau**

where

- **dom** *str*: Name of domain.
- **geom** *str*
- **nu** *float*
- **utau** *float*

### 21.41 Champ\_front\_xyz\_debit

Description: This field is used to define a flow rate field with a velocity profil which will be normalized to match the flow rate chosen.

See also: `champ_front_base` ([21.1](#))

Usage:

**champ\_front\_xyz\_debit** *str*

**Read** *str* {

[ **velocity\_profil** *champ\_front\_base*]

```

 flow_rate champ_front_base
}
where

```

- **velocity\_profil** *champ\_front\_base* (21.1): velocity\_profil 0 velocity field to define the profil of velocity.
- **flow\_rate** *champ\_front\_base* (21.1): flow\_rate 1 uniform field in space to define the flow rate. It could be, for example, champ\_front\_uniforme, ch\_front\_input\_uniform or champ\_front\_fonc\_t

## 22 interpolation\_ibm\_base

Description: Base class for all the interpolation methods available in the Immersed Boundary Method (IBM).

See also: objet\_u (48) ibm\_element\_fluide (22.3) ibm\_aucune (22.2) ibm\_gradient\_moyen (22.5)

Usage:

```

interpolation_ibm_base [impr] [nb_histo_boxes_impr]
where

```

- **impr** : To print IBM-related data
- **nb\_histo\_boxes\_impr** *int*: number of histogram boxes for printed data

### 22.1 Interpolation\_ibm\_power\_law\_tbl\_u\_star

Description: Immersed Boundary Method (IBM): law u star.

See also: ibm\_gradient\_moyen (22.5)

Usage:

```

Interpolation_IBM_power_law_tbl_u_star str
Read str {

```

```

 points_solides champ_base
 est_dirichlet champ_base
 correspondance_elements champ_base
 elements_solides champ_base
 [impr]
 [nb_histo_boxes_impr int]

```

```

}
where

```

- **points\_solides** *champ\_base* (20.1): Node field giving the projection of the node on the immersed boundary
- **est\_dirichlet** *champ\_base* (20.1): Node field of booleans indicating whether the node belong to an element where the interface is
- **correspondance\_elements** *champ\_base* (20.1): Cell field giving the SALOME cell number
- **elements\_solides** *champ\_base* (20.1): Node field giving the element number containing the solid point
- **impr** for inheritance: To print IBM-related data
- **nb\_histo\_boxes\_impr** *int* for inheritance: number of histogram boxes for printed data

## 22.2 Ibm\_aucune

Synonymous: **interpolation\_ibm\_aucune**

Description: Immersed Boundary Method (IBM): no interpolation.

See also: [interpolation\\_ibm\\_base \(22\)](#)

Usage:

**ibm\_aucune** [ **impr** ] [ **nb\_histo\_boxes\_impr** ]

where

- **impr** : To print IBM-related data
- **nb\_histo\_boxes\_impr** *int*: number of histogram boxes for printed data

## 22.3 Ibm\_element\_fluide

Synonymous: **interpolation\_ibm\_element\_fluide**

Description: Immersed Boundary Method (IBM): fluid element interpolation.

See also: [interpolation\\_ibm\\_base \(22\)](#) [ibm\\_power\\_law\\_tbl \(22.6\)](#) [ibm\\_hybride \(22.4\)](#)

Usage:

**ibm\_element\_fluide** *str*

**Read** *str* {

```
 points_fluides champ_base
 points_solides champ_base
 elements_fluides champ_base
 correspondance_elements champ_base
 [impr]
 [nb_histo_boxes_impr int]
```

}

where

- **points\_fluides** *champ\_base* [\(20.1\)](#): Node field giving the projection of the point below (points-\_solides) falling into the pure cell fluid
- **points\_solides** *champ\_base* [\(20.1\)](#): Node field giving the projection of the node on the immersed boundary
- **elements\_fluides** *champ\_base* [\(20.1\)](#): Node field giving the number of the element (cell) containing the pure fluid point
- **correspondance\_elements** *champ\_base* [\(20.1\)](#): Cell field giving the SALOME cell number
- **impr** for inheritance: To print IBM-related data
- **nb\_histo\_boxes\_impr** *int* for inheritance: number of histogram boxes for printed data

## 22.4 Ibm\_hybride

Synonymous: **interpolation\_ibm\_hybride**

Description: Immersed Boundary Method (IBM): hybrid (fluid/mean gradient) interpolation.

See also: [ibm\\_element\\_fluide \(22.3\)](#)

Usage:

**ibm\_hybride** *str*

**Read** *str* {

```
 est_dirichlet champ_base
 elements_solides champ_base
 points_fluides champ_base
 points_solides champ_base
 elements_fluides champ_base
 correspondance_elements champ_base
 [impr]
 [nb_histo_boxes_impr int]
```

}

where

- **est\_dirichlet** *champ\_base* (20.1): Node field of booleans indicating whether the node belong to an element where the interface is
- **elements\_solides** *champ\_base* (20.1): Node field giving the element number containing the solid point
- **points\_fluides** *champ\_base* (20.1) for inheritance: Node field giving the projection of the point below (points\_solides) falling into the pure cell fluid
- **points\_solides** *champ\_base* (20.1) for inheritance: Node field giving the projection of the node on the immersed boundary
- **elements\_fluides** *champ\_base* (20.1) for inheritance: Node field giving the number of the element (cell) containing the pure fluid point
- **correspondance\_elements** *champ\_base* (20.1) for inheritance: Cell field giving the SALOME cell number
- **impr** for inheritance: To print IBM-related data
- **nb\_histo\_boxes\_impr** *int* for inheritance: number of histogram boxes for printed data

## 22.5 Ibm\_gradient\_moyen

Synonymous: **interpolation\_ibm\_gradient\_moyen**

Description: Immersed Boundary Method (IBM): mean gradient interpolation.

See also: **interpolation\_ibm\_base** (22) **Interpolation\_IBM\_power\_law\_tbl\_u\_star** (22.1)

Usage:

**ibm\_gradient\_moyen** *str*

**Read** *str* {

```
 points_solides champ_base
 est_dirichlet champ_base
 correspondance_elements champ_base
 elements_solides champ_base
 [impr]
 [nb_histo_boxes_impr int]
```

}

where

- **points\_solides** *champ\_base* (20.1): Node field giving the projection of the node on the immersed boundary



- **est\_dirichlet** *champ\_base* (20.1): Node field of booleans indicating whether the node belong to an element where the interface is
- **correspondance\_elements** *champ\_base* (20.1): Cell field giving the SALOME cell number
- **elements\_solides** *champ\_base* (20.1): Node field giving the element number containing the solid point
- **impr** for inheritance: To print IBM-related data
- **nb\_histo\_boxes\_impr** *int* for inheritance: number of histogram boxes for printed data

## 22.6 Ibm\_power\_law\_tbl

Synonymous: **interpolation\_ibm\_power\_law\_tbl**

Description: Immersed Boundary Method (IBM): power law interpolation.

See also: **ibm\_element\_fluide** (22.3)

Usage:

**ibm\_power\_law\_tbl** *str*

**Read** *str* {

```
[formulation_linear_pwl int]
points_fluides champ_base
points_solides champ_base
elements_fluides champ_base
correspondance_elements champ_base
[impr]
[nb_histo_boxes_impr int]
```

}

where

- **formulation\_linear\_pwl** *int*: Choix formulation lineaire ou non
- **points\_fluides** *champ\_base* (20.1) for inheritance: Node field giving the projection of the point below (points\_solides) falling into the pure cell fluid
- **points\_solides** *champ\_base* (20.1) for inheritance: Node field giving the projection of the node on the immersed boundary
- **elements\_fluides** *champ\_base* (20.1) for inheritance: Node field giving the number of the element (cell) containing the pure fluid point
- **correspondance\_elements** *champ\_base* (20.1) for inheritance: Cell field giving the SALOME cell number
- **impr** for inheritance: To print IBM-related data
- **nb\_histo\_boxes\_impr** *int* for inheritance: number of histogram boxes for printed data

## 23 loi\_etat\_base

Description: Basic class for state laws used with a dilatable fluid.

See also: **objet\_u** (48) **loi\_etat\_gaz\_parfait\_base** (23.7) **loi\_etat\_tpqi\_base** (23.9) **loi\_etat\_gaz\_reel\_base** (23.8)

Usage:

### 23.1 Eos\_qc

Description: Class for using EOS with QC problem

See also: `loi_etat_tppi_base` ([23.9](#))

Usage:

**EOS\_QC** *str*

**Read** *str* {

**Cp** *float*

**fluid** *str*

**model** *str*

}

where

- **Cp** *float*: Specific heat at constant pressure (J/kg/K).
- **fluid** *str*: Fluid name in the EOS model
- **model** *str*: EOS model name

### 23.2 Eos\_wc

Description: Class for using EOS with WC problem

See also: `loi_etat_tppi_base` ([23.9](#))

Usage:

**EOS\_WC** *str*

**Read** *str* {

**Cp** *float*

**fluid** *str*

**model** *str*

}

where

- **Cp** *float*: Specific heat at constant pressure (J/kg/K).
- **fluid** *str*: Fluid name in the EOS model
- **model** *str*: EOS model name

### 23.3 Binaire\_gaz\_parfait\_qc

Description: Class for perfect gas binary mixtures state law used with a quasi-compressible fluid under the iso-thermal and iso-bar assumptions.

See also: `loi_etat_gaz_parfait_base` ([23.7](#))

Usage:

**binaire\_gaz\_parfait\_QC** *str*

**Read** *str* {

**molar\_mass1** *float*

**molar\_mass2** *float*

```

 mu1 float
 mu2 float
 temperature float
 diffusion_coeff float
}
where

```

- **molar\_mass1** *float*: Molar mass of species 1 (in kg/mol).
- **molar\_mass2** *float*: Molar mass of species 2 (in kg/mol).
- **mu1** *float*: Dynamic viscosity of species 1 (in kg/m.s).
- **mu2** *float*: Dynamic viscosity of species 2 (in kg/m.s).
- **temperature** *float*: Temperature (in Kelvin) which will be constant during the simulation since this state law only works for iso-thermal conditions.
- **diffusion\_coeff** *float*: Diffusion coefficient assumed the same for both species (in m<sup>2</sup>/s).

### 23.4 Binaire\_gaz\_parfait\_wc

Description: Class for perfect gas binary mixtures state law used with a weakly-compressible fluid under the iso-thermal and iso-bar assumptions.

See also: `loi_etat_gaz_parfait_base` ([23.7](#))

Usage:

**binaire\_gaz\_parfait\_WC** *str*

**Read** *str* {

```

 molar_mass1 float
 molar_mass2 float
 mu1 float
 mu2 float
 temperature float
 diffusion_coeff float

```

```

}
where

```

- **molar\_mass1** *float*: Molar mass of species 1 (in kg/mol).
- **molar\_mass2** *float*: Molar mass of species 2 (in kg/mol).
- **mu1** *float*: Dynamic viscosity of species 1 (in kg/m.s).
- **mu2** *float*: Dynamic viscosity of species 2 (in kg/m.s).
- **temperature** *float*: Temperature (in Kelvin) which will be constant during the simulation since this state law only works for iso-thermal conditions.
- **diffusion\_coeff** *float*: Diffusion coefficient assumed the same for both species (in m<sup>2</sup>/s).

### 23.5 Coolprop\_qc

Description: Class for using CoolProp with QC problem

See also: `loi_etat_tppi_base` ([23.9](#))

Usage:

**coolprop\_QC** *str*

**Read** *str* {

```

 Cp float
 fluid str
 model str
}
where

```

- **Cp** *float*: Specific heat at constant pressure (J/kg/K).
- **fluid** *str*: Fluid name in the CoolProp model
- **model** *str*: CoolProp model name

## 23.6 Coolprop\_wc

Description: Class for using CoolProp with WC problem

See also: [loi\\_etat\\_tppi\\_base \(23.9\)](#)

Usage:

```

coolprop_WC str
Read str {

```

```

 Cp float
 fluid str
 model str

```

```

}
where

```

- **Cp** *float*: Specific heat at constant pressure (J/kg/K).
- **fluid** *str*: Fluid name in the CoolProp model
- **model** *str*: CoolProp model name

## 23.7 Loi\_etat\_gaz\_parfait\_base

Description: Basic class for perfect gases state laws used with a dilatable fluid.

See also: [loi\\_etat\\_base \(23\)](#) [multi\\_gaz\\_parfait\\_WC \(23.11\)](#) [binaire\\_gaz\\_parfait\\_WC \(23.4\)](#) [gaz\\_parfait\\_WC \(23.13\)](#) [binaire\\_gaz\\_parfait\\_QC \(23.3\)](#) [multi\\_gaz\\_parfait\\_QC \(23.10\)](#) [rhoT\\_gaz\\_parfait\\_QC \(23.14\)](#) [gaz\\_parfait\\_QC \(23.12\)](#)

Usage:

## 23.8 Loi\_etat\_gaz\_reel\_base

Description: Basic class for real gases state laws used with a dilatable fluid.

See also: [loi\\_etat\\_base \(23\)](#) [rhoT\\_gaz\\_reel\\_QC \(23.15\)](#)

Usage:

## 23.9 Loi\_etat\_tppi\_base

Description: Basic class for thermo-physical properties interface (TPPI) used for dilatable problems

See also: loi\_etat\_base (23) EOS\_WC (23.2) coolprop\_WC (23.6) EOS\_QC (23.1) coolprop\_QC (23.5)

Usage:

## 23.10 Multi\_gaz\_parfait\_qc

Description: Class for perfect gas multi-species mixtures state law used with a quasi-compressible fluid.

See also: loi\_etat\_gaz\_parfait\_base (23.7)

Usage:

**multi\_gaz\_parfait\_QC** *str*

**Read** *str* {

*sc float*  
**prandtl** *float*  
[ **cp float** ]  
[ **dtol\_fraction float** ]  
[ **correction\_fraction** ]  
[ **ignore\_check\_fraction** ]

}

where

- **sc float**: Schmidt number of the gas  $Sc = \nu/D$  ( $D$ : diffusion coefficient of the mixing).
- **prandtl float**: Prandtl number of the gas  $Pr = \mu * Cp / \lambda$
- **cp float**: Specific heat at constant pressure of the gas  $C_p$ .
- **dtol\_fraction float**: Delta tolerance on mass fractions for check testing (default value 1.e-6).
- **correction\_fraction** : To force mass fractions between 0. and 1.
- **ignore\_check\_fraction** : Not to check if mass fractions between 0. and 1.

## 23.11 Multi\_gaz\_parfait\_wc

Description: Class for perfect gas multi-species mixtures state law used with a weakly-compressible fluid.

See also: loi\_etat\_gaz\_parfait\_base (23.7)

Usage:

**multi\_gaz\_parfait\_WC** *str*

**Read** *str* {

**species\_number** *int*  
**diffusion\_coeff** *champ\_base*  
**molar\_mass** *champ\_base*  
**mu** *champ\_base*  
**cp** *champ\_base*  
**prandtl** *float*

}

where

- **species\_number int**: Number of species you are considering in your problem.

- **diffusion\_coeff** *champ\_base* (20.1): Diffusion coefficient of each species, defined with a Champ\_uniforme of dimension equals to the species\_number.
- **molar\_mass** *champ\_base* (20.1): Molar mass of each species, defined with a Champ\_uniforme of dimension equals to the species\_number.
- **mu** *champ\_base* (20.1): Dynamic viscosity of each species, defined with a Champ\_uniforme of dimension equals to the species\_number.
- **cp** *champ\_base* (20.1): Specific heat at constant pressure of the gas Cp, defined with a Champ\_uniforme of dimension equals to the species\_number..
- **prandtl** *float*: Prandtl number of the gas  $Pr = \mu * Cp / \lambda$ .

### 23.12 Gaz\_parfait\_qc

Description: Class for perfect gas state law used with a quasi-compressible fluid.

See also: [loi\\_etat\\_gaz\\_parfait\\_base \(23.7\)](#)

Usage:

**gaz\_parfait\_QC** *str*

**Read** *str* {

**Cp** *float*  
 [ **Cv** *float*]  
 [ **gamma** *float*]  
**Prandtl** *float*  
 [ **rho\_constant\_pour\_debug** *champ\_base*]

}

where

- **Cp** *float*: Specific heat at constant pressure (J/kg/K).
- **Cv** *float*: Specific heat at constant volume (J/kg/K).
- **gamma** *float*:  $Cp/Cv$
- **Prandtl** *float*: Prandtl number of the gas  $Pr = \mu * Cp / \lambda$
- **rho\_constant\_pour\_debug** *champ\_base* (20.1): For developers to debug the code with a constant rho.

### 23.13 Gaz\_parfait\_wc

Description: Class for perfect gas state law used with a weakly-compressible fluid.

See also: [loi\\_etat\\_gaz\\_parfait\\_base \(23.7\)](#)

Usage:

**gaz\_parfait\_WC** *str*

**Read** *str* {

**Cp** *float*  
 [ **Cv** *float*]  
 [ **gamma** *float*]  
**Prandtl** *float*

}

where

- **Cp** *float*: Specific heat at constant pressure (J/kg/K).

- **Cv** *float*: Specific heat at constant volume (J/kg/K).
- **gamma** *float*:  $C_p/C_v$
- **Prandtl** *float*: Prandtl number of the gas  $Pr = \mu * C_p / \lambda$

### 23.14 Rhot\_gaz\_parfait\_qc

Description: Class for perfect gas used with a quasi-compressible fluid where the state equation is defined as  $\rho = f(T)$ .

See also: `loi_etat_gaz_parfait_base` ([23.7](#))

Usage:

**rhoT\_gaz\_parfait\_QC** *str*

```
Read str {
 cp float
 [prandtl float]
 [rho_xyz champ_base]
 [rho_t str]
 [t_min float]
}
```

where

- **cp** *float*: Specific heat at constant pressure of the gas  $C_p$ .
- **prandtl** *float*: Prandtl number of the gas  $Pr = \mu * C_p / \lambda$
- **rho\_xyz** *champ\_base* ([20.1](#)): Defined with a `Champ_Fonc_xyz` to define a constant  $\rho$  with time (space dependent)
- **rho\_t** *str*: Expression of  $T$  used to calculate  $\rho$ . This can lead to a variable  $\rho$ , both in space and in time.
- **t\_min** *float*: Temperature may, in some cases, locally and temporarily be very small (and negative) even though computation converges. `T_min` keyword allows to set a lower limit of temperature (in Kelvin, -1000 by default). WARNING: DO NOT USE THIS KEYWORD WITHOUT CHECKING CAREFULLY YOUR RESULTS!

### 23.15 Rhot\_gaz\_reel\_qc

Description: Class for real gas state law used with a quasi-compressible fluid.

See also: `loi_etat_gaz_reel_base` ([23.8](#))

Usage:

**rhoT\_gaz\_reel\_QC** **bloc**

where

- **bloc** *bloc\_lecture* ([3.2](#)): Description.

## 24 loi\_fermeture\_base

Description: Class for appends fermeture to problem

Keyword `Discretize` should have already been used to read the object.

See also: `objet_u` ([48](#)) `loi_fermeture_test` ([24.1](#))

Usage:

## 24.1 Loi\_fermeture\_test

Description: Loi for test only

Keyword Discretize should have already been used to read the object.

See also: loi\_fermeture\_base (24)

Usage:

**loi\_fermeture\_test** *str*

**Read** *str* {

    [ **coef** *float*]

}

where

- **coef** *float*: coefficient

## 25 loi\_horaire

Description: to define the movement with a time-dependant law for the solid interface.

See also: objet\_u (48)

Usage:

**loi\_horaire** *str*

**Read** *str* {

**position** *n word1 word2 ... wordn*

**vitesse** *n word1 word2 ... wordn*

    [ **rotation** *n word1 word2 ... wordn*]

    [ **derivee\_rotation** *n word1 word2 ... wordn*]

    [ **verification\_derivee** *int*]

    [ **impr** *int*]

}

where

- **position** *n word1 word2 ... wordn*: Vecteur position
- **vitesse** *n word1 word2 ... wordn*: Vecteur vitesse
- **rotation** *n word1 word2 ... wordn*: Matrice de passage
- **derivee\_rotation** *n word1 word2 ... wordn*: Derivee matrice de passage
- **verification\_derivee** *int*
- **impr** *int*: Whether to print output

## 26 milieu\_base

Description: Basic class for medium (physics properties of medium).

See also: objet\_u (48) constituant (26.5) solide (26.18) fluide\_base (26.6) fluide\_diphasique (26.8)

Usage:

**milieu\_base** *str*

**Read** *str* {



```

[gravite champ_base]
[porosites_champ champ_base]
[diametre_hyd_champ champ_base]
[porosites porosites]
[rho champ_base]
[lambda champ_base]
[cp champ_base]
}
where

```

- **gravite** *champ\_base* (20.1): Gravity field (optional).
- **porosites\_champ** *champ\_base* (20.1): The porosity is given at each element and the porosity at each face,  $\Psi(\text{face})$ , is calculated by the average of the porosities of the two neighbour elements  $\Psi(\text{elem1})$ ,  $\Psi(\text{elem2})$  :  $\Psi(\text{face}) = 2 / (1/\Psi(\text{elem1}) + 1/\Psi(\text{elem2}))$ . This keyword is optional.
- **diametre\_hyd\_champ** *champ\_base* (20.1): Hydraulic diameter field (optional).
- **porosites** *porosites* (34): Porosities.
- **rho** *champ\_base* (20.1): Density (kg.m-3).
- **lambda** *champ\_base* (20.1): Conductivity (W.m-1.K-1).
- **cp** *champ\_base* (20.1): Specific heat (J.kg-1.K-1).

## 26.1 **Fluide\_diphasique\_ijk**

Description: Two-phase fluid.

See also: **fluide\_diphasique** (26.8)

Usage:

**Fluide\_Diphasique\_IJK** *str*

**Read** *str* {

```

sigma champ_don_base
phase0|fluide0 fluide_base
phase1|fluide1 fluide_base
[chaleur_latente champ_don_base]
[formule_mu str]
[gravite champ_base]
[porosites_champ champ_base]
[diametre_hyd_champ champ_base]
[porosites porosites]
[rho champ_base]
[lambda champ_base]
[cp champ_base]

```

}  
where

- **sigma** *champ\_don\_base* (20.9) for inheritance: surfacic tension (J/m2)
- **phase0|fluide0** *fluide\_base* (26.6) for inheritance: first phase fluid
- **phase1|fluide1** *fluide\_base* (26.6) for inheritance: second phase fluid
- **chaleur\_latente** *champ\_don\_base* (20.9) for inheritance: phase changement enthalpy  $h(\text{phase1}_-) - h(\text{phase0}_-)$  (J/kg/K)
- **formule\_mu** *str* for inheritance: (into=[standard,arithmetic,harmonic]) formula used to calculate average
- **gravite** *champ\_base* (20.1) for inheritance

- **porosites\_champ** *champ\_base* (20.1) for inheritance: The porosity is given at each element and the porosity at each face,  $\Psi(\text{face})$ , is calculated by the average of the porosities of the two neighbour elements  $\Psi(\text{elem1})$ ,  $\Psi(\text{elem2})$  :  $\Psi(\text{face})=2/(1/\Psi(\text{elem1})+1/\Psi(\text{elem2}))$ . This keyword is optional.
- **diametre\_hyd\_champ** *champ\_base* (20.1) for inheritance: Hydraulic diameter field (optional).
- **porosites** *porosites* (34) for inheritance: Porosities.
- **rho** *champ\_base* (20.1) for inheritance: Density (kg.m-3).
- **lambda** *champ\_base* (20.1) for inheritance: Conductivity (W.m-1.K-1).
- **cp** *champ\_base* (20.1) for inheritance: Specific heat (J.kg-1.K-1).

## 26.2 Solid\_particle\_base

Description: base particle type for collision model

Keyword Discretize should have already been used to read the object.

See also: *fluide\_incompressible* (26.9) *Solid\_Particle\_sphere* (26.3) *Solid\_Particle\_spheroid* (26.4)

Usage:

**Solid\_Particle\_base** *str*

**Read** *str* {

```

 e_dry float
 [beta_th champ_base]
 [mu champ_base]
 [beta_co champ_base]
 [rho champ_base]
 [cp champ_base]
 [lambda champ_base]
 [porosites bloc_lecture]
 [indice champ_base]
 [kappa champ_base]
 [gravite champ_base]
 [porosites_champ champ_base]
 [diametre_hyd_champ champ_base]

```

}

where

- **e\_dry** *float*: dry coefficient
- **beta\_th** *champ\_base* (20.1) for inheritance: Thermal expansion (K-1).
- **mu** *champ\_base* (20.1) for inheritance: Dynamic viscosity (kg.m-1.s-1).
- **beta\_co** *champ\_base* (20.1) for inheritance: Volume expansion coefficient values in concentration.
- **rho** *champ\_base* (20.1) for inheritance: Density (kg.m-3).
- **cp** *champ\_base* (20.1) for inheritance: Specific heat (J.kg-1.K-1).
- **lambda** *champ\_base* (20.1) for inheritance: Conductivity (W.m-1.K-1).
- **porosites** *bloc\_lecture* (3.2) for inheritance: Porosity (optional)
- **indice** *champ\_base* (20.1) for inheritance: Refractivity of fluid.
- **kappa** *champ\_base* (20.1) for inheritance: Absorptivity of fluid (m-1).
- **gravite** *champ\_base* (20.1) for inheritance: Gravity field (optional).
- **porosites\_champ** *champ\_base* (20.1) for inheritance: The porosity is given at each element and the porosity at each face,  $\Psi(\text{face})$ , is calculated by the average of the porosities of the two neighbour elements  $\Psi(\text{elem1})$ ,  $\Psi(\text{elem2})$  :  $\Psi(\text{face})=2/(1/\Psi(\text{elem1})+1/\Psi(\text{elem2}))$ . This keyword is optional.
- **diametre\_hyd\_champ** *champ\_base* (20.1) for inheritance: Hydraulic diameter field (optional).

## 26.3 Solid\_particle\_sphere

Description: spherical particle for collision model

Keyword Discretize should have already been used to read the object.

See also: Solid\_Particle\_base (26.2)

Usage:

**Solid\_Particle\_sphere** *str*

**Read** *str* {

```
 radius float
 e_dry float
 [beta_th champ_base]
 [mu champ_base]
 [beta_co champ_base]
 [rho champ_base]
 [cp champ_base]
 [lambda champ_base]
 [porosites bloc_lecture]
 [indice champ_base]
 [kappa champ_base]
 [gravite champ_base]
 [porosites_champ champ_base]
 [diametre_hyd_champ champ_base]
```

}

where

- **radius** *float*: radius of a spherical particle
- **e\_dry** *float* for inheritance: dry coefficient
- **beta\_th** *champ\_base* (20.1) for inheritance: Thermal expansion (K-1).
- **mu** *champ\_base* (20.1) for inheritance: Dynamic viscosity (kg.m-1.s-1).
- **beta\_co** *champ\_base* (20.1) for inheritance: Volume expansion coefficient values in concentration.
- **rho** *champ\_base* (20.1) for inheritance: Density (kg.m-3).
- **cp** *champ\_base* (20.1) for inheritance: Specific heat (J.kg-1.K-1).
- **lambda** *champ\_base* (20.1) for inheritance: Conductivity (W.m-1.K-1).
- **porosites** *bloc\_lecture* (3.2) for inheritance: Porosity (optional)
- **indice** *champ\_base* (20.1) for inheritance: Refractivity of fluid.
- **kappa** *champ\_base* (20.1) for inheritance: Absorptivity of fluid (m-1).
- **gravite** *champ\_base* (20.1) for inheritance: Gravity field (optional).
- **porosites\_champ** *champ\_base* (20.1) for inheritance: The porosity is given at each element and the porosity at each face,  $\Psi(\text{face})$ , is calculated by the average of the porosities of the two neighbour elements  $\Psi(\text{elem1})$ ,  $\Psi(\text{elem2})$  :  $\Psi(\text{face}) = 2 / (1/\Psi(\text{elem1}) + 1/\Psi(\text{elem2}))$ . This keyword is optional.
- **diametre\_hyd\_champ** *champ\_base* (20.1) for inheritance: Hydraulic diameter field (optional).

## 26.4 Solid\_particle\_spheroid

Description: spheroid particle for collision model

Keyword Discretize should have already been used to read the object.

See also: Solid\_Particle\_base (26.2)

Usage:

**Solid\_Particle\_spheroid** *str*

**Read** *str* {

```
 half_small_axis float
 half_long_axis float
 e_dry float
 [beta_th champ_base]
 [mu champ_base]
 [beta_co champ_base]
 [rho champ_base]
 [cp champ_base]
 [lambda champ_base]
 [porosites bloc_lecture]
 [indice champ_base]
 [kappa champ_base]
 [gravite champ_base]
 [porosites_champ champ_base]
 [diametre_hyd_champ champ_base]
```

}

where

- **half\_small\_axis** *float*: small half-axis of the spheroid
- **half\_long\_axis** *float*: long half-axis of the spheroid
- **e\_dry** *float* for inheritance: dry coefficient
- **beta\_th** *champ\_base* (20.1) for inheritance: Thermal expansion (K-1).
- **mu** *champ\_base* (20.1) for inheritance: Dynamic viscosity (kg.m-1.s-1).
- **beta\_co** *champ\_base* (20.1) for inheritance: Volume expansion coefficient values in concentration.
- **rho** *champ\_base* (20.1) for inheritance: Density (kg.m-3).
- **cp** *champ\_base* (20.1) for inheritance: Specific heat (J.kg-1.K-1).
- **lambda** *champ\_base* (20.1) for inheritance: Conductivity (W.m-1.K-1).
- **porosites** *bloc\_lecture* (3.2) for inheritance: Porosity (optional)
- **indice** *champ\_base* (20.1) for inheritance: Refractivity of fluid.
- **kappa** *champ\_base* (20.1) for inheritance: Absorptivity of fluid (m-1).
- **gravite** *champ\_base* (20.1) for inheritance: Gravity field (optional).
- **porosites\_champ** *champ\_base* (20.1) for inheritance: The porosity is given at each element and the porosity at each face,  $\Psi(\text{face})$ , is calculated by the average of the porosities of the two neighbour elements  $\Psi(\text{elem1})$ ,  $\Psi(\text{elem2})$  :  $\Psi(\text{face}) = 2 / (1/\Psi(\text{elem1}) + 1/\Psi(\text{elem2}))$ . This keyword is optional.
- **diametre\_hyd\_champ** *champ\_base* (20.1) for inheritance: Hydraulic diameter field (optional).

## 26.5 Constituant

Description: Constituent.

See also: *milieu\_base* (26)

Usage:

**constituant** *str*

**Read** *str* {

```
 [coefficient_diffusion champ_base]
 [is_multi_scalar]
 [gravite champ_base]
```

```

[porosites_champ champ_base]
[diametre_hyd_champ champ_base]
[porosites porosites]
[rho champ_base]
[lambda champ_base]
[cp champ_base]
}
where

```

- **coefficient\_diffusion** *champ\_base* (20.1): Constituent diffusion coefficient value (m<sup>2</sup>.s<sup>-1</sup>). If a multi-constituent problem is being processed, the diffusivity will be a vectorial and each components will be the diffusion of the constituent.
- **is\_multi\_scalar** : Flag to activate the multi\_scalar diffusion operator
- **gravite** *champ\_base* (20.1) for inheritance: Gravity field (optional).
- **porosites\_champ** *champ\_base* (20.1) for inheritance: The porosity is given at each element and the porosity at each face,  $\Psi(\text{face})$ , is calculated by the average of the porosities of the two neighbour elements  $\Psi(\text{elem1})$ ,  $\Psi(\text{elem2})$  :  $\Psi(\text{face}) = 2 / (1/\Psi(\text{elem1}) + 1/\Psi(\text{elem2}))$ . This keyword is optional.
- **diametre\_hyd\_champ** *champ\_base* (20.1) for inheritance: Hydraulic diameter field (optional).
- **porosites** *porosites* (34) for inheritance: Porosities.
- **rho** *champ\_base* (20.1) for inheritance: Density (kg.m<sup>-3</sup>).
- **lambda** *champ\_base* (20.1) for inheritance: Conductivity (W.m<sup>-1</sup>.K<sup>-1</sup>).
- **cp** *champ\_base* (20.1) for inheritance: Specific heat (J.kg<sup>-1</sup>.K<sup>-1</sup>).

## 26.6 Fluide\_base

Description: Basic class for fluids.

Keyword Discretize should have already been used to read the object.

See also: *milieu\_base* (26) *fluide\_incompressible* (26.9) *fluide\_reel\_base* (26.13) *fluide\_dilatable\_base* (26.7)

Usage:

**fluide\_base** *str*

**Read** *str* {

```

[indice champ_base]
[kappa champ_base]
[gravite champ_base]
[porosites_champ champ_base]
[diametre_hyd_champ champ_base]
[porosites porosites]
[rho champ_base]
[lambda champ_base]
[cp champ_base]
}
where

```

- **indice** *champ\_base* (20.1): Refractivity of fluid.
- **kappa** *champ\_base* (20.1): Absorptivity of fluid (m<sup>-1</sup>).
- **gravite** *champ\_base* (20.1) for inheritance: Gravity field (optional).

- **porosites\_champ** *champ\_base* (20.1) for inheritance: The porosity is given at each element and the porosity at each face,  $\Psi(\text{face})$ , is calculated by the average of the porosities of the two neighbour elements  $\Psi(\text{elem1})$ ,  $\Psi(\text{elem2})$  :  $\Psi(\text{face})=2/(1/\Psi(\text{elem1})+1/\Psi(\text{elem2}))$ . This keyword is optional.
- **diametre\_hyd\_champ** *champ\_base* (20.1) for inheritance: Hydraulic diameter field (optional).
- **porosites** *porosites* (34) for inheritance: Porosities.
- **rho** *champ\_base* (20.1) for inheritance: Density (kg.m-3).
- **lambda** *champ\_base* (20.1) for inheritance: Conductivity (W.m-1.K-1).
- **cp** *champ\_base* (20.1) for inheritance: Specific heat (J.kg-1.K-1).

## 26.7 Fluide\_dilatable\_base

Description: Basic class for dilatable fluids.

Keyword Discretize should have already been used to read the object.

See also: *fluide\_base* (26.6) *fluide\_weakly\_compressible* (26.17) *fluide\_quasi\_compressible* (26.11)

Usage:

**fluide\_dilatable\_base** *str*

**Read** *str* {

```
[indice champ_base]
[kappa champ_base]
[gravite champ_base]
[porosites_champ champ_base]
[diametre_hyd_champ champ_base]
[porosites porosites]
[rho champ_base]
[lambda champ_base]
[cp champ_base]
```

}

where

- **indice** *champ\_base* (20.1) for inheritance: Refractivity of fluid.
- **kappa** *champ\_base* (20.1) for inheritance: Absorptivity of fluid (m-1).
- **gravite** *champ\_base* (20.1) for inheritance: Gravity field (optional).
- **porosites\_champ** *champ\_base* (20.1) for inheritance: The porosity is given at each element and the porosity at each face,  $\Psi(\text{face})$ , is calculated by the average of the porosities of the two neighbour elements  $\Psi(\text{elem1})$ ,  $\Psi(\text{elem2})$  :  $\Psi(\text{face})=2/(1/\Psi(\text{elem1})+1/\Psi(\text{elem2}))$ . This keyword is optional.
- **diametre\_hyd\_champ** *champ\_base* (20.1) for inheritance: Hydraulic diameter field (optional).
- **porosites** *porosites* (34) for inheritance: Porosities.
- **rho** *champ\_base* (20.1) for inheritance: Density (kg.m-3).
- **lambda** *champ\_base* (20.1) for inheritance: Conductivity (W.m-1.K-1).
- **cp** *champ\_base* (20.1) for inheritance: Specific heat (J.kg-1.K-1).

## 26.8 Fluide\_diphastique

Description: Two-phase fluid.

See also: *milieu\_base* (26) *Fluide\_Diphastique\_IJK* (26.1)

Usage:

**fluide\_diphasique** *str*

**Read** *str* {

```
 sigma champ_don_base
 phase0fluide0 fluide_base
 phase1fluide1 fluide_base
 [chaleur_latente champ_don_base]
 [formule_mu str]
 [gravite champ_base]
 [porosites_champ champ_base]
 [diametre_hyd_champ champ_base]
 [porosites porosites]
 [rho champ_base]
 [lambda champ_base]
 [cp champ_base]
```

}

where

- **sigma** *champ\_don\_base* (20.9): surfacic tension (J/m2)
- **phase0fluide0** *fluide\_base* (26.6): first phase fluid
- **phase1fluide1** *fluide\_base* (26.6): second phase fluid
- **chaleur\_latente** *champ\_don\_base* (20.9): phase changement enthalpy  $h(\text{phase1}_) - h(\text{phase0}_)$  (J/kg/K)
- **formule\_mu** *str*: (into=[standard,arithmetic,harmonic]) formula used to calculate average
- **gravite** *champ\_base* (20.1)
- **porosites\_champ** *champ\_base* (20.1) for inheritance: The porosity is given at each element and the porosity at each face,  $\Psi(\text{face})$ , is calculated by the average of the porosities of the two neighbour elements  $\Psi(\text{elem1})$ ,  $\Psi(\text{elem2})$ :  $\Psi(\text{face}) = 2 / (1/\Psi(\text{elem1}) + 1/\Psi(\text{elem2}))$ . This keyword is optional.
- **diametre\_hyd\_champ** *champ\_base* (20.1) for inheritance: Hydraulic diameter field (optional).
- **porosites** *porosites* (34) for inheritance: Porosities.
- **rho** *champ\_base* (20.1) for inheritance: Density (kg.m-3).
- **lambda** *champ\_base* (20.1) for inheritance: Conductivity (W.m-1.K-1).
- **cp** *champ\_base* (20.1) for inheritance: Specific heat (J.kg-1.K-1).

## 26.9 Fluide\_incompressible

Description: Class for non-compressible fluids.

Keyword Discretize should have already been used to read the object.

See also: *fluide\_base* (26.6) *fluide\_ostwald* (26.10) *Solid\_Particle\_base* (26.2)

Usage:

**fluide\_incompressible** *str*

**Read** *str* {

```
 [beta_th champ_base]
 [mu champ_base]
 [beta_co champ_base]
 [rho champ_base]
 [cp champ_base]
 [lambda champ_base]
 [porosites bloc_lecture]
```

```

[indice champ_base]
[kappa champ_base]
[gravite champ_base]
[porosites_champ champ_base]
[diametre_hyd_champ champ_base]
}
where

```

- **beta\_th** *champ\_base* (20.1): Thermal expansion (K-1).
- **mu** *champ\_base* (20.1): Dynamic viscosity (kg.m-1.s-1).
- **beta\_co** *champ\_base* (20.1): Volume expansion coefficient values in concentration.
- **rho** *champ\_base* (20.1): Density (kg.m-3).
- **cp** *champ\_base* (20.1): Specific heat (J.kg-1.K-1).
- **lambda** *champ\_base* (20.1): Conductivity (W.m-1.K-1).
- **porosites** *bloc\_lecture* (3.2): Porosity (optional)
- **indice** *champ\_base* (20.1) for inheritance: Refractivity of fluid.
- **kappa** *champ\_base* (20.1) for inheritance: Absorptivity of fluid (m-1).
- **gravite** *champ\_base* (20.1) for inheritance: Gravity field (optional).
- **porosites\_champ** *champ\_base* (20.1) for inheritance: The porosity is given at each element and the porosity at each face, Psi(face), is calculated by the average of the porosities of the two neighbour elements Psi(elem1), Psi(elem2) : Psi(face)=2/(1/Psi(elem1)+1/Psi(elem2)). This keyword is optional.
- **diametre\_hyd\_champ** *champ\_base* (20.1) for inheritance: Hydraulic diameter field (optional).

## 26.10 Fluides\_ostwald

Description: Non-Newtonian fluids governed by Ostwald's law. The law applicable to stress tensor is:

$\tau = K(T) \cdot (D:D)^{((n-1)/2)} \cdot D$  Where:

D refers to the deformation tensor

K refers to fluid consistency (may be a function of the temperature T)

n refers to the fluid structure index  $n=1$  for a Newtonian fluid,  $n<1$  for a rheofluidifier fluid,  $n>1$  for a rheothickening fluid.

Keyword Discretize should have already been used to read the object.

See also: *fluide\_incompressible* (26.9)

Usage:

**fluide\_ostwald** *str*

**Read** *str* {

```

[k champ_base]
[n champ_base]
[beta_th champ_base]
[mu champ_base]
[beta_co champ_base]
[rho champ_base]
[cp champ_base]
[lambda champ_base]
[porosites bloc_lecture]
[indice champ_base]
[kappa champ_base]
[gravite champ_base]
[porosites_champ champ_base]

```



```
[diametre_hyd_champ champ_base]
```

```
}
```

where

- **k** *champ\_base* (20.1): Fluid consistency.
- **n** *champ\_base* (20.1): Fluid structure index.
- **beta\_th** *champ\_base* (20.1) for inheritance: Thermal expansion (K-1).
- **mu** *champ\_base* (20.1) for inheritance: Dynamic viscosity (kg.m-1.s-1).
- **beta\_co** *champ\_base* (20.1) for inheritance: Volume expansion coefficient values in concentration.
- **rho** *champ\_base* (20.1) for inheritance: Density (kg.m-3).
- **cp** *champ\_base* (20.1) for inheritance: Specific heat (J.kg-1.K-1).
- **lambda** *champ\_base* (20.1) for inheritance: Conductivity (W.m-1.K-1).
- **porosites** *bloc\_lecture* (3.2) for inheritance: Porosity (optional)
- **indice** *champ\_base* (20.1) for inheritance: Refractivity of fluid.
- **kappa** *champ\_base* (20.1) for inheritance: Absorptivity of fluid (m-1).
- **gravite** *champ\_base* (20.1) for inheritance: Gravity field (optional).
- **porosites\_champ** *champ\_base* (20.1) for inheritance: The porosity is given at each element and the porosity at each face,  $\Psi(\text{face})$ , is calculated by the average of the porosities of the two neighbour elements  $\Psi(\text{elem1})$ ,  $\Psi(\text{elem2})$  :  $\Psi(\text{face}) = 2 / (1/\Psi(\text{elem1}) + 1/\Psi(\text{elem2}))$ . This keyword is optional.
- **diametre\_hyd\_champ** *champ\_base* (20.1) for inheritance: Hydraulic diameter field (optional).

## 26.11 **Fluide\_quasi\_compressible**

Description: Quasi-compressible flow with a low mach number assumption; this means that the thermodynamic pressure (used in state law) is uniform in space.

Keyword Discretize should have already been used to read the object.

See also: **fluide\_dilatable\_base** (26.7)

Usage:

**fluide\_quasi\_compressible** *str*

**Read** *str* {

```
[sutherland bloc_sutherland]
```

```
[pression float]
```

```
[loi_etat loi_etat_base]
```

```
[traitement_pth str into ['edo', 'constant', 'conservation_masse']]
```

```
[traitement_rho_gravite str into ['standard', 'moins_rho_moyen']]
```

```
[temps_debut_prise_en_compte_drho_dt float]
```

```
[omega_relaxation_drho_dt float]
```

```
[lambda champ_base]
```

```
[mu champ_base]
```

```
[indice champ_base]
```

```
[kappa champ_base]
```

```
[gravite champ_base]
```

```
[porosites_champ champ_base]
```

```
[diametre_hyd_champ champ_base]
```

```
[porosites porosites]
```

```
[rho champ_base]
```

```
[cp champ_base]
```

```
}
```

where

- **sutherland** *bloc\_sutherland* (26.12): Sutherland law for viscosity and for conductivity.
- **pression** *float*: Initial thermo-dynamic pressure used in the associated state law.
- **loi\_etat** *loi\_etat\_base* (23): The state law that will be associated to the Quasi-compressible fluid.
- **traitement\_pth** *str into ['edo', 'constant', 'conservation\_masse']*: Particular treatment for the thermodynamic pressure Pth ; there are three possibilities:
  - 1) with the keyword 'edo' the code computes Pth solving an O.D.E. ; in this case, the mass is not strictly conserved (it is the default case for quasi compressible computation);
  - 2) the keyword 'conservation\_masse' forces the conservation of the mass (closed geometry or with periodic boundaries condition)
  - 3) the keyword 'constant' makes it possible to have a constant Pth ; it's the good choice when the flow is open (e.g. with pressure boundary conditions).
 It is possible to monitor the volume averaged value for temperature and density, plus Pth evolution in the .evol\_glob file.
- **traitement\_rho\_gravite** *str into ['standard', 'moins\_rho\_moyen']*: It may be :1) `standard`: the gravity term is evaluated with  $\rho * g$  (It is the default). 2) `moins_rho_moyen`: the gravity term is evaluated with  $(\rho - \rho_{moy}) * g$ . Unknown pressure is then  $P^* = P + \rho_{moy} * g * z$ . It is useful when you apply uniform pressure boundary condition like  $P^* = 0$ .
- **temps\_debut\_prise\_en\_compte\_drho\_dt** *float*: While  $time < value$ ,  $dRho/dt$  is set to zero (Rho, volumic mass). Useful for some calculation during the first time steps with big variation of temperature and volumic mass.
- **omega\_relaxation\_drho\_dt** *float*: Optional option to have a relaxed algorithm to solve the mass equation. value is used (1 per default) to specify omega.
- **lambda** *champ\_base* (20.1): Conductivity (W.m-1.K-1).
- **mu** *champ\_base* (20.1): Dynamic viscosity (kg.m-1.s-1).
- **indice** *champ\_base* (20.1) for inheritance: Refractivity of fluid.
- **kappa** *champ\_base* (20.1) for inheritance: Absorptivity of fluid (m-1).
- **gravite** *champ\_base* (20.1) for inheritance: Gravity field (optional).
- **porosites\_champ** *champ\_base* (20.1) for inheritance: The porosity is given at each element and the porosity at each face,  $\Psi(\text{face})$ , is calculated by the average of the porosities of the two neighbour elements  $\Psi(\text{elem1})$ ,  $\Psi(\text{elem2})$  :  $\Psi(\text{face}) = 2 / (1/\Psi(\text{elem1}) + 1/\Psi(\text{elem2}))$ . This keyword is optional.
- **diametre\_hyd\_champ** *champ\_base* (20.1) for inheritance: Hydraulic diameter field (optional).
- **porosites** *porosites* (34) for inheritance: Porosities.
- **rho** *champ\_base* (20.1) for inheritance: Density (kg.m-3).
- **cp** *champ\_base* (20.1) for inheritance: Specific heat (J.kg-1.K-1).

## 26.12 Bloc\_sutherland

Description: Sutherland law for viscosity  $\mu(T) = \mu_0 * ((T_0 + C)/(T + C)) * (T/T_0)^{1.5}$  and (optional) for conductivity  $\lambda(T) = \mu_0 * C_p / Prandtl * ((T_0 + S\lambda)/(T + S\lambda)) * (T/T_0)^{1.5}$

See also: [objet\\_lecture \(47\)](#)

Usage:

**problem\_name mu0 mu0\_val t0 t0\_val [ Slambda ] [ s ] C c\_val**

where

- **problem\_name** *str*: Name of problem.
- **mu0** *str into ['mu0']*
- **mu0\_val** *float*
- **t0** *str into ['T0']*
- **t0\_val** *float*
- **Slambda** *str into ['Slambda']*
- **s** *float*

- **C** *str* into ['C']
- **c\_val** *float*

## 26.13 **Fluide\_reel\_base**

Description: Class for real fluids.

Keyword Discretize should have already been used to read the object.

See also: [fluide\\_base \(26.6\)](#) [fluide\\_sodium\\_liquide \(26.15\)](#) [fluide\\_sodium\\_gaz \(26.14\)](#) [fluide\\_stiffened-gas \(26.16\)](#)

Usage:

**fluide\_reel\_base** *str*

**Read** *str* {

- [ **indice** *champ\_base*]
- [ **kappa** *champ\_base*]
- [ **gravite** *champ\_base*]
- [ **porosites\_champ** *champ\_base*]
- [ **diametre\_hyd\_champ** *champ\_base*]
- [ **porosites** *porosites*]
- [ **rho** *champ\_base*]
- [ **lambda** *champ\_base*]
- [ **cp** *champ\_base*]

}

where

- **indice** *champ\_base* [\(20.1\)](#) for inheritance: Refractivity of fluid.
- **kappa** *champ\_base* [\(20.1\)](#) for inheritance: Absorptivity of fluid (m-1).
- **gravite** *champ\_base* [\(20.1\)](#) for inheritance: Gravity field (optional).
- **porosites\_champ** *champ\_base* [\(20.1\)](#) for inheritance: The porosity is given at each element and the porosity at each face,  $\Psi(\text{face})$ , is calculated by the average of the porosities of the two neighbour elements  $\Psi(\text{elem1})$ ,  $\Psi(\text{elem2})$  :  $\Psi(\text{face}) = 2 / (1/\Psi(\text{elem1}) + 1/\Psi(\text{elem2}))$ . This keyword is optional.
- **diametre\_hyd\_champ** *champ\_base* [\(20.1\)](#) for inheritance: Hydraulic diameter field (optional).
- **porosites** *porosites* [\(34\)](#) for inheritance: Porosities.
- **rho** *champ\_base* [\(20.1\)](#) for inheritance: Density (kg.m-3).
- **lambda** *champ\_base* [\(20.1\)](#) for inheritance: Conductivity (W.m-1.K-1).
- **cp** *champ\_base* [\(20.1\)](#) for inheritance: Specific heat (J.kg-1.K-1).

## 26.14 **Fluide\_sodium\_gaz**

Description: Class for Fluide\_sodium\_liquide

Keyword Discretize should have already been used to read the object.

See also: [fluide\\_reel\\_base \(26.13\)](#)

Usage:

**fluide\_sodium\_gaz** *str*

**Read** *str* {

- [ **P\_ref** *float*]
- [ **T\_ref** *float*]

```

[indice champ_base]
[kappa champ_base]
[gravite champ_base]
[porosites_champ champ_base]
[diametre_hyd_champ champ_base]
[porosites porosites]
[rho champ_base]
[lambda champ_base]
[cp champ_base]

```

```

}

```

where

- **P\_ref** *float*: Use to set the pressure value in the closure law. If not specified, the value of the pressure unknown will be used
- **T\_ref** *float*: Use to set the temperature value in the closure law. If not specified, the value of the temperature unknown will be used
- **indice** *champ\_base* (20.1) for inheritance: Refractivity of fluid.
- **kappa** *champ\_base* (20.1) for inheritance: Absorptivity of fluid (m-1).
- **gravite** *champ\_base* (20.1) for inheritance: Gravity field (optional).
- **porosites\_champ** *champ\_base* (20.1) for inheritance: The porosity is given at each element and the porosity at each face,  $\Psi(\text{face})$ , is calculated by the average of the porosities of the two neighbour elements  $\Psi(\text{elem1})$ ,  $\Psi(\text{elem2})$  :  $\Psi(\text{face}) = 2 / (1/\Psi(\text{elem1}) + 1/\Psi(\text{elem2}))$ . This keyword is optional.
- **diametre\_hyd\_champ** *champ\_base* (20.1) for inheritance: Hydraulic diameter field (optional).
- **porosites** *porosites* (34) for inheritance: Porosities.
- **rho** *champ\_base* (20.1) for inheritance: Density (kg.m-3).
- **lambda** *champ\_base* (20.1) for inheritance: Conductivity (W.m-1.K-1).
- **cp** *champ\_base* (20.1) for inheritance: Specific heat (J.kg-1.K-1).

## 26.15 Fluide\_sodium\_liquide

Description: Class for Fluide\_sodium\_liquide

Keyword Discretize should have already been used to read the object.

See also: [fluide\\_reel\\_base](#) (26.13)

Usage:

**fluide\_sodium\_liquide** *str*

**Read** *str* {

```

[P_ref float]
[T_ref float]
[indice champ_base]
[kappa champ_base]
[gravite champ_base]
[porosites_champ champ_base]
[diametre_hyd_champ champ_base]
[porosites porosites]
[rho champ_base]
[lambda champ_base]
[cp champ_base]

```

```

}

```

where

- **P\_ref** *float*: Use to set the pressure value in the closure law. If not specified, the value of the pressure unknown will be used
- **T\_ref** *float*: Use to set the temperature value in the closure law. If not specified, the value of the temperature unknown will be used
- **indice** *champ\_base* (20.1) for inheritance: Refractivity of fluid.
- **kappa** *champ\_base* (20.1) for inheritance: Absorptivity of fluid (m-1).
- **gravite** *champ\_base* (20.1) for inheritance: Gravity field (optional).
- **porosites\_champ** *champ\_base* (20.1) for inheritance: The porosity is given at each element and the porosity at each face,  $\Psi(\text{face})$ , is calculated by the average of the porosities of the two neighbour elements  $\Psi(\text{elem1})$ ,  $\Psi(\text{elem2})$  :  $\Psi(\text{face}) = 2 / (1/\Psi(\text{elem1}) + 1/\Psi(\text{elem2}))$ . This keyword is optional.
- **diametre\_hyd\_champ** *champ\_base* (20.1) for inheritance: Hydraulic diameter field (optional).
- **porosites** *porosites* (34) for inheritance: Porosities.
- **rho** *champ\_base* (20.1) for inheritance: Density (kg.m-3).
- **lambda** *champ\_base* (20.1) for inheritance: Conductivity (W.m-1.K-1).
- **cp** *champ\_base* (20.1) for inheritance: Specific heat (J.kg-1.K-1).

## 26.16 **Fluide\_stiffened\_gas**

Description: Class for Stiffened Gas

Keyword Discretize should have already been used to read the object.

See also: *fluide\_reel\_base* (26.13)

Usage:

**fluide\_stiffened\_gas** *str*

**Read** *str* {

```
[gamma float]
[pinf float]
[mu float]
[lambda float]
[Cv float]
[q float]
[q_prim float]
[indice champ_base]
[kappa champ_base]
[gravite champ_base]
[porosites_champ champ_base]
[diametre_hyd_champ champ_base]
[porosites porosites]
[rho champ_base]
[lambda champ_base]
[cp champ_base]
```

}

where

- **gamma** *float*: Heat capacity ratio ( $C_p/C_v$ )
- **pinf** *float*: Stiffened gas pressure constant (if set to zero, the state law becomes identical to that of perfect gases)
- **mu** *float*: Dynamic viscosity
- **lambda** *float*: Thermal conductivity
- **Cv** *float*: Thermal capacity at constant volume

- **q** *float*: Reference energy
- **q\_prim** *float*: Model constant
- **indice** *champ\_base* (20.1) for inheritance: Refractivity of fluid.
- **kappa** *champ\_base* (20.1) for inheritance: Absorptivity of fluid (m-1).
- **gravite** *champ\_base* (20.1) for inheritance: Gravity field (optional).
- **porosites\_champ** *champ\_base* (20.1) for inheritance: The porosity is given at each element and the porosity at each face,  $\Psi(\text{face})$ , is calculated by the average of the porosities of the two neighbour elements  $\Psi(\text{elem1})$ ,  $\Psi(\text{elem2})$  :  $\Psi(\text{face}) = 2 / (1/\Psi(\text{elem1}) + 1/\Psi(\text{elem2}))$ . This keyword is optional.
- **diametre\_hyd\_champ** *champ\_base* (20.1) for inheritance: Hydraulic diameter field (optional).
- **porosites** *porosites* (34) for inheritance: Porosities.
- **rho** *champ\_base* (20.1) for inheritance: Density (kg.m-3).
- **lambda** *champ\_base* (20.1) for inheritance: Conductivity (W.m-1.K-1).
- **cp** *champ\_base* (20.1) for inheritance: Specific heat (J.kg-1.K-1).

## 26.17 **Fluide\_weakly\_compressible**

Description: Weakly-compressible flow with a low mach number assumption; this means that the thermodynamic pressure (used in state law) can vary in space.

Keyword Discretize should have already been used to read the object.

See also: *fluide\_dilatable\_base* (26.7)

Usage:

**fluide\_weakly\_compressible** *str*

**Read** *str* {

```
[loi_etat loi_etat_base
 [sutherland bloc_sutherland
 [traitement_pth str into ['constant']]
 [lambda champ_base
 [mu champ_base
 [pression_thermo float
 [pression_xyz champ_base
 [use_total_pressure int
 [use_hydrostatic_pressure int
 [use_grad_pression_eos int
 [time_activate_ptot float
 [indice champ_base
 [kappa champ_base
 [gravite champ_base
 [porosites_champ champ_base
 [diametre_hyd_champ champ_base
 [porosites porosites
 [rho champ_base
 [cp champ_base
```

}

where

- **loi\_etat** *loi\_etat\_base* (23): The state law that will be associated to the Weakly-compressible fluid.
- **sutherland** *bloc\_sutherland* (26.12): Sutherland law for viscosity and for conductivity.
- **traitement\_pth** *str* into ['constant']: Particular treatment for the thermodynamic pressure Pth ; there is currently one possibility:

1) the keyword 'constant' makes it possible to have a constant Pth but not uniform in space ; it's the good choice when the flow is open (e.g. with pressure boundary conditions).

- **lambda** *champ\_base* (20.1): Conductivity (W.m-1.K-1).
- **mu** *champ\_base* (20.1): Dynamic viscosity (kg.m-1.s-1).
- **pression\_thermo** *float*: Initial thermo-dynamic pressure used in the associated state law.
- **pression\_xyz** *champ\_base* (20.1): Initial thermo-dynamic pressure used in the associated state law. It should be defined with as a Champ\_Fonc\_xyz.
- **use\_total\_pressure** *int*: Flag (0 or 1) used to activate and use the total pressure in the associated state law. The default value of this Flag is 0.
- **use\_hydrostatic\_pressure** *int*: Flag (0 or 1) used to activate and use the hydro-static pressure in the associated state law. The default value of this Flag is 0.
- **use\_grad\_pression\_eos** *int*: Flag (0 or 1) used to specify whether or not the gradient of the thermo-dynamic pressure will be taken into account in the source term of the temperature equation (case of a non-uniform pressure). The default value of this Flag is 1 which means that the gradient is used in the source.
- **time\_activate\_ptot** *float*: Time (in seconds) at which the total pressure will be used in the associated state law.
- **indice** *champ\_base* (20.1) for inheritance: Refractivity of fluid.
- **kappa** *champ\_base* (20.1) for inheritance: Absorptivity of fluid (m-1).
- **gravite** *champ\_base* (20.1) for inheritance: Gravity field (optional).
- **porosites\_champ** *champ\_base* (20.1) for inheritance: The porosity is given at each element and the porosity at each face, Psi(face), is calculated by the average of the porosities of the two neighbour elements  $\Psi(\text{elem1}), \Psi(\text{elem2}) : \Psi(\text{face}) = 2 / (1/\Psi(\text{elem1}) + 1/\Psi(\text{elem2}))$ . This keyword is optional.
- **diametre\_hyd\_champ** *champ\_base* (20.1) for inheritance: Hydraulic diameter field (optional).
- **porosites** *porosites* (34) for inheritance: Porosities.
- **rho** *champ\_base* (20.1) for inheritance: Density (kg.m-3).
- **cp** *champ\_base* (20.1) for inheritance: Specific heat (J.kg-1.K-1).

## 26.18 Solide

Description: Solid with cp and/or rho non-uniform.

See also: milieu\_base (26)

Usage:

**solide** *str*

**Read** *str* {

```
[rho champ_base]
[cp champ_base]
[lambda champ_base]
[user_field champ_base]
[gravite champ_base]
[porosites_champ champ_base]
[diametre_hyd_champ champ_base]
[porosites porosites]
```

}

where

- **rho** *champ\_base* (20.1): Density (kg.m-3).
- **cp** *champ\_base* (20.1): Specific heat (J.kg-1.K-1).
- **lambda** *champ\_base* (20.1): Conductivity (W.m-1.K-1).

- **user\_field** *champ\_base* (20.1): user defined field.
- **gravite** *champ\_base* (20.1) for inheritance: Gravity field (optional).
- **porosites\_champ** *champ\_base* (20.1) for inheritance: The porosity is given at each element and the porosity at each face,  $\Psi(\text{face})$ , is calculated by the average of the porosities of the two neighbour elements  $\Psi(\text{elem1})$ ,  $\Psi(\text{elem2})$  :  $\Psi(\text{face}) = 2 / (1/\Psi(\text{elem1}) + 1/\Psi(\text{elem2}))$ . This keyword is optional.
- **diametre\_hyd\_champ** *champ\_base* (20.1) for inheritance: Hydraulic diameter field (optional).
- **porosites** *porosites* (34) for inheritance: Porosities.

## 27 milieu\_v2\_base

Description: Basic class for medium (physics properties of medium) composed of constituents (fluids and solids).

See also: *objet\_u* (48)

Usage:

## 28 modele\_rayonnement\_base

Description: Basic class for wall thermal radiation model.

See also: *objet\_u* (48) *modele\_rayonnement\_milieu\_transparent* (28.1)

Usage:

### 28.1 Modele\_rayonnement\_milieu\_transparent

Description: Wall thermal radiation model for a transparent gas and resolving a radiation-conduction-thermohydraulics coupled problem in VDF or VEF.

Keyword Discretize should have already been used to read the object.

See also: *modele\_rayonnement\_base* (28)

Usage:

**modele\_rayonnement\_milieu\_transparent bloc**  
where

- **bloc** *bloc\_lecture* (3.2): *Modele\_Rayonnement\_Milieu\_Transparent* mod  

```

Read mod {
 nom_pb_rayonnant
 problem_name
 fichier_fij
 file_name
 fichier_face_rayo
 file_name
 [fichier_matrice | fichier_matrice_binaire file_name]
}
```

*nom\_pb\_rayonnant problem\_name* : *problem\_name* is the name of the radiating fluid problem  
*fichier\_fij file\_name* : *file\_name* is the name of the file which contains the shape factor matrix between all the faces.  
*fichier\_face\_rayo file\_name* : *file\_name* is the name of the file which contains the radiating faces



characteristics (area, emission value ...)

fichier\_matrice|fichier\_matrice\_binaire file\_name : file\_name is the name of the ASCII (or binary) file which contains the inverted shape factor matrix. It is an optional keyword, if not defined, the inverted shape factor matrix will be calculated and written in a file.

The two first files can be generated by a preprocessor, they allow the radiating face characteristics to be entered (set of faces considered to be uniform with respect to radiation for emission value, flux, etc.) and the form factors for these various faces. These files have the following format:

File on radiating faces:

N M -> N is the number of radiating faces (=edges) and M equals the number of non-zero emission radiating faces

Nom(i) S(i) E(i) -> Name of the edge i, surface area of the edge i -> emission value (between 0 and 1)

Example:

13 4

Gauche 50.0 0.0

Droit1 50.0 0.5

Bas 10.0 0.0

Haut 10.0 0.0

Arriere 5.0 0.0

Avant 5.0 0.0

Droit2 30.0 0.5

Bas1 40.0 0.0

Haut1 20.0 0.0

Avant1 20.0 0.0

Arriere1 20.0 0.0

Entree 20.0 0.5

Sortie 20.0 0.5

File on form factors:

N -> Number of radiating faces

Fij -> Matrix of form factors where i, j between 1 and N

Example:

13

```
1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 0.00 0.24 0.20 0.10 0.10 0.10 0.10 0.16
0.00 0.00 1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.00 0.00 0.00 0.00 1.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
0.00 0.40 0.00 0.00 0.00 0.00 0.00 0.20 0.10 0.10 0.10 0.10 0.00
0.00 0.25 0.00 0.00 0.00 0.00 0.15 0.00 0.15 0.10 0.10 0.15 0.10
0.00 0.25 0.00 0.00 0.00 0.00 0.15 0.30 0.00 0.10 0.10 0.00 0.10
0.00 0.25 0.00 0.00 0.00 0.00 0.15 0.20 0.10 0.00 0.10 0.10 0.10
0.00 0.25 0.00 0.00 0.00 0.00 0.15 0.20 0.10 0.10 0.00 0.10 0.10
0.00 0.25 0.00 0.00 0.00 0.00 0.15 0.30 0.00 0.10 0.10 0.00 0.10
0.00 0.40 0.00 0.00 0.00 0.00 0.00 0.20 0.10 0.10 0.10 0.10 0.00
```

Caution:

a) The radiation model's precision is decided by the user when he/she names the domain edges. In fact, a radiating face is recognised by the preprocessor as the set of domain edges faces bearing the same name. Thus, if the user subdivides the edge into two edges which are named differently, he/she thus creates two radiating faces instead of one.

b) The form factors are entered by the user, the preprocessor carries out no calculations other than checking preservation relationships on form factors.

c) The fluid is considered to be a transparent gas.

## 29 modele\_turbulence\_scal\_base

Description: Basic class for turbulence model for energy equation.

See also: objet\_u (48) schmidt (29.4) prandtl (29.3) null (29.2) sous\_maille\_dyn (29.5)

Usage:

**modele\_turbulence\_scal\_base** *str*

**Read** *str* {

[ **dt\_impr\_nusselt** *float*]

[ **dt\_impr\_nusselt\_mean\_only** *dt\_impr\_nusselt\_mean\_only*]

[ **turbulence\_paro** *turbulence\_paro\_scalaire\_base*]

}

where

- **dt\_impr\_nusselt** *float*: Keyword to print local values of Nusselt number and temperature near a wall during a turbulent calculation. The values will be printed in the \_Nusselt.face file each dt\_impr\_nusselt time period. The local Nusselt expression is as follows :  $Nu = ((\lambda + \lambda_t)/\lambda) * d_{wall}/d_{eq}$  where  $d_{wall}$  is the distance from the first mesh to the wall and  $d_{eq}$  is given by the wall law. This option also gives the value of  $d_{eq}$  and  $h = (\lambda + \lambda_t)/d_{eq}$  and the fluid temperature of the first mesh near the wall.  
For the Neumann boundary conditions (flux\_impose), the «equivalent» wall temperature given by the wall law is also printed (Tparoi equiv.) preceded for VEF calculation by the edge temperature «T face de bord».
- **dt\_impr\_nusselt\_mean\_only** *dt\_impr\_nusselt\_mean\_only* (29.1): This keyword is used to print the mean values of Nusselt ( obtained with the wall laws) on each boundary, into a file named datafile-\_ProblemName\_nusselt\_mean\_only.out. periode refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword boundaries, all the boundaries will be considered. If you use it, you must specify nb\_boundaries which is the number of boundaries on which you want to calculate the mean values, then you have to specify their names.
- **turbulence\_paro** *turbulence\_paro\_scalaire\_base* (45): Keyword to set the wall law.

### 29.1 Dt\_impr\_nusselt\_mean\_only

Description: not\_set

See also: objet\_lecture (47)

Usage:

{

**dt\_impr** *float*

[ **boundaries** *n word1 word2 ... wordn*]

}

where

- **dt\_impr** *float*
- **boundaries** *n word1 word2 ... wordn*

## 29.2 Null

Description: Null scalar turbulence model (turbulent diffusivity = 0) which can be used with a turbulent problem.

See also: modele\_turbulence\_scal\_base (29)

Usage:

**null** *str*

**Read** *str* {

[ **dt\_impr\_nusselt** *float*]

[ **dt\_impr\_nusselt\_mean\_only** *dt\_impr\_nusselt\_mean\_only*]

}

where

- **dt\_impr\_nusselt** *float* for inheritance: Keyword to print local values of Nusselt number and temperature near a wall during a turbulent calculation. The values will be printed in the \_Nusselt.face file each dt\_impr\_nusselt time period. The local Nusselt expression is as follows :  $Nu = ((\lambda + \lambda_t)/\lambda) * d_{wall}/d_{eq}$  where  $d_{wall}$  is the distance from the first mesh to the wall and  $d_{eq}$  is given by the wall law. This option also gives the value of  $d_{eq}$  and  $h = (\lambda + \lambda_t)/d_{eq}$  and the fluid temperature of the first mesh near the wall.

For the Neumann boundary conditions (flux\_impose), the «equivalent» wall temperature given by the wall law is also printed (Tparoi equiv.) preceded for VEF calculation by the edge temperature «T face de bord».

- **dt\_impr\_nusselt\_mean\_only** *dt\_impr\_nusselt\_mean\_only* (29.1) for inheritance: This keyword is used to print the mean values of Nusselt ( obtained with the wall laws) on each boundary, into a file named datafile\_ProblemName\_nusselt\_mean\_only.out. periode refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword boundaries, all the boundaries will be considered. If you use it, you must specify nb\_boundaries which is the number of boundaries on which you want to calculate the mean values, then you have to specify their names.

## 29.3 Prandtl

Description: The Prandtl model. For the scalar equations, only the model based on Reynolds analogy is available. If K\_Epsilon was selected in the hydraulic equation, Prandtl must be selected for the convection-diffusion temperature equation coupled to the hydraulic equation and Schmidt for the concentration equations.

See also: modele\_turbulence\_scal\_base (29)

Usage:

**prandtl** *str*

**Read** *str* {

[ **prdt** *str*]

[ **prandt\_turbulent\_fonction\_nu\_t\_alpha** *str*]

[ **dt\_impr\_nusselt** *float*]

[ **dt\_impr\_nusselt\_mean\_only** *dt\_impr\_nusselt\_mean\_only*]

[ **turbulence\_paro** *turbulence\_paro\_scalaire\_base*]

}

where

- **prdt** *str*: Keyword to modify the constant (Prdt) of Prandtl model :  $\text{Alphat} = \text{Nut} / \text{Prdt}$  Default value is 0.9
- **prandt\_turbulent\_function\_nu\_t\_alpha** *str*: Optional keyword to specify turbulent diffusivity (by default,  $\alpha_t = \text{nu}_t / \text{Prt}$ ) with another formulae, for example:  $\alpha_t = \text{nu}_t^2 / (0.7 * \alpha + 0.85 * \text{nu}_t)$  with the string  $\text{nu}_t * \text{nu}_t / (0.7 * \alpha + 0.85 * \text{nu}_t)$  where  $\alpha$  is the thermal diffusivity.
- **dt\_impr\_nusselt** *float* for inheritance: Keyword to print local values of Nusselt number and temperature near a wall during a turbulent calculation. The values will be printed in the `_Nusselt.face` file each `dt_impr_nusselt` time period. The local Nusselt expression is as follows :  $\text{Nu} = ((\lambda + \lambda_{\text{td}}) / \lambda) * d_{\text{wall}} / d_{\text{eq}}$  where  $d_{\text{wall}}$  is the distance from the first mesh to the wall and  $d_{\text{eq}}$  is given by the wall law. This option also gives the value of  $d_{\text{eq}}$  and  $h = (\lambda + \lambda_{\text{td}}) / d_{\text{eq}}$  and the fluid temperature of the first mesh near the wall.  
For the Neumann boundary conditions (flux\_impose), the «equivalent» wall temperature given by the wall law is also printed (Tparoi equiv.) preceded for VEF calculation by the edge temperature «T face de bord».
- **dt\_impr\_nusselt\_mean\_only** *dt\_impr\_nusselt\_mean\_only* (29.1) for inheritance: This keyword is used to print the mean values of Nusselt ( obtained with the wall laws) on each boundary, into a file named `datafile_ProblemName_nusselt_mean_only.out`. `periode` refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword `boundaries`, all the boundaries will be considered. If you use it, you must specify `nb_boundaries` which is the number of boundaries on which you want to calculate the mean values, then you have to specify their names.
- **turbulence\_pari** *turbulence\_pari\_scalaire\_base* (45) for inheritance: Keyword to set the wall law.

## 29.4 Schmidt

Description: The Schmidt model. For the scalar equations, only the model based on Reynolds analogy is available. If `K_Epsilon` was selected in the hydraulic equation, Schmidt must be selected for the convection-diffusion temperature equation coupled to the hydraulic equation and Schmidt for the concentration equations.

See also: `modele_turbulence_scal_base` (29)

Usage:

**schmidt** *str*

**Read** *str* {

[ **scturb** *float*]  
 [ **dt\_impr\_nusselt** *float*]  
 [ **dt\_impr\_nusselt\_mean\_only** *dt\_impr\_nusselt\_mean\_only*]  
 [ **turbulence\_pari** *turbulence\_pari\_scalaire\_base*]

}

where

- **scturb** *float*: Keyword to modify the constant (Sct) of Schmlidt model :  $\text{Dt} = \text{Nut} / \text{Sct}$  Default value is 0.7.
- **dt\_impr\_nusselt** *float* for inheritance: Keyword to print local values of Nusselt number and temperature near a wall during a turbulent calculation. The values will be printed in the `_Nusselt.face` file each `dt_impr_nusselt` time period. The local Nusselt expression is as follows :  $\text{Nu} = ((\lambda + \lambda_{\text{td}}) / \lambda) * d_{\text{wall}} / d_{\text{eq}}$  where  $d_{\text{wall}}$  is the distance from the first mesh to the wall and  $d_{\text{eq}}$  is given by the wall law. This option also gives the value of  $d_{\text{eq}}$  and  $h = (\lambda + \lambda_{\text{td}}) / d_{\text{eq}}$  and the fluid temperature of the first mesh near the wall.  
For the Neumann boundary conditions (flux\_impose), the «equivalent» wall temperature given by the wall law is also printed (Tparoi equiv.) preceded for VEF calculation by the edge temperature «T face de bord».

- **dt\_impr\_nusselt\_mean\_only** *dt\_impr\_nusselt\_mean\_only* (29.1) for inheritance: This keyword is used to print the mean values of Nusselt ( obtained with the wall laws) on each boundary, into a file named `datafile_ProblemName_nusselt_mean_only.out`. `periode` refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword `boundaries`, all the boundaries will be considered. If you use it, you must specify `nb_boundaries` which is the number of boundaries on which you want to calculate the mean values, then you have to specify their names.
- **turbulence\_paro** *turbulence\_paro\_scalaire\_base* (45) for inheritance: Keyword to set the wall law.

## 29.5 Sous\_maille\_dyn

Description: Dynamic sub-grid turbulence modele.

Warning : Available in VDF only. Not coded in VEF yet.

See also: `modele_turbulence_scal_base` (29)

Usage:

**sous\_maille\_dyn** *str*

**Read** *str* {

```
[stabilise str into ['6_points', 'moy_euler', 'plans_paralleles']]
[nb_points int]
[dt_impr_nusselt float]
[dt_impr_nusselt_mean_only dt_impr_nusselt_mean_only]
[turbulence_paro turbulence_paro_scalaire_base]
```

}

where

- **stabilise** *str* into ['6\_points', 'moy\_euler', 'plans\_paralleles']
- **nb\_points** *int*
- **dt\_impr\_nusselt** *float* for inheritance: Keyword to print local values of Nusselt number and temperature near a wall during a turbulent calculation. The values will be printed in the `_Nusselt.face` file each `dt_impr_nusselt` time period. The local Nusselt expression is as follows :  $Nu = ((\lambda + \lambda_{t})/\lambda) * d_{wall}/d_{eq}$  where `d_wall` is the distance from the first mesh to the wall and `d_eq` is given by the wall law. This option also gives the value of `d_eq` and  $h = (\lambda + \lambda_{t})/\lambda$  and the fluid temperature of the first mesh near the wall.  
For the Neumann boundary conditions (`flux_impose`), the «equivalent» wall temperature given by the wall law is also printed (`Tparoi equiv.`) preceded for VEF calculation by the edge temperature «T face de bord».
- **dt\_impr\_nusselt\_mean\_only** *dt\_impr\_nusselt\_mean\_only* (29.1) for inheritance: This keyword is used to print the mean values of Nusselt ( obtained with the wall laws) on each boundary, into a file named `datafile_ProblemName_nusselt_mean_only.out`. `periode` refers to the printing period, this value is expressed in seconds. If you don't use the optional keyword `boundaries`, all the boundaries will be considered. If you use it, you must specify `nb_boundaries` which is the number of boundaries on which you want to calculate the mean values, then you have to specify their names.
- **turbulence\_paro** *turbulence\_paro\_scalaire\_base* (45) for inheritance: Keyword to set the wall law.

## 30 moyenne\_imposee\_deriv

Description: `not_set`

See also: [objet\\_u \(48\)](#) [profil \(30.5\)](#) [connexion\\_exacte \(30.2\)](#) [connexion\\_approchee \(30.1\)](#) [interpolation \(30.3\)](#) [logarithmique \(30.4\)](#)

Usage:

### 30.1 Connexion\_approchee

Description: To read the imposed field from a file where positions and values are given (it is not necessary that the coordinates of points match the coordinates of the boundary faces, indeed, the nearest point of each face of the boundary will be used).

See also: [moyenne\\_imposee\\_deriv \(30\)](#)

Usage:

**connexion\_approchee fichier file1**

where

- **fichier** *str* into [*'fichier'*]
- **file1** *str*: filename. The format of the file is:  
N  
x(1) y(1) [z(1)] valx(1) valy(1) [valz(1)]  
x(2) y(2) [z(2)] valx(2) valy(2) [valz(2)]  
...  
x(N) y(N) [z(N)] valx(N) valy(N) [valz(N)]

### 30.2 Connexion\_exacte

Description: To read the imposed field from two files.

See also: [moyenne\\_imposee\\_deriv \(30\)](#)

Usage:

**connexion\_exacte fichier file1 [ file2 ]**

where

- **fichier** *str* into [*'fichier'*]
- **file1** *str*: first file, contains the points coordinates (which should be the same as the coordinates of the boundary faces). The format of this file is:  
N  
1 x(1) y(1) [z(1)]  
2 x(2) y(2) [z(2)]  
...  
N x(N) y(N) [z(N)]
- **file2** *str*: second file, contains the mean values. The format of this file is:  
N  
1 valx(1) valy(1) [valz(1)]  
2 valx(2) valy(2) [valz(2)]  
...  
N valx(N) valy(N) [valz(N)]

### 30.3 Interpolation

Synonymous: **champ\_post\_interpolation**

Description: To create an imposed field built by interpolation of values read from a file. The imposed field is applied on the direction given by the keyword `direction_anisotrope` (the field is zero for the other directions).

See also: `moyenne_imposee_deriv` (30)

Usage:

**interpolation fichier file1**

where

- **fichier** *str* into [*'fichier'*]: The format of the file is:  
pos(1) val(1)  
pos(2) val(2)  
...  
pos(N) val(N)  
If direction given by direction  
- `_anisotrope` is 1 (or 2 or 3), then pos will be X (or Y or Z) coordinate and val will be X value (or Y value, or Z value) of the imposed field.
- **file1** *str*: name of `geom_face_perio`

### 30.4 Logarithmique

Description: To specify the imposed field (in this case, velocity) by an analytical logarithmic law of the wall:

$$g(x,y,z) = u\_tau * ( \log(0.5*diametre*u\_tau/visco\_cin)/Kappa + 5.1 )$$

with  $g(x,y,z)=u(x,y,z)$  if direction is set to 1,  $g=v(x,y,z)$  if direction is set to 2 and  $g=w(x,y,z)$  if it is set to 3

See also: `moyenne_imposee_deriv` (30)

Usage:

**logarithmique diametre val u\_tau val\_u\_tau visco\_cin val\_visco\_cin direction val\_direction**

where

- **diametre** *str* into [*'diametre'*]
- **val** *float*: diameter
- **u\_tau** *str* into [*'u\_tau'*]
- **val\_u\_tau** *float*: value of `u_tau`
- **visco\_cin** *str* into [*'visco\_cin'*]
- **val\_visco\_cin** *float*: value of `visco_cin`
- **direction** *str* into [*'direction'*]
- **val\_direction** *int*: direction

### 30.5 Profil

Description: To specify analytic profile for the imposed `g` field.

See also: `moyenne_imposee_deriv` (30)

Usage:

**profil profile**

where

- **profile** *n word1 word2 ... wordn*: specifies the analytic profile: 2|3 valx(x,y,z,t) valy(x,y,z,t) [valz(x,y,z,t)]

## 31 nom

Description: Class to name the TRUST objects.

See also: objet\_u (48) nom\_anonyme (31.1)

Usage:

**nom** [ **mot** ]

where

- **mot** *str*: Chain of characters.

### 31.1 Nom\_anonyme

Description: not\_set

See also: nom (31)

Usage:

[ **mot** ]

where

- **mot** *str*: Chain of characters.

## 32 partitionneur\_deriv

Description: not\_set

See also: objet\_u (48) sous\_zones (32.6) fichier\_med (32.1) metis (32.3) sous\_dom (32.5) tranche (32.7) union (32.8) fichier\_decoupage (32.2) partition (32.4)

Usage:

**partitionneur\_deriv** *str*

**Read** *str* {

    [ **nb\_parts** *int*]

}

where

- **nb\_parts** *int*: The number of non empty parts that must be generated (generally equal to the number of processors in the parallel run).



## 32.1 Fichier\_med

Description: Partitioning a domain using a MED file containing an integer field providing for each element the processor number on which the element should be located.

See also: [partitionneur\\_deriv \(32\)](#)

Usage:

**fichier\_med** *str*

```
Read str {
 file str
 [field str]
 [nb_parts int]
```

```
}
```

where

- **file** *str*: file name of the MED file to load
- **field** *str*: field name of the integer (or double) field to load
- **nb\_parts** *int* for inheritance: The number of non empty parts that must be generated (generally equal to the number of processors in the parallel run).

## 32.2 Fichier\_decoupage

Description: This algorithm reads an array of integer values on the disc, one value for each mesh element. Each value is interpreted as the target part number  $n \geq 0$  for this element. The number of parts created is the highest value in the array plus one. Empty parts can be created if some values are not present in the array.

The file format is ASCII, and contains space, tab or carriage-return separated integer values. The first value is the number `nb_elem` of elements in the domain, followed by `nb_elem` integer values (positive or zero). This algorithm has been designed to work together with the 'ecrire\_decoupage' option. You can generate a partition with any other algorithm, write it to disc, modify it, and read it again to generate the .Zone files. Contrary to other partitioning algorithms, no correction is applied by default to the partition (eg. element 0 on processor 0 and corrections for periodic boundaries). If 'corriger\_partition' is specified, these corrections are applied.

See also: [partitionneur\\_deriv \(32\)](#)

Usage:

**fichier\_decoupage** *str*

```
Read str {
 fichier str
 [corriger_partition]
 [nb_parts int]
```

```
}
```

where

- **fichier** *str*: File name
- **corriger\_partition**
- **nb\_parts** *int* for inheritance: The number of non empty parts that must be generated (generally equal to the number of processors in the parallel run).

### 32.3 Metis

Description: Metis is an external partitionning library. It is a general algorithm that will generate a partition of the domain.

See also: `partitionneur_deriv` ([32](#))

Usage:

**metis** *str*

**Read** *str* {

    [ **kmetis** ]

    [ **use\_weights** ]

    [ **nb\_parts** *int*]

}

where

- **kmetis** : The default values are pmetis, default parameters are automatically chosen by Metis. 'kmetis' is faster than pmetis option but the last option produces better partitioning quality. In both cases, the partitioning quality may be slightly improved by increasing the nb\_essais option (by default N=1). It will compute N partitions and will keep the best one (smallest edge cut number). But this option is CPU expensive, taking N=10 will multiply the CPU cost of partitioning by 10. Experiments show that only marginal improvements can be obtained with non default parameters.
- **use\_weights** : If use\_weights is specified, weighting of the element-element links in the graph is used to force metis to keep opposite periodic elements on the same processor. This option can slightly improve the partitionning quality but it consumes more memory and takes more time. It is not mandatory since a correction algorithm is always applied afterwards to ensure a correct partitionning for periodic boundaries.
- **nb\_parts** *int* for inheritance: The number of non empty parts that must be generated (generally equal to the number of processors in the parallel run).

### 32.4 Partition

Synonymous: **decouper**

Description: This algorithm re-use the partition of the domain named DOMAINE\_NAME. It is useful to partition for example a post processing domain. The partition should match with the calculation domain.

See also: `partitionneur_deriv` ([32](#))

Usage:

**partition** *str*

**Read** *str* {

**domaine** *str*

    [ **nb\_parts** *int*]

}

where

- **domaine** *str*: domain name
- **nb\_parts** *int* for inheritance: The number of non empty parts that must be generated (generally equal to the number of processors in the parallel run).

## 32.5 Sous\_dom

Description: Given a global partition of a global domain, 'sous-domaine' allows to produce a conform partition of a sub-domain generated from the bigger one using the keyword `create_domain_from_sub_domain`. The sub-domain will be partitionned in a conform fashion with the global domain.

See also: `partitionneur_deriv` ([32](#))

Usage:

**sous\_dom** *str*

**Read** *str* {

**fichier** *str*

    [ **fichier\_ssz** *str*]

    [ **name\_ssz** *str*]

    [ **nb\_parts** *int*]

}

where

- **fichier** *str*: fichier
- **fichier\_ssz** *str*: fichier sous zone
- **name\_ssz** *str*: nom sous zone
- **nb\_parts** *int* for inheritance: The number of non empty parts that must be generated (generally equal to the number of processors in the parallel run).

## 32.6 Sous\_zones

Description: This algorithm will create one part for each specified subdomaine/domain. All elements contained in the first subdomaine/domain are put in the first part, all remaining elements contained in the second subdomaine/domain in the second part, etc...

If all elements of the current domain are contained in the specified subdomaines/domain, then N parts are created, otherwise, a supplemental part is created with the remaining elements.

If no subdomaine is specified, all subdomaines defined in the domain are used to split the mesh.

See also: `partitionneur_deriv` ([32](#))

Usage:

**sous\_zones** *str*

**Read** *str* {

    [ **sous\_zones** *n word1 word2 ... wordn*]

    [ **domaines** *n word1 word2 ... wordn*]

    [ **nb\_parts** *int*]

}

where

- **sous\_zones** *n word1 word2 ... wordn*: N SUBZONE\_NAME\_1 SUBZONE\_NAME\_2 ...
- **domaines** *n word1 word2 ... wordn*: N DOMAIN\_NAME\_1 DOMAIN\_NAME\_2 ...
- **nb\_parts** *int* for inheritance: The number of non empty parts that must be generated (generally equal to the number of processors in the parallel run).

## 32.7 Tranche

Description: This algorithm will create a geometrical partitioning by slicing the mesh in the two or three axis directions, based on the geometric center of each mesh element. `nz` must be given if `dimension=3`. Each slice contains the same number of elements (slices don't have the same geometrical width, and for VDF meshes, slice boundaries are generally not flat except if the number of mesh elements in each direction is an exact multiple of the number of slices). First, `nx` slices in the X direction are created, then each slice is split in `ny` slices in the Y direction, and finally, each part is split in `nz` slices in the Z direction. The resulting number of parts is `nx*ny*nz`. If one particular direction has been declared periodic, the default slicing (0, 1, 2, ..., `n-1`) is replaced by (0, 1, 2, ... `n-1`, 0), each of the two '0' slices having twice less elements than the other slices.

See also: `partitionneur_deriv` (32)

Usage:

**tranche** *str*

**Read** *str* {

    [ **tranches** *n1 n2 (n3)*]

    [ **nb\_parts** *int*]

}

where

- **tranches** *n1 n2 (n3)*: Partitioned by `nx` in the X direction, `ny` in the Y direction, `nz` in the Z direction. Works only for structured meshes. No warranty for unstructured meshes.
- **nb\_parts** *int* for inheritance: The number of non empty parts that must be generated (generally equal to the number of processors in the parallel run).

## 32.8 Union

Description: Let several local domains be generated from a bigger one using the keyword `create_domain_from_sub_domain`, and let their partitions be generated in the usual way. Provided the list of partition files for each small domain, the keyword 'union' will partition the global domain in a conform fashion with the smaller domains.

See also: `partitionneur_deriv` (32)

Usage:

**union** *liste* [ **nb\_parts** ]

where

- **liste** *bloc\_lecture* (3.2): List of the partition files with the following syntaxe: {`sous_domaine1 decoupage1 ... sous_domaineim decoupageim` } where `sous_domaine1 ... sous_zomeim` are small domains names and `decoupage1 ... decoupageim` are partition files.
- **nb\_parts** *int*: The number of non empty parts that must be generated (generally equal to the number of processors in the parallel run).

## 33 pb\_champ\_evaluateur

Description: specifies problem name, the field name belonging to the problem and number of field components.

See also: `objet_u` (48)

Usage:

**pb champ ncomp**

where

- **pb** *str*: name of the problem where the source fields will be searched.
- **champ** *str*: name of the field
- **ncomp** *int*: number of components

## 34 porosites

Description: To define the volume porosity and surface porosity that are uniform in every direction in space on a sub-area.

Porosity was only usable in VDF discretization, and now available for VEF P1NC/P0.

Observations :

- Surface porosity values must be given in every direction in space (set this value to 1 if there is no porosity),
  - Prior to defining porosity, the problem must have been discretized.
- Can't be used in VEF discretization, use Porosites\_champ instead.

See also: objet\_u (48)

Usage:

**porosites aco sous\_zone1|sous\_zone bloc [ sous\_zone2 ] [ bloc2 ] acof**

where

- **aco** *str* into ['{']: Opening curly bracket.
- **sous\_zone1|sous\_zone** *str*: Name of the sub-area to which porosity are allocated.
- **bloc** *bloc\_lecture\_poro* (34.1): Surface and volume porosity values.
- **sous\_zone2** *str*: Name of the 2nd sub-area to which porosity are allocated.
- **bloc2** *bloc\_lecture\_poro* (34.1): Surface and volume porosity values.
- **acof** *str* into ['}']: Closing curly bracket.

### 34.1 Bloc\_lecture\_poro

Description: Surface and volume porosity values.

See also: objet\_lecture (47)

Usage:

{

**volumique** *float*  
**surfacique** *n x1 x2 ... xn*

}

where

- **volumique** *float*: Volume porosity value.
- **surfacique** *n x1 x2 ... xn*: Surface porosity values (in X, Y, Z directions).

## 35 precondition\_base

Description: Basic class for preconditioning.

See also: objet\_u (48) ssor\_bloc (35.4) precondsolv (35.2) ssor (35.3) ilu (35.1)

Usage:

### 35.1 Ilu

Description: This preconditionner can be only used with the generic GEN solver.

See also: precondition\_base (35)

Usage:

**ilu** *str*

**Read** *str* {

[ **type** *int*]

[ **filling** *int*]

}

where

- **type** *int*: values can be 0|1|2|3 for null|left|right|left-and-right preconditionning (default value = 2)
- **filling** *int*: default value = 1.

### 35.2 Precondsolv

Description: not\_set

See also: precondition\_base (35)

Usage:

**precondsolv** *solveur*

where

- **solveur** *solveur\_sys\_base* (14.19): Solver type.

### 35.3 Ssor

Description: Symmetric successive over-relaxation algorithm.

See also: precondition\_base (35)

Usage:

**ssor** *str*

**Read** *str* {

[ **omega** *float*]

}

where

- **omega** *float*: Over-relaxation facteur (between 1 and 2, default value 1.6).

## 35.4 Ssor\_bloc

Description: not\_set

See also: [precond\\_base \(35\)](#)

Usage:

**ssor\_bloc** *str*

**Read** *str* {

```
[precond0 precond_base]
[precond1 precond_base]
[preconda precond_base]
[alpha_0 float]
[alpha_1 float]
[alpha_a float]
```

}

where

- **precond0** *precond\_base* ([35](#))
- **precond1** *precond\_base* ([35](#))
- **preconda** *precond\_base* ([35](#))
- **alpha\_0** *float*
- **alpha\_1** *float*
- **alpha\_a** *float*

## 36 preconditionneur\_petsc\_deriv

Description: Preconditioners available with petsc solvers

See also: [objet\\_u \(48\)](#) [diag \(36.6\)](#) [c-amg \(36.5\)](#) [sa-amg \(36.11\)](#) [BLOCK\\_JACOBI\\_ICC \(36.1\)](#) [boomer-amg \(36.4\)](#) [null \(36.9\)](#) [lu \(36.8\)](#) [jacobi \(36.7\)](#) [EISENTAT \(36.2\)](#) [ssor \(36.13\)](#) [block\\_jacobi\\_ilu \(36.3\)](#) [spai \(36.12\)](#) [pilut \(36.10\)](#)

Usage:

### 36.1 Block\_jacobi\_icc

Description: Incomplete Cholesky factorization for symmetric matrix with the PETSc implementation.

See also: [preconditionneur\\_petsc\\_deriv \(36\)](#)

Usage:

**BLOCK\_JACOBI\_ICC** *str*

**Read** *str* {

```
[level int]
[ordering str into ['natural', 'rcm']]
```

}

where

- **level** *int*: factorization level (default value, 1). In parallel, the factorization is done by block (one per processor by default).

- **ordering** *str* into [*natural*, *rcm*]: The ordering of the local matrix is natural by default, but rcm ordering, which reduces the bandwidth of the local matrix, may interestingly improve the quality of the decomposition and reduce the number of iterations.

## 36.2 Eisentat

Description: SSOR version with Eisenstat trick which reduces the number of computations and thus CPU cost...

See also: `preconditionneur_petsc_deriv` (36)

Usage:

**EISENTAT** *str*

**Read** *str* {

    [ **omega** *float*]

}

where

- **omega** *float*: relaxation factor

## 36.3 Block\_jacobi\_ilu

Description: preconditionner

See also: `preconditionneur_petsc_deriv` (36)

Usage:

**block\_jacobi\_ilu** *str*

**Read** *str* {

    [ **level** *int*]

}

where

- **level** *int*

## 36.4 Boomeramg

Description: Multigrid preconditioner (no option is available yet, look at CLI command and Petsc documentation to try other options).

See also: `preconditionneur_petsc_deriv` (36)

Usage:

## 36.5 C-amg

Description: preconditionner

See also: `preconditionneur_petsc_deriv` (36)

Usage:



## 36.6 Diag

Description: Diagonal (Jacobi) preconditioner.

See also: `preconditionneur_petsc_deriv` (36)

Usage:

## 36.7 Jacobi

Description: preconditionner

See also: `preconditionneur_petsc_deriv` (36)

Usage:

## 36.8 Lu

Description: preconditionner

See also: `preconditionneur_petsc_deriv` (36)

Usage:

## 36.9 Null

Description: No preconditioner used

See also: `preconditionneur_petsc_deriv` (36)

Usage:

## 36.10 Pilut

Description: Dual Threashold Incomplete LU factorization.

See also: `preconditionneur_petsc_deriv` (36)

Usage:

**pilut** *str*

**Read** *str* {

    [ **level** *int*]

    [ **epsilon** *float*]

}

where

- **level** *int*: factorization level
- **epsilon** *float*: drop tolerance

### 36.11 Sa-amg

Description: preconditionner

See also: [preconditionneur\\_petsc\\_deriv \(36\)](#)

Usage:

### 36.12 Spai

Description: Spai Approximate Inverse algorithm from Parasails Hypr library.

See also: [preconditionneur\\_petsc\\_deriv \(36\)](#)

Usage:

**spai** *str*

**Read** *str* {

    [ **level** *int*]

    [ **epsilon** *float*]

}

where

- **level** *int*: first parameter
- **epsilon** *float*: second parameter

### 36.13 Ssor

Description: Symmetric Successive Over Relaxation algorithm.

See also: [preconditionneur\\_petsc\\_deriv \(36\)](#)

Usage:

**ssor** *str*

**Read** *str* {

    [ **omega** *float*]

}

where

- **omega** *float*: relaxation factor (default value, 1.5)

## 37 schema\_temps\_base

Description: Basic class for time schemes. This scheme will be associated with a problem and the equations of this problem.

See also: [objet\\_u \(48\)](#) [runge\\_kutta\\_rationnel\\_ordre\\_2 \(37.14\)](#) [schema\\_adams\\_bashforth\\_order\\_2 \(37.15\)](#) [schema\\_implicite\\_base \(37.22\)](#) [schema\\_adams\\_bashforth\\_order\\_3 \(37.16\)](#) [leap\\_frog \(37.5\)](#) [scheme\\_euler\\_explicit \(37.4\)](#) [schema\\_predictor\\_corrector \(37.24\)](#) [runge\\_kutta\\_ordre\\_2 \(37.7\)](#) [runge\\_kutta\\_ordre\\_3 \(37.9\)](#) [runge\\_kutta\\_ordre\\_4\\_d3p \(37.11\)](#) [runge\\_kutta\\_ordre\\_2\\_classique \(37.8\)](#) [runge\\_kutta\\_ordre\\_3\\_classique \(37.10\)](#) [runge\\_kutta\\_ordre\\_4\\_classique \(37.12\)](#) [runge\\_kutta\\_ordre\\_4\\_classique\\_3\\_8 \(37.13\)](#) [Sch\\_CN\\_iteratif \(37.3\)](#) [schema\\_phase\\_field \(37.23\)](#) [schema\\_euler\\_explicite\\_ALE \(37.25\)](#)

Usage:

**schema\_temps\_base** *str*

**Read** *str* {

```
[tinit float]
[tmax float]
[tcpumax float]
[dt_min float]
[dt_max str]
[dt_sauv float]
[nb_sauv_max int]
[dt_impr float]
[facsec str]
[seuil_statio float]
[residuals residuals]
[diffusion_implicite int]
[seuil_diffusion_implicite float]
[impr_diffusion_implicite int]
[impr_extremums int]
[no_error_if_not_converged_diffusion_implicite int]
[no_conv_subiteration_diffusion_implicite int]
[dt_start dt_start]
[nb_pas_dt_max int]
[niter_max_diffusion_implicite int]
[precision_impr int]
[periode_sauvegarde_securite_en_heures float]
[no_check_disk_space]
[disable_progress]
[disable_dt_ev]
[gnuplot_header int]
```

}

where

- **tinit** *float*: Value of initial calculation time (0 by default).
- **tmax** *float*: Time during which the calculation will be stopped (1e30s by default).
- **tcpumax** *float*: CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **dt\_min** *float*: Minimum calculation time step (1e-16s by default).
- **dt\_max** *str*: Maximum calculation time step as function of time (1e30s by default).
- **dt\_sauv** *float*: Save time step value (1e30s by default). Every **dt\_sauv**, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that **dt\_sauv** is in terms of physical time (not cpu time).
- **nb\_sauv\_max** *int*: Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of **nb\_sauv\_max** timesteps in the file), the next checkpoints will overwrite the first ones
- **dt\_impr** *float*: Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **facsec** *str*: Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the **facsec** to 0.5.

Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema\_Adams\_Bashforth\_order\_3.

- **seuil\_statio** *float*: Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives  $dG_i/dt$  of all the unknown transported values  $G_i$  have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **residuals** *residuals* (3.127): To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **diffusion\_implicit** *int*: Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ( $dt=facsec*dt_{convection}$ ). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore  $dt=facsec*dt_{max}$ .
- **seuil\_diffusion\_implicit** *float*: This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **impr\_diffusion\_implicit** *int*: Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **impr\_extremums** *int*: Print unknowns extremas
- **no\_error\_if\_not\_converged\_diffusion\_implicit** *int*
- **no\_conv\_subiteration\_diffusion\_implicit** *int*
- **dt\_start** *dt\_start* (14.11):  $dt_{start} dt_{min}$  : the first iteration is based on  $dt_{min}$ .  
 $dt_{start} dt_{calc}$  : the time step at first iteration is calculated in agreement with CFL condition.  
 $dt_{start} dt_{fixe}$  value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity).  
By default, the first iteration is based on  $dt_{calc}$ .
- **nb\_pas\_dt\_max** *int*: Maximum number of calculation time steps (1e9 by default).
- **niter\_max\_diffusion\_implicit** *int*: This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **precision\_impr** *int*: Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **periode\_sauvegarde\_securite\_en\_heures** *float*: To change the default period (23 hours) between the save of the fields in .sauv file.
- **no\_check\_disk\_space** : To disable the check of the available amount of disk space during the calculation.
- **disable\_progress** : To disable the writing of the .progress file.
- **disable\_dt\_ev** : To disable the writing of the .dt\_ev file.
- **gnuplot\_header** *int*: Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

### 37.1 Implicit\_euler\_steady\_scheme

Synonymous: **schema\_euler\_implicit\_stationnaire**

Description: This is the Implicit Euler scheme using a dual time step procedure (using local and global  $dt$ ) for steady problems. Remark: the only possible solver choice for this scheme is the implicit\_steady solver.

See also: **schema\_implicit\_base** (37.22)

Usage:

**implicit\_euler\_steady\_scheme** *str*

**Read** *str* {

```
[max_iter_implicite int]
[steady_security_facteur float]
[steady_global_dt float]
solveur solveur_implicite_base
[tinit float]
[tmax float]
[tcpumax float]
[dt_min float]
[dt_max str]
[dt_sauv float]
[nb_sauv_max int]
[dt_impr float]
[facsec str]
[seuil_statio float]
[residuals residuals]
[diffusion_implicite int]
[seuil_diffusion_implicite float]
[impr_diffusion_implicite int]
[impr_extremums int]
[no_error_if_not_converged_diffusion_implicite int]
[no_conv_subiteration_diffusion_implicite int]
[dt_start dt_start]
[nb_pas_dt_max int]
[niter_max_diffusion_implicite int]
[precision_impr int]
[periode_sauvegarde_securite_en_heures float]
[no_check_disk_space]
[disable_progress]
[disable_dt_ev]
[gnuplot_header int]
```

}

where

- **max\_iter\_implicite** *int*: Maximum number of iterations allowed for the solver (by default 200)
- **steady\_security\_facteur** *float*: Parameter used in the local time step calculation procedure in order to increase or decrease the local dt value (by default 0.5). We expect a strictly positive value
- **steady\_global\_dt** *float*: This is the global time step used in the dual time step algorithm (by default 100). We expect a strictly positive value
- **solveur** *solveur\_implicite\_base* (39) for inheritance: This keyword is used to designate the solver selected in the situation where the time scheme is an implicit scheme. *solver* is the name of the solver that allows equation diffusion and convection operators to be set as implicit terms. Keywords corresponding to this functionality are Simple (SIMPLE type algorithm), Simpler (SIMPLER type algorithm) for incompressible systems, Piso (Pressure Implicit with Split Operator), and Implicite (similar to PISO, but as it looks like a simplified solver, it will use fewer timesteps, and ICE (for PB\_multiphase). But it may run faster because the pressure matrix is not re-assembled and thus provides CPU gains.

Advice: Since the 1.6.0 version, we recommend to use first the Implicite or Simple, then Piso, and at least Simpler. Because the two first give a fastest convergence (several times) than Piso and the Simpler has not been validated. It seems also than Implicite and Piso schemes give better results than the Simple scheme when the flow is not fully stationary. Thus, if the solution obtained with Simple is not stationary, it is recommended to switch to Piso or Implicite scheme.

- **tinit** *float* for inheritance: Value of initial calculation time (0 by default).
- **tmax** *float* for inheritance: Time during which the calculation will be stopped (1e30s by default).
- **tcpumax** *float* for inheritance: CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **dt\_min** *float* for inheritance: Minimum calculation time step (1e-16s by default).
- **dt\_max** *str* for inheritance: Maximum calculation time step as function of time (1e30s by default).
- **dt\_sauv** *float* for inheritance: Save time step value (1e30s by default). Every **dt\_sauv**, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that **dt\_sauv** is in terms of physical time (not cpu time).
- **nb\_sauv\_max** *int* for inheritance: Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of **nb\_sauv\_max** timesteps in the file), the next checkpoints will overwrite the first ones
- **dt\_impr** *float* for inheritance: Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **facsec** *str* for inheritance: Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the **facsec** to 0.5.  
Warning: Some schemes needs a **facsec** lower than 1 (0.5 is a good start), for example Schema\_Adams\_Bashforth\_order\_3.
- **seuil\_statio** *float* for inheritance: Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives  $dG_i/dt$  of all the unknown transported values  $G_i$  have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **residuals** *residuals* (3.127) for inheritance: To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **diffusion\_implicit** *int* for inheritance: Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ( $dt=facsec*dt_{convection}$ ). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large **facsec** value. Start with a **facsec** value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore  $dt=facsec*dt_{max}$ .
- **seuil\_diffusion\_implicit** *float* for inheritance: This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **impr\_diffusion\_implicit** *int* for inheritance: Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **impr\_extremums** *int* for inheritance: Print unknowns extremas
- **no\_error\_if\_not\_converged\_diffusion\_implicit** *int* for inheritance
- **no\_conv\_subiteration\_diffusion\_implicit** *int* for inheritance
- **dt\_start** *dt\_start* (14.11) for inheritance: **dt\_start dt\_min** : the first iteration is based on **dt\_min**.  
**dt\_start dt\_calc** : the time step at first iteration is calculated in agreement with CFL condition.  
**dt\_start dt\_fixe** value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity).  
By default, the first iteration is based on **dt\_calc**.
- **nb\_pas\_dt\_max** *int* for inheritance: Maximum number of calculation time steps (1e9 by default).
- **niter\_max\_diffusion\_implicit** *int* for inheritance: This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.

- **precision\_impr** *int* for inheritance: Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **periode\_sauvegarde\_securite\_en\_heures** *float* for inheritance: To change the default period (23 hours) between the save of the fields in .sauv file.
- **no\_check\_disk\_space** for inheritance: To disable the check of the available amount of disk space during the calculation.
- **disable\_progress** for inheritance: To disable the writing of the .progress file.
- **disable\_dt\_ev** for inheritance: To disable the writing of the .dt\_ev file.
- **gnuplot\_header** *int* for inheritance: Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

## 37.2 Sch\_cn\_ex\_iteratif

Description: This keyword also describes a Crank-Nicholson method of second order accuracy but here, for scalars, because of instabilities encountered when  $dt > dt_{CFL}$ , the Crank Nicholson scheme is not applied to scalar quantities. Scalars are treated according to Euler-Explicite scheme at the end of the CN treatment for velocity flow fields (by doing  $p$  Euler explicite under-iterations at  $dt \leq dt_{CFL}$ ). Parameters are the same (but default values may change) compare to the Sch\_CN\_iterative scheme plus a relaxation keyword: *niter\_min* (2 by default), *niter\_max* (6 by default), *niter\_avg* (3 by default), *facsec\_max* (20 by default), *seuil* (0.05 by default)

See also: Sch\_CN\_iteratif ([37.3](#))

Usage:

**Sch\_CN\_EX\_iteratif** *str*

**Read** *str* {

```
[omega float]
[seuil float]
[niter_min int]
[niter_max int]
[niter_avg int]
[facsec_max float]
[tinit float]
[tmax float]
[tcpumax float]
[dt_min float]
[dt_max str]
[dt_sauv float]
[nb_sauv_max int]
[dt_impr float]
[facsec str]
[seuil_statio float]
[residuals residuals]
[diffusion_implicite int]
[seuil_diffusion_implicite float]
[impr_diffusion_implicite int]
[impr_extremums int]
[no_error_if_not_converged_diffusion_implicite int]
[no_conv_subiteration_diffusion_implicite int]
[dt_start dt_start]
[nb_pas_dt_max int]
[niter_max_diffusion_implicite int]
```

```

[precision_impr int]
[periode_sauvegarde_securite_en_heures float]
[no_check_disk_space]
[disable_progress]
[disable_dt_ev]
[gnuplot_header int]
}
where

```

- **omega** *float*: relaxation factor (0.1 by default)
- **seuil** *float* for inheritance: criteria for ending iterative process ( $\text{Max}(\|u(p) - u(p-1)\|/\text{Max}\|u(p)\|) < \text{seuil}$ ) (0.001 by default)
- **niter\_min** *int* for inheritance: minimal number of p-iterations to satisfy convergence criteria (2 by default)
- **niter\_max** *int* for inheritance: number of maximum p-iterations allowed to satisfy convergence criteria (6 by default)
- **niter\_avg** *int* for inheritance: threshold of p-iterations (3 by default). If the number of p-iterations is greater than niter\_avg, facsec is reduced, if lesser than niter\_avg, facsec is increased (but limited by the facsec\_max value).
- **facsec\_max** *float* for inheritance: maximum ratio allowed between dynamical time step returned by iterative process and stability time returned by CFL condition (2 by default).
- **tinit** *float* for inheritance: Value of initial calculation time (0 by default).
- **tmax** *float* for inheritance: Time during which the calculation will be stopped (1e30s by default).
- **tcumax** *float* for inheritance: CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **dt\_min** *float* for inheritance: Minimum calculation time step (1e-16s by default).
- **dt\_max** *str* for inheritance: Maximum calculation time step as function of time (1e30s by default).
- **dt\_sauv** *float* for inheritance: Save time step value (1e30s by default). Every dt\_sauv, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that dt\_sauv is in terms of physical time (not cpu time).
- **nb\_sauv\_max** *int* for inheritance: Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of nb\_sauv\_max timesteps in the file), the next checkpoints will overwrite the first ones
- **dt\_impr** *float* for inheritance: Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **facsec** *str* for inheritance: Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5.  
Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema-Adams-Bashforth\_order\_3.
- **seuil\_statio** *float* for inheritance: Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives  $dG_i/dt$  of all the unknown transported values  $G_i$  have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **residuals** *residuals* (3.127) for inheritance: To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **diffusion\_implicit** *int* for inheritance: Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ( $dt = \text{facsec} * dt_{\text{convection}}$ ). Thus, in some circumstances, an important



gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore  $dt = facsec * dt\_max$ .

- **seuil\_diffusion\_implicit** *float* for inheritance: This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **impr\_diffusion\_implicit** *int* for inheritance: Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **impr\_extremums** *int* for inheritance: Print unknowns extremas
- **no\_error\_if\_not\_converged\_diffusion\_implicit** *int* for inheritance
- **no\_conv\_subiteration\_diffusion\_implicit** *int* for inheritance
- **dt\_start** *dt\_start* (14.11) for inheritance:  $dt\_start$   $dt\_min$  : the first iteration is based on  $dt\_min$ .  
 $dt\_start$   $dt\_calc$  : the time step at first iteration is calculated in agreement with CFL condition.  
 $dt\_start$   $dt\_fixe$  value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity).  
By default, the first iteration is based on  $dt\_calc$ .
- **nb\_pas\_dt\_max** *int* for inheritance: Maximum number of calculation time steps (1e9 by default).
- **niter\_max\_diffusion\_implicit** *int* for inheritance: This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **precision\_impr** *int* for inheritance: Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **periode\_sauvegarde\_securite\_en\_heures** *float* for inheritance: To change the default period (23 hours) between the save of the fields in .sauv file.
- **no\_check\_disk\_space** for inheritance: To disable the check of the available amount of disk space during the calculation.
- **disable\_progress** for inheritance: To disable the writing of the .progress file.
- **disable\_dt\_ev** for inheritance: To disable the writing of the .dt\_ev file.
- **gnuplot\_header** *int* for inheritance: Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

### 37.3 Sch\_cn\_iteratif

Description: The Crank-Nicholson method of second order accuracy. A mid-point rule formulation is used (Euler-centered scheme). The basic scheme is:

$$u(t + 1) = u(t) + du/dt(t + 1/2) * dt$$

The estimation of the time derivative  $du/dt$  at the level  $(t+1/2)$  is obtained either by iterative process. The time derivative  $du/dt$  at the level  $(t+1/2)$  is calculated iteratively with a simple under-relaxations method. Since the method is implicit, neither the cfl nor the fourier stability criteria must be respected. The time step is calculated in a way that the iterative procedure converges with the less iterations as possible.

Remark : for stationary or RANS calculations, no limitation can be given for time step through high value of facsec\_max parameter (for instance : facsec\_max 1000). In counterpart, for LES calculations, high values of facsec\_max may engender numerical instabilities.

See also: schema\_temps\_base (37) Sch\_CN\_EX\_iteratif (37.2)

Usage:

**Sch\_CN\_iteratif** *str*

**Read** *str* {

[ **seuil** *float*]

```

[niter_min int]
[niter_max int]
[niter_avg int]
[facsec_max float]
[tinit float]
[tmax float]
[tcpumax float]
[dt_min float]
[dt_max str]
[dt_sauv float]
[nb_sauv_max int]
[dt_impr float]
[facsec str]
[seuil_statio float]
[residuals residuals]
[diffusion_implicite int]
[seuil_diffusion_implicite float]
[impr_diffusion_implicite int]
[impr_extremums int]
[no_error_if_not_converged_diffusion_implicite int]
[no_conv_subiteration_diffusion_implicite int]
[dt_start dt_start]
[nb_pas_dt_max int]
[niter_max_diffusion_implicite int]
[precision_impr int]
[periode_sauvegarde_securite_en_heures float]
[no_check_disk_space]
[disable_progress]
[disable_dt_ev]
[gnuplot_header int]
}

```

where

- **seuil** *float*: criteria for ending iterative process ( $\text{Max}(\|u(p) - u(p-1)\| / \text{Max} \|u(p)\|) < \text{seuil}$ ) (0.001 by default)
- **niter\_min** *int*: minimal number of p-iterations to satisfy convergence criteria (2 by default)
- **niter\_max** *int*: number of maximum p-iterations allowed to satisfy convergence criteria (6 by default)
- **niter\_avg** *int*: threshold of p-iterations (3 by default). If the number of p-iterations is greater than **niter\_avg**, **facsec** is reduced, if lesser than **niter\_avg**, **facsec** is increased (but limited by the **facsec\_max** value).
- **facsec\_max** *float*: maximum ratio allowed between dynamical time step returned by iterative process and stability time returned by CFL condition (2 by default).
- **tinit** *float* for inheritance: Value of initial calculation time (0 by default).
- **tmax** *float* for inheritance: Time during which the calculation will be stopped (1e30s by default).
- **tcpumax** *float* for inheritance: CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **dt\_min** *float* for inheritance: Minimum calculation time step (1e-16s by default).
- **dt\_max** *str* for inheritance: Maximum calculation time step as function of time (1e30s by default).
- **dt\_sauv** *float* for inheritance: Save time step value (1e30s by default). Every **dt\_sauv**, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that **dt\_sauv** is in terms of physical time (not cpu time).

- **nb\_sauv\_max** *int* for inheritance: Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of nb\_sauv\_max timesteps in the file), the next checkpoints will overwrite the first ones
- **dt\_impr** *float* for inheritance: Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **facsec** *str* for inheritance: Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5.  
Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema\_Adams\_Bashforth\_order\_3.
- **seuil\_statio** *float* for inheritance: Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives  $dG_i/dt$  of all the unknown transported values  $G_i$  have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **residuals** *residuals* (3.127) for inheritance: To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **diffusion\_implicit** *int* for inheritance: Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ( $dt=facsec*dt_{convection}$ ). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore  $dt=facsec*dt_{max}$ .
- **seuil\_diffusion\_implicit** *float* for inheritance: This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **impr\_diffusion\_implicit** *int* for inheritance: Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **impr\_extremums** *int* for inheritance: Print unknowns extremas
- **no\_error\_if\_not\_converged\_diffusion\_implicit** *int* for inheritance
- **no\_conv\_subiteration\_diffusion\_implicit** *int* for inheritance
- **dt\_start** *dt\_start* (14.11) for inheritance:  $dt_{start}$   $dt_{min}$  : the first iteration is based on  $dt_{min}$ .  
 $dt_{start}$   $dt_{calc}$  : the time step at first iteration is calculated in agreement with CFL condition.  
 $dt_{start}$   $dt_{fixe}$  value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity).  
By default, the first iteration is based on  $dt_{calc}$ .
- **nb\_pas\_dt\_max** *int* for inheritance: Maximum number of calculation time steps (1e9 by default).
- **niter\_max\_diffusion\_implicit** *int* for inheritance: This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **precision\_impr** *int* for inheritance: Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **periode\_sauvegarde\_securite\_en\_heures** *float* for inheritance: To change the default period (23 hours) between the save of the fields in .sauv file.
- **no\_check\_disk\_space** for inheritance: To disable the check of the available amount of disk space during the calculation.
- **disable\_progress** for inheritance: To disable the writing of the .progress file.
- **disable\_dt\_ev** for inheritance: To disable the writing of the .dt\_ev file.
- **gnuplot\_header** *int* for inheritance: Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

## 37.4 Scheme\_euler\_explicit

Synonymous: **schema\_euler\_explicite**

Description: This is the Euler explicit scheme.

See also: `schema_temps_base` (37)

Usage:

**scheme\_euler\_explicit** *str*

**Read** *str* {

```
[tinit float]
[tmax float]
[tcpumax float]
[dt_min float]
[dt_max str]
[dt_sauv float]
[nb_sauv_max int]
[dt_impr float]
[facsec str]
[seuil_statio float]
[residuals residuals]
[diffusion_implicite int]
[seuil_diffusion_implicite float]
[impr_diffusion_implicite int]
[impr_extremums int]
[no_error_if_not_converged_diffusion_implicite int]
[no_conv_subiteration_diffusion_implicite int]
[dt_start dt_start]
[nb_pas_dt_max int]
[niter_max_diffusion_implicite int]
[precision_impr int]
[periode_sauvegarde_securite_en_heures float]
[no_check_disk_space]
[disable_progress]
[disable_dt_ev]
[gnuplot_header int]
```

}

where

- **tinit** *float* for inheritance: Value of initial calculation time (0 by default).
- **tmax** *float* for inheritance: Time during which the calculation will be stopped (1e30s by default).
- **tcpumax** *float* for inheritance: CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **dt\_min** *float* for inheritance: Minimum calculation time step (1e-16s by default).
- **dt\_max** *str* for inheritance: Maximum calculation time step as function of time (1e30s by default).
- **dt\_sauv** *float* for inheritance: Save time step value (1e30s by default). Every `dt_sauv`, fields are saved in the `.sauv` file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the `.sauv` files, you must specify 0. Note that `dt_sauv` is in terms of physical time (not cpu time).
- **nb\_sauv\_max** *int* for inheritance: Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is

reached (ie we already stored the data of nb\_sauv\_max timesteps in the file), the next checkpoints will overwrite the first ones

- **dt\_impr** *float* for inheritance: Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **facsec** *str* for inheritance: Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5.  
Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema\_Adams\_Bashforth\_order\_3.
- **seuil\_statio** *float* for inheritance: Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives  $dG_i/dt$  of all the unknown transported values  $G_i$  have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **residuals** *residuals* (3.127) for inheritance: To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **diffusion\_implicite** *int* for inheritance: Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ( $dt=facsec*dt_{convection}$ ). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore  $dt=facsec*dt_{max}$ .
- **seuil\_diffusion\_implicite** *float* for inheritance: This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **impr\_diffusion\_implicite** *int* for inheritance: Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **impr\_extremums** *int* for inheritance: Print unknowns extremas
- **no\_error\_if\_not\_converged\_diffusion\_implicite** *int* for inheritance
- **no\_conv\_subiteration\_diffusion\_implicite** *int* for inheritance
- **dt\_start** *dt\_start* (14.11) for inheritance:  $dt_{start} dt_{min}$  : the first iteration is based on  $dt_{min}$ .  
 $dt_{start} dt_{calc}$  : the time step at first iteration is calculated in agreement with CFL condition.  
 $dt_{start} dt_{fixe}$  value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity).  
By default, the first iteration is based on  $dt_{calc}$ .
- **nb\_pas\_dt\_max** *int* for inheritance: Maximum number of calculation time steps (1e9 by default).
- **niter\_max\_diffusion\_implicite** *int* for inheritance: This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **precision\_impr** *int* for inheritance: Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **periode\_sauvegarde\_securite\_en\_heures** *float* for inheritance: To change the default period (23 hours) between the save of the fields in .sauv file.
- **no\_check\_disk\_space** for inheritance: To disable the check of the available amount of disk space during the calculation.
- **disable\_progress** for inheritance: To disable the writing of the .progress file.
- **disable\_dt\_ev** for inheritance: To disable the writing of the .dt\_ev file.
- **gnuplot\_header** *int* for inheritance: Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

## 37.5 Leap\_frog

Description: This is the leap-frog scheme.

See also: `schema_temps_base` (37)

Usage:

**leap\_frog** *str*

**Read** *str* {

```
[tinit float]
[tmax float]
[tcpumax float]
[dt_min float]
[dt_max str]
[dt_sauv float]
[nb_sauv_max int]
[dt_impr float]
[facsec str]
[seuil_statio float]
[residuals residuals]
[diffusion_implicite int]
[seuil_diffusion_implicite float]
[impr_diffusion_implicite int]
[impr_extremums int]
[no_error_if_not_converged_diffusion_implicite int]
[no_conv_subiteration_diffusion_implicite int]
[dt_start dt_start]
[nb_pas_dt_max int]
[niter_max_diffusion_implicite int]
[precision_impr int]
[periode_sauvegarde_securite_en_heures float]
[no_check_disk_space]
[disable_progress]
[disable_dt_ev]
[gnuplot_header int]
```

}

where

- **tinit** *float* for inheritance: Value of initial calculation time (0 by default).
- **tmax** *float* for inheritance: Time during which the calculation will be stopped (1e30s by default).
- **tcpumax** *float* for inheritance: CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **dt\_min** *float* for inheritance: Minimum calculation time step (1e-16s by default).
- **dt\_max** *str* for inheritance: Maximum calculation time step as function of time (1e30s by default).
- **dt\_sauv** *float* for inheritance: Save time step value (1e30s by default). Every `dt_sauv`, fields are saved in the `.sauv` file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the `.sauv` files, you must specify 0. Note that `dt_sauv` is in terms of physical time (not cpu time).
- **nb\_sauv\_max** *int* for inheritance: Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of `nb_sauv_max` timesteps in the file), the next checkpoints will overwrite the first ones

- **dt\_impr** *float* for inheritance: Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **facsec** *str* for inheritance: Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5.  
Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema\_Adams\_Bashforth\_order\_3.
- **seuil\_statio** *float* for inheritance: Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives  $dG_i/dt$  of all the unknown transported values  $G_i$  have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **residuals** *residuals* (3.127) for inheritance: To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **diffusion\_implicit** *int* for inheritance: Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ( $dt=facsec*dt_{convection}$ ). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore  $dt=facsec*dt_{max}$ .
- **seuil\_diffusion\_implicit** *float* for inheritance: This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **impr\_diffusion\_implicit** *int* for inheritance: Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **impr\_extremums** *int* for inheritance: Print unknowns extremas
- **no\_error\_if\_not\_converged\_diffusion\_implicit** *int* for inheritance
- **no\_conv\_subiteration\_diffusion\_implicit** *int* for inheritance
- **dt\_start** *dt\_start* (14.11) for inheritance:  $dt\_start$   $dt\_min$  : the first iteration is based on  $dt\_min$ .  
 $dt\_start$   $dt\_calc$  : the time step at first iteration is calculated in agreement with CFL condition.  
 $dt\_start$   $dt\_fixe$  value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity).  
By default, the first iteration is based on  $dt\_calc$ .
- **nb\_pas\_dt\_max** *int* for inheritance: Maximum number of calculation time steps (1e9 by default).
- **niter\_max\_diffusion\_implicit** *int* for inheritance: This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **precision\_impr** *int* for inheritance: Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **periode\_sauvegarde\_securite\_en\_heures** *float* for inheritance: To change the default period (23 hours) between the save of the fields in .sauv file.
- **no\_check\_disk\_space** for inheritance: To disable the check of the available amount of disk space during the calculation.
- **disable\_progress** for inheritance: To disable the writing of the .progress file.
- **disable\_dt\_ev** for inheritance: To disable the writing of the .dt\_ev file.
- **gnuplot\_header** *int* for inheritance: Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

## 37.6 Rk3\_ft

Description: Keyword for Runge Kutta time scheme for Front\_Tracking calculation.

See also: `runge_kutta_ordre_3` ([37.9](#))

Usage:

**rk3\_ft** *str*

**Read** *str* {

```
[tinit float]
[tmax float]
[tcpumax float]
[dt_min float]
[dt_max str]
[dt_sauv float]
[nb_sauv_max int]
[dt_impr float]
[facsec str]
[seuil_statio float]
[residuals residuals]
[diffusion_implicite int]
[seuil_diffusion_implicite float]
[impr_diffusion_implicite int]
[impr_extremums int]
[no_error_if_not_converged_diffusion_implicite int]
[no_conv_subiteration_diffusion_implicite int]
[dt_start dt_start]
[nb_pas_dt_max int]
[niter_max_diffusion_implicite int]
[precision_impr int]
[periode_sauvegarde_securite_en_heures float]
[no_check_disk_space]
[disable_progress]
[disable_dt_ev]
[gnuplot_header int]
```

}

where

- **tinit** *float* for inheritance: Value of initial calculation time (0 by default).
- **tmax** *float* for inheritance: Time during which the calculation will be stopped (1e30s by default).
- **tcpumax** *float* for inheritance: CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **dt\_min** *float* for inheritance: Minimum calculation time step (1e-16s by default).
- **dt\_max** *str* for inheritance: Maximum calculation time step as function of time (1e30s by default).
- **dt\_sauv** *float* for inheritance: Save time step value (1e30s by default). Every `dt_sauv`, fields are saved in the `.sauv` file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the `.sauv` files, you must specify 0. Note that `dt_sauv` is in terms of physical time (not cpu time).
- **nb\_sauv\_max** *int* for inheritance: Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of `nb_sauv_max` timesteps in the file), the next checkpoints will overwrite the first ones



- **dt\_impr** *float* for inheritance: Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **facsec** *str* for inheritance: Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5.  
Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema\_Adams\_Bashforth\_order\_3.
- **seuil\_statio** *float* for inheritance: Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives  $dG_i/dt$  of all the unknown transported values  $G_i$  have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **residuals** *residuals* (3.127) for inheritance: To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **diffusion\_implicit** *int* for inheritance: Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ( $dt=facsec*dt_{convection}$ ). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore  $dt=facsec*dt_{max}$ .
- **seuil\_diffusion\_implicit** *float* for inheritance: This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **impr\_diffusion\_implicit** *int* for inheritance: Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **impr\_extremums** *int* for inheritance: Print unknowns extremas
- **no\_error\_if\_not\_converged\_diffusion\_implicit** *int* for inheritance
- **no\_conv\_subiteration\_diffusion\_implicit** *int* for inheritance
- **dt\_start** *dt\_start* (14.11) for inheritance:  $dt\_start$   $dt\_min$  : the first iteration is based on  $dt\_min$ .  
 $dt\_start$   $dt\_calc$  : the time step at first iteration is calculated in agreement with CFL condition.  
 $dt\_start$   $dt\_fixe$  value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity).  
By default, the first iteration is based on  $dt\_calc$ .
- **nb\_pas\_dt\_max** *int* for inheritance: Maximum number of calculation time steps (1e9 by default).
- **niter\_max\_diffusion\_implicit** *int* for inheritance: This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **precision\_impr** *int* for inheritance: Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **periode\_sauvegarde\_securite\_en\_heures** *float* for inheritance: To change the default period (23 hours) between the save of the fields in .sauv file.
- **no\_check\_disk\_space** for inheritance: To disable the check of the available amount of disk space during the calculation.
- **disable\_progress** for inheritance: To disable the writing of the .progress file.
- **disable\_dt\_ev** for inheritance: To disable the writing of the .dt\_ev file.
- **gnuplot\_header** *int* for inheritance: Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

## 37.7 Runge\_kutta\_ordre\_2

Description: This is a low-storage Runge-Kutta scheme of second order that uses 2 integration points. The method is presented by Williamson (case 1) in <https://www.sciencedirect.com/science/article/pii/0021999180900339>

See also: `schema_temps_base` (37)

Usage:

**runge\_kutta\_ordre\_2** *str*

```
Read str {
 [tinit float]
 [tmax float]
 [tcpumax float]
 [dt_min float]
 [dt_max str]
 [dt_sauv float]
 [nb_sauv_max int]
 [dt_impr float]
 [facsec str]
 [seuil_statio float]
 [residuals residuals]
 [diffusion_implicite int]
 [seuil_diffusion_implicite float]
 [impr_diffusion_implicite int]
 [impr_extremums int]
 [no_error_if_not_converged_diffusion_implicite int]
 [no_conv_subiteration_diffusion_implicite int]
 [dt_start dt_start]
 [nb_pas_dt_max int]
 [niter_max_diffusion_implicite int]
 [precision_impr int]
 [periode_sauvegarde_securite_en_heures float]
 [no_check_disk_space]
 [disable_progress]
 [disable_dt_ev]
 [gnuplot_header int]
}
```

where

- **tinit** *float* for inheritance: Value of initial calculation time (0 by default).
- **tmax** *float* for inheritance: Time during which the calculation will be stopped (1e30s by default).
- **tcpumax** *float* for inheritance: CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **dt\_min** *float* for inheritance: Minimum calculation time step (1e-16s by default).
- **dt\_max** *str* for inheritance: Maximum calculation time step as function of time (1e30s by default).
- **dt\_sauv** *float* for inheritance: Save time step value (1e30s by default). Every `dt_sauv`, fields are saved in the `.sauv` file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the `.sauv` files, you must specify 0. Note that `dt_sauv` is in terms of physical time (not cpu time).
- **nb\_sauv\_max** *int* for inheritance: Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of `nb_sauv_max` timesteps in the file), the next checkpoints will overwrite the first ones

- **dt\_impr** *float* for inheritance: Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **facsec** *str* for inheritance: Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5.  
Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema\_Adams\_Bashforth\_order\_3.
- **seuil\_statio** *float* for inheritance: Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives  $dG_i/dt$  of all the unknown transported values  $G_i$  have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **residuals** *residuals* (3.127) for inheritance: To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **diffusion\_implicit** *int* for inheritance: Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ( $dt=facsec*dt_{convection}$ ). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore  $dt=facsec*dt_{max}$ .
- **seuil\_diffusion\_implicit** *float* for inheritance: This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **impr\_diffusion\_implicit** *int* for inheritance: Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **impr\_extremums** *int* for inheritance: Print unknowns extremas
- **no\_error\_if\_not\_converged\_diffusion\_implicit** *int* for inheritance
- **no\_conv\_subiteration\_diffusion\_implicit** *int* for inheritance
- **dt\_start** *dt\_start* (14.11) for inheritance:  $dt\_start$   $dt\_min$  : the first iteration is based on  $dt\_min$ .  
 $dt\_start$   $dt\_calc$  : the time step at first iteration is calculated in agreement with CFL condition.  
 $dt\_start$   $dt\_fixe$  value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity).  
By default, the first iteration is based on  $dt\_calc$ .
- **nb\_pas\_dt\_max** *int* for inheritance: Maximum number of calculation time steps (1e9 by default).
- **niter\_max\_diffusion\_implicit** *int* for inheritance: This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **precision\_impr** *int* for inheritance: Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **periode\_sauvegarde\_securite\_en\_heures** *float* for inheritance: To change the default period (23 hours) between the save of the fields in .sauv file.
- **no\_check\_disk\_space** for inheritance: To disable the check of the available amount of disk space during the calculation.
- **disable\_progress** for inheritance: To disable the writing of the .progress file.
- **disable\_dt\_ev** for inheritance: To disable the writing of the .dt\_ev file.
- **gnuplot\_header** *int* for inheritance: Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

## 37.8 Runge\_kutta\_ordre\_2\_classique

Description: This is a classical Runge-Kutta scheme of second order that uses 2 integration points.

See also: `schema_temps_base` (37)

Usage:

**runge\_kutta\_ordre\_2\_classique** *str*

```
Read str {
 [tinit float]
 [tmax float]
 [tcpumax float]
 [dt_min float]
 [dt_max str]
 [dt_sauv float]
 [nb_sauv_max int]
 [dt_impr float]
 [facsec str]
 [seuil_statio float]
 [residuals residuals]
 [diffusion_implicite int]
 [seuil_diffusion_implicite float]
 [impr_diffusion_implicite int]
 [impr_extremums int]
 [no_error_if_not_converged_diffusion_implicite int]
 [no_conv_subiteration_diffusion_implicite int]
 [dt_start dt_start]
 [nb_pas_dt_max int]
 [niter_max_diffusion_implicite int]
 [precision_impr int]
 [periode_sauvegarde_securite_en_heures float]
 [no_check_disk_space]
 [disable_progress]
 [disable_dt_ev]
 [gnuplot_header int]
}
```

where

- **tinit** *float* for inheritance: Value of initial calculation time (0 by default).
- **tmax** *float* for inheritance: Time during which the calculation will be stopped (1e30s by default).
- **tcpumax** *float* for inheritance: CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **dt\_min** *float* for inheritance: Minimum calculation time step (1e-16s by default).
- **dt\_max** *str* for inheritance: Maximum calculation time step as function of time (1e30s by default).
- **dt\_sauv** *float* for inheritance: Save time step value (1e30s by default). Every **dt\_sauv**, fields are saved in the `.sauv` file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the `.sauv` files, you must specify 0. Note that **dt\_sauv** is in terms of physical time (not cpu time).
- **nb\_sauv\_max** *int* for inheritance: Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of **nb\_sauv\_max** timesteps in the file), the next checkpoints will overwrite the first ones

- **dt\_impr** *float* for inheritance: Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **facsec** *str* for inheritance: Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5.  
Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema\_Adams\_Bashforth\_order\_3.
- **seuil\_statio** *float* for inheritance: Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives  $dG_i/dt$  of all the unknown transported values  $G_i$  have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **residuals** *residuals* (3.127) for inheritance: To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **diffusion\_implicit** *int* for inheritance: Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ( $dt=facsec*dt_{convection}$ ). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore  $dt=facsec*dt_{max}$ .
- **seuil\_diffusion\_implicit** *float* for inheritance: This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **impr\_diffusion\_implicit** *int* for inheritance: Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **impr\_extremums** *int* for inheritance: Print unknowns extremas
- **no\_error\_if\_not\_converged\_diffusion\_implicit** *int* for inheritance
- **no\_conv\_subiteration\_diffusion\_implicit** *int* for inheritance
- **dt\_start** *dt\_start* (14.11) for inheritance:  $dt\_start$   $dt\_min$  : the first iteration is based on  $dt\_min$ .  
 $dt\_start$   $dt\_calc$  : the time step at first iteration is calculated in agreement with CFL condition.  
 $dt\_start$   $dt\_fixe$  value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity).  
By default, the first iteration is based on  $dt\_calc$ .
- **nb\_pas\_dt\_max** *int* for inheritance: Maximum number of calculation time steps (1e9 by default).
- **niter\_max\_diffusion\_implicit** *int* for inheritance: This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **precision\_impr** *int* for inheritance: Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **periode\_sauvegarde\_securite\_en\_heures** *float* for inheritance: To change the default period (23 hours) between the save of the fields in .sauv file.
- **no\_check\_disk\_space** for inheritance: To disable the check of the available amount of disk space during the calculation.
- **disable\_progress** for inheritance: To disable the writing of the .progress file.
- **disable\_dt\_ev** for inheritance: To disable the writing of the .dt\_ev file.
- **gnuplot\_header** *int* for inheritance: Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

### 37.9 Runge\_kutta\_ordre\_3

Description: This is a low-storage Runge-Kutta scheme of third order that uses 3 integration points. The method is presented by Williamson (case 7) in <https://www.sciencedirect.com/science/article/pii/0021999180900339>

See also: `schema_temps_base` (37) `rk3_ft` (37.6)

Usage:

**runge\_kutta\_ordre\_3** *str*

```
Read str {
 [tinit float]
 [tmax float]
 [tcpumax float]
 [dt_min float]
 [dt_max str]
 [dt_sauv float]
 [nb_sauv_max int]
 [dt_impr float]
 [facsec str]
 [seuil_statio float]
 [residuals residuals]
 [diffusion_implicite int]
 [seuil_diffusion_implicite float]
 [impr_diffusion_implicite int]
 [impr_extremums int]
 [no_error_if_not_converged_diffusion_implicite int]
 [no_conv_subiteration_diffusion_implicite int]
 [dt_start dt_start]
 [nb_pas_dt_max int]
 [niter_max_diffusion_implicite int]
 [precision_impr int]
 [periode_sauvegarde_securite_en_heures float]
 [no_check_disk_space]
 [disable_progress]
 [disable_dt_ev]
 [gnuplot_header int]
}
```

where

- **tinit** *float* for inheritance: Value of initial calculation time (0 by default).
- **tmax** *float* for inheritance: Time during which the calculation will be stopped (1e30s by default).
- **tcpumax** *float* for inheritance: CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **dt\_min** *float* for inheritance: Minimum calculation time step (1e-16s by default).
- **dt\_max** *str* for inheritance: Maximum calculation time step as function of time (1e30s by default).
- **dt\_sauv** *float* for inheritance: Save time step value (1e30s by default). Every `dt_sauv`, fields are saved in the `.sauv` file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the `.sauv` files, you must specify 0. Note that `dt_sauv` is in terms of physical time (not cpu time).
- **nb\_sauv\_max** *int* for inheritance: Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of `nb_sauv_max` timesteps in the file), the next checkpoints will overwrite the first ones

- **dt\_impr** *float* for inheritance: Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **facsec** *str* for inheritance: Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5.  
Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema\_Adams\_Bashforth\_order\_3.
- **seuil\_statio** *float* for inheritance: Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives  $dG_i/dt$  of all the unknown transported values  $G_i$  have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **residuals** *residuals* (3.127) for inheritance: To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **diffusion\_implicit** *int* for inheritance: Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ( $dt=facsec*dt_{convection}$ ). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore  $dt=facsec*dt_{max}$ .
- **seuil\_diffusion\_implicit** *float* for inheritance: This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **impr\_diffusion\_implicit** *int* for inheritance: Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **impr\_extremums** *int* for inheritance: Print unknowns extremas
- **no\_error\_if\_not\_converged\_diffusion\_implicit** *int* for inheritance
- **no\_conv\_subiteration\_diffusion\_implicit** *int* for inheritance
- **dt\_start** *dt\_start* (14.11) for inheritance:  $dt\_start$   $dt\_min$  : the first iteration is based on  $dt\_min$ .  
 $dt\_start$   $dt\_calc$  : the time step at first iteration is calculated in agreement with CFL condition.  
 $dt\_start$   $dt\_fixe$  value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity).  
By default, the first iteration is based on  $dt\_calc$ .
- **nb\_pas\_dt\_max** *int* for inheritance: Maximum number of calculation time steps (1e9 by default).
- **niter\_max\_diffusion\_implicit** *int* for inheritance: This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **precision\_impr** *int* for inheritance: Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **periode\_sauvegarde\_securite\_en\_heures** *float* for inheritance: To change the default period (23 hours) between the save of the fields in .sauv file.
- **no\_check\_disk\_space** for inheritance: To disable the check of the available amount of disk space during the calculation.
- **disable\_progress** for inheritance: To disable the writing of the .progress file.
- **disable\_dt\_ev** for inheritance: To disable the writing of the .dt\_ev file.
- **gnuplot\_header** *int* for inheritance: Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.



### 37.10 Runge\_kutta\_ordre\_3\_classique

Description: This is a classical Runge-Kutta scheme of third order that uses 3 integration points.

See also: `schema_temps_base` (37)

Usage:

**runge\_kutta\_ordre\_3\_classique** *str*

```
Read str {
 [tinit float]
 [tmax float]
 [tcpumax float]
 [dt_min float]
 [dt_max str]
 [dt_sauv float]
 [nb_sauv_max int]
 [dt_impr float]
 [facsec str]
 [seuil_statio float]
 [residuals residuals]
 [diffusion_implicite int]
 [seuil_diffusion_implicite float]
 [impr_diffusion_implicite int]
 [impr_extremums int]
 [no_error_if_not_converged_diffusion_implicite int]
 [no_conv_subiteration_diffusion_implicite int]
 [dt_start dt_start]
 [nb_pas_dt_max int]
 [niter_max_diffusion_implicite int]
 [precision_impr int]
 [periode_sauvegarde_securite_en_heures float]
 [no_check_disk_space]
 [disable_progress]
 [disable_dt_ev]
 [gnuplot_header int]
}
```

where

- **tinit** *float* for inheritance: Value of initial calculation time (0 by default).
- **tmax** *float* for inheritance: Time during which the calculation will be stopped (1e30s by default).
- **tcpumax** *float* for inheritance: CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **dt\_min** *float* for inheritance: Minimum calculation time step (1e-16s by default).
- **dt\_max** *str* for inheritance: Maximum calculation time step as function of time (1e30s by default).
- **dt\_sauv** *float* for inheritance: Save time step value (1e30s by default). Every **dt\_sauv**, fields are saved in the `.sauv` file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the `.sauv` files, you must specify 0. Note that **dt\_sauv** is in terms of physical time (not cpu time).
- **nb\_sauv\_max** *int* for inheritance: Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of **nb\_sauv\_max** timesteps in the file), the next checkpoints will overwrite the first ones



- **dt\_impr** *float* for inheritance: Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **facsec** *str* for inheritance: Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5.  
Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema\_Adams\_Bashforth\_order\_3.
- **seuil\_statio** *float* for inheritance: Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives  $dG_i/dt$  of all the unknown transported values  $G_i$  have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **residuals** *residuals* (3.127) for inheritance: To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **diffusion\_implicit** *int* for inheritance: Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ( $dt=facsec*dt_{convection}$ ). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore  $dt=facsec*dt_{max}$ .
- **seuil\_diffusion\_implicit** *float* for inheritance: This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **impr\_diffusion\_implicit** *int* for inheritance: Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **impr\_extremums** *int* for inheritance: Print unknowns extremas
- **no\_error\_if\_not\_converged\_diffusion\_implicit** *int* for inheritance
- **no\_conv\_subiteration\_diffusion\_implicit** *int* for inheritance
- **dt\_start** *dt\_start* (14.11) for inheritance:  $dt\_start$   $dt\_min$  : the first iteration is based on  $dt\_min$ .  
 $dt\_start$   $dt\_calc$  : the time step at first iteration is calculated in agreement with CFL condition.  
 $dt\_start$   $dt\_fixe$  value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity).  
By default, the first iteration is based on  $dt\_calc$ .
- **nb\_pas\_dt\_max** *int* for inheritance: Maximum number of calculation time steps (1e9 by default).
- **niter\_max\_diffusion\_implicit** *int* for inheritance: This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **precision\_impr** *int* for inheritance: Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **periode\_sauvegarde\_securite\_en\_heures** *float* for inheritance: To change the default period (23 hours) between the save of the fields in .sauv file.
- **no\_check\_disk\_space** for inheritance: To disable the check of the available amount of disk space during the calculation.
- **disable\_progress** for inheritance: To disable the writing of the .progress file.
- **disable\_dt\_ev** for inheritance: To disable the writing of the .dt\_ev file.
- **gnuplot\_header** *int* for inheritance: Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

### 37.11 Runge\_kutta\_ordre\_4\_d3p

Synonymous: **runge\_kutta\_ordre\_4**

Description: This is a low-storage Runge-Kutta scheme of fourth order that uses 3 integration points. The method is presented by Williamson (case 17) in <https://www.sciencedirect.com/science/article/pii/0021999180900339>

See also: `schema_temps_base` (37)

Usage:

**runge\_kutta\_ordre\_4\_d3p** *str*

**Read** *str* {

```
[tinit float]
[tmax float]
[tcpumax float]
[dt_min float]
[dt_max str]
[dt_sauv float]
[nb_sauv_max int]
[dt_impr float]
[facsec str]
[seuil_statio float]
[residuals residuals]
[diffusion_implicite int]
[seuil_diffusion_implicite float]
[impr_diffusion_implicite int]
[impr_extremums int]
[no_error_if_not_converged_diffusion_implicite int]
[no_conv_subiteration_diffusion_implicite int]
[dt_start dt_start]
[nb_pas_dt_max int]
[niter_max_diffusion_implicite int]
[precision_impr int]
[periode_sauvegarde_securite_en_heures float]
[no_check_disk_space]
[disable_progress]
[disable_dt_ev]
[gnuplot_header int]
```

}

where

- **tinit** *float* for inheritance: Value of initial calculation time (0 by default).
- **tmax** *float* for inheritance: Time during which the calculation will be stopped (1e30s by default).
- **tcpumax** *float* for inheritance: CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **dt\_min** *float* for inheritance: Minimum calculation time step (1e-16s by default).
- **dt\_max** *str* for inheritance: Maximum calculation time step as function of time (1e30s by default).
- **dt\_sauv** *float* for inheritance: Save time step value (1e30s by default). Every `dt_sauv`, fields are saved in the `.sauv` file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the `.sauv` files, you must specify 0. Note that `dt_sauv` is in terms of physical time (not cpu time).
- **nb\_sauv\_max** *int* for inheritance: Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with

parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of `nb_sauv_max` timesteps in the file), the next checkpoints will overwrite the first ones

- **dt\_impr** *float* for inheritance: Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **facsec** *str* for inheritance: Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5.  
Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema\_Adams\_Bashforth\_order\_3.
- **seuil\_statio** *float* for inheritance: Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives  $dG_i/dt$  of all the unknown transported values  $G_i$  have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **residuals** *residuals* (3.127) for inheritance: To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **diffusion\_implicite** *int* for inheritance: Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ( $dt=facsec*dt_{convection}$ ). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore  $dt=facsec*dt_{max}$ .
- **seuil\_diffusion\_implicite** *float* for inheritance: This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **impr\_diffusion\_implicite** *int* for inheritance: Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **impr\_extremums** *int* for inheritance: Print unknowns extremas
- **no\_error\_if\_not\_converged\_diffusion\_implicite** *int* for inheritance
- **no\_conv\_subiteration\_diffusion\_implicite** *int* for inheritance
- **dt\_start** *dt\_start* (14.11) for inheritance: `dt_start dt_min` : the first iteration is based on `dt_min`.  
`dt_start dt_calc` : the time step at first iteration is calculated in agreement with CFL condition.  
`dt_start dt_fixe` value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity).  
By default, the first iteration is based on `dt_calc`.
- **nb\_pas\_dt\_max** *int* for inheritance: Maximum number of calculation time steps (1e9 by default).
- **niter\_max\_diffusion\_implicite** *int* for inheritance: This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **precision\_impr** *int* for inheritance: Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **periode\_sauvegarde\_securite\_en\_heures** *float* for inheritance: To change the default period (23 hours) between the save of the fields in .sauv file.
- **no\_check\_disk\_space** for inheritance: To disable the check of the available amount of disk space during the calculation.
- **disable\_progress** for inheritance: To disable the writing of the .progress file.
- **disable\_dt\_ev** for inheritance: To disable the writing of the .dt\_ev file.
- **gnuplot\_header** *int* for inheritance: Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

## 37.12 Runge\_kutta\_ordre\_4\_classique

Description: This is a classical Runge-Kutta scheme of fourth order that uses 4 integration points.

See also: `schema_temps_base` (37)

Usage:

**runge\_kutta\_ordre\_4\_classique** *str*

```
Read str {
 [tinit float]
 [tmax float]
 [tcpumax float]
 [dt_min float]
 [dt_max str]
 [dt_sauv float]
 [nb_sauv_max int]
 [dt_impr float]
 [facsec str]
 [seuil_statio float]
 [residuals residuals]
 [diffusion_implicite int]
 [seuil_diffusion_implicite float]
 [impr_diffusion_implicite int]
 [impr_extremums int]
 [no_error_if_not_converged_diffusion_implicite int]
 [no_conv_subiteration_diffusion_implicite int]
 [dt_start dt_start]
 [nb_pas_dt_max int]
 [niter_max_diffusion_implicite int]
 [precision_impr int]
 [periode_sauvegarde_securite_en_heures float]
 [no_check_disk_space]
 [disable_progress]
 [disable_dt_ev]
 [gnuplot_header int]
}
```

where

- **tinit** *float* for inheritance: Value of initial calculation time (0 by default).
- **tmax** *float* for inheritance: Time during which the calculation will be stopped (1e30s by default).
- **tcpumax** *float* for inheritance: CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **dt\_min** *float* for inheritance: Minimum calculation time step (1e-16s by default).
- **dt\_max** *str* for inheritance: Maximum calculation time step as function of time (1e30s by default).
- **dt\_sauv** *float* for inheritance: Save time step value (1e30s by default). Every **dt\_sauv**, fields are saved in the `.sauv` file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the `.sauv` files, you must specify 0. Note that **dt\_sauv** is in terms of physical time (not cpu time).
- **nb\_sauv\_max** *int* for inheritance: Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of **nb\_sauv\_max** timesteps in the file), the next checkpoints will overwrite the first ones

- **dt\_impr** *float* for inheritance: Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **facsec** *str* for inheritance: Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5.  
Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema\_Adams\_Bashforth\_order\_3.
- **seuil\_statio** *float* for inheritance: Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives  $dG_i/dt$  of all the unknown transported values  $G_i$  have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **residuals** *residuals* (3.127) for inheritance: To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **diffusion\_implicit** *int* for inheritance: Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ( $dt=facsec*dt_{convection}$ ). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore  $dt=facsec*dt_{max}$ .
- **seuil\_diffusion\_implicit** *float* for inheritance: This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **impr\_diffusion\_implicit** *int* for inheritance: Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **impr\_extremums** *int* for inheritance: Print unknowns extremas
- **no\_error\_if\_not\_converged\_diffusion\_implicit** *int* for inheritance
- **no\_conv\_subiteration\_diffusion\_implicit** *int* for inheritance
- **dt\_start** *dt\_start* (14.11) for inheritance:  $dt\_start$   $dt\_min$  : the first iteration is based on  $dt\_min$ .  
 $dt\_start$   $dt\_calc$  : the time step at first iteration is calculated in agreement with CFL condition.  
 $dt\_start$   $dt\_fixe$  value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity).  
By default, the first iteration is based on  $dt\_calc$ .
- **nb\_pas\_dt\_max** *int* for inheritance: Maximum number of calculation time steps (1e9 by default).
- **niter\_max\_diffusion\_implicit** *int* for inheritance: This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **precision\_impr** *int* for inheritance: Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **periode\_sauvegarde\_securite\_en\_heures** *float* for inheritance: To change the default period (23 hours) between the save of the fields in .sauv file.
- **no\_check\_disk\_space** for inheritance: To disable the check of the available amount of disk space during the calculation.
- **disable\_progress** for inheritance: To disable the writing of the .progress file.
- **disable\_dt\_ev** for inheritance: To disable the writing of the .dt\_ev file.
- **gnuplot\_header** *int* for inheritance: Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

### 37.13 Runge\_kutta\_ordre\_4\_classique\_3\_8

Description: This is a classical Runge-Kutta scheme of fourth order that uses 4 integration points and the 3/8 rule.

See also: `schema_temps_base` (37)

Usage:

**runge\_kutta\_ordre\_4\_classique\_3\_8** *str*

```
Read str {
 [tinit float]
 [tmax float]
 [tcpumax float]
 [dt_min float]
 [dt_max str]
 [dt_sauv float]
 [nb_sauv_max int]
 [dt_impr float]
 [facsec str]
 [seuil_statio float]
 [residuals residuals]
 [diffusion_implicite int]
 [seuil_diffusion_implicite float]
 [impr_diffusion_implicite int]
 [impr_extremums int]
 [no_error_if_not_converged_diffusion_implicite int]
 [no_conv_subiteration_diffusion_implicite int]
 [dt_start dt_start]
 [nb_pas_dt_max int]
 [niter_max_diffusion_implicite int]
 [precision_impr int]
 [periode_sauvegarde_securite_en_heures float]
 [no_check_disk_space]
 [disable_progress]
 [disable_dt_ev]
 [gnuplot_header int]
}
```

where

- **tinit** *float* for inheritance: Value of initial calculation time (0 by default).
- **tmax** *float* for inheritance: Time during which the calculation will be stopped (1e30s by default).
- **tcpumax** *float* for inheritance: CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **dt\_min** *float* for inheritance: Minimum calculation time step (1e-16s by default).
- **dt\_max** *str* for inheritance: Maximum calculation time step as function of time (1e30s by default).
- **dt\_sauv** *float* for inheritance: Save time step value (1e30s by default). Every `dt_sauv`, fields are saved in the `.sauv` file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the `.sauv` files, you must specify 0. Note that `dt_sauv` is in terms of physical time (not cpu time).
- **nb\_sauv\_max** *int* for inheritance: Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of `nb_sauv_max` timesteps in the file), the next checkpoints will overwrite the first ones

- **dt\_impr** *float* for inheritance: Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **facsec** *str* for inheritance: Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5.  
Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema\_Adams\_Bashforth\_order\_3.
- **seuil\_statio** *float* for inheritance: Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives  $dG_i/dt$  of all the unknown transported values  $G_i$  have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **residuals** *residuals* (3.127) for inheritance: To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **diffusion\_implicit** *int* for inheritance: Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ( $dt=facsec*dt_{convection}$ ). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore  $dt=facsec*dt_{max}$ .
- **seuil\_diffusion\_implicit** *float* for inheritance: This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **impr\_diffusion\_implicit** *int* for inheritance: Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **impr\_extremums** *int* for inheritance: Print unknowns extremas
- **no\_error\_if\_not\_converged\_diffusion\_implicit** *int* for inheritance
- **no\_conv\_subiteration\_diffusion\_implicit** *int* for inheritance
- **dt\_start** *dt\_start* (14.11) for inheritance:  $dt\_start$   $dt\_min$  : the first iteration is based on  $dt\_min$ .  
 $dt\_start$   $dt\_calc$  : the time step at first iteration is calculated in agreement with CFL condition.  
 $dt\_start$   $dt\_fixe$  value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity).  
By default, the first iteration is based on  $dt\_calc$ .
- **nb\_pas\_dt\_max** *int* for inheritance: Maximum number of calculation time steps (1e9 by default).
- **niter\_max\_diffusion\_implicit** *int* for inheritance: This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **precision\_impr** *int* for inheritance: Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **periode\_sauvegarde\_securite\_en\_heures** *float* for inheritance: To change the default period (23 hours) between the save of the fields in .sauv file.
- **no\_check\_disk\_space** for inheritance: To disable the check of the available amount of disk space during the calculation.
- **disable\_progress** for inheritance: To disable the writing of the .progress file.
- **disable\_dt\_ev** for inheritance: To disable the writing of the .dt\_ev file.
- **gnuplot\_header** *int* for inheritance: Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.



### 37.14 Runge\_kutta\_rationnel\_ordre\_2

Description: This is the Runge-Kutta rational scheme of second order. The method is described in the note: Wambeck - Rational Runge-Kutta methods for solving systems of ordinary differential equations, at the link: <https://link.springer.com/article/10.1007/BF02252381>. Although rational methods require more computational work than linear ones, they can have some other properties, such as a stable behaviour with explicitness, which make them preferable. The CFD application of this RRK2 scheme is described in the note: [https://link.springer.com/content/pdf/10.1007%2F3-540-13917-6\\_112.pdf](https://link.springer.com/content/pdf/10.1007%2F3-540-13917-6_112.pdf).

See also: `schema_temps_base` (37)

Usage:

**runge\_kutta\_rationnel\_ordre\_2** *str*

```
Read str {
 [tinit float]
 [tmax float]
 [tcpumax float]
 [dt_min float]
 [dt_max str]
 [dt_sauv float]
 [nb_sauv_max int]
 [dt_impr float]
 [facsec str]
 [seuil_statio float]
 [residuals residuals]
 [diffusion_implicite int]
 [seuil_diffusion_implicite float]
 [impr_diffusion_implicite int]
 [impr_extremums int]
 [no_error_if_not_converged_diffusion_implicite int]
 [no_conv_subiteration_diffusion_implicite int]
 [dt_start dt_start]
 [nb_pas_dt_max int]
 [niter_max_diffusion_implicite int]
 [precision_impr int]
 [periode_sauvegarde_securite_en_heures float]
 [no_check_disk_space]
 [disable_progress]
 [disable_dt_ev]
 [gnuplot_header int]
}
where
```

- **tinit** *float* for inheritance: Value of initial calculation time (0 by default).
- **tmax** *float* for inheritance: Time during which the calculation will be stopped (1e30s by default).
- **tcpumax** *float* for inheritance: CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **dt\_min** *float* for inheritance: Minimum calculation time step (1e-16s by default).
- **dt\_max** *str* for inheritance: Maximum calculation time step as function of time (1e30s by default).
- **dt\_sauv** *float* for inheritance: Save time step value (1e30s by default). Every **dt\_sauv**, fields are saved in the `.sauv` file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the `.sauv` files, you must specify 0. Note that **dt\_sauv** is in terms of physical time (not cpu time).



- **nb\_sauv\_max** *int* for inheritance: Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of nb\_sauv\_max timesteps in the file), the next checkpoints will overwrite the first ones
- **dt\_impr** *float* for inheritance: Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **facsec** *str* for inheritance: Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5.  
Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema\_Adams\_Bashforth\_order\_3.
- **seuil\_statio** *float* for inheritance: Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives  $dG_i/dt$  of all the unknown transported values  $G_i$  have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **residuals** *residuals* (3.127) for inheritance: To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **diffusion\_implicit** *int* for inheritance: Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ( $dt=facsec*dt_{convection}$ ). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore  $dt=facsec*dt_{max}$ .
- **seuil\_diffusion\_implicit** *float* for inheritance: This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **impr\_diffusion\_implicit** *int* for inheritance: Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **impr\_extremums** *int* for inheritance: Print unknowns extremas
- **no\_error\_if\_not\_converged\_diffusion\_implicit** *int* for inheritance
- **no\_conv\_subiteration\_diffusion\_implicit** *int* for inheritance
- **dt\_start** *dt\_start* (14.11) for inheritance:  $dt_{start} dt_{min}$  : the first iteration is based on  $dt_{min}$ .  
 $dt_{start} dt_{calc}$  : the time step at first iteration is calculated in agreement with CFL condition.  
 $dt_{start} dt_{fixe}$  value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity).  
By default, the first iteration is based on  $dt_{calc}$ .
- **nb\_pas\_dt\_max** *int* for inheritance: Maximum number of calculation time steps (1e9 by default).
- **niter\_max\_diffusion\_implicit** *int* for inheritance: This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **precision\_impr** *int* for inheritance: Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **periode\_sauvegarde\_securite\_en\_heures** *float* for inheritance: To change the default period (23 hours) between the save of the fields in .sauv file.
- **no\_check\_disk\_space** for inheritance: To disable the check of the available amount of disk space during the calculation.
- **disable\_progress** for inheritance: To disable the writing of the .progress file.
- **disable\_dt\_ev** for inheritance: To disable the writing of the .dt\_ev file.
- **gnuplot\_header** *int* for inheritance: Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

### 37.15 Schema\_adams\_bashforth\_order\_2

Description: not\_set

See also: schema\_temps\_base (37)

Usage:

**schema\_adams\_bashforth\_order\_2** *str*

**Read** *str* {

```
[tinit float]
[tmax float]
[tcpumax float]
[dt_min float]
[dt_max str]
[dt_sauv float]
[nb_sauv_max int]
[dt_impr float]
[facsec str]
[seuil_statio float]
[residuals residuals]
[diffusion_implicite int]
[seuil_diffusion_implicite float]
[impr_diffusion_implicite int]
[impr_extremums int]
[no_error_if_not_converged_diffusion_implicite int]
[no_conv_subiteration_diffusion_implicite int]
[dt_start dt_start]
[nb_pas_dt_max int]
[niter_max_diffusion_implicite int]
[precision_impr int]
[periode_sauvegarde_securite_en_heures float]
[no_check_disk_space]
[disable_progress]
[disable_dt_ev]
[gnuplot_header int]
```

}

where

- **tinit** *float* for inheritance: Value of initial calculation time (0 by default).
- **tmax** *float* for inheritance: Time during which the calculation will be stopped (1e30s by default).
- **tcpumax** *float* for inheritance: CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **dt\_min** *float* for inheritance: Minimum calculation time step (1e-16s by default).
- **dt\_max** *str* for inheritance: Maximum calculation time step as function of time (1e30s by default).
- **dt\_sauv** *float* for inheritance: Save time step value (1e30s by default). Every **dt\_sauv**, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that **dt\_sauv** is in terms of physical time (not cpu time).
- **nb\_sauv\_max** *int* for inheritance: Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of **nb\_sauv\_max** timesteps in the file), the next checkpoints will overwrite the first ones

- **dt\_impr** *float* for inheritance: Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **facsec** *str* for inheritance: Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5.  
Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema\_Adams\_Bashforth\_order\_3.
- **seuil\_statio** *float* for inheritance: Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives  $dG_i/dt$  of all the unknown transported values  $G_i$  have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **residuals** *residuals* (3.127) for inheritance: To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **diffusion\_implicit** *int* for inheritance: Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ( $dt=facsec*dt_{convection}$ ). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore  $dt=facsec*dt_{max}$ .
- **seuil\_diffusion\_implicit** *float* for inheritance: This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **impr\_diffusion\_implicit** *int* for inheritance: Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **impr\_extremums** *int* for inheritance: Print unknowns extremas
- **no\_error\_if\_not\_converged\_diffusion\_implicit** *int* for inheritance
- **no\_conv\_subiteration\_diffusion\_implicit** *int* for inheritance
- **dt\_start** *dt\_start* (14.11) for inheritance:  $dt\_start$   $dt\_min$  : the first iteration is based on  $dt\_min$ .  
 $dt\_start$   $dt\_calc$  : the time step at first iteration is calculated in agreement with CFL condition.  
 $dt\_start$   $dt\_fixe$  value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity).  
By default, the first iteration is based on  $dt\_calc$ .
- **nb\_pas\_dt\_max** *int* for inheritance: Maximum number of calculation time steps (1e9 by default).
- **niter\_max\_diffusion\_implicit** *int* for inheritance: This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **precision\_impr** *int* for inheritance: Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **periode\_sauvegarde\_securite\_en\_heures** *float* for inheritance: To change the default period (23 hours) between the save of the fields in .sauv file.
- **no\_check\_disk\_space** for inheritance: To disable the check of the available amount of disk space during the calculation.
- **disable\_progress** for inheritance: To disable the writing of the .progress file.
- **disable\_dt\_ev** for inheritance: To disable the writing of the .dt\_ev file.
- **gnuplot\_header** *int* for inheritance: Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

## 37.16 Schema\_adams\_bashforth\_order\_3

Description: not\_set

See also: schema\_temps\_base (37)

Usage:

**schema\_adams\_bashforth\_order\_3** *str*

**Read** *str* {

```
[tinit float]
[tmax float]
[tcpumax float]
[dt_min float]
[dt_max str]
[dt_sauv float]
[nb_sauv_max int]
[dt_impr float]
[facsec str]
[seuil_statio float]
[residuals residuals]
[diffusion_implicite int]
[seuil_diffusion_implicite float]
[impr_diffusion_implicite int]
[impr_extremums int]
[no_error_if_not_converged_diffusion_implicite int]
[no_conv_subiteration_diffusion_implicite int]
[dt_start dt_start]
[nb_pas_dt_max int]
[niter_max_diffusion_implicite int]
[precision_impr int]
[periode_sauvegarde_securite_en_heures float]
[no_check_disk_space]
[disable_progress]
[disable_dt_ev]
[gnuplot_header int]
```

}

where

- **tinit** *float* for inheritance: Value of initial calculation time (0 by default).
- **tmax** *float* for inheritance: Time during which the calculation will be stopped (1e30s by default).
- **tcpumax** *float* for inheritance: CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **dt\_min** *float* for inheritance: Minimum calculation time step (1e-16s by default).
- **dt\_max** *str* for inheritance: Maximum calculation time step as function of time (1e30s by default).
- **dt\_sauv** *float* for inheritance: Save time step value (1e30s by default). Every **dt\_sauv**, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that **dt\_sauv** is in terms of physical time (not cpu time).
- **nb\_sauv\_max** *int* for inheritance: Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of **nb\_sauv\_max** timesteps in the file), the next checkpoints will overwrite the first ones

- **dt\_impr** *float* for inheritance: Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **facsec** *str* for inheritance: Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5.  
Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema\_Adams\_Bashforth\_order\_3.
- **seuil\_statio** *float* for inheritance: Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives  $dG_i/dt$  of all the unknown transported values  $G_i$  have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **residuals** *residuals* (3.127) for inheritance: To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **diffusion\_implicit** *int* for inheritance: Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ( $dt=facsec*dt_{convection}$ ). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore  $dt=facsec*dt_{max}$ .
- **seuil\_diffusion\_implicit** *float* for inheritance: This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **impr\_diffusion\_implicit** *int* for inheritance: Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **impr\_extremums** *int* for inheritance: Print unknowns extremas
- **no\_error\_if\_not\_converged\_diffusion\_implicit** *int* for inheritance
- **no\_conv\_subiteration\_diffusion\_implicit** *int* for inheritance
- **dt\_start** *dt\_start* (14.11) for inheritance:  $dt_{start} dt_{min}$  : the first iteration is based on  $dt_{min}$ .  
 $dt_{start} dt_{calc}$  : the time step at first iteration is calculated in agreement with CFL condition.  
 $dt_{start} dt_{fixe}$  value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity).  
By default, the first iteration is based on  $dt_{calc}$ .
- **nb\_pas\_dt\_max** *int* for inheritance: Maximum number of calculation time steps (1e9 by default).
- **niter\_max\_diffusion\_implicit** *int* for inheritance: This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **precision\_impr** *int* for inheritance: Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **periode\_sauvegarde\_securite\_en\_heures** *float* for inheritance: To change the default period (23 hours) between the save of the fields in .sauv file.
- **no\_check\_disk\_space** for inheritance: To disable the check of the available amount of disk space during the calculation.
- **disable\_progress** for inheritance: To disable the writing of the .progress file.
- **disable\_dt\_ev** for inheritance: To disable the writing of the .dt\_ev file.
- **gnuplot\_header** *int* for inheritance: Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

### 37.17 Schema\_adams\_moulton\_order\_2

Description: not\_set

See also: schema\_implicite\_base ([37.22](#))

Usage:

**schema\_adams\_moulton\_order\_2** *str*

**Read** *str* {

```
[facsec_max float]
[max_iter_implicite int]
solveur solveur_implicite_base
[tinit float]
[tmax float]
[tcpumax float]
[dt_min float]
[dt_max str]
[dt_sauv float]
[nb_sauv_max int]
[dt_impr float]
[facsec str]
[seuil_statio float]
[residuals residuals]
[diffusion_implicite int]
[seuil_diffusion_implicite float]
[impr_diffusion_implicite int]
[impr_extremums int]
[no_error_if_not_converged_diffusion_implicite int]
[no_conv_subiteration_diffusion_implicite int]
[dt_start dt_start]
[nb_pas_dt_max int]
[niter_max_diffusion_implicite int]
[precision_impr int]
[periode_sauvegarde_securite_en_heures float]
[no_check_disk_space]
[disable_progress]
[disable_dt_ev]
[gnuplot_header int]
```

}

where

- **facsec\_max** *float*: Maximum ratio allowed between time step and stability time returned by CFL condition. The initial ratio given by facsec keyword is changed during the calculation with the implicit scheme but it couldn't be higher than facsec\_max value.

Warning: Some implicit schemes do not permit high facsec\_max, example Schema\_Adams\_Moulton\_order\_3 needs facsec=facsec\_max=1.

Advice:

The calculation may start with a facsec specified by the user and increased by the algorithm up to the facsec\_max limit. But the user can also choose to specify a constant facsec (facsec\_max will be set to facsec value then). Faster convergence has been seen and depends on the kind of calculation:

- Hydraulic only or thermal hydraulic with forced convection and low coupling between velocity and temperature (Boussinesq value beta low), facsec between 20-30
- Thermal hydraulic with forced convection and strong coupling between velocity and temperature

(Boussinesq value beta high), facsec between 90-100

-Thermohydraulic with natural convection, facsec around 300

-Conduction only, facsec can be set to a very high value (1e8) as if the scheme was unconditionally stable

These values can also be used as rule of thumb for initial facsec with a facsec\_max limit higher.

- **max\_iter\_implicit** *int* for inheritance: Maximum number of iterations allowed for the solver (by default 200).
- **solveur** *solveur\_implicit\_base* (39) for inheritance: This keyword is used to designate the solver selected in the situation where the time scheme is an implicit scheme. *solveur* is the name of the solver that allows equation diffusion and convection operators to be set as implicit terms. Keywords corresponding to this functionality are Simple (SIMPLE type algorithm), Simpler (SIMPLER type algorithm) for incompressible systems, PISO (Pressure Implicit with Split Operator), and Implicite (similar to PISO, but as it looks like a simplified solver, it will use fewer timesteps, and ICE (for PB\_multiphase). But it may run faster because the pressure matrix is not re-assembled and thus provides CPU gains.

Advice: Since the 1.6.0 version, we recommend to use first the Implicite or Simple, then PISO, and at least Simpler. Because the two first give a fastest convergence (several times) than PISO and the Simpler has not been validated. It seems also than Implicite and PISO schemes give better results than the Simple scheme when the flow is not fully stationary. Thus, if the solution obtained with Simple is not stationary, it is recommended to switch to PISO or Implicite scheme.

- **tinit** *float* for inheritance: Value of initial calculation time (0 by default).
- **tmax** *float* for inheritance: Time during which the calculation will be stopped (1e30s by default).
- **tcumax** *float* for inheritance: CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **dt\_min** *float* for inheritance: Minimum calculation time step (1e-16s by default).
- **dt\_max** *str* for inheritance: Maximum calculation time step as function of time (1e30s by default).
- **dt\_sauv** *float* for inheritance: Save time step value (1e30s by default). Every *dt\_sauv*, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that *dt\_sauv* is in terms of physical time (not cpu time).
- **nb\_sauv\_max** *int* for inheritance: Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of *nb\_sauv\_max* timesteps in the file), the next checkpoints will overwrite the first ones
- **dt Impr** *float* for inheritance: Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **facsec** *str* for inheritance: Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5.

Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema\_Adams\_Bashforth\_order\_3.

- **seuil\_statio** *float* for inheritance: Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives  $dG_i/dt$  of all the unknown transported values  $G_i$  have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **residuals** *residuals* (3.127) for inheritance: To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **diffusion\_implicit** *int* for inheritance: Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ( $dt=facsec*dt_{convection}$ ). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened



meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore  $dt=facsec*dt\_max$ .

- **seuil\_diffusion\_implicite** *float* for inheritance: This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **impr\_diffusion\_implicite** *int* for inheritance: Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **impr\_extremums** *int* for inheritance: Print unknowns extremas
- **no\_error\_if\_not\_converged\_diffusion\_implicite** *int* for inheritance
- **no\_conv\_subiteration\_diffusion\_implicite** *int* for inheritance
- **dt\_start** *dt\_start* (14.11) for inheritance:  $dt\_start\ dt\_min$  : the first iteration is based on  $dt\_min$ .  
 $dt\_start\ dt\_calc$  : the time step at first iteration is calculated in agreement with CFL condition.  
 $dt\_start\ dt\_fixe$  value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity).  
By default, the first iteration is based on  $dt\_calc$ .
- **nb\_pas\_dt\_max** *int* for inheritance: Maximum number of calculation time steps (1e9 by default).
- **niter\_max\_diffusion\_implicite** *int* for inheritance: This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **precision\_impr** *int* for inheritance: Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **periode\_sauvegarde\_securite\_en\_heures** *float* for inheritance: To change the default period (23 hours) between the save of the fields in .sauv file.
- **no\_check\_disk\_space** for inheritance: To disable the check of the available amount of disk space during the calculation.
- **disable\_progress** for inheritance: To disable the writing of the .progress file.
- **disable\_dt\_ev** for inheritance: To disable the writing of the .dt\_ev file.
- **gnuplot\_header** *int* for inheritance: Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

### 37.18 Schema\_adams\_moulton\_order\_3

Description: not\_set

See also: schema\_implicite\_base (37.22)

Usage:

**schema\_adams\_moulton\_order\_3** *str*

**Read** *str* {

```
[facsec_max float]
[max_iter_implicite int]
solveur solveur_implicite_base
[tinit float]
[tmax float]
[tcpumax float]
[dt_min float]
[dt_max str]
[dt_sauv float]
[nb_sauv_max int]
[dt_impr float]
[facsec str]
```



```

[seuil_statio float]
[residuals residuals]
[diffusion_implicite int]
[seuil_diffusion_implicite float]
[impr_diffusion_implicite int]
[impr_extremums int]
[no_error_if_not_converged_diffusion_implicite int]
[no_conv_subiteration_diffusion_implicite int]
[dt_start dt_start]
[nb_pas_dt_max int]
[niter_max_diffusion_implicite int]
[precision_impr int]
[periode_sauvegarde_securite_en_heures float]
[no_check_disk_space]
[disable_progress]
[disable_dt_ev]
[gnuplot_header int]
}

```

where

- **facsec\_max** *float*: Maximum ratio allowed between time step and stability time returned by CFL condition. The initial ratio given by facsec keyword is changed during the calculation with the implicit scheme but it couldn't be higher than facsec\_max value.

Warning: Some implicit schemes do not permit high facsec\_max, example Schema\_Adams\_Moulton\_order\_3 needs facsec=facsec\_max=1.

Advice:

The calculation may start with a facsec specified by the user and increased by the algorithm up to the facsec\_max limit. But the user can also choose to specify a constant facsec (facsec\_max will be set to facsec value then). Faster convergence has been seen and depends on the kind of calculation:

-Hydraulic only or thermal hydraulic with forced convection and low coupling between velocity and temperature (Boussinesq value beta low), facsec between 20-30

-Thermal hydraulic with forced convection and strong coupling between velocity and temperature (Boussinesq value beta high), facsec between 90-100

-Thermohydraulic with natural convection, facsec around 300

-Conduction only, facsec can be set to a very high value (1e8) as if the scheme was unconditionally stable

These values can also be used as rule of thumb for initial facsec with a facsec\_max limit higher.

- **max\_iter\_implicite** *int* for inheritance: Maximum number of iterations allowed for the solver (by default 200).
- **solveur** *solveur\_implicite\_base* (39) for inheritance: This keyword is used to designate the solver selected in the situation where the time scheme is an implicit scheme. *solveur* is the name of the solver that allows equation diffusion and convection operators to be set as implicit terms. Keywords corresponding to this functionality are Simple (SIMPLE type algorithm), Simpler (SIMPLER type algorithm) for incompressible systems, Piso (Pressure Implicit with Split Operator), and Implicite (similar to PISO, but as it looks like a simplified solver, it will use fewer timesteps, and ICE (for PB\_multiphase). But it may run faster because the pressure matrix is not re-assembled and thus provides CPU gains.

Advice: Since the 1.6.0 version, we recommend to use first the Implicite or Simple, then Piso, and at least Simpler. Because the two first give a fastest convergence (several times) than Piso and the Simpler has not been validated. It seems also than Implicite and Piso schemes give better results than the Simple scheme when the flow is not fully stationary. Thus, if the solution obtained with Simple is not stationary, it is recommended to switch to Piso or Implicite scheme.

- **tinit** *float* for inheritance: Value of initial calculation time (0 by default).
- **tmax** *float* for inheritance: Time during which the calculation will be stopped (1e30s by default).

- **tcpumax** *float* for inheritance: CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **dt\_min** *float* for inheritance: Minimum calculation time step (1e-16s by default).
- **dt\_max** *str* for inheritance: Maximum calculation time step as function of time (1e30s by default).
- **dt\_sauv** *float* for inheritance: Save time step value (1e30s by default). Every **dt\_sauv**, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that **dt\_sauv** is in terms of physical time (not cpu time).
- **nb\_sauv\_max** *int* for inheritance: Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of **nb\_sauv\_max** timesteps in the file), the next checkpoints will overwrite the first ones
- **dt\_impr** *float* for inheritance: Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **facsec** *str* for inheritance: Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the **facsec** to 0.5.  
Warning: Some schemes needs a **facsec** lower than 1 (0.5 is a good start), for example Schema\_Adams\_Bashforth\_order\_3.
- **seuil\_statio** *float* for inheritance: Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives  $dG_i/dt$  of all the unknown transported values  $G_i$  have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **residuals** *residuals* (3.127) for inheritance: To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **diffusion\_implicit** *int* for inheritance: Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ( $dt=facsec*dt_{convection}$ ). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large **facsec** value. Start with a **facsec** value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore  $dt=facsec*dt_{max}$ .
- **seuil\_diffusion\_implicit** *float* for inheritance: This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **impr\_diffusion\_implicit** *int* for inheritance: Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **impr\_extremums** *int* for inheritance: Print unknowns extremas
- **no\_error\_if\_not\_converged\_diffusion\_implicit** *int* for inheritance
- **no\_conv\_subiteration\_diffusion\_implicit** *int* for inheritance
- **dt\_start** *dt\_start* (14.11) for inheritance: **dt\_start dt\_min** : the first iteration is based on **dt\_min**.  
**dt\_start dt\_calc** : the time step at first iteration is calculated in agreement with CFL condition.  
**dt\_start dt\_fixe** value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity).  
By default, the first iteration is based on **dt\_calc**.
- **nb\_pas\_dt\_max** *int* for inheritance: Maximum number of calculation time steps (1e9 by default).
- **niter\_max\_diffusion\_implicit** *int* for inheritance: This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **precision\_impr** *int* for inheritance: Optional keyword to define the digit number for flux values printed into .out files (by default 3).

- **periode\_sauvegarde\_securite\_en\_heures** *float* for inheritance: To change the default period (23 hours) between the save of the fields in .sauv file.
- **no\_check\_disk\_space** for inheritance: To disable the check of the available amount of disk space during the calculation.
- **disable\_progress** for inheritance: To disable the writing of the .progress file.
- **disable\_dt\_ev** for inheritance: To disable the writing of the .dt\_ev file.
- **gnuplot\_header** *int* for inheritance: Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

### 37.19 Schema\_backward\_differentiation\_order\_2

Description: not\_set

See also: [schema\\_implicite\\_base \(37.22\)](#)

Usage:

**schema\_backward\_differentiation\_order\_2** *str*

```
Read str {
 [facsec_max float]
 [max_iter_implicite int]
 solveur solveur_implicite_base
 [tinit float]
 [tmax float]
 [tcpumax float]
 [dt_min float]
 [dt_max str]
 [dt_sauv float]
 [nb_sauv_max int]
 [dt_impr float]
 [facsec str]
 [seuil_statio float]
 [residuals residuals]
 [diffusion_implicite int]
 [seuil_diffusion_implicite float]
 [impr_diffusion_implicite int]
 [impr_extremums int]
 [no_error_if_not_converged_diffusion_implicite int]
 [no_conv_subiteration_diffusion_implicite int]
 [dt_start dt_start]
 [nb_pas_dt_max int]
 [niter_max_diffusion_implicite int]
 [precision_impr int]
 [periode_sauvegarde_securite_en_heures float]
 [no_check_disk_space]
 [disable_progress]
 [disable_dt_ev]
 [gnuplot_header int]
}
```

where

- **facsec\_max** *float*: Maximum ratio allowed between time step and stability time returned by CFL condition. The initial ratio given by facsec keyword is changed during the calculation with the implicit scheme but it couldn't be higher than facsec\_max value.

Warning: Some implicit schemes do not permit high `facsec_max`, example `Schema_Adams_Moulton_order_3` needs `facsec=facsec_max=1`.

Advice:

The calculation may start with a `facsec` specified by the user and increased by the algorithm up to the `facsec_max` limit. But the user can also choose to specify a constant `facsec` (`facsec_max` will be set to `facsec` value then). Faster convergence has been seen and depends on the kind of calculation:

- Hydraulic only or thermal hydraulic with forced convection and low coupling between velocity and temperature (Boussinesq value `beta` low), `facsec` between 20-30
- Thermal hydraulic with forced convection and strong coupling between velocity and temperature (Boussinesq value `beta` high), `facsec` between 90-100
- Thermohydraulic with natural convection, `facsec` around 300
- Conduction only, `facsec` can be set to a very high value ( $1e8$ ) as if the scheme was unconditionally stable

These values can also be used as rule of thumb for initial `facsec` with a `facsec_max` limit higher.

- **max\_iter\_implicit** *int* for inheritance: Maximum number of iterations allowed for the solver (by default 200).
- **solveur** *solveur\_implicit\_base* (39) for inheritance: This keyword is used to designate the solver selected in the situation where the time scheme is an implicit scheme. `solveur` is the name of the solver that allows equation diffusion and convection operators to be set as implicit terms. Keywords corresponding to this functionality are Simple (SIMPLE type algorithm), Simpler (SIMPLER type algorithm) for incompressible systems, PISO (Pressure Implicit with Split Operator), and Implicit (similar to PISO, but as it looks like a simplified solver, it will use fewer timesteps, and ICE (for PB\_multiphase). But it may run faster because the pressure matrix is not re-assembled and thus provides CPU gains.

Advice: Since the 1.6.0 version, we recommend to use first the Implicit or Simple, then PISO, and at least Simpler. Because the two first give a fastest convergence (several times) than PISO and the Simpler has not been validated. It seems also than Implicit and PISO schemes give better results than the Simple scheme when the flow is not fully stationary. Thus, if the solution obtained with Simple is not stationary, it is recommended to switch to PISO or Implicit scheme.

- **tinit** *float* for inheritance: Value of initial calculation time (0 by default).
- **tmax** *float* for inheritance: Time during which the calculation will be stopped ( $1e30$ s by default).
- **tcputmax** *float* for inheritance: CPU time limit (must be specified in hours) for which the calculation is stopped ( $1e30$ s by default).
- **dt\_min** *float* for inheritance: Minimum calculation time step ( $1e-16$ s by default).
- **dt\_max** *str* for inheritance: Maximum calculation time step as function of time ( $1e30$ s by default).
- **dt\_sauv** *float* for inheritance: Save time step value ( $1e30$ s by default). Every `dt_sauv`, fields are saved in the `.sauv` file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the `.sauv` files, you must specify 0. Note that `dt_sauv` is in terms of physical time (not cpu time).
- **nb\_sauv\_max** *int* for inheritance: Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of `nb_sauv_max` timesteps in the file), the next checkpoints will overwrite the first ones
- **dt\_impr** *float* for inheritance: Scheme parameter printing time step in time ( $1e30$ s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the `.out` file.
- **facsec** *str* for inheritance: Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the `facsec` to 0.5.

Warning: Some schemes needs a `facsec` lower than 1 (0.5 is a good start), for example `Schema_Adams_Bashforth_order_3`.

- **seuil\_statio** *float* for inheritance: Value of the convergence threshold ( $1e-12$  by default). Problems

using this type of time scheme converge when the derivatives  $dG_i/dt$  of all the unknown transported values  $G_i$  have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.

- **residuals** *residuals* (3.127) for inheritance: To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **diffusion\_implicit** *int* for inheritance: Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ( $dt=facsec*dt\_convection$ ). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large *facsec* value. Start with a *facsec* value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore  $dt=facsec*dt\_max$ .
- **seuil\_diffusion\_implicit** *float* for inheritance: This keyword changes the default value ( $1e-6$ ) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **impr\_diffusion\_implicit** *int* for inheritance: Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **impr\_extremums** *int* for inheritance: Print unknowns extremas
- **no\_error\_if\_not\_converged\_diffusion\_implicit** *int* for inheritance
- **no\_conv\_subiteration\_diffusion\_implicit** *int* for inheritance
- **dt\_start** *dt\_start* (14.11) for inheritance: *dt\_start dt\_min* : the first iteration is based on *dt\_min*.  
*dt\_start dt\_calc* : the time step at first iteration is calculated in agreement with CFL condition.  
*dt\_start dt\_fixe* value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity).  
By default, the first iteration is based on *dt\_calc*.
- **nb\_pas\_dt\_max** *int* for inheritance: Maximum number of calculation time steps ( $1e9$  by default).
- **niter\_max\_diffusion\_implicit** *int* for inheritance: This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **precision\_impr** *int* for inheritance: Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **periode\_sauvegarde\_securite\_en\_heures** *float* for inheritance: To change the default period (23 hours) between the save of the fields in .sauv file.
- **no\_check\_disk\_space** for inheritance: To disable the check of the available amount of disk space during the calculation.
- **disable\_progress** for inheritance: To disable the writing of the .progress file.
- **disable\_dt\_ev** for inheritance: To disable the writing of the .dt\_ev file.
- **gnuplot\_header** *int* for inheritance: Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

## 37.20 Schema\_backward\_differentiation\_order\_3

Description: *not\_set*

See also: *schema\_implicit\_base* (37.22)

Usage:

**schema\_backward\_differentiation\_order\_3** *str*

**Read** *str* {

[ **facsec\_max** *float*]  
[ **max\_iter\_implicit** *int*]  
**solveur** *solveur\_implicit\_base*

```

[tinit float]
[tmax float]
[tcpumax float]
[dt_min float]
[dt_max str]
[dt_sauv float]
[nb_sauv_max int]
[dt_impr float]
[facsec str]
[seuil_statio float]
[residuals residuals]
[diffusion_implicit int]
[seuil_diffusion_implicit float]
[impr_diffusion_implicit int]
[impr_extremums int]
[no_error_if_not_converged_diffusion_implicit int]
[no_conv_subiteration_diffusion_implicit int]
[dt_start dt_start]
[nb_pas_dt_max int]
[niter_max_diffusion_implicit int]
[precision_impr int]
[periode_sauvegarde_securite_en_heures float]
[no_check_disk_space]
[disable_progress]
[disable_dt_ev]
[gnuplot_header int]
}

```

where

- **facsec\_max** *float*: Maximum ratio allowed between time step and stability time returned by CFL condition. The initial ratio given by **facsec** keyword is changed during the calculation with the implicit scheme but it couldn't be higher than **facsec\_max** value.

Warning: Some implicit schemes do not permit high **facsec\_max**, example **Schema\_Adams\_Moulton\_order\_3** needs **facsec=facsec\_max=1**.

Advice:

The calculation may start with a **facsec** specified by the user and increased by the algorithm up to the **facsec\_max** limit. But the user can also choose to specify a constant **facsec** (**facsec\_max** will be set to **facsec** value then). Faster convergence has been seen and depends on the kind of calculation:

- Hydraulic only or thermal hydraulic with forced convection and low coupling between velocity and temperature (Boussinesq value **beta** low), **facsec** between 20-30
- Thermal hydraulic with forced convection and strong coupling between velocity and temperature (Boussinesq value **beta** high), **facsec** between 90-100
- Thermohydraulic with natural convection, **facsec** around 300
- Conduction only, **facsec** can be set to a very high value (1e8) as if the scheme was unconditionally stable

These values can also be used as rule of thumb for initial **facsec** with a **facsec\_max** limit higher.

- **max\_iter\_implicit** *int* for inheritance: Maximum number of iterations allowed for the solver (by default 200).
- **solveur** *solveur\_implicit\_base* (39) for inheritance: This keyword is used to designate the solver selected in the situation where the time scheme is an implicit scheme. **solveur** is the name of the solver that allows equation diffusion and convection operators to be set as implicit terms. Keywords corresponding to this functionality are **Simple** (**SIMPLE** type algorithm), **Simpler** (**SIMPLER** type algorithm) for incompressible systems, **Piso** (**Pressure Implicit with Split Operator**), and **Implicit** (similar to **PISO**, but as it looks like a simplified solver, it will use fewer timesteps, and **ICE** (for



PB\_multiphase). But it may run faster because the pressure matrix is not re-assembled and thus provides CPU gains.

Advice: Since the 1.6.0 version, we recommend to use first the Implicite or Simple, then Piso, and at least Simplr. Because the two first give a fastest convergence (several times) than Piso and the Simplr has not been validated. It seems also than Implicite and Piso schemes give better results than the Simple scheme when the flow is not fully stationary. Thus, if the solution obtained with Simple is not stationary, it is recommended to switch to Piso or Implicite scheme.

- **tinit** *float* for inheritance: Value of initial calculation time (0 by default).
- **tmax** *float* for inheritance: Time during which the calculation will be stopped (1e30s by default).
- **tcputmax** *float* for inheritance: CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **dt\_min** *float* for inheritance: Minimum calculation time step (1e-16s by default).
- **dt\_max** *str* for inheritance: Maximum calculation time step as function of time (1e30s by default).
- **dt\_sauv** *float* for inheritance: Save time step value (1e30s by default). Every dt\_sauv, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that dt\_sauv is in terms of physical time (not cpu time).
- **nb\_sauv\_max** *int* for inheritance: Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of nb\_sauv\_max timesteps in the file), the next checkpoints will overwrite the first ones
- **dt\_impr** *float* for inheritance: Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **facsec** *str* for inheritance: Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5.  
Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema\_Adams\_Bashforth\_order\_3.
- **seuil\_statio** *float* for inheritance: Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives  $dG_i/dt$  of all the unknown transported values  $G_i$  have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **residuals** *residuals* (3.127) for inheritance: To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **diffusion\_implicite** *int* for inheritance: Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ( $dt=facsec*dt_{convection}$ ). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore  $dt=facsec*dt_{max}$ .
- **seuil\_diffusion\_implicite** *float* for inheritance: This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **impr\_diffusion\_implicite** *int* for inheritance: Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **impr\_extremums** *int* for inheritance: Print unknowns extremas
- **no\_error\_if\_not\_converged\_diffusion\_implicite** *int* for inheritance
- **no\_conv\_subiteration\_diffusion\_implicite** *int* for inheritance
- **dt\_start** *dt\_start* (14.11) for inheritance: dt\_start dt\_min : the first iteration is based on dt\_min.  
dt\_start dt\_calc : the time step at first iteration is calculated in agreement with CFL condition.

dt\_start dt\_fixe value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity).

By default, the first iteration is based on dt\_calc.

- **nb\_pas\_dt\_max** *int* for inheritance: Maximum number of calculation time steps (1e9 by default).
- **niter\_max\_diffusion\_implicit** *int* for inheritance: This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **precision\_impr** *int* for inheritance: Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **periode\_sauvegarde\_securite\_en\_heures** *float* for inheritance: To change the default period (23 hours) between the save of the fields in .sauv file.
- **no\_check\_disk\_space** for inheritance: To disable the check of the available amount of disk space during the calculation.
- **disable\_progress** for inheritance: To disable the writing of the .progress file.
- **disable\_dt\_ev** for inheritance: To disable the writing of the .dt\_ev file.
- **gnuplot\_header** *int* for inheritance: Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

## 37.21 Scheme\_euler\_implicit

Synonymous: **schema\_euler\_implicit**

Description: This is the Euler implicit scheme.

See also: **schema\_implicit\_base** ([37.22](#))

Usage:

**scheme\_euler\_implicit** *str*

**Read** *str* {

```
[facsec_max float]
[facsec_expert facsec_expert]
[facsec_func str]
[resolution_monolithique bloc_lecture]
[max_iter_implicit int]
solveur solveur_implicit_base
[tinit float]
[tmax float]
[tcpumax float]
[dt_min float]
[dt_max str]
[dt_sauv float]
[nb_sauv_max int]
[dt_impr float]
[facsec str]
[seuil_statio float]
[residuals residuals]
[diffusion_implicit int]
[seuil_diffusion_implicit float]
[impr_diffusion_implicit int]
[impr_extremums int]
[no_error_if_not_converged_diffusion_implicit int]
[no_conv_subiteration_diffusion_implicit int]
```



```

[dt_start dt_start]
[nb_pas_dt_max int]
[niter_max_diffusion_implicit int]
[precision_impr int]
[periode_sauvegarde_securite_en_heures float]
[no_check_disk_space]
[disable_progress]
[disable_dt_ev]
[gnuplot_header int]
}
where

```

- **facsec\_max** *float*: For old syntax, see the complete parameters of facsec for details
- **facsec\_expert** *facsec\_expert* (3.73): Advanced facsec specification
- **facsec\_func** *str*: Advanced facsec specification as a function
- **resolution\_monolithique** *bloc\_lecture* (3.2): Activate monolithic resolution for coupled problems. Solves together the equations corresponding to the application domains in the given order. All application domains of the coupled equations must be given to determine the order of resolution. If the monolithic solving is not wanted for a specific application domain, an underscore can be added as prefix. For example, resolution\_monolithique { dom1 { dom2 dom3 } \_dom4 } will solve in a single matrix the equations having dom1 as application domain, then the equations having dom2 or dom3 as application domain in a single matrix, then the equations having dom4 as application domain in a sequential way (not in a single matrix).
- **max\_iter\_implicit** *int* for inheritance: Maximum number of iterations allowed for the solver (by default 200).
- **solveur** *solveur\_implicit\_base* (39) for inheritance: This keyword is used to designate the solver selected in the situation where the time scheme is an implicit scheme. *solver* is the name of the solver that allows equation diffusion and convection operators to be set as implicit terms. Keywords corresponding to this functionality are Simple (SIMPLE type algorithm), Simpler (SIMPLER type algorithm) for incompressible systems, Piso (Pressure Implicit with Split Operator), and Implicit (similar to PISO, but as it looks like a simplified solver, it will use fewer timesteps, and ICE (for PB\_multiphase). But it may run faster because the pressure matrix is not re-assembled and thus provides CPU gains.  
Advice: Since the 1.6.0 version, we recommend to use first the Implicit or Simple, then Piso, and at least Simpler. Because the two first give a fastest convergence (several times) than Piso and the Simpler has not been validated. It seems also than Implicit and Piso schemes give better results than the Simple scheme when the flow is not fully stationary. Thus, if the solution obtained with Simple is not stationary, it is recommended to switch to Piso or Implicit scheme.
- **tinit** *float* for inheritance: Value of initial calculation time (0 by default).
- **tmax** *float* for inheritance: Time during which the calculation will be stopped (1e30s by default).
- **tcpumax** *float* for inheritance: CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **dt\_min** *float* for inheritance: Minimum calculation time step (1e-16s by default).
- **dt\_max** *str* for inheritance: Maximum calculation time step as function of time (1e30s by default).
- **dt\_sauv** *float* for inheritance: Save time step value (1e30s by default). Every dt\_sauv, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that dt\_sauv is in terms of physical time (not cpu time).
- **nb\_sauv\_max** *int* for inheritance: Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of nb\_sauv\_max timesteps in the file), the next checkpoints will overwrite the first ones

- **dt\_impr** *float* for inheritance: Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **facsec** *str* for inheritance: Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5.  
Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema\_Adams\_Bashforth\_order\_3.
- **seuil\_statio** *float* for inheritance: Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives  $dG_i/dt$  of all the unknown transported values  $G_i$  have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **residuals** *residuals* (3.127) for inheritance: To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **diffusion\_implicit** *int* for inheritance: Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ( $dt=facsec*dt_{convection}$ ). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore  $dt=facsec*dt_{max}$ .
- **seuil\_diffusion\_implicit** *float* for inheritance: This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **impr\_diffusion\_implicit** *int* for inheritance: Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **impr\_extremums** *int* for inheritance: Print unknowns extremas
- **no\_error\_if\_not\_converged\_diffusion\_implicit** *int* for inheritance
- **no\_conv\_subiteration\_diffusion\_implicit** *int* for inheritance
- **dt\_start** *dt\_start* (14.11) for inheritance:  $dt\_start$   $dt\_min$  : the first iteration is based on  $dt\_min$ .  
 $dt\_start$   $dt\_calc$  : the time step at first iteration is calculated in agreement with CFL condition.  
 $dt\_start$   $dt\_fixe$  value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity).  
By default, the first iteration is based on  $dt\_calc$ .
- **nb\_pas\_dt\_max** *int* for inheritance: Maximum number of calculation time steps (1e9 by default).
- **niter\_max\_diffusion\_implicit** *int* for inheritance: This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **precision\_impr** *int* for inheritance: Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **periode\_sauvegarde\_securite\_en\_heures** *float* for inheritance: To change the default period (23 hours) between the save of the fields in .sauv file.
- **no\_check\_disk\_space** for inheritance: To disable the check of the available amount of disk space during the calculation.
- **disable\_progress** for inheritance: To disable the writing of the .progress file.
- **disable\_dt\_ev** for inheritance: To disable the writing of the .dt\_ev file.
- **gnuplot\_header** *int* for inheritance: Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

## 37.22 Schema\_implicite\_base

Description: Basic class for implicite time scheme.

See also: [schema\\_temps\\_base \(37\)](#) [schema\\_adams\\_moulton\\_order\\_3 \(37.18\)](#) [scheme\\_euler\\_implicit \(37.21\)](#) [schema\\_adams\\_moulton\\_order\\_2 \(37.17\)](#) [schema\\_backward\\_differentiation\\_order\\_3 \(37.20\)](#) [schema\\_backward\\_differentiation\\_order\\_2 \(37.19\)](#) [implicit\\_euler\\_steady\\_scheme \(37.1\)](#)

Usage:

**schema\_implicite\_base** *str*

**Read** *str* {

```
[max_iter_implicite int]
solveur solveur_implicite_base
[tinit float]
[tmax float]
[tcpumax float]
[dt_min float]
[dt_max str]
[dt_sauv float]
[nb_sauv_max int]
[dt_impr float]
[facsec str]
[seuil_statio float]
[residuals residuals]
[diffusion_implicite int]
[seuil_diffusion_implicite float]
[impr_diffusion_implicite int]
[impr_extremums int]
[no_error_if_not_converged_diffusion_implicite int]
[no_conv_subiteration_diffusion_implicite int]
[dt_start dt_start]
[nb_pas_dt_max int]
[niter_max_diffusion_implicite int]
[precision_impr int]
[periode_sauvegarde_securite_en_heures float]
[no_check_disk_space]
[disable_progress]
[disable_dt_ev]
[gnuplot_header int]
```

}

where

- **max\_iter\_implicite** *int*: Maximum number of iterations allowed for the solver (by default 200).
- **solveur** *solveur\_implicite\_base* (39): This keyword is used to designate the solver selected in the situation where the time scheme is an implicit scheme. *solver* is the name of the solver that allows equation diffusion and convection operators to be set as implicit terms. Keywords corresponding to this functionality are Simple (SIMPLE type algorithm), Simpler (SIMPLER type algorithm) for incompressible systems, Piso (Pressure Implicit with Split Operator), and Implicite (similar to PISO, but as it looks like a simplified solver, it will use fewer timesteps, and ICE (for PB\_multiphase). But it may run faster because the pressure matrix is not re-assembled and thus provides CPU gains. Advice: Since the 1.6.0 version, we recommend to use first the Implicite or Simple, then Piso, and at least Simpler. Because the two first give a fastest convergence (several times) than Piso and the Simpler has not been validated. It seems also than Implicite and Piso schemes give better results than

the Simple scheme when the flow is not fully stationary. Thus, if the solution obtained with Simple is not stationary, it is recommended to switch to Piso or Implicite scheme.

- **tinit** *float* for inheritance: Value of initial calculation time (0 by default).
- **tmax** *float* for inheritance: Time during which the calculation will be stopped (1e30s by default).
- **tcputmax** *float* for inheritance: CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **dt\_min** *float* for inheritance: Minimum calculation time step (1e-16s by default).
- **dt\_max** *str* for inheritance: Maximum calculation time step as function of time (1e30s by default).
- **dt\_sauv** *float* for inheritance: Save time step value (1e30s by default). Every dt\_sauv, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that dt\_sauv is in terms of physical time (not cpu time).
- **nb\_sauv\_max** *int* for inheritance: Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of nb\_sauv\_max timesteps in the file), the next checkpoints will overwrite the first ones
- **dt\_impr** *float* for inheritance: Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **facsec** *str* for inheritance: Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the facsec to 0.5.  
Warning: Some schemes needs a facsec lower than 1 (0.5 is a good start), for example Schema\_Adams\_Bashforth\_order\_3.
- **seuil\_statio** *float* for inheritance: Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives  $dG_i/dt$  of all the unknown transported values  $G_i$  have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **residuals** *residuals* (3.127) for inheritance: To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **diffusion\_implicite** *int* for inheritance: Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ( $dt=facsec*dt_{convection}$ ). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large facsec value. Start with a facsec value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore  $dt=facsec*dt_{max}$ .
- **seuil\_diffusion\_implicite** *float* for inheritance: This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **impr\_diffusion\_implicite** *int* for inheritance: Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **impr\_extremums** *int* for inheritance: Print unknowns extremas
- **no\_error\_if\_not\_converged\_diffusion\_implicite** *int* for inheritance
- **no\_conv\_subiteration\_diffusion\_implicite** *int* for inheritance
- **dt\_start** *dt\_start* (14.11) for inheritance: dt\_start dt\_min : the first iteration is based on dt\_min.  
dt\_start dt\_calc : the time step at first iteration is calculated in agreement with CFL condition.  
dt\_start dt\_fixe value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity).  
By default, the first iteration is based on dt\_calc.
- **nb\_pas\_dt\_max** *int* for inheritance: Maximum number of calculation time steps (1e9 by default).
- **niter\_max\_diffusion\_implicite** *int* for inheritance: This keyword changes the default value (num-

ber of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.

- **precision\_impr** *int* for inheritance: Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **periode\_sauvegarde\_securite\_en\_heures** *float* for inheritance: To change the default period (23 hours) between the save of the fields in .sauv file.
- **no\_check\_disk\_space** for inheritance: To disable the check of the available amount of disk space during the calculation.
- **disable\_progress** for inheritance: To disable the writing of the .progress file.
- **disable\_dt\_ev** for inheritance: To disable the writing of the .dt\_ev file.
- **gnuplot\_header** *int* for inheritance: Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

### 37.23 Schema\_phase\_field

Description: Keyword for the only available Scheme for time discretization of the Phase Field problem.

See also: `schema_temps_base` (37)

Usage:

**schema\_phase\_field** *str*

**Read** *str* {

```
[schema_ch schema_temps_base]
[schema_ns schema_temps_base]
[tinit float]
[tmax float]
[tcpumax float]
[dt_min float]
[dt_max str]
[dt_sauv float]
[nb_sauv_max int]
[dt_impr float]
[facsec str]
[seuil_statio float]
[residuals residuals]
[diffusion_implicit int]
[seuil_diffusion_implicit float]
[impr_diffusion_implicit int]
[impr_extremums int]
[no_error_if_not_converged_diffusion_implicit int]
[no_conv_subiteration_diffusion_implicit int]
[dt_start dt_start]
[nb_pas_dt_max int]
[niter_max_diffusion_implicit int]
[precision_impr int]
[periode_sauvegarde_securite_en_heures float]
[no_check_disk_space]
[disable_progress]
[disable_dt_ev]
[gnuplot_header int]
```

}

where

- **schema\_ch** *schema\_temps\_base* (37): Time scheme for the Cahn-Hilliard equation.
- **schema\_ns** *schema\_temps\_base* (37): Time scheme for the Navier-Stokes equation.
- **tinit** *float* for inheritance: Value of initial calculation time (0 by default).
- **tmax** *float* for inheritance: Time during which the calculation will be stopped (1e30s by default).
- **tcputmax** *float* for inheritance: CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **dt\_min** *float* for inheritance: Minimum calculation time step (1e-16s by default).
- **dt\_max** *str* for inheritance: Maximum calculation time step as function of time (1e30s by default).
- **dt\_sauv** *float* for inheritance: Save time step value (1e30s by default). Every **dt\_sauv**, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that **dt\_sauv** is in terms of physical time (not cpu time).
- **nb\_sauv\_max** *int* for inheritance: Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of **nb\_sauv\_max** timesteps in the file), the next checkpoints will overwrite the first ones
- **dt\_impr** *float* for inheritance: Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **facsec** *str* for inheritance: Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the **facsec** to 0.5.  
Warning: Some schemes needs a **facsec** lower than 1 (0.5 is a good start), for example Schema\_Adams\_Bashforth\_order\_3.
- **seuil\_statio** *float* for inheritance: Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives  $dG_i/dt$  of all the unknown transported values  $G_i$  have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **residuals** *residuals* (3.127) for inheritance: To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **diffusion\_implicit** *int* for inheritance: Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ( $dt=facsec*dt_{convection}$ ). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large **facsec** value. Start with a **facsec** value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore  $dt=facsec*dt_{max}$ .
- **seuil\_diffusion\_implicit** *float* for inheritance: This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **impr\_diffusion\_implicit** *int* for inheritance: Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **impr\_extremums** *int* for inheritance: Print unknowns extremas
- **no\_error\_if\_not\_converged\_diffusion\_implicit** *int* for inheritance
- **no\_conv\_subiteration\_diffusion\_implicit** *int* for inheritance
- **dt\_start** *dt\_start* (14.11) for inheritance: **dt\_start dt\_min** : the first iteration is based on **dt\_min**.  
**dt\_start dt\_calc** : the time step at first iteration is calculated in agreement with CFL condition.  
**dt\_start dt\_fixe** value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity).  
By default, the first iteration is based on **dt\_calc**.
- **nb\_pas\_dt\_max** *int* for inheritance: Maximum number of calculation time steps (1e9 by default).
- **niter\_max\_diffusion\_implicit** *int* for inheritance: This keyword changes the default value (num-

ber of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.

- **precision\_impr** *int* for inheritance: Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **periode\_sauvegarde\_securite\_en\_heures** *float* for inheritance: To change the default period (23 hours) between the save of the fields in .sauv file.
- **no\_check\_disk\_space** for inheritance: To disable the check of the available amount of disk space during the calculation.
- **disable\_progress** for inheritance: To disable the writing of the .progress file.
- **disable\_dt\_ev** for inheritance: To disable the writing of the .dt\_ev file.
- **gnuplot\_header** *int* for inheritance: Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

## 37.24 Schema\_predictor\_corrector

Description: This is the predictor-corrector scheme (second order). It is more accurate and economic than MacCormack scheme. It gives best results with a second ordre convective scheme like quick, centre (VDF).

See also: `schema_temps_base` (37)

Usage:

**schema\_predictor\_corrector** *str*

**Read** *str* {

```
[tinit float]
[tmax float]
[tcpumax float]
[dt_min float]
[dt_max str]
[dt_sauv float]
[nb_sauv_max int]
[dt_impr float]
[facsec str]
[seuil_statio float]
[residuals residuals]
[diffusion_implicite int]
[seuil_diffusion_implicite float]
[impr_diffusion_implicite int]
[impr_extremums int]
[no_error_if_not_converged_diffusion_implicite int]
[no_conv_subiteration_diffusion_implicite int]
[dt_start dt_start]
[nb_pas_dt_max int]
[niter_max_diffusion_implicite int]
[precision_impr int]
[periode_sauvegarde_securite_en_heures float]
[no_check_disk_space]
[disable_progress]
[disable_dt_ev]
[gnuplot_header int]
```

}

where



- **tinit** *float* for inheritance: Value of initial calculation time (0 by default).
- **tmax** *float* for inheritance: Time during which the calculation will be stopped (1e30s by default).
- **tcpumax** *float* for inheritance: CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **dt\_min** *float* for inheritance: Minimum calculation time step (1e-16s by default).
- **dt\_max** *str* for inheritance: Maximum calculation time step as function of time (1e30s by default).
- **dt\_sauv** *float* for inheritance: Save time step value (1e30s by default). Every **dt\_sauv**, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that **dt\_sauv** is in terms of physical time (not cpu time).
- **nb\_sauv\_max** *int* for inheritance: Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of **nb\_sauv\_max** timesteps in the file), the next checkpoints will overwrite the first ones
- **dt\_impr** *float* for inheritance: Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **facsec** *str* for inheritance: Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the **facsec** to 0.5.  
Warning: Some schemes needs a **facsec** lower than 1 (0.5 is a good start), for example Schema\_Adams\_Bashforth\_order\_3.
- **seuil\_statio** *float* for inheritance: Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives  $dG_i/dt$  of all the unknown transported values  $G_i$  have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **residuals** *residuals* (3.127) for inheritance: To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **diffusion\_implicit** *int* for inheritance: Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ( $dt=facsec*dt_{convection}$ ). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large **facsec** value. Start with a **facsec** value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore  $dt=facsec*dt_{max}$ .
- **seuil\_diffusion\_implicit** *float* for inheritance: This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **impr\_diffusion\_implicit** *int* for inheritance: Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **impr\_extremums** *int* for inheritance: Print unknowns extremas
- **no\_error\_if\_not\_converged\_diffusion\_implicit** *int* for inheritance
- **no\_conv\_subiteration\_diffusion\_implicit** *int* for inheritance
- **dt\_start** *dt\_start* (14.11) for inheritance: **dt\_start dt\_min** : the first iteration is based on **dt\_min**.  
**dt\_start dt\_calc** : the time step at first iteration is calculated in agreement with CFL condition.  
**dt\_start dt\_fixe** value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity).  
By default, the first iteration is based on **dt\_calc**.
- **nb\_pas\_dt\_max** *int* for inheritance: Maximum number of calculation time steps (1e9 by default).
- **niter\_max\_diffusion\_implicit** *int* for inheritance: This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.



- **precision\_impr** *int* for inheritance: Optional keyword to define the digit number for flux values printed into .out files (by default 3).
- **periode\_sauvegarde\_securite\_en\_heures** *float* for inheritance: To change the default period (23 hours) between the save of the fields in .sauv file.
- **no\_check\_disk\_space** for inheritance: To disable the check of the available amount of disk space during the calculation.
- **disable\_progress** for inheritance: To disable the writing of the .progress file.
- **disable\_dt\_ev** for inheritance: To disable the writing of the .dt\_ev file.
- **gnuplot\_header** *int* for inheritance: Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

### 37.25 Schema\_euler\_explicite\_ale

Description: This is the Euler explicit scheme used for ALE problems.

See also: `schema_temps_base` ([37](#))

Usage:

**schema\_euler\_explicite\_ALE** *str*

**Read** *str* {

```
[tinit float]
[tmax float]
[tcpumax float]
[dt_min float]
[dt_max str]
[dt_sauv float]
[nb_sauv_max int]
[dt_impr float]
[facsec str]
[seuil_statio float]
[residuals residuals]
[diffusion_implicite int]
[seuil_diffusion_implicite float]
[impr_diffusion_implicite int]
[impr_extremums int]
[no_error_if_not_converged_diffusion_implicite int]
[no_conv_subiteration_diffusion_implicite int]
[dt_start dt_start]
[nb_pas_dt_max int]
[niter_max_diffusion_implicite int]
[precision_impr int]
[periode_sauvegarde_securite_en_heures float]
[no_check_disk_space]
[disable_progress]
[disable_dt_ev]
[gnuplot_header int]
```

}

where

- **tinit** *float* for inheritance: Value of initial calculation time (0 by default).
- **tmax** *float* for inheritance: Time during which the calculation will be stopped (1e30s by default).

- **tcpumax** *float* for inheritance: CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).
- **dt\_min** *float* for inheritance: Minimum calculation time step (1e-16s by default).
- **dt\_max** *str* for inheritance: Maximum calculation time step as function of time (1e30s by default).
- **dt\_sauv** *float* for inheritance: Save time step value (1e30s by default). Every **dt\_sauv**, fields are saved in the .sauv file. The file contains all the information saved over time. If this instruction is not entered, results are saved only upon calculation completion. To disable the writing of the .sauv files, you must specify 0. Note that **dt\_sauv** is in terms of physical time (not cpu time).
- **nb\_sauv\_max** *int* for inheritance: Maximum number of timesteps that will be stored in backup file (10 by default). This value is only useful when doing a complete backup of the calculation with parallel PDI (as it needs to allocate the proper amount of dataspace in advance). If this number is reached (ie we already stored the data of **nb\_sauv\_max** timesteps in the file), the next checkpoints will overwrite the first ones
- **dt\_impr** *float* for inheritance: Scheme parameter printing time step in time (1e30s by default). The time steps and the flux balances are printed (incorporated onto every side of processed domains) into the .out file.
- **facsec** *str* for inheritance: Value assigned to the safety factor for the time step (1. by default). It can also be a function of time. The time step calculated is multiplied by the safety factor. The first thing to try when a calculation does not converge with an explicit time scheme is to reduce the **facsec** to 0.5.  
Warning: Some schemes needs a **facsec** lower than 1 (0.5 is a good start), for example Schema\_Adams\_Bashforth\_order\_3.
- **seuil\_statio** *float* for inheritance: Value of the convergence threshold (1e-12 by default). Problems using this type of time scheme converge when the derivatives  $dG_i/dt$  of all the unknown transported values  $G_i$  have a combined absolute value less than this value. This is the keyword used to set the permanent rating threshold.
- **residuals** *residuals* (3.127) for inheritance: To specify how the residuals will be computed (default max norm, possible to choose L2-norm instead).
- **diffusion\_implicit** *int* for inheritance: Keyword to make the diffusive term in the Navier-Stokes equations implicit (in this case, it should be set to 1). The stability time step is then only based on the convection time step ( $dt=facsec*dt_{convection}$ ). Thus, in some circumstances, an important gain is achieved with respect to the time step (large diffusion with respect to convection on tightened meshes). Caution: It is however recommended that the user avoids exceeding the convection time step by selecting a too large **facsec** value. Start with a **facsec** value of 1 and then increase it gradually if you wish to accelerate calculation. In addition, for a natural convection calculation with a zero initial velocity, in the first time step, the convection time is infinite and therefore  $dt=facsec*dt_{max}$ .
- **seuil\_diffusion\_implicit** *float* for inheritance: This keyword changes the default value (1e-6) of convergency criteria for the resolution by conjugate gradient used for implicit diffusion.
- **impr\_diffusion\_implicit** *int* for inheritance: Unactivate (default) or not the printing of the convergence during the resolution of the conjugate gradient.
- **impr\_extremums** *int* for inheritance: Print unknowns extremas
- **no\_error\_if\_not\_converged\_diffusion\_implicit** *int* for inheritance
- **no\_conv\_subiteration\_diffusion\_implicit** *int* for inheritance
- **dt\_start** *dt\_start* (14.11) for inheritance: **dt\_start dt\_min** : the first iteration is based on **dt\_min**.  
**dt\_start dt\_calc** : the time step at first iteration is calculated in agreement with CFL condition.  
**dt\_start dt\_fixe** value : the first time step is fixed by the user (recommended when resuming calculation with Crank Nicholson temporal scheme to ensure continuity).  
By default, the first iteration is based on **dt\_calc**.
- **nb\_pas\_dt\_max** *int* for inheritance: Maximum number of calculation time steps (1e9 by default).
- **niter\_max\_diffusion\_implicit** *int* for inheritance: This keyword changes the default value (number of unknowns) of the maximal iterations number in the conjugate gradient method used for implicit diffusion.
- **precision\_impr** *int* for inheritance: Optional keyword to define the digit number for flux values printed into .out files (by default 3).

- **periode\_sauvegarde\_securite\_en\_heures** *float* for inheritance: To change the default period (23 hours) between the save of the fields in .sauv file.
- **no\_check\_disk\_space** for inheritance: To disable the check of the available amount of disk space during the calculation.
- **disable\_progress** for inheritance: To disable the writing of the .progress file.
- **disable\_dt\_ev** for inheritance: To disable the writing of the .dt\_ev file.
- **gnuplot\_header** *int* for inheritance: Optional keyword to modify the header of the .out files. Allows to use the column title instead of columns number.

## 38 schema\_temps\_base\_ijk

Description: Basic class for time schemes. This scheme will be associated with a problem and the equations of this problem.

See also: objet\_u (48) Schema\_RK3\_IJK (38.1) Schema\_euler\_explicite\_IJK (38.2)

Usage:

**schema\_temps\_base\_IJK** *str*

**Read** *str* {

```

 tinit float
 timestep float
 [timestep_facsec float]
 [cfl float]
 [fo float]
 [oh float]
 nb_pas_dt_max int
 [max_simu_time float]
 [tstep_init int]
 [use_tstep_init int]
 [dt_sauvegarde int]
 [tcpumax float]

```

}

where

- **tinit** *float*: initial time
- **timestep** *float*: Upper limit of the timestep
- **timestep\_facsec** *float*: Security factor on timestep
- **cfl** *float*: To provide a value of the limiting CFL number used for setting the timestep
- **fo** *float*
- **oh** *float*
- **nb\_pas\_dt\_max** *int*: maximum limit for the number of timesteps
- **max\_simu\_time** *float*: maximum limit for the simulation time
- **tstep\_init** *int*: index first iteration for recovery
- **use\_tstep\_init** *int*: use tstep init for constant post-processing step
- **dt\_sauvegarde** *int*: saving frequency (writing files for computation restart)
- **tcpumax** *float*: CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).

### 38.1 Schema\_rk3\_ijk

Description: Basic class for time schemes. This scheme will be associated with a problem and the equations of this problem.

See also: `schema_temps_base_IJK` (38)

Usage:

**Schema\_RK3\_IJK** *str*

**Read** *str* {

```
 tinit float
 timestep float
 [timestep_facsec float]
 [cfl float]
 [fo float]
 [oh float]
 nb_pas_dt_max int
 [max_simu_time float]
 [tstep_init int]
 [use_tstep_init int]
 [dt_sauvegarde int]
 [tcpumax float]
```

}

where

- **tinit** *float* for inheritance: initial time
- **timestep** *float* for inheritance: Upper limit of the timestep
- **timestep\_facsec** *float* for inheritance: Security factor on timestep
- **cfl** *float* for inheritance: To provide a value of the limiting CFL number used for setting the timestep
- **fo** *float* for inheritance
- **oh** *float* for inheritance
- **nb\_pas\_dt\_max** *int* for inheritance: maximum limit for the number of timesteps
- **max\_simu\_time** *float* for inheritance: maximum limit for the simulation time
- **tstep\_init** *int* for inheritance: index first iteration for recovery
- **use\_tstep\_init** *int* for inheritance: use tstep init for constant post-processing step
- **dt\_sauvegarde** *int* for inheritance: saving frequency (writing files for computation restart)
- **tcpumax** *float* for inheritance: CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).

## 38.2 Schema\_euler\_explicite\_ijk

Description: Basic class for time schemes. This scheme will be associated with a problem and the equations of this problem.

See also: `schema_temps_base_IJK` (38)

Usage:

**Schema\_euler\_explicite\_IJK** *str*

**Read** *str* {

```
 tinit float
 timestep float
 [timestep_facsec float]
 [cfl float]
 [fo float]
```

```

[oh float]
nb_pas_dt_max int
[max_simu_time float]
[tstep_init int]
[use_tstep_init int]
[dt_sauvegarde int]
[tcpumax float]
}
where

```

- **tinit** *float* for inheritance: initial time
- **timestep** *float* for inheritance: Upper limit of the timestep
- **timestep\_facsec** *float* for inheritance: Security factor on timestep
- **cfl** *float* for inheritance: To provide a value of the limiting CFL number used for setting the timestep
- **fo** *float* for inheritance
- **oh** *float* for inheritance
- **nb\_pas\_dt\_max** *int* for inheritance: maximum limit for the number of timesteps
- **max\_simu\_time** *float* for inheritance: maximum limit for the simulation time
- **tstep\_init** *int* for inheritance: index first iteration for recovery
- **use\_tstep\_init** *int* for inheritance: use tstep init for constant post-processing step
- **dt\_sauvegarde** *int* for inheritance: saving frequency (writing files for computation restart)
- **tcpumax** *float* for inheritance: CPU time limit (must be specified in hours) for which the calculation is stopped (1e30s by default).

## 39 solveur\_implicite\_base

Description: Class for solver in the situation where the time scheme is the implicit scheme. Solver allows equation diffusion and convection operators to be set as implicit terms.

See also: objet\_u (48) solveur\_lineaire\_std (39.9) simplifier (39.8)

Usage:

### 39.1 Ice

Description: Implicit Continuous-fluid Eulerian solver which is useful for a multiphase problem. Robust pressure reduction resolution.

See also: sets (39.6)

Usage:

**ice** *str*

**Read** *str* {

```

[pression_degeneree int]
[pressure_reduction|reduction_preSSION int]
[criteres_convergence bloc_criteres_convergence]
[iter_min int]
[iter_max int]
[seuil_convergence_implicite float]
[nb_corrections_max int]
[facsec_diffusion_for_sets float]

```

```

[seuil_convergence_solveur float]
[seuil_generation_solveur float]
[seuil_verification_solveur float]
[seuil_test_preliminaire_solveur float]
[solveur solveur_sys_base]
[no_qdm]
[nb_it_max int]
[controle_residu]
}
where

```

- **pression\_degeneree** *int*: Set to 1 if the pressure field is degenerate (ex. : incompressible fluid with no imposed-pressure BCs). Default: autodetected
- **pressure\_reduction** *int*: Set to 1 if the user wants a resolution with a pressure reduction. Otherwise, the value is to be set to 0 so that the complete matrix is considered. The default value of this value is 1.
- **criteres\_convergence** *bloc\_criteres\_convergence* (3.2.2) for inheritance: Set the convergence thresholds for each unknown (i.e: alpha, temperature, velocity and pressure). The default values are respectively 0.01, 0.1, 0.01 and 100
- **iter\_min** *int* for inheritance: Number of minimum iterations (default value 1)
- **iter\_max** *int* for inheritance: Number of maximum iterations (default value 10)
- **seuil\_convergence\_implicit** *float* for inheritance: Convergence criteria.
- **nb\_corrections\_max** *int* for inheritance: Maximum number of corrections performed by the PISO algorithm to achieve the projection of the velocity field. The algorithm may perform less corrections than nb\_corrections\_max if the accuracy of the projection is sufficient. (By default nb\_corrections\_max is set to 21).
- **facsec\_diffusion\_for\_sets** *float* for inheritance: facsec to impose on the diffusion time step in sets while the total time step stays smaller than the convection time step.
- **seuil\_convergence\_solveur** *float* for inheritance: value of the convergence criteria for the resolution of the implicit system build by solving several times per time step the Navier-Stokes equation and the scalar equations if any. This value MUST be used when a coupling between problems is considered (should be set to a value typically of 0.1 or 0.01).
- **seuil\_generation\_solveur** *float* for inheritance: Option to create a GMRES solver and use vrel as the convergence threshold (implicit linear system  $Ax=B$  will be solved if residual error  $\|Ax-B\|$  is lesser than vrel).
- **seuil\_verification\_solveur** *float* for inheritance: Option to check if residual error  $\|Ax-B\|$  is lesser than vrel after the implicit linear system  $Ax=B$  has been solved.
- **seuil\_test\_preliminaire\_solveur** *float* for inheritance: Option to decide if the implicit linear system  $Ax=B$  should be solved by checking if the residual error  $\|Ax-B\|$  is bigger than vrel.
- **solveur** *solveur\_sys\_base* (14.19) for inheritance: Method (different from the default one, Gmres with diagonal preconditioning) to solve the linear system.
- **no\_qdm** for inheritance: Keyword to not solve qdm equation (and turbulence models of these equation).
- **nb\_it\_max** *int* for inheritance: Keyword to set the maximum iterations number for the Gmres.
- **controle\_residu** for inheritance: Keyword of Boolean type (by default 0). If set to 1, the convergence occurs if the residu suddenly increases.

## 39.2 Implicit\_steady

Description: this is the implicit solver using a dual time step. Remark: this solver can be used only with the Implicit\_Euler\_Steady\_Scheme time scheme.

See also: implicit (39.3)

Usage:

**implicit\_steady** *str*

**Read** *str* {

```
[seuil_convergence_implicit float]
[nb_corrections_max int]
[seuil_convergence_solveur float]
[seuil_generation_solveur float]
[seuil_verification_solveur float]
[seuil_test_preliminaire_solveur float]
[solveur solveur_sys_base]
[no_qdm]
[nb_it_max int]
[controle_residu]
```

}

where

- **seuil\_convergence\_implicit** *float* for inheritance: Convergence criteria.
- **nb\_corrections\_max** *int* for inheritance: Maximum number of corrections performed by the PISO algorithm to achieve the projection of the velocity field. The algorithm may perform less corrections than `nb_corrections_max` if the accuracy of the projection is sufficient. (By default `nb_corrections_max` is set to 21).
- **seuil\_convergence\_solveur** *float* for inheritance: value of the convergence criteria for the resolution of the implicit system build by solving several times per time step the Navier\_Stokes equation and the scalar equations if any. This value **MUST** be used when a coupling between problems is considered (should be set to a value typically of 0.1 or 0.01).
- **seuil\_generation\_solveur** *float* for inheritance: Option to create a GMRES solver and use `vrel` as the convergence threshold (implicit linear system  $Ax=B$  will be solved if residual error  $\|Ax-B\|$  is lesser than `vrel`).
- **seuil\_verification\_solveur** *float* for inheritance: Option to check if residual error  $\|Ax-B\|$  is lesser than `vrel` after the implicit linear system  $Ax=B$  has been solved.
- **seuil\_test\_preliminaire\_solveur** *float* for inheritance: Option to decide if the implicit linear system  $Ax=B$  should be solved by checking if the residual error  $\|Ax-B\|$  is bigger than `vrel`.
- **solveur** *solveur\_sys\_base* (14.19) for inheritance: Method (different from the default one, Gmres with diagonal preconditioning) to solve the linear system.
- **no\_qdm** for inheritance: Keyword to not solve qdm equation (and turbulence models of these equation).
- **nb\_it\_max** *int* for inheritance: Keyword to set the maximum iterations number for the Gmres.
- **controle\_residu** for inheritance: Keyword of Boolean type (by default 0). If set to 1, the convergence occurs if the residu suddenly increases.

### 39.3 Implicit

Description: similar to PISO, but as it looks like a simplified solver, it will use fewer timesteps. But it may run faster because the pressure matrix is not re-assembled and thus provides CPU gains.

See also: `piso` (39.5) `implicit_ALE` (39.4) `implicit_steady` (39.2)

Usage:

**implicit** *str*

**Read** *str* {

```
[seuil_convergence_implicit float]
```

```

[nb_corrections_max int]
[seuil_convergence_solveur float]
[seuil_generation_solveur float]
[seuil_verification_solveur float]
[seuil_test_preliminaire_solveur float]
[solveur solveur_sys_base]
[no_qdm]
[nb_it_max int]
[controle_residu]

```

}

where

- **seuil\_convergence\_implicite** *float* for inheritance: Convergence criteria.
- **nb\_corrections\_max** *int* for inheritance: Maximum number of corrections performed by the PISO algorithm to achieve the projection of the velocity field. The algorithm may perform less corrections than nb\_corrections\_max if the accuracy of the projection is sufficient. (By default nb\_corrections\_max is set to 21).
- **seuil\_convergence\_solveur** *float* for inheritance: value of the convergence criteria for the resolution of the implicit system build by solving several times per time step the Navier-Stokes equation and the scalar equations if any. This value MUST be used when a coupling between problems is considered (should be set to a value typically of 0.1 or 0.01).
- **seuil\_generation\_solveur** *float* for inheritance: Option to create a GMRES solver and use vrel as the convergence threshold (implicit linear system  $Ax=B$  will be solved if residual error  $\|Ax-B\|$  is lesser than vrel).
- **seuil\_verification\_solveur** *float* for inheritance: Option to check if residual error  $\|Ax-B\|$  is lesser than vrel after the implicit linear system  $Ax=B$  has been solved.
- **seuil\_test\_preliminaire\_solveur** *float* for inheritance: Option to decide if the implicit linear system  $Ax=B$  should be solved by checking if the residual error  $\|Ax-B\|$  is bigger than vrel.
- **solveur** *solveur\_sys\_base* (14.19) for inheritance: Method (different from the default one, Gmres with diagonal preconditioning) to solve the linear system.
- **no\_qdm** for inheritance: Keyword to not solve qdm equation (and turbulence models of these equation).
- **nb\_it\_max** *int* for inheritance: Keyword to set the maximum iterations number for the Gmres.
- **controle\_residu** for inheritance: Keyword of Boolean type (by default 0). If set to 1, the convergence occurs if the residu suddenly increases.

## 39.4 Implicite\_ale

Description: Implicite solver used for ALE problem

See also: implicite (39.3)

Usage:

**implicite\_ALE** *str*

**Read** *str* {

```

[seuil_convergence_implicite float]
[nb_corrections_max int]
[seuil_convergence_solveur float]
[seuil_generation_solveur float]
[seuil_verification_solveur float]
[seuil_test_preliminaire_solveur float]
[solveur solveur_sys_base]

```



```

[no_qdm]
[nb_it_max int]
[controle_residu]
}
where

```

- **seuil\_convergence\_implicite** *float* for inheritance: Convergence criteria.
- **nb\_corrections\_max** *int* for inheritance: Maximum number of corrections performed by the PISO algorithm to achieve the projection of the velocity field. The algorithm may perform less corrections than nb\_corrections\_max if the accuracy of the projection is sufficient. (By default nb\_corrections\_max is set to 21).
- **seuil\_convergence\_solveur** *float* for inheritance: value of the convergence criteria for the resolution of the implicit system build by solving several times per time step the Navier-Stokes equation and the scalar equations if any. This value MUST be used when a coupling between problems is considered (should be set to a value typically of 0.1 or 0.01).
- **seuil\_generation\_solveur** *float* for inheritance: Option to create a GMRES solver and use vrel as the convergence threshold (implicit linear system  $Ax=B$  will be solved if residual error  $\|Ax-B\|$  is lesser than vrel).
- **seuil\_verification\_solveur** *float* for inheritance: Option to check if residual error  $\|Ax-B\|$  is lesser than vrel after the implicit linear system  $Ax=B$  has been solved.
- **seuil\_test\_preliminaire\_solveur** *float* for inheritance: Option to decide if the implicit linear system  $Ax=B$  should be solved by checking if the residual error  $\|Ax-B\|$  is bigger than vrel.
- **solveur** *solveur\_sys\_base* (14.19) for inheritance: Method (different from the default one, Gmres with diagonal preconditioning) to solve the linear system.
- **no\_qdm** for inheritance: Keyword to not solve qdm equation (and turbulence models of these equation).
- **nb\_it\_max** *int* for inheritance: Keyword to set the maximum iterations number for the Gmres.
- **controle\_residu** for inheritance: Keyword of Boolean type (by default 0). If set to 1, the convergence occurs if the residu suddenly increases.

## 39.5 Piso

Description: Piso (Pressure Implicit with Split Operator) - method to solve N\_S.

See also: simpler (39.8) simple (39.7) implicite (39.3)

Usage:

**piso** *str*

**Read** *str* {

```

[seuil_convergence_implicite float]
[nb_corrections_max int]
[seuil_convergence_solveur float]
[seuil_generation_solveur float]
[seuil_verification_solveur float]
[seuil_test_preliminaire_solveur float]
[solveur solveur_sys_base]
[no_qdm]
[nb_it_max int]
[controle_residu]

```

```

}
where

```

- **seuil\_convergence\_implicit** *float*: Convergence criteria.
- **nb\_corrections\_max** *int*: Maximum number of corrections performed by the PISO algorithm to achieve the projection of the velocity field. The algorithm may perform less corrections than `nb_corrections_max` if the accuracy of the projection is sufficient. (By default `nb_corrections_max` is set to 21).
- **seuil\_convergence\_solveur** *float* for inheritance: value of the convergence criteria for the resolution of the implicit system build by solving several times per time step the Navier-Stokes equation and the scalar equations if any. This value **MUST** be used when a coupling between problems is considered (should be set to a value typically of 0.1 or 0.01).
- **seuil\_generation\_solveur** *float* for inheritance: Option to create a GMRES solver and use `vrel` as the convergence threshold (implicit linear system  $Ax=B$  will be solved if residual error  $\|Ax-B\|$  is lesser than `vrel`).
- **seuil\_verification\_solveur** *float* for inheritance: Option to check if residual error  $\|Ax-B\|$  is lesser than `vrel` after the implicit linear system  $Ax=B$  has been solved.
- **seuil\_test\_preliminaire\_solveur** *float* for inheritance: Option to decide if the implicit linear system  $Ax=B$  should be solved by checking if the residual error  $\|Ax-B\|$  is bigger than `vrel`.
- **solveur** *solveur\_sys\_base* (14.19) for inheritance: Method (different from the default one, Gmres with diagonal preconditioning) to solve the linear system.
- **no\_qdm** for inheritance: Keyword to not solve qdm equation (and turbulence models of these equation).
- **nb\_it\_max** *int* for inheritance: Keyword to set the maximum iterations number for the Gmres.
- **controle\_residu** for inheritance: Keyword of Boolean type (by default 0). If set to 1, the convergence occurs if the `residu` suddenly increases.

## 39.6 Sets

Description: Stability-Enhancing Two-Step solver which is useful for a multiphase problem. Ref : J. H. MAHAFFY, A stability-enhancing two-step method for fluid flow calculations, Journal of Computational Physics, 46, 3, 329 (1982).

See also: `simpler` (39.8) `ice` (39.1)

Usage:

**sets** *str*

**Read** *str* {

```
[criteres_convergence bloc_criteres_convergence]
[iter_min int]
[iter_max int]
[seuil_convergence_implicit float]
[nb_corrections_max int]
[facsec_diffusion_for_sets float]
[seuil_convergence_solveur float]
[seuil_generation_solveur float]
[seuil_verification_solveur float]
[seuil_test_preliminaire_solveur float]
[solveur solveur_sys_base]
[no_qdm]
[nb_it_max int]
[controle_residu]
```

}

where

- **criteres\_convergence** *bloc\_criteres\_convergence* (3.2.2): Set the convergence thresholds for each unknown (i.e: alpha, temperature, velocity and pressure). The default values are respectively 0.01, 0.1, 0.01 and 100
- **iter\_min** *int*: Number of minimum iterations (default value 1)
- **iter\_max** *int*: Number of maximum iterations (default value 10)
- **seuil\_convergence\_implicit** *float*: Convergence criteria.
- **nb\_corrections\_max** *int*: Maximum number of corrections performed by the PISO algorithm to achieve the projection of the velocity field. The algorithm may perform less corrections than nb\_corrections\_max if the accuracy of the projection is sufficient. (By default nb\_corrections\_max is set to 21).
- **facsec\_diffusion\_for\_sets** *float*: facsec to impose on the diffusion time step in sets while the total time step stays smaller than the convection time step.
- **seuil\_convergence\_solveur** *float* for inheritance: value of the convergence criteria for the resolution of the implicit system build by solving several times per time step the Navier\_Stokes equation and the scalar equations if any. This value MUST be used when a coupling between problems is considered (should be set to a value typically of 0.1 or 0.01).
- **seuil\_generation\_solveur** *float* for inheritance: Option to create a GMRES solver and use vrel as the convergence threshold (implicit linear system  $Ax=B$  will be solved if residual error  $\|Ax-B\|$  is lesser than vrel).
- **seuil\_verification\_solveur** *float* for inheritance: Option to check if residual error  $\|Ax-B\|$  is lesser than vrel after the implicit linear system  $Ax=B$  has been solved.
- **seuil\_test\_preliminaire\_solveur** *float* for inheritance: Option to decide if the implicit linear system  $Ax=B$  should be solved by checking if the residual error  $\|Ax-B\|$  is bigger than vrel.
- **solveur** *solveur\_sys\_base* (14.19) for inheritance: Method (different from the default one, Gmres with diagonal preconditioning) to solve the linear system.
- **no\_qdm** for inheritance: Keyword to not solve qdm equation (and turbulence models of these equation).
- **nb\_it\_max** *int* for inheritance: Keyword to set the maximum iterations number for the Gmres.
- **controle\_residu** for inheritance: Keyword of Boolean type (by default 0). If set to 1, the convergence occurs if the residu suddenly increases.

## 39.7 Simple

Description: SIMPLE type algorithm

See also: piso (39.5) solveur\_u\_p (39.10)

Usage:

**simple** *str*

**Read** *str* {

```
[relax_pression float]
[seuil_convergence_implicit float]
[nb_corrections_max int]
[seuil_convergence_solveur float]
[seuil_generation_solveur float]
[seuil_verification_solveur float]
[seuil_test_preliminaire_solveur float]
[solveur solveur_sys_base]
[no_qdm]
[nb_it_max int]
[controle_residu]
```

}

where

- **relax\_pression** *float*: Value between 0 and 1 (by default 1), this keyword is used only by the SIMPLE algorithm for relaxing the increment of pressure.
- **seuil\_convergence\_implicite** *float* for inheritance: Convergence criteria.
- **nb\_corrections\_max** *int* for inheritance: Maximum number of corrections performed by the PISO algorithm to achieve the projection of the velocity field. The algorithm may perform less corrections than nb\_corrections\_max if the accuracy of the projection is sufficient. (By default nb\_corrections\_max is set to 21).
- **seuil\_convergence\_solveur** *float* for inheritance: value of the convergence criteria for the resolution of the implicit system build by solving several times per time step the Navier\_Stokes equation and the scalar equations if any. This value MUST be used when a coupling between problems is considered (should be set to a value typically of 0.1 or 0.01).
- **seuil\_generation\_solveur** *float* for inheritance: Option to create a GMRES solver and use vrel as the convergence threshold (implicit linear system  $Ax=B$  will be solved if residual error  $\|Ax-B\|$  is lesser than vrel).
- **seuil\_verification\_solveur** *float* for inheritance: Option to check if residual error  $\|Ax-B\|$  is lesser than vrel after the implicit linear system  $Ax=B$  has been solved.
- **seuil\_test\_preliminaire\_solveur** *float* for inheritance: Option to decide if the implicit linear system  $Ax=B$  should be solved by checking if the residual error  $\|Ax-B\|$  is bigger than vrel.
- **solveur** *solveur\_sys\_base* (14.19) for inheritance: Method (different from the default one, Gmres with diagonal preconditioning) to solve the linear system.
- **no\_qdm** for inheritance: Keyword to not solve qdm equation (and turbulence models of these equation).
- **nb\_it\_max** *int* for inheritance: Keyword to set the maximum iterations number for the Gmres.
- **controle\_residu** for inheritance: Keyword of Boolean type (by default 0). If set to 1, the convergence occurs if the residu suddenly increases.

## 39.8 Simplifier

Description: Simplifier method for incompressible systems.

See also: solveur\_implicite\_base (39) piso (39.5) sets (39.6)

Usage:

**simpler** *str*

**Read** *str* {

```
 seuil_convergence_implicite float
 [seuil_convergence_solveur float]
 [seuil_generation_solveur float]
 [seuil_verification_solveur float]
 [seuil_test_preliminaire_solveur float]
 [solveur solveur_sys_base]
 [no_qdm]
 [nb_it_max int]
 [controle_residu]
```

}

where

- **seuil\_convergence\_implicite** *float*: Keyword to set the value of the convergence criteria for the resolution of the implicit system build to solve either the Navier\_Stokes equation (only for Simple and Simplifier algorithms) or a scalar equation. It is advised to use the default value (1e6) to solve

the implicit system only once by time step. This value must be decreased when a coupling between problems is considered.

- **seuil\_convergence\_solveur** *float*: value of the convergence criteria for the resolution of the implicit system build by solving several times per time step the Navier\_Stokes equation and the scalar equations if any. This value **MUST** be used when a coupling between problems is considered (should be set to a value typically of 0.1 or 0.01).
- **seuil\_generation\_solveur** *float*: Option to create a GMRES solver and use *vrel* as the convergence threshold (implicit linear system  $Ax=B$  will be solved if residual error  $\|Ax-B\|$  is lesser than *vrel*).
- **seuil\_verification\_solveur** *float*: Option to check if residual error  $\|Ax-B\|$  is lesser than *vrel* after the implicit linear system  $Ax=B$  has been solved.
- **seuil\_test\_preliminaire\_solveur** *float*: Option to decide if the implicit linear system  $Ax=B$  should be solved by checking if the residual error  $\|Ax-B\|$  is bigger than *vrel*.
- **solveur** *solveur\_sys\_base* (14.19): Method (different from the default one, Gmres with diagonal preconditioning) to solve the linear system.
- **no\_qdm** : Keyword to not solve qdm equation (and turbulence models of these equation).
- **nb\_it\_max** *int*: Keyword to set the maximum iterations number for the Gmres.
- **controle\_residu** : Keyword of Boolean type (by default 0). If set to 1, the convergence occurs if the *residu* suddenly increases.

### 39.9 Solveur\_lineaire\_std

Description: not\_set

See also: *solveur\_implicite\_base* (39)

Usage:

**solveur\_lineaire\_std** *str*

**Read** *str* {

    [ **solveur** *solveur\_sys\_base*]

}

where

- **solveur** *solveur\_sys\_base* (14.19)

### 39.10 Solveur\_u\_p

Description: similar to simple.

See also: *simple* (39.7)

Usage:

**solveur\_u\_p** *str*

**Read** *str* {

    [ **relax\_pression** *float*]

    [ **seuil\_convergence\_implicite** *float*]

    [ **nb\_corrections\_max** *int*]

    [ **seuil\_convergence\_solveur** *float*]

    [ **seuil\_generation\_solveur** *float*]

    [ **seuil\_verification\_solveur** *float*]

    [ **seuil\_test\_preliminaire\_solveur** *float*]

    [ **solveur** *solveur\_sys\_base*]

```

[no_qdm]
[nb_it_max int]
[controle_residu]
}
where

```

- **relax\_pression** *float* for inheritance: Value between 0 and 1 (by default 1), this keyword is used only by the SIMPLE algorithm for relaxing the increment of pressure.
- **seuil\_convergence\_implicit** *float* for inheritance: Convergence criteria.
- **nb\_corrections\_max** *int* for inheritance: Maximum number of corrections performed by the PISO algorithm to achieve the projection of the velocity field. The algorithm may perform less corrections than nb\_corrections\_max if the accuracy of the projection is sufficient. (By default nb\_corrections\_max is set to 21).
- **seuil\_convergence\_solveur** *float* for inheritance: value of the convergence criteria for the resolution of the implicit system build by solving several times per time step the Navier-Stokes equation and the scalar equations if any. This value MUST be used when a coupling between problems is considered (should be set to a value typically of 0.1 or 0.01).
- **seuil\_generation\_solveur** *float* for inheritance: Option to create a GMRES solver and use vrel as the convergence threshold (implicit linear system  $Ax=B$  will be solved if residual error  $\|Ax-B\|$  is lesser than vrel).
- **seuil\_verification\_solveur** *float* for inheritance: Option to check if residual error  $\|Ax-B\|$  is lesser than vrel after the implicit linear system  $Ax=B$  has been solved.
- **seuil\_test\_preliminaire\_solveur** *float* for inheritance: Option to decide if the implicit linear system  $Ax=B$  should be solved by checking if the residual error  $\|Ax-B\|$  is bigger than vrel.
- **solveur** *solveur\_sys\_base* (14.19) for inheritance: Method (different from the default one, Gmres with diagonal preconditioning) to solve the linear system.
- **no\_qdm** for inheritance: Keyword to not solve qdm equation (and turbulence models of these equation).
- **nb\_it\_max** *int* for inheritance: Keyword to set the maximum iterations number for the Gmres.
- **controle\_residu** for inheritance: Keyword of Boolean type (by default 0). If set to 1, the convergence occurs if the residu suddenly increases.

## 40 solveur\_petsc\_deriv

Description: Additional information is available in the PETSC documentation: <https://petsc.org/release/manual/>

See also: objet\_u (48) lu (40.14) Cholesky\_superlu (40.4) Cholesky\_pastix (40.3) Cholesky\_umfpack (40.5) Cholesky\_out\_of\_core (40.2) cholesky (40.8) cholesky\_mumps\_blr (40.9) cli (40.10) cli\_quiet (40.11) IBICGSTAB (40.6) BICGSTAB (40.1) gmres (40.13) gcp (40.12) PIPECG (40.7)

Usage:

**solveur\_petsc\_deriv** *str*

**Read** *str* {

```

[seuil float]
[quiet]
[impr]
[rtol float]
[atol float]
[save_matrix_mtx_format]

```

```

}
where

```

- **seuil** *float*: corresponds to the iterative solver convergence value. The iterative solver converges when the Euclidean residue standard  $\|Ax-B\|$  is less than **seuil**.
- **quiet** : is a keyword which is used to not displaying any outputs of the solver.
- **impr** : used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).
- **rtol** *float*
- **atol** *float*
- **save\_matrix\_mtx\_format**

## 40.1 Bicgstab

Description: Stabilized Bi-Conjugate Gradient

See also: `solveur_petsc_deriv` (40)

Usage:

**BICGSTAB** *str*

**Read** *str* {

```
[precond preconditionneur_petsc_deriv]
[seuil float]
[quiet]
[impr]
[rtol float]
[atol float]
[save_matrix_mtx_format]
```

}

where

- **precond** *preconditionneur\_petsc\_deriv* (36)
- **seuil** *float* for inheritance: corresponds to the iterative solver convergence value. The iterative solver converges when the Euclidean residue standard  $\|Ax-B\|$  is less than **seuil**.
- **quiet** for inheritance: is a keyword which is used to not displaying any outputs of the solver.
- **impr** for inheritance: used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).
- **rtol** *float* for inheritance
- **atol** *float* for inheritance
- **save\_matrix\_mtx\_format** for inheritance

## 40.2 Cholesky\_out\_of\_core

Description: Same as the previous one but with a written LU decomposition of disk (save RAM memory but add an extra CPU cost during  $Ax=B$  solve).

See also: `solveur_petsc_deriv` (40)

Usage:

**Cholesky\_out\_of\_core** *str*

**Read** *str* {

```
[seuil float]
[quiet]
[impr]
```

```

 [rtol float]
 [atol float]
 [save_matrix_mtx_format]
}
where

```

- **seuil** *float* for inheritance: corresponds to the iterative solver convergence value. The iterative solver converges when the Euclidean residue standard  $\|Ax-B\|$  is less than **seuil**.
- **quiet** for inheritance: is a keyword which is used to not displaying any outputs of the solver.
- **impr** for inheritance: used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).
- **rtol** *float* for inheritance
- **atol** *float* for inheritance
- **save\_matrix\_mtx\_format** for inheritance

### 40.3 Cholesky\_pastix

Description: Parallelized Cholesky from PASTIX library.

See also: `solveur_petsc_deriv` ([40](#))

Usage:

**Cholesky\_pastix** *str*

```

Read str {
 [seuil float]
 [quiet]
 [impr]
 [rtol float]
 [atol float]
 [save_matrix_mtx_format]
}
where

```

- **seuil** *float* for inheritance: corresponds to the iterative solver convergence value. The iterative solver converges when the Euclidean residue standard  $\|Ax-B\|$  is less than **seuil**.
- **quiet** for inheritance: is a keyword which is used to not displaying any outputs of the solver.
- **impr** for inheritance: used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).
- **rtol** *float* for inheritance
- **atol** *float* for inheritance
- **save\_matrix\_mtx\_format** for inheritance

### 40.4 Cholesky\_superlu

Description: Parallelized Cholesky from SUPERLU\_DIST library (less CPU and RAM, efficient than the previous one)

See also: `solveur_petsc_deriv` ([40](#))

Usage:

**Cholesky\_superlu** *str*

```

Read str {

```



```

[seuil float]
[quiet]
[impr]
[rtol float]
[atol float]
[save_matrix_mtx_format]
}
where

```

- **seuil** *float* for inheritance: corresponds to the iterative solver convergence value. The iterative solver converges when the Euclidean residue standard  $\|Ax-B\|$  is less than **seuil**.
- **quiet** for inheritance: is a keyword which is used to not displaying any outputs of the solver.
- **impr** for inheritance: used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).
- **rtol** *float* for inheritance
- **atol** *float* for inheritance
- **save\_matrix\_mtx\_format** for inheritance

## 40.5 Cholesky\_umfpack

Description: Sequential Cholesky from UMFPACK library (seems fast).

See also: [solveur\\_petsc\\_deriv \(40\)](#)

Usage:

**Cholesky\_umfpack** *str*

```

Read str {
 [seuil float]
 [quiet]
 [impr]
 [rtol float]
 [atol float]
 [save_matrix_mtx_format]
}
where

```

- **seuil** *float* for inheritance: corresponds to the iterative solver convergence value. The iterative solver converges when the Euclidean residue standard  $\|Ax-B\|$  is less than **seuil**.
- **quiet** for inheritance: is a keyword which is used to not displaying any outputs of the solver.
- **impr** for inheritance: used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).
- **rtol** *float* for inheritance
- **atol** *float* for inheritance
- **save\_matrix\_mtx\_format** for inheritance

## 40.6 Ibicgstab

Description: Improved version of previous one for massive parallel computations (only a single global reduction operation instead of the usual 3 or 4).

See also: [solveur\\_petsc\\_deriv \(40\)](#)

Usage:

**IBICGSTAB** *str*

**Read** *str* {

[ **precond** *preconditionneur\_petsc\_deriv*]  
[ **seuil** *float*]  
[ **quiet** ]  
[ **impr** ]  
[ **rtol** *float*]  
[ **atol** *float*]  
[ **save\_matrix\_mtx\_format** ]

}

where

- **precond** *preconditionneur\_petsc\_deriv* (36)
- **seuil** *float* for inheritance: corresponds to the iterative solver convergence value. The iterative solver converges when the Euclidean residue standard  $\|Ax-B\|$  is less than **seuil**.
- **quiet** for inheritance: is a keyword which is used to not displaying any outputs of the solver.
- **impr** for inheritance: used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).
- **rtol** *float* for inheritance
- **atol** *float* for inheritance
- **save\_matrix\_mtx\_format** for inheritance

## 40.7 Pipecg

Description: Pipelined Conjugate Gradient (possible reduced CPU cost during massive parallel calculation due to a single non-blocking reduction per iteration, if TRUST is built with a MPI-3 implementation)... no example in TRUST

See also: *solveur\_petsc\_deriv* (40)

Usage:

**PIPECG** *str*

**Read** *str* {

[ **seuil** *float*]  
[ **quiet** ]  
[ **impr** ]  
[ **rtol** *float*]  
[ **atol** *float*]  
[ **save\_matrix\_mtx\_format** ]

}

where

- **seuil** *float* for inheritance: corresponds to the iterative solver convergence value. The iterative solver converges when the Euclidean residue standard  $\|Ax-B\|$  is less than **seuil**.
- **quiet** for inheritance: is a keyword which is used to not displaying any outputs of the solver.
- **impr** for inheritance: used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).
- **rtol** *float* for inheritance
- **atol** *float* for inheritance
- **save\_matrix\_mtx\_format** for inheritance

## 40.8 Cholesky

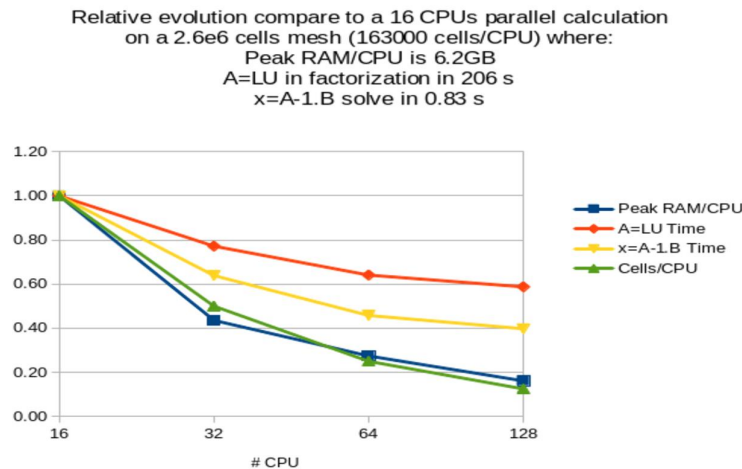
Description: Parallelized version of Cholesky from MUMPS library. This solver accepts an option to select a different ordering than the automatic selected one by MUMPS (and printed by using the `impr` option). The possible choices are Metis, Scotch, PT-Scotch or Parmetis. The two last options can only be used during a parallel calculation, whereas the two first are available for sequential or parallel calculations. It seems that the CPU cost of  $A=LU$  factorization but also of the backward/forward elimination steps may sometimes be reduced by selecting a different ordering (Scotch seems often the best for b/f elimination) than the default one.

Notice that this solver requires a huge amount of memory compared to iterative methods. To know how much RAM you will need by core, then use the `impr` option to have detailed informations during the analysis phase and before the factorisation phase (in the following output, you will learn that the largest memory is taken by the zeroth CPU with 108MB):

Rank of proc needing largest memory in IC facto : 0

Estimated corresponding MBYTES for IC facto : 108

Thanks to the following graph, you read that in order to solve for instance a flow on a mesh with 2.6e6 cells, you will need to run a parallel calculation on 32 CPUs if you have cluster nodes with only 4GB/core (6.2GB\*0.42 2.6GB) :



See also: `solveur_petsc_deriv` (40)

Usage:

**cholesky** *str*

**Read** *str* {

```
[save_matrix|save_matrice]
[save_matrix_petsc_format]
[reduce_ram]
[cli_quiet solveur_petsc_option_cli]
[cli solveur_petsc_option_cli]
[seuil float]
[quiet]
[impr]
[rtol float]
[atol float]
[save_matrix_mtx_format]
```

```
}
```

where

- **save\_matrix|save\_matrice**
- **save\_matrix\_petsc\_format**
- **reduce\_ram**
- **cliQuiet** *solveur\_petsc\_option\_cli* (3.2.1)
- **cli** *solveur\_petsc\_option\_cli* (3.2.1)
- **seuil** *float* for inheritance: corresponds to the iterative solver convergence value. The iterative solver converges when the Euclidean residue standard  $\|Ax-B\|$  is less than **seuil**.
- **quiet** for inheritance: is a keyword which is used to not displaying any outputs of the solver.
- **impr** for inheritance: used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).
- **rtol** *float* for inheritance
- **atol** *float* for inheritance
- **save\_matrix\_mtx\_format** for inheritance

## 40.9 Cholesky\_mumps\_blr

Description: BLR for (Block Low-Rank)

See also: *solveur\_petsc\_deriv* (40)

Usage:

**cholesky\_mumps\_blr** *str*

**Read** *str* {

```
[reduce_ram]
[dropping_parameter float]
[cli solveur_petsc_option_cli]
[seuil float]
[quiet]
[impr]
[rtol float]
[atol float]
[save_matrix_mtx_format]
```

```
}
```

where

- **reduce\_ram**
- **dropping\_parameter** *float*
- **cli** *solveur\_petsc\_option\_cli* (3.2.1)
- **seuil** *float* for inheritance: corresponds to the iterative solver convergence value. The iterative solver converges when the Euclidean residue standard  $\|Ax-B\|$  is less than **seuil**.
- **quiet** for inheritance: is a keyword which is used to not displaying any outputs of the solver.
- **impr** for inheritance: used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).
- **rtol** *float* for inheritance
- **atol** *float* for inheritance
- **save\_matrix\_mtx\_format** for inheritance

## 40.10 Cli

Description: Command Line Interface. Should be used only by advanced users, to access the whole solver/preconditioners from the PETSC API. To find all the available options, run your calculation with the `-ksp_view -help` options:

`trust datafile [N] -ksp_view -help`

`-pc_type` Preconditioner:(one of) none jacobi pbjacobi bjacobi sor lu shell mg eisenstat ilu icc cholesky asm ksp composite redundant nn mat fieldsplit galerkin openmp spai hypre tfs (PCSetType)

HYPRE preconditioner options:

`-pc_hypre_type` pilut (choose one of) pilut parasails boomeramg

HYPRE ParaSails Options

`-pc_hypre_parasails_nlevels` 1: Number of number of levels (None)

`-pc_hypre_parasails_thresh` 0.1: Threshold (None)

`-pc_hypre_parasails_filter` 0.1: filter (None)

`-pc_hypre_parasails_loadbal` 0: Load balance (None)

`-pc_hypre_parasails_logging`: FALSE Print info to screen (None)

`-pc_hypre_parasails_reuse`: FALSE Reuse nonzero pattern in preconditioner (None)

`-pc_hypre_parasails_sym` nonsymmetric (choose one of) nonsymmetric SPD nonsymmetric,SPD

Krylov Method (KSP) Options

`-ksp_type` Krylov method:(one of) cg cgne stcg gltr richardson chebychev gmres tcqmr bcgs bcgsl cgs tfqmr cr lsqr preonly qcg bicg fgmres minres symmlq lgmres lcd (KSPSetType)

`-ksp_max_it` 10000: Maximum number of iterations (KSPSetTolerances)

`-ksp_rtol` 0: Relative decrease in residual norm (KSPSetTolerances)

`-ksp_atol` 1e-12: Absolute value of residual norm (KSPSetTolerances)

`-ksp_divtol` 10000: Residual norm increase cause divergence (KSPSetTolerances)

`-ksp_converged_use_initial_residual_norm`: Use initial residual residual norm for computing relative convergence

`-ksp_monitor_singular_value` stdout: Monitor singular values (KSPMonitorSet)

`-ksp_monitor_short` stdout: Monitor preconditioned residual norm with fewer digits (KSPMonitorSet)

`-ksp_monitor_draw`: Monitor graphically preconditioned residual norm (KSPMonitorSet)

`-ksp_monitor_draw_true_residual`: Monitor graphically true residual norm (KSPMonitorSet)

Example to use the multigrid method as a solver, not only as a preconditioner:

Solveur\_pression Petsc CLI {`-ksp_type richardson -pc_type hypre -pc_hypre_type boomeramg -ksp_atol 1.e-7` }

See also: `solveur_petsc_deriv` (40)

Usage:

**cli cli\_bloc**

where

- **cli\_bloc** *bloc\_lecture* (3.2): bloc

## 40.11 Cli\_quiet

Description: solver

See also: `solveur_petsc_deriv` (40)

Usage:

**cli\_quiet cli\_quiet\_bloc**

where

- **cli\_quiet\_bloc** *bloc\_lecture* (3.2): bloc

## 40.12 Gcp

Description: Preconditioned Conjugate Gradient

See also: *solveur\_petsc\_deriv* (40)

Usage:

**gcp** *str*

**Read** *str* {

```
[precond preconditionneur_petsc_deriv]
[precond_nul]
[rtol float]
[reuse_preconditioner_nb_it_max int]
[cli solveur_petsc_option_cli]
[reorder_matrix int]
[read_matrix]
[save_matrix|save_matrice]
[petsc_decide int]
[pcshell str]
[aij]
[seuil float]
[quiet]
[impr]
[atol float]
[save_matrix_mtx_format]
```

}

where

- **precond** *preconditionneur\_petsc\_deriv* (36): preconditioner
- **precond\_nul** : No preconditioner used, equivalent to precondition null { }
- **rtol** *float*
- **reuse\_preconditioner\_nb\_it\_max** *int*
- **cli** *solveur\_petsc\_option\_cli* (3.2.1)
- **reorder\_matrix** *int*
- **read\_matrix** : *save\_matrix|read\_matrix* are the keywords to save/read into a file the constant matrix A of the linear system  $Ax=B$  solved (eg: matrix from the pressure linear system for an incompressible flow). It is useful when you want to minimize the MPI communications on massive parallel calculation. Indeed, in VEF discretization, the overlapping width (generally 2, specified with the *largeur\_joint* option in the partition keyword partition) can be reduced to 1, once the matrix has been properly assembled and saved. The cost of the MPI communications in TRUST itself (not in PETSc) will be reduced with length messages divided by 2. So the strategy is:
  - I) Partition your VEF mesh with a *largeur\_joint* value of 2
  - II) Run your parallel calculation on 0 time step, to build and save the matrix with the *save\_matrix* option. A file named *Matrix\_NBROWS\_rows\_NCPUS\_cpus.petsc* will be saved to the disk (where NBROWS is the number of rows of the matrix and NCPUS the number of CPUs used).
  - III) Partition your VEF mesh with a *largeur\_joint* value of 1
  - IV) Run your parallel calculation completely now and substitute the *save\_matrix* option by the *read\_matrix* option. Some interesting gains have been noticed when the cost of linear system solve with PETSc is small compared to all the other operations.
- **save\_matrix|save\_matrice** : see *read\_matrix*

- **petsc\_decide** *int*
- **pcshell** *str*
- **aij**
- **seuil** *float* for inheritance: corresponds to the iterative solver convergence value. The iterative solver converges when the Euclidean residue standard  $\|Ax-B\|$  is less than **seuil**.
- **quiet** for inheritance: is a keyword which is used to not displaying any outputs of the solver.
- **impr** for inheritance: used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).
- **atol** *float* for inheritance
- **save\_matrix\_mtx\_format** for inheritance

### 40.13 Gmres

Description: Generalized Minimal Residual

See also: `solveur_petsc_deriv` (40)

Usage:

**gmres** *str*

**Read** *str* {

```
[precond preconditionneur_petsc_deriv]
[reuse_preconditioner_nb_it_max int]
[save_matrix_petsc_format]
[nb_it_max int]
[seuil float]
[quiet]
[impr]
[rtol float]
[atol float]
[save_matrix_mtx_format]
```

}

where

- **precond** *preconditionneur\_petsc\_deriv* (36)
- **reuse\_preconditioner\_nb\_it\_max** *int*
- **save\_matrix\_petsc\_format**
- **nb\_it\_max** *int*: In order to specify a given number of iterations instead of a condition on the residue with the keyword **seuil**. May be useful when defining a PETSc solver for the implicit time scheme where convergence is very fast: 5 or less iterations seems enough.
- **seuil** *float* for inheritance: corresponds to the iterative solver convergence value. The iterative solver converges when the Euclidean residue standard  $\|Ax-B\|$  is less than **seuil**.
- **quiet** for inheritance: is a keyword which is used to not displaying any outputs of the solver.
- **impr** for inheritance: used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).
- **rtol** *float* for inheritance
- **atol** *float* for inheritance
- **save\_matrix\_mtx\_format** for inheritance

### 40.14 Lu

Description: Several solvers through PETSc API are available.

TIPS:

A) Solver for symmetric linear systems (e.g: Pressure system from Navier-Stokes equations):

-The CHOLESKY parallel solver is from MUMPS library. It offers better performance than all others solvers if you have enough RAM for your calculation. A parallel calculation on a cluster with 4GBytes on each processor, 40000 cells/processor seems the upper limit. Seems to be very slow to initialize above 500 cpus/cores.

-When running a parallel calculation with a high number of cpus/cores (typically more than 500) where preconditioner scalability is the key for CPU performance, consider BICGSTAB with BLOCK\_JACOBI\_ICC(1) as preconditioner or if not converges, GCP with BLOCK\_JACOBI\_ICC(1) as preconditioner.

-For other situations, the first choice should be GCP/SSOR. In order to fine tune the solver choice, each one of the previous list should be considered. Indeed, the CPU speed of a solver depends of a lot of parameters. You may give a try to the OPTIMAL solver to help you to find the fastest solver on your study.

B) Solver for non symmetric linear systems (e.g.: Implicit schemes):

The BICGSTAB/DIAG solver seems to offer the best performances.

See also: `solveur_petsc_deriv` (40)

Usage:

**lu** *str*

**Read** *str* {

    [ **seuil** *float* ]  
    [ **quiet** ]  
    [ **impr** ]  
    [ **rtol** *float* ]  
    [ **atol** *float* ]  
    [ **save\_matrix\_mtx\_format** ]

}

where

- **seuil** *float* for inheritance: corresponds to the iterative solver convergence value. The iterative solver converges when the Euclidean residue standard  $\|Ax-B\|$  is less than **seuil**.
- **quiet** for inheritance: is a keyword which is used to not displaying any outputs of the solver.
- **impr** for inheritance: used to request display of the Euclidean residue standard each time this iterates through the conjugated gradient (display to the standard outlet).
- **rtol** *float* for inheritance
- **atol** *float* for inheritance
- **save\_matrix\_mtx\_format** for inheritance

## 41 source\_base

Description: Basic class of source terms introduced in the equation.

See also: `objet_u` (48) `darcy` (41.32) `puissance_thermique` (41.44) `forchheimer` (41.35) `source_constituant` (41.48) `dirac` (41.33) `Correction_Antal` (41.3) `Correction_Tomiyama` (41.5) `radioactive_decay` (41.45) `Source_dep_inco_bases` (41.25) `source_generique` (41.50) `source_th_tdivu` (41.62) `source_qdm_lambdaup` (41.56) `perte_charge_circulaire` (41.38) `perte_charge_anisotrope` (41.37) `perte_charge_isotrope` (41.40) `perte_charge_directionnelle` (41.39) `acceleration` (41.27) `terme_puissance_thermique_echange_impose` (41.71) `boussinesq_temperature` (41.29) `boussinesq_concentration` (41.28) `perte_charge_reguliere` (41.41) `perte_charge_singuliere` (41.43) `coriolis` (41.31) `canal_perio` (41.30) `source_qdm` (41.55) `DP_Impose` (41.6) `vitesse_relative_base`



(41.74) flux\_interfacial (41.34) frottement\_interfacial (41.36) Portance\_interfaciale (41.14) Dispersion\_bulles (41.10) travail\_pression (41.72) source\_transport\_eps (41.64) source\_transport\_k (41.65) source\_transport\_k\_eps (41.66) source\_transport\_k\_eps\_realisable (41.69) Source\_Constituant\_Vortex (41.21) trainee (41.63) flottabilite (41.49) masse\_ajoutee (41.51) source\_rayo\_semi\_transp (41.58) source\_qdm\_phase\_field (41.57) Flux\_2groupes (41.12) Source\_Dissipation\_HZDR (41.22) Injection\_QDM\_nulle (41.13) Production\_energie\_cin\_turb (41.17) Coalescence\_bulles\_1groupe (41.1) Rupture\_bulles\_1groupe (41.18) Coalescence\_bulles\_2groupes (41.2) source\_robin\_scalaire (41.60) Rupture\_bulles\_2groupes (41.19) tenseur\_Reynolds\_externes (41.70) Source\_BIF (41.20) Production\_HZDR (41.15) Diffusion\_supplementaire\_echelle\_temp\_turb (41.9) source\_robin (41.59) source\_con\_phase\_field (41.46) Correction\_Lubchenko (41.4) Source\_Dissipation\_echelle\_temp\_taux\_diss\_turb (41.23) Terme\_dissipation\_energie\_cinetique\_turbulente (41.26) Production\_echelle\_temp\_taux\_diss\_turb (41.16) Dissipation\_echelle\_temp\_taux\_diss\_turb (41.11) Diffusion\_croisee\_echelle\_temp\_taux\_diss\_turb (41.8)

Usage:

### 41.1 Coalescence\_bulles\_1groupe

Description: Source term of interfacial area

See also: source\_base (41)

Usage:

**Coalescence\_bulles\_1groupe** **bloc** **Yao\_Morel**

where

- **bloc** *bloc\_lecture* (3.2): Should be { }
- **Yao\_Morel** *str*: Yao Morel correlation

### 41.2 Coalescence\_bulles\_2groupes

Description: Source term of interfacial area

See also: source\_base (41)

Usage:

**Coalescence\_bulles\_2groupes** **bloc** **smith**

where

- **bloc** *bloc\_lecture* (3.2): hydraulic diameter should be specified within this bloc
- **smith** *str*: Smith correlation

### 41.3 Correction\_antal

Description: Antal correction source term for multiphase problem

See also: source\_base (41)

Usage:

## 41.4 Correction\_lubchenko

Description: not\_set

See also: source\_base (41)

Usage:

**Correction\_Lubchenko** *str*

**Read** *str* {

[ **beta\_lift** *float*]

[ **beta\_disp** *float*]

}

where

- **beta\_lift** *float*
- **beta\_disp** *float*

## 41.5 Correction\_tomiyama

Description: Tomiyama correction source term for multiphase problem

See also: source\_base (41)

Usage:

## 41.6 Dp\_impose

Description: Source term to impose a pressure difference according to the formula :  $DP = dp + dDP/dQ * (Q - Q0)$

See also: source\_base (41)

Usage:

**DP\_Impose** **aco** **dp\_type** **surface** **bloc\_surface** **acof**

where

- **aco** *str* into ['{']: Opening curly bracket.
- **dp\_type** *type\_perte\_charge\_deriv* (41.7): mass flow rate (kg/s).
- **surface** *str* into ['surface']
- **bloc\_surface** *bloc\_lecture* (3.2): Three syntaxes are possible for the surface definition block:  
For VDF and VEF: { X|Y|Z = location subzone\_name }  
Only for VEF: { Surface surface\_name }.  
For polymac { Surface surface\_name Orientation champ\_uniforme }.
- **acof** *str* into ['}']: Closing curly bracket.

## 41.7 Type\_perte\_charge\_deriv

Description: not\_set

See also: objet\_lecture (47) dp (41.7.1) dp\_regul (41.7.2)

Usage:

### 41.7.1 Dp

Description: DP field should have 3 components defining dp, dDP/dQ, Q0

See also: type\_perte\_charge\_deriv (41.7)

Usage:

**dp dp\_field**

where

- **dp\_field** *champ\_base* (20.1): the parameters of the previous formula ( $DP = dp + dDP/dQ * (Q - Q0)$ ): uniform\_field 3 dp dDP/dQ Q0 where Q0 is a mass flow rate (kg/s).

### 41.7.2 Dp\_regul

Description: Keyword used to regulate the DP value in order to match a target flow rate. Syntax : dp\_regul { DP0 d deb d eps e }

See also: type\_perte\_charge\_deriv (41.7)

Usage:

**dp\_regul {**

**DP0** *float*

**deb** *str*

**eps** *str*

**}**

where

- **DP0** *float*: initial value of DP
- **deb** *str*: target flow rate in kg/s
- **eps** *str*: strength of the regulation (low values might be slow to find the target flow rate, high values might oscillate around the target value)

## 41.8 Diffusion\_croisee\_echelle\_temp\_taux\_diss\_turb

Description: Cross-diffusion source term used in the tau and omega equations

See also: source\_base (41)

Usage:

**Diffusion\_croisee\_echelle\_temp\_taux\_diss\_turb** *str*

**Read** *str* {

**[ sigma\_d** *float*

**}**

where

- **sigma\_d** *float*: Constant for the used model

## 41.9 Diffusion\_supplementaire\_echelle\_temp\_turb

Description: not\_set

See also: source\_base (41)

Usage:

**Diffusion\_supplementaire\_echelle\_temp\_turb**

## 41.10 Dispersion\_bulles

Description: Base class for source terms of bubble dispersion in momentum equation.

See also: source\_base (41)

Usage:

**Dispersion\_bulles** *str*

**Read** *str* {

[ **beta** *float*]

}

where

- **beta** *float*: Mutliplying factor for the output of the bubble dispersion source term.

## 41.11 Dissipation\_echelle\_temp\_taux\_diss\_turb

Description: Dissipation source term used in the tau and omega equations

See also: source\_base (41)

Usage:

**Dissipation\_echelle\_temp\_taux\_diss\_turb** *str*

**Read** *str* {

[ **beta\_omega** *float*]

}

where

- **beta\_omega** *float*: Constant for the used model

## 41.12 Flux\_2groupes

Description: Source term of mass and interfacial area transfer

See also: source\_base (41)

Usage:

**Flux\_2groupes** *str*

**Read** *str* {

[ **dh** *float*]

}

where

- **dh** *float*: hydraulic diameter

### 41.13 Injection\_qdm\_nulle

Description: not\_set

See also: source\_base (41)

Usage:

### 41.14 Portance\_interfaciale

Description: Base class for source term of lift force in momentum equation.

See also: source\_base (41)

Usage:

**Portance\_interfaciale** *str*

**Read** *str* {

[ **beta** *float*]

}

where

- **beta** *float*: Multiplying factor for the bubble lift force source term.

### 41.15 Production\_hzdr

Description: Additional source terms in the turbulent kinetic energy equation to model the fluctuations induced by bubbles.

See also: source\_base (41)

Usage:

**Production\_HZDR** *str*

**Read** *str* {

[ **constante\_gravitation** *float*]

[ **c\_k** *float*]

}

where

- **constante\_gravitation** *float*
- **c\_k** *float*

### 41.16 Production\_echelle\_temp\_taux\_diss\_turb

Description: Production source term used in the tau and omega equations

See also: source\_base (41)

Usage:

**Production\_echelle\_temp\_taux\_diss\_turb** *str*

**Read** *str* {

```
[alpha_omega float]
}
```

where

- **alpha\_omega** *float*: Constant for the used model

### 41.17 Production\_energie\_cin\_turb

Description: Production source term for the TKE equation

See also: [source\\_base \(41\)](#)

Usage:

### 41.18 Rupture\_bulles\_1groupe

Description: Source term of interfacial area breakup

See also: [source\\_base \(41\)](#)

Usage:

**Rupture\_bulles\_1groupe** **bloc** **Yao\_Morel**  
where

- **bloc** *bloc\_lecture* (3.2): should be { }
- **Yao\_Morel** *str*: Yao Morel correlation

### 41.19 Rupture\_bulles\_2groupes

Description: Source term of interfacial area breakup

See also: [source\\_base \(41\)](#)

Usage:

**Rupture\_bulles\_2groupes** **bloc** **smith**  
where

- **bloc** *bloc\_lecture* (3.2): hydraulic diameter should be specified
- **smith** *str*: Smith correlation

### 41.20 Source\_bif

Description: Additional fluctuations induced by the movement of bubbles, only available in PolyMAC\_P0

See also: [source\\_base \(41\)](#)

Usage:

### 41.21 Source\_constituant\_vortex

Description: Special treatment for the reactor of vortex effect where reagents are injected just below the free surface in the liquid phase

See also: [source\\_base \(41\)](#)

Usage:

**Source\_Constituant\_Vortex** *str*

**Read** *str* {

```
[senseur_interface bloc_lecture]
[rayon_spot float]
[delta_spot n x1 x2 ... xn]
[integrale float]
[debit float]
```

}

where

- **senseur\_interface** *bloc\_lecture* (3.2): This is to be defined for the concentration equation of the reagents only and in the bloc of the sources. Here the user defines the position of the reagents injection.
- **rayon\_spot** *float*: defines the radius of the concentration spot (tracer) injected in the fluid
- **delta\_spot** *n x1 x2 ... xn*: dimensions of the injection (segment). the syntax is `dim val1 val2 [val3]`
- **integrale** *float*: the molar flowrate of injection
- **debit** *float*: a normalization of the molar flow rate. Advice: keep this value to 1.

### 41.22 Source\_dissipation\_hzdr

Description: Additional source terms in the turbulent dissipation (omega) equation to model the fluctuations induced by bubbles.

See also: [source\\_base \(41\)](#)

Usage:

**Source\_Dissipation\_HZDR** *str*

**Read** *str* {

```
[constante_gravitation float]
[c_k float]
[c_epsilon float]
```

}

where

- **constante\_gravitation** *float*
- **c\_k** *float*
- **c\_epsilon** *float*

### 41.23 Source\_dissipation\_echelle\_temp\_taux\_diss\_turb

Description: Source term which corresponds to the dissipation source term that appears in the transport equation for tau (in the k-tau turbulence model)

See also: `source_base` ([41](#))

Usage:

**Source\_Dissipation\_echelle\_temp\_taux\_diss\_turb**

## 41.24 Source\_transport\_k\_eps\_anisotherme

Description: Keywords to modify the source term constants in the anisotherm standard k-eps model epsilon transport equation. By default, these constants are set to: C1\_eps=1.44 C2\_eps=1.92 C3\_eps=1.0

See also: `source_transport_k_eps` ([41.66](#))

Usage:

**Source\_Transport\_K\_Eps\_anisotherme** *str*

**Read** *str* {

    [ **c3\_eps** *float*]

    [ **c1\_eps** *float*]

    [ **c2\_eps** *float*]

}

where

- **c3\_eps** *float*: Third constant.
- **c1\_eps** *float* for inheritance: First constant.
- **c2\_eps** *float* for inheritance: Second constant.

## 41.25 Source\_dep\_inco\_bases

Description: Basic class of source terms depending of unknown.

See also: `source_base` ([41](#)) `source_pdf_base` ([41.54](#))

Usage:

## 41.26 Terme\_dissipation\_energie\_cinetique\_turbulente

Description: Dissipation source term used in the TKE equation

See also: `source_base` ([41](#))

Usage:

**Terme\_dissipation\_energie\_cinetique\_turbulente** *str*

**Read** *str* {

    [ **beta\_k** *float*]

}

where

- **beta\_k** *float*: Constant for the used model



## 41.27 Acceleration

Description: Momentum source term to take in account the forces due to rotation or translation of a non Galilean referential R' (centre 0') into the Galilean referential R (centre 0).

See also: `source_base` (41)

Usage:

**acceleration** *str*

**Read** *str* {

```
[vitesse champ_base]
[acceleration champ_base]
[omega champ_base]
[domegadt champ_base]
[centre_rotation champ_base]
[option str into ['terme_complet', 'coriolis_seul', 'entrainement_seul']]
```

}

where

- **vitesse** *champ\_base* (20.1): Keyword for the velocity of the referential R' into the R referential ( $d\mathbf{OO'}/dt$  term [m.s-1]). The velocity is mandatory when you want to print the total cinetic energy into the non-mobile Galilean referential R (see `Ec_dans_repere_fixe` keyword).
- **acceleration** *champ\_base* (20.1): Keyword for the acceleration of the referential R' into the R referential ( $d^2\mathbf{OO'}/dt^2$  term [m.s-2]). *field\_base* is a time dependant field (eg: `Champ_Fonc_t`).
- **omega** *champ\_base* (20.1): Keyword for a rotation of the referential R' into the R referential [rad.s-1]. *field\_base* is a 3D time dependant field specified for example by a `Champ_Fonc_t` keyword. The *time\_field* field should have 3 components even in 2D (In 2D: 0 0 omega).
- **domegadt** *champ\_base* (20.1): Keyword to define the time derivative of the previous rotation [rad.s-2]. Should be zero if the rotation is constant. The *time\_field* field should have 3 components even in 2D (In 2D: 0 0 domegadt).
- **centre\_rotation** *champ\_base* (20.1): Keyword to specify the centre of rotation (expressed in R' coordinates) of R' into R (if the domain rotates with the R' referential, the centre of rotation is  $0'=(0,0,0)$ ). The *time\_field* should have 2 or 3 components according the dimension 2 or 3.
- **option** *str* into ['terme\_complet', 'coriolis\_seul', 'entrainement\_seul']: Keyword to specify the kind of calculation: `terme_complet` (default option) will calculate both the Coriolis and centrifugal forces, `coriolis_seul` will calculate the first one only, `entrainement_seul` will calculate the second one only.

## 41.28 Boussinesq\_concentration

Description: Class to describe a source term that couples the movement quantity equation and constituent transport equation with the Boussinesq hypothesis.

See also: `source_base` (41)

Usage:

**boussinesq\_concentration** *str*

**Read** *str* {

```
c0 n x1 x2 ... xn
```

}

where

- **c0** *n x1 x2 ... xn*: Reference concentration field type. The only field type currently available is Champ\_Uniforme (Uniform field).

## 41.29 Boussinesq\_temperature

Description: Class to describe a source term that couples the movement quantity equation and energy equation with the Boussinesq hypothesis.

See also: [source\\_base \(41\)](#)

Usage:

**boussinesq\_temperature** *str*

```
Read str {
 t0 str
 [verif_boussinesq int]
```

```
}
```

where

- **t0** *str*: Reference temperature value (oC or K). It can also be a time dependant function since the 1.6.6 version.
- **verif\_boussinesq** *int*: Keyword to check (1) or not (0) the reference value in comparison with the mean value in the domain. It is set to 1 by default.

## 41.30 Canal\_perio

Description: Momentum source term to maintain flow rate. The expression of the source term is:

$$S(t) = (2*(Q(0) - Q(t)) - (Q(0) - Q(t-dt)))/(coeff*dt*area)$$

Where:

coeff=damping coefficient

area=area of the periodic boundary

Q(t)=flow rate at time t

dt=time step

Three files will be created during calculation on a datafile named DataFile.data. The first file contains the flow rate evolution. The second file is useful for resuming a calculation with the flow rate of the previous stopped calculation, and the last one contains the pressure gradient evolution:

-DataFile\_Channel\_Flow\_Rate\_ProblemName\_BoundaryName

-DataFile\_Channel\_Flow\_Rate\_repr\_ProblemName\_BoundaryName

-DataFile\_Pressure\_Gradient\_ProblemName\_BoundaryName

See also: [source\\_base \(41\)](#)

Usage:

**canal\_perio** *str*

```
Read str {
 [u_etoile float]
 [coeff float]
 [h float]
 bord str
 [debit_impose float]
```

}  
where

- **u\_etoile** *float*
- **coeff** *float*: Damping coefficient (optional, default value is 10).
- **h** *float*: Half height of the channel.
- **bord** *str*: The name of the (periodic) boundary normal to the flow direction.
- **debit\_impose** *float*: Optional option to specify the aimed flow rate  $Q(0)$ . If not used,  $Q(0)$  is computed by the code after the projection phase, where velocity initial conditions are slightly changed to verify incompressibility.

### 41.31 Coriolis

Description: Keyword for a Coriolis term in hydraulic equation. Warning: Only available in VDF.

See also: [source\\_base \(41\)](#)

Usage:

**coriolis** *str*

**Read** *str* {

**omega** *n x1 x2 ... xn*

}

where

- **omega** *n x1 x2 ... xn*: Value of omega.

### 41.32 Darcy

Description: Class for calculation in a porous media with source term of Darcy  $-\nu/K \cdot V$ . This keyword must be used with a permeability model. For the moment there are two models : permeability constant or Ergun's law. Darcy source term is available for quasi compressible calculation. A new keyword is added for porosity (porosite).

See also: [source\\_base \(41\)](#)

Usage:

**darcy bloc**

where

- **bloc** *bloc\_lecture (3.2)*: Description.

### 41.33 Dirac

Description: Class to define a source term corresponding to a volume power release in the energy equation.

See also: [source\\_base \(41\)](#)

Usage:

**dirac position ch**

where

- **position**  $n \times 1 \times 2 \dots \times n$
- **ch** *champ\_base* (20.1): Thermal power field type. To impose a volume power on a domain sub-area, the *Champ\_Uniforme\_Morceaux* (*partly\_uniform\_field*) type must be used.  
Warning : The volume thermal power is expressed in W.m-3.

### 41.34 Flux\_interfacial

Description: Source term of mass transfer between phases connected by the saturation object defined in *saturation\_xxxx*

See also: *source\_base* (41)

Usage:

**flux\_interfacial**

### 41.35 Forchheimer

Description: Class to add the source term of Forchheimer  $-C_f/\sqrt{K} \cdot V^2$  in the Navier-Stokes equations. We must precise a permeability model : constant or Ergun's law. Moreover we can give the constant  $C_f$  : by default its value is 1. Forchheimer source term is available also for quasi compressible calculation. A new keyword is added for porosity (*porosite*).

See also: *source\_base* (41)

Usage:

**forchheimer bloc**

where

- **bloc** *bloc\_lecture* (3.2): Description.

### 41.36 Frottement\_interfacial

Description: Source term which corresponds to the phases friction at the interface

See also: *source\_base* (41)

Usage:

**frottement\_interfacial str**

**Read** *str* {

    [ **a\_res** *float*]  
    [ **dv\_min** *float*]  
    [ **exp\_res** *int*]

}

where

- **a\_res** *float*: void fraction at which the gas velocity is forced to approach liquid velocity (default  $\alpha_{\text{evanescence}} \cdot 100$ )
- **dv\_min** *float*: minimal relative velocity used to linearize interfacial friction at low velocities
- **exp\_res** *int*: exponent that callibrates intensity of velocity convergence (default 2)

### 41.37 Perte\_charge\_anisotrope

Description: Anisotropic pressure loss.

See also: [source\\_base \(41\)](#)

Usage:

**per***te\_charge\_anisotrope str*

**Read** *str* {

**lambda** *str*  
**lambda\_ortho** *str*  
**diam\_hydr** *champ\_don\_base*  
**direction** *champ\_don\_base*  
[ **sous\_zone** *str*]

}

where

- **lambda** *str*: Function for loss coefficient which may be Reynolds dependant (Ex: 64/Re).
- **lambda\_ortho** *str*: Function for loss coefficient in transverse direction which may be Reynolds dependant (Ex: 64/Re).
- **diam\_hydr** *champ\_don\_base* (20.9): Hydraulic diameter value.
- **direction** *champ\_don\_base* (20.9): Field which indicates the direction of the pressure loss.
- **sous\_zone** *str*: Optional sub-area where pressure loss applies.

### 41.38 Perte\_charge\_circulaire

Description: New pressure loss.

See also: [source\\_base \(41\)](#)

Usage:

**per***te\_charge\_circulaire str*

**Read** *str* {

**lambda** *str*  
**diam\_hydr** *champ\_don\_base*  
[ **sous\_zone** *str*]  
**lambda\_ortho** *str*  
**diam\_hydr\_ortho** *champ\_don\_base*  
**direction** *champ\_don\_base*

}

where

- **lambda** *str*: Function  $f(\text{Re}_{\text{tot}}, \text{Re}_{\text{long}}, t, x, y, z)$  for loss coefficient in the longitudinal direction
- **diam\_hydr** *champ\_don\_base* (20.9): Hydraulic diameter value.
- **sous\_zone** *str*: Optional sub-area where pressure loss applies.
- **lambda\_ortho** *str*: function: Function  $f(\text{Re}_{\text{tot}}, \text{Re}_{\text{ortho}}, t, x, y, z)$  for loss coefficient in transverse direction
- **diam\_hydr\_ortho** *champ\_don\_base* (20.9): Transverse hydraulic diameter value.
- **direction** *champ\_don\_base* (20.9): Field which indicates the direction of the pressure loss.

### 41.39 Perte\_charge\_directionnelle

Description: Directional pressure loss (available in VEF and PolyMAC).

See also: [source\\_base \(41\)](#)

Usage:

**perte\_charge\_directionnelle** *str*

```
Read str {
 lambda str
 diam_hydr champ_don_base
 direction champ_don_base
 [sous_zone str]
}
```

where

- **lambda** *str*: Function for loss coefficient which may be Reynolds dependant (Ex: 64/Re).
- **diam\_hydr** *champ\_don\_base* (20.9): Hydraulic diameter value.
- **direction** *champ\_don\_base* (20.9): Field which indicates the direction of the pressure loss.
- **sous\_zone** *str*: Optional sub-area where pressure loss applies.

### 41.40 Perte\_charge\_isotrope

Description: Isotropic pressure loss (available in VEF and PolyMAC).

See also: [source\\_base \(41\)](#)

Usage:

**perte\_charge\_isotrope** *str*

```
Read str {
 lambda str
 diam_hydr champ_don_base
 [sous_zone str]
}
```

where

- **lambda** *str*: Function for loss coefficient which may be Reynolds dependant (Ex: 64/Re).
- **diam\_hydr** *champ\_don\_base* (20.9): Hydraulic diameter value.
- **sous\_zone** *str*: Optional sub-area where pressure loss applies.

### 41.41 Perte\_charge\_reguliere

Description: Source term modelling the presence of a bundle of tubes in a flow.

See also: [source\\_base \(41\)](#)

Usage:

**perte\_charge\_reguliere** **spec** **zone\_name**

where

- **spec** *spec\_pdc\_r\_base* (41.42): Description of longitudinale or transversale type.
- **zone\_name** *str*: Name of the sub-area occupied by the tube bundle. A Sous\_Zone (Sub-area) type object called *zone\_name* should have been previously created.

## 41.42 Spec\_pdc\_r\_base

Description: Class to read the source term modelling the presence of a bundle of tubes in a flow.  $C_f = A \text{Re}^{-B}$ .

See also: `objet_lecture` (47) longitudinale (41.42.1) transversale (41.42.2)

Usage:

**spec\_pdc\_r\_base**

### 41.42.1 Longitudinale

Description: Class to define the pressure loss in the direction of the tube bundle.

See also: `spec_pdc_r_base` (41.42)

Usage:

**longitudinale dir dd ch\_a a [ ch\_b ] [ b ]**

where

- **dir** *str into* ['x', 'y', 'z']: Direction.
- **dd** *float*: Tube bundle hydraulic diameter value. This value is expressed in m.
- **ch\_a** *str into* ['a', 'cf']: Keyword to be used to set law coefficient values for the coefficient of regular pressure losses.
- **a** *float*: Value of a law coefficient for regular pressure losses.
- **ch\_b** *str into* ['b']: Keyword to be used to set law coefficient values for regular pressure losses.
- **b** *float*: Value of a law coefficient for regular pressure losses.

### 41.42.2 Transversale

Description: Class to define the pressure loss in the direction perpendicular to the tube bundle.

See also: `spec_pdc_r_base` (41.42)

Usage:

**transversale dir dd chaine\_d d ch\_a a [ ch\_b ] [ b ]**

where

- **dir** *str into* ['x', 'y', 'z']: Direction.
- **dd** *float*: Value of the tube bundle step.
- **chaine\_d** *str into* ['d']: Keyword to be used to set the value of the tube external diameter.
- **d** *float*: Value of the tube external diameter.
- **ch\_a** *str into* ['a', 'cf']: Keyword to be used to set law coefficient values for the coefficient of regular pressure losses.
- **a** *float*: Value of a law coefficient for regular pressure losses.
- **ch\_b** *str into* ['b']: Keyword to be used to set law coefficient values for regular pressure losses.
- **b** *float*: Value of a law coefficient for regular pressure losses.

## 41.43 Perte\_charge\_singuliere

Description: Source term that is used to model a pressure loss over a surface area (transition through a grid, sudden enlargement) defined by the faces of elements located on the intersection of a subzone named `subzone_name` and a X,Y, or Z plane located at X,Y or Z = location.

See also: `source_base` (41)

Usage:

**perte\_charge\_singuliere** *str*

**Read** *str* {

**dir** *str* into ['kx', 'ky', 'kz', 'K']  
    [ **coeff** *float*]  
    [ **regul** *bloc\_lecture*]  
    **surface** *bloc\_lecture*

}

where

- **dir** *str* into ['kx', 'ky', 'kz', 'K']: KX, KY or KZ designate directional pressure loss coefficients for respectively X, Y or Z direction. Or in the case where you chose a target flow rate with regul. Use K for isotropic pressure loss coefficient
- **coeff** *float*: Value (float) of friction coefficient (KX, KY, KZ).
- **regul** *bloc\_lecture* (3.2): option to have adjustable K with flowrate target  
{ K0 valeur\_initiale\_de\_k deb debit\_cible eps intervalle\_variation\_mutiplicatif }.
- **surface** *bloc\_lecture* (3.2): Three syntaxes are possible for the surface definition block:  
For VDF and VEF: { X|Y|Z = location subzone\_name }  
Only for VEF: { Surface surface\_name }.  
For polymac { Surface surface\_name Orientation champ\_uniforme }

## 41.44 Puissance\_thermique

Description: Class to define a source term corresponding to a volume power release in the energy equation.

See also: `source_base` (41)

Usage:

**puissance\_thermique** *ch*

where

- **ch** *champ\_base* (20.1): Thermal power field type. To impose a volume power on a domain sub-area, the Champ\_Uniforme\_Morceaux (partly\_uniform\_field) type must be used.  
Warning : The volume thermal power is expressed in W.m-3 in 3D (in W.m-2 in 2D). It is a power per volume unit (in a porous media, it is a power per fluid volume unit).

## 41.45 Radioactive\_decay

Description: Radioactive decay source term of the form  $-\lambda_i c_i$ , where  $0 \leq i \leq N$ , N is the number of component of the constituent,  $c_i$  and  $\lambda_i$  are the concentration and the decay constant of the i-th component of the constituent.

See also: `source_base` (41)

Usage:

**radioactive\_decay** *val*

where

- **val** *n x1 x2 ... xn*: n is the number of decay constants to read (int), and val1, val2... are the decay constants (double)



## 41.46 Source\_con\_phase\_field

Description: Keyword to define the source term of the Cahn-Hilliard equation.

See also: `source_base` (41)

Usage:

**source\_con\_phase\_field** *str*

**Read** *str* {

```
[systeme_naire systeme_naire_deriv
temps_d_affichage int
moyenne_de_kappa str
multiplicateur_de_kappa float
couplage_NS_CH str
implication_CH str into ['oui', 'non']
gmres_non_lineaire str into ['oui', 'non']
seuil_cv_iterations_ptfixe float
seuil_residu_ptfixe float
seuil_residu_gmresnl float
dimension_espace_de_krylov int
nb_iterations_gmresnl int
residu_min_gmresnl float
residu_max_gmresnl float
```

}

where

- **systeme\_naire** *systeme\_naire\_deriv* (41.47)
- **temps\_d\_affichage** *int*: Time during the characteristics of the problem are shown before calculation.
- **moyenne\_de\_kappa** *str*: To define how mobility kappa is calculated on faces of the mesh according to cell-centered values (chaîne is arithmetique/harmonique/geometrique).
- **multiplicateur\_de\_kappa** *float*: To define the parameter of the mobility expression when mobility depends on C.
- **couplage\_NS\_CH** *str*: Evaluating time choosen for the term source calculation into the Navier Stokes equation (chaîne is mutilde(n+1/2)/mutilde(n), in order to be conservative, the first choice seems better).
- **implication\_CH** *str* into ['oui', 'non']: To define if the Cahn-Hilliard will be solved using a implicit algorithm or not.
- **gmres\_non\_lineaire** *str* into ['oui', 'non']: To define the algorithm to solve Cahn-Hilliard equation (oui: Newton-Krylov method, non: fixed point method).
- **seuil\_cv\_iterations\_ptfixe** *float*: Convergence threshold (an option of the fixed point method).
- **seuil\_residu\_ptfixe** *float*: Threshold for the matrix inversion used in the method (an option of the fixed point method).
- **seuil\_residu\_gmresnl** *float*: Convergence threshold (an option of the Newton-Krylov method).
- **dimension\_espace\_de\_krylov** *int*: Vector numbers used in the method (an option of the Newton-Krylov method).
- **nb\_iterations\_gmresnl** *int*: Maximal iteration (an option of the Newton-Krylov method).
- **residu\_min\_gmresnl** *float*: Minimal convergence threshold (an option of the Newton-Krylov method).
- **residu\_max\_gmresnl** *float*: Maximal convergence threshold (an option of the Newton-Krylov method).

## 41.47 Systeme\_naire\_deriv

Description: not\_set

See also: objet\_lecture (47) non (41.47.1)

Usage:

### 41.47.1 Non

Description: not\_set

See also: systeme\_naire\_deriv (41.47)

Usage:

```
non {
 alpha float
 beta float
 kappa float
 kappa_variable bloc_kappa_variable
 [potentiel_chimique bloc_potentiel_chim]
}
where
```

- **alpha** float: Internal capillary coefficient alfa.
- **beta** float: Parameter beta of the model.
- **kappa** float: Mobility coefficient kappa0.
- **kappa\_variable** bloc\_kappa\_variable (41.47.2): To define a mobility which depends on concentration C.
- **potentiel\_chimique** bloc\_potentiel\_chim (41.47.3): chemical potential function

### 41.47.2 Bloc\_kappa\_variable

Description: if the parameter of the mobility, kappa, depends on C

See also: objet\_lecture (47)

Usage:

```
expr
where
```

- **expr** bloc\_lecture (3.2): choice for kappa\_variable

### 41.47.3 Bloc\_potentiel\_chim

Description: if the chemical potential function is an univariate function

See also: objet\_lecture (47)

Usage:

```
expr
where
```

- **expr** bloc\_lecture (3.2): choice for potentiel\_chimique

#### 41.48 Source\_constituant

Description: Keyword to specify source rates, in  $[[C]/s]$ , for each one of the nb constituents. [C] is the concentration unit.

See also: `source_base` (41)

Usage:

**source\_constituant** *ch*

where

- **ch** *champ\_base* (20.1): Field type.

#### 41.49 Flottabilite

Description: buoyancy effect

See also: `source_base` (41)

Usage:

**flottabilite**

#### 41.50 Source\_generique

Description: to define a source term depending on some discrete fields of the problem and (or) analytic expression. It is expressed by the way of a generic field usually used for post-processing.

See also: `source_base` (41)

Usage:

**source\_generique** *champ*

where

- **champ** *champ\_generique\_base* (12): the source field

#### 41.51 Masse\_ajoutee

Description: weight added effect

See also: `source_base` (41)

Usage:

**masse\_ajoutee**

#### 41.52 Source\_pdf

Description: Source term for Penalised Direct Forcing (PDF) method.

See also: `source_pdf_base` (41.54)

Usage:

**source\_pdf** *str*

**Read** *str* {

```

aire champ_base
rotation champ_base
[transpose_rotation]
modele bloc_pdf_model
[interpolation interpolation_ibm_base]
}
where

```

- **aire** *champ\_base* (20.1) for inheritance: volumic field: a boolean for the cell (0 or 1) indicating if the obstacle is in the cell
- **rotation** *champ\_base* (20.1) for inheritance: volumic field with 9 components representing the change of basis on cells (local to global). Used for rotating cases for example.
- **transpose\_rotation** for inheritance: whether to transpose the basis change matrix.
- **modele** *bloc\_pdf\_model* (41.53) for inheritance: model used for the Penalized Direct Forcing
- **interpolation** *interpolation\_ibm\_base* (22) for inheritance: interpolation method

### 41.53 Bloc\_pdf\_model

Description: not\_set

See also: objet\_lecture (47)

Usage:

```

{
 eta float
 [bilan_pdf int]
 [temps_relaxation_coefficient_pdf float]
 [echelle_relaxation_coefficient_pdf float]
 [local]
 [vitesse_imposee_data champ_base]
 [vitesse_imposee_fonction n word1 word2 ... wordn]
 [variable_imposee_data champ_base]
 [variable_imposee_fonction n word1 word2 ... wordn]
}
where

```

- **eta** *float*: penalization coefficient
- **bilan\_pdf** *int*: type de bilan du terme PDF (seul/avec temps/avec convection)
- **temps\_relaxation\_coefficient\_pdf** *float*: time relaxation on the forcing term to help
- **echelle\_relaxation\_coefficient\_pdf** *float*: time relaxation on the forcing term to help convergence
- **local** : whether the prescribed velocity is expressed in the global or local basis
- **vitesse\_imposee\_data** *champ\_base* (20.1): Prescribed velocity as a field
- **vitesse\_imposee\_fonction** *n word1 word2 ... wordn*: Prescribed velocity as a set of analytical component
- **variable\_imposee\_data** *champ\_base* (20.1): Prescribed variable as a field
- **variable\_imposee\_fonction** *n word1 word2 ... wordn*: Prescribed variable as a set of analytical component

### 41.54 Source\_pdf\_base

Description: Basic class of source\_PDF terms introduced in the equation.

See also: `Source_dep_inco_bases` (41.25) `source_pdf` (41.52)

Usage:

**source\_pdf\_base** *str*

**Read** *str* {

**aire** *champ\_base*  
**rotation** *champ\_base*  
[ **transpose\_rotation** ]  
**modele** *bloc\_pdf\_model*  
[ **interpolation** *interpolation\_ibm\_base*]

}

where

- **aire** *champ\_base* (20.1): volumic field: a boolean for the cell (0 or 1) indicating if the obstacle is in the cell
- **rotation** *champ\_base* (20.1): volumic field with 9 components representing the change of basis on cells (local to global). Used for rotating cases for example.
- **transpose\_rotation** : whether to transpose the basis change matrix.
- **modele** *bloc\_pdf\_model* (41.53): model used for the Penalized Direct Forcing
- **interpolation** *interpolation\_ibm\_base* (22): interpolation method

## 41.55 Source\_qdm

Description: Momentum source term in the Navier-Stokes equations.

See also: `source_base` (41)

Usage:

**source\_qdm** **ch**

where

- **ch** *champ\_base* (20.1): Field type.

## 41.56 Source\_qdm\_lambdaup

Description: This source term is a dissipative term which is intended to minimise the energy associated to non-conformscales  $u'$  (responsible for spurious oscillations in some cases). The equation for these scales can be seen as:  $du'/dt = -\lambda u' + \text{grad } P'$  where  $-\lambda u'$  represents the dissipative term, with  $\lambda = a/\Delta t$ . For Crank-Nicholson temporal scheme, recommended value for  $a$  is 2.

Remark : This method requires to define a filtering operator.

See also: `source_base` (41)

Usage:

**source\_qdm\_lambdaup** *str*

**Read** *str* {

**lambda** *float*  
[ **lambda\_min** *float* ]  
[ **lambda\_max** *float* ]  
[ **ubar\_umprim\_cible** *float* ]

}  
where

- **lambda** *float*: value of lambda
- **lambda\_min** *float*: value of lambda\_min
- **lambda\_max** *float*: value of lambda\_max
- **ubar\_umprim\_cible** *float*: value of ubar\_umprim\_cible

#### 41.57 Source\_qdm\_phase\_field

Description: Keyword to define the capillary force into the Navier Stokes equation for the Phase Field problem.

See also: [source\\_base \(41\)](#)

Usage:

**source\_qdm\_phase\_field** *str*

**Read** *str* {

**forme\_du\_terme\_source** *int*

}  
where

- **forme\_du\_terme\_source** *int*: Kind of the source term (1, 2, 3 or 4).

#### 41.58 Source\_rayo\_semi\_transp

Description: Radiative term source in energy equation.

See also: [source\\_base \(41\)](#)

Usage:

**source\_rayo\_semi\_transp**

#### 41.59 Source\_robin

Description: This source term should be used when a Paroi\_decalee\_Robin boundary condition is set in a hydraulic equation. The source term will be applied on the N specified boundaries. To post-process the values of tauw, u\_tau and Reynolds\_tau into the files tauw\_robin.dat, reynolds\_tau\_robin.dat and u\_tau\_robin.dat, you must add a block Traitement\_particulier { canal { } }

See also: [source\\_base \(41\)](#)

Usage:

**source\_robin** **bords**

where

- **bords** *vect\_nom* ([3.149](#))

## 41.60 Source\_robin\_scalaire

Description: This source term should be used when a `Paroi_decalee_Robin` boundary condition is set in an energy equation. The source term will be applied on the `N` specified boundaries. The values `temp_wall_valueI` are the temperature specified on the `I`th boundary. The last value `dt_impr` is a printing period which is mandatory to specify in the data file but has no effect yet.

See also: `source_base` ([41](#))

Usage:

**source\_robin\_scalaire bords**

where

- **bords** *listdeuxmots\_sacc* ([41.61](#))

## 41.61 Listdeuxmots\_sacc

Description: List of groups of two words (without curly brackets).

See also: `listobj` ([46.5](#))

Usage:

`n object1 object2 ....`

list of *deuxmots* ([4.9.1](#))

## 41.62 Source\_th\_tdivu

Description: This term source is dedicated for any scalar (called `T`) transport. Coupled with upwind (amont) or muscl scheme, this term gives for final expression of convection :  $\text{div}(\mathbf{U}.T) - T.\text{div}(\mathbf{U}) = \mathbf{U}.\text{grad}(T)$  This ensures, in incompressible flow when divergence free is badly resolved, to stay in a better way in the physical boundaries.

Warning: Only available in VEF discretization.

See also: `source_base` ([41](#))

Usage:

**source\_th\_tdivu**

## 41.63 Trainee

Description: drag effect

See also: `source_base` ([41](#))

Usage:

**trainee**

## 41.64 Source\_transport\_eps

Description: Keyword to alter the source term constants for `eps` in the bicephale `k-eps` model epsilon transport equation. By default, these constants are set to: `C1_eps=1.44` `C2_eps=1.92`

See also: `source_base` ([41](#))

Usage:

**source\_transport\_eps** *str*

**Read** *str* {

    [ **c1\_eps** *float*]

    [ **c2\_eps** *float*]

}

where

- **c1\_eps** *float*: First constant.
- **c2\_eps** *float*: Second constant.

## 41.65 Source\_transport\_k

Description: Keyword to alter the source term constants for k in the bicephale k-eps model epsilon transport equation.

See also: [source\\_base \(41\)](#)

Usage:

## 41.66 Source\_transport\_k\_eps

Description: Keyword to alter the source term constants in the standard k-eps model epsilon transport equation. By default, these constants are set to: C1\_eps=1.44 C2\_eps=1.92

See also: [source\\_base \(41\)](#) [Source\\_Transport\\_K\\_Eps\\_anisotherme \(41.24\)](#) [source\\_transport\\_k\\_eps\\_aniso\\_concen \(41.67\)](#) [source\\_transport\\_k\\_eps\\_aniso\\_therm\\_concen \(41.68\)](#)

Usage:

**source\_transport\_k\_eps** *str*

**Read** *str* {

    [ **c1\_eps** *float*]

    [ **c2\_eps** *float*]

}

where

- **c1\_eps** *float*: First constant.
- **c2\_eps** *float*: Second constant.

## 41.67 Source\_transport\_k\_eps\_aniso\_concen

Description: Keywords to modify the source term constants in the anisotherm standard k-eps model epsilon transport equation. By default, these constants are set to: C1\_eps=1.44 C2\_eps=1.92 C3\_eps=1.0

See also: [source\\_transport\\_k\\_eps \(41.66\)](#)

Usage:

**source\_transport\_k\_eps\_aniso\_concen** *str*

**Read** *str* {

    [ **c3\_eps** *float*]

    [ **c1\_eps** *float*]



```

 [c2_eps float]
}
where

```

- **c3\_eps** *float*: Third constant.
- **c1\_eps** *float* for inheritance: First constant.
- **c2\_eps** *float* for inheritance: Second constant.

#### 41.68 Source\_transport\_k\_eps\_aniso\_therm\_concen

Description: Keywords to modify the source term constants in the anisotherm standard k-eps model epsilon transport equation. By default, these constants are set to: C1\_eps=1.44 C2\_eps=1.92 C3\_eps=1.0

See also: source\_transport\_k\_eps ([41.66](#))

Usage:

**source\_transport\_k\_eps\_aniso\_therm\_concen** *str*

**Read** *str* {

```

 [c3_eps float]
 [c1_eps float]
 [c2_eps float]

```

```

}
where

```

- **c3\_eps** *float*: Third constant.
- **c1\_eps** *float* for inheritance: First constant.
- **c2\_eps** *float* for inheritance: Second constant.

#### 41.69 Source\_transport\_k\_eps\_realisable

Description: Keyword to alter the source term constants in the standard k-eps model epsilon transport equation. By default, these constants are set to: C1\_eps=1.44 C2\_eps=1.92

See also: source\_base ([41](#))

Usage:

**source\_transport\_k\_eps\_realisable** *str*

**Read** *str* {

```

 [c2_eps float]

```

```

}
where

```

- **c2\_eps** *float*: Second constant.

### 41.70 Tenseur\_reynolds\_externe

Description: Use a neural network to estimate the values of the Reynolds tensor. The structure of the neural networks is stored in a file located in the share/reseaux\_neurones directory.

See also: [source\\_base \(41\)](#)

Usage:

**tenseur\_Reynolds\_externe** *str*

**Read** *str* {

**nom\_fichier** *str*

}

where

- **nom\_fichier** *str*: The base name of the file.

### 41.71 Terme\_puissance\_thermique\_echange\_impose

Description: Source term to impose thermal power according to formula :  $P = h_{imp} * (T - T_{ext})$ . Where T is the Trust temperature,  $T_{ext}$  is the outside temperature with which energy is exchanged via an exchange coefficient  $h_{imp}$

See also: [source\\_base \(41\)](#)

Usage:

**terme\_puissance\_thermique\_echange\_impose** *str*

**Read** *str* {

**himp** *champ\_base*

**Text** *champ\_base*

    [ **PID\_controler\_on\_targer\_power** *bloc\_lecture* ]

}

where

- **himp** *champ\_base* ([20.1](#)): the exchange coefficient
- **Text** *champ\_base* ([20.1](#)): the outside temperature
- **PID\_controler\_on\_targer\_power** *bloc\_lecture* ([3.2](#)): PID\_controler\_on\_targer\_power bloc with parameters target\_power (required), Kp, Ki and Kd (at least one of them should be provided)

### 41.72 Travail\_pression

Description: Source term which corresponds to the additional pressure work term that appears when dealing with compressible multiphase fluids

See also: [source\\_base \(41\)](#)

Usage:

**travail\_pression**

### 41.73 Vitesse\_derive\_base

Description: Source term which corresponds to the drift-velocity between a liquid and a gas phase

See also: `vitesse_relative_base` ([41.74](#))

Usage:

**vitesse\_derive\_base**

### 41.74 Vitesse\_relative\_base

Description: Basic class for drift-velocity source term between a liquid and a gas phase

See also: `source_base` ([41](#)) `vitesse_derive_base` ([41.73](#))

Usage:

**vitesse\_relative\_base**

## 42 sous\_zone

Synonymous: **sous\_domaine**

Description: It is an object type describing a domain sub-set.

A `Sous_Zone` (Sub-area) type object must be associated with a `Domaine` type object. The `Read` (`Lire`) interpreter is used to define the items comprising the sub-area.

Caution: The `Domain` type object `nom_domaine` must have been meshed (and triangulated or tetrahedralised in VEF) prior to carrying out the `Associate` (`Associer`) `nom_sous_zone nom_domaine` instruction; this instruction must always be preceded by the `read` instruction.

See also: `objet_u` ([48](#))

Usage:

**sous\_zone** *str*

**Read** *str* {

```
[restriction str]
[rectangle bloc_origine_cotes]
[segment bloc_origine_cotes]
[boite bloc_origine_cotes]
[liste n n1 n2 ... nn]
[fichier str]
[intervalle deuxentiers]
[polynomes bloc_lecture]
[couronne bloc_couronne]
[tube bloc_tube]
[fonction_sous_zone str]
[union str]
```

}

where

- **restriction** *str*: The elements of the sub-area `nom_sous_zone` must be included into the other sub-area named `nom_sous_zone2`. This keyword should be used first in the `Read` keyword.
- **rectangle** *bloc\_origine\_cotes* ([42.1](#)): The sub-area will include all the domain elements whose centre of gravity is within the Rectangle (in dimension 2).

- **segment** *bloc\_origine\_cotes* (42.1)
- **boite** *bloc\_origine\_cotes* (42.1): The sub-area will include all the domain elements whose centre of gravity is within the Box (in dimension 3).
- **liste** *n n1 n2 ... nn*: The sub-area will include n domain items, numbers No. 1 No. i No. n.
- **fichier** *str*: The sub-area is read into the file filename.
- **intervalle** *deuxentiers* (5.22.8): The sub-area will include domain items whose number is between n1 and n2 (where  $n1 \leq n2$ ).
- **polynomes** *bloc\_lecture* (3.2): A REPENDRE
- **couronne** *bloc\_couronne* (42.2): In 2D case, to create a couronne.
- **tube** *bloc\_tube* (42.3): In 3D case, to create a tube.
- **fonction\_sous\_zone** *str*: Keyword to build a sub-area with the the elements included into the area defined by fonction>0.
- **union** *str*: The elements of the sub-area nom\_sous\_zone3 will be added to the sub-area nom\_sous\_zone. This keyword should be used last in the Read keyword.

## 42.1 Bloc\_origine\_cotes

Description: Class to create a rectangle (or a box).

See also: *objet\_lecture* (47)

Usage:

**name origin name2 cotes**

where

- **name** *str into ['Origine']*: Keyword to define the origin of the rectangle (or the box).
- **origin** *x1 x2 (x3)*: Coordinates of the origin of the rectangle (or the box).
- **name2** *str into ['Cotes']*: Keyword to define the length along the axes.
- **cotes** *x1 x2 (x3)*: Length along the axes.

## 42.2 Bloc\_couronne

Description: Class to create a couronne (2D).

See also: *objet\_lecture* (47)

Usage:

**name origin name3 ri name4 re**

where

- **name** *str into ['Origine']*: Keyword to define the center of the circle.
- **origin** *x1 x2 (x3)*: Center of the circle.
- **name3** *str into ['ri']*: Keyword to define the interior radius.
- **ri** *float*: Interior radius.
- **name4** *str into ['re']*: Keyword to define the exterior radius.
- **re** *float*: Exterior radius.

## 42.3 Bloc\_tube

Description: Class to create a tube (3D).

See also: *objet\_lecture* (47)

Usage:

**name origin name2 direction name3 ri name4 re name5 h**

where

- **name** *str* into [*'Origine'*]: Keyword to define the center of the tube.
- **origin** *x1 x2 (x3)*: Center of the tube.
- **name2** *str* into [*'dir'*]: Keyword to define the direction of the main axis.
- **direction** *str* into [*'X', 'Y', 'Z'*]: direction of the main axis X, Y or Z
- **name3** *str* into [*'ri'*]: Keyword to define the interior radius.
- **ri** *float*: Interior radius.
- **name4** *str* into [*'re'*]: Keyword to define the exterior radius.
- **re** *float*: Exterior radius.
- **name5** *str* into [*'hauteur'*]: Keyword to define the heigth of the tube.
- **h** *float*: Heigth of the tube.

## 43 transport\_equation\_deriv

Description: not\_set

See also: objet\_u (48) transport\_k\_epsilon (43.2) transport\_k\_omega (43.3) Transport\_K\_Epsilon\_Realisable (43.1)

Usage:

**transport\_equation\_deriv** *str*

**Read** *str* {

```
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limite condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
```

}

where

- **disable\_equation\_residual** *int*: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2): Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3): Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limite condlims** (4.39.1): Boundary conditions.
- **initial\_conditions|conditions\_initiales condinits** (5.4): Initial conditions.
- **sources** *sources* (5.5): To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur ecrire\_fichier\_xyz\_valeur** (3.57): This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation parametre\_equation\_base** (5.6): Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str*: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time

t0 and t1.  
 Navier\_Sokes\_Standard  
 { equation\_non\_resolue (t>t0)\*(t<t1) }  
 • **renommer\_equation** *str*: Rename the equation with a specific name.

### 43.1 Transport\_k\_epsilon\_realisable

Description: Realizable K-Epsilon Turbulence Model Transport Equations for K and Epsilon.

See also: transport\_equation\_deriv (43)

Usage:

**Transport\_K\_Epsilon\_Realisable** *str*  
**Read** *str* {

[ **disable\_equation\_residual** *int*]  
 [ **convection** *bloc\_convection*]  
 [ **diffusion** *bloc\_diffusion*]  
 [ **boundary\_conditions|conditions\_limites** *condlims*]  
 [ **initial\_conditions|conditions\_initiales** *condinits*]  
 [ **sources** *sources*]  
 [ **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur*]  
 [ **parametre\_equation** *parametre\_equation\_base*]  
 [ **equation\_non\_resolue** *str*]  
 [ **renommer\_equation** *str*]

}

where

- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1.  
 Navier\_Sokes\_Standard  
 { equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.

### 43.2 Transport\_k\_epsilon

Description: The (k-eps) transport equation. To resume from a previous mixing length calculation, an external MED-format file containing reconstructed K and Epsilon quantities can be read (see fichier\_ecriture\_k\_eps) thanks to the Champ\_fonc\_MED keyword.

Warning, When used with the Quasi-compressible model, k and eps should be viewed as rho k and rho epsilon when defining initial and boundary conditions or when visualizing values for k and eps. This bug will be fixed in a future version.

See also: `transport_equation_deriv` (43)

Usage:

**transport\_k\_epsilon** *str*

**Read** *str* {

```
[disable_equation_residual int]
[convection bloc_convection]
[diffusion bloc_diffusion]
[boundary_conditions|conditions_limites condlims]
[initial_conditions|conditions_initiales condinits]
[sources sources]
[ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
[parametre_equation parametre_equation_base]
[equation_non_resolue str]
[renommer_equation str]
[with_nu str into ['yes', 'no']]
[exit_on_negative_k_eps]
[exit_on_big_eps]
[disable_VEF_mean_value_corrections]
```

}

where

- **disable\_equation\_residual** *int*: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2): Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3): Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1): Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4): Initial conditions.
- **sources** *sources* (5.5): To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57): This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6): Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str*: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Exemple: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str*: Rename the equation with a specific name.
- **with\_nu** *str* into ['yes', 'no']: yes/no
- **exit\_on\_negative\_k\_eps** : Flag to exit (with postprocessing of fields) if a negative value is found for k or epsilon
- **exit\_on\_big\_eps** : Flag to exit (with postprocessing of fields) if an excessively big values of epsilon is found
- **disable\_VEF\_mean\_value\_corrections**

### 43.3 Transport\_k\_omega

Description: The (k-omega) transport equation.

See also: `transport_equation_deriv` (43)

Usage:

**transport\_k\_omega** *str*

```
Read str {
 [exit_on_negative_k_omega]
 [exit_on_big_omega]
 [with_nu str into ['yes', 'no']]
 [disable_equation_residual int]
 [convection bloc_convection]
 [diffusion bloc_diffusion]
 [boundary_conditions|conditions_limites condlims]
 [initial_conditions|conditions_initiales condinits]
 [sources sources]
 [ecrire_fichier_xyz_valeur ecrire_fichier_xyz_valeur]
 [parametre_equation parametre_equation_base]
 [equation_non_resolue str]
 [renommer_equation str]
}
where
```

- **exit\_on\_negative\_k\_omega** : Flag to exit (with postprocessing of fields) if a negative value is found for k or omega
- **exit\_on\_big\_omega** : Flag to exit (with postprocessing of fields) if an excessively big values of omega are found
- **with\_nu** *str* into ['yes', 'no']: yes/no (default no)
- **disable\_equation\_residual** *int* for inheritance: The equation residual will not be used for the problem residual used when checking time convergence or computing dynamic time-step
- **convection** *bloc\_convection* (5.2) for inheritance: Keyword to alter the convection scheme.
- **diffusion** *bloc\_diffusion* (5.3) for inheritance: Keyword to specify the diffusion operator.
- **boundary\_conditions|conditions\_limites** *condlims* (4.39.1) for inheritance: Boundary conditions.
- **initial\_conditions|conditions\_initiales** *condinits* (5.4) for inheritance: Initial conditions.
- **sources** *sources* (5.5) for inheritance: To introduce a source term into an equation (in case of several source terms into the same equation, the blocks corresponding to the various terms need to be separated by a comma)
- **ecrire\_fichier\_xyz\_valeur** *ecrire\_fichier\_xyz\_valeur* (3.57) for inheritance: This keyword is used to write the values of a field only for some boundaries in a text file
- **parametre\_equation** *parametre\_equation\_base* (5.6) for inheritance: Keyword used to specify additional parameters for the equation
- **equation\_non\_resolue** *str* for inheritance: The equation will not be solved while condition(t) is verified if equation\_non\_resolue keyword is used. Example: The Navier-Stokes equations are not solved between time t0 and t1.  
Navier\_Sokes\_Standard  
{ equation\_non\_resolue (t>t0)\*(t<t1) }
- **renommer\_equation** *str* for inheritance: Rename the equation with a specific name.



## 44 turbulence\_paro\_base

Description: Basic class for wall laws for Navier-Stokes equations.

See also: objet\_u (48) negligeable (44.7) loi\_puissance\_hydr (44.3) loi\_standard\_hydr (44.4) loi\_standard\_hydr\_old (44.5) paroi\_tble (44.8) utau\_imp (44.11)

Usage:

### 44.1 Loi\_ciofalo\_hydr

Description: A Loi\_ciofalo\_hydr law for wall turbulence for NAVIER STOKES equations.

See also: loi\_standard\_hydr (44.4)

Usage:

**loi\_ciofalo\_hydr**

### 44.2 Loi\_expert\_hydr

Description: This keyword is similar to the previous keyword Loi\_standard\_hydr but has several additional options into brackets.

See also: loi\_standard\_hydr (44.4)

Usage:

**loi\_expert\_hydr** *str*

**Read** *str* {

```
[u_star_impose float]
[methode_calcul_face_keps_impose str into ['toutes_les_faces_accrochees', 'que_les_faces_des-
_elts_dirichlet']]
[kappa float]
[Erugu float]
[A_plus float]
```

}

where

- **u\_star\_impose** *float*: The value of the friction velocity ( $u^*$ ) is not calculated but given by the user.
- **methode\_calcul\_face\_keps\_impose** *str* into ['toutes\_les\_faces\_accrochees', 'que\_les\_faces\_des-\_elts\_dirichlet']: The available options select the algorithm to apply K and Eps boundaries condition (the algorithms differ according to the faces).  
toutes\_les\_faces\_accrochees : Default option in 2D (the algorithm is the same than the algorithm used in Loi\_standard\_hydr)  
que\_les\_faces\_des\_elts\_dirichlet : Default option in 3D (another algorithm where less faces are concerned when applying K-Eps boundary condition).
- **kappa** *float*: The value can be changed from the default one (0.415)
- **Erugu** *float*: The value of E can be changed from the default one for a smooth wall (9.11). It is also possible to change the value for one boundary wall only with paroi\_rugueuse keyword/
- **A\_plus** *float*: The value can be changed from the default one (26.0)

### 44.3 Loi\_puissance\_hydr

Description: A Loi\_puissance\_hydr law for wall turbulence for NAVIER STOKES equations.

See also: turbulence\_paro\_base ([44](#))

Usage:

### 44.4 Loi\_standard\_hydr

Description: Keyword for the logarithmic wall law for a hydraulic problem. Loi\_standard\_hydr refers to first cell rank eddy-viscosity defined from continuous analytical functions, whereas Loi\_standard\_hydr\_3couches from functions separately defined for each sub-layer

See also: turbulence\_paro\_base ([44](#)) loi\_ww\_hydr ([44.6](#)) loi\_ciofalo\_hydr ([44.1](#)) loi\_expert\_hydr ([44.2](#))

Usage:

**loi\_standard\_hydr**

### 44.5 Loi\_standard\_hydr\_old

Description: not\_set

See also: turbulence\_paro\_base ([44](#))

Usage:

**loi\_standard\_hydr\_old**

### 44.6 Loi\_ww\_hydr

Description: laws have been qualified on channel calculation

See also: loi\_standard\_hydr ([44.4](#))

Usage:

### 44.7 Negligeable

Description: Keyword to suppress the calculation of a law of the wall with a turbulence model. The wall stress is directly calculated with the derivative of the velocity, in the direction perpendicular to the wall ( $\tau_{\text{tan}}/\rho = \nu \, dU/dy$ ).

Warning: This keyword is not available for k-epsilon models. In that case you must choose a wall law.

See also: turbulence\_paro\_base ([44](#))

Usage:

**negligeable**

### 44.8 Paroi\_tble

Description: Keyword for the Thin Boundary Layer Equation wall-model (a more complete description of the model can be found into this PDF file). The wall shear stress is evaluated thanks to boundary layer equations applied in a one-dimensional fine grid in the near-wall region.

See also: `turbulence_paro_base` ([44](#))

Usage:

**paroi\_tble** *str*

**Read** *str* {

```
[n int]
[facteur float]
[modele_visco str]
[stats twofloat]
[sonde_tble liste_sonde_tble]
[restart]
[stationnaire floatfloat]
[lambda str]
[mu str]
[sans_source_boussinesq]
[alpha float]
[kappa float]
```

}

where

- **n** *int*: Number of nodes in the TBLE grid (mandatory option).
- **facteur** *float*: Stretching ratio for the TBLE grid (to refine, the TBLE facteur must be greater than 1).
- **modele\_visco** *str*: File name containing the description of the eddy viscosity model.
- **stats** *twofloat* ([44.9](#)): Statistics of the TBLE velocity and turbulent viscosity profiles. 2 values are required : the starting time and ending time of the statistics computation.
- **sonde\_tble** *liste\_sonde\_tble* ([44.10](#))
- **restart**
- **stationnaire** *floatfloat* ([5.19](#))
- **lambda** *str*
- **mu** *str*
- **sans\_source\_boussinesq**
- **alpha** *float*
- **kappa** *float*

## 44.9 Twofloat

Description: two reals.

See also: `objet_lecture` ([47](#))

Usage:

**a b**

where

- **a** *float*: First real.
- **b** *float*: Second real.

## 44.10 Liste\_sonde\_tble

Description: `not_set`

See also: `listobj` ([46.5](#))

Usage:  
n object1 object2 ....  
list of *sonde\_tble* (44.10.1)

#### 44.10.1 Sonde\_tble

Description: not\_set

See also: objet\_lecture (47)

Usage:  
**name point**  
where

- **name** *str*
- **point** *un\_point* (3.4.7)

#### 44.11 Utau\_imp

Description: Keyword to impose the friction velocity on the wall with a turbulence model for thermohydraulic problems. There are two possibilities to use this keyword :

1 - we can impose directly the value of the friction velocity  $u_{\text{star}}$ .

2 - we can also give the friction coefficient and hydraulic diameter. So, TRUST determines the friction velocity by :  $u_{\text{star}} = U \cdot \sqrt{(\lambda_{\text{c}}/8)}$ .

See also: turbulence\_paro\_base (44)

Usage:  
**utau\_imp** *str*  
**Read** *str* {  
    [ **u\_tau** *champ\_base*]  
    [ **lambda\_c** *str*]  
    [ **diam\_hydr** *champ\_base*]  
}  
where

- **u\_tau** *champ\_base* (20.1): Field type.
- **lambda\_c** *str*: The friction coefficient. It can be function of the spatial coordinates x,y,z, the Reynolds number  $Re$ , and the hydraulic diameter.
- **diam\_hydr** *champ\_base* (20.1): The hydraulic diameter.

### 45 turbulence\_paro\_scalaire\_base

Description: Basic class for wall laws for energy equation.

See also: objet\_u (48) negligeable\_scalaire (45.7) loi\_odvm (45.4) loi\_WW\_scalaire (45.1) loi\_standard\_hydr\_scalaire (45.6) loi\_analytique\_scalaire (45.2) paroi\_tble\_scal (45.8) loi\_paro\_nu\_impose (45.5)

Usage:

## 45.1 Loi\_ww\_scalaire

Description: not\_set

See also: turbulence\_paro\_scalaire\_base (45)

Usage:

**loi\_WW\_scalaire**

## 45.2 Loi\_analytique\_scalaire

Description: not\_set

See also: turbulence\_paro\_scalaire\_base (45)

Usage:

**loi\_analytique\_scalaire**

## 45.3 Loi\_expert\_scalaire

Description: Keyword similar to keyword Loi\_standard\_hydr\_scalaire but with additional option.

See also: loi\_standard\_hydr\_scalaire (45.6)

Usage:

**loi\_expert\_scalaire** *str*

**Read** *str* {

    [ **prdt\_sur\_kappa** *float*]

    [ **calcul\_ldp\_en\_flux\_impose** *int into [0, 1]*]

}

where

- **prdt\_sur\_kappa** *float*: This option is to change the default value of 2.12 in the scalable wall function.
- **calcul\_ldp\_en\_flux\_impose** *int into [0, 1]*: By default (value set to 0), the law of the wall is not applied for a wall with a Neumann condition. With value set to 1, the law is applied even on a wall with Neumann condition.

## 45.4 Loi\_odvm

Description: Thermal wall-function based on the simultaneous 1D resolution of a turbulent thermal boundary-layer and a variance transport equation, adapted to conjugate heat-transfer problems with fluid/solid thermal interaction (where a specific boundary condition should be used : Paroi\_Echange\_Contact\_OVDM\_VDF). This law is also available with isothermal walls.

See also: turbulence\_paro\_scalaire\_base (45)

Usage:

**loi\_odvm** *str*

**Read** *str* {

**n** *int*

**gamma** *float*

```

 [stats floatfloat]
 [check_files]
}

```

where

- **n** *int*: Number of points per face in the 1D uniform meshes. n should be chosen in order to have the first point situated near  $\Delta y = 1/3$ .
- **gamma** *float*: Smoothing parameter of the signal between 10e-5 (no smoothing) and 10e-1 (high averaging).
- **stats** *floatfloat* (5.19): value\_t0 value\_dt : Only for plane channel flow, it gives mean and root mean square profiles in the fine meshes, since value\_t0 and every value\_dt seconds. The values are printed into files named ODVM\_fields\*.dat.
- **check\_files** : It gives for one boundary face a historical view of local instantaneous and filtered values, as well as the calculated variance profiles from the resolution of the equation. The printed values are into the file Suivi\_ndeb.dat.

## 45.5 Loi\_paroι\_nu\_impose

Description: Keyword to impose Nusselt numbers on the wall for the thermohydraulic problems. To use this option, it is necessary to give in the data file the value of the hydraulic diameter and the expression of the Nusselt number.

See also: turbulence\_paroι\_scalaire\_base (45)

Usage:

```

loi_paroι_nu_impose str
Read str {
 nusselt str
 diam_hydr champ_base
}

```

where

- **nusselt** *str*: The Nusselt number. This expression can be a function of x, y, z, Re (Reynolds number), Pr (Prandtl number).
- **diam\_hydr** *champ\_base* (20.1): The hydraulic diameter.

## 45.6 Loi\_standard\_hydr\_scalaire

Description: Keyword for the law of the wall.

See also: turbulence\_paroι\_scalaire\_base (45) loi\_expert\_scalaire (45.3)

Usage:

```

loi_standard_hydr_scalaire

```

## 45.7 Negligeable\_scalaire

Description: Keyword to suppress the calculation of a law of the wall with a turbulence model for thermohydraulic problems. The wall stress is directly calculated with the derivative of the velocity, in the direction perpendicular to the wall.

See also: [turbulence\\_paroï\\_scalaire\\_base \(45\)](#)

Usage:

**negligeable\_scalaire**

## 45.8 Paroi\_tble\_scal

Description: Keyword for the Thin Boundary Layer Equation thermal wall-model.

See also: [turbulence\\_paroï\\_scalaire\\_base \(45\)](#)

Usage:

**paroi\_tble\_scal** *str*

**Read** *str* {

```
[n int]
[facteur float]
[modele_visco str]
[nb_comp int]
[stats fourfloat]
[sonde_tble liste_sonde_tble]
[prandtl float]
```

}

where

- **n** *int*: Number of nodes in the TBLE grid (mandatory option).
- **facteur** *float*: Stretching ratio for the TBLE grid (to refine, the TBLE facteur must be greater than 1).
- **modele\_visco** *str*: File name containing the description of the eddy viscosity model.
- **nb\_comp** *int*: Number of component to solve in the fine grid (1 if 2D simulation (2D not available yet), 2 if 3D simulation).
- **stats** *fourfloat* ([45.9](#)): Statistics of the TBLE velocity and turbulent viscosity profiles. 4 values are required : the starting time of velocity averaging, the starting time of the RMS fluctuations, the ending time of the statistics computation and finally the print time period for the statistics.
- **sonde\_tble** *liste\_sonde\_tble* ([44.10](#))
- **prandtl** *float*

## 45.9 Fourfloat

Description: Four reals.

See also: [objet\\_lecture \(47\)](#)

Usage:

**a b c d**

where

- **a** *float*: First real.
- **b** *float*: Second real.
- **c** *float*: Third real.
- **d** *float*: Fourth real.

## 46 listobj\_impl

Description: not\_set

See also: objet\_u (48) listobj (46.5)

Usage:

### 46.1 Milieu\_musig

Description: MUSIG medium made of several sub mediums.

See also: listobj (46.5)

Usage:

{ object1 object2 .... }

list of *milieu\_base* (26)

### 46.2 Milieu\_composite

Description: Composite medium made of several sub mediums.

See also: listobj (46.5)

Usage:

{ object1 object2 .... }

list of *milieu\_base* (26)

### 46.3 List\_un\_pb

Description: pour les groupes

See also: listobj (46.5)

Usage:

{ object1 , object2 .... }

list of *un\_pb* (46.4) separeted with ,

### 46.4 Un\_pb

Description: pour les groupes

See also: objet\_lecture (47)

Usage:

**mot**

where

- **mot** *str*: the string



## 46.5 Listobj

Description: List of objects.

See also: [listobj\\_impl \(46\)](#) [listchamp\\_generique \(12.2\)](#) [champs\\_a\\_post \(4.2.2\)](#) [definition\\_champs \(4.2.4\)](#) [sondes \(4.2.7\)](#) [list\\_stat\\_post \(4.2.29\)](#) [post\\_processings \(4.3\)](#) [liste\\_post\\_ok \(4.4\)](#) [liste\\_post \(4.5\)](#) [list\\_un\\_pb \(46.3\)](#) [list\\_list\\_nom \(4.37\)](#) [condlims \(4.39.1\)](#) [condinits \(5.4\)](#) [sources \(5.5\)](#) [coarsen\\_operators \(3.98\)](#) [listpoints \(3.4.6\)](#) [vect\\_nom \(3.149\)](#) [list\\_nom \(3.134\)](#) [list\\_nom\\_virgule \(12.3\)](#) [list\\_bord \(3.90.4\)](#) [list\\_bloc\\_mailler \(3.90\)](#) [list\\_info\\_med \(4.74\)](#) [listsous\\_zone\\_valeur \(5.2.19\)](#) [reactions \(13.1\)](#) [listeqn \(4.14\)](#) [Milieu\\_composite \(46.2\)](#) [Milieu\\_MUSIG \(46.1\)](#) [liste\\_sonde\\_tble \(44.10\)](#) [listdeuxmots\\_acc \(4.9\)](#) [listdeuxmots\\_sacc \(41.61\)](#)

Usage:

## 47 objet\_lecture

Description: Auxiliary class for reading.

See also: [objet\\_u \(48\)](#) [bloc\\_lecture \(3.2\)](#) [deuxmots \(4.9.1\)](#) [troismots \(7.2\)](#) [quatremots \(47.1\)](#) [deuxentiers \(5.22.8\)](#) [floatfloat \(5.19\)](#) [entierfloat \(47.2\)](#) [bloc\\_lecture\\_poro \(34.1\)](#) [postraitement\\_base \(4.4.2\)](#) [champ\\_a\\_post \(4.2.3\)](#) [interface\\_posts \(4.2.1\)](#) [definition\\_champ \(4.2.5\)](#) [definition\\_champs\\_fichier \(4.2.6\)](#) [sonde\\_base \(4.2.9\)](#) [sonde \(4.2.8\)](#) [sondes\\_fichier \(4.2.24\)](#) [champs\\_posts \(4.2.25\)](#) [bloc\\_fichier \(4.2.27\)](#) [champs\\_posts\\_fichier \(4.2.26\)](#) [stat\\_post\\_deriv \(4.2.30\)](#) [stats\\_posts \(4.2.28\)](#) [stats\\_posts\\_fichier \(4.2.36\)](#) [stats\\_serie\\_posts \(4.2.37\)](#) [stats\\_serie\\_posts\\_fichier \(4.2.38\)](#) [un\\_postraitement \(4.3.1\)](#) [nom\\_postraitement \(4.4.1\)](#) [type\\_un\\_post \(4.5.2\)](#) [type\\_postraitement\\_ft\\_lata \(4.5.3\)](#) [un\\_postraitement\\_spec \(4.5.1\)](#) [format\\_file\\_base \(4.6\)](#) [un\\_pb \(46.4\)](#) [troisf \(3.70\)](#) [convection\\_deriv \(5.2.1\)](#) [bloc\\_convection \(5.2\)](#) [diffusion\\_deriv \(5.3.1\)](#) [op\\_implicite \(5.3.23\)](#) [bloc\\_diffusion \(5.3\)](#) [condlimlu \(4.39.2\)](#) [condinit \(5.4.1\)](#) [parametre\\_equation\\_base \(5.6\)](#) [traitement\\_particulier\\_base \(5.18.1\)](#) [dt\\_impr\\_ustar\\_mean\\_only \(5.22.1\)](#) [modele\\_turbulence\\_hyd\\_deriv \(5.22\)](#) [form\\_a\\_nb\\_points \(5.22.3\)](#) [Coarsen\\_Operator\\_Uniform \(3.98.1\)](#) [un\\_point \(3.4.7\)](#) [format\\_lata\\_to\\_CGNS \(3.83\)](#) [format\\_lata\\_to\\_med \(3.85\)](#) [bloc\\_pdf\\_model \(41.53\)](#) [defbord \(3.90.7\)](#) [bord\\_base \(3.90.5\)](#) [mailler\\_base \(3.90.1\)](#) [bloc\\_origine\\_cotes \(42.1\)](#) [bloc\\_couronne \(42.2\)](#) [bloc\\_tube \(42.3\)](#) [lecture\\_bloc\\_moment\\_base \(3.42\)](#) [bloc\\_pave \(3.90.3\)](#) [remove\\_elem\\_bloc \(3.122\)](#) [bloc\\_decouper \(3.103\)](#) [bloc\\_lec\\_champ\\_init\\_canal\\_sinal \(20.20\)](#) [fonction\\_champ\\_reprise \(20.16\)](#) [info\\_med \(4.74.1\)](#) [verifiercoin\\_bloc \(3.152\)](#) [bloc\\_diffusion\\_standard \(5.3.8\)](#) [bloc\\_ef \(5.2.7\)](#) [sous\\_zone\\_valeur \(5.2.20\)](#) [dt\\_impr\\_nusselt\\_mean\\_only \(29.1\)](#) [traitement\\_particulier \(5.18\)](#) [reaction \(13.1.1\)](#) [spec\\_pdc\\_r\\_base \(41.42\)](#) [type\\_perte\\_charge\\_deriv \(41.7\)](#) [bloc\\_sutherland \(26.12\)](#) [type\\_diffusion\\_turbulente\\_multiphase\\_deriv \(5.3.10\)](#) [floatentier \(5.22.9\)](#) [modele\\_fonction\\_bas\\_reynolds\\_base \(5.22.23\)](#) [twofloat \(44.9\)](#) [sonde\\_tble \(44.10.1\)](#) [fourfloat \(45.9\)](#) [bloc\\_lecture\\_turb\\_synt \(21.11\)](#) [paroi\\_ft\\_disc\\_deriv \(17.77\)](#) [methode\\_transport\\_deriv \(5.62\)](#) [bloc\\_lecture\\_remaillage \(5.63\)](#) [objet\\_lecture\\_maintien\\_temperature \(5.41\)](#) [interpolation\\_champ\\_face\\_deriv \(5.65\)](#) [type\\_indic\\_faces\\_deriv \(5.66\)](#) [parcours\\_interface \(5.64\)](#) [injection\\_marqueur \(5.69\)](#) [penalisation\\_forcage \(5.51\)](#) [eq\\_rayo\\_semi\\_transp \(4.39\)](#) [type\\_diffusion\\_turbulente\\_multiphase\\_multiple\\_deriv \(47.3\)](#) [bloc\\_rho\\_fonc\\_c \(5.55.2\)](#) [bloc\\_boussinesq \(5.55.1\)](#) [approx\\_boussinesq \(5.55\)](#) [bloc\\_mu\\_fonc\\_c \(5.56.2\)](#) [bloc\\_visco2 \(5.56.1\)](#) [visco\\_dyn\\_cons \(5.56\)](#) [bloc\\_lecture\\_Structural\\_dynamic\\_mesh\\_model \(3.30\)](#) [ceg\\_areva \(5.18.11\)](#) [ceg\\_cea\\_jaea \(5.18.12\)](#) [NewmarkTimeScheme\\_deriv \(3.4.2\)](#) [bloc\\_poutre \(3.4.1\)](#) [bloc\\_lecture\\_beam\\_model \(3.4\)](#) [systeme\\_naire\\_deriv \(41.47\)](#) [bloc\\_kappa\\_variable \(41.47.2\)](#) [bloc\\_potentiel\\_chim \(41.47.3\)](#)

Usage:

### 47.1 Quatremots

Description: Three words.

See also: [objet\\_lecture \(47\)](#)

Usage:

**mot\_1 mot\_2 mot\_3 mot\_4**

where

- **mot\_1** *str*: First word.
- **mot\_2** *str*: Snd word.
- **mot\_3** *str*: Third word.
- **mot\_4** *str*: Fourth word.

## 47.2 Entierfloat

Description: An integer and a real.

See also: `objet_lecture` ([47](#))

Usage:

**the\_int the\_float**

where

- **the\_int** *int*: Integer.
- **the\_float** *float*: Real.

## 47.3 Type\_diffusion\_turbulente\_multiphase\_multiple\_deriv

Description: `not_set`

See also: `objet_lecture` ([47](#)) `k_omega` ([5.3.18](#)) `sato` ([5.3.19](#))

Usage:

## 48 index

## Index

/\*, 320  
#, 340  
 , 27, 39, 43, 69, 72, 192, 198, 219, 437, 522  
aire\_interfaciale, 203  
associer, 40  
champ\_post\_interpolation, 326, 431  
champ\_post\_statistiques\_correlation, 104, 323  
champ\_post\_statistiques\_ecart\_type, 104, 324  
champ\_post\_statistiques\_moyenne, 103, 327  
champ\_uniforme, 384  
decoupebord, 44  
decouper, 71, 434  
decouper\_multi, 72  
discretiser, 46  
divergence, 323  
echange\_externe\_radiatif, 346  
ecrire\_fichier, 91  
extraction, 324  
fin, 55  
frontiere\_ouverte\_temperature\_imposee, 352  
gradient, 325  
interpolation\_ibm\_aucune, 399  
interpolation\_ibm\_element\_fluide, 399  
interpolation\_ibm\_gradient\_moyen, 400  
interpolation\_ibm\_hybride, 399  
interpolation\_ibm\_power\_law\_tbl, 401  
lata\_to\_med, 58  
lata\_to\_other, 59  
lire, 76  
lire\_fichier, 76  
lire\_fichier\_bin, 77  
lire\_med, 37  
lml\_to\_lata, 60  
morceau\_equation, 326  
operateur\_eqn, 321  
postraitement, 107  
postraitements, 106  
probleme\_ft\_disc\_gen, 113  
raffiner\_simplexes, 75  
rectify\_mesh, 77  
reduction\_0d, 328  
refchamp, 329  
resoudre, 82  
runge\_kutta\_ordre\_4, 466  
schema\_euler\_explicite, 452  
schema\_euler\_implicite, 488  
schema\_euler\_implicite\_stationnaire, 444  
sous\_domaine, 547  
temperature\_imposee\_parois, 367  
tparois\_vof, 329  
transformation, 330  
vefprep1b, 368  
0, 81  
1, 81  
2, 81  
6\_points, 242, 429  
≤, 63  
=, 63  
A, 347  
a, 535  
a\_ext, 348, 351, 352  
A\_i\_ext, 348, 351, 352  
all\_times, 32  
amont, 195  
analytique, 304, 306  
ancien, 262–265  
antisym, 193  
approx, 318, 319  
arrete, 230–245  
avec\_energie\_cinetique, 273, 274  
avec\_les\_cl, 217, 218, 227, 228, 257–260, 283–287, 289–295, 298–302  
avec\_sources, 217, 218, 227, 228, 257–260, 283–287, 289–295, 298–302  
avec\_sources\_et\_operateurs, 217, 218, 227, 228, 257–260, 283–287, 289–295, 298–302  
average, 328, 329  
b, 535  
binaire, 47, 101, 376  
both, 318, 319  
C, 419  
C\_ext, 348, 351, 352  
celsius, 346  
centre, 195  
cf, 535  
cgns, 58, 59, 73, 93, 94, 107, 108  
chakravarthy, 195  
Champ\_Fonc\_Fonction, 296, 297  
champ\_frontiere, 325  
Champ\_Uniforme, 296  
check\_pass, 30  
chsom, 96  
coarsen\_i, 69  
coarsen\_j, 69  
coarsen\_k, 69  
composante, 330, 331  
concentration, 296, 297  
conservation\_masse, 417, 418  
constant, 417, 418, 422  
coriolis\_seul, 529

CORRECTION\_GHOST\_INDIC , 287, 288  
Cotes , 548  
d , 535  
debit\_total , 57  
default , 326  
default\_bar , 193, 200  
diametre , 431  
dir , 549  
direction , 431  
disabled , 30  
distant , 64  
divrhout\_moins\_Tdivrhout , 262–265  
divuT\_moins\_Tdivu , 262–265  
domaine , 72  
double , 68, 69  
dt\_integr , 105, 106  
dt\_post , 101, 102, 105  
edo , 417, 418  
elem , 67, 68, 95, 103, 104, 371, 372, 376  
emissivite , 346, 347  
entrainement\_seul , 529  
euclidian\_norm , 328, 329  
exact , 318, 319  
faces , 95, 103, 104  
fichier , 430, 431  
filtrer\_resu , 193, 200, 201  
Fluctu\_Temperature\_ext , 348, 351, 352  
flux\_bords , 326, 327  
Flux\_Chaleur\_Turb\_ext , 348, 351, 352  
flux\_surfacique\_bords , 326, 327  
fonction , 377  
format\_post\_sup , 58, 59  
formatte , 47, 101, 376  
formule , 330, 331  
grad\_i , 57, 58, 287, 288  
grad\_Ubar , 200, 201  
grav , 96  
gravcl , 96  
H\_ext , 348, 351, 352  
h\_imp , 346, 360, 361  
hauteur , 549  
homogene , 64  
implicite , 205  
initiale , 304, 306  
integrale\_en\_z , 57  
interp\_ai\_based , 287, 288  
interp\_modifiee , 287, 288  
interp\_standard , 287, 288  
K , 536  
k , 364  
K\_Eps\_ext , 348, 351, 352  
k\_ext , 348, 351, 352  
K\_Omega\_ext , 348, 351, 352  
kelvin , 346  
kx , 536  
ky , 536  
kz , 536  
L1\_norm , 328, 329  
L2 , 81  
L2\_norm , 328, 329  
last\_time , 32  
lata , 58, 59, 73, 93, 94, 107, 108  
lata\_v2 , 58, 59, 73, 93, 94, 107, 108  
left\_value , 328, 329  
lml , 58, 59, 73, 93, 94, 107, 108  
local , 64  
max , 81, 328, 329  
mean\_volumic\_velocity , 303, 305  
med , 58, 59, 73, 93, 94, 107, 108  
Mfront\_library , 39  
Mfront\_material\_property , 39  
Mfront\_model\_name , 39  
min , 328, 329  
minmod , 195  
mixed , 68, 69  
modifiee , 304, 306  
moins\_rho\_moyen , 417, 418  
moy\_euler , 242, 429  
moyenne , 328, 329  
moyenne\_ponderee , 328, 329  
mpi-io , 93, 94, 108  
mu0 , 418  
multiple , 93, 94, 108  
muscl , 195  
name , 27  
natural , 439, 440  
nb\_beam , 26  
nb\_pas\_dt\_post , 101, 102, 105  
no , 326, 551, 552  
nodes , 96  
non , 70, 295, 296, 537  
normalized\_euclidian\_norm , 328, 329  
norme , 330, 331  
nu , 200, 201  
nu\_transp , 200, 201  
nut , 200, 201  
nut\_transp , 200, 201  
omega\_ext , 348, 351, 352  
one\_way\_coupling , 312, 313  
Origine , 548, 549  
oui , 70, 295, 296, 537  
pdi , 376  
periode , 96  
plans\_paralleles , 242, 429  
post\_processing , 110  
postraitement , 110  
postraitement\_ft\_lata , 110  
produit\_scalaire , 330, 331

que\_les\_faces\_des\_elts\_dirichlet , 553  
 rcm , 439, 440  
 re , 548, 549  
 rho\_g , 57, 58, 287, 288  
 ri , 548, 549  
 sans\_energie\_cinetique , 273, 274  
 sans\_rien , 217, 218, 227, 228, 257–260, 283–287, 289–295, 298–302  
 scotti , 230–245  
 SEMI\_TRANSP , 355  
 simple , 93, 94, 108  
 simplifiee , 304, 306  
 single\_hdf , 376  
 single\_lata , 73, 93, 94, 107, 108  
 Slambda , 418  
 solveur , 205  
 som , 67, 68, 95, 96, 103, 104, 371, 372, 376  
 somme , 328, 329  
 somme\_ponderee , 328, 329  
 somme\_ponderee\_porosite , 328, 329  
 stabilite , 326, 327  
 standard , 417, 418  
 suivi , 312, 313  
 sum , 328, 329  
 superbee , 195  
 surface , 522  
 T0 , 418  
 T\_ext , 348, 351, 352  
 t\_ext , 346, 360, 361  
 tau\_ext , 348, 351, 352  
 temperature\_unit , 346  
 terme\_complet , 529  
 toutes\_les\_faces\_accrochees , 553  
 trace , 325  
 TRANSP , 355  
 transportant\_bar , 193  
 transporte\_bar , 193  
 two\_way\_coupling , 312, 313  
 u\_tau , 431  
 uniforme , 304, 306  
 V2\_ext , 348, 351, 352  
 valeur\_a\_elem , 303, 305  
 valeur\_a\_gauche , 328, 329  
 valeur\_normale , 395  
 vanalbada , 195  
 vanleer , 195  
 vdf\_lineaire , 303, 305  
 vecteur , 330, 331  
 visco\_cin , 431  
 vitesse\_interpoelee , 312, 313  
 vitesse\_parois , 364  
 vitesse\_particules , 312, 313  
 vitesse\_tangentielle , 397  
 volume , 230–245  
 volume\_sans\_lissage , 230–245  
 weighted\_average , 328, 329  
 weighted\_sum , 328, 329  
 weighted\_sum\_porosity , 328, 329  
 write\_pass , 30  
 X , 63, 82, 549  
 x , 535  
 xyz , 376  
 Y , 63, 82, 549  
 y , 535  
 Y\_ext , 348, 351, 352  
 yes , 326, 551, 552  
 Z , 63, 82, 549  
 z , 535  
 , 26, 39, 43, 69, 72, 192, 198, 219, 437, 522  
**all\_options** , 70  
**champs** , 94, 109  
**champs\_fichier** , 94, 109  
**conditions\_initiales** , 191, 207, 209–216, 218, 227, 228, 259–276, 278–280, 282–284, 286, 289, 291, 293, 295, 299, 300, 302–304, 311, 312, 549–552  
**conditions\_limites** , 148, 191, 207, 209–216, 218, 227, 228, 258, 260–276, 278–280, 282–284, 286, 289, 291, 293, 295, 299, 300, 302, 303, 306, 311, 313, 549–552  
**definition\_champs\_fichier** , 94, 108  
**domain** , 38  
**domaine** , 74  
**exclude\_groups** , 38  
**fichier** , 74, 96, 101  
**file** , 38  
**fluide0** , 409, 415  
**fluide1** , 409, 415  
**hydraulic\_equation** , 318  
**include\_additional\_face\_groups** , 38  
**interface\_equation** , 318  
**is\_multi\_scalar** , 209, 266–268, 274  
**limiter** , 204  
**maillage\_vdf** , 36  
**mesh** , 38  
**name\_of\_initial\_domaines** , 37  
**name\_of\_new\_domaines** , 37  
**par\_sous\_zone** , 29  
**partitionneur** , 71  
**postraitements** , 92, 112, 114–117, 119–122, 124, 126–130, 132–137, 139–141, 143–145, 147, 149, 150, 152–156, 158–161, 163–167, 169–172, 174–176, 178–181, 183–186, 188, 190  
**postraitements** , 92, 112, 114–117, 119–122, 124, 126–130, 132–137, 139–141, 143–145, 147, 149, 150, 152–156, 158–161, 163–167, 169–172, 174–176, 178–180, 182–185, 187, 188, 190

pr\_t , 203  
 prandtl\_eps , 250, 252, 253, 255, 256  
 prandtl\_k , 247, 250, 252, 253, 255, 256  
 prandtl\_omega , 247  
 Read\_file , 91  
 reduction\_pression , 502  
 sans\_dec , 35  
 save\_matrice , 335–339, 516, 518  
 sigma , 202  
 sondes , 94, 108  
 sondes\_fichier , 94, 108  
 sondes mobiles , 94, 108  
 sondes mobiles\_fichier , 94, 108  
 sous\_domaine , 50, 94, 108  
 statistiques , 94, 109  
 statistiques\_en\_serie , 94, 109  
 statistiques\_en\_serie\_fichier , 94, 109  
 statistiques\_fichier , 94, 109  
 tension\_superficielle , 316, 317  
 thermal\_equation , 318  
 a0 , 333  
 A\_plus , 553  
 a\_res , 532  
 Abse\_file\_name , 27  
 acceleration , 529  
 activate\_collision\_before\_impact , 314, 340  
 activation\_distance\_percentage\_diameter , 314, 340  
 adjoint , 258  
 adjust\_meso\_ml , 319  
 aij , 519  
 aire , 540, 541  
 ajout\_init\_a\_reprise , 226  
 alias , 209, 266–268, 274  
 alpha , 33, 196, 365, 538, 555  
 alpha\_0 , 439  
 alpha\_1 , 439  
 alpha\_a , 439  
 alpha\_omega , 526  
 alpha\_sous\_zone , 196  
 amont\_sous\_zone , 196  
 ampli\_bruit , 378  
 ampli\_fluctuation , 396  
 ampli\_moyenne\_imposee , 396  
 ampli\_moyenne\_recyclee , 396  
 ampli\_sin , 378  
 approximation\_de\_boussinesq , 294  
 areva , 223  
 ascii , 37, 84  
 atol , 511–514, 516, 519, 520  
 autre\_bord , 345, 346  
 autre\_champ\_indicatrice , 346  
 autre\_champ\_temperature , 345  
 autre\_champ\_temperature\_indic0 , 346  
 autre\_champ\_temperature\_indic1 , 346  
 autre\_probleme , 345, 346  
 avec\_certains\_bords , 31, 52  
 avec\_certains\_bords\_pour\_extraire\_surface , 51  
 avec\_les\_bords , 31, 52  
 avoid\_duplicata , 282  
 BaseCenterCoordinates , 28  
 bench\_ijk\_splitting\_read , 36  
 bench\_ijk\_splitting\_write , 36  
 bendingDirection , 27  
 beta , 365, 524, 525, 538  
 beta\_co , 410–412, 416, 417  
 beta\_disp , 522  
 beta\_k , 204, 528  
 beta\_lift , 522  
 beta\_omega , 524  
 beta\_th , 410–412, 416, 417  
 bilan\_pdf , 540  
 binaire , 45, 73  
 binary\_file , 48  
 block\_size\_bytes , 36  
 block\_size\_megabytes , 36  
 boite , 548  
 bord , 43, 66, 220, 531  
 bords\_a\_decouper , 45  
 boundaries , 48, 230, 426  
 boundary\_conditions , 148, 191, 207, 209–216, 218, 227, 228, 258, 260–276, 278–280, 282–284, 286, 289, 291, 293, 295, 299, 300, 302, 303, 306, 311, 313, 549–552  
 boundary\_xmax , 66  
 boundary\_xmin , 66  
 boundary\_ymax , 66  
 boundary\_ymin , 66  
 boundary\_zmax , 66  
 boundary\_zmin , 66  
 boussinesq\_approximation , 288  
 btd , 197  
 c , 223  
 c0 , 529  
 c1\_eps , 528, 544, 545  
 c2\_eps , 528, 544, 545  
 c3\_eps , 528, 545  
 c\_epsilon , 527  
 c\_k , 525, 527  
 calc\_spectre , 221, 222  
 calcul\_ldp\_en\_flux\_impose , 557  
 canal , 235  
 canalx , 234  
 cea\_jaea , 223  
 centre\_rotation , 529  
 cfl , 499–501  
 chaleur\_latente , 409, 415  
 champ\_front\_normal\_et\_tangentiel\_robin , 365

champ\_med , 57  
 changement\_de\_base\_p1bulle , 369  
 check\_divergence , 35  
 check\_files , 558  
 checkpoint\_fname , 112  
 CI\_file\_name , 28  
 cl\_pression\_sommet\_faible , 369  
 cli , 516, 518  
 cli\_quiet , 516  
 clipping\_courbure\_interface , 287  
 close\_every\_n , 34  
 cmu , 250, 253  
 coarsen\_operators , 68  
 coef , 408  
 coef\_ammortissement , 226  
 coef\_force\_time\_n , 227  
 coef\_immobilisation , 226  
 coef\_mean\_force , 227  
 coef\_rayon\_force\_rappel , 227  
 coeff , 531, 536  
 coeff\_evol\_volume , 226  
 coeffa , 318  
 coeffb , 318  
 coefficient\_diffusion , 413  
 coefficients\_activites , 332  
 collision\_duration , 314, 340  
 collision\_model , 314, 340  
 collision\_model\_fpi , 306  
 collisions , 305  
 compo , 322, 327  
 compute\_distance\_autres\_interfaces , 58, 282  
 compute\_force\_init , 226  
 compute\_particles\_rms , 306  
 condition\_elements , 31, 50, 52  
 condition\_faces , 31, 52  
 condition\_geometrique , 45  
 Conduction , 92, 128  
 Conduction\_ibm , 112  
 conservation\_Ec , 221, 222  
 constante\_cinetique , 209  
 constante\_gravitation , 525, 527  
 constante\_modele\_micro\_melange , 331  
 constante\_taux\_reaction , 332  
 constituant , 92, 112, 114–118, 120–122, 124, 125, 127–130, 132–137, 139–141, 143–145, 147, 149, 150, 152–157, 159–161, 163–167, 169–172, 174–176, 178–181, 183–186, 188, 190  
 contre\_energie\_activation , 332  
 contre\_reaction , 332  
 contribution\_one\_way , 313  
 controle\_residu , 337, 502–510  
 convection , 191, 207, 209–216, 218, 227, 228, 258, 260–276, 278–280, 282–284, 286, 289, 291, 293, 295, 298, 300, 302, 303, 306, 311, 313, 549–552  
 convection\_diffusion\_chaleur\_QC , 133, 170, 178  
 convection\_diffusion\_chaleur\_turbulent\_qc , 135, 180, 185  
 convection\_diffusion\_chaleur\_WC , 171, 179  
 convection\_diffusion\_concentration , 115, 136, 155, 156, 172, 174  
 convection\_diffusion\_concentration\_turbulent , 116, 137, 158, 159, 175, 176  
 convection\_diffusion\_espece\_binaire\_QC , 161  
 Convection\_Diffusion\_Espece\_Binaire\_Turbulent-QC , 164  
 convection\_diffusion\_espece\_binaire\_WC , 163  
 convection\_diffusion\_phase\_field , 166  
 convection\_diffusion\_temperature , 132, 136, 140, 169, 172, 174, 183  
 convection\_diffusion\_temperature\_ibm , 181  
 convection\_diffusion\_temperature\_ibm\_turbulent , 139  
 Convection\_Diffusion\_Temperature\_Sensibility , 142  
 convection\_diffusion\_temperature\_turbulent , 134, 137, 141, 175, 176, 184, 186  
 convection\_sensibility , 211  
 convertalltopoly , 38  
 correction\_calcul\_pression\_initiale , 290  
 correction\_force , 226  
 correction\_fraction , 405  
 correction\_gradt\_ordre , 276  
 correction\_matrice\_pression , 290  
 correction\_matrice\_projection\_initiale , 290  
 correction\_mpoint\_diff\_conv\_energy , 276  
 correction\_parcours\_thomas , 309  
 correction\_pression\_modifie , 291  
 correction\_variable\_initiale , 207, 277, 291  
 correction\_visco\_turb\_pour\_controle\_pas\_de\_temps , 229, 231–233, 235–239, 241–250, 252, 254–257  
 correction\_visco\_turb\_pour\_controle\_pas\_de\_temps-\_parametre , 229, 231–233, 235–237, 239–250, 253–257  
 correction\_vitesse\_modifie , 291  
 correction\_vitesse\_projection\_initiale , 290  
 corrections\_qdm , 227  
 correlations , 124, 125, 127, 131, 143, 169  
 correspondance\_elements , 398–401  
 corriger\_partition , 433  
 couplage\_NS\_CH , 537  
 couronne , 548  
 Cp , 402, 404, 406  
 cp , 48, 357, 358, 405–407, 409–423  
 crank , 206  
 critere\_absolu , 53

**critere\_arete** , 79, 308  
**critere\_longueur\_fixe** , 309  
**criteres\_convergence** , 502, 506  
**cs** , 202, 233  
**cstdiff** , 203  
**Cv** , 406, 421  
**cw** , 201, 232  
**d** , 383, 385, 389  
**deactivate** , 319  
**deactivate\_arete\_mixte** , 70  
**deb** , 523  
**debit** , 357, 358, 527  
**debit\_impose** , 531  
**debug** , 223  
**debut\_stat** , 219  
**decoup** , 371, 372, 376  
**default\_value** , 371  
**definition\_champs** , 94, 108  
**definition\_champs\_file** , 94, 108  
**delta** , 356  
**delta\_spot** , 527  
**deprecatedkeepduplicatedprobes** , 94, 108  
**derivee\_rotation** , 408  
**detection\_method** , 314, 340  
**dh** , 357, 358, 524  
**diag** , 337  
**diam\_bulle\_monodisperse** , 226  
**diam\_hydr** , 533, 534, 556, 558  
**diam\_hydr\_ortho** , 533  
**diametre\_hyd\_champ** , 409–424  
**diffusion** , 191, 207, 209–216, 218, 227, 228, 258, 260–276, 278–280, 282–284, 286, 289, 291, 293, 295, 298, 300, 302, 303, 306, 311, 313, 549–552  
**diffusion\_alternative** , 226  
**diffusion\_coeff** , 403, 405  
**diffusion\_implicite** , 444, 446, 448, 451, 453, 455, 457, 459, 461, 463, 465, 467, 469, 471, 473, 475, 477, 479, 482, 485, 487, 490, 492, 494, 496, 498  
**dim\_espace\_krilov** , 337  
**dimension\_espace\_de\_krylov** , 537  
**dir** , 357, 536  
**dir\_flow** , 378  
**dir\_fluct** , 388  
**dir\_wall** , 378  
**direct\_write** , 39, 40  
**direction** , 43, 52–54, 220, 533, 534  
**direction\_anisotrope** , 396  
**disable\_convection\_qdm** , 226  
**disable\_diffusion\_qdm** , 226  
**disable\_diphasique** , 35  
**disable\_dt\_ev** , 444, 447, 449, 451, 453, 455, 457, 459, 461, 463, 465, 467, 469, 471, 473, 475, 477, 480, 483, 485, 488, 490, 493, 495, 497, 499  
**disable\_equation\_residual** , 191, 207, 209–216, 218, 227, 228, 258, 260–280, 282–284, 286, 289, 291, 293, 295, 298, 300, 302, 303, 306, 311, 313, 549–552  
**disable\_progress** , 444, 447, 449, 451, 453, 455, 457, 459, 461, 463, 465, 467, 469, 471, 473, 475, 477, 480, 483, 485, 488, 490, 493, 495, 497, 499  
**disable\_solveur\_poisson** , 226  
**disable\_source\_interf** , 226  
**disable\_VEF\_mean\_value\_corrections** , 551  
**distance\_plan** , 395  
**distance\_projete\_faces** , 306  
**distri\_first\_facette** , 319  
**dmax** , 234  
**dom\_dist** , 371  
**dom\_loc** , 371  
**domain** , 65, 74, 372, 376  
**domaine** , 31, 38, 43, 45, 50, 51, 53, 54, 94, 108, 325, 326, 434  
**domaine\_final** , 29, 52  
**domaine\_flottant\_fluide** , 290  
**domaine\_grossier** , 45  
**domaine\_init** , 29, 52  
**domaines** , 74, 435  
**domegadt** , 529  
**DP0** , 523  
**dropping\_parameter** , 516  
**dt** , 48  
**dt\_impr** , 230, 357, 358, 426, 443, 446, 448, 451, 453, 454, 456, 458, 460, 462, 464, 467, 468, 470, 473, 474, 476, 479, 482, 484, 487, 489, 492, 494, 496, 498  
**dt\_impr\_moy\_spat** , 219  
**dt\_impr\_moy\_temp** , 219  
**dt\_impr\_nusselt** , 426–429  
**dt\_impr\_nusselt\_mean\_only** , 426–429  
**dt\_impr\_ustar** , 229, 231–233, 235–241, 243–246, 248–250, 252–256  
**dt\_impr\_ustar\_mean\_only** , 229, 231–233, 235–241, 243–246, 248–250, 252, 254–256  
**dt\_injection** , 314  
**dt\_max** , 443, 446, 448, 450, 452, 454, 456, 458, 460, 462, 464, 466, 468, 470, 472, 474, 476, 479, 482, 484, 487, 489, 492, 494, 496, 498  
**dt\_min** , 443, 446, 448, 450, 452, 454, 456, 458, 460, 462, 464, 466, 468, 470, 472, 474, 476, 479, 482, 484, 487, 489, 492, 494, 496, 498  
**dt\_post** , 94, 108, 223



**dt\_projection** , 217, 228, 258, 259, 284, 285, 288, 291, 292, 294, 298, 300, 301  
**dt\_sauv** , 443, 446, 448, 450, 452, 454, 456, 458, 460, 462, 464, 466, 468, 470, 472, 474, 476, 479, 482, 484, 487, 489, 492, 494, 496, 498  
**dt\_sauvegarde** , 499–501  
**dt\_start** , 444, 446, 449, 451, 453, 455, 457, 459, 461, 463, 465, 467, 469, 471, 473, 475, 477, 480, 482, 485, 487, 490, 492, 494, 496, 498  
**dt\_uniforme** , 319  
**dtol\_fraction** , 405  
**dual** , 74  
**dv\_min** , 532  
**e\_dry** , 410–412  
**Ec** , 220  
**Ec\_dans\_repere\_fixe** , 220  
**echelle\_relaxation\_coefficient\_pdf** , 540  
**Echelle\_temporelle\_turbulente** , 124, 125, 127  
**ecrire\_decoupage** , 71  
**ecrire\_fichier\_xyz\_valeur** , 191, 207, 209–216, 218, 227, 229, 259–276, 278–284, 286, 289, 291, 293, 295, 299, 300, 302, 303, 307, 312, 313, 549–552  
**ecrire\_frontiere** , 74  
**ecrire\_lata** , 72  
**ecrire\_med** , 72  
**elements\_fluides** , 399–401  
**elements\_solides** , 398, 400, 401  
**emissivite\_pour\_rayonnement\_entre\_deux\_plaques\_quasi\_infinies** , 358  
**energie\_activation** , 332  
**Energie\_cinetique\_turbulente** , 124, 125, 127  
**Energie\_cinetique\_turbulente\_WIT** , 124, 125, 127  
**Energie\_Multiphase** , 124, 127  
**Energie\_Multiphase\_h** , 125  
**ensemble\_points** , 314  
**enthalpie\_reaction** , 332  
**epaisseur** , 51, 53  
**eps** , 523  
**eps\_max** , 248, 250, 252, 253  
**eps\_min** , 248, 250, 252, 253  
**epsilon** , 441, 442  
**eq\_rayo\_semi\_transp** , 147  
**equation\_frequence\_resolue** , 207  
**equation\_interface** , 208, 267, 276, 318  
**equation\_interfaces\_proprietes\_fluide** , 287  
**equation\_interfaces\_vitesse\_imposee** , 287  
**equation\_navier\_stokes** , 276, 318  
**equation\_non\_resolue** , 191, 207, 209–218, 227, 229, 259–275, 277–284, 286, 289, 291, 293, 295, 299, 300, 302, 303, 307, 312, 313, 549–552  
**equation\_nu\_t** , 209  
**equation\_temperature** , 318  
**equation\_temperature\_mpoint** , 288  
**equation\_temperature\_mpoint\_vapeur** , 288  
**equations\_interfaces\_vitesse\_imposee** , 287  
**equations\_scalaires\_passifs** , 150, 156, 159, 174, 176, 178–180, 183, 186  
**equations\_source\_chimie** , 208  
**equilateral** , 79  
**Erugu** , 553  
**erugu** , 364  
**espece** , 271, 273  
**espece\_en\_competition\_micro\_melange** , 331  
**est\_dirichlet** , 398, 400  
**eta** , 540  
**evanescence** , 261  
**exclure\_groupes** , 38  
**exit\_on\_big\_eps** , 551  
**exit\_on\_big\_omega** , 552  
**exit\_on\_negative\_k\_eps** , 551  
**exit\_on\_negative\_k\_omega** , 552  
**exp\_res** , 532  
**expert\_mode** , 248  
**exposant\_beta** , 332  
**expression** , 331  
**expression\_derivee\_facteur\_variable\_source** , 227  
**expression\_derivee\_force** , 226  
**expression\_p\_ana** , 93, 108  
**expression\_p\_init** , 226  
**expression\_potential\_phi** , 227  
**expression\_variable\_source\_x** , 226  
**expression\_variable\_source\_y** , 226  
**expression\_variable\_source\_z** , 226  
**expression\_vitesse\_upstream** , 226  
**expression\_vx\_ana** , 93, 108  
**expression\_vx\_init** , 226  
**expression\_vy\_ana** , 93, 108  
**expression\_vy\_init** , 226  
**expression\_vz\_ana** , 93, 108  
**expression\_vz\_init** , 226  
**facon\_init** , 221, 222  
**facsec** , 443, 446, 448, 451, 453, 455, 457, 459, 461, 463, 465, 467, 469, 471, 473, 475, 477, 479, 482, 484, 487, 490, 492, 494, 496, 498  
**facsec\_diffusion\_for\_sets** , 502, 507  
**facsec\_expert** , 489  
**facsec\_func** , 489  
**facsec\_ini** , 55  
**facsec\_max** , 55, 448, 450, 478, 481, 483, 486, 489  
**facteur** , 197, 555, 559  
**facteur\_longueur\_ideale** , 79, 309

facteur\_variable\_source\_init , 226  
 facteurs , 61  
 fichier , 38, 93, 102, 108, 234, 373, 386, 396, 433, 435, 548  
 fichier\_distance\_paroï , 251  
 fichier\_ecriture\_K\_Eps , 234  
 fichier\_matrice , 84  
 fichier\_post , 43  
 fichier\_reprise\_interface , 57, 281  
 fichier\_reprise\_vitesse , 226  
 fichier\_secmem , 84  
 fichier\_solution , 84  
 fichier\_solveur , 84  
 fichier\_solveur\_non\_recree , 338  
 fichier\_sortie , 57  
 fichier\_ssz , 435  
 field , 372, 376, 433  
 fields , 48, 94, 109  
 fields\_file , 94, 109  
 file , 74, 96, 101, 372, 376, 433  
 file\_coord\_x , 65  
 file\_coord\_y , 65  
 file\_coord\_z , 66  
 file\_coords , 315  
 file\_name , 319  
 file\_per\_comm\_group , 34  
 filename , 30  
 filling , 438  
 fin\_stat , 219  
 flow\_rate , 398  
 fluid , 402, 404  
 fluide\_diphasique , 113, 145, 190  
 Fluide\_Diphasique\_IJK , 145, 190  
 fluide\_incompressible , 113, 115–118, 120–122, 136–138, 140–142, 145, 152–157, 159, 160, 165, 166, 168, 172, 174–176, 181, 183, 184, 186, 190  
 fluide\_ostwald , 131, 142, 168, 181  
 fluide\_quasi\_compressible , 161, 164, 170, 177, 180, 185  
 fluide\_sodium\_gaz , 131, 143, 169  
 fluide\_sodium\_liquide , 131, 142, 169  
 fluide\_weakly\_compressible , 162, 171, 179  
 flush\_every\_n , 34  
 flux\_paroï , 341  
 fo , 499–501  
 fonction , 80, 243  
 fonction\_filtre , 67  
 fonction\_sous\_zone , 548  
 forage , 227  
 force , 336  
 force\_on\_two\_phase\_elem , 314, 340  
 format , 73, 94, 108  
 format\_post , 67  
 forme\_du\_terme\_source , 542  
 formulation\_a\_nb\_points , 230, 232–235, 237–244  
 formulation\_linear\_pwl , 401  
 formule\_mu , 409, 415  
 frequence\_recalc , 338  
 frontiere , 223  
 frozen\_velocity , 226  
 function\_coord\_x , 65  
 function\_coord\_y , 65  
 function\_coord\_z , 65  
 gamma , 406, 407, 421, 558  
 gas\_turb , 202, 204  
 genere\_fichier\_solveur , 84  
 ghost\_size , 68  
 ghost\_thickness , 65  
 gmres\_non\_lineaire , 537  
 gnuplot\_header , 444, 447, 449, 451, 453, 455, 457, 459, 461, 463, 465, 467, 469, 471, 473, 475, 477, 480, 483, 485, 488, 490, 493, 495, 497, 499  
 gradient\_pression\_qdm\_modifie , 291  
 gram\_schmidt , 35  
 gravite , 294, 409–424  
 group\_nb , 33  
 groupes , 146, 151, 189  
 h , 378, 531  
 half\_long\_axis , 412  
 half\_small\_axis , 412  
 harmonic\_nu\_in\_calc\_with\_indicatrice , 226  
 harmonic\_nu\_in\_diff\_operator , 226  
 haspi , 223  
 hexa\_old , 52  
 himp , 546  
 Hlsat , 317  
 Hvsat , 317  
 i , 385  
 ignore\_check\_fraction , 405  
 ijk\_splitting\_ft\_extension , 315  
 implicitation\_CH , 537  
 implicite , 313  
 impr , 69, 84, 309, 334–337, 339, 398–401, 408, 511–514, 516, 519, 520  
 impr\_diffusion\_implicite , 444, 446, 449, 451, 453, 455, 457, 459, 461, 463, 465, 467, 469, 471, 473, 475, 477, 480, 482, 485, 487, 490, 492, 494, 496, 498  
 impr\_extremums , 444, 446, 449, 451, 453, 455, 457, 459, 461, 463, 465, 467, 469, 471, 473, 475, 477, 480, 482, 485, 487, 490, 492, 494, 496, 498  
 improved\_initial\_pressure\_guess , 226  
 include\_pressure\_gradient\_in\_ustar , 226  
 inclure\_groupes\_faces\_additionnels , 38

**indic\_faces\_modifiee** , 306  
**indice** , 410–414, 416–423  
**info** , 199  
**init\_Ec** , 221, 222  
**initial\_cl\_xcoord** , 318  
**initial\_conditions** , 191, 207, 209–216, 218, 227, 228, 259–276, 278–280, 282–284, 286, 289, 291, 293, 295, 299, 300, 302–304, 311, 312, 549–552  
**initial\_field** , 380  
**initial\_value** , 379, 380, 389, 390  
**injecteur\_interfaces** , 306  
**injection** , 312  
**inout\_method** , 319  
**input\_field** , 380  
**integrale** , 527  
**interfaces** , 93, 108  
**interp\_vel** , 36  
**interpol\_indic\_pour\_dI\_dt** , 288  
**interpolation** , 540, 541  
**interpolation\_champ\_face** , 306  
**interpolation\_repere\_local** , 306  
**intervalle** , 548  
**inverse\_condition\_element** , 51  
**is\_multi\_scalar** , 413  
**is\_multi\_scalar\_diffusion** , 209, 266–268, 274  
**iter\_max** , 502, 507  
**iter\_min** , 502, 507  
**iterations\_mixed\_solver** , 69  
**joints\_non\_postraites** , 74  
**k** , 417  
**k\_min** , 246, 248–250, 252–256  
**k\_omega** , 203  
**kappa** , 410–414, 416–423, 538, 553, 555  
**kappa\_variable** , 538  
**KeOverKmin** , 388  
**kmetis** , 434  
**l\_melange** , 202  
**lambda** , 357, 358, 409–423, 533, 534, 542, 555  
**lambda\_c** , 556  
**lambda\_max** , 542  
**lambda\_min** , 542  
**lambda\_ortho** , 533  
**larg\_joint** , 71  
**last\_time** , 371, 372, 376  
**lata\_meshname** , 57, 281  
**lenghtScale** , 388  
**level** , 439–442  
**limiteur** , 204  
**Lire\_fichier** , 91  
**lissage\_courbure\_coeff** , 79, 309  
**lissage\_courbure\_iterations\_si\_remaillage** , 79, 309  
**lissage\_courbure\_iterations\_systematique** , 79, 309  
**lissage\_tcl** , 319  
**list\_equations** , 119, 120, 140, 141, 149  
**liste** , 80, 548  
**liste\_cas** , 49  
**liste\_de\_postraitements** , 92, 112, 114–117, 119–122, 124, 126–130, 132–136, 138–141, 143–145, 147, 149, 150, 152–156, 158–161, 163–167, 169–171, 173–176, 178–180, 182–185, 187, 188, 190  
**liste\_postraitements** , 92, 112, 114–116, 118–122, 124, 126–130, 132–136, 138–141, 143–145, 147, 149, 150, 152–156, 158–161, 163–167, 169–171, 173–175, 177–180, 182–185, 187, 188, 190  
**loc** , 372, 376  
**local** , 540  
**localisation** , 68, 326, 331  
**loi\_etat** , 418, 422  
**longitudinal\_axis** , 27  
**longueur\_boite** , 221, 222  
**longueur\_maille** , 230, 232–235, 237–243, 245  
**longueurs** , 61  
**ls** , 318  
**Lvap** , 317  
**maillage** , 38, 305  
**maillage\_ft\_ijk** , 282  
**main** , 73  
**maintien\_temperature** , 276  
**Mass\_and\_stiffness\_file\_name** , 27  
**mass\_source** , 285  
**masse\_molaire** , 48, 209, 266–268, 274  
**Masse\_Multiphase** , 124, 125, 127  
**matrice\_pression\_invariante** , 288  
**matrice\_pression\_penalisee\_H1** , 290  
**max\_iter\_implicite** , 445, 479, 481, 484, 486, 489, 491  
**max\_simu\_time** , 499–501  
**mesh** , 371, 372, 376  
**methode** , 57, 325, 326, 328, 330  
**methode\_calcul\_face\_keps\_impose** , 553  
**methode\_calcul\_pression\_initiale** , 218, 228, 258, 260, 284, 286, 289, 291, 293, 295, 298, 300, 302  
**methode\_couplage** , 313  
**methode\_interpolation\_v** , 305  
**methode\_transport** , 305, 313  
**milieu** , 92, 112, 114–117, 119–122, 124, 125, 127–130, 132–137, 139–141, 143–145, 147, 149, 150, 152–156, 158–161, 163–167, 169–172, 174–176, 178–181, 183–186, 188, 190  
**milieu\_composite** , 123, 125, 127  
**Milieu\_MUSIG** , 123, 125, 127  
**min\_critere\_q\_sur\_max\_critere\_q** , 224  
**min\_dir\_flow** , 378

min\_dir\_wall , 378  
 mobile\_probes , 94, 108  
 mobile\_probes\_file , 94, 108  
 Modal\_deformation\_file\_name , 27  
 mode , 30  
 mode\_calcul\_convection , 263, 265  
 model , 402, 404  
 model\_variant , 247  
 modele , 540, 541  
 modele\_cinetique , 208  
 modele\_fonc\_bas\_reynolds , 249, 253  
 modele\_fonc\_realisable , 255, 256  
 modele\_micro\_melange , 331  
 modele\_turbulence , 209, 210, 228, 265, 268, 273, 278, 279, 288, 292, 299, 301  
 modele\_visco , 555, 559  
 models , 124, 125, 127  
 modif\_div\_face\_dirichlet , 369  
 molar\_mass , 406  
 molar\_mass1 , 403  
 molar\_mass2 , 403  
 moyenne , 388  
 moyenne\_convergee , 327  
 moyenne\_de\_kappa , 537  
 moyenne\_imposee , 396  
 moyenne\_recyclee , 396  
 mpoint\_inactif\_sur\_qdm , 288  
 mpoint\_vapeur\_inactif\_sur\_qdm , 288  
 mu , 48, 357, 358, 406, 410–412, 416–418, 421, 423, 555  
 mu1 , 403  
 mu2 , 403  
 mu\_1 , 274, 297  
 mu\_2 , 274, 297  
 mu\_fonc\_c , 297  
 multigrid\_solver , 225  
 multiplicateur\_de\_kappa , 537  
 n , 358, 417, 555, 558, 559  
 n\_extend\_meso , 318  
 n\_iterations\_distance , 305  
 n\_iterations\_interpolation\_ibc , 306  
 name\_of\_initial\_zones , 37  
 name\_of\_new\_zones , 37  
 name\_ssz , 435  
 nature , 371  
 Navier\_Stokes\_Aposteriori , 154  
 navier\_stokes\_ibm , 160, 181  
 navier\_stokes\_ibm\_turbulent , 117, 138  
 navier\_stokes\_phase\_field , 166  
 navier\_stokes\_QC , 133, 161, 170, 177  
 navier\_stokes\_standard , 115, 118, 129, 132, 136, 140, 143, 152, 155, 156, 169, 172, 174, 183  
 navier\_stokes\_standard\_ALE , 153  
 Navier\_Stokes\_standard\_sensibility , 122, 142  
 navier\_stokes\_turbulent , 116, 120, 130, 134, 137, 141, 157, 159, 165, 175, 176, 184, 186  
 Navier\_Stokes\_Turbulent\_ALE , 121  
 navier\_stokes\_turbulent\_qc , 135, 164, 180, 185  
 navier\_stokes\_WC , 162, 171, 179  
 nb\_comp , 379, 380, 389, 390, 559  
 nb\_corrections\_max , 502–508, 510  
 nb\_diam\_ortho\_shear\_perio , 226  
 nb\_diam\_upstream , 226  
 nb\_full\_mg\_steps , 69  
 nb\_histo\_boxes\_impr , 398–401  
 nb\_it\_max , 335–337, 339, 502–510, 519  
 nb\_ite\_sans\_accel\_max , 55  
 nb\_iter\_barycentrage , 79, 308  
 nb\_iter\_correction\_volume , 79, 309  
 nb\_iter\_remaillage , 79, 308  
 nb\_iteration\_max\_uzawa , 306  
 nb\_iterations , 313  
 nb\_iterations\_correction\_volume , 306  
 nb\_iterations\_gmresnl , 537  
 nb\_lissage\_correction\_volume , 306  
 nb\_mailles\_mini , 224  
 nb\_modes , 27  
 nb\_nodes , 65  
 nb\_parts , 432–436  
 nb\_parts\_geom , 45  
 nb\_parts\_naif , 45  
 nb\_parts\_tot , 72  
 nb\_pas\_dt\_max , 444, 446, 449, 451, 453, 455, 457, 459, 461, 463, 465, 467, 469, 471, 473, 475, 477, 480, 482, 485, 488, 490, 492, 494, 496, 498–501  
 nb\_pas\_dt\_post , 94, 108  
 nb\_pas\_dt\_post\_stats\_bulles , 93, 108  
 nb\_pas\_dt\_post\_stats\_plans , 93, 108  
 nb\_planes , 27  
 nb\_points , 242, 429  
 nb\_points\_par\_phase , 220  
 nb\_procs , 49  
 nb\_sauv\_max , 443, 446, 448, 450, 452, 454, 456, 458, 460, 462, 464, 466, 468, 470, 472, 474, 476, 479, 482, 484, 487, 489, 492, 494, 496, 498  
 nb\_test , 84  
 nb\_tranche , 57  
 nb\_tranches , 52–54  
 nb\_var , 243  
 nbelem , 315  
 nbelem\_i , 370  
 nbelem\_j , 370  
 nbelem\_k , 370  
 nbModes , 388  
 new\_ijk\_splitting , 39, 40

new\_jacobian , 199  
 new\_mass\_source , 288  
 NewmarkTimeScheme , 27  
 ng2 , 203  
 niter\_avg , 448, 450  
 niter\_max , 448, 450  
 niter\_max\_diffusion\_implicite , 206, 444, 446, 449, 451, 453, 455, 457, 459, 461, 463, 465, 467, 469, 471, 473, 475, 477, 480, 482, 485, 488, 490, 492, 494, 496, 498  
 niter\_min , 448, 450  
 nmax , 40  
 no\_alpha , 202  
 no\_check\_disk\_space , 444, 447, 449, 451, 453, 455, 457, 459, 461, 463, 465, 467, 469, 471, 473, 475, 477, 480, 483, 485, 488, 490, 493, 495, 497, 499  
 no\_conv\_subiteration\_diffusion\_implicite , 444, 446, 449, 451, 453, 455, 457, 459, 461, 463, 465, 467, 469, 471, 473, 475, 477, 480, 482, 485, 487, 490, 492, 494, 496, 498  
 no\_error\_if\_not\_converged\_diffusion\_implicite , 444, 446, 449, 451, 453, 455, 457, 459, 461, 463, 465, 467, 469, 471, 473, 475, 477, 480, 482, 485, 487, 490, 492, 494, 496, 498  
 no\_octree\_method , 58  
 no\_qdm , 502–510  
 nom , 379, 380, 389, 390  
 nom\_bord , 52, 53  
 nom\_champ , 371  
 nom\_cl\_derriere , 54  
 nom\_cl\_devant , 54  
 nom\_domaine , 67  
 nom\_fichier , 546  
 nom\_fichier\_post , 67  
 nom\_fichier\_solveur , 338  
 nom\_fichier\_sortie , 45  
 nom\_frontiere , 325  
 nom\_inconnue , 209, 266–268, 274  
 nom\_mon\_indicatrice , 345  
 nom\_pb , 67  
 nom\_reprise , 39, 40, 145, 190  
 nom\_sauvegarde , 39, 40, 145, 190  
 nom\_source , 320–331  
 nom\_zones , 71  
 nombre\_de\_noeuds , 61  
 nombre\_facettes\_retenues\_par\_cellule , 306  
 noms\_champs , 67  
 norm , 81  
 normal\_value , 388  
 normalise , 224  
 nproc , 315  
 nu , 199, 357, 358  
 nu\_transp , 199  
 numero , 327, 331  
 numero\_masse , 322  
 numero\_op , 322  
 numero\_source , 322  
 nusselt , 558  
 nut , 199  
 nut\_max , 229, 231–233, 235–239, 241–250, 252, 254–257  
 nut\_transp , 199  
 oh , 499–501  
 old , 196  
 old\_ijk\_splitting , 39, 40  
 old\_ijk\_splitting\_ft\_extension , 39  
 omega , 378, 438, 440, 442, 448, 529, 531  
 omega\_max , 254  
 omega\_min , 254  
 omega\_relaxation\_drho\_dt , 418  
 ONEFLUID , 282  
 optimisation\_sous\_maillage , 326  
 optimized , 336, 339  
 option , 208, 267, 327, 529  
 order , 34  
 ordering , 439  
 origin , 315  
 origin\_i , 370  
 origin\_j , 370  
 origin\_k , 370  
 Origine , 61  
 origine , 51  
 OutletCorrection\_pour\_dI\_dt , 288  
 Output\_position\_1D , 28  
 Output\_position\_3D , 28  
 p0 , 369  
 p1 , 369  
 p\_imposee\_aux\_faces , 70  
 P\_ref , 317, 420  
 p\_ref , 316, 317  
 P\_sat , 317  
 p\_seuil\_max , 226  
 p\_seuil\_min , 226  
 pa , 369  
 par\_sous\_dom , 29  
 parallel\_over\_zone , 34  
 parallele , 94, 108  
 parametre\_equation , 191, 207, 209–218, 227, 229, 259–275, 277–284, 286, 289, 291, 293, 295, 299, 300, 302, 303, 307, 312, 313, 549–552  
 parcours\_interface , 281, 305  
 parcours\_sans\_tolerance , 309  
 Partition\_tool , 71  
 pas , 308

pas\_de\_solution\_initiale , 84  
 pas\_lissage , 308  
 pas\_remaillage , 79  
 pb\_champ , 328, 329  
 pb\_champ\_evaluateur , 395  
 pb\_dist , 371  
 pb\_loc , 371  
 pb\_name , 73  
 pcshell , 519  
 penalisation\_forage , 288  
 penalisation\_l2\_ftd , 211, 275–277  
 perio , 315  
 perio\_i , 370  
 perio\_j , 370  
 perio\_k , 370  
 perio\_x , 65  
 perio\_y , 65  
 perio\_z , 65  
 periode , 220  
 periode\_calc\_spectre , 221, 222  
 periode\_sauvegarde\_securite\_en\_heures , 444, 447, 449, 451, 453, 455, 457, 459, 461, 463, 465, 467, 469, 471, 473, 475, 477, 480, 482, 485, 488, 490, 493, 495, 497, 498  
 periodique , 72  
 petsc\_decide , 518  
 phase , 208, 267, 276, 345  
 phase0 , 409, 415  
 phase1 , 409, 415  
 phase\_marquee , 313  
 PID\_controler\_on\_targer\_power , 546  
 pinf , 421  
 point1 , 51  
 point2 , 51  
 point3 , 51  
 points\_fluides , 399–401  
 points\_solides , 398–401  
 polynomes , 548  
 polynomial\_chaos , 211, 258  
 porosites , 409–424  
 porosites\_champ , 409–424  
 portee\_force\_repulsion , 282  
 position , 311, 408  
 post\_process\_hydrodynamic\_forces , 306  
 Post\_processing , 92, 112, 114–117, 119–122, 124, 126–130, 132–137, 139–141, 143–145, 147, 149, 150, 152–156, 158–161, 163–167, 169–172, 174–176, 178–181, 183–186, 188, 190  
 Post\_processings , 92, 112, 114–117, 119–122, 124, 126–130, 132–137, 139–141, 143–145, 147, 149, 150, 152–156, 158–161, 163–167, 169–172, 174–176, 178–180, 182–185, 187, 188, 190  
 postraiter\_gradient\_pression\_sans\_masse , 218, 228, 258, 260, 284, 286, 289, 291, 293, 295, 298, 300, 302  
 potentiel\_chimique , 538  
 potentiel\_chimique\_generalise , 274  
 Pr\_t , 202  
 prandtl\_turbulent\_fonction\_nu\_t\_alpha , 428  
 Prandtl , 406, 407  
 prandtl , 405–407, 559  
 prandtl\_turbulent , 203  
 prdt , 427  
 prdt\_sur\_kappa , 557  
 pre\_smooth\_steps , 68  
 precision\_impr , 444, 446, 449, 451, 453, 455, 457, 459, 461, 463, 465, 467, 469, 471, 473, 475, 477, 480, 482, 485, 488, 490, 493, 495, 496, 498  
 precondition , 335, 336, 339, 511, 514, 518, 519  
 precondition0 , 439  
 precondition1 , 439  
 precondition\_diagonal , 336, 339  
 precondition\_nul , 336, 339, 518  
 preconda , 439  
 preconditionnement\_diag , 206  
 prescribed\_mpoint , 276  
 pression , 418  
 pression\_degeneree , 502  
 pression\_reference , 290  
 pression\_thermo , 423  
 pression\_xyz , 423  
 pressure\_order , 34  
 pressure\_reduction , 502  
 print\_more\_infos , 72  
 probes , 94, 108  
 probes\_file , 94, 108  
 probleme , 31, 50–52, 296, 297, 379, 380, 389, 390  
 produits , 332  
 projection\_initiale , 218, 228, 258, 260, 284, 286, 289, 291, 293, 295, 298, 300, 302  
 projection\_normale\_bord , 53  
 proprietes\_particules , 314  
 pulsation\_w , 220  
 q , 421  
 q\_prim , 422  
 QDM\_Multiphase , 124, 125, 127  
 qtcl , 318  
 quiet , 246, 248–250, 252–256, 334–339, 511–514, 516, 519, 520  
 radius , 411  
 rapport\_residus , 55  
 ratioCutoffWavenumber , 388  
 rayon\_spot , 527  
 rc\_inject , 318  
 rc\_tcl\_gridn , 318

reactifs , 332  
 reactions , 331  
 read\_matrix , 518  
 rectangle , 547  
 reduce\_ram , 516  
 refuse\_patch\_conservation\_qdm\_rk3\_source\_interfacs\_passer\_par\_le2d , 52  
 , 226  
 regul , 536  
 reinjection\_tcl , 318  
 relative , 81  
 relax\_barycentrage , 79, 308  
 relax\_jacobi , 68  
 relax\_pression , 508, 510  
 remaillage , 305  
 remaillage\_ft\_ijk , 58, 281  
 renommer\_equation , 191, 208–218, 227, 229,  
 259–267, 269–275, 277–283, 285, 286, 289,  
 292, 293, 295, 299, 300, 302, 303, 307,  
 312, 313, 550–552  
 reorder , 72  
 reorder\_matrix , 518  
 reprise , 92, 112, 114, 115, 117–121, 123, 124, 126,  
 127, 129–136, 138–140, 142–144, 146, 147,  
 149, 150, 152–155, 157–160, 162–165, 167–  
 170, 172–175, 177–179, 181–185, 187, 189,  
 190, 220  
 reprise\_correlation , 357, 358  
 reprise\_liq\_velocity\_tmoy , 226  
 reprise\_vap\_velocity\_tmoy , 226  
 residu\_max\_gmresnl , 537  
 residu\_min\_gmresnl , 537  
 residuals , 444, 446, 448, 451, 453, 455, 457, 459,  
 461, 463, 465, 467, 469, 471, 473, 475,  
 477, 479, 482, 485, 487, 490, 492, 494,  
 496, 498  
 resolution\_explicite , 207  
 resolution\_fluctuations , 226  
 resolution\_monolithique , 489  
 restart , 555  
 Restart\_file\_name , 28  
 restriction , 547  
 resume\_last\_time , 92, 113, 114, 116–120, 122–  
 124, 126, 128–135, 137–140, 142–144, 146,  
 147, 149, 151–154, 156–160, 162–165, 167–  
 170, 172–175, 177–179, 181–184, 186, 187,  
 189, 190  
 reuse\_preconditioner\_nb\_it\_max , 518, 519  
 reynolds\_stress\_isotrope , 251  
 rho , 357, 358, 409–423  
 rho\_1 , 274, 296  
 rho\_2 , 274, 296  
 Rho\_beam , 28  
 rho\_constant\_pour\_debug , 406  
 rho\_fonc\_c , 296  
 rho\_t , 407  
 rho\_xyz , 407  
 rotation , 408, 540, 541  
 rt , 369  
 rtol , 511–514, 516, 518–520  
 sans\_solveur\_masse , 322  
 sans\_source\_boussinesq , 555  
 sato , 203  
 sauvegarde , 92, 112, 114–116, 118–122, 124, 126–  
 130, 132–136, 138–141, 143–145, 147, 149,  
 150, 152–155, 157–161, 163–165, 167–  
 171, 173–175, 177–180, 182–185, 187, 188,  
 190  
 sauvegarde\_simple , 92, 112, 114, 115, 117–121,  
 123, 124, 126–128, 130–136, 138–141, 143,  
 144, 146, 147, 149, 150, 152–155, 157–  
 160, 162–165, 167–171, 173–175, 177–  
 179, 181–185, 187, 188, 190  
 sauvegarder\_xyz , 145, 190  
 save\_matrix , 335–339, 516, 518  
 save\_matrix\_mtx\_format , 511–514, 516, 519, 520  
 save\_matrix\_petsc\_format , 516, 519  
 sc , 405  
 schema\_ch , 493  
 schema\_ns , 494  
 scturb , 428  
 segment , 547  
 senseur\_interface , 527  
 serial\_statistics , 94, 109  
 serial\_statistics\_file , 94, 109  
 seuil , 69, 335–337, 339, 448, 450, 510–514, 516,  
 519, 520  
 seuil\_absolu , 30, 31  
 seuil\_convergence\_implicit , 207, 502–505, 507,  
 508, 510  
 seuil\_convergence\_solveur , 207, 502–510  
 seuil\_convergence\_uzawa , 306  
 seuil\_cv\_iterations\_ptfixe , 537  
 seuil\_diffusion\_implicit , 206, 444, 446, 449, 451,  
 453, 455, 457, 459, 461, 463, 465, 467,  
 469, 471, 473, 475, 477, 480, 482, 485,  
 487, 490, 492, 494, 496, 498  
 seuil\_divU , 217, 228, 258, 260, 284, 285, 288,  
 291, 292, 294, 298, 300, 301  
 seuil\_dvolume\_residuel , 79, 309  
 seuil\_generation\_solveur , 502–510  
 seuil\_minimum\_relatif , 30, 31  
 seuil\_relatif , 30, 31  
 seuil\_residu\_gmresnl , 537  
 seuil\_residu\_ptfixe , 537  
 seuil\_statio , 444, 446, 448, 451, 453, 455, 457,  
 459, 461, 463, 465, 467, 469, 471, 473,



475, 477, 479, 482, 484, 487, 490, 492,  
 494, 496, 498  
 seuil\_test\_preliminaire\_solveur , 502–510  
 seuil\_verification , 84  
 seuil\_verification\_solveur , 502–510  
 sharing\_algo , 35  
 shift\_secmem2 , 288  
 sigma , 204, 409, 415  
 sigma\_d , 523  
 sigma\_eps , 250, 252, 253, 255, 256  
 sigma\_k , 247, 250, 252, 253, 255, 256  
 sigma\_k1 , 247  
 sigma\_k2 , 247  
 sigma\_omega , 247  
 sigma\_omega1 , 248  
 sigma\_omega2 , 248  
 sigma\_turbulent , 202  
 single\_hdf , 37, 72  
 single\_precision , 34  
 single\_safe\_file , 34  
 size\_dom , 315  
 sm , 318  
 smooth\_steps , 69  
 solide , 92, 112  
 solv\_elem , 336  
 solved\_equations , 113, 145, 190  
 solver\_precision , 69  
 solveur , 84, 148, 206, 207, 445, 479, 481, 484,  
 486, 489, 491, 502–510  
 solveur0 , 335  
 solveur1 , 335  
 solveur\_bar , 218, 228, 258, 260, 284, 286, 289,  
 291, 293, 294, 298, 300, 302  
 solveur\_grossier , 69  
 solveur\_pression , 217, 228, 258, 259, 261, 284,  
 285, 288, 291, 292, 294, 298, 299, 301  
 sonde\_tble , 555, 559  
 source , 320–331  
 source\_reference , 320–331  
 sources , 191, 207, 209–216, 218, 227, 228, 259–  
 276, 278–280, 282–284, 286, 289, 291,  
 293, 295, 299, 300, 302, 303, 306, 311,  
 313, 320–331, 549–552  
 sources\_reference , 320–331  
 sous\_zone , 50, 94, 108, 379, 380, 389, 390, 533,  
 534  
 sous\_zones , 435  
 species\_number , 405  
 spectre\_1D , 221, 222  
 spectre\_3D , 221, 222  
 splitting , 65  
 stabilise , 242, 429  
 standard , 199  
 state , 258  
 stationnaire , 555  
 statistics , 94, 109  
 statistics\_file , 94, 109  
 stats , 555, 558, 559  
 steady\_global\_dt , 445  
 steady\_security\_facteur , 445  
 stencil\_width , 276  
 suffix\_for\_reset , 94, 109  
 suppression\_rejetons , 226  
 surface , 358, 536  
 surface\_tension , 316, 317  
 surfacic\_flux , 66  
 surfacique , 437  
 sutherland , 417, 422  
 symx , 61  
 symy , 61  
 symz , 61  
 systeme\_naire , 537  
 t0 , 530  
 t\_deb , 223, 322–324, 327  
 t\_debut\_injection , 314  
 t\_debut\_statistiques , 93, 108  
 t\_fin , 223, 322–324, 327  
 t\_min , 407  
 T\_ref , 317, 420, 421  
 t\_ref , 316, 317  
 T\_sat , 317  
 table\_temps , 371  
 table\_temps\_lue , 371  
 Taux\_dissipation\_turbulent , 124, 125, 127  
 tcpumax , 443, 446, 448, 450, 452, 454, 456, 458,  
 460, 462, 464, 466, 468, 470, 472, 474,  
 476, 479, 481, 484, 487, 489, 492, 494,  
 496, 497, 499–501  
 tdivu , 196  
 temperature , 403  
 temperature\_order , 35  
 temperature\_paroie , 341  
 temperature\_state , 211  
 temps\_d\_affichage , 537  
 temps\_debut\_prise\_en\_compte\_drho\_dt , 418  
 temps\_relaxation\_coefficient\_pdf , 540  
 terme\_force\_init , 226  
 terme\_gravite , 58, 282, 288  
 test , 196  
 test\_etapes\_et\_bilan , 226  
 Text , 546  
 theta\_app , 318  
 thetac\_tcl , 318  
 thi , 235  
 thickness , 311  
 time , 372, 376  
 time\_activate\_ptot , 423  
 timeScale , 388



timestep , 499–501  
 timestep\_facsec , 499–501  
 timestep\_reprise\_interface , 57, 281  
 timestep\_reprise\_vitesse , 226  
 tinf , 357  
 tinit , 443, 445, 448, 450, 452, 454, 456, 458, 460, 462, 464, 466, 468, 470, 472, 474, 476, 479, 481, 484, 487, 489, 492, 494, 495, 497, 499–501  
 tmax , 443, 446, 448, 450, 452, 454, 456, 458, 460, 462, 464, 466, 468, 470, 472, 474, 476, 479, 481, 484, 487, 489, 492, 494, 496, 497  
 toutes\_les\_options , 70  
 traitement\_axi , 36  
 traitement\_coins , 70  
 traitement\_gradients , 70  
 traitement\_particulier , 217, 228, 258, 259, 284, 285, 288, 291, 292, 294, 298, 300, 301  
 traitement\_pth , 418, 422  
 traitement\_rho\_gravite , 418  
 tranches , 436  
 transformation\_bulles , 312  
 transport\_equation , 246, 248–250, 252–256  
 transpose\_rotation , 540, 541  
 triangle , 51  
 Triple\_Line\_Model\_FT\_Disc , 114, 145, 190  
 trois\_tetra , 52  
 tstep\_init , 499–501  
 tsup , 357, 358  
 tube , 548  
 turbDissRate , 388  
 turbKinEn , 388  
 turbulence\_paroι , 229, 230, 232, 233, 235–241, 243–246, 248–250, 252–256, 426, 428, 429  
 tuyauz , 234  
 type , 327, 438  
 type\_indic\_faces , 306  
 type\_vitesse\_imposee , 306  
 u , 383, 385, 389  
 u\_etoile , 531  
 u\_star\_impose , 553  
 u\_tau , 556  
 ubar\_umprim\_cible , 542  
 ucent , 378  
 uncertain\_variable , 211, 258  
 uniform\_domain\_size\_i , 370  
 uniform\_domain\_size\_j , 370  
 uniform\_domain\_size\_k , 370  
 union , 548  
 unite , 327, 331  
 upstream\_dir , 226  
 upstream\_stencil , 226  
 use\_existing\_domain , 371, 372, 376  
 use\_grad\_pression\_eos , 423  
 use\_hydrostatic\_pressure , 423  
 use\_inv\_rho\_for\_mass\_solver\_and\_calculer\_rho\_v , 226  
 use\_inv\_rho\_in\_poisson\_solver , 226  
 use\_links , 34  
 use\_osqp , 36  
 use\_overlapdec , 371  
 use\_total\_pressure , 423  
 use\_tryggvason\_interfacial\_source , 58, 282  
 use\_tstep\_init , 499–501  
 use\_weights , 434  
 user\_field , 423  
 val\_Ec , 221, 222  
 variable\_imposee\_data , 540  
 variable\_imposee\_fonction , 540  
 vdf\_mesh , 36  
 velocity\_convection\_op , 226  
 velocity\_diffusion\_op , 226  
 velocity\_order , 34  
 velocity\_profil , 398  
 velocity\_reset , 226  
 velocity\_state , 211  
 verif\_boussinesq , 530  
 verification\_derivee , 408  
 via\_extraire\_surface , 51  
 vingt\_tetra , 52  
 viscosite\_dynamique\_constante , 294  
 vitesse , 408, 529  
 vitesse\_entree , 226  
 vitesse\_fluide\_explicite , 310  
 vitesse\_imposee\_data , 540  
 vitesse\_imposee\_fonction , 540  
 vitesse\_imposee\_regularisee , 306  
 vitesse\_upstream , 226  
 voflike\_correction\_volume , 306  
 vol\_bulle\_monodisperse , 226  
 vol\_bulles , 226  
 volume , 357  
 volume\_impose\_phase\_1 , 305  
 volumes\_etendus , 196  
 volumes\_non\_etendus , 196  
 volumique , 437  
 with\_nu , 551, 552  
 without\_dec , 35  
 writing\_processes , 36  
 xinf , 358  
 xm , 318  
 xsup , 358  
 xtanh , 62  
 xtanh\_dilatation , 62  
 xtanh\_taille\_premiere\_maille , 62  
 yaml\_fname , 112  
 ylim , 318

ym , 318  
 ymeso , 318  
 Young\_Module , 28  
 ytanh , 62  
 ytanh\_dilatation , 62  
 ytanh\_taille\_premiere\_maille , 62  
 zmax , 57  
 zmin , 57  
 ztanh , 62  
 ztanh\_dilatation , 62  
 ztanh\_taille\_premiere\_maille , 62  
  
 Acceleration, 528  
 Ai\_based, 311  
 Ale, 197  
 Ale\_neumann\_bc\_for\_grid\_problem, 25  
 Algo\_base, 319  
 Algo\_couple\_1, 319  
 Amg, 333  
 Amgx, 333  
 Amont, 192  
 Amont\_old, 194  
 Analyse\_angle, 40  
 Associate, 40  
 Associer\_algo, 41  
 Associer\_pbm\_g\_pbf, 41  
 Associer\_pbm\_g\_pbglobal, 41  
 Axi, 41  
  
 Base, 310  
 Beam\_model, 26  
 Bicgstab, 511  
 Bidim\_axi, 42  
 Binaire, 110  
 Binaire\_gaz\_parfait\_qc, 402  
 Binaire\_gaz\_parfait\_wc, 403  
 Block\_jacobi\_icc, 439  
 Block\_jacobi\_ilu, 440  
 Boomeramg, 440  
 Bord, 62  
 Bord\_base, 62  
 Boundary\_field\_inward, 388  
 Boundary\_field\_keps\_from\_ud, 385  
 Boundary\_field\_uniform\_keps\_from\_ud, 388  
 Boussinesq\_concentration, 529  
 Boussinesq\_temperature, 530  
 Brech, 222  
 Btd, 197  
  
 C-amg, 440  
 Calcul, 42  
 Calculer\_moments, 42  
 Canal, 219  
 Canal\_perio, 530  
  
 Ceg, 222  
 Centre, 192  
 Centre4, 192  
 Centre\_de\_gravite, 42  
 Centre\_old, 194  
 Ch\_front\_input, 389  
 Ch\_front\_input\_ale, 385  
 Ch\_front\_input\_uniforme, 389  
 Champ\_base, 370  
 Champ\_composite, 373  
 Champ\_don\_base, 374  
 Champ\_don\_lu, 374  
 Champ\_fonc\_fonction, 374  
 Champ\_fonc\_fonction\_txyz, 375  
 Champ\_fonc\_fonction\_txyz\_morceaux, 375  
 Champ\_fonc\_interp, 370  
 Champ\_fonc\_med, 375  
 Champ\_fonc\_med\_table\_temps, 371  
 Champ\_fonc\_med\_tabule, 372  
 Champ\_fonc\_reprise, 376  
 Champ\_fonc\_t, 377  
 Champ\_fonc\_tabule, 377  
 Champ\_fonc\_tabule\_morceaux\_interp, 373  
 Champ\_fonc\_txyz, 382  
 Champ\_fonc\_xyz, 383  
 Champ\_front\_ale, 386  
 Champ\_front\_ale\_beam, 386  
 Champ\_front\_base, 384  
 Champ\_front\_bruite, 390  
 Champ\_front\_calc, 390  
 Champ\_front\_composite, 391  
 Champ\_front\_contact\_rayo\_semi\_transp\_vef, 391  
 Champ\_front\_contact\_rayo\_transp\_vef, 391  
 Champ\_front\_contact\_vef, 392  
 Champ\_front\_debit, 392  
 Champ\_front\_debit\_massique, 392  
 Champ\_front\_debit\_qc\_vdf, 386  
 Champ\_front\_debit\_qc\_vdf\_fonc\_t, 387  
 Champ\_front\_fonc\_pois\_ipsn, 392  
 Champ\_front\_fonc\_pois\_tube, 393  
 Champ\_front\_fonc\_t, 393  
 Champ\_front\_fonc\_txyz, 393  
 Champ\_front\_fonc\_xyz, 393  
 Champ\_front\_fonction, 394  
 Champ\_front\_lu, 394  
 Champ\_front\_med, 390  
 Champ\_front\_musig, 394  
 Champ\_front\_normal\_vef, 394  
 Champ\_front\_parametrique, 386  
 Champ\_front\_pression\_from\_u, 395  
 Champ\_front\_recyclage, 395  
 Champ\_front\_synt, 387  
 Champ\_front\_tabule, 396  
 Champ\_front\_tabule\_lu, 396

Champ\_front\_tangentiel\_vef, 396  
 Champ\_front\_uniforme, 397  
 Champ\_front\_vortex, 397  
 Champ\_front\_xyz\_debit, 397  
 Champ\_front\_xyz\_tabule, 385  
 Champ\_generique\_base, 320  
 Champ\_init\_canal\_sinal, 377  
 Champ\_input\_base, 378  
 Champ\_input\_p0, 379  
 Champ\_input\_p0\_composite, 379  
 Champ\_musig, 380  
 Champ\_ostwald, 380  
 Champ\_parametrique, 373  
 Champ\_post\_de\_champs\_post, 320  
 Champ\_post\_extraction, 324  
 Champ\_post\_morceau\_equation, 326  
 Champ\_post\_operateur\_base, 321  
 Champ\_post\_operateur\_divergence, 323  
 Champ\_post\_operateur\_eqn, 321  
 Champ\_post\_operateur\_gradient, 325  
 Champ\_post\_reduction\_0d, 328  
 Champ\_post\_refchamp, 329  
 Champ\_post\_statistiques\_base, 322  
 Champ\_post\_tparoi\_vef, 329  
 Champ\_post\_transformation, 330  
 Champ\_som\_lu\_vdf, 380  
 Champ\_som\_lu\_vef, 381  
 Champ\_tabule\_lu, 381  
 Champ\_tabule\_morceaux, 372  
 Champ\_tabule\_temps, 381  
 Champ\_uniforme\_morceaux, 382  
 Champ\_uniforme\_morceaux\_tabule\_temps, 382  
 Champ\_front\_fonc\_txyz, 22  
 Chimie, 331  
 Chmoy\_faceperio, 222  
 Cholesky, 334, 514  
 Cholesky\_mumps\_blr, 516  
 Cholesky\_out\_of\_core, 511  
 Cholesky\_pastix, 512  
 Cholesky\_superlu, 512  
 Cholesky\_umfpack, 513  
 Circle, 100  
 Circle\_3, 100  
 Class\_generic, 332  
 Cli, 516  
 Cli\_quiet, 517  
 Coalescence\_bulles\_1groupe, 521  
 Coalescence\_bulles\_2groupes, 521  
 Collision\_model\_fpi\_deriv, 339  
 Collision\_model\_ft\_base, 314  
 Collision\_model\_ft\_sphere, 339  
 Combinaison, 243  
 Cond\_lim\_k\_complique\_transition\_flux\_nul\_demi, 341  
 Cond\_lim\_k\_simple\_flux\_nul, 341  
 Cond\_lim\_omega\_demi, 341  
 Cond\_lim\_omega\_dix, 341  
 Condinits, 205  
 Condlim\_base, 340  
 Condlims, 148  
 Conduction, 191  
 Conduction\_ibm, 207  
 Connexion\_approchee, 430  
 Connexion\_exacte, 430  
 Constant, 363  
 Constituant, 412  
 Contact\_vdf\_vef, 344  
 Contact\_vef\_vdf, 344  
 Convection\_deriv, 192  
 Convection\_diffusion\_chaleur\_qc, 262  
 Convection\_diffusion\_chaleur\_turbulent\_qc, 264  
 Convection\_diffusion\_chaleur\_wc, 263  
 Convection\_diffusion\_concentration, 265  
 Convection\_diffusion\_concentration\_ft\_disc, 266  
 Convection\_diffusion\_concentration\_turbulent, 267  
 Convection\_diffusion\_concentration\_turbulent\_ft\_disc, 208  
 Convection\_diffusion\_espece\_binaire\_qc, 269  
 Convection\_diffusion\_espece\_binaire\_turbulent\_qc, 209  
 Convection\_diffusion\_espece\_binaire\_wc, 269  
 Convection\_diffusion\_espece\_multi\_qc, 270  
 Convection\_diffusion\_espece\_multi\_turbulent\_qc, 272  
 Convection\_diffusion\_espece\_multi\_wc, 271  
 Convection\_diffusion\_phase\_field, 273  
 Convection\_diffusion\_temperature, 274  
 Convection\_diffusion\_temperature\_ft\_disc, 275  
 Convection\_diffusion\_temperature\_ibm, 277  
 Convection\_diffusion\_temperature\_ibm\_turbulent, 278  
 Convection\_diffusion\_temperature\_sensibility, 210  
 Convection\_diffusion\_temperature\_turbulent, 279  
 Coolprop\_qc, 403  
 Coolprop\_wc, 404  
 Coriolis, 531  
 Correction\_antal, 521  
 Correction\_lubchenko, 521  
 Correction\_tomiyama, 522  
 Correlation, 102, 104, 323  
 Corriger\_frontiere\_periodique, 43  
 Create\_domain\_from\_sub\_domain, 29  
 Cudss, 334  
 Darcy, 531  
 Debog, 43  
 Debogft, 30  
 Debogijk, 30  
 Decoupebord\_pour\_rayonnement, 44  
 Decouper\_bord\_coincident, 45  
 Dg, 367

Di\_l2, 194  
 Diag, 440  
 Diffusion\_croisee\_echelle\_temp\_taux\_diss\_turb, 523  
 Diffusion\_deriv, 198  
 Diffusion\_supplementaire\_echelle\_temp\_turb, 523  
 Dilate, 45  
 Dimension, 45  
 Dirac, 531  
 Dirichlet, 344  
 Disable\_tu, 46  
 Discretisation\_base, 367  
 Discretiser\_domaine, 46  
 Discretize, 46  
 Dispersion\_bulles, 524  
 Dissipation\_echelle\_temp\_taux\_diss\_turb, 524  
 Distance\_paro, 46  
 Dns\_qc\_double, 29  
 Domain, 64  
 Domaine, 369  
 Domaine\_ale, 370  
 Domaine\_base, 314  
 Domaine\_ijk, 314  
 Domaineaxild, 369  
 Dp, 522  
 Dp\_impose, 522  
 Dp\_regul, 523  
 Dt\_calc, 334  
 Dt\_fixe, 334  
 Dt\_min, 335  
 Dt\_start, 335  
 Dt\_post, 102, 104  
  
 Easm\_baglietto, 251  
 Ec, 220  
 Ecart\_type, 103, 324  
 Ecart\_type, 102, 104  
 Echange\_contact\_rayo\_transp\_vdf, 344  
 Echange\_contact\_vdf\_ft\_disc, 345  
 Echange\_contact\_vdf\_ft\_disc\_solid, 345  
 Echange\_couplage\_thermique, 341  
 Echelle\_temporelle\_turbulente, 211  
 Ecrire, 91  
 Ecrire\_champ\_med, 47  
 Ecrire\_fichier\_bin, 91  
 Ecrire\_fichier\_formatte, 47  
 Ecriturelecturespecial, 48  
 Ef, 193, 367  
 Ef\_axi, 367  
 Ef\_stab, 196  
 Eisentat, 440  
 End, 55  
 Energie\_cinetique\_turbulente, 214  
 Energie\_cinetique\_turbulente\_wit, 215  
 Energie\_multiphase, 212  
 Energie\_multiphase\_h, 213  
 Enthalpie\_imposee\_paro, 366  
 Entree\_temperature\_imposee\_h, 346  
 Eos\_qc, 401  
 Eos\_wc, 402  
 Epsilon, 64  
 Eqn\_base, 280  
 Execute\_parallel, 48  
 Export, 49  
 Extract\_2d\_from\_3d, 49  
 Extract\_2daxi\_from\_3d, 49  
 Extraire\_domaine, 50  
 Extraire\_plan, 50  
 Extraire\_surface, 51  
 Extraire\_surface\_ale, 31  
 Extrudebord, 52  
 Extrudeparoi, 52  
 Extruder, 53  
 Extruder\_en20, 53  
 Extruder\_en3, 54  
  
 Fd, 28  
 Fichier\_decoupage, 433  
 Fichier\_med, 432  
 Field\_uniform\_keps\_from\_ud, 383  
 Flottabilite, 539  
 Fluide\_base, 413  
 Fluide\_dilatable\_base, 414  
 Fluide\_diphasique, 414  
 Fluide\_diphasique\_ijk, 409  
 Fluide\_incompressible, 415  
 Fluide\_ostwald, 416  
 Fluide\_quasi\_compressible, 417  
 Fluide\_reel\_base, 419  
 Fluide\_sodium\_gaz, 419  
 Fluide\_sodium\_liquide, 420  
 Fluide\_stiffened\_gas, 421  
 Fluide\_weakly\_compressible, 422  
 Flux\_2groupes, 524  
 Flux\_interfacial, 532  
 Flux\_radiatif, 346  
 Flux\_radiatif\_vdf, 347  
 Flux\_radiatif\_vef, 347  
 Forchheimer, 532  
 Formatte, 110  
 Frontiere\_ouverte, 347  
 Frontiere\_ouverte\_alpha\_impose, 348  
 Frontiere\_ouverte\_concentration\_imposee, 348  
 Frontiere\_ouverte\_enthalpie\_imposee, 352  
 Frontiere\_ouverte\_fraction\_massique\_imposee, 348  
 Frontiere\_ouverte\_gradient\_pression\_impose, 348  
 Frontiere\_ouverte\_gradient\_pression\_impose\_vefprep1b, 349  
 Frontiere\_ouverte\_gradient\_pression\_libre\_vef, 349

Frontiere\_ouverte\_gradient\_pression\_libre\_vefprep1b, 349  
 Frontiere\_ouverte\_k\_eps\_impose, 349  
 Frontiere\_ouverte\_k\_omega\_impose, 350  
 Frontiere\_ouverte\_pression\_imposee, 350  
 Frontiere\_ouverte\_pression\_imposee\_orlansky, 350  
 Frontiere\_ouverte\_pression\_moyenne\_imposee, 350  
 Frontiere\_ouverte\_rayo\_semi\_transp, 351  
 Frontiere\_ouverte\_rayo\_transp, 351  
 Frontiere\_ouverte\_rayo\_transp\_vdf, 351  
 Frontiere\_ouverte\_rayo\_transp\_vef, 351  
 Frontiere\_ouverte\_rho\_u\_impose, 352  
 Frontiere\_ouverte\_temperature\_imposee\_rayo\_semi-transp, 352  
 Frontiere\_ouverte\_temperature\_imposee\_rayo\_transp, 353  
 Frontiere\_ouverte\_vitesse\_imposee, 353  
 Frontiere\_ouverte\_vitesse\_imposee\_ale, 353  
 Frontiere\_ouverte\_vitesse\_imposee\_sortie, 353  
 Frottement\_interfacial, 532  
  
 Gaz\_parfait\_qc, 406  
 Gaz\_parfait\_wc, 406  
 Gcp, 338, 518  
 Gcp\_ns, 335  
 Gen, 336  
 Generic, 194  
 Gmres, 336, 519  
  
 Hht, 28  
  
 Ibcgstab, 513  
 Ibm\_aucune, 398  
 Ibm\_element\_fluide, 399  
 Ibm\_gradient\_moyen, 400  
 Ibm\_hybride, 399  
 Ibm\_power\_law\_tbl, 401  
 Ice, 501  
 Ijk, 367  
 Ijk\_grid\_geometry, 369  
 Ijk\_interfaces, 281  
 Ijk\_thermals, 282  
 Ilu, 438  
 Implicit\_euler\_steady\_scheme, 444  
 Implicit\_steady, 502  
 Implicite, 503  
 Implicite\_ale, 504  
 Imposer\_vit\_bords\_ale, 56  
 Imprimer\_flux, 56  
 Imprimer\_flux\_sum, 56  
 Init\_par\_partie, 383  
 Injection\_qdm\_nulle, 524  
 Integrer\_champ\_med, 56  
 Interface\_base, 315  
 Interface\_sigma\_constant, 316  
 Interfacial\_area, 203  
 Internes, 64  
 Interpolation, 325, 430  
 Interpolation\_champ\_face\_deriv, 309  
 Interpolation\_ibm\_base, 398  
 Interpolation\_ibm\_power\_law\_tbl\_u\_star, 398  
 Interprete, 24  
 Interprete\_geometrique\_base, 58  
  
 Jacobi, 441  
 Jones\_laundry, 251  
  
 K\_epsilon, 249  
 K\_epsilon\_bicephale, 253  
 K\_epsilon\_realisable, 256  
 K\_epsilon\_realisable\_bicephale, 255  
 K\_omega, 203, 204, 247  
 K\_tau, 204  
 Kquick, 195  
  
 L\_melange, 202  
 Lam\_bremhorst, 250  
 Lata\_2\_med, 58  
 Lata\_2\_other, 59  
 Lata\_to\_cgns, 58  
 Laundry\_sharma, 251  
 Leap\_frog, 453  
 Lineaire, 310  
 Link\_cgns\_files, 31  
 Lire\_ideas, 59  
 Lire\_tgrid, 77  
 List\_bloc\_mailler, 60  
 List\_bord, 62  
 List\_nom, 83  
 List\_nom\_virgule, 321  
 Liste\_post, 109  
 Liste\_post\_ok, 106  
 Listobj, 560  
 Listobj\_impl, 560  
 Lml\_2\_lata, 60  
 Logarithmique, 431  
 Loi\_analytique\_scalaire, 557  
 Loi\_ciofalo\_hydr, 553  
 Loi\_etat\_base, 401  
 Loi\_etat\_gaz\_parfait\_base, 404  
 Loi\_etat\_gaz\_reel\_base, 404  
 Loi\_etat\_tppi\_base, 404  
 Loi\_expert\_hydr, 553  
 Loi\_expert\_scalaire, 557  
 Loi\_fermeture\_base, 407  
 Loi\_fermeture\_test, 407  
 Loi\_horaire, 307, 408  
 Loi\_odvm, 557

Loi\_paroι\_nu\_impose, 558  
 Loi\_puissance\_hydr, 553  
 Loi\_standard\_hydr, 554  
 Loi\_standard\_hydr\_old, 554  
 Loi\_standard\_hydr\_scalaire, 558  
 Loi\_ww\_hydr, 554  
 Loi\_ww\_scalaire, 556  
 Longitudinale, 535  
 Longueur\_melange, 233  
 Lu, 441, 519  
  
 Ma, 28  
 Mailler, 60  
 Mailler\_base, 60  
 Maillerparallel, 65  
 Masse\_ajoutee, 539  
 Masse\_multiphase, 216  
 Merge\_med, 32  
 Methode\_transport\_deriv, 307  
 Metis, 433  
 Milieu\_base, 408  
 Milieu\_composite, 560  
 Milieu\_musig, 560  
 Milieu\_v2\_base, 424  
 Mixte\_shear, 342  
 Mkdir, 66  
 Mod\_turb\_hyd\_rans, 246  
 Mod\_turb\_hyd\_rans\_bicephale, 252  
 Mod\_turb\_hyd\_rans\_keps, 248  
 Mod\_turb\_hyd\_rans\_komega, 254  
 Mod\_turb\_hyd\_ss\_maille, 230  
 Modele\_fonc\_realisable\_base, 332  
 Modele\_fonction\_bas\_reynolds\_base, 250  
 Modele\_rayo\_semi\_transp, 146  
 Modele\_rayonnement\_base, 424  
 Modele\_rayonnement\_milieu\_transparent, 424  
 Modele\_shih\_zhu\_lumley\_vdf, 332  
 Modele\_turbulence\_hyd\_deriv, 229  
 Modele\_turbulence\_scal\_base, 426  
 Modif\_bord\_to\_raccord, 66  
 Modifiee, 310  
 Modifydomaineaxi1d, 67  
 Mor\_eqn, 191  
 Moyenne, 102–106, 327  
 Moyenne\_imposee\_deriv, 429  
 Moyenne\_volumique, 67  
 Multi\_gaz\_parfait\_qc, 405  
 Multi\_gaz\_parfait\_wc, 405  
 Multiple, 203  
 Multiplefiles, 32  
 Muscl, 194  
 Muscl3, 195  
 Muscl\_new, 195  
 Muscl\_old, 193

My\_comm\_group, 32  
  
 Navier\_stokes\_aposteriori, 217  
 Navier\_stokes\_ft\_disc, 286  
 Navier\_stokes\_ftd\_ijk, 224  
 Navier\_stokes\_ibm, 290  
 Navier\_stokes\_ibm\_turbulent, 292  
 Navier\_stokes\_phase\_field, 293  
 Navier\_stokes\_qc, 283  
 Navier\_stokes\_standard, 297  
 Navier\_stokes\_standard\_sensibility, 257  
 Navier\_stokes\_std\_ale, 259  
 Navier\_stokes\_turbulent, 299  
 Navier\_stokes\_turbulent\_ale, 227  
 Navier\_stokes\_turbulent\_qc, 301  
 Navier\_stokes\_wc, 285  
 Negligeable, 192, 198, 554  
 Negligeable\_scalaire, 558  
 Nettoiepasnoeuds, 69  
 Neumann, 354  
 Neumann\_homogene, 343  
 Neumann\_paroι, 343  
 Neumann\_paroι\_adiabatique, 343  
 Newmarktimescheme\_deriv, 28  
 Nom, 432  
 Non, 538  
 Null, 245, 426, 441  
 Numero\_elem\_sur\_maitre, 98  
  
 Objet\_lecture, 561  
 Op\_conv\_ef\_stab\_polymac\_face, 33  
 Op\_conv\_ef\_stab\_polymac\_p0\_face, 33  
 Op\_conv\_ef\_stab\_polymac\_p0p1nc\_elem, 33  
 Op\_conv\_ef\_stab\_polymac\_p0p1nc\_face, 33  
 Optimal, 337  
 Option, 199  
 Option\_cgns, 33  
 Option\_dg, 34  
 Option\_ijk, 35  
 Option\_interpolation, 35  
 Option\_polymac, 35  
 Option\_vdf, 70  
 Orientefacesbord, 70  
 Orienter\_simplexes, 77  
  
 P1b, 200  
 P1ncp1b, 199  
 Parallel\_io\_parameters, 36  
 Parametre\_diffusion\_implicit, 206  
 Parametre\_equation\_base, 205  
 Parametre\_implicit, 206  
 Paroι, 343  
 Paroι\_adiabatique, 354  
 Paroι\_contact, 354

Paroi\_contact\_fictif, 355  
 Paroi\_contact\_rayo, 355  
 Paroi\_decalee\_robin, 355  
 Paroi\_defilante, 356  
 Paroi\_echange\_contact\_correlation\_vdf, 356  
 Paroi\_echange\_contact\_correlation\_vdf, 357  
 Paroi\_echange\_contact\_odvm\_vdf, 358  
 Paroi\_echange\_contact\_rayo\_semi\_transp\_vdf, 358  
 Paroi\_echange\_contact\_vdf, 359  
 Paroi\_echange\_contact\_vdf\_ft, 359  
 Paroi\_echange\_externer\_impose, 359  
 Paroi\_echange\_externer\_impose\_h, 360  
 Paroi\_echange\_externer\_impose\_rayo\_semi\_transp, 360  
 Paroi\_echange\_externer\_impose\_rayo\_transp, 360  
 Paroi\_echange\_externer\_radiatif, 346  
 Paroi\_echange\_global\_impose, 361  
 Paroi\_echange\_interne\_global\_impose, 342  
 Paroi\_echange\_interne\_global\_parfait, 342  
 Paroi\_echange\_interne\_impose, 342  
 Paroi\_echange\_interne\_parfait, 342  
 Paroi\_fixe, 361  
 Paroi\_fixe\_iso\_genepi2\_sans\_contribution\_aux\_vitesses\_sommets, 361  
 Paroi\_flux\_impose, 361  
 Paroi\_flux\_impose\_rayo\_semi\_transp\_vdf, 362  
 Paroi\_flux\_impose\_rayo\_semi\_transp\_vdf, 362  
 Paroi\_flux\_impose\_rayo\_transp, 362  
 Paroi\_frottante\_loi, 343  
 Paroi\_frottante\_simple, 344  
 Paroi\_ft\_disc, 362  
 Paroi\_ft\_disc\_deriv, 363  
 Paroi\_knudsen\_non\_negligeable, 363  
 Paroi\_rugueuse, 364  
 Paroi\_tble, 554  
 Paroi\_tble\_scal, 559  
 Paroi\_temperature\_imposee, 364  
 Paroi\_temperature\_imposee\_rayo\_semi\_transp, 364  
 Paroi\_temperature\_imposee\_rayo\_transp, 364  
 Partition, 70, 434  
 Partition\_multi, 72  
 Partitionneur\_deriv, 432  
 Pave, 60  
 Pb\_avec\_liste\_conc, 148  
 Pb\_avec\_passif, 149  
 Pb\_base, 143  
 Pb\_conduction, 91  
 Pb\_conduction\_ibm, 112  
 Pb\_couple\_rayo\_semi\_transp, 151  
 Pb\_couple\_rayonnement, 189  
 Pb\_frontracking\_disc, 113  
 Pb\_gen\_base, 91  
 Pb\_hem, 126  
 Pb\_hydraulique, 151  
 Pb\_hydraulique\_ale, 152  
 Pb\_hydraulique\_aposteriori, 153  
 Pb\_hydraulique\_cloned\_concentration, 114  
 Pb\_hydraulique\_cloned\_concentration\_turbulent, 116  
 Pb\_hydraulique\_concentration, 154  
 Pb\_hydraulique\_concentration\_scalaires\_passifs, 156  
 Pb\_hydraulique\_concentration\_turbulent, 157  
 Pb\_hydraulique\_concentration\_turbulent\_scalaires\_passifs, 158  
 Pb\_hydraulique\_ibm, 159  
 Pb\_hydraulique\_ibm\_turbulent, 117  
 Pb\_hydraulique\_list\_concentration, 118  
 Pb\_hydraulique\_list\_concentration\_turbulent, 119  
 Pb\_hydraulique\_melange\_binaire\_qc, 161  
 Pb\_hydraulique\_melange\_binaire\_turbulent\_qc, 163  
 Pb\_hydraulique\_melange\_binaire\_wc, 162  
 Pb\_hydraulique\_sensibility, 122  
 Pb\_hydraulique\_turbulent, 164  
 Pb\_hydraulique\_turbulent\_ale, 121  
 Pb\_mg, 165  
 Pb\_multiphase, 123  
 Pb\_multiphase\_h, 124  
 Pb\_phase\_field, 166  
 Pb\_rayo\_conduction, 128  
 Pb\_rayo\_hydraulique, 129  
 Pb\_rayo\_hydraulique\_turbulent, 130  
 Pb\_rayo\_thermohydraulique, 131  
 Pb\_rayo\_thermohydraulique\_qc, 132  
 Pb\_rayo\_thermohydraulique\_turbulent, 133  
 Pb\_rayo\_thermohydraulique\_turbulent\_qc, 134  
 Pb\_thermohydraulique, 168  
 Pb\_thermohydraulique\_cloned\_concentration, 135  
 Pb\_thermohydraulique\_cloned\_concentration\_turbulent, 137  
 Pb\_thermohydraulique\_concentration, 172  
 Pb\_thermohydraulique\_concentration\_scalaires\_passifs, 173  
 Pb\_thermohydraulique\_concentration\_turbulent, 174  
 Pb\_thermohydraulique\_concentration\_turbulent\_scalaires\_passifs, 176  
 Pb\_thermohydraulique\_especes\_qc, 177  
 Pb\_thermohydraulique\_especes\_turbulent\_qc, 179  
 Pb\_thermohydraulique\_especes\_wc, 178  
 Pb\_thermohydraulique\_ibm, 181  
 Pb\_thermohydraulique\_ibm\_turbulent, 138  
 Pb\_thermohydraulique\_list\_concentration, 139  
 Pb\_thermohydraulique\_list\_concentration\_turbulent, 140  
 Pb\_thermohydraulique\_qc, 169  
 Pb\_thermohydraulique\_scalaires\_passifs, 182  
 Pb\_thermohydraulique\_sensibility, 142  
 Pb\_thermohydraulique\_turbulent, 183  
 Pb\_thermohydraulique\_turbulent\_qc, 184  
 Pb\_thermohydraulique\_turbulent\_scalaires\_passifs, 186  
 Pb\_thermohydraulique\_wc, 170



Pbc\_med, 187  
 Pdi, 111  
 Pdi\_expert, 111  
 Perio, 365  
 Periodique, 365  
 Perte\_charge\_anisotrope, 532  
 Perte\_charge\_circulaire, 533  
 Perte\_charge\_directionnelle, 533  
 Perte\_charge\_isotrope, 534  
 Perte\_charge\_reguliere, 534  
 Perte\_charge\_singuliere, 535  
 Petsc, 338  
 Petsc\_gpu, 338  
 Pilote\_icoco, 72  
 Pilut, 441  
 Pipecg, 514  
 Piso, 505  
 Plan, 99  
 Point, 98  
 Points, 98  
 Polyedriser, 73  
 Polymac, 367  
 Polymac\_p0, 368  
 Polymac\_p0p1nc, 368  
 Porosites, 437  
 Portance\_interfaciale, 525  
 Position\_like, 99  
 Post\_processing, 107  
 Post\_processings, 106  
 Postraitement\_base, 107  
 Postraitement\_ft\_lata, 109  
 Postraiter\_domaine, 73  
 Prandtl, 202, 427  
 Precisiongeom, 74  
 Precond\_base, 438  
 Preconditionneur\_petsc\_deriv, 439  
 Precondsolv, 438  
 Predefini, 328  
 Pression, 102, 104–106  
 Problem\_read\_generic, 188  
 Probleme\_couple, 146  
 Probleme\_ftd\_ijk, 189  
 Probleme\_ftd\_ijk\_cut\_cell, 144  
 Production\_echelle\_temp\_taux\_diss\_turb, 525  
 Production\_energie\_cin\_turb, 526  
 Production\_hzdr, 525  
 Profil, 431  
 Profils\_thermo, 219  
 Projection\_ale\_boundary, 36  
 Puissance\_thermique, 536  
 Qdm\_multiphase, 260  
 Quick, 194  
 Raccord, 63  
 Radioactive\_decay, 536  
 Radius, 98  
 Raffiner\_anisotrope, 74  
 Raffiner\_isotrope, 74  
 Raffiner\_isotrope\_parallele, 37  
 Read, 76  
 Read\_file, 76  
 Read\_file\_binary, 76  
 Read\_med, 37  
 Read\_unsupported\_ascii\_file\_from\_icem, 77  
 Redresser\_hexaedres\_vdf, 77  
 Refine\_mesh, 78  
 Regroupebord, 78  
 Remove\_elem, 79  
 Remove\_invalid\_internal\_boundaries, 80  
 Reordonner, 81  
 Reorienter\_tetraedres, 80  
 Reorienter\_triangles, 80  
 Rhot\_gaz\_parfait\_qc, 407  
 Rhot\_gaz\_reel\_qc, 407  
 Rk3\_ft, 455  
 Robin\_vef, 365  
 Rotation, 81  
 Rt, 198  
 Runge\_kutta\_ordre\_2, 457  
 Runge\_kutta\_ordre\_2\_classique, 459  
 Runge\_kutta\_ordre\_3, 461  
 Runge\_kutta\_ordre\_3\_classique, 463  
 Runge\_kutta\_ordre\_4\_classique, 467  
 Runge\_kutta\_ordre\_4\_classique\_3\_8, 469  
 Runge\_kutta\_ordre\_4\_d3p, 465  
 Runge\_kutta\_rationnel\_ordre\_2, 471  
 Rupture\_bulles\_1groupe, 526  
 Rupture\_bulles\_2groupes, 526  
 Sa-amg, 441  
 Sato, 204  
 Saturation\_base, 316  
 Saturation\_constant, 316  
 Saturation\_sodium, 317  
 Scalaire\_impose\_parois, 365  
 Scatter, 82  
 Scattermed, 82  
 Sch\_cn\_ex\_iteratif, 447  
 Sch\_cn\_iteratif, 449  
 Schema\_adams\_bashforth\_order\_2, 474  
 Schema\_adams\_bashforth\_order\_3, 475  
 Schema\_adams\_moulton\_order\_2, 477  
 Schema\_adams\_moulton\_order\_3, 480  
 Schema\_backward\_differentiation\_order\_2, 483  
 Schema\_backward\_differentiation\_order\_3, 485  
 Schema\_euler\_explicite\_ale, 497  
 Schema\_euler\_explicite\_ijk, 500



Schema\_implicite\_base, 490  
 Schema\_phase\_field, 493  
 Schema\_predictor\_corrector, 495  
 Schema\_rk3\_ijk, 499  
 Schema\_temps\_base, 442  
 Schema\_temps\_base\_ijk, 499  
 Scheme\_euler\_explicit, 452  
 Scheme\_euler\_implicit, 488  
 Schmidt, 428  
 Segment, 100  
 Segmentfacesx, 97  
 Segmentfacesy, 97  
 Segmentfacesz, 97  
 Segmentpoints, 98  
 Sensibility, 198  
 Sets, 506  
 Sgdh, 201  
 Shih\_zhu\_lumley, 333  
 Simple, 507  
 Simplifier, 508  
 Single\_hdf, 111  
 Smago, 202  
 Solid\_particle\_base, 410  
 Solid\_particle\_sphere, 410  
 Solid\_particle\_spheroid, 411  
 Solide, 423  
 Solve, 82  
 Solver\_moving\_mesh\_ale, 38  
 Solveur\_implicite\_base, 501  
 Solveur\_lineaire\_std, 509  
 Solveur\_petsc\_deriv, 510  
 Solveur\_sys\_base, 339  
 Solveur\_u\_p, 509  
 Sonde\_base, 96  
 Sortie\_libre\_rho\_variable, 366  
 Sortie\_libre\_temperature\_imposee\_h, 366  
 Source\_base, 520  
 Source\_bif, 526  
 Source\_con\_phase\_field, 536  
 Source\_constituant, 538  
 Source\_constituant\_vortex, 526  
 Source\_dep\_inco\_bases, 528  
 Source\_dissipation\_echelle\_temp\_taux\_diss\_turb, 527  
 Source\_dissipation\_hzdr, 527  
 Source\_generique, 539  
 Source\_pdf, 539  
 Source\_pdf\_base, 540  
 Source\_qdm, 541  
 Source\_qdm\_lambdaup, 541  
 Source\_qdm\_phase\_field, 542  
 Source\_rayo\_semi\_transp, 542  
 Source\_robin, 542  
 Source\_robin\_scalaire, 542  
 Source\_th\_tdivu, 543  
 Source\_transport\_eps, 543  
 Source\_transport\_k, 544  
 Source\_transport\_k\_eps, 544  
 Source\_transport\_k\_eps\_aniso\_concen, 544  
 Source\_transport\_k\_eps\_aniso\_therm\_concen, 545  
 Source\_transport\_k\_eps\_anisotherme, 528  
 Source\_transport\_k\_eps\_realisable, 545  
 Sources, 205  
 Sous\_dom, 434  
 Sous\_maille, 244  
 Sous\_maille\_1elt, 238  
 Sous\_maille\_1elt\_selectif\_mod, 239  
 Sous\_maille\_axi, 240  
 Sous\_maille\_dyn, 429  
 Sous\_maille\_selectif, 237  
 Sous\_maille\_selectif\_mod, 235  
 Sous\_maille\_smago, 232  
 Sous\_maille\_smago\_dyn, 242  
 Sous\_maille\_smago\_filtre, 241  
 Sous\_maille\_wale, 231  
 Sous\_zone, 547  
 Sous\_zones, 435  
 Spai, 442  
 Spec\_pdcrr\_base, 534  
 Ssor, 438, 442  
 Ssor\_bloc, 438  
 Stab, 199  
 Standard, 200, 310  
 Standard\_keps, 251  
 Stat\_per\_proc\_perf\_log, 83  
 Stat\_post\_deriv, 102  
 Statistiques, 102, 104–106  
 Statistiques\_en\_serie, 105, 106  
 Structural\_dynamic\_mesh\_model, 38  
 Supg, 197  
 Supprime\_bord, 83  
 Switch\_double, 39  
 Switch\_ft\_double, 39  
 Symetrie, 363, 366  
 System, 83  
 Systeme\_naire\_deriv, 537  
 T\_deb, 103  
 T\_fin, 103  
 Taux\_dissipation\_turbulent, 261  
 Tayl\_green, 383  
 Temperature, 220  
 Tenseur\_reynolds\_externes, 204, 545  
 Terme\_dissipation\_energie\_cinetique\_turbulente, 528  
 Terme\_puissance\_thermique\_echange\_impose, 546  
 Test\_solveur, 84  
 Test\_sse\_kernels, 40  
 Testeur, 84  
 Testeur\_medcoupling, 84

Tetraedriser, [85](#)  
 Tetraedriser\_homogene, [85](#)  
 Tetraedriser\_homogene\_compact, [86](#)  
 Tetraedriser\_homogene\_fin, [86](#)  
 Tetraedriser\_par\_prisme, [87](#)  
 Thi, [220](#)  
 Thi\_thermo, [221](#)  
 Trainee, [543](#)  
 Traitement\_particulier\_base, [219](#)  
 Tranche, [435](#)  
 Transformer, [88](#)  
 Transport\_epsilon, [302](#)  
 Transport\_equation\_deriv, [549](#)  
 Transport\_interfaces\_ft\_disc, [303](#)  
 Transport\_k, [311](#)  
 Transport\_k\_epsilon, [550](#)  
 Transport\_k\_epsilon\_realisable, [550](#)  
 Transport\_k\_omega, [551](#)  
 Transport\_marqueur\_ft, [312](#)  
 Transversale, [535](#)  
 Travail\_pression, [546](#)  
 Trianguler, [88](#)  
 Trianguler\_fin, [89](#)  
 Trianguler\_h, [89](#)  
 Triple\_line\_model\_ft\_disc, [317](#)  
 Turbulence\_paro\_base, [553](#)  
 Turbulence\_paro\_scalaire\_base, [556](#)  
 Turbulente, [201](#)  
 type, [102](#), [104](#)  
 Type\_diffusion\_turbulente\_multiphase\_deriv, [201](#)  
 Type\_diffusion\_turbulente\_multiphase\_multiple\_deriv,  
     [562](#)  
 Type\_indic\_faces\_deriv, [310](#)  
 Type\_perte\_charge\_deriv, [522](#)  
  
 Uniform\_field, [384](#)  
 Union, [436](#)  
 Utau\_imp, [556](#)  
  
 Valeur\_totale\_sur\_volume, [384](#)  
 Vdf, [368](#)  
 Vect\_nom, [90](#)  
 Vef, [368](#)  
 Verifier\_qualite\_raffinements, [90](#)  
 Verifier\_simplexes, [90](#)  
 Verifiercoin, [90](#)  
 Vitesse\_derive\_base, [546](#)  
 Vitesse\_imposee, [307](#)  
 Vitesse\_interpolee, [307](#)  
 Vitesse\_relative\_base, [547](#)  
 Volume, [99](#)  
  
 Wale, [201](#)  
 Write\_med, [31](#)  
  
 Xyz, [111](#)  
 xyz, [22](#)